JavaScript Handbook

Core language · functions & closures · objects & classes · async & the event loop · DOM · browser APIs · Node.js · interview prep

19 parts · one place to study: DesignGurus courses + classic books + the web, combined, beginner to intermediate

what it is · problem · how it works · trade-offs · real world · interview · self-check · TL;DR

Personal study notes. Course diagrams belong to DesignGurus. Not for redistribution.

Contents

1. **Chapter 1 — Introduction and Setup**
   - 1.1 What JavaScript is
   - 1.2 ECMAScript and TC39
   - 1.3 JavaScript engines — how your code actually runs
   - 1.4 Where JavaScript runs — hosts
   - 1.5 Setting up and running code
   - 1.6 Hello, World! — your first program and the basic rules
   - 1.7 Comments
   - 1.8 Strict mode
   - Chapter 1 TL;DR
2. **Chapter 2 — Variables, Types and Operators**
   - 2.1 Variables: `let`, `const` and `var`
   - 2.2 Hoisting and the Temporal Dead Zone
   - 2.3 Data types and `typeof`
   - 2.4 Primitives vs objects
   - 2.5 Arithmetic operators
   - 2.6 Comparison operators and equality
   - 2.7 Logical operators, `??` and `?.`
   - 2.8 Bitwise and assignment operators
   - 2.9 Operator precedence
   - 2.10 Numbers in depth
   - 2.11 Booleans, truthy and falsy
   - 2.12 Type conversion summary
   - Chapter 2 TL;DR
3. **Chapter 3 — Control Flow and Loops**
   - 3.1 Conditional statements — making decisions
   - 3.2 `if`, `else` and `else if`
   - 3.3 The conditional (ternary) operator
   - 3.4 `switch`
   - 3.5 Loops — why and which
   - 3.6 The `for` loop
   - 3.7 The `while` loop
   - 3.8 The `do…while` loop
   - 3.9 The `for…in` loop
   - 3.10 The `for…of` loop
   - 3.11 `break`, `continue` and labels
   - 3.12 Choosing the right loop
   - Chapter 3 TL;DR
4. **Chapter 4 — Functions, Scope and Closures**
   - 4.1 What a function is
   - 4.2 Declarations vs expressions (and hoisting)
   - 4.3 Arrow functions
   - 4.4 Parameters: defaults, rest, `arguments` and spread
   - 4.5 Scope: global, function and block
   - 4.6 Hoisting — the complete picture
   - 4.7 Closures
   - 4.8 Higher-order functions and callbacks
   - 4.9 IIFE — Immediately Invoked Function Expressions
   - 4.10 Recursion
   - 4.11 Pure functions
   - Chapter 4 TL;DR
5. **Chapter 5 — Objects, Math, Date and JSON**
   - 5.1 What an object is
   - 5.2 Reading, adding, updating and deleting properties
   - 5.3 Methods and `this`
   - 5.4 Spread, rest and copying objects
   - 5.5 Destructuring objects
   - 5.6 `Object.*` static methods, descriptors and locking
   - 5.7 Getters and setters
   - 5.8 Optional chaining `?.`
   - 5.9 JavaScript's built-in (native) objects
   - 5.10 `Math`
   - 5.11 `Date`
   - 5.12 JSON
   - Chapter TL;DR
6. **Chapter 6 — Strings and Regular Expressions**
   - 6.1 What a string is
   - 6.2 Creating strings
   - 6.3 Template literals
   - 6.4 Working with strings: indexing, immutability, coercion, String objects
   - 6.5 Unicode: code units, code points and graphemes
   - 6.6 String methods
   - 6.7 Regular expressions: the pattern language
   - 6.8 Regex methods and techniques
   - 6.9 Modern regex features and practical recipes
   - Chapter TL;DR
7. **Chapter 7 — Arrays**
   - 7.1 Why arrays?
   - 7.2 Creating arrays
   - 7.3 Reading, changing, adding and removing
   - 7.4 The methods table: mutating vs non-mutating
   - 7.5 Iteration methods: map, filter, reduce and friends
   - 7.6 Sorting correctly
   - 7.7 Spread, rest and destructuring with arrays
   - 7.8 Arrays vs objects (vs Map/Set)
   - 7.9 Typed arrays (a short note)
   - Chapter TL;DR
8. **Chapter 8 — Maps, Sets, Iterators and Generators**
   - 8.1 `Map`
   - 8.2 `Set`

- 8.3 `WeakMap`, `WeakSet`, `WeakRef` and `FinalizationRegistry`
- 8.4 Iterators and the iterable protocol
- 8.5 Generators
- 8.6 Iterator helpers (ES2025)
- 8.7 Grouping: `Object.groupBy` and `Map.groupBy` (ES2024)
- Version summary (verified September 2026)
- Chapter TL;DR

9. **Chapter 9 — `this`, Prototypes and Classes**
   - 9.1 `this` — the call-time context
   - 9.2 `call`, `apply` and `bind`
   - 9.3 Prototypes and the prototype chain
   - 9.4 Constructor functions and `new`
   - 9.5 Classes
   - 9.6 Inheritance with `extends` and `super`
   - 9.7 `instanceof` and type checks
   - 9.8 Mixins
   - 9.9 Composition vs inheritance
   - Chapter 9 TL;DR
10. **Chapter 10 — Modules and Tooling**
   - 10.1 Why modules?
   - 10.2 `export` and `import` syntax
   - 10.3 How ES modules load
   - 10.4 Dynamic `import()`
   - 10.5 Top-level `await`
   - 10.6 `import.meta` and JSON modules
   - 10.7 Modules in the browser
   - 10.8 CommonJS vs ES modules in Node
   - 10.9 npm and `package.json`
   - 10.10 Semantic versioning, ranges and lockfiles
   - 10.11 Bundlers
   - 10.12 Transpilers and polyfills
   - 10.13 Linters and formatters
   - 10.14 Putting it together — a modern project
   - Chapter 10 TL;DR
11. **Chapter 11 — Errors and Debugging**
   - 11.1 Errors and the built-in error types
   - 11.2 `throw`, `try`, `catch`, `finally`
   - 11.3 Custom errors
   - 11.4 Error chaining with `cause`
   - 11.5 Errors in asynchronous code (preview)
   - 11.6 Guaranteed cleanup: `finally` and `using`
   - 11.7 The console toolbox
   - 11.8 Reading stack traces and source maps
   - 11.9 Debugging with DevTools and the Node inspector
   - 11.10 Common runtime errors — a field guide
   - Chapter 11 TL;DR
12. **Chapter 12 — Asynchronous JavaScript**
   - 12.1 One thread, and why blocking hurts
   - 12.2 The event loop
   - 12.3 Ordering puzzles
   - 12.4 Node's event loop: `nextTick`, `setImmediate` and phases
   - 12.5 Timers
   - 12.6 Callbacks and callback hell
   - 12.7 Promises
   - 12.8 Promise combinators: `all`, `allSettled`, `race`, `any`
   - 12.9 `async` / `await`
   - 12.10 Error handling across async code — a checklist
   - 12.11 Cancellation: `AbortController` and `AbortSignal`
   - 12.12 Async iteration: `for await…of` and async generators
   - 12.13 Real-world async patterns
   - Chapter 12 TL;DR
13. **Chapter 13 — The DOM and Events**
   - 13.1 The DOM tree
   - 13.2 Selecting and traversing
   - 13.3 Creating, inserting and removing nodes
   - 13.4 Attributes, properties, classes, data and styles
   - 13.5 Events and listeners
   - 13.6 Propagation: capture, target, bubble
   - 13.7 Event delegation
   - 13.8 Default actions and `preventDefault`
   - 13.9 Forms
   - 13.10 Watching the DOM: `MutationObserver`
   - 13.11 Rendering performance: reflow, repaint and layout thrashing
   - 13.12 When scripts run: `DOMContentLoaded`, `load` and `defer`
   - Chapter 13 TL;DR
14. **Chapter 14 — Browser APIs and Security**
   - 14.1 `fetch` and HTTP
   - 14.2 The same-origin policy and CORS
   - 14.3 Web Storage: `localStorage` and `sessionStorage`
   - 14.4 Cookies and IndexedDB — choosing storage
   - 14.5 Timers and `requestAnimationFrame`
   - 14.6 `URL` and `URLSearchParams`
   - 14.7 The History API
   - 14.8 Web Workers
   - 14.9 `IntersectionObserver` and `ResizeObserver`
   - 14.10 Cross-site scripting (XSS) and sanitizing
   - 14.11 Cross-site request forgery (CSRF)
   - 14.12 Content Security Policy (CSP)
   - Chapter 14 TL;DR
15. **Chapter 15 — Node.js Basics**
   - 15.1 What Node.js is
   - 15.2 The event loop, the thread pool and blocking

- 15.3 Modules in Node — the short version
- 15.4 Files with `node:fs/promises`
- 15.5 Paths and file URLs: `node:path`, `node:url`
- 15.6 `process`: arguments, environment and exit codes
- 15.7 `Buffer`: raw bytes
- 15.8 Events: `EventEmitter`
- 15.9 Streams
- 15.10 Building an HTTP server with `node:http`
- 15.11 Child processes and npm scripts
- 15.12 Everyday Node flags, and Node vs Deno vs Bun
- Chapter 15 TL;DR

16. **Chapter 16 — Testing and TypeScript**

- 16.1 Why test, and what kinds of tests exist
- 16.2 Assertions with `node:assert/strict`
- 16.3 Writing and running tests with `node:test`
- 16.4 Async tests and how tests fail
- 16.5 Test doubles and mocking
- 16.6 Snapshot tests and code coverage
- 16.7 Good tests, and other runners
- 16.8 TypeScript: what it is and how it's checked
- 16.9 Running TypeScript: Node's type stripping and other tools
- 16.10 Everyday types
- 16.11 Narrowing
- 16.12 Generics and utility types
- 16.13 Classes, `as const`, `satisfies`, and enums
- 16.14 Tests written in TypeScript
- 16.15 JSDoc types: TypeScript checking for plain JavaScript
- Chapter 16 TL;DR

17. **Chapter 17 — JavaScript for Interviews**

- 17.1 The JavaScript DSA toolkit
- 17.2 Coding patterns
- 17.3 Implement-from-scratch classics
- 17.4 Fifty-three tricky interview questions
- Chapter 17 TL;DR

18. **Chapter 18 — Cheat Sheets and Glossary**

- 18.1 Syntax, variables and types
- 18.2 Operators and the coercion table
- 18.3 Control flow and loops
- 18.4 Functions, scope, closures and `this`
- 18.5 Objects, Math, Date and JSON
- 18.6 Classes and prototypes
- 18.7 Strings and regular expressions
- 18.8 Arrays: mutating vs non-mutating
- 18.9 Map, Set, iterators and generators
- 18.10 Modules and tooling
- 18.11 Errors and debugging
- 18.12 Async and the event loop
- 18.13 DOM and events
- 18.14 Browser APIs and security
- 18.15 Node.js core modules
- 18.16 Testing and TypeScript
- 18.17 Which API for which job
- 18.18 ECMAScript versions ES2015 → ES2026
- 18.19 Where to go next
- 18.20 Interview question index
- 18.21 Glossary (A–Z)
- Chapter 18 TL;DR

19. **Chapter 19 — Interview Question Bank**

- How to use this bank
- 1. Types, values and coercion
- 2. Variables, hoisting, scope and closures
- 3. Functions and functional patterns
- 4. `this`, `call` / `apply` / `bind`, prototypes and classes
- 5. Objects, arrays, strings and collections
- 6. The event loop, promises and `async` / `await`
- 7. Errors, modules and tooling
- 8. The DOM and events
- 9. Browser APIs, storage, networking and security
- 10. Performance and memory
- 11. Node.js
- 12. TypeScript basics
- 13. Output-prediction puzzles ("What will this print?")
- 14. Coding tasks: implement-from-scratch classics
- 15. LeetCode "30 Days of JavaScript"
- Sources of the question lists

Chapter 1 — Introduction and Setup

JavaScript is the language that makes web pages *do* things: open a menu when you click it, check a form before it is sent, update a chat without reloading the page. Today the same language also runs web servers (programs that answer requests from browsers and apps over the network), command-line tools (programs you start by typing a command in a text window), desktop apps and mobile apps. This chapter explains what JavaScript actually is, who defines it, what software runs it, and how to run your own first programs — in a browser and in Node.js. It finishes with two small but important habits: writing good comments and switching on strict mode.

No previous programming experience is assumed. All you need is a computer, a modern browser (Chrome, Firefox, Safari or Edge) and, for the Node.js examples, Node installed (§1.5). The goal of the whole book is that by the end you can read and write real-world JavaScript with confidence and are ready for frameworks such as React (a **framework** is a ready-made structure of code that your app plugs into, so you don't build every common piece yourself).

What you'll learn

1. What JavaScript is, where it came from, and why it is worth learning.
2. The difference between *JavaScript* and *ECMAScript*, and how TC39 adds features every year.
3. What a JavaScript *engine* is (V8, SpiderMonkey, JavaScriptCore) and how it turns text into running code.
4. The idea of a *host* (browser vs Node.js) and why `document` exists in one but not the other.
5. Four ways to run code: browser console, `<script>` tag, the `node` command and the Node REPL.
6. Your first program (`console.log`), statements, semicolons and the basic rules of the syntax.
7. Comments: single-line, multi-line, JSDoc, and when (not) to write them.
8. Strict mode: what `"use strict"` changes and why modern code is strict by default.

All runnable code is in `code/`. Every "Output:" block was produced by running the file with Node 24.

1.1 What JavaScript is

What it is

JavaScript (JS) is a high-level, general-purpose programming language. *High-level* means you write instructions in terms of ideas (numbers, text, lists, functions) rather than in terms of the computer's memory and processor; the language takes care of those details for you, including freeing memory you no longer use (**garbage collection**).

A few more words you will meet constantly. (A **variable** is a name that holds a value, like a labelled box; a **compile** step translates source code into a lower-level form the machine can run.)

Term	Meaning for JavaScript
Interpreted / JIT-compiled	You do not run a separate compile step. You hand the source text to an engine, which compiles it on the fly ("just in time") while running it (§1.3).
Dynamically typed	Variables do not have fixed types. The *value* has a type; the same variable can hold a number now and text later (Chapter 2).
Multi-paradigm	You can write in an imperative style (step-by-step), an object-oriented style (objects and classes — Chapter 9) or a functional style (functions passed around as values — Chapter 4).
Single-threaded with an event loop	Your code runs one piece at a time on one thread (a single sequence of instructions being carried out); slow work (network, timers, files) is handed off and its result comes back later as an event (Chapter 12).

Analogy: if a web page were a house, **HTML** is the structure (walls, rooms), **CSS** is the decoration (paint, furniture layout), and **JavaScript** is the electricity and plumbing — the part that makes things

work when someone flips a switch.

A short history

Year	Event
1995	Brendan Eich writes the first version at Netscape (maker of the leading 1990s web browser) in about ten days. It is called Mocha, then LiveScript, and finally renamed **JavaScript** as a marketing tie-in with Java, which was fashionable then. The two languages are otherwise unrelated.
1996–97	Netscape hands the language to **Ecma International** for standardisation (November 1996). The first edition of the standard, called **ECMAScript** (document ECMA-262), is published in June 1997.
1999	ES3 — the version that most of the early web was written in.
2008	ES4, a very ambitious redesign, is abandoned. Google releases Chrome with the fast **V8** engine.
2009	ES5 adds strict mode, JSON (JavaScript Object Notation, a plain-text format for data) and array methods like `map`. Ryan Dahl releases **Node.js**, which runs V8 outside the browser.
2015	**ES2015 (ES6)** — the biggest update ever: `let` / `const`, arrow functions, classes, modules, promises, template literals.
2016 → now	A new edition every June, named by year (ES2016, ES2017, …). This book targets **ES2025**.

Don't worry if feature names such as *arrow functions*, *promises* or *modules* mean nothing yet — each one is explained in its own chapter.

Why it matters — reasons to learn JavaScript

- **It is everywhere.** It is the only programming language that every web browser runs natively. If you build for the web, you use JavaScript (directly, or through something that compiles to it, like TypeScript — JavaScript plus optional type labels that are checked before the code runs).
- **One language, many platforms.** Browser front ends (the part of an app the user sees and clicks); back ends (the server-side part that stores data and does the work) with **Node.js**; mobile apps with **React Native**; desktop apps with **Electron** (VS Code, Slack and Discord desktop are Electron apps); serverless functions at the "edge" (Cloudflare Workers). **Serverless** means you upload just a function and the provider runs it on demand without you managing a machine; the **edge** means servers spread around the world close to users (System Design Fundamentals).
- **Huge ecosystem.** npm, the JavaScript package registry (an online store of free, reusable code bundles called **packages**), is the largest software registry in the world, so most problems already have a **library** (code someone else wrote that you call from your program) (Chapter 10).
- **Jobs.** Front-end, back-end and full-stack (both ends) roles all ask for it; it has topped the Stack Overflow developer survey's "most used language" list for over a decade.
- **Gentle start.** You can run your first line of code in a browser tab with nothing installed. The language has sharp corners (you will meet them — coercion, the automatic conversion of a value from one type to another, in Chapter 2, `this` in Chapter 9), but the basics are approachable, which makes it a common first language.

Common mistakes

- **"JavaScript is Java for the web."** No — different design, different syntax details, different runtime. The name is a historical marketing accident.
- **"JavaScript only runs in browsers."** It runs anywhere there is an engine; Node.js is the most common non-browser host.
- **Treating "JavaScript" and "the browser" as the same thing.** `document.getElementById` is *not* part of the language — it belongs to the browser (§1.4). This distinction explains many "why doesn't this work in Node?" questions.

TL;DR

(TL;DR = "too long; didn't read" — a short summary of the section.)

JavaScript is a high-level, dynamically typed, garbage-collected, multi-paradigm language, standardised as ECMAScript, that runs in every browser and — through Node.js and friends — almost everywhere else.

1.2 ECMAScript and TC39

What it is

ECMAScript (ES) is the official specification of the language: a long document (ECMA-262) that says exactly what every piece of syntax means and how every built-in (a ready-made object or function the language provides, such as `Math`, `Array`, `Promise` …) behaves. **JavaScript** is the everyday name for implementations of that spec. ("JavaScript" is technically a trademark owned by Oracle, which is one reason the standard uses a different name.)

Analogy: ECMAScript is the rule book of chess; Chrome, Firefox and Node.js are chess sets made by different companies. The rule book guarantees a knight moves the same way on every set.

TC39 (Technical Committee 39 of Ecma International) is the group that maintains the spec. Its members are browser makers, companies that use JavaScript heavily, and invited experts. Anyone can follow its work on GitHub (github.com/tc39), a website that hosts code projects and their discussions.

How it works — how a feature gets into the language

New features go through a public **proposal process** with numbered stages:

1. A **champion** (a TC39 member) presents an idea and a problem statement.
2. The committee agrees the problem is worth solving (Stage 1) and works on a draft (Stage 2).
3. Stage 2.7 (added in 2024) means the design is approved and conformance tests (shared tests that check an engine follows the spec exactly) are being written.
4. At Stage 3, browsers and Node ship it, often behind a flag first (switched off unless you turn on a special setting), and real-world feedback arrives.
5. When there are two compatible shipping implementations and full tests, it reaches Stage 4 and is included in the next yearly edition (published each June).

Why it matters

- **Version names.** "ES6" and "ES2015" are the same thing. After 2015, people mostly use the year: "ES2020 added `??` and `?.`".
- **Compatibility.** Features can be used in browsers before the yearly edition is published (Stage 3/4). Whether *your users'* browsers support a feature is a separate question; sites like MDN's compatibility tables and caniuse.com answer it, and tools can translate new syntax into older syntax (Chapter 10).
- **The web never breaks.** TC39 follows the rule "don't break the web": old code must keep working. That is why odd old behaviours (like `typeof null === "object"`, Chapter 2) can never be fixed — they are fixed around instead (e.g. strict mode, `===`, `let`).

Editions you will hear about

Edition	Headline features (a selection)
ES5 (2009)	strict mode, `JSON`, `Array.prototype.forEach/map/filter`, `Object.defineProperty`
ES2015 / ES6	`let` / `const`, arrow functions, classes, modules, promises, template literals, destructuring, `Map` / `Set`, `for…of`
ES2017	`async` / `await`
ES2020	`??`, `?.`, `BigInt`, `globalThis`, `Promise.allSettled`
ES2022	class fields and `#private`, top-level `await`, `.at()`, `Error` `cause`
ES2023	`toSorted`, `toReversed`, `with`, `findLast`
ES2024	`Object.groupBy`, `Map.groupBy`, `Promise.withResolvers`
ES2025	iterator helpers (`.map/.filter` on iterators), new `Set` methods (`union`, `intersection` …), `Promise.try`, `RegExp.escape`, JSON modules/import attributes, `Float16Array`
ES2026	`Array.fromAsync`, `Error.isError`, `Math.sumPrecise`, `Uint8Array` base64/hex, `Map.prototype.getOrInsert` (upsert), `Iterator.concat`, `JSON.parse` source text access — only some are in Node 24
ES2027 (next)	**`Temporal`** (modern dates), `using` declarations (explicit resource management), `Atomics.pause`, `Iterator.zip` — finished proposals waiting for the June 2027 edition

Each feature is taught in its own chapter; this table is only a map.

In the interview

30-second answer — "What's the difference between JavaScript and ECMAScript?" ECMAScript is the standard (ECMA-262) maintained by TC39; JavaScript is the language as implemented by engines such as V8 and SpiderMonkey, plus, in practice, the host APIs around it. (An **API**, Application Programming Interface, is the set of functions and objects one piece of software offers for other code to call.) Since 2015 a new edition ships every year through a staged proposal process; Stage 4 features go into the next edition.

- *Follow-up: Is `document` part of JavaScript?* No. It is a browser (DOM) API provided by the host — the **DOM** (Document Object Model) is the browser's tree of objects representing the page, which scripts read and change; ECMAScript does not define it.
- *Follow-up: What does Stage 3 mean?* The design is complete and engines are implementing it; changes after this point are rare and usually driven by implementation feedback.

TL;DR

ECMAScript is the spec, JavaScript is its implementations. TC39 moves proposals through Stages 0–4, and a new edition (ES2016, ES2017 …) is published every June.

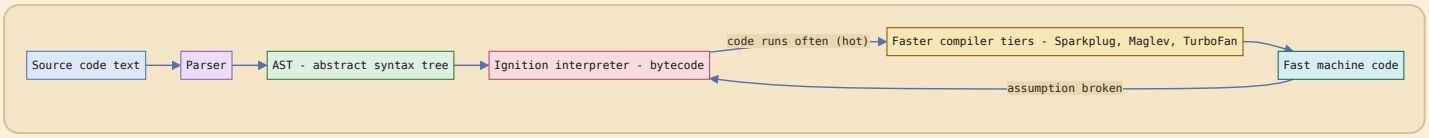
1.3 JavaScript engines — how your code actually runs

What it is

A **JavaScript engine** is the program that reads JavaScript source code and executes it. The main ones:

Engine	Made by	Used in
V8	Google	Chrome, Edge, Opera, Brave, **Node.js**, Deno
SpiderMonkey	Mozilla	Firefox (the first JS engine ever, descended from Eich's original)
JavaScriptCore (JSC)	Apple	Safari, most iOS browsers (Apple long required it; EU rules now allow others), **Bun**

How it works — from text to execution (V8 as the example)



1. **Parse.** The engine reads the text and checks the grammar. A grammar error (e.g. a missing `)`) is a `SyntaxError`, reported *before any line of that script runs*.
2. **Build an AST.** The code becomes a tree data structure — the **abstract syntax tree** — that represents its meaning.
3. **Interpret bytecode.** V8's **Ignition** interpreter (a program that carries out instructions one by one, without first translating the whole program) turns the tree into compact **bytecode** (simple, low-level instructions designed for the engine's own virtual machine rather than for the real processor) and starts running it immediately, so start-up is fast.
4. **Profile and optimise.** While running, the engine counts which functions are "hot" (called a lot) and what kinds of values they see. Hot code is recompiled into machine code (the raw instructions the processor executes directly) by faster tiers: **Sparkplug** (a quick, non-optimising baseline compiler), then **Maglev** and **TurboFan** (optimising compilers). The optimising tiers build on assumptions such as "this variable always holds a small integer".
5. **Deoptimise when needed.** If an assumption breaks (suddenly a string arrives), the engine throws away the optimised code and falls back to bytecode. Results stay correct; only speed changes.
6. Meanwhile a **garbage collector** frees memory that nothing references any more.

This whole approach is called **just-in-time (JIT) compilation**. Other engines use the same general idea with different names for their tiers.

Why it matters

- You never compile JavaScript yourself; you ship source code (usually minified — shrunk by removing spaces, comments and shortening names, without changing behaviour) and the engine does the rest.
- **Syntax errors stop the whole file**; runtime errors (problems found only while the code is running, such as calling something that isn't a function) stop at the line where they happen (Chapter 11).
- Writing code whose values keep consistent types (a function that always receives numbers) helps the optimiser. You rarely need to think about this as a beginner; clarity comes first.

Engine vs runtime

An engine alone can only run the core language: variables, functions, `Math`, `Array`, `JSON`, promises. It cannot draw on a screen, read a file or talk to the network. The program that *embeds* an engine and adds those abilities is the **host** or **runtime** — the next section.

One more runtime piece belongs to the host, not the engine: the **event loop**, the mechanism that runs your callbacks (functions you hand over to be called later) when a timer fires or data arrives. It is the reason JavaScript can do many things "at once" on one thread; it gets its own chapter (Chapter 12).

Self-check

1. Which engine does Node.js use? → V8.
2. When is a `SyntaxError` reported? → While parsing, before any code in that script runs.
3. What does "deoptimisation" mean? → The engine discards optimised machine code because an assumption it made (e.g. about value types) turned out false, and continues in slower bytecode.

TL;DR

An engine (V8, SpiderMonkey, JavaScriptCore) parses your code into an AST, runs it as bytecode and JIT-compiles hot parts to machine code. Everything beyond the core language comes from the host.

1.4 Where JavaScript runs — hosts

What it is

A **host environment** (or **runtime**) is a program that embeds a JS engine and gives your code extra objects and functions — **host APIs** — for talking to the outside world.

Host	Engine	Adds (examples)	Typical use
Web browser	V8 / SpiderMonkey / JSC	`window`, `document` (the **DOM**), `fetch`, `localStorage`, events, `setTimeout`	Interactive web pages (Ch. 13, Ch. 14)
Node.js	V8	`process`, `fs` (files), `http` (servers), `path`, `Buffer`, plus `fetch` and `setTimeout`	Servers, CLIs (command-line programs), build tools (programs that prepare code for release) (Ch. 15)
Deno	V8	Web-style APIs, built-in TypeScript, permission system	Servers and scripts
Bun	JavaScriptCore	Node-compatible APIs, bundler (combines many code files into a few), test runner	Servers, tooling
Others	various	React Native (mobile), Electron (desktop = Chromium, the open-source core of Chrome, + Node), Cloudflare Workers (edge)	Apps and edge functions

The **language** is the same everywhere. What changes is the set of host objects. The **global object** (the top-level object whose properties are visible everywhere as global variables) also has different names — `window` in browsers, `global` in Node — so ES2020 added **`globalThis`**, which works in all of them.

Example — asking "where am I?"

```
// code/04_where_am_i.js
console.log("typeof globalThis :", typeof globalThis);  // standard name for the global object everywhere
console.log("typeof window     :", typeof window);      // only in browsers
console.log("typeof document   :", typeof document);    // only in browsers (the DOM)
console.log("typeof process    :", typeof process);     // only in Node.js
console.log("Node version      :", process.version.split(".")[0] + ".x");
console.log("V8 is the engine  :", typeof process.versions.v8 === "string");
console.log("Math.max(3, 7)    :", Math.max(3, 7));     // ECMAScript built-in
console.log("typeof setTimeout :", typeof setTimeout);  // host API (browser and Node both have it)
console.log("typeof fetch      :", typeof fetch);       // web API, also built into Node 18+
```

Output (Node 24):

```
typeof globalThis : object
typeof window     : undefined
typeof document   : undefined
typeof process    : object
Node version      : v24.x
V8 is the engine  : true
Math.max(3, 7)    : 7
typeof setTimeout : function
typeof fetch      : function
```

`typeof` (explained in Chapter 2) is safe to use on a name that doesn't exist — it returns `"undefined"` instead of throwing — which makes it the classic way to feature-detect (check whether a feature exists

before using it).

Client-side vs server-side

- **Client-side JavaScript** runs in the user's browser. It reacts to clicks and typing and updates the page *without reloading it*, which is what makes modern web apps feel responsive.
- **Server-side JavaScript** runs on a server (usually Node.js). It handles requests (messages from a client asking for something, usually over HTTP, the web's request/response protocol), talks to databases (programs that store and search data) and sends responses (System Design Fundamentals). Using JS on both sides means a team can build the whole application — front end and back end — in one language and even share code (for example, validation rules).

Common mistakes

- Calling `document.querySelector` in a Node script → `ReferenceError: document is not defined`. It's a browser API. (For testing, libraries like jsdom emulate the DOM in Node — this book uses it in Chapter 13.)
- Using `require` (Node's older way of loading another file, Chapter 10) in browser code or `window` in Node code. Check which host you are in.

TL;DR

Engine = runs the language; host = engine + APIs. Browsers give you the DOM, Node gives you files, processes and servers; `globalThis` is the portable name for the global object.

1.5 Setting up and running code

There are four everyday ways to run JavaScript. You will use all of them.

1. The browser console (zero install)

1. Open any web page (even a blank tab, `about:blank`).
2. Open **DevTools** (the developer tools built into every browser): `F12`, or `Ctrl+Shift+J` (Windows/Linux) / `Cmd+Option+J` (Mac) in Chrome and Edge; `Ctrl+Shift+K` / `Cmd+Option+K` in Firefox; in Safari first enable *Settings → Advanced → Show features for web developers*, then `Cmd+Option+C`.
3. Click the **Console** tab, type `2 + 3` and press Enter. The console is a **REPL** (**R**ead a line, **E**valuate it, **P**rint the result, **L**oop), so it prints `5`.

Good for quick experiments and for inspecting a live page. Code you type there is lost when you close the tab.

2. A `<script>` tag in an HTML page

A web page loads JavaScript with the `<script>` element — either inline code or a separate file via `src`:

```
<!-- code/hello.html (key parts) -->
<head>
  <meta charset="utf-8">
  <title>Hello from JavaScript</title>
  <!-- defer: download now, run after the HTML is parsed, so the #out element exists -->
  <script src="hello.js" defer></script>
</head>
<body>
  <h1>My first page with JavaScript</h1>
  <p>The box below is empty in the HTML file. The script fills it in:</p>
  <div id="out"></div>
  <script>
    // An inline script runs as soon as the parser reaches it.
    console.log("inline script ran");   // visible in DevTools > Console
  </script>
</body>
```

```
// code/hello.js — a browser script (uses the DOM, so it can't run in Node)
const out = document.getElementById("out");
const now = new Date(2026, 0, 15, 9, 30); // fixed date so the screenshot is reproducible
out.innerHTML =
  "<strong>Hello, World!</strong> This text was written by <code>hello.js</code>.<br>" +
  "2 + 3 = " + (2 + 3) + " &middot; page built at " + now.toLocaleTimeString("en-US");
console.log("hello.js ran");
```

Opening `hello.html` in a browser gives:

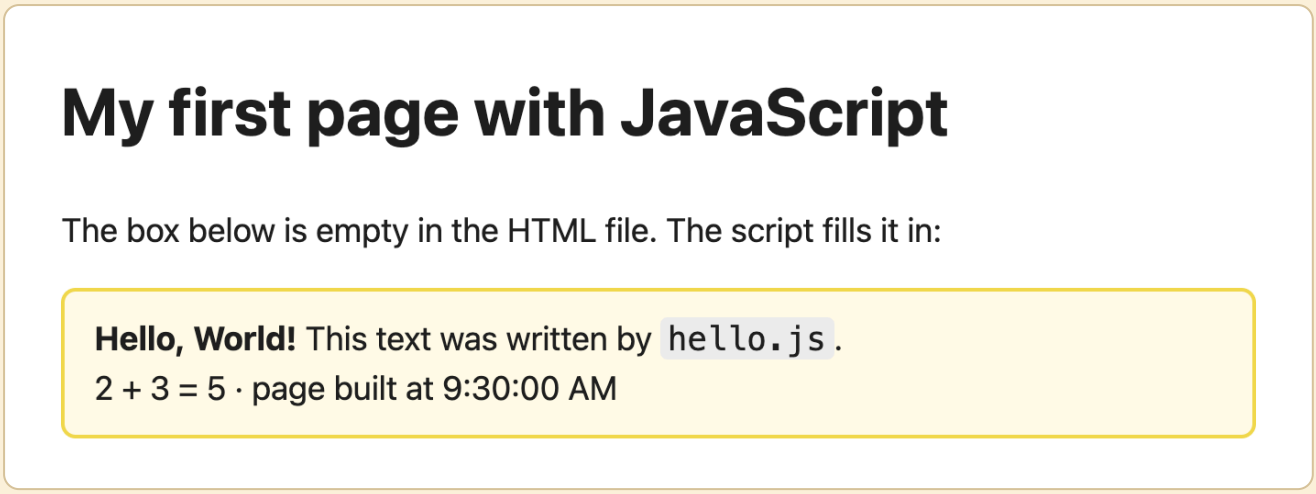


Figure — `hello.html` rendered in headless Chrome (Chrome run by a program, with no visible window). The yellow box was empty in the HTML; the script wrote its contents.

In the Console you would see `inline script ran` first and `hello.js ran` second: the inline script runs as soon as the parser reaches it, while the deferred script waits until the whole document has been parsed.

How a `<script>` is loaded matters, because a plain script **pauses HTML parsing** while it downloads and runs:

Form	Downloads	Runs	Order kept?	Use for
`<script src>` (in `<head>`)	blocks parsing	immediately	yes	rarely — it delays the page
`<script src>` at end of `<body>`	when reached	immediately	yes	the classic old fix
`<script src defer>`	in parallel	after HTML is parsed	**yes**	most page scripts
`<script src async>`	in parallel	as soon as downloaded	**no**	independent scripts (analytics)
`<script type="module" src>`	in parallel	after parsing (deferred by default)	yes	modern code using `import` (Ch. 10)

3. Node.js from the terminal

1. Install Node.js from nodejs.org — pick the **LTS** (long-term support) version. Even-numbered Node major versions (the first number, e.g. 24 in 24.1.0) become LTS; this book uses Node 24 (LTS since October 2025).
2. Check it: `node --version` → `v24.x.x`.
3. Save code in a file, e.g. `hello.js`, and run `node hello.js`. Output appears in the terminal (the text window where you type commands).

4. The Node REPL

Type `node` with no file name. You get a `>` prompt that evaluates each line, just like the browser console. `.help` lists commands, `.exit` (or `Ctrl+D`) quits. The special variable `_` holds the last

result.

```
$ node
> 2 + 3
5
> _ * 10
50
> .exit
```

An editor

For anything longer than a few lines, use a code editor. **Visual Studio Code** (free) is the most common choice for JavaScript: syntax highlighting, error squiggles, an integrated terminal (so you can run `node file.js` in the same window) and a debugger (a tool that pauses your program so you can inspect values step by step — Chapter 11). Online playgrounds (CodePen, JSFiddle, StackBlitz) are handy for sharing small experiments.

Common mistakes

- Putting a plain `<script src>` in `<head>` that touches page elements → `null` errors, because the elements don't exist yet. Use `defer`.
- Opening the HTML file and looking for `console.log` output *on the page*. It goes to the DevTools Console, not the page.
- Mixing up the Node REPL (`>` prompt) and the system shell (`$` prompt; the shell is the program that reads your terminal commands): `node hello.js` must be typed in the shell, not inside the REPL.

TL;DR

Console and REPL for quick experiments; `<script defer>` (or `type="module"`) to run code in a page; `node file.js` to run code outside the browser; VS Code to edit.

1.6 Hello, World! — your first program and the basic rules

What it is

The tradition when learning a language is to make the computer print "Hello, World!". In JavaScript that is one line:

```
// code/01_hello_world.js — run with:  node 01_hello_world.js
console.log("Hello, World!");          // double quotes
console.log('Hello, JavaScript!');     // single quotes work the same way
console.log(`2 + 3 = ${2 + 3}`);       // backticks = template literal, can embed expressions
console.log("Many", "values", 42, true); // console.log accepts several arguments, separated by spaces
```

Output:

```
Hello, World!
Hello, JavaScript!
2 + 3 = 5
Many values 42 true
```

What each part does:

- **`console`** is an object (a bundle of named values and functions — Chapter 5) provided by the host (browser or Node) for writing diagnostic messages.
- **`.log(...)`** is a *method* (a function belonging to an object) that prints its arguments (the values you pass between the parentheses). In the browser the message goes to the DevTools Console; in Node it goes to the terminal (standard output, the default place a program writes its text).
- **`"Hello, World!"`** is a **string** — a piece of text. Text can be written between double quotes `"…"` or single quotes `'…'` — they behave identically; pick one style and stay consistent. **Backticks** `` `…` `` create *template literals*, which can embed any expression inside `${ }` (Chapter 6).

- The line ends with a **semicolon** `;`, which ends the statement.

`console.log` is how you will look at values throughout this book, and it remains a first-line debugging tool for professionals. Its siblings — `console.error`, `console.warn`, `console.table`, `console.time` — are covered in Chapter 11.

Try it yourself: change the message to `"Hello, <your name>!"` and run the file again.

The basic rules of the syntax

Rule	Detail
Statements	A program is a list of statements run top to bottom. A statement does something (`console.log(...)`, `let x = 5`).
Expressions	Anything that produces a value: `2 + 3`, `"hi"`, `x > 1`, `max(a, b)`. Expressions can appear inside statements.
Case-sensitive	`console`, `Console` and `CONSOLE` are different names; only the first exists.
Identifiers	Names of variables and functions: letters, digits, `_` and `$`, not starting with a digit. `userName`, `_count`, `$el` are fine; `2fast` is not. Reserved words (`let`, `class`, `return` …) can't be used as names. Convention: **camelCase** (first word lowercase, later words capitalised: `totalPrice`).
Blocks	Curly braces `{ … }` group statements (after `if`, `for`, function bodies).
Whitespace	Spaces, tabs and newlines between tokens (the smallest meaningful pieces: names, numbers, symbols) are mostly ignored, so format for readability (a formatter like Prettier does it for you — Chapter 10).

Semicolons and ASI

Semicolons end statements, but JavaScript has **Automatic Semicolon Insertion (ASI)**: if a line break appears where the grammar can't continue, the parser pretends a semicolon is there. So code without semicolons usually works. Two situations where it doesn't do what you expect:

```
function f() {
  return          // ASI inserts ";" right here, because a newline follows "return"
    { ok: true }; // never reached — f() returns undefined
}

const a = 1
const b = a
[1, 2].forEach(n => console.log(n)) // parsed as  a[1, 2].forEach(...)  -> TypeError
```

Rules of thumb: always put the returned value on the same line as `return`; if you skip semicolons, start any line that begins with `(`, `[` or `` ` `` with a `;`. Most teams either always use semicolons or use a formatter that handles this — this book always uses them.

Self-check

1. Where does `console.log` output appear in a browser? → In the DevTools Console, not on the page.
2. Are `'hi'` and `"hi"` different? → No, both are the same string.
3. Why does `return` followed by a newline and then a value return `undefined`? → ASI inserts a semicolon after `return`.

TL;DR

`console.log(value)` prints values; strings use `'…'`, `"…"` or `` `…` ``; code is case-sensitive; end statements with `;` and never break a line straight after `return`.

1.7 Comments

What it is

A **comment** is text in your source file that the engine ignores completely. It exists only for human readers — your teammates, and you six months from now. Comments have no effect on what the program does or how fast it runs.

Two kinds of comments are part of the language, plus one widely used convention:

Kind	Syntax	Ends at	Typical use
Single-line	`// text`	end of the line	Short notes; switching off one line
Multi-line (block)	`/* text */`	the closing `*/` (may span many lines)	Longer explanations; switching off several lines
JSDoc (a convention built on block comments)	`/** … */` with tags like `@param`, `@returns`	`*/`	Documenting functions; editors show it as hover help and can type-check with it (Chapter 16)

Example

```
// code/02_comments.js
// A single-line comment runs from // to the end of the line.
console.log("Line 1 runs");   // a comment can also follow code on the same line
// console.log("Line 2 is switched off, so nothing prints for it");

/*
  A multi-line (block) comment can span many lines.
  Handy for a longer explanation, or to switch off a whole block:
console.log("switched off too");
console.log("and this");
*/
console.log("Line 3 runs");

/**
 * A JSDoc comment: starts with slash-star-star. Editors and tools read the tags.
 * @param {number} celsius - temperature in degrees Celsius
 * @returns {number} the same temperature in Fahrenheit
 */
function toFahrenheit(celsius) {
  // Why, not what: 9/5 and 32 come from the Fahrenheit scale definition.
  return celsius * 9 / 5 + 32;
}
console.log("100 C =", toFahrenheit(100), "F");

const total = 10 /* inline block comments are legal too */ + 5;
console.log("total =", total);
```

Output:

```
Line 1 runs
Line 3 runs
100 C = 212 F
total = 15
```

Notice that the three `console.log` calls inside comments printed nothing. Turning code into a comment ("**commenting out**") is a quick way to disable it temporarily while debugging without deleting it. In VS Code, `Ctrl+/` (`Cmd+/` on Mac) toggles `//` on the selected lines.

Best practices

1. **Explain *why*, not *what*.** `i++ // add one to i` repeats the code. `// the API counts pages from 1, not 0` tells the reader something the code can't.

2. **Keep comments in sync.** When you change code, update or delete its comment. A wrong comment is worse than none.
3. **Don't over-comment.** Prefer clear names (`isLoggedIn`, `totalPrice`) and small functions so the code explains itself; add comments where intent, a trade-off or a gotcha isn't obvious.
4. **Delete commented-out code before committing** (saving a snapshot of your changes in version control). Version control (git — a tool that records every saved version of your files) remembers old code; dead blocks just clutter the file.
5. Use conventional markers sparingly: `// TODO:` for planned work, `// FIXME:` for known problems — many editors list them.

Common mistakes

- **Nesting block comments.** `/* outer /* inner */ still outer? */` — the first `*/` ends the comment, so `still outer? */` becomes code and causes a `SyntaxError`. Use `//` on each line when commenting out code that already contains block comments.
- **Putting secrets or passwords in comments.** Browser JavaScript is downloaded by every visitor; anyone can read it, comments included (unless a build step — an automatic processing pass before publishing — strips them).
- **HTML comments inside JS.** `<!-- -->` is HTML syntax. (For historical reasons it is still treated as a single-line comment in classic browser scripts, but never rely on that.)

TL;DR

`//` for one line, `/* … */` for many, `/** … */` for JSDoc. Comments are ignored by the engine; write them to explain intent, keep them current, and prefer self-explanatory code.

1.8 Strict mode

What it is

Strict mode is an opt-in, stricter variant of JavaScript introduced in ES5 (2009). It turns several silent mistakes into errors and removes a few confusing features. You switch it on with the **directive** `"use strict";` as the very first statement of a script or of a function. Code without it runs in what is informally called **sloppy mode**.

Crucially, **modern code is usually strict automatically**:

- every **ES module** (files loaded with `import`/`export`, `<script type="module">`, `.mjs` files in Node) is strict;
- every **class** body is strict.

So if you write modern modular code you are already in strict mode, and the directive is only needed in classic scripts.

The problem it solves

Early JavaScript tried hard never to complain: a misspelt variable silently created a new global, writing to a read-only property (a **property** is a named value stored on an object) silently did nothing. That made bugs hard to find. Because the web can't be broken (§1.2), these behaviours couldn't simply be changed — so strict mode was added as an opt-in "fixed" version.

Example — sloppy vs strict

```
// code/03_strict_mode.js — a classic CommonJS script, so it starts in sloppy mode
function sloppyTypo() {
  countr = 1;               // typo: meant "counter". Sloppy mode silently creates a global!
  return typeof globalThis.countr;
}
console.log("sloppy: accidental global created ->", sloppyTypo());

function strictTypo() {
  "use strict";
  try {
    countr2 = 1;            // strict mode refuses to invent a global
  } catch (e) {
    return `${e.name}: ${e.message}`;
  }
}
console.log("strict:", strictTypo());

function sloppyThis() { return this === globalThis; }
function strictThis() { "use strict"; return this; }
console.log("plain call, sloppy this is globalThis ->", sloppyThis());
console.log("plain call, strict this ->", strictThis());
// ... (frozen-object and duplicate-parameter checks, see the file)
```

Output:

```
sloppy: accidental global created -> number
strict: ReferenceError: countr2 is not defined
plain call, sloppy this is globalThis -> true
plain call, strict this -> undefined
write to frozen object: sloppy keeps 1 | strict throws TypeError
duplicate params in strict code -> SyntaxError
```

(The example file is a **CommonJS** script — Node's original module format, which loads files with `require`. `try`/`catch` catches an error so the program can continue and print it — see Chapter 11. `this` is explained in Chapter 9.)

What strict mode changes

Situation	Sloppy mode	Strict mode
Assigning to an undeclared variable (`x = 1`)	creates a global variable	`ReferenceError`
`this` in a plain function call `f()`	the global object	`undefined`
Writing to a read-only or frozen (locked with `Object.freeze`) property, adding to a non-extensible (no new properties allowed) object	silently ignored	`TypeError`
Deleting a non-deletable property (`delete Object.prototype`)	returns `false`	`TypeError`
Duplicate parameter names `function f(a, a)`	allowed	`SyntaxError`
Legacy octal (base-8) literal `010`	means 8	`SyntaxError` (use `0o10`)
`with` statement (treats an object's properties as variables)	allowed	`SyntaxError`
`eval("var x = 1")` (runs a string as code)	leaks `x` into the surrounding scope (the region of code where a name is visible)	`x` stays inside the eval
`arguments` object	stays linked to the parameters	an independent copy (Chapter 4)
Future reserved words (`implements`, `interface`, `package`, `private`, `protected`, `public`, `static`, `yield`, `let`) as names	allowed	`SyntaxError`

How to use it

1. **Modules** (`.mjs`, `"type": "module"` in `package.json` (the project's settings file, Chapter 10), `<script type="module">`): nothing to do — already strict.
2. **Classic scripts**: put `"use strict";` as the first statement of the file (only comments may come before it). It applies to the whole script.
3. **One function only**: put it as the first statement of the function body.

Common mistakes

- **Placing the directive after other code.** It must be the first statement; otherwise it is just an unused string and does nothing.
- **Concatenating files.** If a strict file is glued before a sloppy one in a single classic script, the directive at the top now applies to both (or vice versa). Bundlers handle this; hand-concatenation doesn't.
- **Adding `"use strict"` to a function with default, rest or destructured parameters** (fancier parameter forms from Chapter 4) — that is itself a `SyntaxError`. Put the directive at the file level instead.
- **Assuming Node `.js` files are strict.** A `.js` file in Node is a CommonJS script (sloppy) unless `package.json` says `"type": "module"` or the file uses `import`/`export` syntax. The code in this book's early chapters uses plain `.js` files for that reason, and turns strictness on where it matters.

In the interview

30-second answer — "What is strict mode?" An opt-in restricted variant of JavaScript enabled with `"use strict"` at the top of a script or function, and automatically on in ES modules and classes. It turns silent errors into thrown ones (undeclared assignments, writes to read-only properties), makes `this` `undefined` in plain function calls, and forbids error-prone syntax like `with`, duplicate parameters and legacy octals.

- *Follow-up: Do you still need to write `"use strict"`?* Not in ES modules or class bodies; only in classic scripts or CommonJS files.
- *Follow-up: Why not make strict mode the default everywhere?* Because it would break existing sites that rely on sloppy behaviour; the web must stay backward compatible (new versions must not break code written for old ones).
- *Follow-up: Why does strict mode make `this` undefined?* So that a function accidentally called without an object can't silently read or modify the global object.

Self-check

1. In a `.mjs` file, does `x = 5` (no declaration) work? → No; modules are strict, so it throws `ReferenceError`.
2. Where must `"use strict"` appear? → As the first statement of the script or function body.
3. What is `this` inside `function f() { "use strict"; return this; }` when called as `f()`? → `undefined`.

TL;DR

Strict mode turns silent failures into errors and removes risky features. Modules and classes are strict automatically; in classic scripts, start with `"use strict";`.

Chapter 1 TL;DR

- **JavaScript** is a high-level, dynamically typed, garbage-collected, multi-paradigm language; created in 1995, standardised as **ECMAScript**, maintained by **TC39**, updated every June (this book: ES2025).
- An **engine** (V8, SpiderMonkey, JavaScriptCore) parses code into an AST, runs bytecode and JIT-compiles hot code. A **host** (browser, Node.js, Deno, Bun) wraps the engine and adds APIs like the DOM or the file system; `globalThis` is the global object everywhere.
- Run code in the **browser console**, in a page with **`<script defer>`** (or `type="module"`), with **`node file.js`**, or in the **Node REPL**.
- `console.log` prints; strings take `'`, `"` or `` ` ``; code is case-sensitive; use semicolons and never break a line after `return`.
- Comments: `//`, `/* */`, `/** JSDoc */` — explain *why*, keep them current.
- **Strict mode** (`"use strict"`, automatic in modules and classes) turns silent mistakes into errors.

Next: Chapter 2 — Variables, Types and Operators.

Chapter 2 — Variables, Types and Operators

Every program stores values and computes new values from them. This chapter covers the three tools for that: **variables** (names for values), **types** (what kind of value something is) and **operators** (the symbols that combine values: `+`, `===`, `&&`, `??` …). It also covers the part of JavaScript that surprises newcomers most — **type coercion**, the automatic conversion of one type into another — so that `"3" + 4`, `0 == ""` and `0.1 + 0.2` stop being mysteries.

What you'll learn

1. Declaring variables with `let`, `const` and `var`, and which one to choose.
2. Hoisting and the Temporal Dead Zone (TDZ).
3. The seven primitive types, objects, and the `typeof` operator (with its quirks).
4. Primitives vs objects: copying by value vs by reference.
5. Arithmetic, comparison, logical, nullish (`??`), optional-chaining (`?.`), bitwise and assignment operators, and precedence.
6. `==` vs `===` and the exact coercion rules.
7. Numbers in depth: floating-point precision, `NaN`, `Infinity`, safe integers, `Number` properties and methods, `BigInt`.
8. Booleans, truthy and falsy values, and explicit conversions.

Code: `code/` — every "Output:" block is real output from Node 24. Files are plain `.js` (sloppy-mode scripts) unless noted; Chapter 1 §1.8 explains the difference.

2.1 Variables: `let`, `const` and `var`

What it is

A **variable** is a named storage slot for a value. You pick a name, put a value in it, and later use the name to get the value back — or put a different value in.

Analogy: a labelled box. The label (`score`) stays the same; what's inside can be swapped. A `const` is a box whose label is glued to one particular thing: you can't swap the contents for something else, although if the thing is itself a container (an object) you can still rearrange what's inside it.

Three words matter:

- **Declare** — create the name: `let score;`
- **Assign** — store a value: `score = 10;`
- **Initialise** — declare and assign in one step: `let score = 10;`

A declared-but-unassigned variable holds the special value `undefined`.

How it works — the three keywords

```
// code/01_declarations.js
let score;                    // declared, no value yet
console.log("score before assignment:", score);
score = 10;                   // assign later
score = score + 5;            // reassign (let allows it)
console.log("score:", score);

const PI_APPROX = 3.14;       // const must be initialised and can't be reassigned
try { eval("PI_APPROX = 3"); } catch (e) { console.log("reassign const ->", e.name + ":", e.message); }

// const protects the BINDING, not the object: the contents can still change.
const cart = { items: 1 };
cart.items = 2;               // allowed: we mutate the object, not the variable
console.log("const object mutated:", cart);

// var: the old way. Function-scoped, can be redeclared, hoisted with value undefined.
var legacy = "first";
var legacy = "second";        // redeclaring with var is silently allowed
console.log("var redeclared:", legacy);
try { eval("let dup = 1; let dup = 2;"); } catch (e) { console.log("let redeclared ->", e.name); }

let a = 1, b = 2, c = a + b;  // several declarations in one statement
console.log("a, b, c =", a, b, c);

const greeting = "Hello";
const who = "World";
console.log(greeting + " " + who);
```

Output:

```
score before assignment: undefined
score: 15
reassign const -> TypeError: Assignment to constant variable.
const object mutated: { items: 2 }
var redeclared: second
let redeclared -> SyntaxError
a, b, c = 1 2 3
Hello World
```

(`eval` runs a string as code; it is used here only so a deliberate error can be caught and printed. Don't use it in real code.)

Comparison table

	`let`	`const`	`var`
Introduced	ES2015	ES2015	original (1995)
Scope (where the name is visible)	**block** `{ }`	**block** `{ }`	**function** (or global)
Reassign	yes	**no** (`TypeError`)	yes
Redeclare in same scope	no (`SyntaxError`)	no	**yes, silently**
Must initialise	no	**yes**	no
Use before the declaration line	`ReferenceError` (TDZ, §2.2)	`ReferenceError` (TDZ)	gives `undefined`
At top level of a classic script, becomes a property of `globalThis`	no	no	**yes**

Scope — *where* a variable is visible — is covered properly in Chapter 4 §4.5. For now: a `let`/`const` declared inside `{ }` exists only inside those braces; a `var` leaks out to the whole function.

Which one should I use?

1. `const` **by default.** Most names never need to point at something else, and `const` tells the reader so.
2. `let` **when the value must change** — counters, accumulators, values assigned in branches.
3. `var` **never, in new code.** You need to *recognise* it because older code and tutorials use it everywhere (and some lessons you'll see online use `var` "to keep things simple"), but its function scope, silent redeclaration and hoisting cause bugs that `let`/`const` were designed to prevent.

Naming rules and conventions

- Allowed characters: letters, digits, `_`, `$`; the first character can't be a digit. Unicode letters work (`café`) — Unicode is the worldwide standard that gives every character in every script a number — but stick to ASCII (the basic 128-character English set: plain letters, digits, punctuation) in shared code.
- Case matters: `total` and `Total` are different variables.
- Reserved words (`let`, `class`, `return`, `if`, `for`, `new`, `typeof` …) can't be names.
- Conventions: **camelCase** for variables and functions (`totalPrice`), **PascalCase** for classes (`ShoppingCart`; a class is a blueprint for making objects — Chapter 9), **UPPER_SNAKE_CASE** for fixed configuration constants (`MAX_RETRIES`). Name booleans as questions: `isOpen`, `hasAccess`.

Common mistakes

- **Thinking `const` makes an object read-only.** It only prevents reassigning the variable. To freeze the contents, use `Object.freeze` (Chapter 5).
- **Forgetting the keyword** — `total = 5` without `let`/`const` creates an accidental global in sloppy mode and throws in strict mode.
- **Declaring with `let` inside a block and using it outside** → `ReferenceError`.

TL;DR

`const` by default, `let` when you must reassign, `var` only when reading old code. `const` freezes the binding (the link between a name and the value it points to), not the value.

2.2 Hoisting and the Temporal Dead Zone

What it is

Before running any code in a scope, the engine first scans it and creates all declared names. This is called **hoisting** (as if the declarations were "hoisted" to the top). What differs is *what value the name has before its declaration line runs*:

Declaration	Before its line runs
`var x`	exists, value `undefined`
`let x`, `const x`, `class X`	exists but **uninitialised** — touching it throws `ReferenceError`
`function f() {}`	exists and is **fully usable** (covered in Chapter 4)

The period between entering the scope and the declaration line for a `let`/`const` is the **Temporal Dead Zone (TDZ)**.

Example

```
// code/02_tdz_and_hoisting.js
console.log("var before its line:", typeof hoistedVar, hoistedVar); // declared + set to undefined at start
var hoistedVar = "ready";

try {
  console.log(notYet);        // let exists but is uninitialised until its line runs
} catch (e) {
  console.log("let before its line ->", e.name + ":", e.message);
}
let notYet = "ready";

try { typeof inZone; let inZone = 1; } catch (e) { console.log("typeof in TDZ ->", e.name); }
{
  // The TDZ is about TIME, not position: a function may mention a later let,
  // as long as it is called after the let line has run.
  const show = () => later;
  let later = "defined by the time show() is called";
  console.log(show());
}
```

Output:

```
var before its line: undefined undefined
let before its line -> ReferenceError: Cannot access 'notYet' before initialization
typeof in TDZ -> ReferenceError
defined by the time show() is called
```

Why it matters

The TDZ is a feature: with `var`, reading a variable too early silently gives `undefined` and the bug surfaces far away; with `let`/`const` you get an immediate, clearly named error. Note that `typeof` — normally safe on unknown names — also throws inside the TDZ.

In the interview

"What is the Temporal Dead Zone?" `let`, `const` and `class` declarations are hoisted — the scope knows about them from the start — but they aren't initialised until their declaration is evaluated. Accessing them in that window throws a `ReferenceError`. `var` is hoisted *and* initialised to `undefined`, so early access silently returns `undefined`.

- *Follow-up: Are `let` and `const` hoisted?* Yes — which is why an inner `let x` shadows an outer `x` (hides it, because the inner name wins inside its block) even on lines above it (you get a TDZ error instead of the outer value).
- *Follow-up: Is the TDZ about lines or time?* Time: a closure (a function that remembers the variables around where it was created — Chapter 4) that refers to a later `let` is fine as long as it runs after the declaration executes.

TL;DR

All declarations are hoisted; `var` starts as `undefined`, `let`/`const` start in the TDZ and throw if touched early.

2.3 Data types and `typeof`

What it is

Every value has a **type**. JavaScript has **eight** types: seven **primitive** types and one **object** type.

Type	Example values	Notes
`number`	`42`, `3.14`, `-0`, `Infinity`, `NaN`	One type for integers and decimals: 64-bit floating point (§2.10)
`bigint`	`10n`, `2n ** 64n`	Whole numbers of any size (ES2020)
`string`	`"hi"`, `'hi'`, `` `hi ${x}` ``	Text, immutable (can't be changed in place) (Chapter 6)
`boolean`	`true`, `false`	§2.11
`undefined`	`undefined`	"No value assigned yet" — what JS itself gives you
`null`	`null`	"Intentionally empty" — what *you* assign to mean "nothing"
`symbol`	`Symbol("id")`	Unique identifiers, mostly for object keys (property names) (Chapter 8)
`object`	`{ a: 1 }`, `[1, 2]`, functions, dates, maps …	Everything that isn't a primitive

JavaScript is **dynamically typed**: variables don't have types, values do. The same variable can hold a number and later a string.

`typeof` — asking a value its type

```
// code/03_types_typeof.js (abridged)
const samples = [
  ["number", 42], ["number (float)", 3.14], ["NaN", NaN], ["bigint", 10n],
  ["string", "hi"], ["boolean", true], ["undefined", undefined], ["null", null],
  ["symbol", Symbol("id")], ["object", { a: 1 }], ["array", [1, 2]],
  ["function", function () {}], ["date", new Date(0)],
];
for (const [label, value] of samples) {
  console.log(label.padEnd(15), "typeof ->", typeof value);
}
console.log("Array.isArray([1,2]) ->", Array.isArray([1, 2]));
let box = 5;      console.log("box:", typeof box);
box = "five";     console.log("box:", typeof box);
```

Output:

```
number          typeof -> number
number (float)  typeof -> number
NaN             typeof -> number
bigint          typeof -> bigint
string          typeof -> string
boolean         typeof -> boolean
undefined       typeof -> undefined
null            typeof -> object
symbol          typeof -> symbol
object          typeof -> object
array           typeof -> object
function        typeof -> function
date            typeof -> object
Array.isArray([1,2]) -> true
value === null      -> true
box: number
box: string
```

Three quirks to memorise:

1. **`typeof null === "object"`** — a bug from the very first implementation that can never be fixed (it would break the web). Test for null with `value === null`.
2. **`typeof NaN === "number"`** — "Not-a-Number" is a special *number* value.
3. **Functions report `"function"`**, even though they are objects; arrays report `"object"` — use `Array.isArray` to detect them.

`typeof` always returns a string, and it is safe on undeclared names (`typeof notDeclared` → `"undefined"`), except inside a TDZ.

`undefined` vs `null`

	`undefined`	`null`
Meaning	value not set	deliberately empty
Who produces it	the language: unassigned variables, missing properties, missing arguments, functions without `return`	your code (and some APIs, e.g. `document.getElementById` when nothing matches)
`typeof`	`"undefined"`	`"object"` (quirk)
`Number(…)`	`NaN`	`0`
`== null`	`true`	`true`

A handy idiom (a common, widely recognised coding pattern): `value == null` is true for *both* `null` and `undefined` and nothing else — the one widely accepted use of `==`.

TL;DR

Seven primitives (`number`, `bigint`, `string`, `boolean`, `undefined`, `null`, `symbol`) plus objects. `typeof` identifies them, except `null` (→ `"object"`) and arrays (use `Array.isArray`).

2.4 Primitives vs objects

What it is

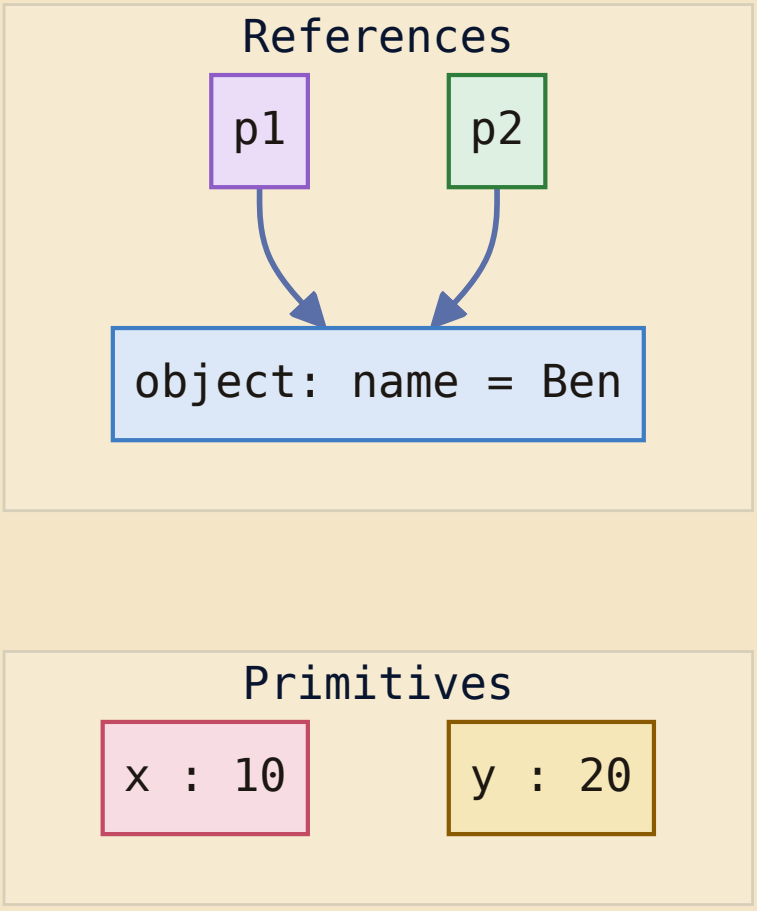
- A **primitive** is a simple, **immutable** value. The variable holds the value itself. Copying copies the value.
- An **object** is a mutable (changeable after creation) collection of properties (named values) living in memory. The variable holds a **reference** — like an address — to the object. Copying copies the address, so both variables point at the *same* object.

Example

```
// code/04_primitives_vs_objects.js
let x = 10;
let y = x;        // copies the value 10
y = 20;
console.log("x, y:", x, y);

const p1 = { name: "Ana" };
const p2 = p1;    // copies the REFERENCE — both names point at one object
p2.name = "Ben";
console.log("p1.name:", p1.name);

console.log("{} === {}:", {} === {});           // two different objects
console.log("p1 === p2:", p1 === p2);           // same object
console.log("'a' === 'a':", "a" === "a");       // primitives compare by value

const s = "hello";
const upper = s.toUpperCase();                  // string methods return NEW strings
console.log(s, upper);
console.log("length of 'hello':", s.length, "| (255).toString(16):", (255).toString(16));
console.log("typeof new Number(5):", typeof new Number(5), "| typeof Number('5'):", typeof Number("5"));
```

Output:

```
x, y: 10 20
p1.name: Ben
{} === {}: false
p1 === p2: true
'a' === 'a': true
hello HELLO
length of 'hello': 5 | (255).toString(16): ff
typeof new Number(5): object | typeof Number('5'): number
```



Autoboxing and wrapper objects

Primitives have no properties, yet `"hello".length` and `(255).toString(16)` work. When you access a property on a primitive, the engine temporarily wraps it in a matching **wrapper object** (`String`, `Number`, `Boolean`, `BigInt`, `Symbol`), reads the property, and throws the wrapper away. This is called **autoboxing**.

You can also create wrappers yourself with `new Number(5)`, `new String("x")` or `new Boolean(false)`, but **don't**: they are objects, so `typeof` says `"object"`, `new Number(5) === 5` is `false`, and `new Boolean(false)` is *truthy* (§2.11). Calling the same functions **without** `new` — `Number("5")`, `String(5)`, `Boolean(0)` — is the correct way to *convert* values and returns primitives.

Why it matters

- Passing an object to a function lets the function change *your* object (Chapter 4).
- `===` on objects asks "same object?", not "same contents?". Comparing contents needs a loop or a library (ready-made code written by someone else).
- Frameworks like React rely on this: they detect a change by checking whether you created a *new* object, which is why "don't mutate state" is a React rule (*mutate* = change in place; *state* = the data that decides what the screen currently shows).

TL;DR

Primitives: immutable, copied and compared by value. Objects: mutable, copied and compared by reference. Use `Number()/String()/Boolean()` without `new` to convert.

2.5 Arithmetic operators

What it is

An **operator** is a symbol that takes one or more values (**operands**) and produces a new value. Unary operators take one operand (`-x`, `!x`), binary take two (`a + b`), and there is one ternary operator (`c ? a : b`, Chapter 3).

Operator	Name	Example	Result
`+`	addition (or string concatenation — joining texts end to end)	`10 + 5`	`15`
`-`	subtraction	`10 - 5`	`5`
`*`	multiplication	`10 * 5`	`50`
`/`	division (always floating point)	`7 / 2`	`3.5`
`%`	remainder	`10 % 3`	`1`
`**`	exponentiation (ES2016)	`2 ** 10`	`1024`
`++`	increment by 1	`n++` / `++n`	see below
`--`	decrement by 1	`n--` / `--n`	see below
unary `+` / `-`	convert to number / negate	`+"42"`	`42`

Example

```
// code/05_arithmetic.js
let a = 10, b = 5;
console.log("a + b =", a + b);
console.log("a - b =", a - b);
console.log("a * b =", a * b);
console.log("a / b =", a / b);
console.log("a % b =", a % b);
console.log("2 ** 10 =", 2 ** 10);
console.log("7 / 2 =", 7 / 2, "(no integer division)");
console.log("Math.trunc(7 / 2) =", Math.trunc(7 / 2));
console.log("-7 % 3 =", -7 % 3, "(sign follows the left operand)");
console.log("1 / 0 =", 1 / 0, "| -1 / 0 =", -1 / 0, "| 0 / 0 =", 0 / 0);

let n = 5;
console.log("n++ returns", n++, "then n is", n);   // returns old value
console.log("++n returns", ++n, "then n is", n);   // returns new value
b--; console.log("after b--:", b);

console.log('"3" + 4 =', "3" + 4);
console.log('"3" - 4 =', "3" - 4);
console.log('"3" * "4" =', "3" * "4");
console.log('1 + 2 + "3" =', 1 + 2 + "3");
console.log('"1" + 2 + 3 =', "1" + 2 + 3);
console.log('+"42" =', +"42", '| -"5" =', -"5", '| +"" =', +"", '| +"abc" =', +"abc");
```

Output:

```
a + b = 15
a - b = 5
a * b = 50
a / b = 2
a % b = 0
2 ** 10 = 1024
7 / 2 = 3.5 (no integer division)
Math.trunc(7 / 2) = 3
-7 % 3 = -1 (sign follows the left operand)
1 / 0 = Infinity | -1 / 0 = -Infinity | 0 / 0 = NaN
n++ returns 5 then n is 6
++n returns 7 then n is 7
after b--: 4
"3" + 4 = 34
"3" - 4 = -1
"3" * "4" = 12
1 + 2 + "3" = 33
"1" + 2 + 3 = 123
+"42" = 42 | -"5" = -5 | +"" = 0 | +"abc" = NaN
```

How it works — the rules behind the surprises

1. **Division never throws.** Dividing by zero gives `Infinity`/`-Infinity`; `0 / 0` gives `NaN`.
2. **`%` is a remainder, not a mathematical modulo:** the result has the sign of the left operand (`-7 % 3` is `-1`). For an always-positive modulo use `((a % n) + n) % n`.
3. **Prefix vs postfix.** Both `n++` and `++n` add one to `n`. The difference is the value of the *expression*: postfix returns the value *before* the change, prefix the value *after*. On a line by itself (`i++;`) there is no difference.
4. **`+` is overloaded** (one symbol, more than one job). If either operand is a string (after converting objects to primitives), `+` joins strings; otherwise it adds numbers. Evaluation goes left to right, so `1 + 2 + "3"` is `3 + "3"` = `"33"`, while `"1" + 2 + 3` is `"12" + 3` = `"123"`.
5. **Other arithmetic operators always convert to numbers**, so `"3" - 4` is `-1` and `"3" * "4"` is `12`.
6. **Unary `+`** is the shortest way to convert to a number (`+"42"` → `42`); the empty string becomes `0`, non-numeric text becomes `NaN`.
7. **`-2 ** 2` is a `SyntaxError`** — the language forces you to write `(-2) ** 2` or `-(2 ** 2)` to avoid ambiguity. `**` is right-associative (grouped from the right, §2.9): `2 ** 3 ** 2` is `2 ** 9` = `512`.

Common mistakes

- Adding form input values: `input.value` is always a string, so `"2" + "3"` gives `"23"`. Convert first: `Number(input.value)`.
- Expecting `7 / 2` to be `3`. Use `Math.trunc`, `Math.floor` or `Math.round` explicitly (Chapter 5).

2.6 Comparison operators and equality

What it is

Comparison operators compare two values and always produce a **boolean** (`true` or `false`).

Operator	Meaning	Converts types?
`===`	strictly equal: same type **and** same value	no
`!==`	strictly not equal	no
`==`	loosely equal: equal after converting types	**yes**
`!=`	loosely not equal	**yes**
`>` `<` `>=` `<=`	greater / less (or equal)	yes (to number, unless both are strings)

Example

```
// code/06_comparison_equality.js (abridged)
const x = 5, y = "5";
console.log("x == y  :", x == y);
console.log("x === y :", x === y);
console.log("x != y  :", x != y);
console.log("x !== y :", x !== y);
console.log("x > 2, x < 10, x >= 5, x <= 5:", x > 2, x < 10, x >= 5, x <= 5);
// ... loops over a table of pairs, printing == and === for each
```

Output:

```
x == y  : true
x === y : false
x != y  : false
x !== y : true
x > 2, x < 10, x >= 5, x <= 5: true true true true
--- loose equality surprises ---
0         == ""        -> true  | === -> false
0         == "0"       -> true  | === -> false
""        == "0"       -> false | === -> false
false     == "0"       -> true  | === -> false
false     == ""        -> true  | === -> false
null      == undefined -> true  | === -> false
null      == 0         -> false | === -> false
undefined == 0         -> false | === -> false
NaN       == NaN       -> false | === -> false
[1]       == 1         -> true  | === -> false
[]        == ""        -> true  | === -> false
[1,2]     == "1,2"     -> true  | === -> false
--- relational comparisons ---
"10" < "9"  : true (both strings: compared character by character)
"10" < 9    : false (one number: string converted to number)
"apple" < "banana": true | "Z" < "a": true
null >= 0   : true | null == 0: false
NaN < 1, NaN >= 1: false false
--- Object.is ---
NaN === NaN: false | Object.is(NaN, NaN): true
0 === -0: true | Object.is(0, -0): false
```

Look at the first three surprise rows: `0 == ""` and `0 == "0"` are both true, yet `"" == "0"` is false. Loose equality isn't even *transitive* (the rule "if a equals b and b equals c, then a equals c" fails). That is the main reason to avoid it.

How it works — the `==` algorithm (simplified from the spec)

Given `a == b`:

1. **Same type?** Compare like `===`.
2. `null` **and** `undefined` are equal to each other and to **nothing else** (so `null == 0` is false).
3. **Number vs string** → convert the string to a number (`"5"` → `5`, `""` → `0`).
4. **Boolean vs anything** → convert the boolean to a number first (`true` → `1`, `false` → `0`). That's why `false == "0"` is true (`0 == "0"` → `0 == 0`), and why `if (x == true)` is a trap: `"2" == true` is `2 == 1`, false.
5. **Object vs primitive** → convert the object to a primitive (via `valueOf` / `toString`; arrays become comma-joined strings: `[1,2]` → `"1,2"`, `[]` → `""`), then compare again.
6. **BigInt vs number** → compare mathematical values (`1n == 1` is true).
7. `NaN` **is never equal to anything**, including itself.

`===` skips all of that: different types → `false`. Only two oddities remain: `NaN === NaN` is `false`, and `0 === -0` is `true`. `Object.is(a, b)` is identical to `===` except it fixes those two cases.

Relational comparisons (`<`, `>` …)

1. Convert objects to primitives.

2. If **both are strings**, compare them character by character by UTF-16 code unit (UTF-16 is how JavaScript stores text internally: each character becomes one or two 16-bit numbers called code units) — i.e. dictionary order by character code. So `"10" < "9"` (because `"1"` < `"9"`), and all uppercase letters come before lowercase (`"Z" < "a"`). For human-friendly sorting use `localeCompare` (Chapter 6).
3. Otherwise convert both to numbers. `null` → `0`, so `null >= 0` is true even though `null == 0` is false (`==` uses rule 2 above, relational operators don't). Any comparison involving `NaN` is false.

Trade-offs — `==` vs `===`

	`===`	`==`
Predictable	yes	needs the rules above
Catches type bugs	yes (`"5" === 5` false)	hides them
Useful idiom	—	`x == null` (null **or** undefined)
Linters (tools that scan code for likely bugs; ESLint rule `eqeqeq`)	recommended	flagged, except `== null` if configured

In the interview

30-second answer — "Difference between `==` and `===`?" `===` compares type and value with no conversion. `==` converts operands first: null and undefined equal only each other; strings and booleans are turned into numbers; objects into primitives. Those rules make `==` non-transitive (`0 == ""`, `0 == "0"`, but `"" != "0"`), so I use `===` everywhere except the `x == null` idiom.

- *Follow-up: How do you check for `NaN`?* `Number.isNaN(x)` (or `Object.is(x, NaN)`); `x === NaN` is always false.
- *Follow-up: What's `[] == ![]`?* `true`: `![]` is `false` (arrays are truthy), then `[] == false` → `"" == 0` → `0 == 0`.
- *Follow-up: Why is `null >= 0` true but `null == 0` false?* Relational operators convert `null` to `0`; `==` has a special rule that null equals only undefined.

TL;DR

Use `===`/`!==`. `==` coerces with non-obvious, non-transitive rules. String-vs-string comparisons are by character code; everything else is compared numerically; `NaN` compares false with everything.

2.7 Logical operators, `??` and `?.`

What it is

Logical operators combine conditions — "is the user logged in **and** an admin?", "is the cart empty **or** the store closed?".

Operator	Name	Returns
`a && b`	AND	`a` if `a` is falsy, otherwise `b`
`a || b`	OR	`a` if `a` is truthy, otherwise `b`
`!a`	NOT	`true` if `a` is falsy, else `false` (always a boolean)
`a ?? b`	nullish coalescing (ES2020)	`a` unless it is `null` / `undefined`, then `b`
`a?.b`, `a?.[k]`, `a?.()`	optional chaining (ES2020)	`undefined` if `a` is `null` / `undefined`, otherwise the normal access/call

With booleans they behave like the truth tables you'd expect (`true && false` → `false`). The important detail is the "Returns" column: **`&&` and `||` return one of their operands, not necessarily `true` / `false`.** *Truthy* and *falsy* (§2.11) describe how any value behaves in a boolean context.

How it works — short-circuit evaluation

1. `&&` works from the left and **returns the first falsy value it meets**, without evaluating the rest. If none is falsy, it returns the last value.
2. `||` works from the left and **returns the first truthy value it meets**, without evaluating the rest. If none is truthy, it returns the last value.
3. Because they stop early, the right side may never run. That's used deliberately: `user && user.logout()` calls `logout` only if `user` exists; `isDebug && console.log(...)` logs only in debug mode.

Example

```
// code/07_logical_nullish_optional.js (abridged)
console.log('"cat" && "dog":', "cat" && "dog");    // both truthy -> last
console.log('0 || "default":', 0 || "default");     // first truthy
console.log('"" || null || 0:', "" || null || 0);   // none truthy -> last

let calls = 0;
const touch = () => { calls++; return true; };
false && touch();
true || touch();
console.log("right side evaluated", calls, "times");

const settings = { volume: 0, title: "", theme: null };
console.log("volume || 50:", settings.volume || 50, "| volume ?? 50:", settings.volume ?? 50);
console.log('theme ?? "light":', settings.theme ?? "light");

const user = { name: "Ana", address: null, greet() { return "hi"; } };
console.log("user.address?.city:", user.address?.city);
console.log("user.profile?.photo?.url:", user.profile?.photo?.url);
console.log("user.greet?.():", user.greet?.(), "| user.wave?.():", user.wave?.());
try { user.address.city; } catch (e) { console.log("without ?. ->", e.constructor.name); }

const cfg = { retries: 0, name: "", port: undefined };
cfg.retries ||= 3;    // 0 is falsy -> replaced
cfg.name ??= "app";   // "" is not nullish -> kept
cfg.port ??= 8080;    // undefined -> replaced
let token = "abc"; token &&= token.toUpperCase();
console.log(cfg, token);
```

Output (full run):

```
t && f: false | t || f: true | !f: true
"cat" && "dog": dog
"" && "dog": ""
0 || "default": default
"" || null || 0: 0
!!'text': true | !!0: false
right side evaluated 0 times
volume || 50: 50 | volume ?? 50: 0
title || "Untitled": Untitled | title ?? "Untitled": ""
theme ?? "light": light
user.address?.city: undefined
user.profile?.photo?.url: undefined
user.greet?.(): hi | user.wave?.(): undefined
user.tags?.[0]: undefined
without ?. -> TypeError
{ retries: 3, name: '', port: 8080 } ABC
true || false && false: true
mixing ?? and || without () -> SyntaxError
(null || undefined) ?? 'x': x
```

`||` vs `??` for defaults

`||` falls back on **any falsy** value, which includes legitimate values like `0`, `""` and `false`. A volume of `0` is a real setting, but `volume || 50` throws it away. `??` falls back **only on** `null` **or** `undefined` — "no value" — so `volume ?? 50` keeps the `0`. Rule: use `??` for defaults unless you really mean "any falsy value".

Optional chaining `?.`

Reading a property of `null` or `undefined` throws a `TypeError` ("Cannot read properties of null"). `?.` checks the value on its left: if it's `null`/`undefined`, the **whole chain** stops and evaluates to `undefined`; otherwise it continues normally. Three forms: `obj?.prop`, `obj?.[expr]`, `fn?.(args)` (call only if `fn` exists). You cannot assign through it (`obj?.x = 1` is a `SyntaxError`).

Logical assignment (ES2021)

Operator	Equivalent	Assigns when `a` is…
`a ||= b`	`a || (a = b)`	falsy
`a &&= b`	`a && (a = b)`	truthy
`a ??= b`	`a ?? (a = b)`	`null` / `undefined`

They short-circuit too: if no assignment is needed, `b` isn't even evaluated.

Common mistakes

- `if (x = 5)` — a single `=` assigns (and the result `5` is truthy); you meant `===`.
- **Overusing** `?.` — if a value *must* exist, let the error surface instead of hiding a bug behind `undefined`.
- **Mixing** `??` **with** `||`/`&&` **without parentheses** → `SyntaxError`; the language forces you to be explicit.
- Forgetting precedence: `&&` binds tighter than `||`, so `a || b && c` is `a || (b && c)`.

TL;DR

`&&`/`||` short-circuit and return an operand. Use `??` for defaults (keeps `0` and `""`), `?.` to read possibly-missing data safely, and `||=`/`&&=`/`??=` to assign conditionally.

2.8 Bitwise and assignment operators

Bitwise operators

Bitwise operators treat numbers as 32-bit signed integers and work on the individual binary digits (**bits**). Beginners can skim this; they appear in low-level code (graphics, compression, hashing (turning data into a short fixed-size fingerprint number), permission flags) and in interview puzzles.

Operator	Name	Rule per bit
`&`	AND	1 if both bits are 1
`|`	OR	1 when either bit (or both) is 1
`^`	XOR	1 if the bits differ
`~`	NOT	flips every bit (`~x` equals `-x - 1`)
`<<`	left shift	moves bits left, filling with 0 (×2 per step)
`>>`	sign-propagating right shift	moves bits right, copying the sign bit (÷2, rounding down)
`>>>`	zero-fill right shift	moves bits right, filling with 0 (result is unsigned)

```
// code/08_bitwise_assignment.js (first half)
let c = 5, d = 2;               // 5 = 101, 2 = 010
console.log("c & d  =", c & d, bin(c & d));
console.log("c | d  =", c | d, bin(c | d));
console.log("c ^ d  =", c ^ d, bin(c ^ d));
console.log("~d     =", ~d);
console.log("c << 1 =", c << 1);
console.log("c >> 1 =", c >> 1, "| -5 >> 1 =", -5 >> 1, "(keeps sign)");
console.log("c >>> 1 =", c >>> 1, "| -5 >>> 1 =", -5 >>> 1, "(fills with 0)");
const READ = 1, WRITE = 2, EXEC = 4;
let perms = READ | WRITE;
console.log("can write?", (perms & WRITE) !== 0, "| can exec?", (perms & EXEC) !== 0);
```

Output:

```
c & d  = 0 000
c | d  = 7 111
c ^ d  = 7 111
~d     = -3
c << 1 = 10
c >> 1 = 2 | -5 >> 1 = -3 (keeps sign)
c >>> 1 = 2 | -5 >>> 1 = 2147483645 (fills with 0)
can write? true | can exec? false
```

Walk-through: `101 & 010` has no position where both are 1 → `000` = 0. OR and XOR both give `111` = 7 because 5 and 2 never share a 1-bit. `~2` = `-3` (two's complement — the standard way computers store negative integers in binary: flip all bits and add 1). The **flags** idiom packs several yes/no settings into one number: each flag is a power of two, `|` combines them, `&` tests one. Unix (Linux/macOS) file permissions (`chmod 755`) and many binary file formats use this idea.

Caveat: bitwise operators first truncate their operands (cut off anything that doesn't fit) to 32-bit integers, so they break for numbers beyond $\pm 2^{31}$ (about 2.1 billion). Old tricks like `x | 0` to truncate decimals share this limit — prefer `Math.trunc`.

Assignment operators

`=` stores the right-hand value in the variable on the left. **Compound assignment** operators combine an operation with assignment: `x += y` means `x = x + y`.

Operator	Same as		Operator	Same as
`x = y`	assign		`x <<= y`	`x = x << y`
`x += y`	`x = x + y`		`x >>= y`	`x = x >> y`
`x -= y`	`x = x - y`		`x >>>= y`	`x = x >>> y`
`x *= y`	`x = x * y`		`x &= y`	`x = x & y`
`x /= y`	`x = x / y`		`x |= y`	`x = x | y`
`x %= y`	`x = x % y`		`x ^= y`	`x = x ^ y`
`x **= y`	`x = x ** y`		`||=` `&&=` `??=`	§2.7

```
// code/08_bitwise_assignment.js (second half)
let e = 10;
const log = [];
e += 5;  log.push(e);  // 15
e -= 3;  log.push(e);  // 12
e *= 2;  log.push(e);  // 24
e /= 4;  log.push(e);  // 6
e %= 4;  log.push(e);  // 2
e **= 3; log.push(e);  // 8
e <<= 1; log.push(e);  // 16
e >>= 2; log.push(e);  // 4
e &= 6;  log.push(e);  // 4   (100 & 110 = 100)
e |= 1;  log.push(e);  // 5   (100 | 001 = 101)
e ^= 3;  log.push(e);  // 6   (101 ^ 011 = 110)
console.log("e after each step:", log.join(" -> "));
let p, q;
p = q = 7;                      // right to left: q = 7, then p = (q = 7) which is 7
console.log("p, q:", p, q);
```

Output:

```
e after each step: 15 -> 12 -> 24 -> 6 -> 2 -> 8 -> 16 -> 4 -> 4 -> 5 -> 6
p, q: 7 7
```

An assignment is itself an expression whose value is the assigned value; that is what makes `p = q = 7` work (and what makes `if (x = 5)` a silent bug).

2.9 Operator precedence

Precedence decides which operator is applied first when several appear in one expression (`2 + 3 * 4` is `14`, not `20`). **Associativity** decides the order among operators of equal precedence (most are left-to-right; `**`, assignment and `?:` are right-to-left). From highest to lowest (simplified):

Level	Operators
1	`( … )` grouping
2	`.` `[]` `?.` function call `()` `new` with arguments
3	postfix `x++` `x--`
4	`!` `~` unary `+` `-` prefix `++x` `--x` `typeof` `void` `delete` `await`
5	`**` (right-to-left)
6	`*` `/` `%`
7	`+` `-`
8	`<<` `>>` `>>>`
9	`<` `<=` `>` `>=` `in` `instanceof`
10	`==` `!=` `===` `!==`
11–13	`&`, then `^`, then `|`
14	`&&`
15	`||` and `??` (cannot be mixed without parentheses)
16	`? :` (right-to-left)
17	`=` and all compound assignments, `=>` (right-to-left)
18	`,` comma

Practical advice: you don't need to memorise this. **When in doubt, add parentheses** — they cost nothing and make intent obvious to the reader.

2.10 Numbers in depth

What it is

All ordinary JavaScript numbers — integers and decimals alike — are stored as **64-bit IEEE-754 double-precision floating-point** values ("doubles"), the same format as `double` in Java or C.

> **Key term — floating point:** a way of storing numbers in binary as a fixed number of significant digits plus an exponent (like scientific notation, 1.23×10^5), so the decimal point can "float" to handle both tiny and huge values. **IEEE-754** is the international standard that defines this format; "double precision" means 64 bits per number. The trade-off: range is enormous, but most decimal fractions are stored as close approximations, not exactly. That single choice explains almost everything unusual about numbers in JS:

- about 15–17 significant decimal digits (meaningful digits, ignoring leading zeros) of precision;
- integers are exact only up to $2^{53} - 1$ (9,007,199,254,740,991);
- decimals like `0.1` can't be represented exactly in binary, just as 1/3 can't be written exactly in decimal;
- special values `Infinity`, `-Infinity`, `NaN` and a separate `-0`.

The global `Number` is the function/wrapper that holds number-related constants and helper methods. Calling `Number(value)` converts to a number; `new Number(value)` creates a wrapper object you should not use (§2.4).

Example

```
// code/09_numbers.js (abridged — see the file for every line)
console.log("0.1 + 0.2 =", 0.1 + 0.2);
console.log("0.1 + 0.2 === 0.3 ?", 0.1 + 0.2 === 0.3);
console.log("close enough?", Math.abs(0.1 + 0.2 - 0.3) < Number.EPSILON);
console.log("money in cents: 10 + 20 =", (10 + 20) / 100);
console.log("MAX_VALUE        ", Number.MAX_VALUE);
// ... MIN_VALUE, MAX/MIN_SAFE_INTEGER, EPSILON, infinities
console.log("2**53 + 1        ", 2 ** 53 + 1, "(not representable!)");
console.log("literals:", 1_000_000, 0xff, 0o17, 0b1010, 1.5e3, 2e-3, .5);
const num = 123.456;
console.log("toFixed(2):", num.toFixed(2), "| toFixed(0):", num.toFixed(0));
console.log("toPrecision(4):", num.toPrecision(4), "| toPrecision(2):", num.toPrecision(2));
console.log("(255).toString(2):", (255).toString(2), "| (255).toString(16):", (255).toString(16));
console.log("(1.005).toFixed(2):", (1.005).toFixed(2), "(binary rounding!)");
console.log("parseInt('123px'):", Number.parseInt("123px"), "| parseInt('px123'):", Number.parseInt("px123"));
console.log("Number('123px'):", Number("123px"), "| Number(''):", Number(""), "| Number(' 42 '):", Number(" 42
"));
console.log("isNaN('abc'):", isNaN("abc"), "| Number.isNaN('abc'):", Number.isNaN("abc"), "| Number.isNaN(NaN):",
Number.isNaN(NaN));
const big = 2n ** 64n;
console.log("2n ** 64n =", big, "| typeof:", typeof big);
try { 1n + 1; } catch (e) { console.log("1n + 1 ->", e.name + ":", e.message); }
```

Output:

```
0.1 + 0.2 = 0.30000000000000004
0.1 + 0.2 === 0.3 ? false
close enough? true
money in cents: 10 + 20 = 0.3
MAX_VALUE         1.7976931348623157e+308
MIN_VALUE         5e-324
MAX_SAFE_INTEGER  9007199254740991
MIN_SAFE_INTEGER  -9007199254740991
EPSILON           2.220446049250313e-16
POSITIVE_INFINITY Infinity | NEGATIVE_INFINITY -Infinity
MAX_VALUE * 2     Infinity
2**53 + 1         9007199254740992 (not representable!)
isSafeInteger(2**53): false
literals: 1000000 255 15 10 1500 0.002 0.5
toFixed(2): 123.46 | toFixed(0): 123
toPrecision(4): 123.5 | toPrecision(2): 1.2e+2
(255).toString(2): 11111111 | (255).toString(16): ff
(1.005).toFixed(2): 1.00 (binary rounding!)
toLocaleString: 1,234,567.891 | 1.234.567,891
valueOf of wrapper: 7
parseInt('123px'): 123 | parseInt('px123'): NaN
parseInt('ff', 16): 255 | parseInt('101', 2): 5
parseFloat('3.14abc'): 3.14
Number('123px'): NaN | Number(''): 0 | Number(' 42 '): 42
Number(null): 0 | Number(undefined): NaN | Number(true): 1
isInteger(5.0): true | isInteger(5.5): false
isFinite('5'): true | Number.isFinite('5'): false
isNaN('abc'): true | Number.isNaN('abc'): false | Number.isNaN(NaN): true
2n ** 64n = 18446744073709551616n | typeof: bigint
9007199254740993n + 1n = 9007199254740994n
7n / 2n = 3n (truncates)
1n + 1 -> TypeError: Cannot mix BigInt and other types, use explicit conversions
BigInt(10) + 1n = 11n | Number(5n) + 1 = 6
```

Number literals

Form	Example	Value
decimal	`42`, `3.14`, `.5`	
exponent	`1.5e3`, `2e-3`	1500, 0.002
hex / octal / binary	`0xff`, `0o17`, `0b1010`	255, 15, 10
numeric separators (ES2021)	`1_000_000`	1000000 (underscores are ignored, just for readability)
BigInt	`123n`	a `bigint`

`Number` properties

Property	Value	Meaning
`Number.MAX_VALUE`	≈ 1.797e308	largest finite number; beyond it you get `Infinity`
`Number.MIN_VALUE`	5e-324	smallest **positive** number (closest to zero) — *not* the most negative number
`Number.MAX_SAFE_INTEGER`	$2^{53} - 1$	largest integer `n` such that `n` and `n + 1` are both exactly representable
`Number.MIN_SAFE_INTEGER`	$-(2^{53} - 1)$	the negative counterpart
`Number.EPSILON`	≈ 2.22e-16	gap between 1 and the next representable double; a tolerance for comparing decimals near 1
`Number.POSITIVE_INFINITY` / `NEGATIVE_INFINITY`	`Infinity` / `-Infinity`	results of overflow (a result too large to store) or division by zero
`Number.NaN`	`NaN`	same as the global `NaN`

`Number` methods

Instance methods (called on one particular number value, the "instance"; they return strings except `valueOf`):

Method	What it does	Example → result
`toFixed(d)`	round to `d` digits after the decimal point	`(123.456).toFixed(2)` → `"123.46"`
`toPrecision(p)`	round to `p` significant digits (may switch to exponent form)	`(123.456).toPrecision(4)` → `"123.5"`
`toString(radix)`	write in base 2–36 (the radix is the number base)	`(255).toString(16)` → `"ff"`
`toLocaleString(locale)`	format for humans (thousands separators, currency) using a locale (language + region, e.g. `"en-US"`)	`(1234.5).toLocaleString("en-US")` → `"1,234.5"`
`valueOf()`	the primitive inside a `Number` wrapper	`new Number(7).valueOf()` → `7`

To call a method directly on an integer literal, wrap it: `(255).toString(16)` — `255.toString()` is a syntax error because the dot is read as a decimal point.

Static methods (called on `Number` itself):

Method	What it does	Watch out
`Number.isFinite(v)`	`true` only for finite *numbers*	the global `isFinite("5")` converts first and says `true`
`Number.isInteger(v)`	integer-valued number?	`5.0` is an integer (it's the same value as `5`)
`Number.isSafeInteger(v)`	integer within $\pm(2^{53} - 1)$?	use before trusting large IDs
`Number.isNaN(v)`	is it exactly `NaN`?	the global `isNaN("abc")` converts first and says `true`
`Number.parseFloat(s)`	read a decimal from the *start* of a string	stops at the first invalid character
`Number.parseInt(s, radix)`	read an integer from the *start* of a string in base `radix`	**always pass the radix** (usually 10)

`parseInt`/`parseFloat` vs `Number()`: the parse functions read as much of a leading number as they can and ignore the rest (`"123px"` → `123`), which suits CSS values and user text; `Number()` requires the *whole* string (after trimming spaces) to be numeric, else `NaN`, and treats `""` as `0`. Choose deliberately.

`NaN` and `Infinity`

- **`NaN`** ("Not a Number") is the result of a failed numeric operation: `Number("abc")`, `0 / 0`, `Math.sqrt(-1)`. It spreads — any arithmetic with `NaN` gives `NaN` — and it's the only value not equal to itself. Detect it with `Number.isNaN`.
- **`Infinity`** results from overflow (`Number.MAX_VALUE * 2`) or division by zero. `Number.isFinite(x)` rejects `Infinity`, `-Infinity` and `NaN` in one check.

Floating-point precision — what to do about it

1. **Never compare decimals with `===`.** Compare with a tolerance: `Math.abs(a - b) < 1e-9` (or `Number.EPSILON` for values near 1).
2. **Money: work in integer cents** (store 1999, display 19.99), or use a decimal library. Payment APIs such as Stripe take amounts in the smallest currency unit for this reason.
3. **Round only for display**, with `toFixed` or `Intl.NumberFormat`/`toLocaleString`. Be aware `toFixed` rounds the *binary* value: `(1.005).toFixed(2)` is `"1.00"` because 1.005 is stored as 1.00499999…
4. **Large integers** (database IDs, Twitter/X post IDs, nanosecond timestamps — a timestamp is a number marking a moment in time) beyond 2^{53} lose precision silently. Keep them as **strings** or use **`BigInt`**.

`BigInt`

`bigint` (ES2020) represents whole numbers of any size. Write it with an `n` suffix (`10n`) or convert with `BigInt(10)`. Rules: you **can't mix** `bigint` and `number` in arithmetic (`TypeError`) — convert explicitly; division truncates (`7n / 2n` is `3n`); `Math` functions don't accept BigInts; and `JSON.stringify` can't serialise them (turn them into JSON text for saving or sending) without help. Use BigInt for big IDs, cryptography (the maths behind encryption and secure signatures) and exact large-integer maths; use `number` for everything else (it's faster).

In the interview

"Why is `0.1 + 0.2 !== 0.3`?" Numbers are IEEE-754 binary doubles; 0.1 and 0.2 have no exact binary representation, so each is stored as the nearest double, and their sum is the double just above 0.3 (0.30000000000000004). Compare with a tolerance, or use integer units such as cents.

- *Follow-up: What's `Number.MAX_SAFE_INTEGER` and why does it matter?* $2^{53} - 1$; above it not every integer is representable, so `2**53 + 1 === 2**53`. Keep big IDs as strings or BigInt.
- *Follow-up: `isNaN` vs `Number.isNaN`?* The global one converts its argument to a number first, so `isNaN("abc")` is true; `Number.isNaN` is true only for the actual `NaN` value.
- *Follow-up: `parseInt("08")`?* `8` in modern engines, but always pass the radix — `parseInt(x, 10)` — to be explicit. Also note `parseInt(0.0000005)` is `5`, because the number is first turned into the string

`"5e-7"`.

Self-check

1. `Number("")` vs `parseInt("")`? → `0` vs `NaN`.
2. What is `Number.MIN_VALUE`? → The smallest *positive* double, 5e-324.
3. `typeof NaN`? → `"number"`.
4. `10n + 5`? → `TypeError` — convert one side first.

TL;DR

One `number` type (64-bit float): exact integers up to $2^{53} - 1$, inexact decimals, plus `NaN`, `±Infinity`. Use `Number.isNaN/isFinite/isInteger`, `toFixed` for display, integer cents for money, `BigInt` for huge integers.

2.11 Booleans, truthy and falsy

What it is

The **boolean** type has exactly two values, `true` and `false`. Comparisons produce booleans, and conditions (`if`, `while`, `? :`, Chapter 3) consume them.

But a condition can be *any* value. JavaScript converts it to a boolean using a fixed list: the **falsy** values become `false`; **everything else is truthy**.

Falsy values (the complete list)
`false`
`0`, `-0` and `0n` (BigInt zero)
`""` — the empty string, whichever quotes are used
`null`
`undefined`
`NaN`

(Browsers have one historical oddity, `document.all`, which is also falsy. You'll never need it.)

Surprising **truthy** values: `"0"`, `"false"`, `" "` (a space), `[]` (empty array), `{}` (empty object), `-1`, `Infinity`, every function, and every wrapper object — including `new Boolean(false)`.

Example

```
// code/10_boolean_truthiness.js
const falsy = [["false", false], ["0", 0], ["-0", -0], ["0n", 0n], ['""', ""], ["null", null], ["undefined",
undefined], ["NaN", NaN]];
console.log("falsy:", falsy.map(([label, v]) => `${label}->${Boolean(v)}`).join("  "));
const truthy = ["0", "false", " ", [], {}, -1, Infinity, function () {}, new Boolean(false)];
console.log("truthy:", truthy.map((v) => Boolean(v)).every(Boolean));

if (false || 0 || "" || null || undefined || NaN) {
  console.log("never printed");
} else {
  console.log("All values are falsy.");
}
console.log('Boolean("hi"):', Boolean("hi"), '| !!"":', !!"", "| !!42:", !!42);
const wrapped = new Boolean(false);
console.log("typeof wrapped:", typeof wrapped, "| valueOf():", wrapped.valueOf());
console.log("if (new Boolean(false)) runs?", wrapped ? "yes (object is truthy)" : "no");
console.log("new Boolean(true).valueOf():", new Boolean(true).valueOf(), "| toString():", new
Boolean(true).toString());
const age = 20;
const isAdult = age >= 18;
console.log("isAdult:", isAdult, typeof isAdult);
console.log("true + true =", true + true, "| [true, false, true].filter(Boolean).length =", [true, false,
true].filter(Boolean).length);
```

Output:

```
falsy: false->false  0->false  -0->false  0n->false  ""->false  null->false  undefined->false  NaN->false
truthy: true
All values are falsy.
Boolean("hi"): true | !!"": false | !!42: true
typeof wrapped: object | valueOf(): false
if (new Boolean(false)) runs? yes (object is truthy)
new Boolean(true).valueOf(): true | toString(): true
isAdult: true boolean
true + true = 2 | [true, false, true].filter(Boolean).length = 2
```

In the `if` test, `||` walks through six values looking for a truthy one; all six are falsy, so the whole expression returns the last one (`NaN`), which is falsy, and the `else` branch runs.

Converting to boolean

- **`Boolean(value)`** — explicit and readable.
- **`!!value`** — double NOT: the first `!` converts and inverts, the second inverts back. Common shorthand.
- **`array.filter(Boolean)`** — passes `Boolean` as a function to drop all falsy entries.

The `Boolean` wrapper object

`new Boolean(x)` creates an **object** that wraps a boolean. Its only notable methods are `valueOf()` (the primitive inside) and `toString()` (`"true"`/`"false"`). Because it's an object, it is *always truthy* — even `new Boolean(false)`. That trap is why nobody uses `new Boolean`; call `Boolean(x)` (no `new`) to convert.

Common mistakes

- **`if (arr)` to test "is the list non-empty?"** — `[]` is truthy. Use `if (arr.length > 0)`.
- **`if (count)` when `0` is a valid count** — `0` is falsy. Use `if (count !== undefined)` or `count ?? …`.
- **`if (x == true)`** — converts `true` to `1`, so `"yes" == true` is false. Write `if (x)` for truthiness or `x === true` for the exact value.
- **Reading `"false"` from a form or URL** and testing it with `if` — the string `"false"` is truthy. Compare: `value === "true"`.

In the interview

"List the falsy values." `false`, `0`, `-0`, `0n`, `""`, `null`, `undefined`, `NaN` (plus the legacy `document.all`). Everything else — including `"0"`, `[]`, `{}` and `new Boolean(false)` — is truthy.

- *Follow-up:* `!!"false"`*?* `true` — it's a non-empty string.
- *Follow-up: Why is* `new Boolean(false)` *truthy?* It's an object, and all objects are truthy.

TL;DR

Eight falsy values; everything else is truthy. Convert with `Boolean(x)` or `!!x`; never use `new Boolean`.

2.12 Type conversion summary

Type conversion (or **coercion**) is turning a value of one type into another. **Explicit** conversion is when you ask for it (`Number(x)`, `String(x)`, `Boolean(x)`); **implicit** conversion is when an operator does it for you (`"3" * 2`, `if (x)`, `x == y`). Knowing the three target conversions covers almost every case:

```
// code/11_conversions.js (the table loop)
const values = ["42", "", " ", "4px", "0x1f", null, undefined, true, false, [], [7], [1, 2], {}];
for (const v of values) {
  console.log(show(v).padEnd(10), String(Number(v)).padEnd(10), JSON.stringify(String(v)).padEnd(18), Boolean(v));
}
```

Output:

```
value      Number()   String()           Boolean()
"42"       42         "42"               true
""         0          ""                 false
" "        0          " "                true
"4px"      NaN        "4px"              true
"0x1f"     31         "0x1f"             true
null       0          "null"             false
undefined  NaN        "undefined"        false
true       1          "true"             true
false      0          "false"            false
[]         0          ""                 true
[7]        7          "7"                true
[1,2]      NaN        "1,2"              true
{}         NaN        "[object Object]"  true
parseInt vs Number on '08.5kg': 8 NaN
String(123) + 1 = 1231 | `${123}`: 123
```

Rules behind the table:

1. **To number:** whitespace-only strings → `0`; numeric strings (including `0x…`, `0b…`, `0o…`) → that number; anything else → `NaN`. `null` → `0`, `undefined` → `NaN`, booleans → `1`/`0`. Objects → first to a primitive (arrays → joined string), then to number.
2. **To string:** the obvious text; arrays join with commas (`[]` → `""`); plain objects → `"[object Object]"` (use `JSON.stringify` to see contents).
3. **To boolean:** the falsy list from §2.11; everything else `true`.

Which operators trigger which conversion:

Context	Converts to
`if`, `while`, `? :`, `!`, `&&`, `||`	boolean
`-` `*` `/` `%` `**`, unary `+`/`-`, `<` `>` (unless both strings)	number
`+` when either side is a string; template literals `` `${x}` ``	string
`==`	follows the §2.6 algorithm

Prefer **explicit** conversions in your own code — `Number(input.value)` says exactly what you mean — and `===` to avoid implicit ones.

Chapter 2 TL;DR

- **Declare** with `const` (default) or `let` (when reassigning); avoid `var` (function-scoped, redeclarable, hoisted as `undefined`). `let`/`const` are block-scoped and throw in the **TDZ**.
- **Types:** seven primitives (`number`, `bigint`, `string`, `boolean`, `undefined`, `null`, `symbol`) + objects. `typeof null` is `"object"`; use `Array.isArray` for arrays.
- **Primitives** are immutable and copied by value; **objects** are copied and compared by reference.
- **Arithmetic:** `/` is always floating point; `%` keeps the left sign; `+` concatenates if either side is a string; postfix `++` returns the old value.
- **Equality:** use `===`; `==` coerces with non-transitive rules (`x == null` is the one common exception). `Object.is` fixes `NaN` and `-0`.
- **Logic:** `&&`/`||` short-circuit and return operands; `??` defaults only on `null`/`undefined`; `?.` safely reads possibly-missing data; `||=`, `&&=`, `??=` assign conditionally.
- **Numbers:** IEEE-754 doubles — `0.1 + 0.2 !== 0.3`, safe integers up to $2^{53} - 1$, `NaN`/`Infinity`; use `Number.isNaN`, `toFixed` for display, cents for money, `BigInt` for huge integers; `parseInt(s, 10)` vs `Number(s)` differ on trailing text.
- **Truthiness:** falsy = `false, 0, -0, 0n, "", null, undefined, NaN`; everything else (including `[]`, `"0"`, `new Boolean(false)`) is truthy.

Next: Chapter 3 — Control Flow and Loops.

Chapter 3 — Control Flow and Loops

Left alone, a program runs its statements once each, top to bottom. **Control flow** is everything that changes that: *choosing* which statements run (conditionals) and *repeating* statements (loops). With the values and operators from Chapter 2, these two tools are enough to express any algorithm (a precise, step-by-step recipe for solving a problem).

What you'll learn

1. What conditional statements are and where real applications use them.
2. `if`, `else`, `else if`, guard clauses, and how truthiness affects conditions.
3. The conditional (ternary) operator `? :`.
4. `switch`: cases, `default`, `break`, grouped cases, strict comparison, fall-through.
5. Why loops exist, and the five loop statements: `for`, `while`, `do…while`, `for…in`, `for…of`.
6. Nested loops.
7. `break`, `continue` and labels.
8. How to choose the right loop (and when an array method is better than any loop).

Code: `code/`. Every "Output:" block is real Node 24 output.

3.1 Conditional statements — making decisions

What it is

A **conditional statement** evaluates a **condition** — an expression — and, depending on whether it is truthy or falsy (treated as `true` or as `false` when used as a condition), runs one block of code or another. It's the program's way of saying "if this, do that; otherwise do something else".

Analogy: a railway switch. The train (your program) always moves forward, but the switch decides which track it takes.

JavaScript has two conditional **statements** and one conditional **operator**:

Tool	Best for
`if … else if … else`	any conditions, including ranges and combinations (`age >= 18 && hasId`)
`switch`	comparing **one** value against many fixed options (`"Mon"`, `"Tue"` …)
`condition ? a : b`	choosing between two **values** inside an expression

Where you meet them in real applications

- **Form validation** — is the email field filled in and shaped like an email? Only then enable the Submit button.
- **Authentication and permissions** (checking *who* the user is, then *what* they may do) — show the admin panel only if the user is logged in *and* has the admin role.
- **Interactive UI** (user interface — the buttons, menus and screens people interact with) — if the menu is open, close it on click; otherwise open it. If the user prefers dark mode, apply the dark theme.
- **Filtering data** — show a product only if its price is inside the range the shopper selected.
- **Error handling** — if the server answered with status 404, show "Not found"; if 500, show "Try again later". (An HTTP status code is the number a web server sends back to say how a request went: 2xx success, 4xx your mistake, 5xx the server's — see System Design Fundamentals.)

TL;DR

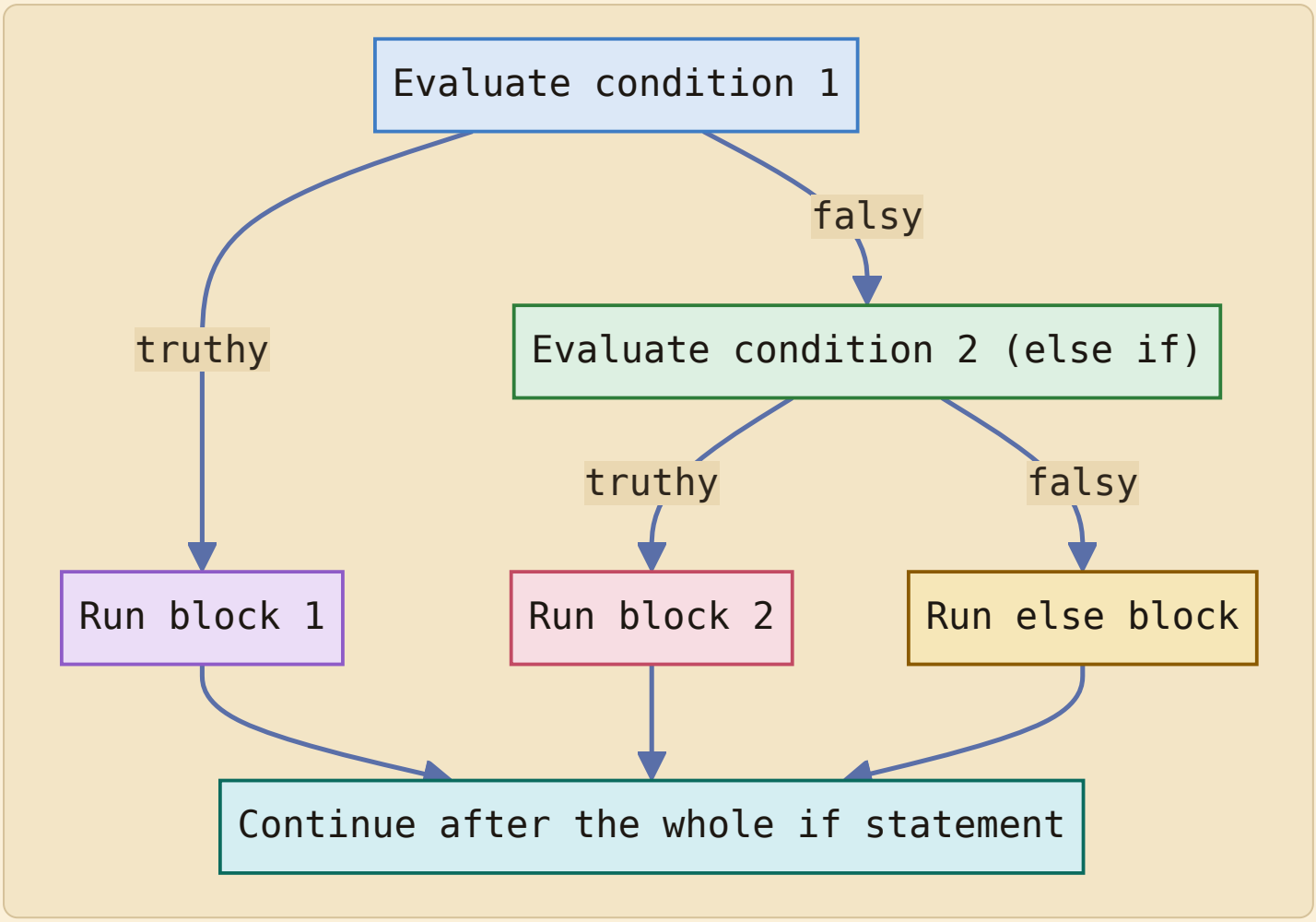
Conditionals choose between code paths based on a condition's truthiness. `if` handles anything, `switch` handles one-value-many-options, the ternary chooses between two values.

3.2 `if`, `else` and `else if`

What it is

- `if (condition) { … }` runs the block only when the condition is truthy.
- `else { … }` (optional) is the fallback, run when the `if` condition is falsy.
- `else if (condition) { … }` (any number) adds more conditions, tested **in order**; the first truthy one wins and the rest are skipped.

How it works



1. The condition in parentheses is evaluated and converted to a boolean using the truthy/falsy rules (Chapter 2 §2.11).
2. If truthy, its block runs, and execution jumps to after the whole statement.
3. If falsy, the next `else if` condition is evaluated, and so on.
4. If no condition was truthy, the `else` block runs (if there is one). At most **one** block runs.

Example

```
// code/01_if_else.js (part 1)
function votingMessage(age) {
  if (age >= 18) {
    return "You can vote.";
  } else {
    return "Not old enough to vote yet.";
  }
}
console.log("age 18 ->", votingMessage(18));
console.log("age 16 ->", votingMessage(16));

function letterGrade(score) {
  if (score >= 90) {
    return "A";
  } else if (score >= 80) {
    return "B";
  } else if (score >= 70) {
    return "C";
  } else if (score >= 60) {
    return "D";
  } else {
    return "F";
  }
}
for (const s of [95, 85, 75, 65, 40]) console.log(`score ${s} -> grade ${letterGrade(s)}`);

function badGrade(score) {
  if (score >= 70) return "C";   // checked first, so 95 also lands here
  if (score >= 90) return "A";
  return "F";
}
console.log("wrong order: 95 ->", badGrade(95));
```

Output:

```
age 18 -> You can vote.
age 16 -> Not old enough to vote yet.
score 95 -> grade A
score 85 -> grade B
score 75 -> grade C
score 65 -> grade D
score 40 -> grade F
wrong order: 95 -> C
```

(Functions and `return` are introduced properly in Chapter 4; here they just package each example. The `for…of` loop is §3.10.)

The grade ladder works because the checks go from **most specific to least specific**: a score of 85 fails `>= 90`, then passes `>= 80`, and nothing further is tested. `badGrade` shows what happens when the order is wrong — the broad condition `>= 70` catches 95 before the `>= 90` test is ever reached.

Conditions are truthy/falsy, not just `true`/`false`

```
// code/01_if_else.js (part 2)
const inputs = ["", "Ana", 0, 3, null, []];
for (const v of inputs) {
  console.log(JSON.stringify(v).padEnd(6), v ? "truthy -> if-branch" : "falsy  -> else-branch");
}
```

Output:

```
""     falsy  -> else-branch
"Ana"  truthy -> if-branch
0      falsy  -> else-branch
3      truthy -> if-branch
null   falsy  -> else-branch
[]     truthy -> if-branch
```

`if (name)` is a common, readable way to say "if `name` is a non-empty string". Just remember `0` and `""` are falsy and `[]` is truthy, and write an explicit comparison when that matters (`if (items.length > 0)`).

Guard clauses — flatter code

Deeply nested `if`s are hard to read. A **guard clause** handles a special case at the top of a function and returns immediately, so the main logic isn't indented:

```
// code/01_if_else.js (part 3)
function shippingCost(order) {
  if (!order) return "no order";
  if (order.items === 0) return "empty cart";
  if (order.total >= 50) return "free shipping";
  return "shipping: $4.99";
}
const user = { loggedIn: true, role: "editor" };
if (user.loggedIn && (user.role === "admin" || user.role === "editor")) {
  console.log("can publish");
}
```

Output:

```
no order | empty cart | free shipping | shipping: $4.99
can publish
```

Conditions can be combined with `&&`, `||` and `!`; use parentheses to make grouping explicit.

Braces

If a block has only one statement, the braces can be omitted (`if (x) doSomething();`). Keep braces for anything multi-line — this classic bug happens without them:

```
if (isAdmin)
  showPanel();
  logAccess();   // looks conditional, but ALWAYS runs — only the first line belongs to the if
```

A formatter (a tool that automatically rewrites code layout — e.g. Prettier) re-indents code to expose this; many teams simply always use braces.

Common mistakes

- **`=` instead of `===`**: `if (role = "admin")` assigns and is always truthy.
- **Wrong order in an else-if ladder** (see `badGrade`).
- **Testing an array or object for emptiness with `if (arr)`** — always truthy.
- **Writing `else if` conditions that overlap or leave gaps** (`< 60` and `> 60` forgets exactly 60). Test the boundaries.

Self-check

1. With `if (a) {…} else if (b) {…}`, can both blocks run? → No; at most one block of an if/else chain runs.
2. What does `if ("0")` do? → Runs the block — `"0"` is a non-empty string, hence truthy.
3. Why check `score >= 90` before `score >= 80`? → Because the first truthy branch wins; broader checks must come later.

TL;DR

`if` → optional `else if`s → optional `else`; the first truthy branch runs, others are skipped. Order from specific to general, keep braces, prefer guard clauses to deep nesting.

3.3 The conditional (ternary) operator

What it is

`condition ? valueIfTruthy : valueIfFalsy` is an **expression** (it produces a value) that picks one of two values. It is JavaScript's only operator with three operands, hence the nickname *ternary operator*.

Example

```
// code/02_ternary.js
const age = 20;
const label = age >= 18 ? "adult" : "minor";
console.log("label:", label);

const count = 1;
console.log(`You have ${count} new ${count === 1 ? "message" : "messages"}`);

const temp = 12;
const feel =
  temp < 0 ? "freezing" :
  temp < 15 ? "cold" :
  temp < 25 ? "pleasant" :
  "hot";
console.log("feel:", feel);

const price = 100, isMember = true;
console.log("to pay:", price * (isMember ? 0.9 : 1));
```

Output:

```
label: adult
You have 1 new message
feel: cold
to pay: 90
```

When to use it (and when not)

Use the ternary when…	Use `if` when…
choosing between two **values**	running **statements** (side effects — changes beyond returning a value, like printing or updating a variable — several lines)
it fits on one line and reads naturally	the logic needs explanation or more than two outcomes
you need an expression: inside a template literal, an argument, a `const` initialiser, JSX in React (HTML-like syntax inside JavaScript)	—

Chained ternaries are acceptable when formatted one condition per line as above; nesting them inside each other's branches quickly becomes unreadable.

TL;DR

`cond ? a : b` returns `a` or `b`. It's for choosing values, not for running code.

3.4 `switch`

What it is

A `switch` statement evaluates one expression once and compares the result against a list of `case` values, running the code of the first match. It's a tidy replacement for a long `if … else if` chain that keeps testing *the same value* for equality.

```
switch (expression) {
  case value1:
    // runs when expression === value1
    break;
  case value2:
    // runs when expression === value2
    break;
  default:
    // runs when no case matched (optional)
}
```

Part	Role
`expression`	evaluated **once**; its value is compared with each case
`case value:`	a candidate match, compared with `===` (strict equality)
`break`	leaves the switch; without it, execution **falls through** into the next case's code
`default:`	optional catch-all when nothing matched — the switch equivalent of `else`

How it works

1. Evaluate the `switch` expression.
2. Compare it, top to bottom, with each `case` value using `===`.
3. At the first match, start executing statements *from that point*.
4. Keep going until a `break` (or `return`), or the end of the switch — **including through the following cases' code**.
5. If nothing matched and there is a `default`, start executing there.

Examples

```
// code/03_switch.js (selected parts)
function describeType(value) {
  switch (typeof value) {
    case "number":  return "a number";
    case "string":  return "a string";
    case "boolean": return "a boolean";
    default:        return "something else";
  }
}

function menu(option) {
  let result;
  switch (option) {
    case 1: result = "Home"; break;
    case 2: result = "Profile"; break;
    case 3: result = "Settings"; break;
    default: result = "Invalid selection";
  }
  return result;
}

function openStatus(day) {          // grouped cases share one block
  switch (day) {
    case "Mon": case "Tue": case "Wed": case "Thu": case "Fri":
      return "open 9-17";
    case "Sat": return "open 10-14";
    case "Sun": return "closed";
    default:    return "unknown day";
  }
}

function ageCase(age) {             // strict comparison: "18" is not 18
  switch (age) {
    case 18:   return "matched the NUMBER 18";
    case "18": return "matched the STRING '18'";
    default:   return "no match";
  }
}
console.log(describeType(42), "|", describeType("hi"), "|", describeType(null));
console.log([1, 2, 3, 4].map(menu).join(", "));
console.log(["Tue", "Sat", "Sun", "Funday"].map(openStatus).join(" | "));
console.log(ageCase("18"), "|", ageCase(18), "|", ageCase(18.0));
```

Output:

```
a number | a string | something else
Home, Profile, Settings, Invalid selection
open 9-17 | open 10-14 | closed | unknown day
matched the STRING '18' | matched the NUMBER 18 | matched the NUMBER 18
```

Notes:

- In `describeType`, `typeof null` is `"object"`, which matches no case, so `default` runs.
- Inside a function, `return` both produces the result and leaves the switch, so no `break` is needed.
- **Grouped cases**: empty cases fall through to the next non-empty one, so five weekdays share one block without repeating code.
- **Strict comparison**: `"18"` (string) only matches `case "18"`. `18.0` is the same number as `18`. If your value might arrive as a string (form input, URL parameter), convert it first: `switch (Number(age))`.

Fall-through — feature and trap

```
// code/03_switch.js — no breaks on purpose: each level also gets the lower levels' rights
function withoutBreak(level) {
  const granted = [];
  switch (level) {
    case "admin":  granted.push("delete");   // falls through
    case "editor": granted.push("edit");     // falls through
    case "viewer": granted.push("read");
  }
  return granted;
}
console.log("admin:", withoutBreak("admin"), "viewer:", withoutBreak("viewer"));
```

Output:

```
admin: [ 'delete', 'edit', 'read' ] viewer: [ 'read' ]
```

Intentional fall-through like this is rare; accidental fall-through (a forgotten `break`) is a classic bug. If you do it on purpose, add a `// falls through` comment — ESLint's `no-fallthrough` rule recognises it (ESLint is a linter: a tool that scans code for likely bugs).

Two useful patterns

```
// code/03_switch.js
function bmiCategory(bmi) {          // switch (true): each case is a boolean condition
  switch (true) {
    case bmi < 18.5: return "underweight";
    case bmi < 25:   return "normal";
    case bmi < 30:   return "overweight";
    default:         return "obese";
  }
}
const statusText = { 200: "OK", 404: "Not Found", 500: "Server Error" };  // lookup object
console.log([17, 22, 27, 33].map(bmiCategory).join(", "));
console.log(statusText[404], "|", statusText[418] ?? "Unknown status");
```

Output:

```
underweight, normal, overweight, obese
Not Found | Unknown status
```

- `switch (true)` matches the first case whose expression is `true` — a way to use ranges. It works, but many developers find a plain `if … else if` clearer.
- **A lookup object** (or a `Map` — a built-in key-to-value collection, Chapter 8) is often cleaner than a switch whose only job is to translate one value to another; add a fallback with `??`.

Scope inside cases

The whole switch body is **one block**, so `const r` declared in two different cases collides. Wrap each case body in its own braces when declaring variables:

```
case "circle": {
  const r = shape.r;
  return +(Math.PI * r * r).toFixed(2);
}
```

Trade-offs

	`switch`	`if … else if`	lookup object/Map
Compares	one value, `===` only	any condition	exact keys
Ranges (`< 18`)	only via `switch (true)`	natural	no
Risk	forgotten `break`	long chains	keys are strings (objects)
Best when	many discrete values, some sharing code	mixed or range conditions	pure value-to-value mapping

In the interview

"How does `switch` compare values?" With strict equality (`===`), evaluated once against each case top-down; execution starts at the first match and continues until `break`/`return` or the end — that's fall-through. `default` runs when nothing matches, and can appear anywhere (it's conventionally last).

- *Follow-up: Why is `switch("1")` not matching `case 1`?* Different types under `===`.
- *Follow-up: What's wrong with `let` in two cases?* The switch body is a single block, so two `let x` declarations are a redeclaration `SyntaxError`; wrap case bodies in braces.

TL;DR

`switch` = one value vs many `===` cases. End each case with `break` (or `return`), group cases to share code, use `default` as a catch-all, and wrap case bodies in `{ }` to declare variables.

3.5 Loops — why and which

What it is

A **loop** repeats a block of code — the **body** — while some condition holds or for each item in a collection. One run of the body is an **iteration**.

Without loops, printing "Hello" ten times means writing ten `console.log` lines, and processing a 10,000-item shopping list is impossible to write out. With a loop you write the body once and describe *how often* it should run.

Real uses: rendering every product in a cart, summing an order, retrying a network request, reading lines of a file, walking a grid in a game, polling (checking again and again at intervals) until a job is done.

Loop	Repeats…	Checks condition	Typical use
`for`	a counted number of times	before each iteration	index-based loops (using each item's position number, starting at 0) over arrays, counting
`while`	while a condition is truthy	before each iteration (may run 0 times)	unknown number of iterations
`do…while`	while a condition is truthy	**after** each iteration (runs at least once)	"do it, then decide whether to repeat"
`for…in`	once per enumerable (visible to loops, §3.9) **property name** of an object	—	inspecting an object's keys
`for…of`	once per **value** of an iterable (anything that can hand out values one by one, §3.10: array, string, Map, Set …)	—	the everyday way to walk a collection

3.6 The `for` loop

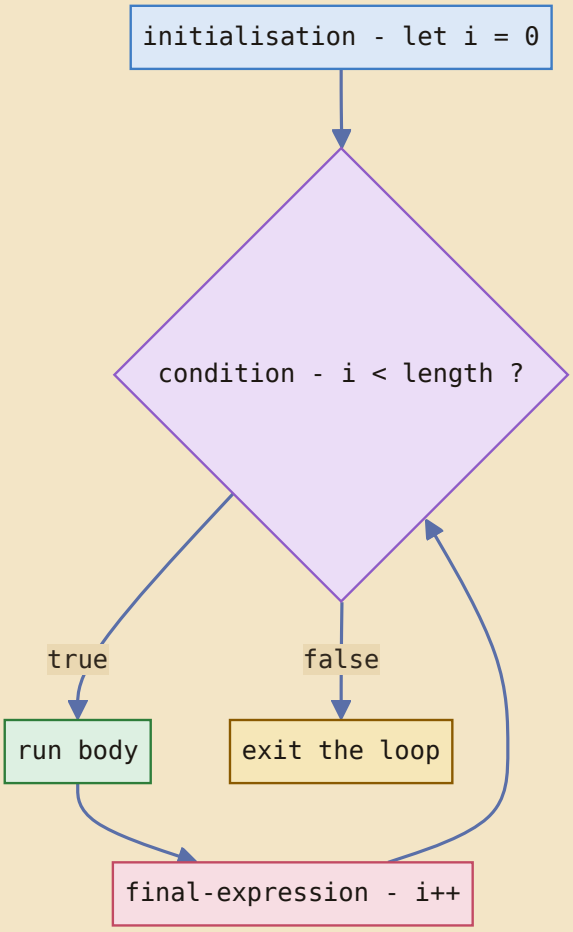
What it is

The classic `for` **loop** puts everything that controls the repetition in one header:

```
for (initialisation; condition; final-expression) {
  // body
}
```

Part	When it runs	Typical content
initialisation	once, before the loop starts	declare a counter: `let i = 0`
condition	before **every** iteration; falsy → stop	`i < items.length`
final-expression	after every iteration	update the counter: `i++`

How it works



Example

```
// code/04_for_loop.js (part 1)
const fruits = ["Apple", "Banana", "Cherry"];
for (let i = 0; i < fruits.length; i++) {
  console.log(i, fruits[i]);
}

const prices = [12, 5, 30, 8];
let total = 0;
for (let i = 0; i < prices.length; i++) {
  total += prices[i];
}
console.log("total:", total);

let down = "";
for (let i = 5; i > 0; i--) down += i + " ";
console.log("countdown:", down.trim());
let evens = [];
for (let i = 0; i <= 10; i += 2) evens.push(i);
console.log("evens:", evens.join(","));
for (let lo = 0, hi = 4; lo < hi; lo++, hi--) console.log(`pair lo=${lo} hi=${hi}`);
```

Output:

```
0 Apple
1 Banana
2 Cherry
total: 55
countdown: 5 4 3 2 1
evens: 0,2,4,6,8,10
pair lo=0 hi=4
pair lo=1 hi=3
```

Trace of the first loop: `i = 0` → `0 < 3` true → print → `i` becomes 1 → `1 < 3` true → … → `i` becomes 3 → `3 < 3` false → exit. Arrays are indexed from 0, so the last valid index is `length - 1`; that's why the condition is `<`, not `<=`.

The header is flexible: count down with `i--`, step with `i += 2`, or manage two variables separated by commas (the "two pointers" pattern used in many interview problems — Chapter 17).

Nested loops

A loop inside another loop is **nested**. For **each single iteration of the outer loop, the inner loop runs through all of its iterations**, then the outer loop moves on and the inner loop starts again from the beginning. With an outer loop of m iterations and an inner loop of n, the inner body runs $m \times n$ times.

```
// code/04_for_loop.js (part 2)
const grid = [
  [1, 2, 3],
  [4, 5, 6],
];
for (let row = 0; row < grid.length; row++) {
  let line = "";
  for (let col = 0; col < grid[row].length; col++) {
    line += `[${row},${col}]=${grid[row][col]} `;
  }
  console.log(line.trim());
}
let iterations = 0;
for (let a = 1; a <= 4; a++) {
  let row = "";
  for (let b = 1; b <= 4; b++) { row += String(a * b).padStart(3); iterations++; }
  console.log(row);
}
console.log("inner body ran", iterations, "times");
```

Output:

```
[0,0]=1 [0,1]=2 [0,2]=3
[1,0]=4 [1,1]=5 [1,2]=6
  1  2  3  4
  2  4  6  8
  3  6  9 12
  4  8 12 16
inner body ran 16 times
```

Nested loops are how you walk two-dimensional data — matrices (grids of numbers arranged in rows and columns), spreadsheet rows and columns, game boards, image pixels — and how you compare every item with every other item. That $m \times n$ cost is also why nested loops are the first thing to look at when code is slow: two nested loops over 10,000 items each is 100 million iterations. Interview problems often ask you to replace such an O(n²) nested loop (Big-O notation describes how work grows with input size n; O(n²) means doubling the input roughly quadruples the work) with a single loop and a `Map` or `Set` (Chapter 8).

Edge cases

```js
// code/04_for_loop.js (part 3)
let n = 0;
for (;;) { if (++n === 3) break; }       // all three parts are optional
console.log("stopped at", n);

const letters = ["a", "b"];
for (let i = 0; i <= letters.length; i++) console.log("i =", i, "->", letters[i]);
```

Output:

```
stopped at 3
i = 0 -> a
i = 1 -> b
i = 2 -> undefined
```

- `for (;;)` is an intentional infinite loop; something inside must `break`.
- **Off-by-one**: `<=` visits index 2, which doesn't exist — JavaScript doesn't throw, it quietly gives `undefined`.

`let` in the loop header

Declare the counter with `let`. Each iteration gets a **fresh copy** of `i`, which matters when a function created inside the loop remembers `i` (a closure). With `var` all iterations share one variable — a famous interview puzzle explained in Chapter 4 §4.7.

Common mistakes

- `i <= arr.length` instead of `<` (off-by-one).
- **Modifying the array you are looping over by index** (removing items shifts the rest, so elements get skipped). Loop backwards or build a new array with `filter`.
- **Forgetting to update the counter** (or updating the wrong one in a nested loop, e.g. `i++` in the inner loop) → infinite loop.

TL;DR

`for (init; condition; update)` — init once, check before each iteration, update after. Use `<` with `length`. Nested loops multiply iteration counts.

3.7 The `while` loop

What it is

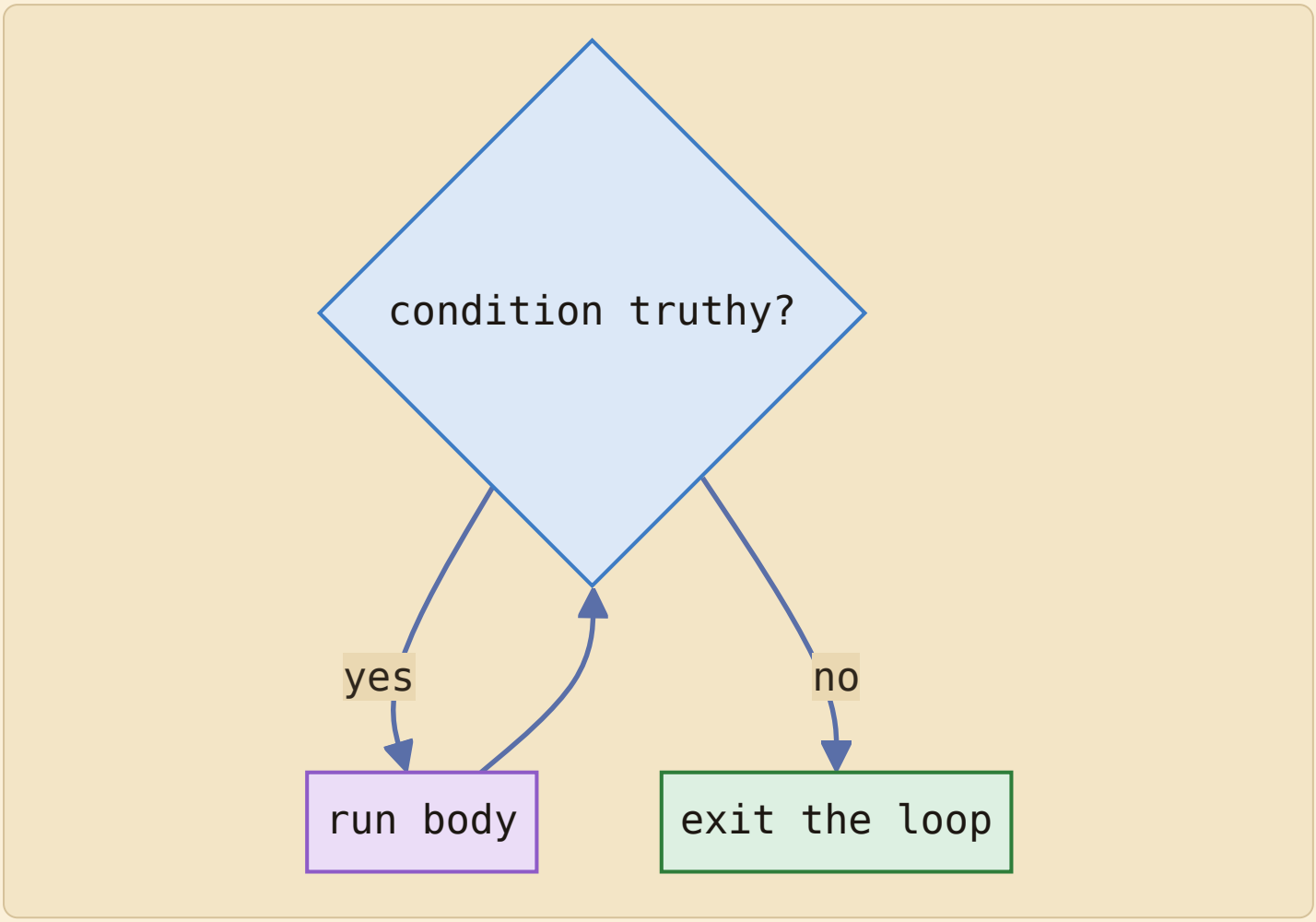
A `while` **loop** keeps running its body *as long as* its condition is truthy. The condition is checked **before** each iteration, so if it is falsy at the start, the body never runs.

```
while (condition) {
  // body — must eventually make the condition falsy (or break)
}
```

Use it when you **don't know in advance** how many iterations you need: keep reading until the input ends, keep retrying until it works, keep halving until the value is small enough. (A `for` loop is really a `while` loop with its counter bookkeeping gathered into the header — anything one can do, the other can.)

Analogy: "keep stirring while the sauce is still lumpy". You don't know how many stirs it will take; you check before each one.

How it works



Example

```
// code/05_while_do_while.js (part 1)
let count = 1;
let out = [];
while (count <= 5) {
  out.push(count);
  count++;
}
console.log("while 1..5:", out.join(" "));

const readings = [5, 3, 7, -1, 2];
let sum = 0, i = 0;
while (i < readings.length && readings[i] >= 0) {
  sum += readings[i];
  i++;
}
console.log(`sum before first negative: ${sum} (stopped at index ${i})`);

let value = 1000, steps = 0;
while (value >= 1) { value /= 2; steps++; }
console.log("halvings:", steps);
```

Output:

```
while 1..5: 1 2 3 4 5
sum before first negative: 15 (stopped at index 3)
halvings: 10
```

The second loop has a **compound condition**: stop at the end of the array *or* at the first negative number, whichever comes first. The order inside `&&` matters: `i < readings.length` is checked first, so `readings[i]` is never read past the end — short-circuit evaluation (Chapter 2 §2.7) protects it.

Common mistakes

- **Infinite loop** — the body never changes anything the condition depends on (`while (count <= 5) { console.log(count); }` with no `count++`). In the browser the tab freezes; in Node, press `Ctrl+C`.
- **Using `while` for a simple count** — a `for` loop keeps the counter logic in one visible place.
- **Busy-waiting** — `while (!dataArrived) {}` to wait for a network response never works in JavaScript: the single thread is stuck in your loop, so the response can never be processed. Waiting is done with callbacks (functions handed over to run later), promises (objects standing for a result that will arrive later) and `await` (Chapter 12).

TL;DR

`while (cond) { … }` checks first, may run zero times, and is for an unknown number of iterations. Make sure the body moves toward ending the loop.

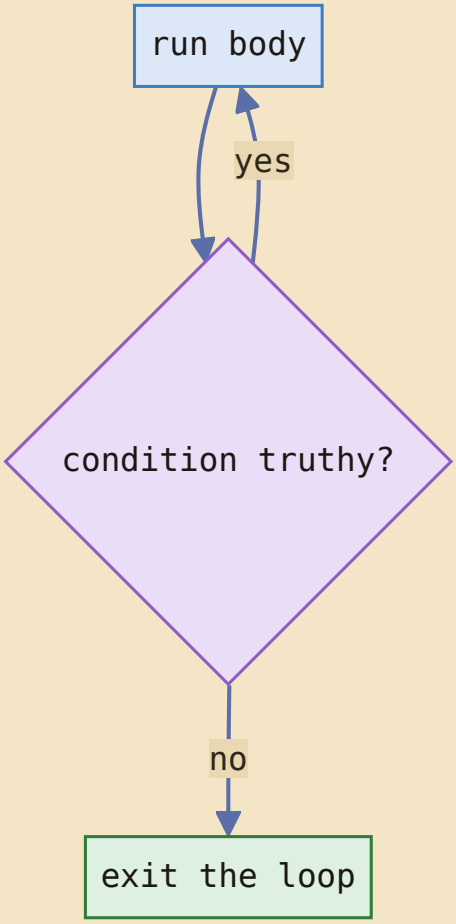
3.8 The `do…while` loop

What it is

A `do…while` **loop** runs its body **first** and checks the condition **afterwards**. The body therefore always runs **at least once**, even when the condition was already false before the loop began.

```
do {
  // body — always runs once
} while (condition);   // note the semicolon
```

How it works



Example

```
// code/05_while_do_while.js (part 2)
let x = 10;
while (x < 5) { console.log("while body (never)"); }
do { console.log("do-while body runs once, x =", x); } while (x < 5);

let c = 1; out = [];
do {
  out.push(c);
  c++;
} while (c <= 5);
console.log("do-while 1..5:", out.join(" "));

function makeRandom(seed) {            // tiny deterministic random generator, so the output is repeatable
  return () => (seed = (seed * 1103515245 + 12345) % 2 ** 31) / 2 ** 31;
}
const random = makeRandom(42);
let roll, rolls = [];
do {
  roll = Math.floor(random() * 6) + 1;   // 1..6
  rolls.push(roll);
} while (roll !== 6);
console.log("rolled until a 6:", rolls.join(" "));

let attempt = 0, ok = false;
do {
  attempt++;
  ok = attempt === 3;                    // pretend the 3rd try succeeds
  console.log(`attempt ${attempt}: ${ok ? "success" : "failed"}`);
} while (!ok && attempt < 5);
```

Output:

```
do-while body runs once, x = 10
do-while 1..5: 1 2 3 4 5
rolled until a 6: 4 4 6
attempt 1: failed
attempt 2: failed
attempt 3: success
```

The first two lines show the key difference: with the same false condition, `while` ran zero times and `do…while` ran once. The dice example is the natural shape for "generate something, then decide whether to try again" — you can't test the roll before you've rolled. (In real code you'd use `Math.random()`, Chapter 5; the seeded generator — one that starts from a fixed starting number, the *seed*, so it produces the same "random" sequence every time — keeps the printed output identical on every run.) The retry loop combines "until success" with a safety cap on attempts so it can't run forever.

Trade-offs — `while` vs `do…while`

	`while`	`do…while`
Condition checked	before each iteration	after each iteration
Minimum runs	0	1
Typical use	process while data remains	prompt/generate/try first, then decide
How common	very	rare — a `while` with a flag or `break` is often clearer

TL;DR

`do { … } while (cond);` runs the body once before testing. Use it only when the first run must happen unconditionally.

3.9 The `for…in` loop

What it is

`for…in` loops over the **property names (keys)** of an object — all **enumerable string-keyed** properties, **including inherited ones** (properties the object gets from another object it is linked to, rather than its own). It's useful when you need to inspect an object whose shape you don't know in advance.

```
for (const key in object) {
  // key is a string: the current property's name
}
```

(An **enumerable** property is one that shows up in loops like this — ordinary properties you create are enumerable; built-in methods like `toString` are not. Property attributes are covered in Chapter 5.)

Example

```
// code/06_for_in.js
const person = { firstName: "Grace", lastName: "Hopper", born: 1906 };
for (const key in person) {
  console.log(`${key}: ${person[key]}`);
}

const laptop = { brand: "Acme", ram: 16, ssd: 512, weight: 1.3 };
let propertyCount = 0;
for (const key in laptop) propertyCount++;
console.log("laptop has", propertyCount, "properties | Object.keys:", Object.keys(laptop).length);

const base = { kind: "vehicle" };
const bike = Object.create(base);   // bike's prototype is base (see Chapter 9)
bike.wheels = 2;
const all = [], own = [];
for (const key in bike) {
  all.push(key);
  if (Object.hasOwn(bike, key)) own.push(key);   // filter to own properties (ES2022)
}
console.log("all keys:", all, "| own keys:", own);

const colors = ["red", "green", "blue"];
colors.note = "primary-ish";
for (const index in colors) {
  console.log(typeof index, index, "->", colors[index]);
}

const mixed = { b: 1, 2: "two", a: 2, 1: "one" };
const order = [];
for (const k in mixed) order.push(k);
console.log("order:", order.join(", "));
```

Output:

```
firstName: Grace
lastName: Hopper
born: 1906
laptop has 4 properties | Object.keys: 4
all keys: [ 'wheels', 'kind' ] | own keys: [ 'wheels' ]
string 0 -> red
string 1 -> green
string 2 -> blue
string note -> primary-ish
order: 1, 2, b, a
```

What the output teaches:

1. **The loop variable is the key**; use bracket notation `person[key]` to get the value (`person.key` would look for a property literally named "key").
2. **Inherited properties appear too.** `bike` has only `wheels` of its own, but the loop also visited `kind` from its prototype (the object it falls back to when a property is missing — Chapter 9). Filter with `Object.hasOwn(obj, key)` (ES2022; older code uses `obj.hasOwnProperty(key)`), or avoid the issue with `Object.keys(obj)`, which returns only own enumerable keys.
3. **On arrays it's a trap.** Arrays are objects whose indexes are property names, so `for…in` gives you **strings** (`"0"`, not `0`) and also visits any extra property someone attached (`note`) — or any enumerable property a library added to `Array.prototype`. Use `for…of`, a `for` loop or array methods for arrays.
4. **Order:** integer-like keys come first in ascending numeric order, then other string keys in the order they were created.

Modern alternatives

Need	Use
own keys	`Object.keys(obj)`
own values	`Object.values(obj)`
own `[key, value]` pairs	`for (const [k, v] of Object.entries(obj))`
number of own properties	`Object.keys(obj).length`

These are what most modern code uses; `for…in` is mainly seen in older code and when you deliberately want inherited properties.

TL;DR

`for…in` iterates enumerable property **names** (strings), inherited ones included. Fine for plain objects (with `Object.hasOwn` if needed); don't use it on arrays.

3.10 The `for…of` loop

What it is

`for…of` (ES2015) loops over the **values** of any **iterable** — an object that knows how to hand out its values one at a time. Built-in iterables: arrays, strings, `Map`, `Set`, typed arrays (fixed-type arrays of raw numbers, used for binary data), the `arguments` object (a function's list of passed values), DOM collections such as `NodeList` (lists of page elements), and generators (functions that produce values one at a time). (How iterables work under the hood — the `Symbol.iterator` protocol, i.e. an agreed set of method names an object must provide — is in Chapter 8.)

```
for (const value of iterable) {
  // value is the current element
}
```

It gives you the values directly, with no index bookkeeping, which makes it the everyday choice for walking a collection.

Example

```
// code/07_for_of.js
const numbers = [10, 20, 30];
for (const n of numbers) console.log("number", n);

for (const ch of "Hi!") console.log("char", ch);

const word = "a😀b";
console.log("length:", word.length, "| for...of count:", [...word].length);
for (const ch of word) console.log("code point:", ch);

const tasks = ["write", "test", "ship"];
for (const [i, task] of tasks.entries()) console.log(`${i + 1}. ${task}`);

const stock = new Map([["apples", 5], ["pears", 0]]);
for (const [item, qty] of stock) console.log(item, "=", qty);
for (const tag of new Set(["js", "css", "js"])) console.log("tag", tag);

const scores = { ana: 90, ben: 72 };
try { for (const x of scores) {} } catch (e) { console.log("for...of on object ->", e.name + ":", e.message); }
for (const [name, score] of Object.entries(scores)) console.log(name, score);

const arr = ["x", "y"];
const ins = [], ofs = [];
for (const k in arr) ins.push(k);
for (const v of arr) ofs.push(v);
console.log("for...in ->", ins, "| for...of ->", ofs);

const nums = [1, 2, 3];
for (let n of nums) n *= 10;
console.log("unchanged:", nums);
```

Output:

```
number 10
number 20
number 30
char H
char i
char !
length: 4 | for...of count: 3
code point: a
code point: 😀
code point: b
1. write
2. test
3. ship
apples = 5
pears = 0
tag js
tag css
for...of on object -> TypeError: scores is not iterable
ana 90
ben 72
for...in -> [ '0', '1' ] | for...of -> [ 'x', 'y' ]
unchanged: [ 1, 2, 3 ]
```

What the output teaches:

1. **Strings** are iterated character by character — and correctly by **Unicode code point** (the unique number Unicode assigns to each character): the emoji counts as one character in `for…of`, while `.length` reports 4 because the emoji occupies two UTF-16 units (Chapter 6).
2. **Need the index as well?** Iterate `array.entries()`, which yields `[index, value]` pairs, and unpack them with array destructuring `[i, task]` (syntax that pulls items out of an array into separate variables) (Chapter 5).
3. **Maps** yield `[key, value]` pairs; **Sets** yield each unique value once.
4. **Plain objects are not iterable** → `TypeError`. Iterate `Object.keys/values/entries(obj)` instead.

5. `for…in` gives index *strings*; `for…of` gives *values*.
6. The loop variable is a **copy of each value**: reassigning it does not change the array. (If the values are objects, you *can* change their properties — the copy is a reference to the same object, Chapter 2 §2.4.) Use `const` for the loop variable unless you need to reassign it.

Trade-offs — `for…in` vs `for…of`

	`for…in`	`for…of`
Gives you	keys (always strings)	values
Works on	any object	iterables only (arrays, strings, Map, Set …)
Includes inherited properties	yes (enumerable ones)	n/a
Arrays	avoid	✓
Plain objects	✓	✗ — use `Object.entries(obj)`
Supports `break`/`continue`	yes	yes

TL;DR

`for…of` walks the values of arrays, strings, Maps, Sets and other iterables — the default loop for collections. Use `.entries()` for indexes and `Object.entries()` for plain objects.

3.11 `break`, `continue` and labels

What it is

Three tools change the normal flow of a loop:

Statement	Effect
`break`	exits the innermost loop (or `switch`) immediately; execution continues after it
`continue`	skips the rest of the current iteration and starts the next one (in a `for` loop, the final-expression still runs first)
`label:`	names a statement (usually an outer loop) so `break label` / `continue label` can target it from inside a nested loop

The syntax is `break;`, `continue;`, `break labelName;`, `continue labelName;`, and a label is any identifier followed by a colon placed before a statement: `outer: for (…) { … }`.

Example

```
// code/08_loop_control.js
for (let i = 0; i < 5; i++) {
  if (i === 3) break;           // leave the loop entirely
  console.log("break demo i =", i);
}

for (let i = 0; i < 5; i++) {
  if (i === 2) continue;        // skip the rest of this iteration only
  console.log("continue demo i =", i);
}

const orders = [{ id: 1, paid: true }, { id: 2, paid: false }, { id: 3, paid: false }];
let firstUnpaid = null;
for (const order of orders) {
  if (!order.paid) { firstUnpaid = order; break; }
}
console.log("first unpaid order:", firstUnpaid.id);

outer: for (let i = 0; i < 3; i++) {
  for (let j = 0; j < 3; j++) {
    if (i === 1 && j === 1) break outer;
    console.log(`labelled break: i=${i}, j=${j}`);
  }
}

const matrix = [[1, 2], [-3, 4], [5, 6]];
const rowsWithoutNegatives = [];
rows: for (const row of matrix) {
  for (const v of row) {
    if (v < 0) continue rows;
  }
  rowsWithoutNegatives.push(row);
}
console.log("rows without negatives:", JSON.stringify(rowsWithoutNegatives));

let k = 0; const odd = [];
while (k < 6) {
  k++;                          // advance BEFORE continue, or the loop never ends
  if (k % 2 === 0) continue;
  odd.push(k);
}
console.log("odd numbers:", odd.join(","));

console.log("some unpaid?", orders.some((o) => !o.paid), "| find:", orders.find((o) => !o.paid).id);
```

Output:

```
break demo i = 0
break demo i = 1
break demo i = 2
continue demo i = 0
continue demo i = 1
continue demo i = 3
continue demo i = 4
first unpaid order: 2
labelled break: i=0, j=0
labelled break: i=0, j=1
labelled break: i=0, j=2
labelled break: i=1, j=0
rows without negatives: [[1,2],[5,6]]
odd numbers: 1,3,5
some unpaid? true | find: 2
```

Walk-through:

- `break` at `i === 3` stops the loop, so only 0, 1, 2 print.
- `continue` at `i === 2` skips that one print; the loop carries on with 3 and 4.

- **Search-and-stop**: once the first unpaid order is found there's no reason to check the rest — `break` saves work.
- **Labelled `break`**: a plain `break` inside the inner loop would exit only the inner loop and the outer loop would continue with `i = 2`. `break outer` ends both loops at `i=1, j=1`.
- **Labelled `continue`**: as soon as a row contains a negative number, `continue rows` abandons that row and moves the *outer* loop to the next row, so the push is skipped.
- **`continue` in a `while` loop** jumps straight back to the condition — if the counter update were below the `continue`, it would be skipped and the loop would never end.

When not to use them

- Many "loop + `break`" searches are clearer as array methods that stop early by themselves: `find`, `findIndex`, `some`, `every` (Chapter 7) — as in the last line.
- Labels are legitimate but rare; if you need them often, consider moving the nested loops into a function and using `return`, which exits all loops at once.
- `break` and `continue` do **not** work inside `forEach` callbacks (`forEach` isn't a loop statement — the callback is a separate function). Use `for…of` if you need to stop early.

In the interview

"How do you exit a nested loop from the inner loop?" Label the outer loop and use `break outerLabel`, or put the loops in a function and `return`. A plain `break` only exits the innermost loop.

- *Follow-up: Can you `break` out of `forEach`?* No — it's a `SyntaxError` inside the callback (not a loop); throwing works but is hacky. Use `for…of`, `some` or `find`.
- *Follow-up: Does `continue` in a `for` loop skip the update expression?* No — the final-expression (e.g. `i++`) still runs before the next condition check.

TL;DR

`break` exits the loop, `continue` skips to the next iteration, labels let either target an outer loop. Prefer `find`/`some` for search-and-stop.

3.12 Choosing the right loop

Situation	Best choice
Visit every element of an array/string/Map/Set	`for…of`
Need the index too	`for…of` over `arr.entries()`, or a classic `for`
Iterate backwards, skip by steps, two pointers	classic `for`
Repeat until something happens (unknown count)	`while`
Must run once before checking	`do…while`
Keys of a plain object	`Object.keys/entries` + `for…of` (or `for…in` + `Object.hasOwn`)
Transform every element into a new array	`arr.map(...)` (Ch. 7)
Keep only some elements	`arr.filter(...)`
Combine all elements into one value	`arr.reduce(...)` or a `for…of` with an accumulator
Find the first match / check any / check all	`find` / `some` / `every`
Wait for asynchronous work (work that finishes later, e.g. a network request)	`for await…of`, `await` in a loop (Ch. 12) — never a busy `while`

Performance is rarely a reason to prefer one loop over another; modern engines optimise all of them. Choose the one that states your intent most clearly.

Chapter 3 TL;DR

- **Conditionals** choose a path: `if / else if / else` (first truthy branch wins — order specific to general), the ternary `c ? a : b` for choosing values, and `switch` for one value against many `===` cases (`break` to stop fall-through, `default` as catch-all, braces for per-case variables).
- **Loops** repeat a body: `for` (init; condition; update) for counting, `while` (check first, may run 0 times) for unknown counts, `do…while` (runs at least once), `for…in` (property **names**, inherited included — not for arrays), `for…of` (**values** of iterables — the default for collections).
- **Nested loops** run the inner loop fully for each outer iteration (m × n work).
- `break` exits, `continue` skips to the next iteration, **labels** target an outer loop; `forEach` can't be broken — use `for…of`, `find` or `some`.
- Watch for off-by-one (`<` vs `<=`), infinite loops (update the counter before `continue`), and truthiness surprises in conditions.

Next: Chapter 4 — Functions, Scope and Closures.

Chapter 4 — Functions, Scope and Closures

Functions are the heart of JavaScript. They let you name a piece of behaviour, reuse it, pass it to other code, and even return it from other functions. Understanding functions properly also means understanding **scope** (which variables a piece of code can see) and **closures** (functions that remember the variables around them) — the ideas behind private state (data only certain functions can touch), event handlers (functions that run when something happens, like a click), React hooks (functions such as `useState` that give React components memory between screen updates) and a large share of interview questions.

What you'll learn

1. Defining and calling functions: parameters vs arguments, `return`, missing and extra arguments, how values are passed.
2. Function declarations vs function expressions, and how hoisting treats each.
3. Arrow functions: every syntax form and every way they differ from regular functions.
4. Parameters in depth: defaults (evaluated at call time), rest parameters, the `arguments` object, spread, options objects.
5. Scope: global, function and block scope; lexical scoping; the scope chain; shadowing.
6. Hoisting — the complete picture.
7. Closures: counters, private state (the module pattern), factories, `once`, `memoize`, and the loop puzzle.
8. Higher-order functions and callbacks; composition and currying.
9. IIFEs, recursion and pure functions.

Code: `code/` — every "Output:" is real Node 24 output. The `this` keyword gets its own chapter: Chapter 9 — `this`, Prototypes and Classes.

4.1 What a function is

What it is

A **function** is a named, reusable block of code. You **define** it once — describing what inputs it takes and what it does — and then **call** (or **invoke**) it as many times as you like, from anywhere it's visible. A function can take inputs (**parameters**) and can hand back one output (a **return value**).

Analogy: a function is a vending machine. Building the machine (defining the function) doesn't produce a snack. Each time someone presses the button with some coins (calls it with arguments), it runs the same mechanism and gives something back (the return value).

Why it matters

- **Reuse** — write the logic once, call it wherever needed.
- **Readability** — `calculateShipping(order)` explains itself; forty inline lines don't.
- **Modularity and maintenance** — small functions with clear inputs and outputs are easy to test (Chapter 16) and to change without breaking unrelated code.
- **Abstraction** — callers need to know *what* a function does, not *how*.

How it works — anatomy of a function declaration

```
function functionName(parameter1, parameter2) {
  // body: statements that run when the function is called
  return someValue;   // optional
}
```

Part	Meaning
`function`	keyword that starts a function **declaration**
`functionName`	the name you'll call it by; by convention a verb in camelCase (`getUser`, `isValid`)
`(parameter1, parameter2)`	**parameters**: local variables that receive the inputs; there can be zero or more
`{ … }`	the **body**
`return value`	ends the function and sends `value` back to the caller

When you call `functionName(a, b)`:

1. Each **argument** expression (`a`, `b`) is evaluated, left to right.
2. A new **execution context** is created — a private workspace with its own local variables — and pushed onto the **call stack** (the engine's list of functions currently running).
3. Parameters are bound to the argument values, in order.
4. The body runs until a `return` statement or its closing brace.
5. The context is popped off the stack; the call expression evaluates to the returned value — or to `undefined` if nothing was returned.

Parameter vs argument: a *parameter* is the name in the definition (`name`); an *argument* is the actual value supplied in a call (`"Alice"`).

Example

```
// code/01_function_basics.js (abridged)
function greet() {                      // declaration: no parameters, no return
  console.log("Hello, world!");
}
greet();                                // call (invoke) it
greet();                                // ...as often as you like

function greetPerson(name) {            // one parameter
  console.log("Hello, " + name + "!");
}
greetPerson("Alice");                   // "Alice" is the argument

function square(number) {               // return sends a value back to the caller
  return number * number;
}
const result = square(4);
console.log("square(4) =", result, "| square(square(2)) =", square(square(2)));

function noReturn() {}
console.log("function without return gives:", noReturn());

function early(n) {
  if (n < 0) return "negative";         // return also ends the function immediately
  return "non-negative";
}
console.log(early(-1), early(3));

function describe(a, b) { return `a=${a}, b=${b}`; }
console.log(describe(1), "|", describe(1, 2, 3));

const isEven = function (number) {      // function expression
  return number % 2 === 0;
};
console.log("isEven(4):", isEven(4), "| isEven(5):", isEven(5));
console.log("typeof isEven:", typeof isEven, "| name:", isEven.name, "| length:", describe.length);
console.log("square without () is the function itself:", typeof square, "| with ():", square(3));

function tryToChange(num, obj) {
  num = 100;                  // changes only the local copy
  obj.changed = true;         // changes the shared object
  obj = { replaced: true };  // rebinding the local name doesn't affect the caller
}
let n = 1; const o = {};
tryToChange(n, o);
console.log("after call: n =", n, "| o =", o);
```

Output:

```
Hello, world!
Hello, world!
Hello, Alice!
square(4) = 16 | square(square(2)) = 16
function without return gives: undefined
negative non-negative
a=1, b=undefined | a=1, b=2
isEven(4): true | isEven(5): false
typeof isEven: function | name: isEven | length: 2
square without () is the function itself: function | with (): 9
after call: n = 1 | o = { changed: true }
```

What the output teaches:

1. **A function's result can feed another call** — `square(square(2))` is `square(4)`.
2. **No `return` → `undefined`.** And `return` exits immediately, so code after it never runs.
3. **JavaScript never checks argument counts.** Missing arguments become `undefined`; extra ones are ignored (though still reachable, §4.4).

4. **Functions are values** (**first-class** functions): they can be stored in variables, passed around and given properties. `typeof` reports `"function"`; `.name` is the function's name; `.length` is the number of declared parameters.
5. `square` **vs** `square()`: without parentheses you refer to the function itself; with them you call it. Passing `handleClick()` where `handleClick` was expected is a classic bug — you pass its *result*.
6. **Arguments are passed by value**, but for objects the value is a **reference** (Chapter 2 §2.4). The function can mutate (change in place) the caller's object (`obj.changed = true`), but reassigning the parameter only changes the local name. This is sometimes called "call by sharing".

Common mistakes

- **Forgetting** `return` in a function meant to compute something → the caller gets `undefined`.
- **Putting the return value on the next line** — `return` followed by a newline returns `undefined` (automatic semicolon insertion, Chapter 1 §1.6).
- **Calling instead of passing**: `button.addEventListener("click", save())` runs `save` immediately.

TL;DR

Define once, call many times. Parameters receive arguments (missing → `undefined`), `return` hands back a value (none → `undefined`), and functions are values you can store and pass.

4.2 Declarations vs expressions (and hoisting)

What it is

There are two classic ways to create a function with the `function` keyword:

	Function declaration	**Function expression**
Looks like	`function add(a, b) { … }` as a statement on its own	`const add = function (a, b) { … };` — a function used as a value
Name	required	optional (anonymous, or named for internal use)
Hoisting	**fully hoisted** — callable before its line	follows the variable: `const` / `let` → TDZ (Temporal Dead Zone, the stretch before the declaration line) error, `var` → `undefined` (so calling it gives `TypeError`)
Typical use	top-level helpers, named utilities	callbacks (functions handed to other code to call later), conditional definitions, values stored in objects

A third form, the **arrow function** (`const add = (a, b) => a + b;`), is also an expression; §4.3 covers it.

Example

```
// code/02_declarations_expressions_hoisting.js
console.log("declared later, called early:", add(2, 3));
function add(a, b) { return a + b; }

try {
  multiply(2, 3);
} catch (e) {
  console.log("const expression called early ->", e.name + ":", e.message);
}
const multiply = function (a, b) { return a * b; };

try {
  divide(6, 3);
} catch (e) {
  console.log("var expression called early ->", e.name + ":", e.message);
}
var divide = function (a, b) { return a / b; };

const factorial = function fact(n) { return n <= 1 ? 1 : n * fact(n - 1); };
console.log("factorial(5):", factorial(5), "| typeof fact outside:", typeof fact);

const anon = function () {};
const arrowFn = () => {};
console.log("inferred names:", anon.name, arrowFn.name);

(function () {
  "use strict";
  if (true) { function insideBlock() { return "hi"; } }
  console.log("strict: block function visible outside block?", typeof insideBlock);
})();
```

Output:

```
declared later, called early: 5
const expression called early -> ReferenceError: Cannot access 'multiply' before initialization
var expression called early -> TypeError: divide is not a function
factorial(5): 120 | typeof fact outside: undefined
inferred names: anon arrowFn
strict: block function visible outside block? undefined
```

What it shows:

1. A **declaration** can be called above the line where it's written — the engine creates the whole function when it enters the scope. This lets you put the "story" of a file at the top and helpers below.
2. A `const` **expression** is in the Temporal Dead Zone until its line (Chapter 2 §2.2); a `var` **expression** exists but holds `undefined`, so calling it gives "is not a function".
3. A **named function expression** (`function fact(n) {…}`) can refer to itself by that name inside its body — useful for recursion (a function calling itself, §4.10) and clearer stack traces (the list of nested function calls printed with an error) — but the name is not visible outside.
4. Anonymous functions assigned to variables get an **inferred** `name`, which is what shows up in error stack traces.
5. In strict code (modules, classes), a function *declared inside a block* is scoped to that block. (Sloppy mode has murky legacy rules for this; avoid block-level function declarations there.)

Which to use?

Both are fine; teams usually pick one style. Common guidance: declarations (or `const` + arrow) for named top-level functions, arrow functions for callbacks. What matters more is consistency and not relying on calling `const` functions before their definition.

TL;DR

Declarations are hoisted whole; expressions behave like the variable that holds them. Named function expressions can call themselves by name.

4.3 Arrow functions

What it is

Arrow functions (ES2015) are a compact syntax for function expressions: `(params) => body`. Beyond being shorter, they differ from regular functions in a few important ways — most notably, they don't have their own `this` (the keyword that refers to the object a function is currently working on — Chapter 9).

Syntax forms

Form	Example	Notes
one parameter	`n => n * n`	parentheses optional for exactly one simple parameter
several parameters	`(a, b) => a + b`	parentheses required
no parameters	`() => "hello"`	empty parentheses **required**
default / rest / destructured parameter	`(name = "Guest") => …`, `(...xs) => …`, `({ id }) => …`	parentheses required
expression body	`x => x * 2`	the expression's value is returned automatically (**implicit return**)
block body	`x => { const y = x * 2; return y; }`	braces → you must write `return`
returning an object literal (an object written directly in `{ }`)	`(x, y) => ({ x, y })`	wrap in parentheses, or the braces are read as a block

Example

```
// code/03_arrow_functions.js
"use strict"; // strict: a plain function call gets this === undefined
const square = n => n * n;                      // one parameter: parentheses optional
const add = (a, b) => a + b;                    // several parameters: parentheses required
const hello = () => "hello";                    // no parameters: empty parentheses required
const greet = (name = "Guest") => "Hello, " + name + "!"; // default parameter
const describe = (user) => {                    // block body: needs an explicit return
  const words = user.name.split(" ");
  return `${words.length} word(s)`;
};
console.log(square(5), add(2, 3), hello(), greet(), greet("Ana"), describe({ name: "Ada Lovelace" }));

const makePointWrong = (x, y) => { x: x, y };    // parsed as a block with a label -> returns undefined
const makePoint = (x, y) => ({ x, y });
console.log("without ():", makePointWrong(1, 2), "| with ():", makePoint(1, 2));

const prices = [5, 12, 30];
console.log("with tax:", prices.map(p => +(p * 1.2).toFixed(2)), "| over 10:", prices.filter(p => p > 10));

const timer = {
  seconds: 0,
  tickRegular() {
    return [1, 2, 3].map(function () { return this === undefined ? "undefined" : typeof this; });
  },
  tickArrow() {
    return [1, 2, 3].map(() => { this.seconds++; return this.seconds; });
  },
};
console.log("regular callback this:", timer.tickRegular()[0], "| arrow callback:", timer.tickArrow());

const counter = {
  count: 5,
  regular() { return this.count; },
  arrow: () => this?.count,                       // `this` comes from the surrounding (file-level) code
};
console.log("method regular:", counter.regular(), "| method arrow:", counter.arrow());

function outer() {
  const inner = () => arguments.length;         // uses outer's arguments
  return inner();
}
console.log("arrow sees outer arguments.length:", outer(1, 2, 3));
try { new square(2); } catch (e) { console.log("new on arrow ->", e.name + ":", e.message); }
```

Output:

```
25 5 hello Hello, Guest! Hello, Ana! 2 word(s)
without (): undefined | with (): { x: 1, y: 2 }
with tax: [ 6, 14.4, 36 ] | over 10: [ 12, 30 ]
regular callback this: undefined | arrow callback: [ 1, 2, 3 ]
method regular: 5 | method arrow: undefined
arrow sees outer arguments.length: 3
new on arrow -> TypeError: square is not a constructor
```

How arrows differ from regular functions

Feature	Regular `function`	Arrow `=>`
`this`	set by **how** the function is called (Ch. 9)	**lexical**: taken from the surrounding code, fixed forever (even `call`/`bind`, methods that run a function with a chosen `this`, can't change it)
`arguments` object	yes	no — uses the enclosing function's, if any
Can be called with `new`	yes (declarations/expressions)	no — `TypeError`; no `prototype` property
`super` (reaches the parent class), `new.target` (tells whether `new` was used)	own	lexical
Can be a generator (a function that pauses and hands out values one at a time with `yield`, Ch. 8)	yes	no
Hoisted	declarations yes	no (it's an expression)
Good as object method	yes	**no** — `this` won't be the object
Good as callback inside a method	needs `bind` / `self = this`	**yes** — sees the method's `this`

The `timer` example shows why lexical `this` matters: in `tickRegular`, the inner `function` callback is called plainly by `map`, so in strict mode its `this` is `undefined` — it lost the object. In `tickArrow`, the arrow borrows `this` from `tickArrow`, which *is* `timer`, so `this.seconds++` works. Before ES2015, people wrote `var self = this;` or `.bind(this)` to solve this; arrows made that unnecessary. The flip side is `counter.arrow`: an arrow used *as* a method doesn't get the object as `this` either — its `this` is whatever it was where the object literal was written. Here that is the top level of a CommonJS file (Node's original module format, using `require`/`module.exports`), where Node sets `this` to `module.exports` (an empty object), so `this.count` is `undefined`; in an ES module the top-level `this` is `undefined`, so `this?.count` would give `undefined` there too.

Why it matters / benefits

- **Concise** callbacks: `items.filter(i => i.inStock).map(i => i.price)`.
- **Predictable** `this` in callbacks, timers and event handlers inside class methods.
- Common in React code (React is a popular library for building user interfaces out of *components*, reusable pieces of UI) and functional-style code.

When not to use arrows

- **Object methods** that use `this` → use method shorthand `name() { … }`.
- **Constructors** (functions called with `new` to build objects) → use a `class` (Ch. 9).
- **DOM event handlers that need `this` to be the element** → use a regular function or `event.currentTarget` (Ch. 13).
- **Long, multi-step logic** — a named declaration can read better.

In the interview

30-second answer — "Arrow vs regular functions?" Arrows are shorter and have an implicit return for expression bodies, but the real difference is that they don't bind their own `this`, `arguments`, `super` or `new.target` — they take them from the enclosing scope. So they're great as callbacks inside methods, but wrong as object methods or constructors; they can't be used with `new` and have no `prototype`.

- *Follow-up: Can you change an arrow's `this` with `bind`?* No — `bind`/`call`/`apply` can pass arguments, but the `this` argument is ignored.
- *Follow-up: How do you return an object literal from an arrow?* Wrap it in parentheses: `() => ({ a: 1 })`.
- *Follow-up: How do you get all arguments in an arrow?* Use a rest parameter: `(...args) => args.length`.

> **TL;DR**
>
> `(params) => expression` returns the expression; `=> { … }` needs `return`. Arrows have lexical `this` and no `arguments`/`new` — perfect callbacks, poor methods.

4.4 Parameters: defaults, rest, `arguments` and spread

Default parameters — what problem they solve

Before ES2015, a missing argument had to be patched inside the body, typically with `||`:

```
function greet(name) {
  name = name || "Guest";      // old pattern
  return "Hello, " + name + "!";
}
```

That's extra code in every function, and it has a hidden bug: `||` replaces **every falsy value**, so a deliberate `""` or `0` gets overwritten. **Default parameters** put the fallback directly in the signature (the function's name plus its parameter list) — `function greet(name = "Guest")` — which is shorter, self-documenting, and replaces **only** `undefined`.

How defaults work

1. When the function is called, parameters are initialised **left to right**.
2. If the argument for a parameter is `undefined` (missing, or passed explicitly as `undefined`), the default expression is used. `null`, `0`, `""` and `false` are real values and are kept.
3. The default is an **expression evaluated at call time**, every time it's needed — not once when the function is defined. It can call functions (`new Date()`, `nextId()`) and refer to **earlier** parameters.
4. Parameters with defaults behave like `let` declarations: a default can't refer to a *later* parameter (TDZ).

Example

```
// code/04_parameters.js (part 1)
function oldGreet(name) {
  name = name || "Guest";
  return "Hello, " + name + "!";
}
function greet(name = "Guest") {
  return "Hello, " + name + "!";
}
console.log(oldGreet(), "|", greet(), "|", greet("Ana"));
console.log('empty string -> old:', oldGreet(""), "| new:", JSON.stringify(greet("")));
console.log("undefined triggers the default:", greet(undefined), "| null does not:", greet(null));

function orderPizza(type = "Cheese", size = "Medium") {
  return `Ordering a ${size} ${type} pizza.`;
}
console.log(orderPizza());
console.log(orderPizza("Pepperoni"));
console.log(orderPizza(undefined, "Large"));   // skip the first by passing undefined

let counter = 0;
const nextId = () => ++counter;
function createUser(name, id = nextId(), tag = `${name.toLowerCase()}-${id}`) {
  return { name, id, tag };
}
console.log(createUser("Ana"), createUser("Ben"), createUser("Cy", 99));

function addItem(item, list = []) { list.push(item); return list; }
console.log(addItem("a"), addItem("b"));

const clock = { now: () => "09:30:00" };
function logMessage(message, timestamp = clock.now()) {
  return `${timestamp}: ${message}`;
}
console.log(logMessage("Server started"), "|", logMessage("Custom", "10:00:00"));

function required(name) { throw new Error(`Missing parameter: ${name}`); }
function connect(url = required("url")) { return "connecting to " + url; }
try { connect(); } catch (e) { console.log(e.message); }
```

Output:

```
Hello, Guest! | Hello, Guest! | Hello, Ana!
empty string -> old: Hello, Guest! | new: "Hello, !"
undefined triggers the default: Hello, Guest! | null does not: Hello, null!
Ordering a Medium Cheese pizza.
Ordering a Medium Pepperoni pizza.
Ordering a Large Cheese pizza.
{ name: 'Ana', id: 1, tag: 'ana-1' } { name: 'Ben', id: 2, tag: 'ben-2' } { name: 'Cy', id: 99, tag: 'cy-99' }
[ 'a' ] [ 'b' ]
09:30:00: Server started | 10:00:00: Custom
Missing parameter: url
```

Things to notice:

- **Several defaults**: each parameter has its own; supplying the first argument overrides only the first. To use the default for an *earlier* parameter while supplying a later one, pass `undefined` explicitly — or better, use an options object (below).
- **Call-time evaluation**: `nextId()` ran once per call that needed it, and `tag` was built from the *earlier* parameters `name` and `id`. In real code a timestamp default like `timestamp = new Date().toLocaleTimeString()` gives each call its own current time; the example uses a fixed clock so the output is reproducible.
- **A fresh default every call**: `list = []` creates a new array per call, so `addItem("b")` doesn't see `"a"`. (Python programmers: JavaScript does *not* have Python's shared-mutable-default trap.)
- **Required parameters**: a default that throws turns a forgotten argument into a clear error.

Rest parameters, `arguments` and spread

```
// code/04_parameters.js (part 2)
function sum(...numbers) {
  let total = 0;
  for (const n of numbers) total += n;
  return total;
}
console.log("sum():", sum(), "| sum(1,2,3):", sum(1, 2, 3));
function tagAll(tag, ...words) { return words.map(w => `${tag}:${w}`); }
console.log(tagAll("js", "let", "const"));

function legacySum() {
  let total = 0;
  for (let i = 0; i < arguments.length; i++) total += arguments[i];
  return { total, isArray: Array.isArray(arguments), length: arguments.length };
}
console.log(legacySum(4, 5, 6));

const nums = [3, 9, 1];
console.log("Math.max(...nums):", Math.max(...nums), "| sum(...nums, 10):", sum(...nums, 10));
console.log("lengths:", orderPizza.length, sum.length, tagAll.length, createUser.length);

function makeButton({ label = "OK", color = "blue", disabled = false } = {}) {
  return `${label} (${color}${disabled ? ", disabled" : ""})`;
}
console.log(makeButton(), "|", makeButton({ label: "Save", disabled: true }));
```

Output:

```
sum(): 0 | sum(1,2,3): 6
[ 'js:let', 'js:const' ]
{ total: 15, isArray: false, length: 3 }
Math.max(...nums): 9 | sum(...nums, 10): 23
lengths: 0 0 1 1
OK (blue) | Save (blue, disabled)
```

Tool	Where	What it does
Rest parameter `...args`	last parameter in a definition	collects all remaining arguments into a **real array** (empty if none)
`arguments`	inside any non-arrow function	legacy **array-like** object of all arguments: has `length` and indexes, but no array methods; absent in arrows
Spread `...array`	in a call (or array/object literal)	expands an iterable (anything `for…of` can loop over: arrays, strings …) into separate arguments — the mirror image of rest
Options object with destructuring (pulling properties out into variables)	`function f({ a = 1, b = 2 } = {})`	named, order-independent parameters with defaults; the `= {}` lets you call `f()` with no argument at all
`fn.length`	property	number of parameters **before** the first one with a default or rest

Prefer rest parameters over `arguments` in new code: they're real arrays, work in arrows, and make the signature self-describing. Destructuring syntax is explained fully in Chapter 5; the options-object pattern is how most JavaScript APIs (the functions a library exposes for you to call) and React props (the settings object passed into a component) take many optional settings.

Common mistakes

- Expecting `null` to trigger a default — only `undefined` does. Use `??` inside the body if `null` should also fall back.

- Putting required parameters after optional ones (`function f(a = 1, b)`) — callers must pass `undefined` for `a`. Put defaults last or use an options object.
- Mutating `arguments` in sloppy mode also changes the named parameters (and vice versa); in strict mode they're independent. Another reason to prefer rest parameters.

In the interview

- *"When is a default parameter's expression evaluated?"* At call time, each time the argument is `undefined`, left to right, with access to earlier parameters.
- *"Difference between rest and spread?"* Same `...` syntax, opposite jobs: rest *collects* many values into an array (in a parameter list or destructuring pattern); spread *expands* an iterable into many values (in calls and literals).

TL;DR

Defaults replace only `undefined`, are evaluated per call and may use earlier parameters. `...rest` collects remaining arguments into an array; spread expands arrays into arguments; options objects give named parameters.

4.5 Scope: global, function and block

What it is

Scope is the region of code where a variable is visible. When code uses a name, JavaScript looks it up in the current scope, then in the enclosing scope, and so on outward. There are three kinds of scope you create with ordinary code:

Scope	Created by	Variables that live there	Visible
Global	top level of a script	top-level `var`, `let`, `const`, functions	everywhere
Function	every function body	`var`, `let`, `const` and parameters declared in the function	inside that function (and functions nested in it)
Block	any `{ … }`: `if`, `for`, `while`, `switch`, a bare block	`let`, `const`, `class` (and functions in strict mode)	only inside those braces

The key difference between the keywords: `var` **ignores blocks** — it's scoped to the nearest *function* (or global). `let` **and** `const` **respect blocks.** (ES modules also have a **module scope**: top-level names in a module are visible throughout that file but not globally — Chapter 10.)

Analogy: scopes are like one-way mirrored rooms nested inside each other. From an inner room you can see out into every room around it, but people in the outer rooms can't see in.

Example

```
// code/05_scope.js (abridged)
var globalVar = "global var";
let globalLet = "global let";
const globalConst = "global const";

function readGlobals() {
  return [globalVar, globalLet, globalConst].join(" | ");
}
console.log("inside function:", readGlobals());

function testScope() {
  if (true) {
    var functionScoped = "var: visible in the whole function";
    let blockLet = "let: only in this block";
    const blockConst = "const: only in this block";
    console.log("inside block:", blockLet, "/", blockConst);
  }
  console.log("after block:", functionScoped);
  try { blockLet; } catch (e) { console.log("after block, let ->", e.name + ":", e.message); }
}
testScope();
try { functionScoped; } catch (e) { console.log("outside function, var ->", e.name); }

for (var vi = 0; vi < 3; vi++) {}
for (let li = 0; li < 3; li++) {}
console.log("var loop counter after loop:", vi, "| let counter exists?", typeof li);

const color = "red";
function paint() {
  const color = "blue";            // shadows the outer color inside paint
  return color;
}
console.log("shadowed:", paint(), "| outer still:", color);

const level = "global";
function showLevel() { return level; }
function caller() {
  const level = "caller";
  return showLevel();              // showLevel looks up `level` where it was DEFINED
}
console.log("lexical scope result:", caller());

const a = 1;
function outer() {
  const b = 2;
  function inner() {
    const c = 3;
    return a + b + c;              // c local, b from outer, a from global
  }
  return inner();
}
console.log("scope chain sum:", outer());
```

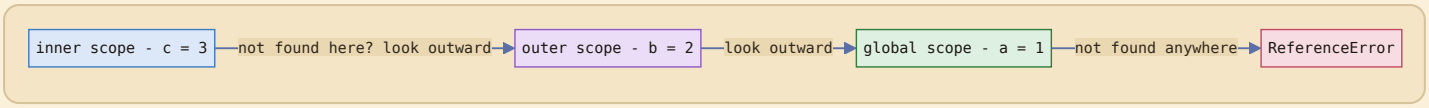
Output:

```
inside function: global var | global let | global const
inside block: let: only in this block / const: only in this block
after block: var: visible in the whole function
after block, let -> ReferenceError: blockLet is not defined
outside function, var -> ReferenceError
var loop counter after loop: 3 | let counter exists? undefined
shadowed: blue | outer still: red
lexical scope result: global
scope chain sum: 6
classic script globals -> var on global object: true | let: false | const: false
this CommonJS file's top-level var on globalThis? false
```

How it works — lexical scope and the scope chain

JavaScript uses **lexical (static) scoping**: the scopes a function can see are decided by **where the function is written** in the source code, not by where it's called from. That's why `caller()` returned `"global"`: `showLevel` was written at the top level, so it sees the global `level`, no matter that it was called from inside `caller`, which has its own `level`.

The nested scopes form the **scope chain**:



1. Look for the name in the current scope.
2. Not there? Look in the scope that lexically encloses it.
3. Repeat up to the global scope.
4. Still not found → `ReferenceError: x is not defined` (or, when *assigning* in sloppy mode, an accidental global is created).

Shadowing: an inner declaration with the same name as an outer one hides the outer one within the inner scope (`paint`'s `color`). The outer variable is untouched. Shadowing is legal but can confuse readers; linters (tools such as ESLint that scan code for likely mistakes) can flag it.

The global object and scripts vs modules

In a **classic browser script**, top-level `var` declarations and function declarations become properties of the global object (`window` / `globalThis`); top-level `let`/`const` are global *variables* but not properties. The example demonstrates this by running a snippet as a classic script with Node's `vm` module. In **ES modules** and **Node CommonJS files**, top-level declarations are private to the file — which is why `globalVar` did *not* appear on `globalThis` in the last line.

Why it matters

- **Fewer collisions**: two functions can both use a variable named `i` or `result` without interfering.
- **Encapsulation**: variables that are only visible where they're needed can't be changed by accident elsewhere. Global variables are shared by *every* script on a page — including third-party ones (code written by other companies, such as ads or analytics) — so keep the global scope clean.
- **Garbage collection** (the engine automatically reclaiming memory nothing uses any more): once no code can reach a scope's variables, their memory can be freed.

Common mistakes

- Declaring with `var` inside an `if` or `for` and being surprised it's visible after the block (the `vi` example).
- Relying on a variable from the *calling* function — scopes are lexical, not dynamic.
- Forgetting `let`/`const` and creating an accidental global (sloppy mode) — strict mode turns this into an error (Chapter 1 §1.8).

In the interview

"Explain scope in JavaScript." Scope decides where a name is visible. There's global scope, function scope and block scope (plus module scope). `var` is function-scoped; `let`/`const` are block-scoped. Scoping is lexical: a function sees the variables of the places where it's *defined*, walking outward along the scope chain, which is exactly what makes closures possible.

- *Follow-up: What's shadowing?* An inner declaration hiding an outer variable of the same name within the inner scope.
- *Follow-up: Does a top-level `let` create a `window` property?* No; only top-level `var` and function declarations in classic scripts do.

TL;DR

Global → function → block. `var` is function-scoped, `let`/`const` are block-scoped. Lookups walk outward along the lexical scope chain; unresolved names throw `ReferenceError`.

4.6 Hoisting — the complete picture

Chapter 2 §2.2 introduced hoisting for variables. Here is every kind of declaration side by side, observed *before* its line runs:

```
// code/06_hoisting.js (the report part)
function demo() {
  const report = [];
  report.push(["function declaration", typeof hoistedFn === "function" ? hoistedFn() : "missing"]);
  report.push(["var", String(v)]);
  try { l; } catch (e) { report.push(["let", e.name]); }
  try { c; } catch (e) { report.push(["const", e.name]); }
  try { new K(); } catch (e) { report.push(["class", e.name]); }
  try { fe(); } catch (e) { report.push(["var function expression", e.name + " (" + typeof fe + ")"]); }

  function hoistedFn() { return "callable"; }
  var v = 1;
  let l = 2;
  const c = 3;
  class K {}
  var fe = function () {};
  return report;
}
```

Output (full run):

```
function declaration     -> callable
var                      -> undefined
let                      -> ReferenceError
const                    -> ReferenceError
class                    -> ReferenceError
var function expression  -> TypeError (undefined)
ReferenceError: inner x is in its TDZ
var shadow hoisting: [ undefined, 'grumpy' ]
```

Declaration	Hoisted?	Value before its line
`function f() {}`	yes, with its body	the function (callable)
`var x`	yes	`undefined`
`let x`, `const x`	yes	uninitialised → `ReferenceError` (TDZ)
`class X {}`	yes	uninitialised → `ReferenceError` (TDZ)
`var f = function () {}` / `const f = () => {}`	as the variable	`undefined` (→ `TypeError` if called) / TDZ

The last two lines of output show the most confusing consequences:

- **`let` shadowing with TDZ**: a function containing `let x` later on can't read an *outer* `x` above that line — the inner `x` already exists (in its TDZ) for the whole block.
- **`var` shadowing**: inside `moodSwing`, `mood` reads as `undefined` rather than the global `"happy"`, because the function's own `var mood` is hoisted to the top of the function.

Mental model: before running a scope, the engine does a first pass that registers every declaration; then it runs the code. Nothing physically moves.

4.7 Closures

What it is

A **closure** is a function together with the variables from the scope where it was created. Because of lexical scoping (§4.5), an inner function can use the variables of its outer function. The surprising part: it can **keep using them after the outer function has returned**. Those variables stay alive for as long as the inner function exists.

Analogy: a backpack. When a function is created inside another function, it packs a backpack containing (references to) the surrounding variables and carries it wherever it goes. Whenever it runs — even much later, somewhere else — it can open the backpack.

Example — a counter with private state

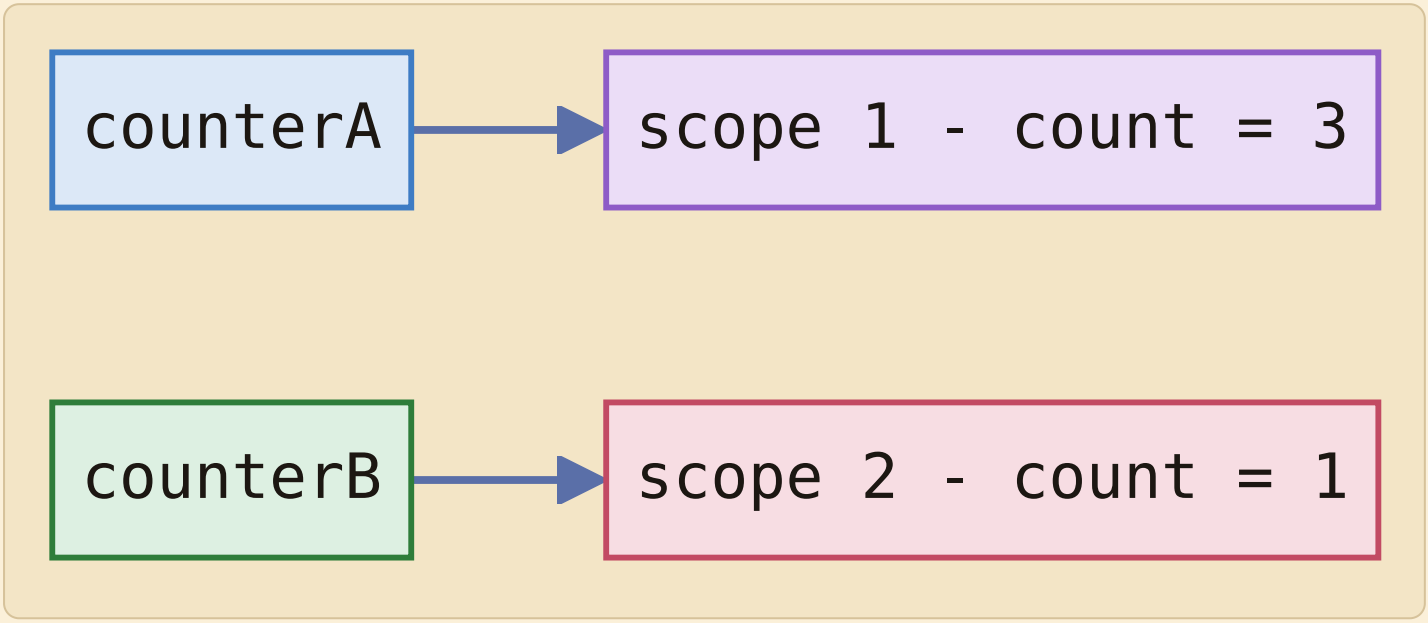
```
// code/07_closures.js (part 1)
function makeCounter() {
  let count = 0;                      // private: only the returned function can reach it
  return function () {
    count++;
    return count;
  };
}
const counterA = makeCounter();
const counterB = makeCounter();         // an independent closure with its own count
console.log("A:", counterA(), counterA(), counterA(), "| B:", counterB());
```

Output:

```
A: 1 2 3 | B: 1
```

How it works, step by step:

1. `makeCounter()` runs, creating a fresh scope with `count = 0`.
2. It creates the inner function — which captures a reference to that scope — and returns it.
3. `makeCounter` finishes. Normally its local variables would be discarded, but the returned function still references the scope, so it survives.
4. Each `counterA()` call increments *that* `count`. Nothing else in the program can read or change it.
5. `makeCounter()` called again creates a **new** scope, so `counterB` has its own independent `count`.



The module pattern — several functions sharing private state

```
// code/07_closures.js (part 2)
function createBankAccount(initial) {
  let balance = initial;
  const history = [];
  function record(type, amount) { history.push(`${type} ${amount}`); }
  return {
    deposit(amount) { balance += amount; record("deposit", amount); return balance; },
    withdraw(amount) {
      if (amount > balance) throw new Error("Insufficient funds");
      balance -= amount; record("withdraw", amount); return balance;
    },
    get balance() { return balance; },
    history: () => [...history],
  };
}
const acct = createBankAccount(100);
acct.deposit(50);
acct.withdraw(30);
console.log("balance:", acct.balance, "| direct access:", acct.balanceVar, "| history:", acct.history());
try { acct.withdraw(1000); } catch (e) { console.log("withdraw 1000 ->", e.message); }
```

Output:

```
balance: 120 | direct access: undefined | history: [ 'deposit 50', 'withdraw 30' ]
withdraw 1000 -> Insufficient funds
```

All the returned methods close over the same `balance` and `history`. The only way to change the balance is through `deposit`/`withdraw`, which enforce the rules. `history()` returns a copy so callers can't tamper with the real array. This is **encapsulation** without classes — the basis of the pre-ES2015 "module pattern" (§4.9). Classes now offer `#private` fields for the same goal (Chapter 9).

More closure patterns

```
// code/07_closures.js (part 3)
function makeMultiplier(factor) { return (x) => x * factor; }
const double = makeMultiplier(2), triple = makeMultiplier(3);
console.log("double(5):", double(5), "| triple(5):", triple(5));

function makeReader() {
  let message = "first";
  const read = () => message;
  message = "changed later";
  return read;
}
console.log("closure sees latest value:", makeReader()());

function once(fn) {
  let done = false, result;
  return (...args) => {
    if (!done) { done = true; result = fn(...args); }
    return result;
  };
}
const init = once(() => { console.log("  initialising..."); return 42; });
console.log("init:", init(), init(), init());

function memoize(fn) {
  const cache = new Map();
  return (n) => {
    if (cache.has(n)) return cache.get(n);
    const value = fn(n);
    cache.set(n, value);
    return value;
  };
}
let slowCalls = 0;
const slowSquare = (n) => { slowCalls++; return n * n; };
const fastSquare = memoize(slowSquare);
fastSquare(9); fastSquare(9); fastSquare(9);
console.log("memoized: underlying function ran", slowCalls, "time(s)");
```

Output:

```
double(5): 10 | triple(5): 15
closure sees latest value: changed later
  initialising...
init: 42 42 42
memoized: underlying function ran 1 time(s)
```

- **Function factories** — configure a function once (`factor`), get a specialised function back.
- **Closures capture variables, not values** — `read` returned `"changed later"`, the value at *call* time, because it holds a live reference to the variable.
- `once` — the `done` flag lives in the closure; used for one-time initialisation, "submit only once" buttons.
- `memoize` (memoization = remembering the results of earlier calls) — the cache (a store of saved results for quick reuse) lives in the closure; repeated calls with the same input skip the work. (Real memoizers must think about cache size and multi-argument keys.)

The classic loop puzzle

```
// code/07_closures.js (part 4)
const withVar = [], withLet = [];
for (var i = 0; i < 3; i++) withVar.push(() => i);
for (let j = 0; j < 3; j++) withLet.push(() => j);
console.log("var closures:", withVar.map(f => f()), "| let closures:", withLet.map(f => f()));

for (var t = 0; t < 3; t++) setTimeout(() => process.stdout.write(`var-timer ${t}  `), 0);
for (let u = 0; u < 3; u++) setTimeout(() => process.stdout.write(`let-timer ${u}  `), 0);
setTimeout(() => console.log(), 0);
```

Output:

```
var closures: [ 3, 3, 3 ] | let closures: [ 0, 1, 2 ]
var-timer 3  var-timer 3  var-timer 3  let-timer 0  let-timer 1  let-timer 2
```

Why: with `var` there is **one** `i` for the whole loop (function scope). All three closures capture that same variable, and by the time they run the loop has finished and `i` is 3. With `let` in a `for` header, JavaScript creates a **new binding** (a fresh name-to-value slot) **for each iteration**, copying the current value into it, so each closure captures its own `j`. The timer version is the same story with asynchronous callbacks (Chapter 12). Pre-ES2015 fixes used an IIFE per iteration (§4.9); today, just use `let`.

Why closures matter — real-world examples

- **Event handlers** remember the data they were created with: `button.addEventListener("click", () => deleteItem(item.id))` (Ch. 13).
- **Debounce / throttle** utilities (debounce = wait until calls stop for a moment, then run once; throttle = run at most once per time window) keep their timer IDs in a closure (Ch. 17).
- **React hooks**: every render's (each time React redraws a component) event handlers close over that render's props and state (the component's own remembered data) — which is also the source of "stale closure" bugs.
- **Callbacks with context**: `setTimeout(() => notify(user), 1000)` keeps `user` alive until the timer fires.
- **Private state** and factories, as above.

Common mistakes and costs

- **Stale closures**: a closure keeps reading a variable from an old scope (e.g. an old render in React) when you expected the latest value.
- **Memory**: anything a live closure references can't be garbage-collected. A long-lived event listener (a function registered to run on an event such as a click) that captures a huge array keeps that array in memory — remove listeners you no longer need.
- **The loop puzzle** — use `let`.

In the interview

30-second answer — "What is a closure?" A function bundled with references to the variables of the scope where it was defined. Because scope is lexical, an inner function can access outer variables — and it keeps that access after the outer function returns, so those variables stay alive. Closures give us private state (counters, the module pattern), function factories, memoization, and are what event handlers and callbacks rely on.

- *Follow-up: Why does `for (var i…) setTimeout(() => log(i))` print the final value three times?* One shared function-scoped `i`; the callbacks run after the loop ends. `let` gives one binding per iteration.
- *Follow-up: Do closures copy values?* No, they keep live references to the variables themselves, so later changes show up.
- *Follow-up: Can closures cause memory leaks (memory that is never freed because something still points at it)?* They keep referenced variables reachable; long-lived closures (listeners, timers, caches) holding large objects keep them in memory.

Self-check

1. After `const c = makeCounter(); c(); c();`, what does `c()` return? → `3`.
2. Do `makeCounter()` calls share a count? → No; each call creates a new scope.
3. How would you make a variable that only two functions can modify? → Declare it in an outer function and return both functions (the module pattern).

TL;DR

A closure = function + the variables around its definition, kept alive as long as the function is. Use it for private state, factories, `once`/`memoize`; use `let` in loops; beware stale values and retained memory.

4.8 Higher-order functions and callbacks

What it is

Because functions are values (**first-class**), they can be passed to and returned from other functions.

- A **callback** is a function you pass to another function so that it can call it ("call back") at the right moment.
- A **higher-order function** is a function that takes a function as an argument, returns a function, or both.

Analogy: leaving your phone number with a restaurant (the callback). You don't wait at the door; they call you when the table is ready.

Example

```
// code/08_higher_order_callbacks.js (abridged)
function repeat(times, action) {
  for (let i = 0; i < times; i++) action(i);
}
repeat(3, (i) => console.log("callback call", i));

const orders = [
  { id: 1, total: 25, status: "paid" },
  { id: 2, total: 90, status: "pending" },
  { id: 3, total: 40, status: "paid" },
];
const paidTotals = orders.filter((o) => o.status === "paid").map((o) => o.total);
const revenue = paidTotals.reduce((sum, t) => sum + t, 0);
console.log("paid totals:", paidTotals, "| revenue:", revenue);
console.log("sorted by total desc:", [...orders].sort((a, b) => b.total - a.total).map((o) => o.id));

function greaterThan(limit) { return (n) => n > limit; }
console.log("[5, 15, 25] > 10:", [5, 15, 25].filter(greaterThan(10)));

const compose = (...fns) => (x) => fns.reduceRight((acc, fn) => fn(acc), x);
const pipe = (...fns) => (x) => fns.reduce((acc, fn) => fn(acc), x);
const trim = (s) => s.trim(), lower = (s) => s.toLowerCase(), dashes = (s) => s.replace(/\s+/g, "-");
const slugify = pipe(trim, lower, dashes);
console.log("slug:", slugify("  Hello Big World  "), "| compose order:", compose(dashes, lower, trim)("  A B "));

const curriedAdd = (a) => (b) => (c) => a + b + c;
const add10 = curriedAdd(10);
console.log("curried:", curriedAdd(1)(2)(3), add10(5)(5));
function volume(l, w, h) { return l * w * h; }
const baseTen = volume.bind(null, 10);
console.log("partial:", baseTen(2, 3));

console.log("A: before");
[1].forEach(() => console.log("B: forEach callback runs immediately (synchronous)"));
setTimeout(() => console.log("D: setTimeout callback runs later (asynchronous)"), 0);
console.log("C: after");

function hits() { hits.count = (hits.count ?? 0) + 1; }
hits(); hits();
console.log("function property count:", hits.count);
```

Output:

```
callback call 0
callback call 1
callback call 2
paid totals: [ 25, 40 ] | revenue: 65
sorted by total desc: [ 2, 3, 1 ]
[5, 15, 25] > 10: [ 15, 25 ]
slug: hello-big-world | compose order: a-b
curried: 6 20
partial: 60
A: before
B: forEach callback runs immediately (synchronous)
C: after
function property count: 2
D: setTimeout callback runs later (asynchronous)
```

The patterns

Pattern	Idea	Example above
Callback	pass behaviour in	`repeat(3, action)`
Array HOFs (higher-order functions)	`map`, `filter`, `reduce`, `sort`, `find`, `some` … take callbacks (Ch. 7)	revenue of paid orders
Returning a function	build a predicate (a function that answers true/false)/handler from settings	`greaterThan(10)`
Composition	combine small functions into a pipeline; `pipe` runs left→right, `compose` right→left	`slugify`
Currying	turn `f(a, b, c)` into `f(a)(b)(c)` so arguments can be supplied one at a time	`curriedAdd`
Partial application	fix some arguments now, supply the rest later (`bind` can do it)	`baseTen`

Synchronous vs asynchronous callbacks

Look at the order **A, B, C … D**. `forEach` calls its callback *immediately*, before moving on — a **synchronous** callback. `setTimeout` only *registers* its callback; it runs later, after the current code (and even after the other logs) has finished — an **asynchronous** callback. Event listeners, timers and network responses all use asynchronous callbacks; how the engine schedules them is the event loop (Chapter 12), where you'll also meet "callback hell" and its cure, promises.

The last line shows that functions are objects: you can attach properties to them (rarely needed, but it explains why functions have `.name`, `.length`, `.bind` and so on).

Why it matters

- Separates **what** to do from **how/when**: `filter` knows how to loop; you only supply the rule.
- Declarative (describing *what* you want rather than each step), reusable code; the foundation of functional programming (a style built from small functions that return new values instead of changing existing ones) and of every event-driven API.

When not to use

- Deep chains of tiny anonymous callbacks can hurt readability and debugging — name them.
- For heavy loops in hot code (code that runs very often), a plain loop can be marginally faster than chained `map/filter`, but measure before optimising.

TL;DR

Callbacks are functions passed in; higher-order functions take or return functions. They enable array methods, composition, currying and every event/async API.

4.9 IIFE — Immediately Invoked Function Expressions

What it is

An **IIFE** (pronounced "iffy") is a function expression that is called right where it's defined: `(function () { … })();` or `(() => { … })();`. The surrounding parentheses turn the `function` keyword into an *expression* (a statement starting with `function` would be a declaration, which can't be called immediately — that's a `SyntaxError`).

The problem it solves

Before `let`/`const` and modules, the only way to create a private scope was a function. Wrapping a script in an IIFE kept its variables out of the global scope, preventing clashes between scripts on the

same page. The IIFE returning an object of public functions was *the* **module pattern** of the 2000s (jQuery plugins, early libraries; jQuery was the dominant library for working with web pages in that era).

Example

```
// code/09_iife.js
(function () {
  const secret = "only visible in here";
  console.log("IIFE ran:", secret);
})();
console.log("secret outside?", typeof secret);

const config = ((env) => {
  const base = { retries: 3 };
  return env === "prod" ? { ...base, debug: false } : { ...base, debug: true };
})("dev");
console.log("config:", config);

const Cart = (function () {
  const items = [];                          // private
  function total() { return items.reduce((s, i) => s + i.price, 0); }
  return {
    add(name, price) { items.push({ name, price }); return this; },
    total,
    count: () => items.length,
  };
})();
Cart.add("pen", 2).add("book", 12);
console.log("Cart:", Cart.count(), "items, total", Cart.total(), "| items exposed?", "items" in Cart);

(async () => {
  const value = await Promise.resolve("async IIFE done");
  console.log(value);
})();

try {
  eval("function () {}()");
} catch (e) {
  console.log("function statement called directly ->", e.name);
}
```

Output:

```
IIFE ran: only visible in here
secret outside? undefined
config: { retries: 3, debug: true }
Cart: 2 items, total 14 | items exposed? false
function statement called directly -> SyntaxError
async IIFE done
```

(The async IIFE's message appears last because `await` always resumes asynchronously — Chapter 12.)

Today

Old need	Modern replacement
private scope for a script	ES modules (each file has its own scope) or a plain `{ }` block with `let`/`const`
module pattern	ES modules with `export` (Ch. 10) or classes with `#private`
capture loop variable	`let` in the loop header
`await` at the top level	top-level `await` in modules

IIFEs still appear in bundler output (the combined files produced by tools that merge many modules into one), in snippets that must not leak globals (analytics tags — small scripts that track page visits), and as async IIFEs in CommonJS scripts. Recognise them; you'll rarely need to write them. If a line

starts with `(` and the previous line has no semicolon, ASI (Automatic Semicolon Insertion) won't insert one — which is why IIFEs are sometimes written `;(function () { … })()`.

> **TL;DR**
>
> `(function () { … })()` creates and runs a private scope at once. Modules and block scope have mostly replaced it.

4.10 Recursion

What it is

A **recursive** function is one that calls itself to solve a smaller version of the same problem. Every recursive function needs:

1. a **base case** — an input simple enough to answer directly, which stops the recursion;
2. a **recursive case** — which calls the function on a *smaller* input, moving toward the base case.

Analogy: Russian nesting dolls. To find the smallest doll you open the current one (recursive case) until you reach one that doesn't open (base case).

Example

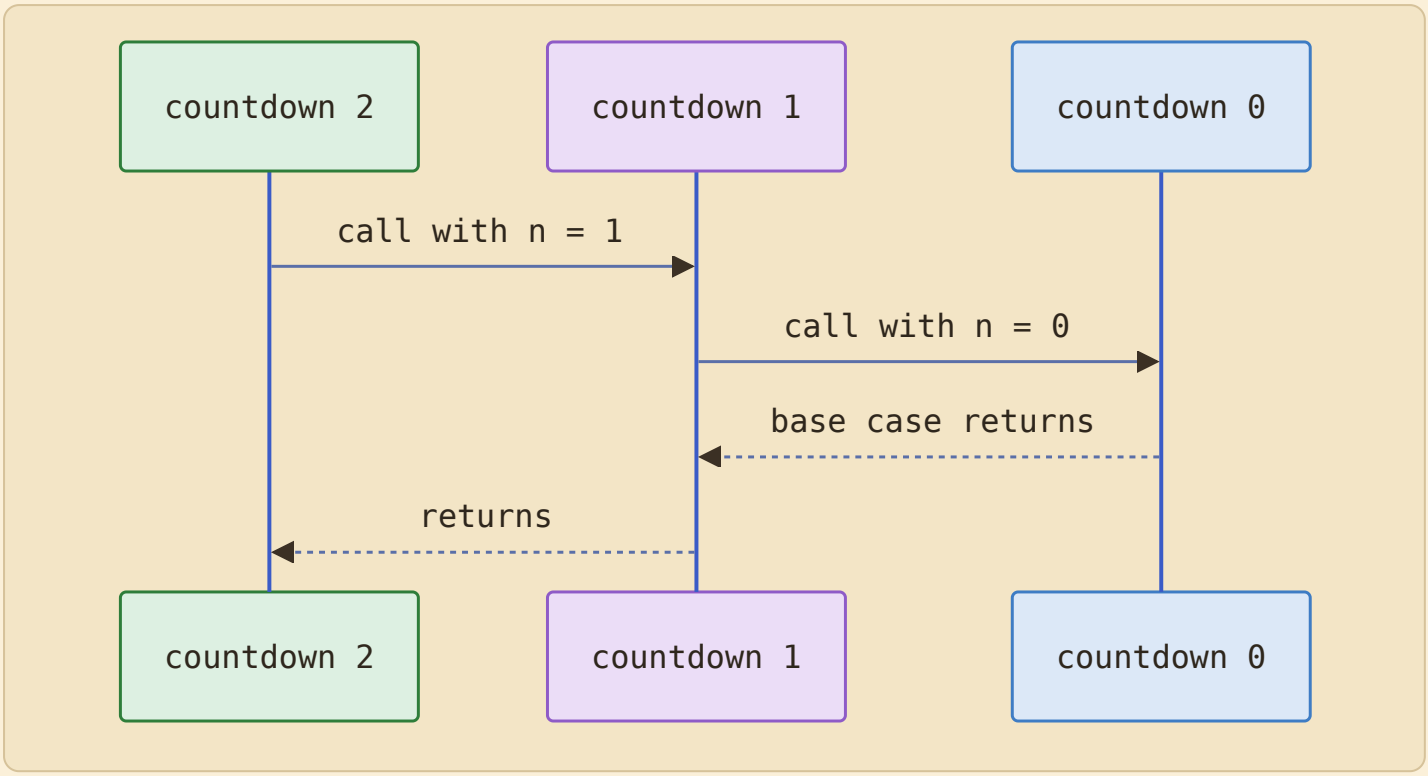
```
// code/10_recursion.js (abridged)
function factorial(n) {
  if (n <= 1) return 1;          // base case: stops the recursion
  return n * factorial(n - 1);   // recursive case: smaller input
}
console.log("factorial(5):", factorial(5));

function countdown(n, depth = 0) {
  const pad = "  ".repeat(depth);
  if (n === 0) { console.log(pad + "liftoff (base case)"); return; }
  console.log(pad + "countdown(" + n + ")");
  countdown(n - 1, depth + 1);
  console.log(pad + "back in countdown(" + n + ")");
}
countdown(2);

function totalSize(node) {       // folder tree: size of node + sizes of all children
  let sum = node.size;
  for (const child of node.children) sum += totalSize(child);
  return sum;
}
console.log("total size:", totalSize(folder));

function flatten(arr) {
  const out = [];
  for (const item of arr) {
    if (Array.isArray(item)) out.push(...flatten(item));
    else out.push(item);
  }
  return out;
}
console.log("flatten:", flatten([1, [2, [3, [4]], 5]]));

function forever(n) { return forever(n + 1); }
try { forever(0); } catch (e) { console.log("no base case ->", e.name + ":", e.message); }

let calls = 0;
function fib(n) { calls++; return n < 2 ? n : fib(n - 1) + fib(n - 2); }
fib(25);
console.log("naive fib(25) calls:", calls);
function fibLoop(n) { let a = 0, b = 1; for (let i = 0; i < n; i++) [a, b] = [b, a + b]; return a; }
console.log("fibLoop(25):", fibLoop(25), "| fibLoop(50):", fibLoop(50));
```

Output:

```
factorial(5): 120
countdown(2)
  countdown(1)
    liftoff (base case)
  back in countdown(1)
back in countdown(2)
total size: 15
flatten: [ 1, 2, 3, 4, 5 ]
no base case -> RangeError: Maximum call stack size exceeded
naive fib(25) calls: 242785
fibLoop(25): 75025 | fibLoop(50): 12586269025
```

How it works — the call stack

Each call gets its own execution context on the **call stack**. The countdown trace shows it: calls go *down* (each waiting for the next to finish), the base case returns, and then each waiting call resumes and finishes in reverse order.



The stack has a limited size (roughly ten thousand frames — one *frame* is the stored record of one active call — depending on the engine and frame size). Recursing too deep — or forever, with no base case — throws `RangeError: Maximum call stack size exceeded`, the JavaScript "stack overflow". JavaScript engines (except Safari's) do **not** implement tail-call optimisation (reusing the current stack frame when a function's very last action is a call to itself), so deep recursion can't be made safe by writing it in tail-call form.

When to use it (and when not)

Good fit	Better as a loop
tree-shaped data: file systems, the DOM, JSON, org charts, comments with replies	simple counting / summing over a flat list
divide-and-conquer algorithms (split the problem into halves, solve each, combine): merge sort, quicksort, binary search	inputs that could nest tens of thousands of calls deep
backtracking (try a choice, undo it if it leads nowhere): permutations, sudoku	naive recursions that repeat work (Fibonacci) unless memoized

The naive `fib(25)` made **242,785** calls because it recomputes the same values over and over — exponential time (each step up in *n* multiplies the work). Memoization (§4.7) or a simple loop makes it linear (work grows in step with *n*). Also note `fibLoop` stops at 50 on purpose: `fib(79)` and above exceed `Number.MAX_SAFE_INTEGER`, so results would silently lose precision (Chapter 2 §2.10) — use `BigInt` for those.

TL;DR

Recursion = base case + smaller recursive case. Natural for trees and divide-and-conquer; watch stack depth (`RangeError`) and repeated work (memoize).

4.11 Pure functions

What it is

A **pure function**:

1. always returns the **same output for the same inputs** (it depends only on its arguments), and
2. has **no side effects** — it doesn't modify anything outside itself (no mutating arguments or globals, no I/O (input/output, such as reading files or the keyboard), no DOM changes (edits to the web page), no logging, no network).

Anything else is **impure**. `Math.max` is pure; `Math.random`, `Date.now`, `console.log` and `array.push` are not.

Example

```
// code/11_pure_functions.js (abridged)
const addTax = (price, rate) => +(price * (1 + rate)).toFixed(2);   // pure
console.log("pure:", addTax(100, 0.2), addTax(100, 0.2));

let rate = 0.2;
const addTaxImpure = (price) => +(price * (1 + rate)).toFixed(2);   // reads outside state
console.log("impure before:", addTaxImpure(100));
rate = 0.25;
console.log("impure after someone changed rate:", addTaxImpure(100));

function addItemMutating(cart, item) { cart.push(item); return cart; }  // side effect
function addItemPure(cart, item) { return [...cart, item]; }          // returns a new array
const cart = ["apple"];
const newCart = addItemPure(cart, "pear");
console.log("pure add -> original:", cart, "| new:", newCart);
addItemMutating(cart, "plum");
console.log("mutating add -> original changed:", cart);

const assert = require("node:assert");
assert.strictEqual(addTax(10, 0.1), 11);
assert.deepStrictEqual(addItemPure([], "x"), ["x"]);
console.log("tests passed");
```

Output:

```
pure: 120 120
impure before: 120
impure after someone changed rate: 125
pure add -> original: [ 'apple' ] | new: [ 'apple', 'pear' ]
mutating add -> original changed: [ 'apple', 'plum' ]
Date.now() is impure: number
tests passed
```

Why it matters

- **Predictable and testable** — no setup, no mocks (fake stand-ins for real dependencies used in tests): call it and compare the result.
- **Safe to cache** (memoize) and to run in any order or in parallel.

- **Frameworks depend on it**: React requires components to be pure functions of their props and state during rendering; Redux reducers (in the Redux state library, functions that compute the next state from the current state and an action) must be pure. Returning a new array (`[...cart, item]`) instead of mutating is what lets them detect changes by reference (Chapter 2 §2.4).

Trade-offs

A program with no side effects does nothing visible — you must eventually write to the screen, disk or network. The practical goal is a **pure core, impure shell**: keep calculations pure and push side effects to the edges (event handlers, API calls). Copying instead of mutating costs some memory and time; for most UI-sized data it's negligible.

TL;DR

Pure = same inputs → same output, no side effects. Keep business logic (the rules specific to your app, like prices and discounts) pure; isolate I/O and mutation.

Chapter 4 TL;DR

- **Functions** are reusable, first-class values: define once, call many times; missing arguments are `undefined`, no `return` gives `undefined`; objects are passed as shared references.
- **Declarations** are hoisted whole; **expressions** (including arrows) follow their variable's hoisting (`const` → TDZ).
- **Arrow functions**: concise (`x => x * 2`, `() => ({…})`), lexical `this`, no `arguments`, can't be `new`ed — ideal callbacks, wrong as object methods.
- **Parameters**: defaults replace only `undefined` and are evaluated per call (can use earlier params); `...rest` collects into an array; spread expands; options objects give named parameters.
- **Scope**: global, function (`var`), block (`let`/`const`); lookups walk the **lexical** scope chain; inner names shadow outer ones.
- **Closures**: functions keep live access to the variables where they were defined — private state, factories, `once`, `memoize`, event handlers. `let` in loops gives one binding per iteration.
- **Higher-order functions & callbacks**: pass/return functions; `map/filter/reduce`, composition, currying; callbacks can be synchronous or asynchronous.
- **IIFE**: `(() => { … })()` — legacy private scope/module pattern, mostly replaced by modules.
- **Recursion**: base case + smaller case; watch `RangeError` stack overflows and repeated work.
- **Pure functions**: same input → same output, no side effects — easy to test, required by React and Redux.

Next: Chapter 5 — Objects, Math, Date and JSON. For `this`, `call/apply/bind` and classes, see Chapter 9; for error handling with `try/catch`, Chapter 11.

Chapter 5 — Objects, Math, Date and JSON

Almost everything interesting in a JavaScript program is an **object**: a user profile, an HTTP response (the reply a web server sends back to a request — see System Design Fundamentals), a configuration file, a DOM element (one piece of a web page, like a button, as the browser represents it to JavaScript), even an array or a function. This chapter teaches the everyday object toolkit — creating objects, reading and changing their properties, copying and merging them, locking them down — and then three built-in objects you will use constantly: `Math` for numbers, `Date` for time, and `JSON` for sending data between programs.

What you'll learn

1. What an object is, object literals, and how keys and values are stored.
2. Dot vs bracket access, computed keys, shorthand properties, adding/updating/deleting properties, checking whether a key exists, key order.
3. Methods and `this` inside a method (the short version — the full rules are in Chapter 9).
4. Spread and rest with objects, shallow vs deep copies, and destructuring.
5. The `Object.*` static methods: `keys`, `values`, `entries`, `fromEntries`, `assign`, `create`, `is`, and the reflection/locking family.
6. Property descriptors, `freeze` / `seal` / `preventExtensions`, getters and setters.
7. Optional chaining `?.` for safely reading deep data.
8. The built-in ("native") objects at a glance.
9. `Math`: constants, rounding, min/max, powers, trigonometry, random numbers.
10. `Date`: creating, reading, changing, comparing and formatting dates — plus the state of **Temporal**, its modern replacement.
11. `JSON.stringify` and `JSON.parse`, including replacers, revivers and what JSON cannot represent.

Before you start. You should know variables, primitive types and operators (Chapter 2) and functions and arrow functions (Chapter 4). Arrays get their own chapter (Chapter 7), and `Map` / `Set` — which are often better than plain objects for lookup tables — are in Chapter 8.

All runnable code is in `code/`. Every file is an ES module (`.mjs` — a file that uses the modern `import`/`export` system, Chapter 10), which means it runs in **strict mode** (a stricter variant of JavaScript where some silent failures become errors). Outputs shown are real, produced with Node 24.

5.1 What an object is

What it is

An **object** is a collection of **properties**. Each property is a **key** (a name, normally a string) paired with a **value** (any JavaScript value — a number, a string, another object, a function). A property whose value is a function is called a **method**.

Think of an object as a labelled filing box: every drawer has a label (the key) and holds something (the value). A real car is a good mental model: it has *properties* such as colour, brand and model, and it can *do* things such as drive or brake — those actions are its methods.

Compare that with a **primitive** (string, number, boolean, `null`, `undefined`, bigint, symbol), which is a single, unchangeable value. An object groups many related values — and the behaviour that works on them — under one name.

Why it matters

Real data is structured. A user is not "a string"; it is a name, an email, a list of roles, a signup date. Objects let you keep those pieces together, pass them around as one thing, and attach behaviour to them. JSON (section 5.12), the format of nearly every web API (a service other programs call over the internet to fetch or send data), is basically a text version of JavaScript objects and arrays.

How it works — creating objects with a literal

The simplest and by far the most common way to make an object is an **object literal**: curly braces containing `key: value` pairs separated by commas.

```
const car = {
  color: 'red',
  brand: 'Toyota',
  model: 'Corolla',
  drive() {                       // a method
    console.log('The car is moving');
  },
};
```

- `{ ... }` creates a new object each time this line runs.
- `color`, `brand`, `model` are properties holding strings.
- `drive` is a method. `drive() { ... }` is **method shorthand** (ES2015) for `drive: function () { ... }`.
- A trailing comma after the last property is allowed and makes diffs (the line-by-line "what changed" view in version control) cleaner.

Other ways to create objects exist — `new Object()` (never needed), `Object.create(proto)` (section 5.6), constructor functions and classes (Chapter 9) — but literals cover 90% of everyday code.

5.2 Reading, adding, updating and deleting properties

Dot notation vs bracket notation

There are two ways to reach a property:

	Dot notation `obj.key`	**Bracket notation** `obj['key']` / `obj[expr]`
Key must be a valid identifier?	Yes (letters, digits, `_`, `$`; cannot start with a digit; no spaces or dashes)	No — any string (or symbol, a unique key type from Chapter 8) works
Key known while writing the code?	Yes — the name is typed literally	Not necessary — the key can come from a variable or expression
Readability	Cleaner; the default choice	A little noisier
Typical use	`user.name`, `config.port`	`headers['content-type']`, `row[columnName]`, `scores[2024]`

Rule of thumb: **use dot notation unless you can't** — the key has special characters, or it is only known at run time (while the program is running, not when you write it).

Computed keys and shorthand properties

Two ES2015 features make literals more flexible:

- **Computed keys** — write `[expression]: value` and the *result* of the expression becomes the key.
- **Shorthand properties** — `{ name, age }` means `{ name: name, age: age }` when variables with those names exist.

`code/01_object_basics.mjs` (first part):

```
const field = 'price';

const laptop = {
  brand: 'Lenovo',            // plain property
  'model name': 'ThinkPad',   // key that is not a valid identifier -> must be quoted
  [field]: 1299,              // computed key: the value of `field` becomes the key
  [`${field}WithTax`]: 1299 * 1.2,
  open() {                    // method shorthand
    return `${this.brand} ${this['model name']} is open`;
  },
};

console.log(laptop.brand);                // known, valid identifier
console.log(laptop['model name']);        // spaces -> brackets required
const key = 'price';
console.log(laptop[key], laptop.priceWithTax);
console.log(laptop.color);                // missing property -> undefined, not an error

const name = 'Ada', age = 36;
const user = { name, age };
console.log(user);
```

Output:

```
Lenovo
ThinkPad
1299 1558.8
undefined
{ name: 'Ada', age: 36 }
```

Reading a property that does not exist gives `undefined` — no error. Reading a property *of* `undefined` (for example `laptop.color.length`) **is** an error (`TypeError`); section 5.8 shows how `?.` avoids it.

Adding, updating and deleting

Objects are **mutable**: you can change them after creation. Assigning to a key that exists updates it; assigning to a new key adds it. `delete obj.key` removes a property entirely.

```
laptop.brand = 'Dell';          // update (dot)
laptop['ram'] = '16GB';         // add (bracket)
delete laptop.priceWithTax;     // remove
console.log(laptop.open(), '|', Object.keys(laptop).join(','));
```

Output:

```
Dell ThinkPad is open | brand,model name,price,open,ram
```

Does the object have this key?

```
console.log('ram' in laptop, Object.hasOwn(laptop, 'ram'), laptop.ram !== undefined);
const withUndefined = { x: undefined };
console.log('x' in withUndefined, withUndefined.x !== undefined);
```

Output:

```
true true true
true false
```

Check	Looks at inherited (prototype) keys?	Works when the value is `undefined`?
`'key' in obj`	Yes (so `'toString' in {}` is `true`)	Yes
`Object.hasOwn(obj, 'key')` (ES2022)	No — own properties only	Yes
`obj.hasOwnProperty('key')`	No	Yes, but fails on objects without a prototype; prefer `Object.hasOwn`
`obj.key !== undefined`	Yes	**No** — a key explicitly set to `undefined` looks missing

(*Inherited* keys come from the object's **prototype** — a fallback object JavaScript checks when a key isn't found on the object itself; section 5.6.)

Key order

People often say "objects have no order". That was never quite true and has been fully specified since ES2020 for the common operations (`Object.keys`, `for...in`, `JSON.stringify`, …). The order of own keys is:

1. **Integer-like keys** (`'0'`, `'1'`, `'42'`) in ascending numeric order;
2. then other **string keys** in the order they were added;
3. then **symbol keys** in the order they were added (only reported by functions that include symbols).

```
const ordered = { b: 1, 2: 'two', a: 2, 1: 'one', [Symbol('s')]: 3 };
console.log(Object.keys(ordered));
console.log(Reflect.ownKeys(ordered));
```

Output:

```
[ '1', '2', 'b', 'a' ]
[ '1', '2', 'b', 'a', Symbol(s) ]
```

So the order is deterministic (always the same for the same input), but it is *not* pure insertion order when keys look like numbers. If you need a keyed collection where insertion order always wins (or keys that are not strings), use a `Map` (Chapter 8).

Objects are references

A variable holding an object holds a **reference** — an arrow pointing at the object — not the object itself. Copying the variable copies the arrow.

```
const a = { n: 1 };
const b = a;           // same object, second name
b.n = 99;
console.log(a.n, a === b, { n: 1 } === { n: 1 });

const cfg = { debug: false };
cfg.debug = true;      // allowed
console.log(cfg);
try { cfg = {}; } catch (e) { console.log(e.constructor.name + ':', e.message); }
```

Output:

```
99 true false
{ debug: true }
TypeError: Assignment to constant variable.
```

- `===` on objects asks "same object?", not "same contents?". Two separate literals with equal contents are never `===`.
- `const` freezes the **binding** (the variable cannot point elsewhere), not the object. To stop mutation, use `Object.freeze` (section 5.6).

Common mistakes

- Writing `obj.key` when the key is in a variable: `obj.field` looks for a property literally named `"field"`. Use `obj[field]`.
- Using a number-like string key and expecting insertion order.
- Believing `const` makes an object immutable.
- Comparing objects with `===` to test "equal contents" — compare the fields, or serialize both deliberately (convert each to a string such as JSON and compare the strings).

Self-check

1. How do you read a property named `first-name`? → `obj['first-name']` — the dash makes it an invalid identifier.
2. `const k = 'x'; ({ [k]: 1 })` produces what? → `{ x: 1 }`.
3. `'toString' in {}` vs `Object.hasOwn({}, 'toString')`? → `true` (inherited) vs `false` (not own).
4. What does `Object.keys({ z: 1, 10: 1, 2: 1 })` return? → `['2', '10', 'z']`.

5.3 Methods and `this`

What it is

A **method** is a function stored as a property. When you call it as `obj.method()`, JavaScript sets the special keyword `this` inside the function to `obj` — the object to the left of the dot. That is how one method can read the object's other properties.

```
const person = {
  firstName: 'Grace',
  lastName: 'Hopper',
  greet() {
    console.log(`Hi, I'm ${this.firstName} ${this.lastName}`);
  },
};
person.greet();   // Hi, I'm Grace Hopper
```

Methods can be called with brackets too: `person['greet']()`.

Common mistakes

- **Arrow functions as methods.** Arrow functions do not get their own `this`; they borrow it from the surrounding code. An arrow method on an object literal therefore does *not* see the object:

```
const counter = {
  count: 0,
  incArrow: () => typeof this,          // `this` is the module's this (undefined)
  inc() { this.count++; return this.count; },
};
console.log(counter.incArrow(), counter.inc());
```

Output (from `code/05_getters_setters_optional.mjs`):

```
undefined 1
```

- **Detaching a method.** `const f = person.greet; f();` loses `this` (it becomes `undefined` in strict mode). The full set of rules — default, implicit, explicit (`call`/`apply`/`bind`) and `new` binding — is in Chapter 9, section 9.1.

Returning `this` for chaining

A method that returns `this` lets calls be chained, the style used by jQuery (a classic library for manipulating web pages), query builders (libraries that build database queries step by step) and many test libraries:

```
deposit(amount) { this._balance += amount; return this; }
// account.deposit(10).deposit(5)
```

5.4 Spread, rest and copying objects

What it is

- **Spread** `{ ...source }` (ES2018) copies the *own* (not inherited), *enumerable* (visible to loops and listing methods) properties of `source` into a new object literal.
- **Rest** in destructuring `const { a, ...others } = obj` collects "everything I didn't name" into a new object.

Both use the same three dots; spread *expands* an object into a literal, rest *gathers* leftovers into a variable.

How it works

1. Properties are copied left to right.
2. If the same key appears twice, the **later one wins** — this is how you apply overrides on top of defaults.
3. Only the top level is copied. Nested objects are shared (a **shallow copy**).

From `code/02_object_static_methods.mjs`:

```
const defaults = { theme: 'light', lang: 'en', fontSize: 14 };
const prefs = { theme: 'dark' };
const merged = Object.assign({}, defaults, prefs); // later sources win
console.log(merged);

console.log({ ...defaults, ...prefs, fontSize: 16 });

const { password, ...safeUser } = { id: 7, name: 'Lin', password: 'x' };
console.log(safeUser);

const original = { name: 'Kit', address: { city: 'Pune' } };
const shallow = { ...original };
const deep = structuredClone(original);
shallow.address.city = 'Delhi';
console.log(original.address.city, deep.address.city);
```

Output:

```
{ theme: 'dark', lang: 'en', fontSize: 14 }
{ theme: 'dark', lang: 'en', fontSize: 16 }
{ id: 7, name: 'Lin' }
Delhi Pune
```

The last line is the classic bug: changing `shallow.address.city` also changed `original`, because both point to the *same* inner address object. **`structuredClone(value)`** (a global in browsers and Node 17+) makes a true **deep copy** — it copies nested objects, arrays, `Date`, `Map`, `Set`, typed arrays and even circular references (an object that points back to itself, directly or through others). It cannot copy functions, DOM nodes or class identity (instances — objects created from a class — come back as plain objects).

Trade-offs: ways to copy or merge

(A *getter* is a property that runs a function when read; a *setter* runs one when written — section 5.7.)

Technique	Depth	Mutates target?	Copies getters as getters?	Notes
`{ ...a, ...b }`	Shallow	No (new object)	No — getters are *called*, values copied	Most common; readable
`Object.assign(target, a, b)`	Shallow	**Yes** — writes into `target`	No — getters called, setters on target triggered	Use `Object.assign({}, ...)` for a fresh object
`structuredClone(obj)`	Deep	No	No	Handles Date/Map/Set/cycles; throws on functions
`JSON.parse(JSON.stringify(obj))`	Deep	No	No	Lossy: Dates → strings, drops `undefined`/functions, fails on cycles and BigInt (section 5.12)

Real-world examples

- Redux reducers (functions that compute an app's next state in the Redux library) and React state updates (changing the data a React component displays): `return { ...state, loading: false }` — create a new object instead of mutating.
- Config loaders: `{ ...defaultConfig, ...fileConfig, ...envConfig }` — later layers override earlier ones.
- Removing sensitive fields before logging or sending: `const { password, ...safe } = user`.

In the interview

> **30-second answer:** "Spread copies an object's own enumerable properties into a new literal; later keys override earlier ones. It's shallow — nested objects are shared. For a deep copy I use `structuredClone`, which handles Dates, Maps, Sets and cycles; the JSON round-trip is lossy."

- *Is `Object.assign` the same as spread?* Nearly; but `assign` mutates its first argument and triggers setters on it, while spread always creates a new object.
- *How would you deep-merge two configs?* Recurse: for each key, if both values are plain objects, merge them recursively; otherwise take the right-hand value. Or use a library such as lodash `merge` (lodash is a popular collection of utility functions).

Self-check

1. `{ ...{ a: 1 }, a: 2 }` → `{ a: 2 }`.
2. After `const c = { ...o }`, does `c.list.push(1)` change `o.list`? → Yes; shallow copy.
3. What does `structuredClone` do with a method? → Throws a `DataCloneError`.

5.5 Destructuring objects

What it is

Destructuring (ES2015) unpacks properties into variables in one statement: `const { a, b } = obj` is short for `const a = obj.a, b = obj.b`.

How it works

From `code/03_destructuring.mjs`:

```
const order = {
  id: 'A-17',
  customer: { name: 'Mira', address: { city: 'Lyon' } },
  items: 3,
};

const { id, items } = order;                      // basic
const { id: orderId } = order;                    // rename: take `id`, call it orderId
const { status = 'pending' } = order;             // default when the value is undefined
const { customer: { name, address: { city } } } = order; // nested
console.log(id, items, orderId, status, name, city);

const { a = 1, b = 2 } = { a: undefined, b: null };
console.log(a, b);

function createUser({ name, role = 'viewer', active = true } = {}) {
  return `${name ?? 'anonymous'} / ${role} / ${active}`;
}
console.log(createUser({ name: 'Jo', role: 'admin' }));
console.log(createUser());

let x, y;
({ x, y } = { x: 10, y: 20 });
console.log(x, y);

try { const { q } = null; } catch (e) { console.log(e.constructor.name); }
```

Output:

```
A-17 3 A-17 pending Mira Lyon
1 null
Jo / admin / true
anonymous / viewer / true
10 20
TypeError
```

Key points:

- `{ id: orderId }` reads as "take property `id`, store it in `orderId`" — the name on the **right** of the colon is the new variable.
- Defaults (`= 'pending'`) apply only when the value is `undefined` — **not** when it is `null`.
- In the nested pattern `customer: { name }`, `customer` is only a path; no `customer` variable is created.
- **Parameter destructuring** gives JavaScript "named arguments": callers pass an object, order does not matter, and each option has its own default. The trailing `= {}` makes calling with no argument safe.
- Assigning to *existing* variables needs parentheses, otherwise `{` starts a block.
- Destructuring `null` or `undefined` throws a `TypeError`.

Array destructuring (`const [first, second] = arr`) is covered in Chapter 7.

Self-check

1. `const { a: { b } } = { a: { b: 5 } }` — which variables exist? → Only `b` (value 5).
2. `const { n = 10 } = { n: 0 }` → `n` is `0`; defaults only replace `undefined`.
3. Why write `function f({ x } = {})`? → So `f()` with no argument does not throw.

5.6 `Object.*` static methods, descriptors and locking

What it is

The global `Object` holds **static methods** — helpers you call as `Object.method(obj)`, not on the object itself. They fall into four groups:

Group	Methods	What they do
Listing	`Object.keys(o)`, `Object.values(o)`, `Object.entries(o)`, `Object.fromEntries(pairs)`	Own enumerable string-keyed names / values / `[key, value]` pairs; `fromEntries` rebuilds an object from pairs
Copying & creating	`Object.assign(target, ...sources)`, `Object.create(proto, descriptors?)`	Copy properties into a target; make a new object with a chosen prototype
Reflection (code inspecting the structure of objects at run time)	`Object.getOwnPropertyNames(o)` (includes non-enumerable string keys), `Object.getOwnPropertySymbols(o)`, `Object.getOwnPropertyDescriptor(o, k)`, `Object.getOwnPropertyDescriptors(o)`, `Object.getPrototypeOf(o)`, `Object.setPrototypeOf(o, p)`, `Object.hasOwn(o, k)`, `Object.is(a, b)`	Inspect keys, flags and the prototype; compare values
Defining & locking	`Object.defineProperty(o, k, desc)`, `Object.defineProperties(o, descs)`, `Object.preventExtensions`, `Object.seal`, `Object.freeze`, and the tests `isExtensible`, `isSealed`, `isFrozen`	Control exactly how a property behaves; stop an object from changing

Listing: keys, values, entries, fromEntries

From `code/02_object_static_methods.mjs`:

```
const stock = { apples: 5, pears: 0, plums: 12 };

console.log(Object.keys(stock));
console.log(Object.values(stock));
console.log(Object.entries(stock));

const inStock = Object.fromEntries(
  Object.entries(stock).filter(([, qty]) => qty > 0).map(([k, qty]) => [k.toUpperCase(), qty]),
);
console.log(inStock);

for (const [fruit, qty] of Object.entries(stock)) console.log(`${fruit.padEnd(7)}${qty}`);
```

Output:

```
[ 'apples', 'pears', 'plums' ]
[ 5, 0, 12 ]
[ [ 'apples', 5 ], [ 'pears', 0 ], [ 'plums', 12 ] ]
{ APPLES: 5, PLUMS: 12 }
apples 5
pears  0
plums  12
```

The pattern **entries → array methods → `fromEntries`** is the idiomatic (natural, conventional for the language) way to "map" or "filter" an object, since objects have no `.map` of their own. All three listing methods return only **own, enumerable, string-keyed** properties — no inherited ones, no symbols — in the key order from section 5.2. (`for...in` also walks inherited enumerable keys, which is why `Object.keys`/`entries` with `for...of` is preferred; see Chapter 3.)

`Object.create` and `Object.is`

```
const animal = { speak() { return `${this.name} makes a sound`; } };
const dog = Object.create(animal);
dog.name = 'Rex';
console.log(dog.speak(), Object.getPrototypeOf(dog) === animal, Object.keys(dog));
const dict = Object.create(null);    // no prototype at all: a "pure" dictionary
console.log('toString' in dict, 'toString' in {});

console.log(NaN === NaN, Object.is(NaN, NaN), 0 === -0, Object.is(0, -0));
```

Output:

```
Rex makes a sound true [ 'name' ]
false true
false true true false
```

- `Object.create(animal)` makes an empty object whose **prototype** — the fallback object consulted when a property is not found — is `animal`. `dog` has only `name` of its own; `speak` is found on the prototype. Prototypes are explained properly in Chapter 9.
- `Object.create(null)` gives an object with *no* prototype — no inherited `toString`, `constructor` or `__proto__` — handy as a dictionary (a plain key-to-value lookup table) whose keys come from users.
- `Object.is` is `===` with two fixes: `NaN` equals itself, and `+0` and `-0` are different. `Map` and `Set` use almost the same rule (they treat `+0` and `-0` as equal).

Property descriptors

Every property has hidden settings called **attributes**, collected in a **property descriptor**:

Attribute	Meaning	Default for normal assignment	Default via `defineProperty`
`value`	The stored value	—	`undefined`
`writable`	Can the value be changed?	`true`	`false`
`enumerable`	Shows up in `Object.keys`, `for...in`, spread, `JSON.stringify`?	`true`	`false`
`configurable`	Can the property be deleted or its attributes changed?	`true`	`false`
`get` / `set`	Accessor functions (instead of `value`/`writable`) — section 5.7	—	`undefined`

From `code/04_descriptors_freeze.mjs`:

```
const tryIt = (label, fn) => {
  try { fn(); console.log(label, '-> ok'); } catch (e) { console.log(label, '->', e.constructor.name); }
};

const product = { name: 'Pen' };
console.log(Object.getOwnPropertyDescriptor(product, 'name'));

Object.defineProperty(product, 'sku', { value: 'P-001' });
console.log(Object.getOwnPropertyDescriptor(product, 'sku'));
console.log(Object.keys(product), Object.getOwnPropertyNames(product), product.sku);
tryIt('write read-only sku', () => { product.sku = 'X'; });
tryIt('delete non-configurable sku', () => { delete product.sku; });

Object.defineProperties(product, {
  price: { value: 2.5, writable: true, enumerable: true, configurable: true },
});
console.log(JSON.stringify(product)); // non-enumerable sku is skipped

const secret = Symbol('secret');
product[secret] = 42;
console.log(Object.keys(product), Object.getOwnPropertySymbols(product));
```

Output:

```
{ value: 'Pen', writable: true, enumerable: true, configurable: true }
{
  value: 'P-001',
  writable: false,
  enumerable: false,
  configurable: false
}
[ 'name' ] [ 'name', 'sku' ] P-001
write read-only sku -> TypeError
delete non-configurable sku -> TypeError
{"name":"Pen","price":2.5}
[ 'name', 'price' ] [ Symbol(secret) ]
```

Notice the asymmetry: a property created by plain assignment has every flag `true`; one created with `defineProperty` has every flag you omit set to `false`. In **strict mode** (modules, classes, `'use strict'`) writing a read-only property throws a `TypeError`; in old **sloppy mode** the write is silently ignored — a nasty source of "why didn't my change stick?" bugs.

Locking an object: preventExtensions, seal, freeze

```
const ext = { a: 1 };  Object.preventExtensions(ext);
const sealed = { a: 1 }; Object.seal(sealed);
const frozen = { a: 1, inner: { b: 1 } }; Object.freeze(frozen);

tryIt('preventExtensions: add   ', () => { ext.z = 1; });
tryIt('preventExtensions: change', () => { ext.a = 2; });
tryIt('preventExtensions: delete', () => { delete ext.a; });
tryIt('seal: add                ', () => { sealed.z = 1; });
tryIt('seal: change             ', () => { sealed.a = 2; });
tryIt('seal: delete             ', () => { delete sealed.a; });
tryIt('freeze: change           ', () => { frozen.a = 2; });
tryIt('freeze: nested change    ', () => { frozen.inner.b = 2; }); // freeze is shallow!
console.log(Object.isExtensible(ext), Object.isSealed(sealed), Object.isFrozen(frozen),
Object.isFrozen(frozen.inner));
```

Output:

```
preventExtensions: add    -> TypeError
preventExtensions: change -> ok
preventExtensions: delete -> ok
seal: add                 -> TypeError
seal: change              -> ok
seal: delete              -> TypeError
freeze: change            -> TypeError
freeze: nested change     -> ok
false true true false
```

Level	Add new keys	Delete keys	Change values	Implemented as
`Object.preventExtensions`	✗	✓	✓	object marked non-extensible
`Object.seal`	✗	✗	✓	+ every property `configurable: false`
`Object.freeze`	✗	✗	✗	+ every data property `writable: false`

All three are **shallow**: `frozen.inner` is still an ordinary, changeable object. A small recursive helper freezes everything:

```
function deepFreeze(obj) {
  for (const value of Object.values(obj)) {
    if (value && typeof value === 'object') deepFreeze(value);
  }
  return Object.freeze(obj);
}
const config = deepFreeze({ db: { host: 'localhost', port: 5432 } });
tryIt('deepFreeze: nested change', () => { config.db.port = 1; });   // -> TypeError
```

Finally, `Object.setPrototypeOf(obj, proto)` changes an existing object's prototype. It works, but engines optimise objects by their shape (the engine's internal record of which properties an object has, in which order), and swapping the prototype later can make code using that object slower — create it with the right prototype via `Object.create` instead.

When not to use it

- Don't `freeze` large hot-path (run very frequently, where speed matters) data "just in case"; it adds little safety in modules you control. Freeze exported constants and configuration, or use TypeScript's `readonly` for compile-time safety (errors caught by a checker before the code runs; TypeScript adds type labels to JavaScript) (Chapter 16).
- Don't reach for `defineProperty` for ordinary data — plain assignment is clearer. Use it for library-style work: hidden (non-enumerable) helpers, read-only constants, lazy getters (which compute a value only the first time it is asked for).

Common mistakes

- Assuming `Object.freeze` is deep.
- Using `defineProperty` and forgetting that omitted flags default to `false` — the property silently becomes hidden and read-only.
- Expecting `Object.keys` to include symbol keys or non-enumerable keys; use `Reflect.ownKeys` for "everything own".

In the interview

30-second answer: "Every property has a descriptor: value, writable, enumerable, configurable — or get/set for accessors. `preventExtensions` stops additions, `seal` also stops deletions, `freeze` also stops writes. All three are shallow, and violations throw only in strict mode."

- *How do you make a truly immutable object?* Deep-freeze recursively, or use immutable data structures (ones that are never changed in place — every update produces a new version; e.g. the Immer library, persistent collections) — `const` alone does nothing for contents.

- *What's the difference between `Object.keys` and `Object.getOwnPropertyNames`?* The latter includes non-enumerable string keys (e.g. an array's `length`).
- *When is `Object.is` different from `===`?* `NaN` vs `NaN` (true) and `0` vs `-0` (false).

Self-check

1. After `Object.seal(o)`, can you do `o.x = 5` for an existing `x`? → Yes, values stay writable.
2. What does `Object.defineProperty(o, 'k', { value: 1 })` make `Object.keys(o)` show? → Not `k` — it is non-enumerable.
3. Which method lists symbol keys? → `Object.getOwnPropertySymbols` (or `Reflect.ownKeys` for all).
4. Is `Object.isFrozen({})` true? → No; but `Object.isFrozen(Object.freeze({}))` is.

5.7 Getters and setters

What it is

An **accessor property** looks like a normal property from the outside but runs a function when read (**getter**, `get`) or written (**setter**, `set`). It is how you compute a value on demand or validate an assignment without changing how callers use the object.

How it works

From `code/05_getters_setters_optional.mjs`:

```
const account = {
  owner: 'Sam',
  _balance: 100,                    // underscore = "please treat as internal" (convention only)
  get balance() { return `$${this._balance.toFixed(2)}`; },
  set balance(v) {
    if (typeof v !== 'number' || v < 0) throw new RangeError('balance must be a non-negative number');
    this._balance = v;
  },
  deposit(amount) { this._balance += amount; return this; }, // returning this enables chaining
};

console.log(account.balance);           // getter runs; no parentheses
account.balance = 250;                  // setter runs
console.log(account.deposit(10).deposit(5).balance);
try { account.balance = -1; } catch (e) { console.log(e.name + ':', e.message); }
console.log(Object.getOwnPropertyDescriptor(account, 'balance'));

const rect = { w: 3, h: 4, get area() { return this.w * this.h; } };
rect.w = 10;
console.log(rect.area);
```

Output:

```
$100.00
$265.00
RangeError: balance must be a non-negative number
{
  get: [Function: get balance],
  set: [Function: set balance],
  enumerable: true,
  configurable: true
}
40
```

1. `account.balance` (no parentheses) calls the getter.
2. `account.balance = 250` calls the setter with `250`; the setter validates and stores into the "backing" property `_balance`.
3. The descriptor has `get`/`set` instead of `value`/`writable`.

4. `rect.area` is **derived** — always up to date with `w` and `h`, never stored.

The getter and the backing property **must have different names**; a setter that does `this.balance = v` would call itself forever. The underscore is only a convention; truly private state uses closures (Chapter 4) or class `#private` fields (Chapter 9).

When not to use it

Getters should be cheap and side-effect free — readers assume a property read is instant. Anything slow (network, heavy computation) belongs in an explicit method like `loadBalance()`. A getter with no setter makes the property read-only (the write throws in strict mode).

Real-world examples

Vue's reactivity (v2; Vue is a UI framework, and *reactivity* means the screen updates automatically when data changes) was built on `defineProperty` getters/setters; MobX and many ORMs (Object-Relational Mappers — libraries that map database rows to objects; e.g. computed fields in models) use accessors; DOM properties such as `element.innerHTML` and `input.value` are accessors on prototypes.

5.8 Optional chaining `?.`

The problem it solves

Data from APIs is often incomplete. Reading `res.data.user.name` throws `TypeError: Cannot read properties of undefined` if any link in the chain is missing. Before ES2020 people wrote `res && res.data && res.data.user && res.data.user.name`.

How it works

`a?.b` means: "if `a` is `null` or `undefined`, stop and return `undefined`; otherwise read `a.b`". It comes in three forms: `obj?.prop`, `obj?.[expr]`, and `fn?.(args)`. The whole rest of the chain is skipped once a `?.` short-circuits.

```
const res = { data: { user: { name: 'Ana', tags: ['a'] } } };
const empty = {};
console.log(res.data?.user?.name, empty.data?.user?.name);
console.log(res.data?.user?.tags?.[0], empty.data?.user?.tags?.[0]);   // with brackets
console.log(res.logout?.(), typeof empty.save?.());                    // calling a maybe-missing method
console.log(empty.data?.user?.name ?? 'Guest');                        // pair with ?? for a default
```

Output:

```
Ana undefined
a undefined
undefined undefined
Guest
```

Pair `?.` with the **nullish coalescing operator** `??` (Chapter 2) to supply a default only when the result is `null`/`undefined` (unlike `||`, which also replaces `0`, `''` and `false`).

Common mistakes

- Sprinkling `?.` everywhere. If a value *must* exist, let the code fail loudly — hiding a bug behind `undefined` makes it surface far away.
- `?.` cannot be used on the left side of an assignment: `obj?.x = 1` is a `SyntaxError`.
- `?.` only guards against `null`/`undefined`; `(5)?.toFixed` still works and `''?.length` is `0`.

5.9 JavaScript's built-in (native) objects

JavaScript ships with a standard library of **built-in objects** available in every environment (browser, Node, Deno, Bun) with no import. Some are **constructors** (you call them with `new`, or as converters), some are **namespaces** (plain objects holding static functions, never instantiated):

Built-in	Kind	Used for	Where in this book
`Object`	constructor + static helpers	Base of all objects; `Object.keys`, etc.	this chapter
`String`, `Number`, `Boolean`, `BigInt`, `Symbol`	wrappers / converters	Methods on primitives; conversions	Ch 2, Ch 6
`Array`	constructor	Ordered lists	Ch 7
`Function`	constructor	Every function is an object of this type	Ch 4
`Date`	constructor	Points in time	5.11
`RegExp`	constructor	Pattern matching on text	Ch 6
`Map`, `Set`, `WeakMap`, `WeakSet`, `WeakRef`	constructors	Keyed collections, unique values	Ch 8
`Promise`	constructor	Results that arrive later	Ch 12
`Error` (+ `TypeError`, `RangeError`, …)	constructors	Exceptions (error objects that are thrown to stop normal flow)	Ch 11
`Math`	namespace	Mathematical constants and functions	5.10
`JSON`	namespace	Converting to/from JSON text	5.12
`Intl`	namespace	Locale-aware (adapting to a language and region, e.g. `en-US`) formatting (dates, numbers, lists)	5.11, Ch 6
`Reflect`, `Proxy`	namespace / constructor	Meta-programming: `Reflect` exposes object operations as functions (`Reflect.ownKeys`, 5.2); a `Proxy` wraps an object and intercepts reads and writes (how Vue 3 reactivity and Immer work)	overview only
`ArrayBuffer`, typed arrays	constructors	Raw binary data	Ch 7
`globalThis`	the global object	Access to globals in any environment	Ch 1

Environment objects such as `document`, `window`, `fetch` or Node's `process` are **not** part of the language; they come from the host (the program that runs the engine: browser or Node) — see Chapters 13–15.

5.10 `Math`

What it is

`Math` is a **namespace object**: a plain object holding mathematical constants and functions as properties. It is not a constructor — you never write `new Math()` (that throws a `TypeError`); you call `Math.floor(x)` directly. In class terminology all its members are **static** (belong to `Math` itself rather than to instances).

Constants

Property	Value (approx.)	Meaning
`Math.PI`	3.14159	Circle circumference ÷ diameter
`Math.E`	2.71828	Euler's number, base of natural logarithms
`Math.SQRT2` / `Math.SQRT1_2`	1.41421 / 0.70711	√2 and √½
`Math.LN2` / `Math.LN10`	0.69315 / 2.30259	Natural log of 2 and of 10
`Math.LOG2E` / `Math.LOG10E`	1.44270 / 0.43429	$\log_2(e)$ and $\log_{10}(e)$

Methods by family

Family	Methods	Notes
Rounding	`round`, `floor`, `ceil`, `trunc`, `fround`	See the table below for negatives
Sign & size	`abs`, `sign`	`sign` returns -1, 0, 1 (or -0/NaN)
Extremes	`max(...xs)`, `min(...xs)`	Take separate arguments — spread arrays in
Powers & roots	`pow(x, y)` (same as `x ** y`), `sqrt`, `cbrt`, `hypot(...xs)` (√ of sum of squares)	
Exp & logs	`exp`, `expm1`, `log` (natural), `log10`, `log2`, `log1p`	`log(0)` is `-Infinity`; negative → `NaN`
Trigonometry (radians — an angle unit where a full circle is 2π)	`sin`, `cos`, `tan`, `asin`, `acos`, `atan`, `atan2(y, x)`, and hyperbolic `sinh`, `cosh`, `tanh`, …	`atan2` gives the angle of point (x, y), full −π…π range
Random	`random()`	Float in **[0, 1)** — 0 possible, 1 never
Integer tricks	`clz32`, `imul`	Low-level 32-bit helpers

Newer additions: `Math.f16round` (ES2025, rounds to half-precision float; in Node 24) and `Math.sumPrecise` (ES2026, adds a list of numbers without accumulated rounding error — not yet in Node 24, so check support before use).

How it works — examples

From `code/06_math.mjs`:

```
const r = 5;
console.log('area', Math.PI * r ** 2, Math.PI * Math.pow(r, 2));

const rows = [3.56, 3.5, -3.5, -3.56, 2.4].map((x) => ({
  x, round: Math.round(x), floor: Math.floor(x), ceil: Math.ceil(x), trunc: Math.trunc(x),
}));
console.table(rows);

console.log(Math.round(1.2345 * 100) / 100, (1.005).toFixed(2), (1.2345).toFixed(2));

const temps = [18, 25, 21, 30, 17];
console.log(Math.max(10, 20, 30, 40, 50), Math.min(...temps), Math.max(...temps));
console.log(Math.max(), Math.min(), Math.max(1, 'x'));

const clamp = (v, lo, hi) => Math.min(Math.max(v, lo), hi);
console.log(clamp(120, 0, 100), clamp(-5, 0, 100), clamp(42, 0, 100));
```

Output:

```
area 78.53981633974483 78.53981633974483
┌─────────┬──────┬───────┬───────┬──────┬───────┐
│ (index) │ x    │ round │ floor │ ceil │ trunc │
├─────────┼──────┼───────┼───────┼──────┼───────┤
│ 0       │ 3.56 │ 4     │ 3     │ 4    │ 3     │
│ 1       │ 3.5  │ 4     │ 3     │ 4    │ 3     │
│ 2       │ -3.5 │ -3    │ -4    │ -3   │ -3    │
│ 3       │ -3.56│ -4    │ -4    │ -3   │ -3    │
│ 4       │ 2.4  │ 2     │ 2     │ 3    │ 2     │
└─────────┴──────┴───────┴───────┴──────┴───────┘
1.23 1.00 1.23
50 17 30
-Infinity Infinity NaN
100 0 42
```

What to notice:

- **`round` rounds halves up toward +∞**: `Math.round(-3.5)` is `-3`, not `-4`. This is different from "round half away from zero" in many other languages.
- **`floor` vs `trunc`** only differ for negatives: floor goes down (−3.5 → −4), trunc chops the fraction (−3.5 → −3).
- **`toFixed` returns a string** and suffers from binary floating point: `1.005` is stored as 1.00499999…, so `(1.005).toFixed(2)` is `"1.00"`. For money, work in integer cents or use `Intl.NumberFormat` for display (see Chapter 2 on float precision).
- `Math.max()` with no arguments is `-Infinity` (the identity for "max" — a starting value that never changes the answer), and any argument that converts to `NaN` (such as `'x'`) makes the result `NaN` — numeric strings like `'5'` are converted and work. Spreading a *huge* array into `max`/`min` can exceed the engine's argument limit; use `reduce` for arrays with hundreds of thousands of items.

```
console.log(Math.abs(-7), Math.sign(-7), Math.sqrt(81), Math.cbrt(27), Math.hypot(3, 4));
console.log(Math.pow(2, 10), 2 ** 10, Math.exp(1), Math.log(Math.E), Math.log10(1000), Math.log2(8));

const toRad = (deg) => (deg * Math.PI) / 180;
console.log(Math.sin(toRad(30)).toFixed(4), Math.cos(toRad(60)).toFixed(4), Math.sin(Math.PI));
console.log('angle of (1,1):', (Math.atan2(1, 1) * 180) / Math.PI, 'deg');
```

Output:

```
7 -1 9 3 5
1024 1024 2.718281828459045 1 3 3
0.5000 0.5000 1.2246467991473532e-16
angle of (1,1): 45 deg
```

Trigonometric functions take **radians** (a full circle is 2π radians = 360°). `Math.sin(Math.PI)` is not exactly 0 because π itself cannot be stored exactly.

Random integers

`Math.random()` returns a float `x` with `0 ≤ x < 1`. To get an integer between `min` and `max` **inclusive**:

1. Multiply by the number of possible results, `max - min + 1` → float in `[0, count)`.
2. `Math.floor` → integer `0 … count-1`.
3. Add `min` → integer `min … max`.

```
const randInt = (min, max) => Math.floor(Math.random() * (max - min + 1)) + min;
```

The demo draws 30,000 values from `randInt(1, 3)` and confirms all land in `{1, 2, 3}` roughly evenly. Using `Math.round` instead of `floor` is a classic bug: the two end values then get only half the probability of the middle ones.

`Math.random` is not cryptographically secure (unpredictable enough to resist attackers) — its output can be predicted. For tokens, IDs, passwords or anything security-related use the Web Crypto API: `crypto.randomUUID()` (a UUID is a random 128-bit identifier, practically guaranteed to be unique) or `crypto.getRandomValues(new Uint32Array(1))` (global in browsers and Node).

In the interview

- *Why does `0.1 + 0.2 !== 0.3`, and how do you round to two decimals?* IEEE-754 binary floats (the standard binary format computers use for decimal numbers) can't store 0.1 exactly. Compare with a tolerance (`Math.abs(a - b) < Number.EPSILON`), round with `Math.round(x * 100) / 100`, or use integers (cents).
- *Random integer in [a, b]?* `Math.floor(Math.random() * (b - a + 1)) + a`.
- *Max of an array?* `Math.max(...arr)` for normal sizes; `arr.reduce((m, x) => x > m ? x : m)` for very large ones.

Self-check

1. `Math.round(-2.5)`? → `-2`.
2. `Math.trunc(-4.7)` vs `Math.floor(-4.7)`? → `-4` vs `-5`.
3. Can `Math.random()` return 1? → No; `[0, 1)`.
4. Convert 180° to radians. → `Math.PI`.

5.11 `Date`

What it is

A `Date` object represents **one instant in time**, stored internally as a single number: the milliseconds elapsed since the **Unix epoch**, 1 January 1970 00:00:00 **UTC** (Coordinated Universal Time — the time-zone-free reference clock). Everything else — year, month, hour, weekday — is *computed* from that number, either in **local time** (the time zone of the computer running the code) or in UTC.

That single fact explains most `Date` surprises: a `Date` has no time zone of its own; the time zone comes in only when you read or print it.

Why it matters

Calendars, reminders, booking systems, logs, subscriptions, "posted 3 hours ago" — all need time. `Date` is also infamous for pitfalls (0-based months, mutability, parsing quirks, DST — daylight-saving time, when clocks jump an hour in spring and autumn), which is why the language now has **Temporal** (end of this section).

Creating dates

Form	Creates	Example
`new Date()`	The current instant	`new Date()`
`new Date(ms)`	Instant `ms` milliseconds after the epoch (negative = before 1970)	`new Date(0)` → 1970-01-01T00:00:00Z
`new Date(string)`	Parsed from text; reliable only for **ISO 8601** (the international standard date-time text format: `YYYY-MM-DDTHH:mm:ss.sssZ`)	`new Date('2024-03-10T14:30:00Z')`
`new Date(y, m, d?, h?, min?, s?, ms?)`	Built from **local-time** parts; **month is 0-based** (0 = January, 11 = December); omitted day = 1, omitted time parts = 0	`new Date(2024, 0, 31, 9, 15)`
`Date.now()`	Not a Date — just the current ms number	handy for timing
`Date.UTC(y, m, …)`	ms number for those parts **in UTC**	`new Date(Date.UTC(2024, 0, 1))`

Calling `Date()` *without* `new` returns a string, not a Date — always use `new`.

From `code/07_date.mjs` (the script pins the time zone with `process.env.TZ = 'America/New_York'` — `process.env` holds environment variables, settings passed to a program from outside — so output is

repeatable):

```
const now = new Date();                                  // current moment
console.log('now is a Date:', now instanceof Date, typeof now.getTime());
const epoch = new Date(0);                               // ms since 1970-01-01T00:00:00Z
const fromMs = new Date(1_700_000_000_000);
const fromIso = new Date('2024-03-10T14:30:00Z');        // ISO 8601 string (Z = UTC)
const fromParts = new Date(2024, 0, 31, 9, 15);          // local time; month is 0-based (0 = Jan)
console.log(epoch.toISOString(), fromMs.toISOString());
console.log(fromIso.toISOString(), fromParts.toString());
```

Output:

```
now is a Date: true number
1970-01-01T00:00:00.000Z 2023-11-14T22:13:20.000Z
2024-03-10T14:30:00.000Z Wed Jan 31 2024 09:15:00 GMT-0500 (Eastern Standard Time)
```

Parsing pitfalls

```
console.log(new Date('2023-01-01').toString());
console.log(new Date('2023-01-01T00:00').toString());
console.log(new Date('not a date').getTime(), String(new Date('nope')));
```

Output:

```
Sat Dec 31 2022 19:00:00 GMT-0500 (Eastern Standard Time)
Sun Jan 01 2023 00:00:00 GMT-0500 (Eastern Standard Time)
NaN Invalid Date
```

- A **date-only** ISO string (`'2023-01-01'`) is treated as **UTC midnight** — which in New York is still 31 December. A date-*time* string without `Z` or an offset is treated as **local**. This inconsistency is in the spec, for historical reasons.
- Non-ISO strings (`'March 25, 2015'`, `'03/25/2015'`) are parsed in an implementation-specific way (each engine may do it differently). They usually work in modern engines, but the spec does not guarantee them — prefer ISO strings or numeric parts.
- An unparseable string gives an **Invalid Date**: a Date whose time value is `NaN`. Test with `Number.isNaN(d.getTime())`.

Reading components — getters

Local getter	UTC twin	Returns
`getFullYear()`	`getUTCFullYear()`	4-digit year (never use the obsolete `getYear`)
`getMonth()`	`getUTCMonth()`	**0–11**
`getDate()`	`getUTCDate()`	Day of month **1–31**
`getDay()`	`getUTCDay()`	Day of week **0–6**, 0 = Sunday
`getHours()` / `getMinutes()` / `getSeconds()` / `getMilliseconds()`	`getUTC…`	0–23 / 0–59 / 0–59 / 0–999
`getTime()` / `valueOf()`	—	ms since the epoch (same instant in any zone)
`getTimezoneOffset()`	—	Minutes to add to local time to get UTC (positive west of Greenwich, e.g. 240 for New York in summer)

```
const d = new Date('2024-07-04T23:30:00Z');
console.log('local:', d.getFullYear(), d.getMonth(), d.getDate(), d.getDay(), d.getHours(), d.getMinutes());
console.log('UTC  :', d.getUTCFullYear(), d.getUTCMonth(), d.getUTCDate(), d.getUTCDay(), d.getUTCHours());
console.log('offset minutes (UTC - local):', d.getTimezoneOffset());
```

Output:

```
local: 2024 6 4 4 19 30
UTC  : 2024 6 4 4 23
offset minutes (UTC - local): 240
```

Month `6` is **July**; weekday `4` is **Thursday**. The same instant is 19:30 in New York and 23:30 UTC.

Changing components — setters

Setters (`setFullYear`, `setMonth`, `setDate`, `setHours`, `setMinutes`, `setSeconds`, `setMilliseconds`, `setTime`, plus `setUTC…` twins) **mutate** the Date in place. Two properties make them useful:

1. Some accept several parts at once: `setFullYear(year, month?, day?)`, `setHours(h, m?, s?, ms?)`.
2. Out-of-range values **roll over** into the next unit — day 32 of January becomes 1 February, day 0 becomes the last day of the previous month.

```
const b = new Date(2024, 5, 15);
b.setFullYear(1990, 0, 1);                  // year, month, day in one call
console.log(b.toDateString());
const roll = new Date(2024, 0, 31);
roll.setMonth(1);                           // Feb 31 does not exist -> Mar 2 (2024 is a leap year)
console.log(roll.toDateString());
const addDays = (date, n) => { const c = new Date(date); c.setDate(c.getDate() + n); return c; };
console.log(addDays(new Date(2024, 11, 30), 5).toDateString());
const lastDayOfFeb = new Date(2024, 2, 0).getDate();   // day 0 of March = last day of Feb
console.log('days in Feb 2024:', lastDayOfFeb);
```

Output:

```
Mon Jan 01 1990
Sat Mar 02 2024
Sat Jan 04 2025
days in Feb 2024: 29
```

`addDays` copies first (`new Date(date)`) so the caller's Date isn't mutated, then relies on rollover to cross the year boundary. The "month-end overflow" (`Jan 31 + 1 month = Mar 2`) is a real bug source in billing code — decide explicitly whether you want the last day of the month instead.

Differences and comparisons

Subtracting two Dates converts both to milliseconds (via `valueOf`) and gives the difference as a number.

```
const start = new Date('2023-01-01');
const end = new Date('2023-12-31');
const MS_PER_DAY = 1000 * 60 * 60 * 24;
console.log((end - start) / MS_PER_DAY + ' days');
const beforeDst = new Date(2024, 2, 9), afterDst = new Date(2024, 2, 11);
console.log((afterDst - beforeDst) / MS_PER_DAY, Math.round((afterDst - beforeDst) / MS_PER_DAY));

const x = new Date(2024, 0, 1), y = new Date(2024, 0, 1);
console.log(x === y, x.getTime() === y.getTime(), x < addDays(y, 1));
```

Output:

```
364 days
1.9583333333333333 2
false true true
```

- 1 Jan to 31 Dec is 364 days apart (add 1 if you count both ends).
- Across the March **daylight-saving time** (DST) switch, two local midnights are only 47 hours apart, giving 1.958 "days". Round, or compute in UTC.
- `<` and `>` work on Dates (they compare the numbers), but `===` compares object identity. Compare `getTime()` values for equality.

Formatting

Method	Output (for the instant 2024-03-10 14:30 UTC, local zone New York)	Use for
`toISOString()`	`2024-03-10T14:30:00.000Z`	Storage, APIs, logs — always UTC, sortable
`toJSON()`	same as ISO	Used automatically by `JSON.stringify`
`toString()`	`Sun Mar 10 2024 10:30:00 GMT-0400 (Eastern Daylight Time)`	Debugging
`toDateString()` / `toTimeString()`	`Sun Mar 10 2024` / `10:30:00 GMT-0400 (…)`	Debugging
`toUTCString()`	`Sun, 10 Mar 2024 14:30:00 GMT`	HTTP headers (extra name/value lines sent with a web request or response)
`toLocaleDateString(locale, opts)` / `toLocaleTimeString` / `toLocaleString`	`3/10/2024` (en-US), `10/03/2024` (en-GB), `10.3.2024` (de-DE)	Showing dates to people
`Intl.DateTimeFormat(locale, opts).format(d)`	reusable formatter	Formatting many dates
`Intl.RelativeTimeFormat`	`yesterday`, `in 3 weeks`	"time ago" labels

```
console.log(t.toLocaleString('en-US', { timeZone: 'Asia/Kolkata', dateStyle: 'full', timeStyle: 'short' }));
const fmt = new Intl.DateTimeFormat('en', { month: 'short', day: 'numeric', year: 'numeric', timeZone: 'UTC' });
console.log(fmt.format(t));
const rtf = new Intl.RelativeTimeFormat('en', { numeric: 'auto' });
console.log(rtf.format(-1, 'day'), '|', rtf.format(3, 'week'));
```

Output:

```
Sunday, March 10, 2024 at 8:00 PM
Mar 10, 2024
yesterday | in 3 weeks
```

The `timeZone` option lets you *display* an instant in any IANA zone (the official worldwide time-zone database names: `'Asia/Kolkata'`, `'Europe/Berlin'`) — but `Date` itself still cannot *do arithmetic* in a chosen zone. That gap is what Temporal fills.

Temporal — the modern replacement (status in 2026)

Temporal is a new global namespace of immutable date/time types designed to fix `Date`'s problems. Verified status (September 2026):

- It reached **Stage 4** (finished) at the TC39 meeting of **March 2026**; TC39's finished-proposals list slots it into the **ES2027** edition (some articles say "ES2026"; the ES2026 snapshot was already cut).
- It is shipped by default in **Firefox 139+**, **Chrome/Edge 144+** and **Node.js 26+**. **Safari** stable did not yet ship it at the time of writing, and **Node 24** (this book's runtime) does not have it — our script prints `Temporal available here: false`. Use the `@js-temporal/polyfill` or `temporal-polyfill` package where it's missing (a **polyfill** is code that adds a missing built-in feature to older environments).

Its main types:

Type	Represents	Example
`Temporal.Instant`	An exact moment (like a Date's number, in nanoseconds)	a log timestamp
`Temporal.ZonedDateTime`	Instant + time zone + calendar; DST-aware arithmetic	a meeting in Berlin
`Temporal.PlainDate` / `PlainTime` / `PlainDateTime`	Wall-clock values with **no** time zone	a birthday; a store's opening time
`Temporal.PlainYearMonth` / `PlainMonthDay`	Partial dates	a credit-card expiry; an anniversary
`Temporal.Duration`	A length of time	"2 hours 30 minutes"
`Temporal.Now`	Current time in various forms	`Temporal.Now.plainDateISO()`

A taste (Node 26 / Chrome 144 / Firefox 139 — not run in this book's Node 24):

```
const due = Temporal.PlainDate.from('2024-01-31').add({ months: 1 });
due.toString();                                  // '2024-02-29' — clamps to month end, months are 1-based
const meeting = Temporal.ZonedDateTime.from('2026-03-28T09:00[Europe/Berlin]');
meeting.add({ days: 1 }).toString();             // DST-aware: still 09:00 local the next day
```

Why it's better: values are **immutable** (every operation returns a new object), months are **1-based**, parsing is strict ISO, time zones and calendars are first-class, and there are separate types for "an instant" and "a date on a wall calendar". Until Temporal is everywhere you target, libraries like **date-fns**, **Luxon** or **Day.js** fill the gap.

Common mistakes

- Forgetting that months are 0-based (`new Date(2024, 1, 1)` is **February** 1).
- Confusing `getDate()` (day of month) with `getDay()` (day of week).
- Parsing `'2023-01-01'` and being surprised it shows 31 December in the Americas.
- Mutating a Date that other code also holds (setters change it in place). Copy with `new Date(d)` first.
- Storing local-time strings in databases. Store UTC (ISO strings or epoch ms) and convert only for display.
- Computing "days between" with raw division across DST.

Real-world examples

- APIs send ISO 8601 strings (`"2024-05-01T08:00:00Z"`); clients parse with `new Date(str)` and display with `Intl.DateTimeFormat`.
- Performance timing uses `Date.now()` (or better, `performance.now()` which is monotonic — it only ever goes forward, even if the system clock is changed — and sub-millisecond).
- JWT tokens (JSON Web Tokens — signed login tokens passed between client and server) store expiry as **seconds** since the epoch — multiply by 1000 before passing to `new Date`.

In the interview

30-second answer: "A Date is a wrapper around one number — milliseconds since 1970 UTC. Getters compute local or UTC parts from it. Months are 0-based, setters mutate and roll over, date-only ISO strings parse as UTC. For display I use `Intl.DateTimeFormat`; for serious date math I use Temporal (now standard, shipping in Chrome, Firefox and Node 26) or a library."

- *How do you get the number of days in a month?* `new Date(year, month + 1, 0).getDate()` — day 0 of the next month.
- *How do you check a Date is valid?* `!Number.isNaN(d.getTime())`.

- *Why might `new Date('2024-01-01')` show the wrong day?* It's parsed as UTC midnight, then shown in local time west of UTC.

Self-check

1. `new Date(2025, 11, 25).getMonth()`? → `11` (December).
2. What does `getDay()` return for a Sunday? → `0`.
3. `new Date(2024, 0, 32)` is which date? → 1 February 2024.
4. How many ms in a day? → 86,400,000 (ignoring DST days).
5. Which format should you store? → UTC — ISO 8601 or epoch milliseconds.

5.12 JSON

What it is

JSON (JavaScript Object Notation) is a *text* format for structured data, standardised as ECMA-404 / RFC 8259 (an RFC is a numbered internet standards document). It borrows JavaScript's literal syntax but is much stricter, and it is language-independent — Python, Java, Go and every database speak it. The global `JSON` namespace has two functions:

- `JSON.stringify(value, replacer?, space?)` → turns a value into a JSON **string** (serialisation).
- `JSON.parse(text, reviver?)` → turns a JSON string back into values (deserialisation).

JSON syntax rules (vs JS literals)

Allowed in JSON	Not allowed in JSON (but fine in JS)
Objects `{}` with **double-quoted** keys	Unquoted or single-quoted keys: `{a: 1}`, `{'a': 1}`
Strings in **double quotes**	Single quotes, template literals
Numbers (decimal), `true`, `false`, `null`	`undefined`, `NaN`, `Infinity`, hex `0xFF`, BigInt
Arrays `[]`	Trailing commas `[1, 2,]`
Nesting to any depth	Comments, functions, Dates, Maps, Sets, symbols

How `stringify` works

From `code/08_json.mjs`:

```
const order = {
  id: 42, customer: 'Ravi', total: 19.99, paid: true, coupon: null,
  items: [{ sku: 'A1', qty: 2 }],
  placedAt: new Date('2024-05-01T08:00:00Z'),
};

const text = JSON.stringify(order);
console.log(text);
console.log(typeof text, text.length);
console.log(JSON.stringify({ a: 1, b: [1, 2] }, null, 2));
console.log(JSON.stringify(order, ['id', 'total']));
console.log(JSON.stringify(order, (key, value) => (key === 'customer' ? undefined : value)).slice(0, 60) + '...');
```

Output:

```
{"id":42,"customer":"Ravi","total":19.99,"paid":true,"coupon":null,"items":
[{"sku":"A1","qty":2}],"placedAt":"2024-05-01T08:00:00.000Z"}
string 136
{
  "a": 1,
  "b": [
    1,
    2
  ]
}
{"id":42,"total":19.99}
{"id":42,"total":19.99,"paid":true,"coupon":null,"items":[{"...
```

1. The result is a **string**. The Date became an ISO string because `Date.prototype.toJSON` exists.
2. The third argument `space` (a number of spaces up to 10, or a string such as `'\t'`) pretty-prints.
3. The second argument, the **replacer**, is either an **array** of allowed keys, or a **function** `(key, value) => newValue` called for every property (return `undefined` to drop it).

What JSON silently changes or refuses

```
const weird = {
  f() {}, u: undefined, s: Symbol('x'), nan: NaN, inf: Infinity, negZero: -0,
  map: new Map([['k', 1]]), set: new Set([1]), arr: [undefined, () => 1, NaN],
};
console.log(JSON.stringify(weird));
try { JSON.stringify({ big: 10n }); } catch (e) { console.log('BigInt ->', e.constructor.name); }
const loop = { name: 'me' }; loop.self = loop;
try { JSON.stringify(loop); } catch (e) { console.log('circular ->', e.constructor.name); }

const money = { cents: 1999, toJSON() { return (this.cents / 100).toFixed(2); } };
console.log(JSON.stringify({ price: money }));
```

Output:

```
{"nan":null,"inf":null,"negZero":0,"map":{},"set":{},"arr":[null,null,null]}
BigInt -> TypeError
circular -> TypeError
{"price":"19.99"}
```

Value	In an object property	In an array	At top level
`undefined`, function, symbol	**property dropped**	becomes `null`	`JSON.stringify` returns `undefined`
`NaN`, `Infinity`	`null`	`null`	`"null"`
`-0`	`0`	`0`	`"0"`
`Date`	ISO string (via `toJSON`)	same	same
`Map`, `Set`	`{}` — data lost!	same	same
BigInt	**throws** `TypeError`	throws	throws
Circular reference	**throws** `TypeError`	throws	—
Non-enumerable or symbol keys	skipped	—	—

Any object can define **`toJSON()`** to control its own representation — here a money object serialises as a decimal string. To send a `Map`, convert it first: `Object.fromEntries(map)` or `[...map]`.

How `parse` works

```
const parsed = JSON.parse(text);
console.log(parsed.items[0].qty, typeof parsed.placedAt);          // dates come back as strings
const revived = JSON.parse(text, (key, value) => (key === 'placedAt' ? new Date(value) : value));
console.log(revived.placedAt instanceof Date, revived.placedAt.getUTCHours());

for (const bad of ["{'a': 1}", '{a: 1}', '{"a": 1,}', 'undefined', '']) {
  try { JSON.parse(bad); console.log('ok', bad); } catch (e) { console.log('SyntaxError for',
JSON.stringify(bad)); }
}
console.log(JSON.parse('"just a string"'), JSON.parse('3.5'), JSON.parse('[true,null]'));
console.log(JSON.parse('{"id": 9007199254740993}').id);
```

Output:

```
2 string
true 8
SyntaxError for "{'a': 1}"
SyntaxError for "{a: 1}"
SyntaxError for "{\"a\": 1,}"
SyntaxError for "undefined"
SyntaxError for ""
just a string 3.5 [ true, null ]
9007199254740992
```

- JSON has no date type: dates come back as **strings**. A **reviver** function `(key, value) => newValue` runs on every value (innermost first) and can convert them back.
- Invalid JSON throws a `SyntaxError` — always wrap `JSON.parse` of external input in `try/catch`.
- Any JSON value can be the top level, not only objects.
- Integers above `Number.MAX_SAFE_INTEGER` ($2^{53} - 1$) lose precision: `9007199254740993` became `…992`. APIs such as Twitter/X send large IDs as strings for exactly this reason. The ES2026 feature **"JSON.parse source text access"** gives the reviver a third argument whose `source` property is the original text, so you can rebuild an exact BigInt (Node 24 already supports it):

```
const exact = JSON.parse('{"id": 9007199254740993}', (key, value, ctx) =>
  key === 'id' ? BigInt(ctx.source) : value);
console.log(exact.id);   // 9007199254740993n
```

Deep copy via JSON vs `structuredClone`

```
const src = { when: new Date(0), tags: new Set(['x']), n: 1 };
console.log(JSON.parse(JSON.stringify(src)), structuredClone(src));
```

Output:

```
{ when: '1970-01-01T00:00:00.000Z', tags: {}, n: 1 } { when: 1970-01-01T00:00:00.000Z, tags: Set(1) { 'x' }, n: 1
}
```

The JSON round-trip turned the Date into a string and emptied the Set; `structuredClone` kept both. Use `structuredClone` for copying; use JSON for **transport** (HTTP bodies, files, `localStorage` (the browser's small built-in key-value store), message queues (services that hold messages until another program picks them up — see System Design Fundamentals)).

Real-world examples

- `fetch` responses: `await response.json()` is `JSON.parse` on the body; sending uses `body: JSON.stringify(data)` with `Content-Type: application/json` (Chapter 14).
- `package.json` (a project's settings and dependency list), `tsconfig.json` (TypeScript's settings) and most config files are JSON (tsconfig actually allows comments — it is "JSONC", not strict JSON).
- `localStorage` stores only strings, so objects go through `JSON.stringify` / `JSON.parse`.

- Logging pipelines emit one JSON object per line ("NDJSON") so tools like Elasticsearch (a search engine often used to store and search logs) can index fields (build fast lookup structures for them).

In the interview

> **30-second answer:** "`JSON.stringify` serialises to a string, dropping functions/undefined/symbols in objects, turning NaN/Infinity into null, Dates into ISO strings via `toJSON`, and throwing on BigInt and cycles. `JSON.parse` is strict — double quotes, no trailing commas — and takes a reviver to restore types like Dates."

- *How do you handle circular structures?* Use a replacer with a `WeakSet` (a set of objects that doesn't keep them alive in memory, Chapter 8) of seen objects to drop or mark repeats, or restructure the data; `structuredClone` can *copy* cycles but that doesn't help sending JSON.
- *Why does `JSON.stringify({a: undefined})` give `{}`?* `undefined` properties are omitted.
- *Is JSON a subset of JavaScript?* Since ES2019 ("JSON superset"), yes — every valid JSON text is a valid JS expression.

Self-check

1. `JSON.stringify([undefined])`? → `"[null]"`.
2. `JSON.stringify({ d: new Date(0) })`? → `{"d":"1970-01-01T00:00:00.000Z"}`.
3. Is `{"a":1,}` valid JSON? → No — trailing comma.
4. What's the reviver's job? → Transform parsed values (e.g., strings back into Dates).

Chapter TL;DR

- **Objects** are key → value collections; create with literals `{}`; use **dot** access for fixed identifier keys and **bracket** access for dynamic or unusual keys. Computed keys `[expr]` and shorthand `{ name }` shorten literals.
- Key order is defined: integer-like keys ascending, then string keys by insertion, then symbols.
- Variables hold **references**; `===` compares identity; `const` doesn't freeze contents.
- **Spread** `{...a, ...b}` and **`Object.assign`** are **shallow**; later keys win. **`structuredClone`** deep-copies.
- **Destructuring** unpacks properties with renames (`{ id: orderId }`), defaults (only for `undefined`), nesting, and in parameters.
- **`Object.keys/values/entries/fromEntries`** list and rebuild objects; `hasOwn` checks own keys; `Object.is` fixes `NaN` and `-0`; `Object.create(proto | null)` sets the prototype.
- **Descriptors** (`writable`, `enumerable`, `configurable`, `get`/`set`) control behaviour; `preventExtensions` < `seal` < `freeze`, all shallow.
- **Getters/setters** compute or validate on property access; keep them cheap.
- **`?.`** short-circuits on `null`/`undefined`; pair with **`??`** for defaults.
- **`Math`** is a namespace: `round` (halves toward +∞), `floor`/`ceil`/`trunc`, `max/min(...arr)`, `**`, radians for trig; random int = `Math.floor(Math.random() * (max - min + 1)) + min`; use `crypto` for security.
- **`Date`** = ms since 1970 UTC; months 0-based, `getDay` is weekday, setters mutate and roll over, date-only ISO parses as UTC; format with `Intl`. **Temporal** is standard (Stage 4, March 2026) and shipping in Chrome, Firefox and Node 26, not Node 24/Safari.
- **`JSON.stringify`** drops/changes non-JSON values and throws on BigInt/cycles; **`JSON.parse`** is strict and takes a reviver.

Chapter 6 — Strings and Regular Expressions

Text is everywhere: user names, URLs (web addresses), log lines (one-line records a program writes about what it did), CSV files (comma-separated values: spreadsheet-like data as plain text), chat messages, HTML. This chapter covers how JavaScript represents text (**strings**), the methods you use every day to search, cut, change and compare it, the Unicode details that trip up even experienced developers (why `'😀'.length` is 2), and **regular expressions** — a compact pattern language for finding and transforming text.

What you'll learn

1. Creating strings with single quotes, double quotes and backticks; quotes inside strings; escape sequences.
2. Template literals: interpolation, multi-line strings, tagged templates and `String.raw`.
3. Indexing, `length`, `at()`, immutability, and what happens with out-of-range indexes.
4. String + number coercion and converting values to and from strings.
5. String primitives vs `String` objects (and auto-boxing).
6. The methods table: searching, extracting, case, whitespace, padding, splitting, replacing, comparing.
7. Unicode: code units vs code points vs grapheme clusters, normalization, `Intl.Segmenter`, well-formed strings.
8. Regular expressions: syntax, character classes, quantifiers, anchors, groups, flags, lookaround.
9. Regex methods: `test`, `exec`, `match`, `matchAll`, `replace`/`replaceAll`, `search`, `split` — and the `lastIndex` trap.
10. Modern regex features: named groups, `d`, `u` and `v` flags, `RegExp.escape`, modifiers, duplicate named groups.

Before you start. You need variables and types (Chapter 2) and functions/arrow functions (Chapter 4). Array methods such as `map` and `join` appear in examples; they are explained fully in Chapter 7.

All code is in `code/`; files are ES modules (files using `import`/`export`, which always run in strict mode — the stricter variant of JavaScript from Chapter 1) and outputs are real, from Node 24.

6.1 What a string is

What it is

A **string** is an ordered, **immutable** sequence of characters used to represent text. It is one of JavaScript's **primitive** types: `typeof 'hi'` is `'string'`. "Immutable" means no operation ever changes a string in place — every method returns a *new* string.

More precisely, a JavaScript string is a sequence of **UTF-16 code units** (16-bit numbers). For everyday English text one code unit = one character; section 6.5 explains when that stops being true.

Why it matters

Most data that enters a program arrives as text — form fields, query strings (the `?key=value&…` part of a URL), JSON, files — and must be cleaned, validated, split and reformatted. String handling bugs (trailing spaces, case differences, emoji breaking a length limit) are among the most common bugs in web apps.

6.2 Creating strings

Three kinds of quotes

Delimiter	Example	Interpolation `${}`	Real line breaks allowed	Typical use
Single quotes `'…'`	`'It works'`	No	No	Plain strings (common style default in style guides — shared formatting rulebooks — e.g. Airbnb/StandardJS)
Double quotes `"…"`	`"It works"`	No	No	Plain strings (Prettier's default); identical behaviour to single quotes
Backticks `` `…` ``	`` `Hi ${name}` ``	**Yes**	**Yes**	Anything built from values, multi-line text, tagged templates

Single and double quotes are exactly equivalent; pick one and stay consistent (a formatter — a tool that rewrites code layout automatically — such as Prettier will enforce it — see Chapter 10).

Quotes inside strings

To put a quote character inside a string, either surround the string with a *different* kind of quote or **escape** the inner quote with a backslash.

From `code/01_string_basics.mjs`:

```
console.log("She said 'hi'", '|', 'He said "bye"', '|', 'It\'s escaped', '|', `mix 'both' "kinds"`);
```

Output:

```
She said 'hi' | He said "bye" | It's escaped | mix 'both' "kinds"
```

Escape sequences

A backslash `\` inside a string starts an **escape sequence**: two or more characters in the source that stand for one special character in the value.

Escape	Produces	Notes
`\'` `\"` `\``	The quote character itself	Needed only when it matches the delimiter
`\\`	One backslash	Windows paths: `'C:\\temp'`
`\n`	Line feed (new line)	The normal line break on Linux/macOS and in most text protocols (agreed formats for messages between programs, like HTTP)
`\r`	Carriage return	Windows line endings are `\r\n`
`\t`	Horizontal tab	
`\b`	Backspace (U+0008)	Rare. Careful: inside a **regex** (regular expression, section 6.7), `\b` means "word boundary"
`\f`	Form feed (U+000C)	Rare (old printers)
`\v`	Vertical tab (U+000B)	Rare; supported by all modern engines
`\0`	Null character (U+0000)	Must not be followed by a digit (`'\01'` is a legacy octal — base-8 — escape, a `SyntaxError` in strict mode)
`\xHH`	Character with hex (hexadecimal, base-16: digits 0–9 then A–F) code HH (2 digits)	`'\x41'` is `'A'`
`\uXXXX`	UTF-16 code unit with hex code XXXX (4 digits)	`'\u00e9'` is `'é'`
`\u{X…}`	Any Unicode code point (the number Unicode gives a character, section 6.5; 1–6 hex digits), ES2015	`'\u{1F600}'` is `'😀'`
`\` + line break	Line continuation (no character added)	Old way to split a long literal across lines

```
console.log('Tab:\t|  Backslash: \\  Unicode: \u00e9 \u{1F600}  Hex: \x41');
console.log('Line 1\nLine 2');
console.log(JSON.stringify('a\r\nb\0c'), 'a\r\nb'.length);
```

Output:

```
Tab:    |  Backslash: \  Unicode: é 😀  Hex: A
Line 1
Line 2
"a\r\nb\u0000c" 4
```

`'a\r\nb'.length` is 4: each escape is a single character in the final string, however many characters it took to type.

6.3 Template literals

What it is

A **template literal** (ES2015) is a string written between backticks. It adds three abilities that quoted strings lack:

1. **Interpolation** — `${expression}` inserts the value of *any* JavaScript expression, converted to a string.
2. **Multi-line text** — real line breaks in the source become line breaks in the value.

3. **Tags** — a function name placed before the backtick receives the pieces and values and can build any result.

How it works

```
const item = 'coffee', price = 3.5, qty = 2;
console.log(`${qty} x ${item} = $${(price * qty).toFixed(2)} ${qty > 1 ? '(bulk)' : ''}`);
const multi = `Dear user,
  your order has shipped.`;
console.log(multi);
```

Output:

```
2 x coffee = $7.00 (bulk)
Dear user,
  your order has shipped.
```

- `$${…}`: the first `$` is a literal dollar sign; `${` starts interpolation.
- Any expression is allowed inside `${}`: arithmetic, method calls, ternaries (`cond ? a : b`), even nested templates.
- In a multi-line template, **indentation is part of the string** — the two spaces before "your" are kept.

Before ES2015, multi-line text needed `\n` escapes and `+` concatenation: `'This is line 1\n' + 'This is line 2'`. Templates made that obsolete.

Tagged templates and `String.raw`

A **tag** is a function called with (a) an array of the literal text pieces and (b) the interpolated values as further arguments. Libraries use tags to escape HTML, build SQL (the language for querying databases) safely, or define CSS (styled-components' `styled.div`…``, GraphQL's `gql`…`` (GraphQL is a query language for APIs), lit-html's `html`…``).

```
function highlight(strings, ...values) {
  return strings.reduce((out, s, i) => out + s + (i < values.length ? `[${values[i]}]` : ''), '');
}
console.log(highlight`Hello ${'Ana'}, you have ${3} messages`);
console.log(String.raw`C:\new\table`, '|', 'C:\new\table'.length, String.raw`C:\new\table`.length);
```

Output:

```
Hello [Ana], you have [3] messages
C:\new\table | 10 12
```

`strings` is `['Hello ', ', you have ', ' messages']` and `values` is `['Ana', 3]`: there is always one more string piece than values. The built-in tag `String.raw` returns the text *without* processing escapes — `\n` stays a backslash and an `n` — which is handy for Windows paths and regex sources. The normal string `'C:\new\table'` has only 10 characters because `\n` and `\t` became a newline and a tab.

Common mistakes

- Using `'${name}'` (normal quotes) and wondering why nothing is substituted — only backticks interpolate.
- Building HTML with templates from user input without escaping → **XSS** (cross-site scripting). Escape or use `textContent` (Chapter 14).
- Building SQL by interpolation → SQL injection (an attacker's input gets run as part of your database command). Use parameterised queries (the database receives the command and the values separately, so values are never treated as code).

6.4 Working with strings: indexing, immutability, coercion, String objects

Indexing, `length` and `at()`

Characters are numbered from **0**. Read one with bracket notation `str[i]`, `str.charAt(i)`, or `str.at(i)` (ES2022), which also accepts **negative** indexes counting from the end.

```
const greeting = 'Hello, World!';
console.log(greeting.length, greeting[0], greeting[7], greeting.at(-1), greeting[50], greeting.charAt(50) === '');
```

Output:

```
13 H W ! undefined true
```

- `length` counts code units, including spaces and punctuation: `'Hello, World!'` has 13.
- The first character is at `0`; the last is at `length - 1` (or `at(-1)`).
- An index past the end gives `undefined` with brackets, and an empty string `''` with `charAt`.

Strings are immutable

```
const s = 'cat';
try { s[0] = 'b'; } catch (e) { console.log('assigning s[0] ->', e.constructor.name); }
console.log(s, 'b' + s.slice(1));
```

Output:

```
assigning s[0] -> TypeError
cat bat
```

Assigning to an index never changes a string. In **sloppy** (non-strict) scripts the assignment is silently ignored; in **strict** mode (modules, classes) it throws a `TypeError`. To "change" a string, build a new one.

Concatenation and coercion

The `+` operator means **addition** for two numbers but **concatenation** as soon as either side is a string — the other side is converted to a string first. Other arithmetic operators (`-`, `*`, `/`) convert strings to numbers instead.

```
console.log('The number is ' + 5, '|', '5' + 3, '5' - 3, '5' * '2', 1 + 2 + '3', '1' + 2 + 3);
console.log(String(42), (42).toString(), `${null} ${undefined}`, String([1, 2]), String({}));
```

Output:

```
The number is 5 | 53 2 10 33 123
42 42 null undefined 1,2 [object Object]
```

`+` is evaluated left to right: `1 + 2 + '3'` is `3 + '3'` = `'33'`, but `'1' + 2 + 3` is `'12' + 3` = `'123'`. To convert **to** a string use `String(x)` (safe for `null`/`undefined`) or a template; `x.toString()` throws on `null`/`undefined`. To convert **from** a string see the `Number`/`parseInt` rows in section 6.6. The complete coercion table is in Chapter 2.

String primitives vs `String` objects

JavaScript also has a `String` **wrapper object**, created with `new String('…')`. You essentially never want it.

```
const obj = new String('Hello');
const prim = 'Hello';
console.log(typeof obj, typeof prim, obj === prim, obj == prim, obj.valueOf() === prim);
console.log(new String('a') === new String('a'), typeof String(123));
console.log(prim.toUpperCase(), 'methods work on primitives via temporary wrapper objects');
```

Output:

```
object string false true true
false string
HELLO methods work on primitives via temporary wrapper objects
```

	String primitive `'Hello'`	**String object `new String('Hello')`**
`typeof`	`'string'`	`'object'`
Equality	By value — two equal primitives are `===`	By reference — two wrappers are never `===`, even with the same text
Truthiness	`''` is falsy	Every object is truthy — `new String('')` is truthy!
Cost	Cheap; engines optimise heavily	Extra allocation (reserving memory for a new object)
Methods	Available through **auto-boxing**	Available directly

Auto-boxing: when you call `'Hello'.toUpperCase()`, the engine behaves as if it briefly wrapped the primitive in a `String` object to find the method on `String.prototype` (the shared object where all strings' methods live — prototypes are explained in Chapter 9), then throws the wrapper away. That is why primitives "have methods" without being objects. `String(123)` **without** `new` is a conversion function and returns a primitive.

Comparing strings

```
console.log('apple' < 'banana', 'Zebra' < 'apple', 'a'.localeCompare('B'), ['é', 'z', 'e'].sort(), ['é', 'z',
'e'].sort((a, b) => a.localeCompare(b)));
```

Output:

```
true true -1 [ 'e', 'z', 'é' ] [ 'e', 'é', 'z' ]
```

`<`, `>` and the default `sort()` compare **code unit by code unit**. All uppercase letters come before all lowercase ones (`'Z'` is 90, `'a'` is 97), and accented letters sort after `z`. For human ordering use **`localeCompare`** or an `Intl.Collator` (section 6.6; `Intl` is the built-in internationalisation namespace for language-aware formatting and sorting).

Self-check

1. `'abc'[-1]` vs `'abc'.at(-1)`? → `undefined` vs `'c'`.
2. `'3' + 4 + 5`? → `'345'`. And `3 + 4 + '5'`? → `'75'`.
3. Is `new String('') ? 'y' : 'n'` `'y'` or `'n'`? → `'y'` — objects are truthy.
4. Why is `'B' < 'a'` true? → Code unit 66 < 97.

6.5 Unicode: code units, code points and graphemes

The problem it solves

Why is the length of `'😀'` 2? Why can reversing a string produce garbage? Why are `'café'` and `'café'` sometimes not equal? The answers are all about how text is encoded.

Three levels of "character"

Level	What it is	Example	JavaScript view
Code unit	One 16-bit number in UTF-16	`'😀'` = two units `D83D DE00`	`length`, `str[i]`, `charCodeAt`, `slice` indexes
Code point	One Unicode character number (U+0000 … U+10FFFF)	`'😀'` = U+1F600	`codePointAt`, `String.fromCodePoint`, `for…of`, `[...str]`
Grapheme cluster	What a person sees as one character	`'👨‍👩‍👧'` = 5 code points, 1 grapheme	`Intl.Segmenter`

Code points up to U+FFFF (the **Basic Multilingual Plane**, which covers most living scripts) fit in one code unit. Code points above it — most emoji, rare CJK (Chinese, Japanese, Korean) characters, musical and math symbols — are stored as a **surrogate pair**: two code units in the ranges D800–DBFF ("high") then DC00–DFFF ("low"). A lone half of a pair is a **lone surrogate**, and a string containing one is **not well-formed**.

From `code/03_unicode.mjs`:

```
const accented = 'café';                    // é as one code point U+00E9
const decomposed = 'cafe\u0301';            // e + combining acute accent U+0301
console.log(accented, decomposed, accented === decomposed, accented.length, decomposed.length);
console.log(accented === decomposed.normalize('NFC'), accented.normalize('NFD').length);

const smile = '😀';
console.log(smile.length, smile.charCodeAt(0).toString(16), smile.charCodeAt(1).toString(16));
console.log(smile.codePointAt(0).toString(16), String.fromCodePoint(0x1f600) === smile, '\u{1F600}' === smile);
console.log(JSON.stringify(smile[0]), smile[0].isWellFormed(), smile.isWellFormed());

const word = 'hi😀!';
console.log(word.length, [...word].length, [...word]);
console.log('naive reverse :', word.split('').reverse().join('').isWellFormed());
console.log('safer reverse :', [...word].reverse().join(''));
```

Output:

```
café café false 4 5
true 5
2 d83d de00
1f600 true true
"\ud83d" false true
5 4 [ 'h', 'i', '😀', '!' ]
naive reverse : false
safer reverse : !😀ih
```

What this shows:

1. **Normalization.** "é" can be one code point (U+00E9) or two (`e` + combining accent U+0301). They look identical but are not `===`. `str.normalize('NFC')` composes to the short form; `'NFD'` decomposes. Normalize user input before comparing or storing it (e.g. usernames). `'NFKC'`/`'NFKD'` additionally fold "compatibility" variants such as `fi` → `fi`.
2. **Surrogate pairs.** `'😀'.length` is 2, and `smile[0]` is a lone surrogate. `codePointAt` / `fromCodePoint` / `\u{…}` work with whole code points.
3. **Iteration is by code point.** `for…of`, spread `[...str]` and `Array.from(str)` split correctly into code points; `split('')` splits into code units and can cut pairs in half, producing a broken string when reversed.

Grapheme clusters and `Intl.Segmenter`

Some visible characters are several code points joined together: family emoji (joined with the invisible **zero-width joiner** U+200D), flags (two "regional indicator" letters), skin-tone variants, letters plus combining accents. Even code-point iteration splits these. `Intl.Segmenter` (in all modern browsers and Node) splits text into graphemes, words or sentences according to Unicode rules:

```
const family = '👨‍👩‍👧';
const flag = '🇮🇳';
console.log(family.length, [...family].length, flag.length, [...flag].length);
const seg = new Intl.Segmenter('en', { granularity: 'grapheme' });
const graphemes = (s) => [...seg.segment(s)].map((x) => x.segment);
console.log(graphemes(`a${family}b${flag}`).length, graphemes('cafe\u0301').length);

const words = [...new Intl.Segmenter('en', { granularity: 'word' }).segment('Hello, big world!')]
  .filter((x) => x.isWordLike).map((x) => x.segment);
console.log(words);
```

Output:

```
8 5 4 2
4 4
[ 'Hello', 'big', 'world' ]
```

The family emoji is 8 code units, 5 code points, but **1** grapheme. Use grapheme counts for user-facing limits ("max 280 characters"), cursor movement and truncation.

Well-formed strings and byte length

```
const broken = 'ab\uD800';
console.log(broken.isWellFormed(), broken.toWellFormed().codePointAt(2).toString(16));
console.log(encodeURIComponent(broken.toWellFormed()));
try { encodeURIComponent(broken); } catch (e) { console.log('encodeURIComponent(lone surrogate) ->',
e.constructor.name); }

const enc = new TextEncoder();
for (const s of ['abc', 'café', '😀']) console.log(s, 'length', s.length, 'utf8 bytes', enc.encode(s).length);
```

Output:

```
false fffd
ab%EF%BF%BD
encodeURIComponent(lone surrogate) -> URIError
abc length 3 utf8 bytes 3
café length 4 utf8 bytes 5
😀 length 2 utf8 bytes 4
```

- `isWellFormed()` / `toWellFormed()` (ES2024) detect and repair lone surrogates (replacing each with U+FFFD, the "replacement character" `&#xFFFD;`). APIs that must encode text — `encodeURIComponent` (converts text into the `%XX` form safe inside a URL), `TextEncoder`, `fetch` bodies — either throw or silently substitute on broken strings.
- Files, network payloads (the data carried in a message) and database columns are usually measured in **UTF-8 bytes**, not in `length`. (A **byte** is 8 bits; **UTF-8** is the most common text encoding on the web, using 1–4 bytes per character.) Use `new TextEncoder().encode(s).length` (browser) or `Buffer.byteLength(s)` (Node) when a byte limit matters.

In the interview

30-second answer: "JS strings are UTF-16 code units. `length` and indexing count units, so characters beyond U+FFFF like most emoji count as 2. `for…of` and spread iterate code points; `Intl.Segmenter` gives user-perceived characters. Normalize with `normalize('NFC')` before comparing text."

- *How do you reverse a string safely?* `[...s].reverse().join('')` is correct for code points; for combining marks and ZWJ emoji, reverse the graphemes from `Intl.Segmenter`.
- *How do you count characters for a tweet-like limit?* Count graphemes, not `length`.

Self-check

1. `'😀'.length`? → 2.

2. `[...'😀'].length`? → 1.
3. Why might two identical-looking strings not be equal? → Different normalization forms (composed vs decomposed).
4. Which API gives a word list for text in any language? → `Intl.Segmenter` with `granularity: 'word'`.

6.6 String methods

How to read this section

Every string method **returns a new value** and leaves the original untouched. The table groups them by job; the code afterwards shows each in action. "Index" arguments are code-unit positions; an `end` index is **exclusive** (the character at `end` is not included).

Job	Method	Returns	Notes
Size	`length` (property)	number of code units	not a method — no `()`
Read a char	`charAt(i)`	1-char string, or `''` if out of range	
	`at(i)` (ES2022)	1-char string or `undefined`; negative `i` counts from end	
	`charCodeAt(i)`	UTF-16 code unit number 0–65535	`codePointAt(i)` for full code points
	`String.fromCharCode(...codes)` / `String.fromCodePoint(...cps)`	new string	**static**: called on `String`, not on a string
Search	`indexOf(sub, from?)` / `lastIndexOf(sub, from?)`	index or **-1**	case-sensitive
	`includes(sub, from?)`	boolean	ES2015
	`startsWith(sub, pos?)` / `endsWith(sub, len?)`	boolean	ES2015
	`search(regex)`	index of first regex match or -1	
	`match(regex)` / `matchAll(regex)`	match info / iterator (an object that hands out results one at a time, Chapter 8)	section 6.8
Extract	`slice(start, end?)`	substring; **negative** indexes count from end	the one to use
	`substring(start, end?)`	substring; negatives become 0; swaps args if start > end	older
	`substr(start, length?)`	substring by **length**	legacy (Annex B, the spec's appendix of old web-compatibility features, "normative optional") — avoid
Case	`toUpperCase()` / `toLowerCase()`	new string	locale-independent Unicode rules
	`toLocaleUpperCase(loc?)` / `toLocaleLowerCase(loc?)`	new string	locale-specific rules (a locale is a language + region setting such as `tr-TR`; e.g. Turkish dotted İ)
Whitespace	`trim()` / `trimStart()` / `trimEnd()`	new string	removes spaces, tabs, newlines and other Unicode whitespace
Padding	`padStart(len, fill?)` / `padEnd(len, fill?)`	new string of at least `len`	ES2017
Build	`repeat(n)`	`n` copies joined	
	`concat(...parts)`	joined string	`+` or templates are more common
Split / join	`split(sep?, limit?)`	**array** of pieces	`sep` can be a string or regex; `arr.join(sep)` reverses it
Replace	`replace(pattern, replacement)`	new string	string pattern → first occurrence only

Job	Method	Returns	Notes
	`replaceAll(pattern, replacement)` (ES2021)	new string	every occurrence; a regex must have `g`
Compare	`localeCompare(other, locale?, options?)`	negative / 0 / positive	language-aware ordering
Unicode	`normalize(form?)`, `isWellFormed()`, `toWellFormed()`	string / boolean	section 6.5
Convert	`toString()` / `valueOf()`	the primitive string	mainly for `String` objects

Searching

From `code/02_string_methods.mjs`:

```
const log = 'ERROR 42: disk full; ERROR 7: fan';
console.log(log.indexOf('ERROR'), log.lastIndexOf('ERROR'), log.indexOf('WARN'), log.indexOf('ERROR', 1));
console.log(log.includes('disk'), log.startsWith('ERROR'), log.endsWith('fan'), log.startsWith('42', 6));
console.log(log.search(/\d+/));
```

Output:

```
0 21 -1 21
true true true true
6
```

Use `includes` when you only need yes/no; `indexOf(...) !== -1` is the older idiom. A common bug is `if (s.indexOf(x))` — that is falsy when `x` is found **at index 0** and truthy (−1) when it is missing.

Extracting: `slice` vs `substring` vs `substr`

```
const text = 'Hello, World!';
console.log(text.slice(0, 5), '|', text.slice(7), '|', text.slice(-6, -1));
console.log(text.substring(0, 5), '|', text.substring(5, 0), '|', text.substring(-3, 2));
console.log(text.slice(5, 0) === '', text.substr(7, 5), '(substr: legacy, avoid)');
console.log(text.charAt(4), text.at(-1), text.charCodeAt(0), String.fromCharCode(72, 105));
```

Output:

```
Hello | World! | World
Hello | Hello | He
true World (substr: legacy, avoid)
o ! 72 Hi
```

- `slice(0, 5)` takes indexes 0–4 → `'Hello'`; the end index 5 (the comma) is excluded. Omit `end` to go to the end.
- `slice` understands negative indexes (`-6` = 6 from the end); `substring` turns them into 0 and swaps reversed arguments; `slice` with start > end returns `''`.
- `substr(7, 5)` means "5 characters starting at 7". It's kept only for old code — use `slice`.

Case, whitespace and padding

```
console.log(text.toUpperCase(), text.toLowerCase());
console.log('istanbul'.toLocaleUpperCase('tr-TR'), 'istanbul'.toUpperCase());
console.log('Straße'.toUpperCase(), 'Dž'.toLowerCase());

const raw = '   padded text \n';
console.log(JSON.stringify(raw.trim()), JSON.stringify(raw.trimStart()), JSON.stringify(raw.trimEnd()));
console.log('7'.padStart(3, '0'), 'ab'.padEnd(5, '.') + '|', '4111222233334444'.slice(-4).padStart(16, '*'));
```

Output:

```
HELLO, WORLD! hello, world!
İSTANBUL ISTANBUL
STRASSE ẞ
"padded text" "padded text \n" "   padded text"
007 ab...| ************4444
```

- Case conversion is Unicode-aware and can change length (`ß` → `SS`). Turkish has a dotted capital `İ`, which only the locale-aware version produces.
- For **case-insensitive comparison**, lowercasing both sides works for simple text; for accents and other languages prefer `a.localeCompare(b, undefined, { sensitivity: 'base' }) === 0`.
- `padStart` is perfect for zero-padding (`'7'` → `'007'`) and masking card numbers.

Building, splitting and replacing

```
console.log('ab'.repeat(3), 'a'.concat('b', 'c', 1), ['x', 'y', 'z'].join('-'));
console.log('a,b,,c'.split(','), 'hello'.split(''), 'a b  c'.split(' '), 'a b  c'.split(/\s+/), 'x'.split(','),
'a,b,c'.split(',', 2));
const s = 'cat, cat, cat';
console.log(s.replace('cat', 'dog'), '|', s.replaceAll('cat', 'dog'), '|', s.replace(/cat/g, 'cow'));
```

Output:

```
ababab abc1 x-y-z
[ 'a', 'b', '', 'c' ] [ 'h', 'e', 'l', 'l', 'o' ] [ 'a', 'b', '', 'c' ] [ 'a', 'b', 'c' ] [ 'x' ] [ 'a', 'b' ]
dog, cat, cat | dog, dog, dog | cow, cow, cow
```

- `split` keeps empty strings between consecutive separators (`'a,b,,c'` → 4 pieces); a regex separator like `/\s+/` collapses runs of whitespace. With no match you get a one-element array; the optional second argument limits how many pieces are returned.
- **`replace` with a string pattern changes only the first occurrence** — the source of countless bugs. Use `replaceAll` or a regex with the `g` flag.

Comparing with `localeCompare` and `Intl.Collator`

```
console.log('a'.localeCompare('b'), 'b'.localeCompare('a'), 'a'.localeCompare('a'));
console.log('résumé'.localeCompare('resume', 'en', { sensitivity: 'base' }), 'ä'.localeCompare('z', 'de'),
'ä'.localeCompare('z', 'sv'));
const collator = new Intl.Collator('en', { numeric: true });
console.log(['file10', 'file2', 'file1'].sort(), ['file10', 'file2', 'file1'].sort(collator.compare));
```

Output:

```
-1 1 0
0 -1 1
[ 'file1', 'file10', 'file2' ] [ 'file1', 'file2', 'file10' ]
```

`localeCompare` returns a negative number, 0, or a positive number — exactly what `sort` expects from a comparator (a function that says which of two items comes first). Order depends on the **language**: in German `ä` sorts with `a` (before `z`); in Swedish it's a separate letter after `z`. `sensitivity: 'base'` ignores accents and case. `numeric: true` gives **natural sort** (`file2` before `file10`). When sorting many strings, create one `Intl.Collator` and reuse its `compare` — it is much faster than calling `localeCompare` repeatedly.

Converting strings to numbers

```
const wrapped = new String('boxed');
console.log(wrapped.toString(), wrapped.valueOf(), typeof wrapped.valueOf());
console.log(Number('42px'), parseInt('42px', 10), parseFloat('3.14abc'), Number(''), Number('  7 '));
```

Output:

```
boxed boxed string
NaN 42 3.14 0 7
```

`Number(str)` is strict — the whole string (ignoring surrounding whitespace) must be numeric, and `''` becomes `0`. `parseInt` / `parseFloat` read as many digits as they can and stop. Always pass the radix (the number base, e.g. 10 for decimal) to `parseInt` (`parseInt(s, 10)`).

Two real tasks

```
const slugify = (title) => title.trim().toLowerCase()
  .normalize('NFKD').replace(/[\u0300-\u036f]/g, '')   // strip accents
  .replace(/[^a-z0-9]+/g, '-').replace(/^-+|-+$/g, '');
console.log(slugify('  Crème Brûlée: 10 Tips & Tricks!  '));

const titleCase = (str) => str.split(' ').map((w) => w.charAt(0).toUpperCase() + w.slice(1).toLowerCase()).join('
');
console.log(titleCase('the qUICK brown fox'));
```

Output:

```
creme-brulee-10-tips-tricks
The Quick Brown Fox
```

`slugify` builds URL slugs (the readable last part of a web address, like `my-first-post`) the way blogs and CMSs (content management systems such as WordPress) do: trim, lowercase, decompose accented letters and delete the combining marks (range U+0300–U+036F), turn every run of non-alphanumerics into `-`, then strip leading/trailing dashes.

Performance note

Because strings are immutable, `s += piece` in a loop *conceptually* creates a new string each time. Modern engines optimise this with internal "rope" structures (a tree of string pieces that is only joined into one string when needed), so it's fine for normal sizes; for building very large output, collecting pieces in an array and calling `join('')` once is a safe, predictable alternative.

Common mistakes

- `replace('a', 'b')` expecting every `a` to change.
- `if (str.indexOf(x))` instead of `includes`.
- Comparing user-entered text without `trim()` and case/accent normalization.
- Using `substr` in new code, or confusing `substring(start, end)` with `substr(start, length)`.
- Sorting names with the default `sort()` (uppercase before lowercase, accents last).

In the interview

30-second answer: "Strings are immutable primitives; methods return new strings. I use `slice` for extraction (supports negatives), `includes` / `startsWith` for tests, `split` / `join` to convert to arrays, `replaceAll` or a `/g` regex to replace everywhere, `padStart` for formatting, and `localeCompare` / `Intl.Collator` for human sorting."

- *Check whether a string is a palindrome* (reads the same backwards, like "level"). `const t = s.toLowerCase().replace(/[^a-z0-9]/g, ''); t === [...t].reverse().join('')`.
- *Count occurrences of a substring.* `s.split(sub).length - 1`, or `s.match(/sub/g)?.length ?? 0` (escape the pattern if it comes from input — `RegExp.escape`, section 6.9).
- *Difference between `slice` and `substring`?* Negative indexes and argument swapping.

Self-check

1. `'JavaScript'.slice(-6)`? → `'Script'`.
2. `'a-b-c'.split('-', 2)`? → `['a', 'b']`.

3. `'5'.padStart(3, '0')`? → `'005'`.
4. Sort `['b', 'A', 'a']` case-insensitively. → `.sort((x, y) => x.localeCompare(y, undefined, { sensitivity: 'base' }))`.

6.7 Regular expressions: the pattern language

What it is

A **regular expression** (regex, regexp) is a small pattern that describes a set of strings — "five digits", "a word followed by `@`", "anything between `<` and `>`". JavaScript regexes are `RegExp` objects. You write them in two ways:

- **Literal** `/pattern/flags` — compiled (turned into the engine's internal matching program) once when the script loads; use when the pattern is fixed.
- **Constructor** `new RegExp('pattern', 'flags')` — built from a string at run time; use when the pattern depends on data. Backslashes must be doubled inside the string: `new RegExp('\\d+')`.

Think of a regex as a tiny program that the regex engine runs against your text, trying positions from left to right until the pattern fits.

Why it matters

Validating form input, extracting fields from logs, search-and-replace across files, routing URLs (`/users/:id` becomes a regex inside routers — the code that decides which handler serves which address), syntax highlighting, tokenizers (programs that chop text into meaningful pieces such as words, numbers and symbols) — all use regexes. They are also easy to get wrong, so knowing the core syntax well pays off.

Syntax cheat sheet

Category	Syntax	Matches
Literal characters	`abc`	the text `abc`
Any character	`.`	any char except line terminators (unless the `s` flag)
Escaping	`\.` `\*` `\\` `\/`	the special character itself
Character class	`[abc]`, `[a-z0-9]`	one char from the set / range
Negated class	`[^abc]`	one char **not** in the set
Shorthand classes	`\d` `\w` `\s`	digit `[0-9]`; word char `[A-Za-z0-9_]`; whitespace
Negated shorthands	`\D` `\W` `\S`	the complements
Unicode property	`\p{L}`, `\p{Nd}`, `\p{Script=Greek}` (needs `u` or `v`)	any letter / decimal digit / Greek char
Quantifiers	`*` `+` `?`	0 or more / 1 or more / 0 or 1
Counted	`{n}` `{n,}` `{n,m}`	exactly n / at least n / n to m
Lazy quantifiers	`*?` `+?` `??` `{n,m}?`	as **few** as possible
Anchors	`^` `$`	start / end of input (of each line with `m`)
Word boundary	`\b` `\B`	position between `\w` and `\W` / not a boundary
Alternation	`a	b`	
Capturing group	`( … )`	groups and remembers the match as `$1`, `m[1]`, …
Named group	`(?<name> … )`	remembered as `groups.name` / `$<name>`
Non-capturing group	`(?: … )`	groups without remembering
Backreference	`\1`, `\k<name>`	the same text a group matched earlier
Lookahead	`(?= … )` / `(?! … )`	position followed / not followed by …
Lookbehind (ES2018)	`(?<= … )` / `(?<! … )`	position preceded / not preceded by …

Flags

Flag	Name	Effect
`g`	global	Find **all** matches (affects `match`, `replace`, `matchAll`; makes `test`/`exec` stateful — they remember something between calls)
`i`	ignoreCase	Case-insensitive
`m`	multiline	`^` and `$` match at each line start/end
`s`	dotAll (ES2018)	`.` also matches newlines
`u`	unicode (ES2015)	Work in code points; enables `\p{…}` and `\u{…}`; stricter syntax
`v`	unicodeSets (ES2024)	Superset of `u`: set operations in classes, string properties; use `u` **or** `v`, not both
`y`	sticky	Match only at exactly `lastIndex` (used by tokenizers)
`d`	hasIndices (ES2022)	Add start/end `indices` for each group

6.8 Regex methods and techniques

Which method to use

Goal	Use	Returns
Does it match?	`regex.test(str)`	`true` / `false`
First match with details	`str.match(regexWithoutG)` or `regex.exec(str)`	match array (`[0]` full match, `[1]…` groups, `.index`, `.groups`) or `null`
All matched strings	`str.match(regexWithG)`	array of strings or **`null`**
All matches with details	`str.matchAll(regexWithG)` (ES2020)	iterator of match arrays (throws without `g`)
Position of first match	`str.search(regex)`	index or -1
Replace	`str.replace(regex, replacement)` / `replaceAll`	new string
Split	`str.split(regex)`	array

`test`, `match`, `matchAll` and named groups

From `code/04_regex_basics.mjs`:

```
const literal = /colou?r/i;
const dynamic = new RegExp('colou?r', 'i');
console.log(literal.test('Nice COLOR'), dynamic.source, dynamic.flags, literal.toString());

const zip = /^\d{5}(-\d{4})?$/;
console.log(['12345', '12345-6789', '1234', '12345-'].map((z) => `${z}:${zip.test(z)}`).join(' '));

const text = 'Order #123 shipped, order #456 pending';
const first = text.match(/#(\d+)/);
console.log(first[0], first[1], first.index);
console.log(text.match(/#\d+/g), 'no match ->', text.match(/#x/g));

for (const m of text.matchAll(/#(?<id>\d+) (?<status>\w+)/g)) {
  console.log(m.index, m.groups.id, m.groups.status);
}

const { groups: { year, month, day } } = '2024-07-04'.match(/(?<year>\d{4})-(?<month>\d{2})-(?<day>\d{2})/);
console.log(year, month, day);
```

Output:

```
true colou?r i /colou?r/i
12345:true 12345-6789:true 1234:false 12345-:false
#123 123 6
[ '#123', '#456' ] no match -> null
6 123 shipped
26 456 pending
2024 07 04
```

- `u?` makes the `u` optional, so both spellings match; the `i` flag ignores case.
- The ZIP pattern is **anchored** with `^…$` — without anchors, `test` would accept `'abc12345xyz'` because a match *anywhere* counts. Validation regexes almost always need anchors.
- `match` without `g` gives the full match, groups and position; with `g` it gives only the matched strings. Watch out: **no match returns** `null`, not an empty array — `str.match(re).length` crashes. Use `str.match(re) ?? []` or `matchAll`.
- **Named groups** `(?<year>…)` make results self-documenting and combine nicely with destructuring.

`replace` with patterns and functions

```
console.log('2024-07-04'.replace(/(\d{4})-(\d{2})-(\d{2})/, '$3/$2/$1'));
console.log('2024-07-04'.replace(/(?<y>\d{4})-(?<m>\d{2})-(?<d>\d{2})/, '$<d>.$<m>.$<y>'));
console.log('price: 10 USD, 25 USD'.replace(/(\d+) USD/g, (all, n) => `€${(n * 0.9).toFixed(2)}`));
console.log('snake_case_name'.replace(/_(\w)/g, (_, c) => c.toUpperCase()));
console.log('a.b.c'.replaceAll('.', '/'), 'x'.replace('x', '$&$&'));
console.log('one, two;three  four'.split(/[,;\s]+/), '1+2-3'.split(/([+-])/));
```

Output:

```
04/07/2024
04.07.2024
price: €9.00, €22.50
snakeCaseName
a/b/c xx
[ 'one', 'two', 'three', 'four' ] [ '1', '+', '2', '-', '3' ]
```

Special patterns inside a replacement **string**: `$1`…`$99` (numbered group), `$<name>` (named group), `$&` (whole match), `$`` / `$'` (text before / after the match), `$$` (a literal `$`). They apply even when the search pattern is a plain string — so a replacement containing user text should be passed through a **function** instead. A replacement **function** receives `(match, group1, group2, …, offset, fullString, groups)` and returns the text to insert — the most powerful form. If you split with a regex containing a **capturing group**, the captured separators are included in the result.

Greedy vs lazy, anchors, flags in action

```
const html = '<b>bold</b> and <i>italic</i>';
console.log(html.match(/<.+>/)[0], '|', html.match(/<.+?>/)[0], '|', html.match(/<[^>]+>/g));

console.log(/\bcat\b/.test('concatenate'), /\bcat\b/.test('a cat sat'));
const lines = 'id: 1\nname: Bo\nid: 2';
console.log(lines.match(/^id: \d+$/g), lines.match(/^id: \d+$/gm));

console.log(/start.+end/.test('start\nend'), /start.+end/s.test('start\nend'));
```

Output:

```
<b>bold</b> and <i>italic</i> | <b> | [ '<b>', '</b>', '<i>', '</i>' ]
false true
null [ 'id: 1', 'id: 2' ]
false true
```

- Quantifiers are **greedy** by default: `.+` grabs as much as it can and then backs off, so `<.+>` spans from the first `<` to the *last* `>`. Adding `?` makes it **lazy**. A negated class (`[^>]+`) is usually clearer and faster than a lazy dot.
- `\b` stops `cat` from matching inside `concatenate`.
- Without `m`, `^`/`$` refer to the whole input, so a multi-line text matches nothing; with `m` they work per line.
- `.` doesn't match `\n` unless you add the `s` flag.

(Parsing real HTML with regexes is fragile — use `DOMParser` in the browser or a parser library; a **parser** reads text in a known format and builds a structured representation of it. The example only illustrates greediness.)

Lookaround and backreferences

```
console.log('$30 and €40 and $5'.match(/(?<=\$)\d+/g));        // numbers preceded by $
console.log('100px 2em 30px'.match(/\d+(?=px)/g));             // numbers followed by px
const strong = /^(?=.*[a-z])(?=.*[A-Z])(?=.*\d).{8,}$/;
console.log(['weakpass', 'Str0ngPass', 'Short1A'].map((p) => `${p}:${strong.test(p)}`).join(' '));
console.log('this is is a test test'.match(/\b(\w+) \1\b/g));
```

Output:

```
[ '30', '5' ]
[ '100', '30' ]
weakpass:false Str0ngPass:true Short1A:false
[ 'is is', 'test test' ]
```

Lookarounds check what is before or after a position *without including it* in the match. Stacking lookaheads at the start (`(?=.*[a-z])(?=.*[A-Z])(?=.*\d)`) expresses "must contain each of these" — the classic password-rule pattern. The backreference `\1` means "the same text group 1 captured", which finds doubled words.

The `lastIndex` trap and `exec` loops

```
const g = /a/g;
console.log(g.test('a'), g.lastIndex, g.test('a'), g.lastIndex, g.test('a'));

const re = /\d+/g; let m; const found = [];
while ((m = re.exec('a1b22c333')) !== null) found.push(`${m[0]}@${m.index}`);
console.log(found);

const sticky = /\d+/y; sticky.lastIndex = 1;
console.log(sticky.exec('a12b')?.[0], (sticky.lastIndex = 0, sticky.exec('a12b')));
```

Output:

```
true 1 false 0 true
[ '1@1', '22@3', '333@6' ]
12 null
```

A regex with `g` (or `y`) remembers where its last match ended in its **`lastIndex`** property, and `test`/`exec` start searching from there. Reusing one global regex to `test` several strings therefore gives alternating `true`/`false` results — a notorious bug when a `/g` regex is stored in a constant and used for validation. Fixes: don't use `g` for `test`, or reset `lastIndex = 0`, or use `matchAll` (which copies the regex internally). The `while (exec)` loop is the pre-2020 way to walk all matches with details; `matchAll` replaces it. The **sticky** `y` flag only matches starting exactly at `lastIndex` — a building block for tokenizers.

6.9 Modern regex features and practical recipes

From `code/05_regex_modern.mjs`:

```
console.log(/^.$/.test('😀'), /^.$/u.test('😀'));
console.log('Ünïcödé 123 ñ'.match(/\p{L}+/gu));
console.log('Price: ৳৫ or 34'.match(/\p{Nd}+/gu));
console.log('Hi 👋 there 🎉'.replace(/\p{Extended_Pictographic}/gu, '*'));

console.log('abcdef'.match(/[\p{L}--[aeiou]]/gv));
console.log('I 🇮🇳 flags'.match(/\p{RGI_Emoji_Flag_Sequence}/v)?.[0]);

const userInput = 'a.b*c';
const safe = new RegExp(RegExp.escape(userInput), 'g');
console.log(RegExp.escape(userInput), 'xa.b*cx axbbc'.replace(safe, '[hit]'));
console.log('without escaping:', 'xa.b*cx axbbc'.replace(new RegExp(userInput, 'g'), '[hit]'));

const withIdx = /(?<key>\w+)=(?<val>\w+)/d.exec('  mode=dark');
console.log(withIdx.indices.groups.key, withIdx.indices.groups.val);

const dateRe = /(?<y>\d{4})-(?<m>\d{2})|(?<m>\d{2})\/(?<y>\d{4})/;
console.log('2024-07'.match(dateRe).groups, '07/2024'.match(dateRe).groups);

console.log(/^(?i:hello) World$/.test('HELLO World'), /^(?i:hello) World$/.test('HELLO WORLD'));
```

Output:

```
false true
[ 'Ünïcödé', 'ñ' ]
[ '৳৫', '34' ]
Hi * there *
[ 'b', 'c', 'd', 'f' ]
🇮🇳
\x61\.b\*c x[hit]x axbbc
without escaping: xa.b*cx [hit]
[ 2, 6 ] [ 7, 11 ]
[Object: null prototype] { y: '2024', m: '07' } [Object: null prototype] { y: '2024', m: '07' }
true false
```

Feature	Version	What it gives you
`u` flag + `\p{…}` property escapes	ES2015 / ES2018	Code-point matching; letters, digits, emoji in **any** script (`\w` and `\d` are ASCII-only)
`v` flag	ES2024	Class **set operations** — difference `--`, intersection `&&`, nested classes — and multi-code-point properties like `\p{RGI_Emoji_Flag_Sequence}`
`d` flag	ES2022	`indices` with `[start, end]` for the match and each group — for editors and highlighters
`RegExp.escape(str)`	**ES2025**	Escapes every regex-special character so user text is matched literally. Note it also hex-escapes a leading letter or digit (`\x61`) so the result is safe even when placed after another escape
Duplicate named groups	**ES2025**	The same group name may appear in different `
Pattern modifiers `(?i:…)`	**ES2025**	Turn `i`, `m` or `s` on/off for part of a pattern

All of these run in Node 24 and current evergreen browsers (browsers that update themselves automatically). Without `RegExp.escape`, the user's `.` and `*` were treated as regex operators and matched the wrong text — an injection bug, and with patterns like `(a+)+` a potential **ReDoS** (regular-expression denial of service: a pattern that takes exponential time on certain inputs and freezes the thread).

Practical recipes

```
const logs = `127.0.0.1 - [10/Oct/2024:13:55:36] "GET /index.html HTTP/1.1" 200 2326
10.0.0.5 - [10/Oct/2024:13:56:01] "POST /api/login HTTP/1.1" 401 87`;
const lineRe = /^(?<ip>\S+) - \[(?<time>[^\]]+)\] "(?<method>[A-Z]+) (?<path>\S+) [^"]+" (?<status>\d{3}) (?
<bytes>\d+)$/gm;
console.table([...logs.matchAll(lineRe)].map((m) => ({ ...m.groups, status: Number(m.groups.status) })));

const email = /^[^\s@]+@[^\s@]+\.[^\s@]{2,}$/;
console.log(['a@b.co', 'no-at.com', 'x@y', 'sp ace@a.io'].map((e) => `${e}:${email.test(e)}`).join(' '));

const n = 1234567.891;
console.log(String(Math.trunc(n)).replace(/\B(?=(\d{3})+(?!\d))/g, ','), n.toLocaleString('en-US'),
n.toLocaleString('de-DE'));

const tpl = 'Hi {name}, your code is {code}. {unknown}';
const data = { name: 'Ana', code: 'X9' };
console.log(tpl.replace(/\{(\w+)\}/g, (m, key) => data[key] ?? m));
```

Output:

```
┌─────────┬─────────────┬──────────────────────┬──────────┬──────────────┬──────────┬──────────┐
│ (index) │ ip          │ time                 │ method   │ path         │ status   │ bytes    │
├─────────┼─────────────┼──────────────────────┼──────────┼──────────────┼──────────┼──────────┤
│ 0       │ '127.0.0.1' │ '10/Oct/2024:13:55:36' │ 'GET'  │ '/index.html' │ 200     │ '2326'   │
│ 1       │ '10.0.0.5'  │ '10/Oct/2024:13:56:01' │ 'POST' │ '/api/login'  │ 401     │ '87'     │
└─────────┴─────────────┴──────────────────────┴──────────┴──────────────┴──────────┴──────────┘
a@b.co:true no-at.com:false x@y:false sp ace@a.io:false
1,234,567 1,234,567.891 1.234.567,891
Hi Ana, your code is X9. {unknown}
```

1. **Log parsing**: `gm` flags + named groups + `matchAll` turn each line into an object. Real log pipelines (systems that collect, parse and ship logs from many servers — e.g. Logstash "grok" patterns, Fluent Bit parsers) are built on exactly this idea.
2. **Email**: a *pragmatic* check ("something@something.tld", no spaces; the TLD — top-level domain — is the last part like `.com`). Fully validating RFC 5322 (the official email-format standard) addresses with a regex is impractical; the only real test is sending a confirmation email. HTML's `<input type="email">` uses a similar simplified rule.

3. **Thousands separators**: the lookahead `(?=(\d{3})+(?!\d))` finds positions followed by groups of three digits — a neat trick, but `toLocaleString` / `Intl.NumberFormat` is correct for every locale.
4. **Templating**: a replacer function looks each `{key}` up and leaves unknown keys untouched.

Trade-offs: regex or string methods?

Situation	Prefer
Fixed substring test / replace	`includes`, `startsWith`, `replaceAll` — clearer and faster
Patterns (digits, optional parts, alternatives)	Regex
Structured formats (HTML, JSON, CSV with quotes, URLs)	A real parser: `DOMParser`, `JSON.parse`, a CSV library, `new URL()`
Human-language text (words, characters)	`Intl.Segmenter`, `\p{…}` with `u` / `v`

When not to use it

Avoid giant "do everything" regexes nobody can read; split the job into steps or write a small parser. Never run a user-supplied pattern without a timeout — a limit after which the work is abandoned — (or a linear-time engine such as RE2, whose running time only grows in proportion to the input) — catastrophic backtracking (the engine trying an explosive number of ways to match, see the interview section below) can hang a Node server, which is how several real outages (for example Cloudflare's July 2019 outage) happened.

Common mistakes

- Forgetting anchors in validation (`/\d{5}/.test('abc123456')` is `true`).
- Forgetting `g` in `replace`, or using `matchAll`/`replaceAll` with a non-global regex (both throw a `TypeError`).
- Reusing a `/g` regex with `test` (the `lastIndex` trap).
- Building `new RegExp(userText)` without `RegExp.escape`.
- Writing `'\d'` instead of `'\\d'` in the `RegExp` constructor string.
- Expecting `\w` to match `é` or `ñ` — it is ASCII-only; use `\p{L}` with `u`.

Real-world examples

- **Routers**: Express (the most popular Node web-server framework) and many front-end routers compile `/users/:id` into a regex with a capture group.
- **Linters and formatters** use regexes for simple rules; editors use the `d` flag's indices for highlighting.
- **Form validation** with the HTML `pattern` attribute is a regex (evaluated with the `v` flag in modern browsers).
- **Log processing** and **search tools** (ripgrep, VS Code search) are built on regex engines.

In the interview

> **30-second answer:** "A regex is a pattern object — literal `/…/flags` or `new RegExp`. I use `test` for yes/no, `matchAll` with named groups to extract, and `replace` with a callback to transform. Key gotchas: anchors for validation, `g` needed for global replace, stateful `lastIndex` with `g`, greedy vs lazy quantifiers, and escaping user input with `RegExp.escape`."

- *What's the difference between `match` and `matchAll`?* `match` with `g` returns only the strings (or `null`); `matchAll` returns an iterator of full match objects with groups and indexes.
- *How do you match a word in any language?* `/\p{L}+/u`.
- *What is catastrophic backtracking?* Nested quantifiers like `(a+)+$` try exponentially many ways to split the input when it almost matches; avoid nested unbounded quantifiers.

Self-check

1. What does `/^\d{3}-\d{4}$/` match? → Exactly three digits, a dash, four digits — e.g. `555-1234`.
2. `'aaa'.replace(/a/, 'b')`? → `'baa'`.
3. How do you refer to named group `year` in a replacement string? → `$<year>`.
4. Why is `/<.+>/` wrong for matching one tag? → Greedy; it stretches to the last `>`. Use `/<[^>]+>/`.
5. Which ES2025 method safely escapes user input for a regex? → `RegExp.escape`.

Chapter TL;DR

- Strings are **immutable UTF-16 primitives**; create them with `'…'`, `"…"` (identical) or backticks (interpolation, multi-line, tags). Escape with `\` (`\n`, `\t`, `\\`, `\'`, `\u{…}`).
- Index with `[i]`, `charAt`, or `at(-1)`; out-of-range brackets give `undefined`. Assigning to an index never changes a string (throws in strict mode).
- `+` concatenates as soon as one side is a string; other arithmetic operators convert to numbers. Avoid `new String` — wrappers are objects compared by reference; primitives get methods via auto-boxing.
- Core methods: `includes`/`startsWith`/`endsWith`/`indexOf`, `slice` (negatives OK), `toUpperCase`/`toLowerCase`, `trim*`, `padStart`/`padEnd`, `repeat`, `split`/`join`, `replace` (first only) / `replaceAll`, `localeCompare`/`Intl.Collator` for human ordering.
- `length` counts code units: emoji are 2. Iterate with `for…of`/spread for code points; use `Intl.Segmenter` for visible characters; `normalize('NFC')` before comparing; `isWellFormed`/`toWellFormed` for lone surrogates.
- Regexes: literals or `new RegExp`; classes, quantifiers (greedy by default, `?` for lazy), anchors, groups (numbered, named, non-capturing), lookaround, backreferences; flags `g i m s u v y d`.
- Methods: `test`, `match` (null on no match), `matchAll` (needs `g`), `replace` with `$1`/`$<name>` or a callback, `split` by regex. Beware `lastIndex` with `/g`.
- ES2024–2025 additions: `v` flag, `RegExp.escape`, pattern modifiers, duplicate named groups — all in Node 24. Prefer string methods for fixed text and real parsers for structured formats.

Chapter 7 — Arrays

An **array** is JavaScript's ordered list: shopping-cart items, search results, rows from a database, pixels in an image, moves in a game. Arrays come with about forty built-in methods, and knowing which ones **change** the array and which ones **return a new one** is the single most useful thing to learn — it prevents a whole class of bugs, especially in React and Redux code (React is a popular library for building user interfaces; Redux is a library for managing an app's shared data, its *state*).

What you'll learn

1. Why arrays exist, and how to create them (literal, `new Array`, `Array.of`, `Array.from`, `fill`).
2. Reading and writing by index, `at()`, `length`, holes (sparse arrays), and arrays as objects.
3. Adding and removing at the ends: `push`, `pop`, `unshift`, `shift` — and their costs.
4. The complete methods table, split into **mutating** and **non-mutating** — including the ES2023 copying methods `toSorted`, `toReversed`, `toSpliced` and `with`.
5. Iteration methods: `forEach`, `map`, `filter`, `reduce`, `find`/`findLast`, `some`/`every`, `flat`/`flatMap`, and chaining them into pipelines.
6. Sorting correctly with comparators (numbers, strings, multiple keys), stability, and shuffling.
7. Spread, rest and array destructuring; shallow vs deep copies; array-likes; everyday recipes.
8. When to use an array vs an object (vs `Map`/`Set`), and a short introduction to typed arrays.

Before you start. You need functions and arrow functions (Chapter 4); callbacks (functions you pass into another function so it can call them) are used everywhere here. Loops (`for`, `for…of`) are in Chapter 3. Object spread/destructuring are in Chapter 5; `Map`, `Set` and the iterator protocol behind `for…of` are in Chapter 8.

All code is in `code/`; outputs are real, from Node 24.

7.1 Why arrays?

What it is

An **array** stores many values, in order, under one name. Each value is an **element**, and each element has a numeric position called its **index**, starting at **0**.

Think of a row of numbered lockers: the row has one name (`fruits`), each locker has a number (0, 1, 2 …), and you open a locker by its number.

The problem it solves

Without arrays you would need one variable per item:

```
const fruit1 = 'apple', fruit2 = 'banana', fruit3 = 'cherry';   // three separate variables...
const fruits = ['apple', 'banana', 'cherry'];                   // ...or one array
```

Separate variables can't be looped over, counted, sorted or passed around as a group, and adding a fourth fruit means changing code. An array handles any number of items with the same code.

How it works

From `code/01_array_basics.mjs`:

```
console.log(fruits, fruits.length, typeof fruits, Array.isArray(fruits), Array.isArray({ length: 0 }));
const mixed = [1, 'two', { three: 3 }, [4], null, () => 5];   // any types, can nest
console.log(mixed.length, mixed[2].three, mixed[3][0], mixed[5]());
```

Output:

```
[ 'apple', 'banana', 'cherry' ] 3 object true false
6 3 4 5
```

- An **array literal** is a comma-separated list inside square brackets `[ ]`.
- `length` is the number of elements (strictly: the highest index + 1).
- Arrays are **objects**: `typeof` says `'object'`. Use `Array.isArray(x)` to test for an array.
- JavaScript arrays can mix types and nest (an array inside an array is how you build a grid or matrix — a table of values in rows and columns). In practice, keep one type per array — it's easier to reason about and engines optimise it better.

7.2 Creating arrays

Way	Example	Result	When to use
Literal	`['red', 'green', 'blue']`	3 elements	Almost always
`new Array(n)` (one number)	`new Array(3)`	length 3, **no elements** (3 empty slots)	Rarely — prefer `Array.from`
`new Array(a, b, …)`	`new Array('a', 'b', 'c')`	those elements	Never needed — use a literal
`Array.of(...items)`	`Array.of(3)`	`[3]`	When a single number must be an element, not a length
`Array.from(iterableOrArrayLike, mapFn?)` (iterable = anything `for…of` can loop over; array-like = see section 7.7)	`Array.from({ length: 4 }, (_, i) => i * i)`	`[0, 1, 4, 9]`	Build from strings, Sets, Maps, NodeLists (lists of page elements returned by DOM queries), `arguments`, or generate by index
`Array(n).fill(v)`	`new Array(3).fill(0)`	`[0, 0, 0]`	Pre-filled with a **primitive**
Spread	`[...set]`, `[...'abc']`	copy of any iterable	Copy or convert
`str.split(sep)`	`'a,b'.split(',')`	`['a', 'b']`	From text

```
console.log(new Array(3), new Array(3).length, new Array('a', 'b', 'c'), new Array(3, 4));
console.log(Array.of(3), Array.of(1, 2, 3));
console.log(Array.from('hey'), Array.from({ length: 4 }, (_, i) => i * i), Array.from(new Set([1, 1, 2])));
console.log(new Array(3).fill(0), Array(2).fill().map(() => []));
```

Output:

```
[ <3 empty items> ] 3 [ 'a', 'b', 'c' ] [ 3, 4 ]
[ 3 ] [ 1, 2, 3 ]
[ 'h', 'e', 'y' ] [ 0, 1, 4, 9 ] [ 1, 2 ]
[ 0, 0, 0 ] [ [], [] ]
```

The `new Array` constructor (a function called with `new` to build an object) is inconsistent: **one** numeric argument means "length", but two arguments mean "these elements". And `new Array(5)` does not contain five `undefined` values — it contains five **empty slots** (holes; section 7.3). `Array.of` and `Array.from` were added in ES2015 to avoid both surprises.

7.3 Reading, changing, adding and removing

Index access, `at()` and `length`

```
const colors = ['red', 'green', 'blue'];
console.log(colors[0], colors[colors.length - 1], colors.at(-1), colors[5]);
colors[2] = 'purple';
console.log(colors);
```

Output:

```
red blue blue undefined
[ 'red', 'green', 'purple' ]
```

- `colors[0]` is the first element; the last is `colors[colors.length - 1]` or, more readably, `colors.at(-1)` (ES2022 — negative indexes count from the end).
- Reading an index that doesn't exist gives `undefined` — not an error.
- Assigning to an existing index replaces that element; the array itself is changed in place.

The four end-methods

```
const nums = [1, 2, 3];
console.log(nums.push(4), nums.unshift(0), nums);          // both return the NEW length
console.log(nums.pop(), nums.shift(), nums);               // both return the REMOVED element
```

Output:

```
4 5 [ 0, 1, 2, 3, 4 ]
4 0 [ 1, 2, 3 ]
```

Method	Where	Returns	Cost
`push(...items)`	adds at the **end**	new length	O(1) amortised (fast on average; the occasional slow resize is spread over many calls)
`pop()`	removes from the **end**	the removed item (`undefined` if empty)	O(1)
`unshift(...items)`	adds at the **start**	new length	**O(n)** — every element shifts up one index
`shift()`	removes from the **start**	the removed item	**O(n)**

(O(1) means the time doesn't grow with the array size; O(n) means it grows in proportion to the number of elements. See Chapter 17 for complexity basics.) So `push`/`pop` make a fast **stack** (last in, first out — like a pile of plates); using `shift` as a **queue** (first in, first out — like a line at a shop) on huge arrays is slow — for big queues use an index pointer or a linked structure (items that each point to the next, so removing the first never moves the others).

`length` is writable, and holes

```
const letters = ['a', 'b', 'c', 'd'];
letters.length = 2;
console.log(letters);
letters[5] = 'f';                                          // writing past the end creates holes
console.log(letters, letters.length, 3 in letters, letters[3]);

const holey = [1, , 3];
const withUndef = [1, undefined, 3];
let visited = 0; holey.forEach(() => visited++);
console.log(holey, visited, withUndef.map((x) => x ?? 'U'), holey.map((x) => x ?? 'U'));
```

Output:

```
[ 'a', 'b' ]
[ 'a', 'b', <3 empty items>, 'f' ] 6 false undefined
[ 1, <1 empty item>, 3 ] 2 [ 1, 'U', 3 ] [ 1, <1 empty item>, 3 ]
```

- Setting `length` smaller **deletes** elements; `arr.length = 0` empties the array in place (every reference sees the empty array).
- Writing beyond the end makes a **sparse array**: the missing positions are **holes** (`3 in letters` is `false`). Reading a hole gives `undefined`, but many methods (`forEach`, `map`, `filter`, `reduce`, `some`, `every`) **skip** holes, while newer ones (`for…of`, `find`, `includes`, `Array.from`, `toSorted`, `with`, …) treat them as `undefined`. Avoid holes — they are confusing and make engines de-optimise the array (switch it to a slower general-purpose storage mode).

Looping over an array

```
for (let i = 0; i < colors.length; i++) process.stdout.write(`${i}:${colors[i]} `);
for (const c of colors) process.stdout.write(c + ' ');
colors.forEach((c, i) => process.stdout.write(`[${i}]${c} `));
for (const [i, c] of colors.entries()) process.stdout.write(`${i}=${c} `);
```

Output (one line per loop):

```
0:red 1:green 2:purple
red green purple
[0]red [1]green [2]purple
0=red 1=green 2=purple
```

Loop	Index?	`break` / `continue` ?	`await` inside works sequentially?	Notes
Classic `for (let i…)`	Yes	Yes	Yes	Most control (step by 2, go backwards)
`for (const x of arr)`	Via `arr.entries()`	Yes	Yes	Cleanest default
`arr.forEach(fn)`	Yes (2nd arg)	**No**	**No** (async callbacks aren't awaited — `await` pauses until a result that arrives later is ready, Chapter 12)	Fine for simple side effects (actions like printing, rather than computing a value)
`for (const i in arr)`	Yes, as **strings**	Yes	Yes	**Avoid** for arrays: iterates keys, includes inherited enumerable ones (loop-visible properties added to `Array.prototype`)

Arrays are references

```
const a = [1, 2]; const b = a; b.push(3);
console.log(typeof a, a, a === b, [1, 2] === [1, 2]);
```

Output:

```
object [ 1, 2, 3 ] true false
```

Like objects (Chapter 5), assigning an array copies the reference; `===` compares identity, not contents; and `const` stops re-assignment but not `push`.

Self-check

1. `[1, 2, 3].push(4)` returns? → `4` (the new length).
2. Last element without `length - 1`? → `arr.at(-1)`.
3. `new Array(2)` vs `Array.of(2)`? → two empty slots vs `[2]`.
4. Why avoid `for…in` on arrays? → It yields string keys, also visits extra and inherited enumerable properties, and isn't meant for ordered iteration — use `for…of`.

7.4 The methods table: mutating vs non-mutating

Why it matters

A **mutating** method changes the array it's called on. That is fine for a local array you own — but if the array is shared (passed into a function, stored in React state, cached), mutation changes it for **everybody** holding a reference. React compares state (the data a component displays) by reference, so mutating an array in place means React may not re-render (redraw that part of the screen) at all. ES2023 added **copying** versions of the classic mutators so you rarely need the old "copy then mutate" dance.

Complete reference

`length` is the only instance **property** you'll use; everything else is a method. ✎ = mutates the original.

Method	✎	What it does / returns	Added
`push(...x)` / `pop()`	✎	Add to / remove from end; new length / removed item	ES1
`unshift(...x)` / `shift()`	✎	Add to / remove from start	ES1
`splice(start, deleteCount?, ...items)`	✎	Remove and/or insert anywhere; returns **removed** items	ES1
`sort(compareFn?)`	✎	Sorts in place; returns the same array	ES1
`reverse()`	✎	Reverses in place; returns the same array	ES1
`fill(value, start?, end?)`	✎	Overwrites a range with one value	ES2015
`copyWithin(target, start, end?)`	✎	Copies a range to another position inside the array	ES2015
`toSorted(compareFn?)`		**New** sorted array	**ES2023**
`toReversed()`		**New** reversed array	**ES2023**
`toSpliced(start, deleteCount?, ...items)`		**New** array with the splice applied	**ES2023**
`with(index, value)`		**New** array with one index replaced (negative OK; out of range → `RangeError`)	**ES2023**
`slice(start?, end?)`		Shallow copy of a range (negatives OK)	ES1
`concat(...arraysOrValues)`		New array joining this with others (flattens one level of array args)	ES1
`map(fn)`		New array of `fn` results, same length	ES5
`filter(fn)`		New array of elements where `fn` is truthy	ES5
`flat(depth = 1)`		New array with nested arrays flattened	ES2019
`flatMap(fn)`		`map` then `flat(1)`, in one pass	ES2019
`forEach(fn)`		Calls `fn` on each element; returns `undefined`	ES5
`reduce(fn, init?)` / `reduceRight(fn, init?)`		Folds (combines step by step) to a single value, left→right / right→left	ES5
`find(fn)` / `findIndex(fn)`		First matching element / its index (`undefined` / −1)	ES2015
`findLast(fn)` / `findLastIndex(fn)`		Same, searching from the end	ES2023
`some(fn)` / `every(fn)`		Does any / do all elements pass? (short-circuit)	ES5
`includes(x, from?)`		Boolean; finds `NaN`	ES2016
`indexOf(x, from?)` / `lastIndexOf(x, from?)`		Index using `===` (cannot find `NaN`), or −1	ES5
`at(i)`		Element at `i`; negatives from the end	ES2022
`join(sep = ',')`		String of elements; `null`/`undefined` → empty	ES1
`toString()` / `toLocaleString()`		Comma-joined string / locale-formatted	ES1
`keys()` / `values()` / `entries()`		Iterators (objects that hand out values one at a time, Chapter 8) over indexes / values / `[index, value]` pairs	ES2015
Static: `Array.isArray(x)`, `Array.of(...)`, `Array.from(x, fn?)`, `Array.fromAsync(x)`		Test / create arrays (`fromAsync` awaits async iterables; ES2026, already in Node 24)	ES5–ES2026

(You may see `toSource()` in old lists; it was a non-standard Firefox-only method and has been removed — ignore it.)

Mutators vs copying twins in action

From `code/02_mutating_vs_copying.mjs` — a small `show(label, returnValue, original)` helper prints what the call returned, then the original array afterwards (the file also demonstrates `reverse`, `fill` and `copyWithin`):

```
let arr = ['a', 'b', 'c', 'd', 'e'];
show('splice(1, 2)', arr.splice(1, 2), arr);                  // remove 2 at index 1
show("splice(1, 0, 'X', 'Y')", arr.splice(1, 0, 'X', 'Y'), arr);  // insert, remove nothing
show("splice(-1, 1, 'Z')", arr.splice(-1, 1, 'Z'), arr);         // replace the last one

arr = [3, 1, 2];
const sorted = arr.sort();
console.log(sorted === arr, arr);

const base = [3, 1, 2];
show('toSorted()', base.toSorted(), base);
show('toReversed()', base.toReversed(), base);
show("toSpliced(1, 1, 'x')", base.toSpliced(1, 1, 'x'), base);
show('with(0, 99)', base.with(0, 99), base);
show('with(-1, 7)', base.with(-1, 7), base);
try { base.with(10, 1); } catch (e) { console.log('with(10, 1) ->', e.constructor.name); }
show('slice(1)', base.slice(1), base);
show('concat([4], 5)', base.concat([4], 5), base);
show('[...base, 4]', [...base, 4], base);
```

Output:

```
splice(1, 2)              ["b","c"]             original: ["a","d","e"]
splice(1, 0, 'X', 'Y')    []                    original: ["a","X","Y","d","e"]
splice(-1, 1, 'Z')        ["e"]                 original: ["a","X","Y","d","Z"]
true [ 1, 2, 3 ]
toSorted()                [1,2,3]               original: [3,1,2]
toReversed()              [2,1,3]               original: [3,1,2]
toSpliced(1, 1, 'x')      [3,"x",2]             original: [3,1,2]
with(0, 99)               [99,1,2]              original: [3,1,2]
with(-1, 7)               [3,1,7]               original: [3,1,2]
with(10, 1) -> RangeError
slice(1)                  [1,2]                 original: [3,1,2]
concat([4], 5)            [3,1,2,4,5]           original: [3,1,2]
[...base, 4]              [3,1,2,4]             original: [3,1,2]
```

`splice` is the do-everything mutator: *where* to start, *how many* to delete, then *what* to insert. `sort` returns **the same array** (`sorted === arr` is `true`) — a frequent source of "I sorted a copy… oh wait". The four ES2023 methods (the "change array by copy" proposal) return a fresh array and leave the original alone. They are supported in all current browsers and Node 20+.

A real bug: a helper that mutates its argument

```
function topTwoBad(scores) { return scores.sort((a, b) => b - a).slice(0, 2); }
function topTwoGood(scores) { return scores.toSorted((a, b) => b - a).slice(0, 2); }
const s1 = [50, 90, 70];
console.log(topTwoBad(s1), 'caller now sees', s1);
const s2 = [50, 90, 70];
console.log(topTwoGood(s2), 'caller still sees', s2);
```

Output:

```
[ 90, 70 ] caller now sees [ 90, 70, 50 ]
[ 90, 70 ] caller still sees [ 50, 90, 70 ]
```

The "bad" version silently reorders the caller's data. Before ES2023 the fix was `[...scores].sort(...)` or `scores.slice().sort(...)`; today `toSorted` says it directly.

Immutable update cheat sheet (React/Redux style)

Goal	Mutating	Non-mutating
Add to end	`arr.push(x)`	`[...arr, x]`
Add to start	`arr.unshift(x)`	`[x, ...arr]`
Insert at `i`	`arr.splice(i, 0, x)`	`arr.toSpliced(i, 0, x)`
Remove at `i`	`arr.splice(i, 1)`	`arr.toSpliced(i, 1)` or `arr.filter((_, j) => j !== i)`
Remove by id	find index + `splice`	`arr.filter((t) => t.id !== id)`
Replace at `i`	`arr[i] = x`	`arr.with(i, x)`
Update item by id	`item.done = true`	`arr.map((t) => (t.id === id ? { ...t, done: true } : t))`
Sort / reverse	`arr.sort(fn)` / `arr.reverse()`	`arr.toSorted(fn)` / `arr.toReversed()`

Common mistakes

- Using `sort`/`reverse`/`splice` on props (the inputs a React component receives from its parent) or state; use the copying twins.
- Confusing `slice` (copy a range, non-mutating) with `splice` (edit in place).
- Expecting `push` to return the array (it returns the new length), so `arr = arr.push(x)` turns `arr` into a number.
- Thinking `const arr = […]` prevents `push`; freeze it with `Object.freeze(arr)` if needed (shallow).

7.5 Iteration methods: map, filter, reduce and friends

What it is

These methods take a **callback** — a function you pass in — and call it for each element with three arguments: `(element, index, array)`. They let you describe *what* to do with each element instead of writing loop bookkeeping. This style is called **declarative** (say what you want) as opposed to **imperative** (say how to loop).

How it works

From `code/03_iteration_methods.mjs`, using this data:

```
const orders = [
  { id: 1, customer: 'Ana', total: 120, status: 'paid' },
  { id: 2, customer: 'Ben', total: 35, status: 'pending' },
  { id: 3, customer: 'Ana', total: 60, status: 'paid' },
  { id: 4, customer: 'Cy', total: 250, status: 'refunded' },
];
```

`forEach`, **`map`**, **`filter`**, **`reduce`**:

```
orders.forEach((o, i) => console.log(i, o.id, o.customer));
console.log(orders.map((o) => o.total));
console.log(['1', '2', '3'].map(Number), ['1', '2', '3'].map(parseInt)); // classic trap
const paid = orders.filter((o) => o.status === 'paid');
console.log(paid.map((o) => o.id));
const revenue = paid.reduce((sum, o) => sum + o.total, 0);
const byCustomer = orders.reduce((acc, o) => { acc[o.customer] = (acc[o.customer] ?? 0) + o.total; return acc; },
{});
console.log(revenue, byCustomer);
try { [].reduce((a, b) => a + b); } catch (e) { console.log('empty reduce without initial ->',
e.constructor.name); }
console.log(['a', 'b', 'c'].reduceRight((acc, x) => acc + x, ''));
```

Output:

```
0 1 Ana
1 2 Ben
2 3 Ana
3 4 Cy
[ 120, 35, 60, 250 ]
[ 1, 2, 3 ] [ 1, NaN, NaN ]
[ 1, 3 ]
180 { Ana: 180, Ben: 35, Cy: 250 }
empty reduce without initial -> TypeError
cba
```

Method	Callback returns	Method returns	Think of it as
`forEach`	ignored	`undefined`	"do something with each"
`map`	the new element	new array, **same length**	"transform each"
`filter`	truthy/falsy	new array, **same or fewer** elements	"keep some"
`reduce`	the new accumulator (the running result carried from one step to the next)	the final accumulator	"combine all into one"

How `reduce` works, step by step, for `sum` over totals `[120, 60]` with initial value `0`:

1. `sum = 0` (the initial value), element 120 → returns 120.
2. `sum = 120`, element 60 → returns 180.
3. No elements left → `reduce` returns 180.

Always pass the initial value. Without it, `reduce` uses the first element as the starting accumulator — which throws on an empty array and gives wrong types when you're building an object.

The `map(parseInt)` trap: `map` passes `(element, index, array)`, and `parseInt` takes `(string, radix)` (radix = the number base, such as 10), so it is called as `parseInt('2', 1)` and `parseInt('3', 2)` — invalid radix and invalid digit → `NaN`. Wrap it: `map((s) => parseInt(s, 10))` or use `map(Number)`.

Chaining into a pipeline:

```
const report = orders
  .filter((o) => o.status !== 'refunded')
  .map((o) => ({ ...o, total: o.total * 1.1 }))
  .toSorted((a, b) => b.total - a.total)
  .map((o) => `${o.id}:${o.total.toFixed(2)}`)
  .join(', ');
console.log(report);
```

Output:

```
1:132.00, 3:66.00, 2:38.50
```

Each step returns a new array, so steps chain naturally and the original `orders` is never modified. Each step also loops over the data once; for normal UI-sized lists (hundreds or thousands) that's irrelevant, but for millions of rows a single `for` loop or lazy iterator helpers (lazy = each value is computed only when it is asked for) (Chapter 8) avoid building intermediate arrays.

Searching:

```
console.log(orders.find((o) => o.total > 100).id, orders.findIndex((o) => o.total > 100));
console.log(orders.findLast((o) => o.customer === 'Ana').id, orders.findLastIndex((o) => o.customer === 'Ana'));
console.log(orders.find((o) => o.total > 1000), orders.findIndex((o) => o.total > 1000));
console.log(orders.some((o) => o.status === 'refunded'), orders.every((o) => o.total > 0), [].every(() => false));
const vals = [1, 2, NaN, 2];
console.log(vals.includes(NaN), vals.indexOf(NaN), vals.indexOf(2), vals.lastIndexOf(2), vals.includes('2'));
```

Output:

```
1 0
3 2
undefined -1
true true true
true -1 1 3 false
```

Question	Method	Not found
Which element matches?	`find` / `findLast`	`undefined`
At what index?	`findIndex` / `findLastIndex`	`-1`
Is this exact value present?	`includes` (handles `NaN`)	`false`
Where is this exact value?	`indexOf` / `lastIndexOf` (strict `===`, never finds `NaN`)	`-1`
Does any element pass?	`some`	`false` (also for an empty array)
Do all elements pass?	`every`	`true` for an empty array ("vacuous truth")

`find`, `some` and `every` **stop early** as soon as the answer is known. Note that `includes` and `indexOf` compare with `===`-style equality, so `includes('2')` doesn't find the number 2, and searching for an *object* only works with the same reference — use `some((o) => o.id === id)` for objects.

Flattening:

```
const nested = [1, [2, [3, [4]]]];
console.log(nested.flat(), nested.flat(2), nested.flat(Infinity));
const sentences = ['hello world', 'good morning'];
console.log(sentences.map((s) => s.split(' ')), sentences.flatMap((s) => s.split(' ')));
console.log([1, 2, 3, 4].flatMap((n) => (n % 2 ? [] : [n, n * 10])));   // filter + map in one pass
```

Output:

```
[ 1, 2, [ 3, [ 4 ] ] ] [ 1, 2, 3, [ 4 ] ] [ 1, 2, 3, 4 ]
[ [ 'hello', 'world' ], [ 'good', 'morning' ] ] [ 'hello', 'world', 'good', 'morning' ]
[ 2, 20, 4, 40 ]
```

`flat(depth)` removes `depth` levels of nesting (default 1; `Infinity` for all). `flatMap` maps each element to an *array* and joins the results one level deep — returning `[]` drops an element, returning several items expands it, so it can filter and map in a single pass.

Iterators, joining, and the callback signature:

```
console.log([...['x', 'y'].keys()], [...['x', 'y'].values()], [...['x', 'y'].entries()]);
console.log([1, [2, 3], null, undefined, 4].join('-'), String([1, [2, 3]]), [1234.5, 99].toLocaleString('de-DE'));
console.log(['a', 'b'].map((el, idx, arr) => `${el}${idx}/${arr.length}`));
let seen = 0;
for (const o of orders) { seen++; if (o.total > 100) break; }
console.log('stopped after', seen);
```

Output:

```
[ 0, 1 ] [ 'x', 'y' ] [ [ 0, 'x' ], [ 1, 'y' ] ]
1-2,3---4 1,2,3 1.234,5,99
[ 'a0/2', 'b1/2' ]
stopped after 1
```

`join` turns `null`/`undefined` into empty strings and nested arrays into their comma-joined form. `forEach` has no way to stop early (a `return` inside the callback only ends that one call); use `for…of` with `break`, or `some`/`find`.

Trade-offs: methods vs loops

	Array methods	`for` / `for…of` loops
Readability	Name the intent (`filter`, `map`)	Generic; intent must be read from the body
Mutation risk	Encourage new arrays	Easy to mutate by accident
Early exit	Only `find` / `some` / `every`	`break` / `return` anywhere
`await` inside	Callbacks are not awaited sequentially	`for…of` + `await` runs in order
Performance	Slight overhead per call; intermediate arrays	Fastest; one pass

Use methods by default; switch to a loop when you need early exit, sequential `await` (Chapter 12), or measured performance.

Real-world examples

- Rendering lists in React: `items.map((item) => <Row key={item.id} {...item} />)` (React book).
- API data cleanup: `rows.filter(isValid).map(normalize)`.
- Totals in a shopping cart: `cart.reduce((sum, i) => sum + i.price * i.qty, 0)`.
- Feature flags (switches that turn a feature on for some users): `user.roles.some((r) => r === 'admin')`.

In the interview

30-second answer: "`map` transforms, `filter` selects, `reduce` folds to one value, `find` returns the first match, `some` / `every` answer yes/no with short-circuiting. They take `(element, index, array)` callbacks and don't mutate. I always give `reduce` an initial value and avoid `forEach` when I need `break` or sequential `await`."

- *Implement `map` with `reduce`.* `arr.reduce((acc, x, i) => { acc.push(fn(x, i, arr)); return acc; }, [])`.
- *Why does `['1','2','3'].map(parseInt)` return `[1, NaN, NaN]`?* The index is passed as the radix.
- *`find` vs `filter`?* `find` returns the first element (or `undefined`) and stops; `filter` always scans everything and returns an array.

Self-check

1. `[1, 2, 3].map((x) => x * 2)`? → `[2, 4, 6]`.
2. `[5, 12, 8].find((x) => x > 6)`? → `12`.
3. `[].every(Boolean)`? → `true`.
4. Sum `[1, 2, 3]` with `reduce`. → `arr.reduce((s, x) => s + x, 0)`.
5. `[[1], [2, [3]]].flat()`? → `[1, 2, [3]]`.

7.6 Sorting correctly

The problem it solves

`sort()` with no arguments converts elements to **strings** and orders them by UTF-16 code units (the 16-bit numbers JavaScript uses to store each character). That's right for simple ASCII (plain English letters and digits) words and **wrong for numbers**.

From `code/04_sorting.mjs`:

```
console.log([10, 9, 1, 100, 25].sort());                  // default: compared as STRINGS
console.log([10, 9, 1, 100, 25].sort((a, b) => a - b));   // ascending numbers
console.log([10, 9, 1, 100, 25].sort((a, b) => b - a));   // descending numbers
console.log([3, undefined, 1, , 2].sort());               // undefined and holes go last
```

Output:

```
[ 1, 10, 100, 25, 9 ]
[ 1, 9, 10, 25, 100 ]
[ 100, 25, 10, 9, 1 ]
[ 1, 2, 3, undefined, <1 empty item> ]
```

How it works — the comparator contract

A **comparator** is a function `(a, b) => number`:

1. Return a **negative** number → `a` comes before `b`.
2. Return a **positive** number → `b` comes before `a`.
3. Return **0** → keep their current relative order.

`a - b` does this for numbers (and for Dates, since subtraction converts them to milliseconds). The comparator must be **consistent** — returning a boolean (`a > b`) is a bug: `false` becomes 0 and `true` becomes 1, never negative, so results depend on the engine's algorithm.

```
console.log('boolean comparator ->', [3, 1, 2].sort((a, b) => a > b));
```

Output:

```
boolean comparator -> [ 3, 1, 2 ]
```

Strings, multiple keys and stability

```
const people = [
  { name: 'Zoe', dept: 'eng', age: 31 },
  { name: 'adam', dept: 'ops', age: 25 },
  { name: 'Émile', dept: 'eng', age: 25 },
  { name: 'Bob', dept: 'ops', age: 31 },
];
console.log(people.toSorted((a, b) => a.age - b.age).map((p) => p.name));
console.log(people.map((p) => p.name).toSorted());                          // code-unit order
console.log(people.map((p) => p.name).toSorted((a, b) => a.localeCompare(b)));  // human order

const byDeptAgeName = (a, b) =>
  a.dept.localeCompare(b.dept) || b.age - a.age || a.name.localeCompare(b.name);
console.log(people.toSorted(byDeptAgeName).map((p) => `${p.dept}/${p.age}/${p.name}`));

const tasks = [['write', 2], ['test', 1], ['deploy', 2], ['plan', 1]];
console.log(tasks.toSorted((a, b) => a[1] - b[1]).map((t) => t[0]));
```

Output:

```
[ 'adam', 'Émile', 'Zoe', 'Bob' ]
[ 'Bob', 'Zoe', 'adam', 'Émile' ]
[ 'adam', 'Bob', 'Émile', 'Zoe' ]
[ 'eng/31/Zoe', 'eng/25/Émile', 'ops/31/Bob', 'ops/25/adam' ]
[ 'test', 'plan', 'write', 'deploy' ]
```

- For strings, use **`localeCompare`** (or a reused `Intl.Collator().compare`) — the default order puts all capitals first and accented letters last (Chapter 6).
- **Multi-key sort**: chain comparisons with `||`. When the first comparison returns 0 (a tie), `||` falls through to the next key. `b.age - a.age` sorts that key descending.
- **Stability**: since ES2019 `sort` must be **stable** — elements that compare equal keep their original order (`test` stays before `plan`, `write` before `deploy`). That lets you sort by a secondary key first and the primary key second.

Computed keys, booleans, dates

```
const words = ['banana', 'Kiwi', 'apple', 'fig'];
const sortedByLength = words
  .map((w) => ({ w, key: w.length }))
  .sort((a, b) => a.key - b.key || a.w.localeCompare(b.w))
  .map((x) => x.w);
console.log(sortedByLength);
console.log([true, false, true].toSorted((a, b) => Number(a) - Number(b)));
const dates = [new Date('2024-03-01'), new Date('2023-12-25'), new Date('2024-01-15')];
console.log(dates.toSorted((a, b) => a - b).map((d) => d.toISOString().slice(0, 10)));
```

Output:

```
[ 'fig', 'Kiwi', 'apple', 'banana' ]
[ false, true, true ]
[ '2023-12-25', '2024-01-15', '2024-03-01' ]
```

When the sort key is expensive to compute (parsing a date string, normalizing text), compute it **once per element** first, sort by the stored key, then strip it — the "decorate–sort–undecorate" pattern (Schwartzian transform). Otherwise the comparator recomputes keys O(n log n) times (a bit more than proportional to *n* — the usual cost of a good sort).

Shuffling

```
function shuffle(arr, rand = Math.random) {
  const a = [...arr];
  for (let i = a.length - 1; i > 0; i--) {
    const j = Math.floor(rand() * (i + 1));
    [a[i], a[j]] = [a[j], a[i]];
  }
  return a;
}
```

This is the **Fisher–Yates** shuffle: walk from the end, swapping each position with a random earlier (or same) position — every ordering is equally likely, in O(n). The popular one-liner `arr.sort(() => Math.random() - 0.5)` is **biased** (the comparator is inconsistent) and slower. (The demo passes a seeded generator — a fake random-number source that always produces the same sequence from the same starting "seed" — so its output is reproducible: `[ 1, 2, 5, 6, 3, 4 ]`.)

Complexity

V8 (Chrome, Node) implements `sort` with TimSort and other engines use merge-sort variants (sorting algorithms that split the list, sort the parts and merge them back): **O(n log n)** comparisons, stable, and fast on partly-sorted data.

In the interview

30-second answer: "Default `sort` compares strings, so numbers need `(a, b) => a - b`. The comparator returns negative, zero or positive; chain keys with `||`; strings use `localeCompare`. `sort` mutates and is stable since ES2019; use `toSorted` to keep the original."

- *Sort objects by `lastName`, then `firstName`?* `(a, b) => a.lastName.localeCompare(b.lastName) || a.firstName.localeCompare(b.firstName)`.
- *Why not shuffle with `sort(() => Math.random() - 0.5)`?* Biased and O(n log n); use Fisher–Yates.

Self-check

1. `[2, 10, 1].sort()`? → `[1, 10, 2]`.
2. Descending numeric comparator? → `(a, b) => b - a`.
3. What does "stable" mean? → Equal elements keep their original relative order.

7.7 Spread, rest and destructuring with arrays

From `code/05_spread_destructuring.mjs`:

```
const a = [1, 2], b = [3, 4];
console.log([...a, ...b], [0, ...a, 2.5, ...b], Math.max(...b), [...'hi'], [...new Set([1, 1, 2])]);

const [first, second, third = 'default', ...rest] = ['x', 'y'];
console.log(first, second, third, rest);
const [, , keep] = ['skip', 'skip', 'take'];
console.log(keep);
let p = 1, q = 2;
[p, q] = [q, p];
console.log(p, q);
const [[r1c1, r1c2], [r2c1]] = [[1, 2], [3, 4]];
console.log(r1c1, r1c2, r2c1);
const { 0: head, length } = ['h', 'e', 'y'];
console.log(head, length);

const minMax = (arr) => [Math.min(...arr), Math.max(...arr)];
const [lo, hi] = minMax([5, 3, 9]);
console.log(lo, hi);
const sum = (...nums) => nums.reduce((s, n) => s + n, 0);
console.log(sum(1, 2, 3), sum(...[10, 20]));
```

Output:

```
[ 1, 2, 3, 4 ] [ 0, 1, 2, 2.5, 3, 4 ] 4 [ 'h', 'i' ] [ 1, 2 ]
x y default []
take
2 1
1 2 3
h 3
6 30
```

- **Spread** `...arr` expands any **iterable** (array, string, Set, Map, generator) into elements — in array literals and in function calls.
- **Array destructuring** assigns by **position**. Defaults apply for `undefined`; commas skip positions; `...rest` gathers the remainder (always an array, possibly empty).
- `[p, q] = [q, p]` swaps two variables.
- Returning an array and destructuring it (`const [lo, hi] = minMax(...)`) is a light way to return multiple values — React's `const [state, setState] = useState(0)` (a *hook* that gives a component a remembered value plus a function to change it) uses exactly this.
- A **rest parameter** `(...nums)` collects any number of arguments into a real array (replacing the old `arguments` object — Chapter 4).

Copies are shallow; the `fill([])` trap

```
const grid = [[0, 0], [0, 0]];
const shallow = [...grid];
const deep = structuredClone(grid);
shallow[0][0] = 9;
console.log(grid[0][0], deep[0][0]);

const bad = Array(3).fill([]);
bad[0].push('x');
const good = Array.from({ length: 3 }, () => []);
good[0].push('x');
console.log(bad, good);
```

Output:

```
9 0
[ [ 'x' ], [ 'x' ], [ 'x' ] ] [ [ 'x' ], [], [] ]
```

`[...arr]`, `slice()`, `concat()` and `Array.from(arr)` copy only the outer array; inner arrays/objects are shared. `structuredClone` copies deeply. `fill(obj)` puts the **same** object into every slot; use `Array.from({ length }, () => newThing)` to create a fresh one per slot — the standard way to build a 2-D grid.

Array-likes

An **array-like** is an object with numeric keys and a `length` but without array methods — the old `arguments` object, DOM `NodeList`/`HTMLCollection`, strings. Convert with `Array.from` (or spread, if it's iterable), or borrow a method with `call`:

```
const arrayLike = { 0: 'a', 1: 'b', length: 2 };
console.log(Array.from(arrayLike), Array.prototype.map.call(arrayLike, (s) => s.toUpperCase()));
```

Output:

```
[ 'a', 'b' ] [ 'A', 'B' ]
```

Everyday recipes

```
const range = (start, end, step = 1) => Array.from({ length: Math.ceil((end - start) / step) }, (_, i) => start + i * step);
const unique = (arr) => [...new Set(arr)];
const chunk = (arr, size) => Array.from({ length: Math.ceil(arr.length / size) }, (_, i) => arr.slice(i * size, i * size + size));
const zip = (xs, ys) => xs.map((x, i) => [x, ys[i]]);
const last = (arr) => arr.at(-1);
const arraysEqual = (x, y) => x.length === y.length && x.every((v, i) => Object.is(v, y[i]));
```

Output of the demo calls:

```
[ 0, 1, 2, 3, 4 ] [ 10, 7, 4, 1 ]
[ 3, 1, 2 ]
[ [ 1, 2 ], [ 3, 4 ], [ 5 ] ]
[ [ 'a', 1 ], [ 'b', 2 ] ]
3 undefined
true true
```

The last `true` is `[1, 2] == '1,2'` — loose equality converts the array to the string `'1,2'`. Never compare arrays with `==`/`===`; compare element by element (as `arraysEqual` does) or, for simple data, compare `JSON.stringify` results. Grouping arrays into objects/Maps (`Object.groupBy`) is in Chapter 8.

7.8 Arrays vs objects (vs Map/Set)

When to pick which

Use…	When	Example
Array	Order matters; you access by position; you iterate, sort, filter	A playlist, search results, a queue of jobs
Object	A fixed set of named fields describing one thing	`{ name, email, age }` — a record
Array of objects	A list of records — the most common shape of real data	API response `[{ id, title }, …]`
Object / Map keyed by id	Frequent lookup by a key	`usersById[id]` — O(1) instead of `find` O(n)
Set	Uniqueness and fast "is it in?" checks	Visited URLs, selected ids

From `code/01_array_basics.mjs`:

```
const fruitList = ['Apple', 'Banana', 'Cherry'];                    // ordered, by position
const fruitDetails = {                                              // lookup by name
  apple: { color: 'red', taste: 'sweet' },
  cherry: { color: 'red', taste: 'tart' },
};
console.log(fruitList[1], fruitDetails.cherry.taste);
const fruitRecords = [                                              // most common in real apps
  { name: 'Apple', color: 'red' }, { name: 'Banana', color: 'yellow' },
];
console.log(fruitRecords.find((f) => f.name === 'Banana').color);
```

Output:

```
Banana tart
yellow
```

A simple **ordered list** of names is an array. Once each fruit carries its own **details** looked up by name, an object keyed by name reads better. Real apps usually hold an **array of objects** and, when lookups by id become frequent, build an index: `Object.fromEntries(list.map((x) => [x.id, x]))` or `new Map(list.map((x) => [x.id, x]))` (Chapter 8).

Common mistakes

- Using `arr.includes(x)` or `arr.find(...)` inside a loop over another big array — O(n²). Build a `Set` or `Map` first.
- Using an object with numeric keys as a list — you lose `length`, order guarantees are subtler, and no array methods.
- Using an array as a dictionary (`arr['name'] = …`) — it works (arrays are objects) but `length` ignores it and methods skip it.

7.9 Typed arrays (a short note)

What it is

A **typed array** is a fixed-length array of raw numbers of one machine type (8-bit unsigned integers, 32-bit floats, …) stored in a contiguous block of memory called an `ArrayBuffer`. Several typed-array **views** can look at the same buffer in different ways. They're used for binary data: files, images, audio, WebGL/WebGPU (browser APIs for GPU graphics), WebSockets (a two-way, always-open connection between browser and server), `fetch` bodies, WebAssembly memory (WebAssembly runs fast compiled code from languages like C or Rust in the browser), and Node's `Buffer` (a `Uint8Array` subclass — a specialised version that inherits all its behaviour).

From `code/06_typed_arrays.mjs`:

```
const buf = new ArrayBuffer(8);                 // 8 raw bytes, all zero
const bytes = new Uint8Array(buf);              // view them as 8 unsigned bytes
const words = new Uint16Array(buf);             // ...or as 4 unsigned 16-bit ints (same memory)
bytes[0] = 0x34; bytes[1] = 0x12;
console.log(bytes, words[0].toString(16), '(little-endian on this machine)');

const u8 = new Uint8Array([255, 256, -1, 3.7]);
const clamped = new Uint8ClampedArray([255, 256, -1, 3.7]);
const i8 = new Int8Array([127, 128]);
console.log(u8, clamped, i8);

const f = new Float64Array([3, 1, 2]);
console.log(f.length, typeof f.push, f.map((x) => x * 2), f.toSorted(), Array.isArray(f));
console.log(Float32Array.of(0.1)[0], new Float16Array([0.1])[0]);

console.log(Array.from(f), [...new TextEncoder().encode('Hi')], new TextDecoder().decode(new Uint8Array([72,
105])));
const view = new DataView(buf);
view.setUint16(2, 0xabcd, false);                                    // big-endian write, e.g. a network protocol
console.log(bytes.slice(2, 4), view.getUint16(2, false).toString(16));
```

Output:

```
Uint8Array(8) [
  52, 18, 0, 0,
   0,  0, 0, 0
] 1234 (little-endian on this machine)
Uint8Array(4) [ 255, 0, 255, 3 ] Uint8ClampedArray(4) [ 255, 255, 0, 4 ] Int8Array(2) [ 127, -128 ]
3 undefined Float64Array(3) [ 6, 2, 4 ] Float64Array(3) [ 1, 2, 3 ] false
0.10000000149011612 0.0999755859375
[ 3, 1, 2 ] [ 72, 105 ] Hi
Uint8Array(2) [ 171, 205 ] abcd
```

Type	Element	Range / precision
`Int8Array` / `Uint8Array` / `Uint8ClampedArray`	1 byte	−128…127 / 0…255 (wraps) / 0…255 (clamps and rounds — used by `<canvas>`, the HTML drawing surface, for pixel data)
`Int16Array` / `Uint16Array`	2 bytes	
`Int32Array` / `Uint32Array`	4 bytes	
`Float16Array` (ES2025) / `Float32Array` / `Float64Array`	2 / 4 / 8 bytes	half / single / double precision
`BigInt64Array` / `BigUint64Array`	8 bytes	64-bit integers as BigInt

Key differences from normal arrays: **fixed length** (no `push`, `pop`, `splice`), every value is **coerced** to the element type (`256` wraps to `0` in a `Uint8Array`; `-1` becomes `255`), and `Array.isArray` is `false`. Byte order (**endianness**) follows the machine for typed arrays; `DataView` lets you choose it explicitly, which binary file formats and network protocols need.

Chapter TL;DR

- Arrays are ordered, zero-indexed lists — and objects (`typeof` → `'object'`; test with `Array.isArray`). Create with literals; use `Array.from({ length }, fn)` / `Array.of` instead of the quirky `new Array`.
- Read with `arr[i]` or `arr.at(-1)`; missing index → `undefined`. `length` is writable; avoid holes.
- `push`/`pop` are O(1) at the end; `unshift`/`shift` are O(n) at the start. `push` returns the new length.
- **Mutators**: `push pop shift unshift splice sort reverse fill copyWithin`. **Copying twins (ES2023)**: `toSorted toReversed toSpliced with`. Non-mutating: `slice concat map filter flat flatMap` and spread. Never mutate shared state/props.
- `map` transforms, `filter` selects, `reduce` folds (always pass the initial value), `find`/`findLast` return the element, `findIndex` the index, `some`/`every` short-circuit, `includes` finds `NaN`,

`flat`/`flatMap` flatten. `forEach` can't `break` or await.

- Default `sort` is by string; pass `(a, b) => a - b` for numbers, `localeCompare` for text, chain keys with `||`. Sort is stable (ES2019). Shuffle with Fisher–Yates.
- Spread copies shallowly; `structuredClone` deep-copies; `Array(n).fill([])` shares one object. Destructure by position, swap with `[a, b] = [b, a]`.
- Arrays for ordered lists, objects for records, Map/Set for lookups and uniqueness. Typed arrays hold fixed-length binary numbers over an `ArrayBuffer`.

Chapter 8 — Maps, Sets, Iterators and Generators

Plain objects and arrays can do a lot, but two jobs come up so often that JavaScript has dedicated collections for them: **looking things up by key** (`Map`) and **keeping values unique** (`Set`). This chapter covers both, their "weak" cousins that don't cause memory leaks (memory that is never freed because something still points at data nobody needs), and then the machinery that makes `for…of`, spread and destructuring work on all of them — the **iteration protocols** — and **generators**, the easiest way to produce your own lazy sequences (sequences whose values are computed only when asked for). We finish with the newest additions: **iterator helpers** (ES2025) and `Object.groupBy` / `Map.groupBy` (ES2024).

What you'll learn

1. `Map`: creating, `set` / `get` / `has` / `delete` / `clear` / `size`, iterating, converting, and when to prefer it over an object.
2. `Set`: uniqueness, methods, iteration, and the ES2025 set-algebra methods (`union`, `intersection`, …).
3. `WeakMap`, `WeakSet`, `WeakRef` and `FinalizationRegistry`: holding objects without keeping them alive.
4. The **iterable** and **iterator** protocols, `Symbol.iterator`, and everything that consumes iterables.
5. **Generators** (`function*`, `yield`, `yield*`), lazy and infinite sequences, two-way communication, async generators.
6. **Iterator helpers** (`map`, `filter`, `take`, `drop`, `flatMap`, `reduce`, `toArray`, …) and `Iterator.from`.
7. **Grouping** with `Object.groupBy` and `Map.groupBy`.

Before you start. You should know objects (Chapter 5), arrays and their callback methods (Chapter 7), and closures (Chapter 4). One example uses a class with `#private` fields, explained in Chapter 9; async generators use promises (objects that stand for a value arriving later), explained in Chapter 12.

All code is in `code/`; outputs are real, from Node 24.

8.1 `Map`

What it is

A `Map` (ES2015) is a collection of **key → value** pairs, like an object used as a dictionary (a lookup table from keys to values), but designed for that job:

- **Keys can be any value** — objects, functions, numbers, `NaN` — not only strings and symbols.
- It **remembers insertion order**, always.
- It knows its own `size`.
- It is **directly iterable** with `for…of`.

Think of a coat-check counter: you hand over *anything* as a ticket (the key) and get back exactly the coat stored under it.

How it works — creating a Map

`new Map()` creates an empty map. Optionally pass an **iterable of `[key, value]` pairs** (for example an array of two-element arrays) to fill it.

From `code/01_map.mjs`:

```
const stock = new Map([
  ['apples', 500],
  ['bananas', 300],
  ['oranges', 200],
]);
console.log(stock, stock.size);
```

Output:

```
Map(3) { 'apples' => 500, 'bananas' => 300, 'oranges' => 200 } 3
```

Properties and methods

Member	What it does	Returns
`size` (property)	Number of entries	number
`set(key, value)`	Adds a new entry or **updates** an existing key	the map itself (so calls chain)
`get(key)`	Looks up a key	the value, or `undefined` if absent
`has(key)`	Checks whether the key exists	boolean
`delete(key)`	Removes one entry	`true` if something was removed, else `false`
`clear()`	Removes all entries	`undefined`
`keys()` / `values()` / `entries()`	Iterators over keys / values / `[key, value]` pairs, in insertion order	iterator
`forEach((value, key, map) => …, thisArg?)`	Calls a function per entry, in insertion order	`undefined`
`[Symbol.iterator]()`	Same as `entries()` — why `for (const [k, v] of map)` works	iterator
`Map.groupBy(items, fn)` (static, ES2024)	Builds a Map of groups (section 8.7)	Map

```
const books = new Map();
books.set('1984', 'George Orwell').set('Dune', 'Frank Herbert');   // set returns the map -> chainable
console.log(books.get('1984'), books.get('Missing'), books.has('Dune'));
books.set('Dune', 'F. Herbert');                                   // same key -> update
console.log(books.size, books.delete('Dune'), books.delete('Dune'), books.size);
```

Output:

```
George Orwell undefined true
2 true false 1
```

Keys of any type

Map compares keys with **SameValueZero**: like `===`, except `NaN` equals `NaN` (and `+0` equals `-0`). Object keys are matched **by reference** — a different object with the same contents is a different key.

```
const m = new Map();
const objKey = { id: 1 };
const fnKey = () => {};
m.set(objKey, 'object as key').set(fnKey, 'function as key').set(1, 'number 1').set('1', 'string 1').set(NaN, 'NaN works');
console.log(m.get(objKey), m.get(fnKey), m.get(1), m.get('1'), m.get(NaN), m.get({ id: 1 }));

const o = {};
o[objKey] = 'x'; o[1] = 'number'; o['1'] = 'string overwrites';
console.log(Object.keys(o), o);
```

Output:

```
object as key function as key number 1 string 1 NaN works undefined
[ '1', '[object Object]' ] { '1': 'string overwrites', '[object Object]': 'x' }
```

With a plain object, every key is converted to a string: the object key becomes the useless `'[object Object]'` (so *all* objects would share one slot), and `1` and `'1'` collide. The Map keeps them distinct.

Iterating and converting

```
for (const [fruit, qty] of stock) console.log(fruit, qty);
console.log([...stock.keys()], [...stock.values()], [...stock.entries()][0]);
stock.forEach((value, key) => process.stdout.write(`${key}=${value} `));

const asObject = Object.fromEntries(stock);
const backToMap = new Map(Object.entries(asObject));
console.log(asObject, backToMap.get('apples'), JSON.stringify(stock), JSON.stringify([...stock]));
```

Output:

```
apples 500
bananas 300
oranges 200
[ 'apples', 'bananas', 'oranges' ] [ 500, 300, 200 ] [ 'apples', 500 ]
apples=500 bananas=300 oranges=200
{ apples: 500, bananas: 300, oranges: 200 } 500 {} [["apples",500],["bananas",300],["oranges",200]]
```

- Note the callback order in `forEach`: **value first, then key** (matching arrays' `(element, index)`).
- `Object.fromEntries(map)` and `new Map(Object.entries(obj))` convert back and forth (only sensible when keys are strings).
- **`JSON.stringify(map)` gives `{}`** — maps are not JSON. Serialize (turn into text for saving or sending) `[...map]` (an array of pairs) or `Object.fromEntries(map)`.

Map vs Object

Feature	Map	Plain object `{}`
Key types	**Any value** (objects, functions, numbers, `NaN`)	Strings and symbols only (others are converted to strings)
Order	Always insertion order	Defined, but integer-like keys come first in ascending order, then strings by insertion (Ch 5)
Size	`map.size`, O(1) (instant, whatever the size)	`Object.keys(obj).length`, O(n) (time grows with the number of keys)
Iteration	Directly iterable (`for…of`)	Via `Object.keys/values/entries`
Accidental keys	None — starts empty	Inherits from `Object.prototype` (`'toString' in {}`); a key named `__proto__` is special. `Object.create(null)` avoids this
Frequent add/delete	Optimised for it	`delete` can make objects slower in engines
JSON	Not directly	Native
Syntax	`map.get(k)` / `map.set(k, v)`	`obj.k` / `obj[k] = v`, literals, destructuring
Best for	Dynamic dictionaries, caches (stores of already-computed results kept for quick reuse), counting, non-string keys	Fixed-shape records (`{ id, name }`), config, JSON data

Real tasks

```
const text = 'the cat and the hat and the bat';
const freq = new Map();
for (const w of text.split(' ')) freq.set(w, (freq.get(w) ?? 0) + 1);
console.log(freq, [...freq].toSorted((a, b) => b[1] - a[1])[0]);

const users = [{ id: 'u1', name: 'Ana' }, { id: 'u2', name: 'Ben' }];
const byId = new Map(users.map((u) => [u.id, u]));
console.log(byId.get('u2').name);
```

Output:

```
Map(5) { 'the' => 3, 'cat' => 1, 'and' => 2, 'hat' => 1, 'bat' => 1 } [ 'the', 3 ]
Ben
```

The **count pattern** `map.set(k, (map.get(k) ?? 0) + 1)` and the **index-by-id pattern** `new Map(list.map((x) => [x.id, x]))` turn O(n) searches into O(1) lookups. They appear constantly in interview problems (Chapter 17).

Because a Map remembers insertion order and supports cheap delete/re-insert, it makes a compact **LRU cache** (least-recently-used: when full, evict the entry unused the longest — how browser caches, CDNs (content delivery networks: servers around the world that keep copies of files close to users — see System Design Fundamentals) and database buffer pools (the database's in-memory copy of recently used disk pages) decide what to throw away):

```
class LRU {
  #max; #map = new Map();
  constructor(max) { this.#max = max; }
  get(k) {
    if (!this.#map.has(k)) return undefined;
    const v = this.#map.get(k); this.#map.delete(k); this.#map.set(k, v); return v;
  }
  set(k, v) {
    this.#map.delete(k); this.#map.set(k, v);
    if (this.#map.size > this.#max) this.#map.delete(this.#map.keys().next().value); // oldest
  }
  keys() { return [...this.#map.keys()]; }
}
const cache = new LRU(2);
cache.set('a', 1); cache.set('b', 2); cache.get('a'); cache.set('c', 3);
console.log(cache.keys());   // [ 'a', 'c' ]  — 'b' was least recently used
```

(The LLD (low-level design) handbook designs a full LRU cache with a doubly linked list — a chain of items where each points to both its neighbours: `../../../lld/`.)

Coming next — upsert (ES2026). `map.getOrInsert(key, default)` and `map.getOrInsertComputed(key, fn)` return the existing value or insert one in a single call (replacing the `has` / `get` / `set` dance). They are in the ES2026 spec but **not in Node 24**, so the demo feature-detects them and prints `false`.

Common mistakes

- Using bracket syntax on a Map: `map['k'] = 1` sets an ordinary *property* on the Map object; `map.get('k')` won't see it and `size` stays 0.
- Expecting `map.get({ id: 1 })` to find an entry stored with a *different* object that looks the same.
- `JSON.stringify(map)` → `{}`.
- Choosing a Map for a small fixed-shape record — an object is simpler and works with JSON and destructuring.

In the interview

30-second answer: "Use a Map for dynamic key–value data: keys can be any type, it keeps insertion order, has `size`, iterates directly, and is optimised for frequent adds and deletes. Use objects for fixed-shape records and JSON. Map keys compare by SameValueZero, so objects are matched by reference."

- *How would you implement an LRU cache in JS?* A Map: on `get`, delete and re-insert; on overflow, delete `map.keys().next().value`.
- *Why not use an object as a hash map* (a key-value table that finds entries by hashing the key)? String-only keys, inherited properties, `__proto__` pitfalls, no `size`, slower churn (frequent adding and removing).

Self-check

1. `new Map([[1, 'a']]).get('1')`? → `undefined` — `1` and `'1'` are different keys.
2. What does `map.set` return? → The map (chainable).
3. Which comes first in `map.forEach`'s callback: key or value? → Value.
4. How do you send a Map as JSON? → `JSON.stringify([...map])` or `Object.fromEntries(map)`.

8.2 `Set`

What it is

A `Set` (ES2015) holds values — primitives or object references — where each value appears **at most once**. Adding a value that is already present does nothing. Like Map, it keeps insertion order, has `size`, and uses **SameValueZero** equality.

Think of a guest list at a door: a name is either on it or not, and writing it twice doesn't add a second guest.

How it works

```
const colors = new Set(['red', 'green', 'blue', 'red']);
console.log(colors, colors.size);

const numbers = new Set();
numbers.add(1).add(2).add(2);                    // add is chainable; duplicates ignored
console.log(numbers.has(1), numbers.has(3), numbers.size, numbers.delete(1), numbers.delete(1), numbers);
numbers.clear();
console.log(numbers.size);
```

Output (from `code/02_set.mjs`):

```
Set(3) { 'red', 'green', 'blue' } 3
true false 2 true false Set(1) { 2 }
0
```

`new Set(iterable)` takes any iterable (array, string, another Set, a Map's keys…) and keeps the first occurrence of each value.

Member	What it does	Returns
`size` (property)	Number of values	number
`add(value)`	Adds a value if not already present	the set (chainable)
`has(value)`	Membership test — **O(1)** on average	boolean
`delete(value)`	Removes a value	`true` if it was present
`clear()`	Removes everything	`undefined`
`values()` / `keys()`	Iterator over values (`keys` is an alias kept for symmetry with Map)	iterator
`entries()`	Iterator of `[value, value]` pairs (again, for Map symmetry)	iterator
`forEach((value, sameValue, set) => …)`	Calls a function per value, insertion order	`undefined`
`union`, `intersection`, `difference`, `symmetricDifference`, `isSubsetOf`, `isSupersetOf`, `isDisjointFrom`	Set algebra (**ES2025**)	new Set / boolean

Iteration and equality

```
const s = new Set([1, 2, 3]);
s.forEach((value) => process.stdout.write(value + ' '));
console.log([...s.values()], [...s.keys()], [...s.entries()]);

const mixed = new Set([NaN, NaN, 0, -0, '1', 1, { a: 1 }, { a: 1 }]);
console.log(mixed.size, mixed);
```

Output:

```
1 2 3
[ 1, 2, 3 ] [ 1, 2, 3 ] [ [ 1, 1 ], [ 2, 2 ], [ 3, 3 ] ]
6 Set(6) { NaN, 0, '1', 1, { a: 1 }, { a: 1 } }
```

`NaN` is stored once and `0`/`-0` count as one value, but `'1'` and `1` are different, and two separate `{ a: 1 }` objects are **both** kept — objects are compared by reference. To deduplicate objects by content, key them by an id in a Map.

Classic uses

```
console.log([...new Set([3, 1, 3, 2, 1])]);           // dedupe an array, keeping first-seen order
const visited = new Set();
for (const url of ['/a', '/b', '/a', '/c', '/b']) {
  if (visited.has(url)) continue;
  visited.add(url);
}
console.log([...visited]);
```

Output:

```
[ 3, 1, 2 ]
[ '/a', '/b', '/c' ]
```

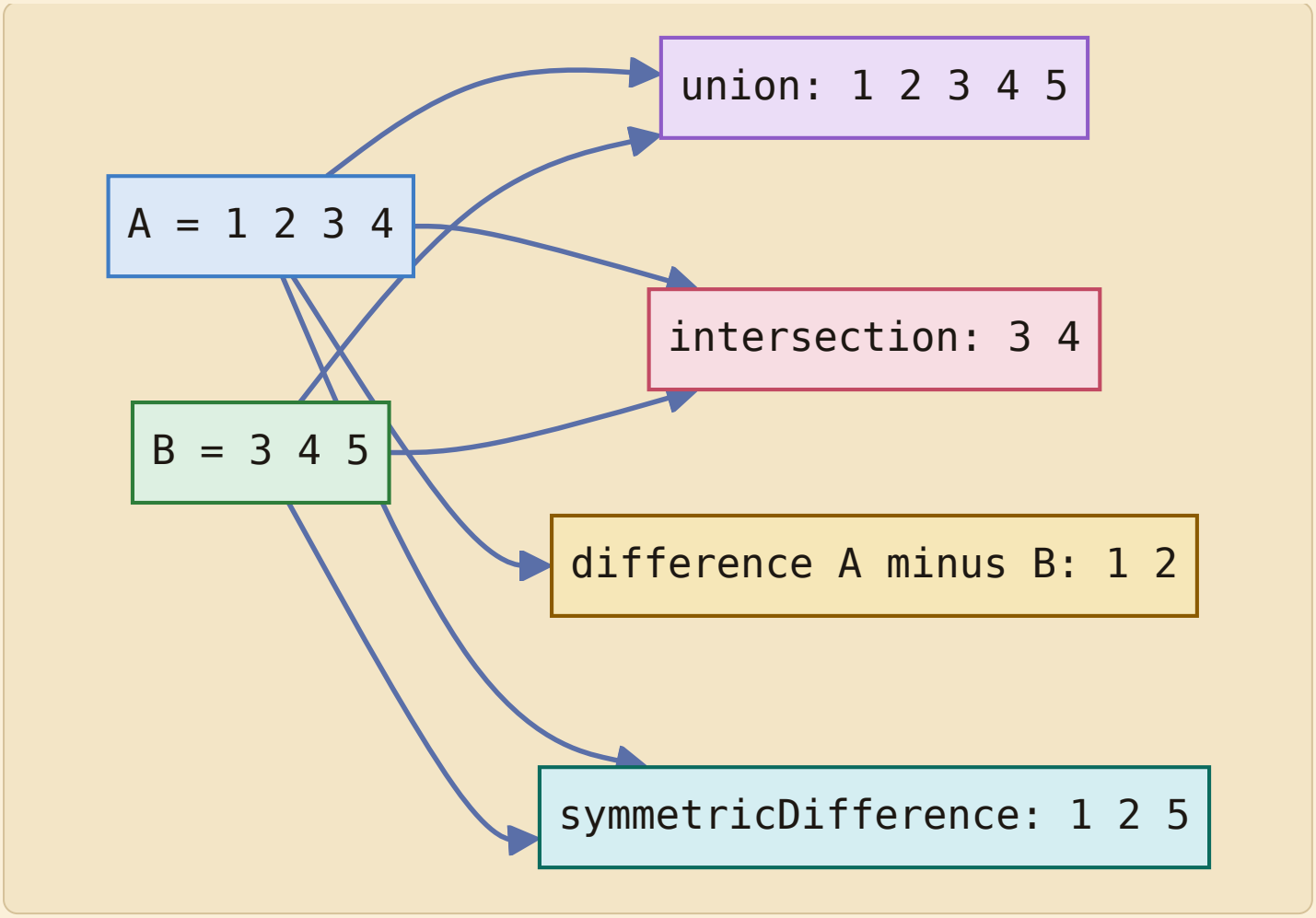
The "visited" set is the heart of web crawlers (programs that follow links to download many pages), graph searches (BFS/DFS — breadth-first and depth-first search, two ways to explore a network of connected nodes) and "don't process this twice" logic.

Set algebra (ES2025)

```
const a = new Set([1, 2, 3, 4]);
const b = new Set([3, 4, 5]);
console.log('union              ', a.union(b));
console.log('intersection       ', a.intersection(b));
console.log('difference a-b     ', a.difference(b));
console.log('symmetricDifference', a.symmetricDifference(b));
console.log('isSubsetOf         ', new Set([3, 4]).isSubsetOf(a), a.isSupersetOf(new Set([1, 2])),
a.isDisjointFrom(new Set([9])));
```

Output:

```
union              Set(5) { 1, 2, 3, 4, 5 }
intersection       Set(2) { 3, 4 }
difference a-b     Set(2) { 1, 2 }
symmetricDifference Set(3) { 1, 2, 5 }
isSubsetOf         true true true
```



Method	Meaning	Returns
`a.union(b)`	In A **or** B	new Set
`a.intersection(b)`	In A **and** B	new Set
`a.difference(b)`	In A but **not** B	new Set
`a.symmetricDifference(b)`	In exactly one of them	new Set
`a.isSubsetOf(b)`	Every element of A is in B	boolean
`a.isSupersetOf(b)`	Every element of B is in A	boolean
`a.isDisjointFrom(b)`	No shared elements	boolean

These methods were standardized in **ES2025** and are available in all current browsers and in **Node 22+**. None of them mutates `a` or `b`. The argument doesn't have to be a Set — any **set-like** object (one with `size`, `has()` and `keys()`) works, such as a Map (its keys are used):

```
const allowed = new Map([['read', true], ['write', false]]);
console.log(new Set(['read', 'delete']).intersection(allowed));   // Set(1) { 'read' }
```

On older runtimes, the classic pattern converts to an array and filters:

```
const setA = new Set([1, 2, 3]);
const setB = new Set([2, 3, 4]);
const intersection = new Set([...setA].filter((x) => setB.has(x)));
console.log(intersection);   // Set(2) { 2, 3 }
```

Similarly, union is `new Set([...a, ...b])` and difference is `new Set([...a].filter((x) => !b.has(x)))`.

Performance: `Set.has` vs `Array.includes`

`array.includes(x)` scans elements one by one — **O(n)**. `set.has(x)` uses hashing (turning the value into a number that points straight at its storage slot) — **O(1)** on average. The demo checks 2,000 lookups against 200,000 numbers and confirms `Set.has` is faster (the exact timing varies by machine, so the script prints only the comparison). Rule of thumb: if you test membership repeatedly, build a Set once.

Common mistakes

- Expecting a Set to deduplicate objects with equal contents.
- Using `set[0]` — Sets have no indexes; convert with `[...set]` or iterate.
- Forgetting that `new Set('hello')` iterates the string: `Set { 'h', 'e', 'l', 'o' }`.
- Calling `JSON.stringify(set)` → `{}`; send `[...set]`.

In the interview

30-second answer: "A Set stores unique values by SameValueZero, keeps insertion order and gives O(1) average `has`. I use it to dedupe (`[...new Set(arr)]`), track visited nodes, and — since ES2025 — for set algebra with `union`, `intersection`, `difference` and friends."

- *Remove duplicates from an array of objects by id?* `[...new Map(arr.map((o) => [o.id, o])).values()]` (keeps the last of each id).
- *Intersection of two arrays in O(n + m)?* `const s = new Set(b); a.filter((x) => s.has(x))`.

Self-check

1. `new Set([1, '1', 1]).size`? → 2.
2. What does `set.add(x)` return? → The set.
3. `new Set([1,2]).union(new Set([2,3]))`? → `Set {1, 2, 3}`.
4. Why is `set.has` better than `arr.includes` in a loop? → O(1) vs O(n) per check.

8.3 `WeakMap`, `WeakSet`, `WeakRef` and `FinalizationRegistry`

What it is

JavaScript frees memory automatically with a **garbage collector** (GC): an object is removed once nothing **reachable** refers to it. A normal Map or Set holds a **strong** reference to its keys/values, so storing an object in a Map keeps it alive forever — a classic memory leak. The **weak** collections hold their keys **weakly**: the entry does not stop the object from being collected, and when it is collected, the entry quietly disappears.

How they work

	`WeakMap`	`WeakSet`	`WeakRef` (ES2021)	`FinalizationRegistry` (ES2021)
Holds	key (weakly) → value	values (weakly)	one target (weakly)	callbacks for targets
Allowed keys/targets	Objects, and non-registered symbols (ES2023)	Same	Same	Same
Methods	`set`, `get`, `has`, `delete`	`add`, `has`, `delete`	`deref()` → target or `undefined`	`register`, `unregister`
`size`, iteration, `clear`	**No**	**No**	—	—

Why no `size` or iteration? Because the contents change whenever the GC runs, which is unpredictable; exposing them would make programs behave differently from run to run.

From `code/03_weak_collections.mjs`:

```
const metadata = new WeakMap();
function track(el) { metadata.set(el, { clicks: 0, firstSeen: 'now' }); }
const button = { tag: 'button' };               // imagine a DOM element
track(button);
metadata.get(button).clicks++;
console.log(metadata.get(button), metadata.has(button), metadata.has({ tag: 'button' }));

const areaCache = new WeakMap();
let computed = 0;
function area(shape) {
  if (!areaCache.has(shape)) { computed++; areaCache.set(shape, shape.w * shape.h); }
  return areaCache.get(shape);
}
const box = { w: 3, h: 4 };
console.log(area(box), area(box), 'computed', computed, 'time(s)');

const processed = new WeakSet();
function processOnce(job) {
  if (processed.has(job)) return 'skipped';
  processed.add(job); return 'processed';
}
const job = { id: 7 };
console.log(processOnce(job), processOnce(job));

try { new WeakMap().set('str', 1); } catch (e) { console.log('primitive key ->', e.constructor.name); }
const wm = new WeakMap();
console.log('size' in wm, typeof wm[Symbol.iterator], typeof wm.keys);
const sym = Symbol('local');
console.log(new WeakMap([[sym, 'symbol key ok (ES2023)']]).get(sym));
try { new WeakSet().add(Symbol.for('registered')); } catch (e) { console.log('registered symbol ->',
e.constructor.name); }
```

Output:

```
{ clicks: 1, firstSeen: 'now' } true false
12 12 computed 1 time(s)
processed skipped
primitive key -> TypeError
false undefined undefined
symbol key ok (ES2023)
registered symbol -> TypeError
```

Three standard uses:

1. **Side tables** — attach extra data to objects you don't own (DOM elements, objects from a library) without adding properties to them. When the element is removed from the page and dropped, its data goes too.
2. **Memoization caches keyed by object** (memoization = remembering the result of a computation so it isn't repeated) — cache a computed result per object; no manual eviction (removing old entries) needed.
3. **Tagging/branding** (marking objects as "seen" or "genuine") with a WeakSet — "have I already processed / validated / created this object?" (This is how some libraries implemented private state before class `#private` fields existed.)

Primitive keys are rejected because primitives are never garbage collected; symbols created with `Symbol.for` (the global registry) are rejected for the same reason.

`WeakRef` and `FinalizationRegistry`

```
let bigObject = { payload: 'x'.repeat(10) };
const ref = new WeakRef(bigObject);
console.log(ref.deref()?.payload.length);        // still reachable -> 10
bigObject = null;                                // after GC, deref() MAY return undefined

const registry = new FinalizationRegistry((heldValue) => console.log('cleaned up', heldValue));
registry.register({ temp: true }, 'temp-object');
```

A `WeakRef` lets you point at an object *without* keeping it alive; `deref()` returns the object or `undefined` once collected. A `FinalizationRegistry` asks to be called back some time after a registered object is collected. Both are advanced tools for caches of large objects and for releasing external resources. Their timing is **unspecified** — the callback may run late or never (for example when the program exits first) — so never put essential logic (saving data, closing files) in them. The TC39 (the committee that designs JavaScript) proposal itself advises avoiding them where possible.

When not to use it

- You need to list, count or iterate the entries → use `Map`/`Set` and remove entries yourself.
- Your keys are strings or numbers → weak collections can't hold them.

In the interview

- *Difference between Map and WeakMap?* WeakMap keys must be objects (or non-registered symbols) and are held weakly, so entries vanish when the key is garbage-collected; as a result it has no `size`, no iteration and no `clear`.
- *Give a real use.* Caching computed data per DOM node or per request object; private per-instance data.

Self-check

1. Can a WeakMap have a string key? → No — `TypeError`.
2. Why can't you iterate a WeakSet? → Its contents depend on unpredictable GC timing.
3. What does `weakRef.deref()` return after the target is collected? → `undefined`.

8.4 Iterators and the iterable protocol

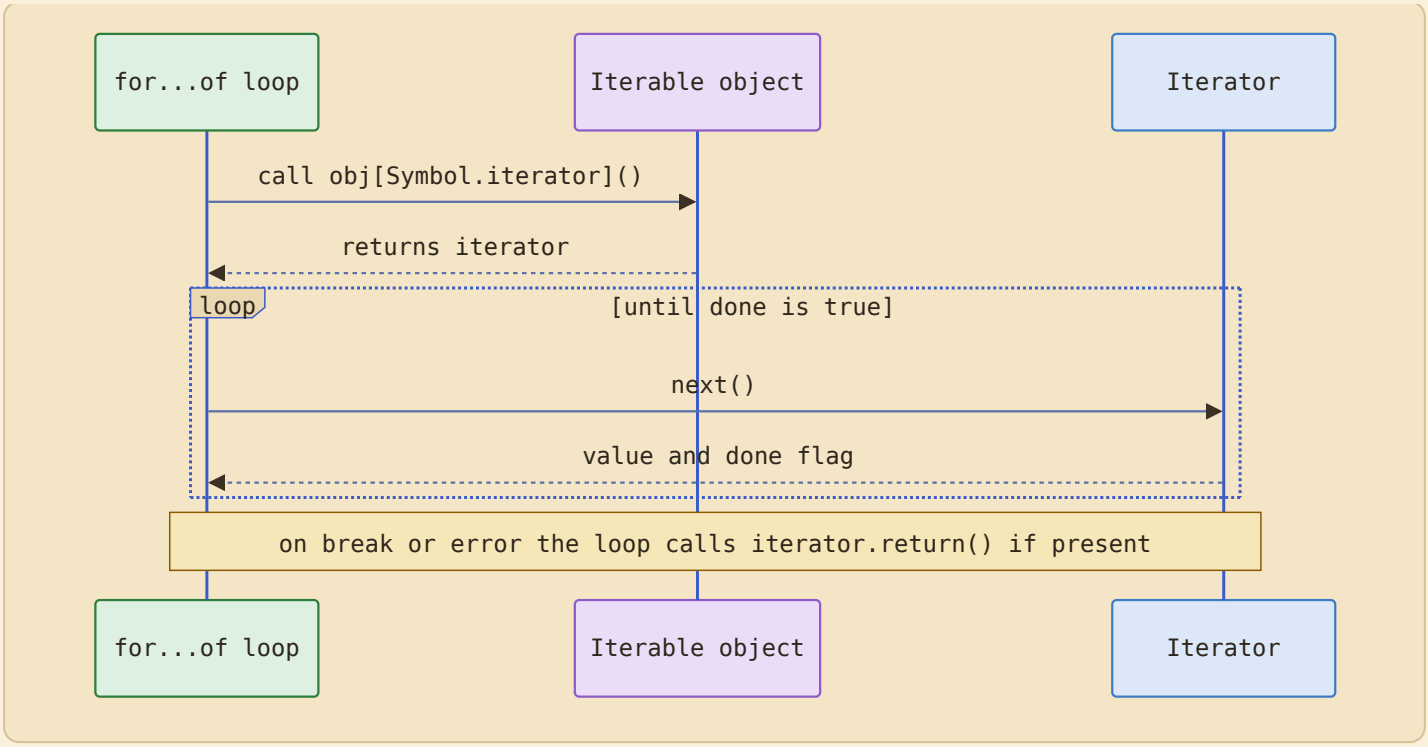
What it is

Two small **protocols** (agreed-upon shapes of objects, ES2015) make `for…of`, spread, destructuring, `Array.from`, `new Map(…)`, `new Set(…)`, `Promise.all` and more work on *any* collection:

- An **iterator** is an object with a `next()` method that hands back a `{ value, done }` result on every call; a result with `done: true` signals the end of the sequence.
- An **iterable** is an object with a method under the key `Symbol.iterator` that returns a fresh iterator.

`Symbol.iterator` is a **well-known symbol**: a built-in unique key the language itself looks for. Using a symbol (instead of a string like `"iterator"`) guarantees it can't clash with ordinary property names.

Think of an iterable as a book and an iterator as a bookmark: you can open the same book many times, each time with a new bookmark that only moves forward.



How it works — driving it by hand

From `code/04_iterators.mjs`:

```
const it = ['a', 'b'][Symbol.iterator]();
console.log(it.next(), it.next(), it.next());

const isIterable = (x) => x != null && typeof x[Symbol.iterator] === 'function';
console.log([['array', [1]], ['string', 'hi'], ['Map', new Map()], ['Set', new Set()], ['object', {}], ['number',
5]]
  .map(([n, v]) => `${n}:${isIterable(v)}`).join(' '));
```

Output:

```
{ value: 'a', done: false } { value: 'b', done: false } { value: undefined, done: true }
array:true string:true Map:true Set:true object:false number:false
```

Built-in iterables: **arrays, strings, Maps, Sets, typed arrays,** `arguments`**, DOM** `NodeList`**s, generator objects**, and the iterators returned by `keys()` / `values()` / `entries()`. Plain objects are **not** iterable — that's why you write `for (const [k, v] of Object.entries(obj))`.

Making your own iterable

```
const countdown = {
  from: 3,
  [Symbol.iterator]() {
    let current = this.from;
    return {
      next: () => (current > 0 ? { value: current--, done: false } : { value: undefined, done: true }),
      [Symbol.iterator]() { return this; },           // iterators are usually iterable too
    };
  },
};
console.log([...countdown], Array.from(countdown), Math.max(...countdown));
for (const n of countdown) process.stdout.write(n + ' ');
console.log('liftoff');
const [first, ...others] = countdown;
console.log(first, others, new Set(countdown).size, new Map([[1, 'x']]).size);
```

Output:

```
[ 3, 2, 1 ] [ 3, 2, 1 ] 3
3 2 1 liftoff
3 [ 2, 1 ] 3 1
```

1. `[Symbol.iterator]()` is a **computed method name** (Chapter 5): the key is the symbol.
2. Each call creates a new iterator with its own `current` (a closure), so the object can be iterated many times.
3. Every consumer — spread, `Array.from`, `Math.max(...)`, `for…of`, destructuring, `new Set` — now works automatically.

Writing `next()` by hand is verbose; a **generator** does the same in two lines (the `Range` class in the demo uses `*[Symbol.iterator]() { … yield v; }` and prints `[ 0, 3, 6, 9 ]` — section 8.5).

Early exit and `return()`

If a consumer stops before `done` (a `break`, a `return`, an exception, or destructuring that takes only the first few values), it calls the iterator's optional **`return()`** method, which is where cleanup (closing a file, releasing a lock — a flag that gives one piece of code exclusive use of a resource) belongs:

```
function resource() {
  let i = 0;
  return {
    [Symbol.iterator]() { return this; },
    next() { i++; return { value: i, done: i > 5 }; },
    return() { console.log('  cleanup after', i, 'items'); return { done: true }; },
  };
}
for (const x of resource()) { if (x === 2) break; }
const [one] = resource();
console.log('first item was', one);
```

Output:

```
  cleanup after 2 items
  cleanup after 1 items
first item was 1
```

Iterators are single-use

```
const arr = [1, 2];
const once = arr.values();
console.log([...once], [...once], [...arr], [...arr]);
```

Output:

```
[ 1, 2 ] [] [ 1, 2 ] [ 1, 2 ]
```

An **iterator** is exhausted after one pass; an **iterable** (the array) hands out a fresh iterator each time. Passing an iterator where code expects to loop twice is a subtle bug.

Everything that consumes iterables

Consumer	Example
`for…of`	`for (const x of set)`
Spread in arrays and calls	`[...map]`, `Math.max(...set)`
Array destructuring	`const [a, b] = iterable`
`Array.from(iterable, mapFn?)`	`Array.from(map.values())`
Collection constructors	`new Map(pairs)`, `new Set(values)`, `new WeakMap(pairs)`
`Object.fromEntries(pairs)`	`Object.fromEntries(map)`
Promise combinators (functions that combine many promises into one)	`Promise.all(iterable)`, `allSettled`, `race`, `any`
`yield*`	inside generators
Grouping	`Object.groupBy(iterable, fn)`

In the interview

30-second answer: "An iterable has a `[Symbol.iterator]()` method returning an iterator; an iterator has `next()` returning `{ value, done }`. `for…of`, spread, destructuring, `Array.from` and the Map/Set constructors all speak this protocol, so implementing it makes any object work with them. Iterators can also define `return()` for cleanup on early exit."

- *Why isn't a plain object iterable?* Its keys, values and entries are all reasonable things to iterate, so the language makes you choose with `Object.keys/values/entries`.
- *`for…in` vs `for…of`?* `for…in` walks enumerable property **keys** (including inherited ones); `for…of` walks the **values** an iterable produces (Chapter 3).

Self-check

1. What must `next()` return? → An object `{ value, done }`.
2. Why does `[...{a: 1}]` throw? → Plain objects are not iterable.
3. When is `return()` called? → When a consumer stops early (break, throw, partial destructuring).

8.5 Generators

What it is

A **generator function** (`function*`, ES2015) is a function that can **pause** in the middle and **resume** later. Calling it doesn't run its body; it returns a **generator object**, which is both an iterator and an iterable. Each `next()` call runs the body until the next `yield`, which hands out a value and pauses right there.

Think of a generator as a TV series rather than a film: calling it gives you the box set, and each `next()` plays one more episode, remembering where you stopped.

How it works

From `code/05_generators.mjs`:

```
function* threeSteps() {
  console.log('  start');
  yield 1;
  console.log('  after first yield');
  yield 2;
  console.log('  after second yield');
  return 3;                                   // return value is NOT included by for...of / spread
}
const gen = threeSteps();                     // nothing runs yet
console.log('created');
console.log(gen.next());
console.log(gen.next());
console.log(gen.next());
console.log(gen.next());
console.log([...threeSteps()]);
```

Output:

```
created
  start
{ value: 1, done: false }
  after first yield
{ value: 2, done: false }
  after second yield
{ value: 3, done: true }
{ value: undefined, done: true }
  start
  after first yield
  after second yield
[ 1, 2 ]
```

Step by step:

1. `threeSteps()` returns a generator object; **no code in the body runs** (`created` prints first).
2. The first `next()` runs until `yield 1` and pauses → `{ value: 1, done: false }`.
3. Each further `next()` resumes right after the last `yield`.
4. `return 3` finishes the generator → `{ value: 3, done: true }`. After that, `next()` always gives `{ value: undefined, done: true }`.
5. `for…of` and spread stop as soon as `done` is `true`, so they **ignore the return value** — `[1, 2]`, not `[1, 2, 3]`.

Syntax variants: `function* name() {}`, `const g = function* () {}`, methods `*name() {}` in objects/classes, and `*[Symbol.iterator]() {}`. There are **no arrow generators**.

Lazy and infinite sequences

Because values are produced only when requested, a generator can describe an **infinite** sequence safely — as long as the consumer stops.

```
function* naturals() { let n = 1; while (true) yield n++; }
function* take(iterable, n) {
  if (n <= 0) return;
  for (const x of iterable) { yield x; if (--n <= 0) return; }
}
console.log([...take(naturals(), 5)]);

function* fibonacci() { let [a, b] = [0, 1]; while (true) { yield a; [a, b] = [b, a + b]; } }
console.log([...take(fibonacci(), 10)]);

function* map(iter, fn) { for (const x of iter) yield fn(x); }
function* filter(iter, pred) { for (const x of iter) if (pred(x)) yield x; }
const firstSquaresOfOdds = take(map(filter(naturals(), (n) => n % 2), (n) => n * n), 4);
console.log([...firstSquaresOfOdds]);
```

Output:

```
[ 1, 2, 3, 4, 5 ]
[
  0, 1,  1,  2,  3,
  5, 8, 13, 21, 34
]
[ 1, 9, 25, 49 ]
```

The pipeline processes **one value at a time** through `filter → map → take` and never builds an intermediate array — the same "lazy evaluation" idea as Python generators, Java streams or Unix pipes. It's how you'd process a huge log file line by line without loading it all into memory. Section 8.6 shows the built-in version.

Delegation with `yield*` and recursion

```
function* inner() { yield 'b'; yield 'c'; return 'inner-done'; }
function* outer() { yield 'a'; const r = yield* inner(); yield r; yield* [1, 2]; }
console.log([...outer()]);

const tree = { value: 1, children: [{ value: 2, children: [{ value: 4, children: [] }] }, { value: 3, children: [] }] };
function* depthFirst(node) { yield node.value; for (const c of node.children) yield* depthFirst(c); }
console.log([...depthFirst(tree)]);
```

Output:

```
[ 'a', 'b', 'c', 'inner-done', 1, 2 ]
[ 1, 2, 4, 3 ]
```

`yield* iterable` yields every value of another iterable (a generator, an array, a string…) as if they were yielded here, and evaluates to that generator's **return value**. Combined with recursion, it makes tree and graph (a network of nodes joined by links) traversal (visiting every node) read almost like the definition: "yield this node, then everything from each child".

Two-way communication: `next(value)`, `return()`, `throw()`

```
function* runningAverage() {
  let sum = 0, count = 0, avg;
  while (true) {
    const value = yield avg;
    sum += value; count++; avg = sum / count;
  }
}
const avg = runningAverage();
avg.next();                                  // prime: run to the first yield
console.log(avg.next(10).value, avg.next(20).value, avg.next(60).value);

function* guarded() {
  try { yield 1; yield 2; } finally { console.log('  finally ran'); }
}
const g1 = guarded(); g1.next();
console.log(g1.return('stopped'), g1.next());
const g2 = guarded(); g2.next();
try { g2.throw(new Error('boom')); } catch (e) { console.log('caught outside:', e.message); }
```

Output:

```
10 15 30
  finally ran
{ value: 'stopped', done: true } { value: undefined, done: true }
  finally ran
caught outside: boom
```

- `gen.next(x)` resumes the generator and makes the paused `yield` expression evaluate to `x`. The first `next()` only runs up to the first `yield` (any argument to it is ignored), so it "primes" the generator.

- `gen.return(v)` finishes the generator as if a `return v` happened at the paused `yield` — `finally` blocks still run.
- `gen.throw(err)` throws `err` at the paused `yield`; if the generator doesn't catch it, it propagates to the caller.

This two-way channel is how libraries like **redux-saga** (a Redux add-on for managing side effects such as network calls) drive effects, and how `async`/`await` was emulated before ES2017 (the `co` library ran generators that yielded promises — see Chapter 12).

More uses: ID makers, state machines, async paging

```
function* idMaker(prefix) { let i = 0; while (true) yield `${prefix}-${++i}`; }
const ids = idMaker('order');
console.log(ids.next().value, ids.next().value);

function* trafficLight() { while (true) { yield 'green'; yield 'yellow'; yield 'red'; } }
const light = trafficLight();
console.log(Array.from({ length: 5 }, () => light.next().value));

async function* fetchPages(totalPages) {
  for (let page = 1; page <= totalPages; page++) {
    await new Promise((r) => setTimeout(r, 5));          // pretend network call
    yield { page, items: [`item${page * 2 - 1}`, `item${page * 2}`] };
  }
}
for await (const { page, items } of fetchPages(3)) console.log('page', page, items);
console.log(await Array.fromAsync(fetchPages(2), (p) => p.page));
```

Output:

```
order-1 order-2
[ 'green', 'yellow', 'red', 'green', 'yellow' ]
page 1 [ 'item1', 'item2' ]
page 2 [ 'item3', 'item4' ]
page 3 [ 'item5', 'item6' ]
[ 1, 2 ]
```

The traffic light is a tiny **state machine** (something that is always in one of a fixed set of states and moves between them by fixed rules). An **async generator** (`async function*`, ES2018) can `await` inside and is consumed with `for await…of`. It's the natural shape for paginated APIs (APIs that return results one page at a time), reading streams (data that arrives in pieces over time) chunk by chunk (Node's readable streams are async iterables — Chapter 15), or polling. `Array.fromAsync` (ES2026, already in Node 24) collects an async iterable into an array.

Trade-offs

	Generator	Array
Memory	One value at a time	All values at once
Infinite sequences	Yes	No
Random access `[i]`, `length`	No	Yes
Reusable	No — create a new generator each time	Yes
Speed for small data	Slightly slower (pause/resume overhead)	Faster

When not to use it

For small, finite data that you'll index, sort or iterate several times, just build an array. Generators shine for large, lazy, infinite or step-by-step data, and for implementing `[Symbol.iterator]`.

Common mistakes

- Calling a generator and expecting its body to run (`gen()` alone does nothing).
- Expecting the `return` value to appear in `for…of` or spread results.
- Reusing an exhausted generator object.
- Spreading an infinite generator without `take` → the program never ends (and runs out of memory).
- Forgetting to prime a generator before sending values with `next(x)`.

In the interview

30-second answer: "A `function*` returns a generator object that's both an iterator and an iterable. `yield` pauses and emits a value; `next()` resumes. That makes lazy and infinite sequences easy, lets you implement `Symbol.iterator` in two lines, and supports two-way communication via `next(x)`, `return` and `throw`. Async generators with `for await` handle paginated or streamed data."

- *Write a `range(start, end)` generator.* `function* range(s, e) { for (let i = s; i < e; i++) yield i; }`
- *What's `yield*`?* Delegation: yields all values from another iterable and evaluates to its return value.
- *How do generators relate to async/await?* Async functions behave like generators that yield promises and are resumed when each settles (finishes, either successfully or with an error).

Self-check

1. What does `[...(function* () { yield 1; return 2; })()]` produce? → `[1]`.
2. What does the first `next()` call do? → Runs to the first `yield` (or the end).
3. Can arrow functions be generators? → No.
4. How do you consume an async generator? → `for await (const x of gen())` or `Array.fromAsync`.

8.6 Iterator helpers (ES2025)

What it is

Iterator helpers add array-like methods — `map`, `filter`, `take`, `drop`, `flatMap`, `reduce`, `toArray`, `forEach`, `some`, `every`, `find` — directly to **iterators** (via a new global `Iterator` class whose prototype every built-in iterator inherits from). Unlike the array methods, the ones that return iterators are **lazy**: nothing is computed until you pull values. Standardized in **ES2025**; available in Node 22+ and all current browsers.

From `code/06_iterator_helpers.mjs`:

```
function* naturals() { let n = 1; while (true) yield n++; }

const result = naturals()
  .filter((n) => n % 3 === 0)
  .map((n) => n * n)
  .take(4)
  .toArray();
console.log(result);

let pulled = 0;
const lazy = naturals().map((n) => { pulled++; return n * 10; }).take(3);
console.log('pulled before use:', pulled);
console.log([...lazy], 'pulled after use:', pulled);

console.log(naturals().drop(5).take(3).toArray());
console.log(['a b', 'c'].values().flatMap((s) => s.split(' ')).toArray());
console.log(naturals().take(5).reduce((sum, n) => sum + n, 0));
console.log(naturals().some((n) => n > 1000), naturals().take(3).every((n) => n < 5), naturals().find((n) => n * n
> 50));

const prices = new Map([['tea', 3], ['cake', 5], ['pie', 7]]);
console.log(prices.entries().filter(([, p]) => p > 4).map(([k]) => k).toArray());

const plain = { i: 0, next() { return this.i < 3 ? { value: this.i++, done: false } : { value: undefined, done:
true }; } };
console.log(Iterator.from(plain).map((x) => x + 100).toArray());

const arr = [1, 2, 3, 4];
console.log(typeof arr.take, arr.values().take(2).toArray());
```

Output:

```
[ 9, 36, 81, 144 ]
pulled before use: 0
[ 10, 20, 30 ] pulled after use: 3
[ 6, 7, 8 ]
[ 'a', 'b', 'c' ]
15
true true 8
[ 'cake', 'pie' ]
[ 100, 101, 102 ]
undefined [ 1, 2 ]
```

Helper	Lazy?	Returns
`map(fn)`, `filter(fn)`, `flatMap(fn)`	Yes	iterator
`take(n)`, `drop(n)`	Yes	iterator
`toArray()`	No — consumes	array
`reduce(fn, init?)`, `forEach(fn)`	No — consumes	value / `undefined`
`some(fn)`, `every(fn)`, `find(fn)`	No — consumes, **short-circuits**	boolean / value
`Iterator.from(obj)` (static)	—	wraps any iterator or iterable so helpers are available

Key points:

- It works on an **infinite** generator because `take(4)` stops pulling once it has four results; `pulled` proves `map` ran only 3 times.
- Map and Set iterators (`entries()`, `values()`) get the helpers too — no need to spread into an array first.
- **Arrays are iterables, not iterators**: `arr.take` is `undefined`. Call `arr.values()` to get the lazy helpers. For ordinary array work, the array methods (Chapter 7) are simpler; use iterator helpers

for large, lazy or infinite data.

- **`Iterator.concat`** (joining several iterators, ES2026) is not in Node 24 — the demo feature-detects it. **Async** iterator helpers are still a separate proposal.

8.7 Grouping: `Object.groupBy` and `Map.groupBy` (ES2024)

What it is

Two static methods that take an iterable and a **key function**, and put each item into the bucket named by the key it returns. Standardized in **ES2024**; in Node 21+ and all current browsers. (They were first proposed as `Array.prototype.group`, but that name broke existing websites, so they became static methods.)

- **`Object.groupBy(items, fn)`** → a **null-prototype object** whose keys are strings (the key function's results converted to strings).
- **`Map.groupBy(items, fn)`** → a **Map**, so keys keep their type — objects, numbers, anything.

From `code/07_groupby.mjs`:

```
const inventory = [
  { name: 'laptop', type: 'electronics', quantity: 4 },
  { name: 'desk', type: 'furniture', quantity: 0 },
  { name: 'monitor', type: 'electronics', quantity: 12 },
  { name: 'chair', type: 'furniture', quantity: 30 },
  { name: 'stapler', type: 'office', quantity: 5 },
];

const byType = Object.groupBy(inventory, (item) => item.type);
console.log(Object.keys(byType), byType.furniture.map((i) => i.name));
console.log(Object.getPrototypeOf(byType) === null);

const byStock = Object.groupBy(inventory, ({ quantity }) => (quantity > 5 ? 'ok' : 'restock'));
console.log(Object.fromEntries(Object.entries(byStock).map(([k, v]) => [k, v.map((i) => i.name)])));

const restock = { label: 'restock' }, sufficient = { label: 'sufficient' };
const byObjectKey = Map.groupBy(inventory, ({ quantity }) => (quantity < 6 ? restock : sufficient));
console.log(byObjectKey.get(restock).map((i) => i.name));
const byLength = Map.groupBy(['a', 'bb', 'cc', 'ddd'], (w) => w.length);
console.log(byLength, byLength.get(2));

console.log(Object.groupBy(new Set([1, 2, 3, 4, 5]), (n) => (n % 2 ? 'odd' : 'even')));

const legacy = inventory.reduce((acc, item) => { (acc[item.type] ??= []).push(item.name); return acc; }, {});
console.log(legacy);
```

Output:

```
[ 'electronics', 'furniture', 'office' ] [ 'desk', 'chair' ]
true
{
  restock: [ 'laptop', 'desk', 'stapler' ],
  ok: [ 'monitor', 'chair' ]
}
[ 'laptop', 'desk', 'stapler' ]
Map(3) { 1 => [ 'a' ], 2 => [ 'bb', 'cc' ], 3 => [ 'ddd' ] } [ 'bb', 'cc' ]
[Object: null prototype] { odd: [ 1, 3, 5 ], even: [ 2, 4 ] }
{
  electronics: [ 'laptop', 'monitor' ],
  furniture: [ 'desk', 'chair' ],
  office: [ 'stapler' ]
}
```

How it works:

1. Walk the iterable in order.

2. Call `fn(item, index)` to get a key.
3. Append the item to that key's array (creating it the first time).
4. Group order follows the first time each key appears; items inside a group keep their original order.

The result's **null prototype** means there are no inherited keys (a group named `toString` or `constructor` is safe), but also no inherited methods — use `Object.keys(result)`, not `result.hasOwnProperty(...)`. The grouped arrays contain the **original items** (not copies).

Before ES2024 everyone wrote the `reduce` version with `??=` (logical nullish assignment: assign only if the left side is `null`/`undefined`); lodash's `_.groupBy` is the library equivalent (lodash is a widely used utility library).

Trade-offs

	`Object.groupBy`	`Map.groupBy`
Keys	Strings (and symbols)	Any value
Result	Null-prototype object	Map
JSON-friendly	Yes	Convert first
Destructuring `const { ok, restock } = …`	Yes	No
Use when	Grouping by a label/category	Grouping by object, number, or you need Map features

Real-world examples

- Dashboards: group orders by status or by day before rendering sections.
- Chat apps: group messages by date ("Today", "Yesterday").
- Data cleanup: group records by email to find duplicates.
- Counting per group: `Object.fromEntries(Object.entries(byType).map(([k, v]) => [k, v.length]))` → `{ electronics: 2, furniture: 2, office: 1 }`.

In the interview

- *How do you group an array by a property?* `Object.groupBy(arr, (x) => x.prop)` in modern runtimes; `reduce` with `(acc[k] ??= []).push(x)` otherwise.
- *When `Map.groupBy` over `Object.groupBy`?* Non-string keys, or when you need Map semantics (order, `size`, object keys).

Self-check

1. Is `groupBy` a method on arrays? → No — static: `Object.groupBy(items, fn)` / `Map.groupBy(items, fn)`.
2. What prototype does `Object.groupBy`'s result have? → `null`.
3. Group numbers into even/odd with a Map. → `Map.groupBy(nums, (n) => n % 2 === 0)` (keys `true`/`false`).

Version summary (verified September 2026)

Feature	Standard	Node 24	Notes
`Map`, `Set`, `WeakMap`, `WeakSet`, iterators, generators	ES2015	✓	
Async generators, `for await…of`	ES2018	✓	
`WeakRef`, `FinalizationRegistry`	ES2021	✓	Timing unspecified
Symbols as WeakMap keys	ES2023	✓	Non-registered symbols only
`Object.groupBy`, `Map.groupBy`	ES2024	✓	Node 21+
Set methods (`union`, `intersection`, …)	ES2025	✓	Node 22+
Iterator helpers, `Iterator.from`	ES2025	✓	Node 22+
`Array.fromAsync`	ES2026	✓	
`Map/WeakMap.prototype.getOrInsert(Computed)` (upsert)	ES2026	✗	Feature-detect
`Iterator.concat`	ES2026	✗	Feature-detect

Chapter TL;DR

- **Map**: key → value with keys of any type (SameValueZero), insertion order, `size`, direct iteration; `set` (chainable, updates), `get`, `has`, `delete`, `clear`, `keys/values/entries`, `forEach(value, key)`. Prefer it for dynamic dictionaries, counting, caches (LRU), non-string keys; prefer objects for fixed records and JSON. `JSON.stringify(map)` is `{}`.
- **Set**: unique values, insertion order, O(1) `has`; `add` (chainable), `delete`, `clear`, `values`, `entries` (`[v, v]`), `forEach`. Dedupe with `[...new Set(arr)]`. ES2025 adds `union`, `intersection`, `difference`, `symmetricDifference`, `isSubsetOf`, `isSupersetOf`, `isDisjointFrom`.
- **WeakMap/WeakSet** hold object (or non-registered symbol) keys weakly: no leaks, but no `size`/iteration. Use for side tables, per-object caches, tagging. **WeakRef/FinalizationRegistry** are advanced; never rely on their timing.
- **Iterable** = has `[Symbol.iterator]()` returning an **iterator** with `next() → { value, done }` (optional `return()` for cleanup). Arrays, strings, Maps, Sets, typed arrays, NodeLists and generators are iterable; plain objects aren't. Consumers: `for…of`, spread, destructuring, `Array.from`, Map/Set constructors, `Promise.all`.
- **Generators** (`function*`) pause at `yield`; calling one returns an iterator without running the body. Great for lazy/infinite sequences, `Symbol.iterator`, tree traversal (`yield*`), two-way coroutines (functions that can pause and exchange values with their caller: `next(x)`, `return`, `throw`). **Async generators** + `for await…of` for paged or streamed data.
- **Iterator helpers** (ES2025): lazy `map/filter/take/drop/flatMap` and consuming `toArray/reduce/forEach/some/every/find` on any iterator; `Iterator.from` wraps custom ones; arrays need `.values()` first.
- **`Object.groupBy`** / **`Map.groupBy`** (ES2024) bucket items by a key function — null-prototype object with string keys, or a Map with any keys.

Chapter 9 — `this`, Prototypes and Classes

JavaScript's object model surprises people who arrive from Java, C# or Python. There are no "real" classes underneath: objects are linked to other objects through a hidden **prototype** link, and a method finds its object at call time through the keyword `this`. The `class` keyword (ES2015) is a cleaner syntax over that same machinery, plus a few genuinely new features (private `#fields`, static blocks). This chapter explains the machinery first, then the syntax, so that nothing about classes feels like magic.

What you'll learn

1. What `this` is and the four rules (plus arrow functions) that decide its value — and their order of precedence.
2. `call`, `apply` and `bind`: choosing `this` yourself, borrowing methods, partial application.
3. The prototype chain: how property lookup, shadowing, `Object.create` and `Object.create(null)` work.
4. Constructor functions and exactly what `new` does (we re-implement it).
5. Classes: fields, private `#` members, static members, getters/setters, static initialization blocks, brand checks.
6. Inheritance with `extends` and `super`, subclassing built-ins like `Array` and `Error`, initialization order.
7. `instanceof`, `isPrototypeOf` and why they can be fooled.
8. Mixins, and when to prefer composition over inheritance.

Before you start. This chapter assumes you know functions, arrow functions and closures (Chapter 4) and object literals, property descriptors and getters/setters (Chapter 5). Symbols such as `Symbol.iterator` are covered in Chapter 8.

All runnable code is in `code/`. Outputs below are real, produced with Node 24. Every file is an ES module (`.mjs` — a file using `import`/`export`, Chapter 10), and **modules always run in strict mode** — this matters for `this`, as you will see.

9.1 `this` — the call-time context

What it is

`this` is a special, automatically defined identifier inside every non-arrow function. Its value is **not** fixed when you write the function; it is decided **each time the function is called**, by *how* it is called. Think of `this` as the word "me" in a sentence: its meaning depends on who is speaking right now, not on who wrote the sentence.

A **call site** is the place in the code where a function is invoked (for example `user.greet()`). To find `this`, you look at the call site — never at where the function was defined.

Why it matters

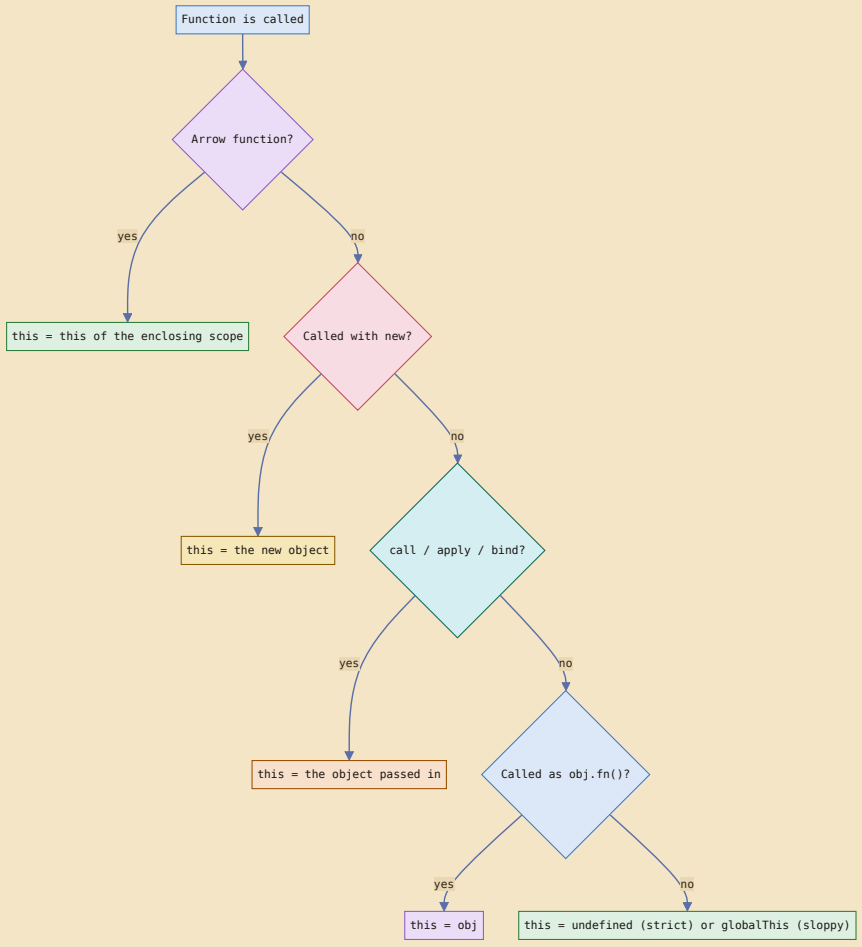
One function can serve many objects. `Array.prototype.push` is a single function shared by every array in your program; it works because, on each call, `this` is the particular array you called it on. The cost of that flexibility is the most common JavaScript bug of all: the **lost** `this`, where a method is passed somewhere as a plain function and suddenly `this` is `undefined`.

How it works — the four rules

For a normal (non-arrow) function, check the call site against these rules. If several apply, the one higher in this list wins:

#	Rule	Call-site shape	`this` is…
4	**`new` binding**	`new Fn(...)`	the brand-new object being constructed
3	**Explicit binding**	`fn.call(obj)`, `fn.apply(obj)`, `fn.bind(obj)()`	`obj`
2	**Implicit binding**	`obj.fn()` or `obj['fn']()`	`obj` — the object just before the dot
1	**Default binding**	plain `fn()`	`undefined` in strict mode; the global object (`globalThis`) in sloppy mode
—	**Arrow function**	any	no own `this`; it uses the `this` of the enclosing scope, *where it was written* (lexical `this` — "lexical" means decided by where code sits in the source)

Precedence, from strongest to weakest: **`new` > explicit > implicit > default**. Arrow functions sit outside the list — nothing (not even `call` or `new`) can change their `this`, and they cannot be used with `new` at all.



Two terms used above:

- **Strict mode** — an opt-in stricter variant of JavaScript (`'use strict'`, or automatically inside modules and class bodies). Among other things it stops a plain call from silently getting the global object as `this`. See Chapter 1.
- **`globalThis`** — the standard name for the global object in every environment (`window` in browsers, `global` in Node).

`code/01_this_rules.mjs`:

```
function whoAmI() {
  return this === undefined ? 'undefined' : this.name;
}

// Rule 1 — default binding: a plain call. Strict mode -> this is undefined.
console.log('1 default :', whoAmI());

// Rule 2 — implicit binding: called as obj.method() -> this is obj.
const alice = { name: 'Alice', whoAmI };
console.log('2 implicit:', alice.whoAmI());

// Lost binding: take the method off the object, then call it plainly.
const detached = alice.whoAmI;
console.log('  lost    :', detached());

// Rule 3 — explicit binding: call / apply / bind choose this for you.
const bob = { name: 'Bob' };
console.log('3 call    :', whoAmI.call(bob));
console.log('  apply   :', whoAmI.apply(bob, []));
const boundToBob = whoAmI.bind(bob);
console.log('  bind    :', boundToBob(), '| still Bob as a method of alice?', { name: 'x', f: boundToBob }.f());

// Rule 4 — new binding: new creates a fresh object and uses it as this.
function Person(name) { this.name = name; }
const carol = new Person('Carol');
console.log('4 new     :', carol.name);

// Arrow functions have NO own this: they use this from the surrounding scope.
const timer = {
  name: 'Timer',
  regular() { return [1].map(function () { return this?.name; })[0]; },
  arrow()   { return [1].map(() => this.name)[0]; },
};
console.log('arrow     : regular callback ->', timer.regular(), '| arrow callback ->', timer.arrow());

// Arrow functions ignore call/apply/bind for this.
const arrowWho = () => (this === undefined ? 'undefined (module top level)' : this);
console.log('arrow+call:', arrowWho.call(bob));

// Precedence: new > explicit (bind) > implicit > default.
const Bound = Person.bind({ name: 'ignored' });
const dave = new Bound('Dave');
console.log('precedence: new beats bind ->', dave.name);
console.log('precedence: call beats implicit ->', alice.whoAmI.call(bob));

// Many array methods accept a thisArg.
const counter = { hits: 0 };
[1, 2, 3].forEach(function () { this.hits++; }, counter);
console.log('thisArg   :', counter.hits);

// Sloppy-mode default binding uses globalThis (Function-created code is never strict here).
const sloppy = new Function('return this === globalThis');
console.log('sloppy default this is globalThis:', sloppy());
```

Output:

```
1 default : undefined
2 implicit: Alice
  lost    : undefined
3 call    : Bob
  apply   : Bob
  bind    : Bob | still Bob as a method of alice? Bob
4 new     : Carol
arrow     : regular callback -> undefined | arrow callback -> Timer
arrow+call: undefined (module top level)
precedence: new beats bind -> Dave
precedence: call beats implicit -> Bob
thisArg   : 3
sloppy default this is globalThis: true
```

What to notice:

- `detached()` prints `undefined` even though the function "came from" `alice`. Functions do not belong to objects; `alice.whoAmI` is just a property holding a reference to a function.
- Inside `timer.regular`, the inner `function` callback is called plainly by `map`, so rule 1 applies and `this` is `undefined`. The arrow callback in `timer.arrow` has no own `this` and sees `arrow()`'s `this`, which is `timer`. This is *the* reason arrow functions were added.
- At the top level of an ES module, `this` is `undefined`. At the top level of a classic browser `<script>` it is `window`, and at the top level of a CommonJS file (Node's older module format using `require`) in Node it is `module.exports` (see Chapter 10).

`this` in event listeners

In the browser, when a regular function is registered with `addEventListener`, the DOM (the browser's object model of the page) calls it with `this` set to the element the listener is attached to (`event.currentTarget`). An arrow listener keeps the outer `this` instead. A class method passed directly as a listener therefore gets the *element* as `this`, not the class instance (the object created from the class with `new`). `code/08_this_in_dom.mjs` runs this in jsdom (a browser-like DOM for Node):

```
class Form {
  constructor(el) {
    this.name = 'SignupForm';
    el.addEventListener('click', this.handle);                // loses this
    el.addEventListener('click', (e) => this.handle2(e));     // arrow keeps this
    el.addEventListener('click', this.handle3.bind(this));    // bind keeps this
  }
  handle() { console.log('method passed directly:', `this is the <${this.tagName.toLowerCase()}> element,
this.name =`, JSON.stringify(this.name)); }
  handle2() { console.log('wrapped in arrow      :', this.name); }
  handle3() { console.log('bound                 :', this.name); }
}
```

Output (including two simpler listeners defined earlier in the file):

```
function listener this: save | same as currentTarget? true
arrow listener this   : undefined
method passed directly: this is the <button> element, this.name = ""
wrapped in arrow      : SignupForm
bound                 : SignupForm
```

The third line is a nasty silent bug: `this.name` did not throw, it read the button's own (empty) `name` attribute. Events are covered fully in Chapter 13.

Common mistakes

- **Passing a method as a callback**: `setTimeout(user.greet, 100)`, `promise.then(obj.handle)` (a promise is an object for a result that arrives later), `arr.map(obj.transform)`. Each of these calls the function plainly. Fix with an arrow wrapper (`() => user.greet()`), `bind`, or an arrow-function class field (§9.5).
- **Arrow functions as object methods**: `const o = { name: 'x', hi: () => this.name }` — the arrow's `this` is the *surrounding* scope's (module: `undefined`), not `o`. Use method shorthand `hi() { … }`.
- **Assuming `this` is the function itself.** It never is; if you need the function, give it a name.
- **Nested regular functions inside methods** lose `this` (the `timer.regular` case). Old code wrote `const self = this;` to work around it; today, use an arrow.

In the interview

30-second answer: "`this` is decided at call time by the call site. In order of precedence: `new` makes it the new object; `call` / `apply` / `bind` set it explicitly; calling as `obj.method()` makes it `obj`; a plain call gives `undefined` in strict mode or the global object in sloppy mode. Arrow

functions don't have their own `this` — they capture it lexically from where they're written, and nothing can rebind it."

- *Q: What does `const f = obj.m; f()` log for `this`?* A: `undefined` in strict code (global object in sloppy code) — the implicit binding is lost because the call site is plain.
- *Q: Can you `new` an arrow function?* A: No — it throws `TypeError: … is not a constructor`. Arrows have no `prototype` and no own `this`.
- *Q: Does `bind` beat `new`?* A: No. `new BoundFn()` still creates a fresh object; the bound `this` is ignored (bound arguments are kept).

Self-check

1. `const o = { v: 1, f() { return () => this.v; } }; o.f()()` returns? → `1` — the arrow captures `f`'s `this`, which is `o`.
2. What is `this` at the top level of an ES module? → `undefined`.
3. `[1,2].forEach(function(){ console.log(this) }, {a:1})` logs? → `{ a: 1 }` twice — `forEach` accepts a `thisArg`.
4. Why does `setTimeout(obj.m, 0)` lose `this`? → `setTimeout` receives only the function and calls it plainly.

TL;DR

Look at the call site: `new` > `call/apply/bind` > `obj.fn()` > plain call (`undefined` in strict code). Arrows borrow `this` from where they were written.

9.2 `call`, `apply` and `bind`

What it is

Every function inherits three methods from `Function.prototype` that let you set `this` explicitly:

Method	Signature	Runs now?	Returns
`call`	`fn.call(thisArg, a, b, …)`	yes	the function's result
`apply`	`fn.apply(thisArg, [a, b, …])`	yes	the function's result
`bind`	`fn.bind(thisArg, a, …)`	no	a new **bound function** with `this` (and optionally leading arguments) fixed

Memory trick: **a**pply takes an **a**rray, **c**all takes **c**ommas.

Why it matters

- **Method borrowing** — reuse a method on an object that doesn't have it (array methods on array-like objects — objects with numbered keys and a `length` but no array methods — for example `arguments` or a DOM `NodeList`).
- **Fixing lost `this`** — `bind` produces a function you can safely hand to a callback API.
- **Partial application** — `bind` can pre-fill leading arguments, producing a more specific function (a form of currying — turning a many-argument function into a chain of one-argument functions; see Chapter 4).

`code/02_call_apply_bind.mjs`:

```
function greet(greeting, punctuation) {
  return `${greeting}, ${this.name}${punctuation}`;
}
const ann = { name: 'Ann' };

console.log(greet.call(ann, 'Hi', '!'));        // args one by one
console.log(greet.apply(ann, ['Hello', '?']));  // args as an array
const hiAnn = greet.bind(ann, 'Hey');           // this AND first argument fixed
console.log(hiAnn('.'));

// Borrowing a method: array methods work on any "array-like" (has length + indexes).
const arrayLike = { 0: 'a', 1: 'b', 2: 'c', length: 3 };
console.log(Array.prototype.join.call(arrayLike, '-'));
console.log(Array.prototype.map.call('abc', (ch) => ch.toUpperCase()));

// apply was the old way to spread an array into arguments; spread syntax replaced it.
const nums = [4, 9, 2];
console.log(Math.max.apply(null, nums), Math.max(...nums));

// A bound function cannot be re-bound.
const boundOnce = greet.bind({ name: 'First' });
const boundTwice = boundOnce.bind({ name: 'Second' });
console.log(boundTwice('Yo', '!'));
console.log('bound name:', boundOnce.name, '| length:', greet.length, '->', hiAnn.length);

// Classic fix for a lost this: bind in advance.
class Button {
  constructor(label) { this.label = label; this.onClick = this.onClick.bind(this); }
  onClick() { return `clicked ${this.label}`; }
}
const { onClick } = new Button('Save');   // detached, but still works
console.log(onClick());
```

Output:

```
Hi, Ann!
Hello, Ann?
Hey, Ann.
a-b-c
[ 'A', 'B', 'C' ]
9 9
Yo, First!
bound name: bound greet | length: 2 -> 1
clicked Save
```

How it works

1. `call` / `apply` invoke the function immediately with the given `this`.
2. `bind` returns an **exotic bound function object** ("exotic" is the spec's word for an object with special built-in behaviour) that remembers the target function, the `this` value and any pre-filled arguments. Calling it calls the target with those values.
3. Binding again has no effect on `this` — the inner binding already won (`Yo, First!`). Extra arguments from later binds are appended, though.
4. A bound function's `name` is `"bound " + original name`; its `length` drops by the number of pre-filled arguments.
5. Passing `null` or `undefined` as `thisArg` in strict code gives `this === null/undefined`; in sloppy code it becomes `globalThis`.

Trade-offs

Need	Best tool today
Spread an array into arguments	spread syntax `f(...arr)` (clearer than `apply`)
Stable callback that uses `this`	arrow wrapper `() => obj.m()`, class arrow field, or `bind`
Pre-fill arguments without caring about `this`	arrow `(x) => f(1, x)` — easier to read than `f.bind(null, 1)`
Borrow a method for an array-like	`Array.from(arrayLike)` then use normal methods, or `Array.prototype.x.call`

Common mistakes

- Calling `bind` inside a render loop (code that redraws the screen repeatedly) or on every event creates a new function each time — a problem when identity matters (for example removing a listener: `removeEventListener('click', fn.bind(this))` removes nothing, because it's a *different* function).
- Expecting `bind` to change an arrow function's `this` — it can't.

Self-check

1. Difference between `f.call(o, 1, 2)` and `f.apply(o, [1, 2])`? → None in effect; only how arguments are passed.
2. What does `f.bind(o)` return? → A new function; it doesn't call `f`.
3. `f.bind(a).bind(b)()` — what is `this`? → `a`.

TL;DR

`call` and `apply` run now with a chosen `this` (commas vs array). `bind` returns a new function with `this` locked forever. Prefer spread and arrows where they read better.

9.3 Prototypes and the prototype chain

What it is

Every JavaScript object has an internal slot called `[[Prototype]]` that points either to another object or to `null`. When you read a property that the object doesn't have itself, the engine follows that link and looks in the prototype, then the prototype's prototype, and so on until it finds the property or reaches `null`. This linked list of objects (each one pointing to the next) is the **prototype chain**, and it is JavaScript's inheritance mechanism (the way one object gets behaviour from another): **delegation**, not copying.

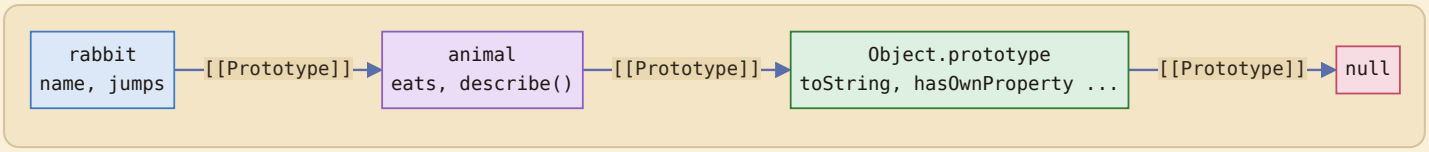
Analogy: asking a question at a help desk. If the clerk doesn't know, they ask their supervisor; the supervisor asks the manager; if nobody up the line knows, the answer is "undefined".

Key vocabulary:

Term	Meaning
`[[Prototype]]`	The hidden link every object has. Read with `Object.getPrototypeOf(obj)`, set with `Object.setPrototypeOf` or at creation with `Object.create(proto)`.
`__proto__`	A legacy getter/setter on `Object.prototype` exposing `[[Prototype]]`. Avoid it in code; the **`__proto__:` key in an object literal** is a separate, standard way to set the prototype at creation.
`Fn.prototype`	An ordinary property that every normal function (and class) has. It is *not* the function's own prototype — it is the object that becomes the `[[Prototype]]` of instances created with `new Fn()`.
Own property	A property stored directly on the object. Check with `Object.hasOwn(obj, key)` (ES2022).
Shadowing	An own property hides a property of the same name further up the chain.

How it works

1. **Reading** `obj.x`: look for own property `x`; if absent, repeat on `Object.getPrototypeOf(obj)`; stop at `null` and return `undefined`.
2. **Writing** `obj.x = v`: normally creates or updates an **own** property on `obj` — the prototype is never modified by assignment. (Two exceptions: if the chain has a *setter* for `x`, the setter runs; if the chain has a *non-writable* `x`, the assignment fails — silently in sloppy mode, `TypeError` in strict mode.)
3. **Deleting** `delete obj.x` removes only an own property, so an inherited value can "reappear".
4. **Methods run with `this` = the object you started from**, not the prototype where the method was found. That's why shared methods can work on per-object data.



`code/03_prototype_chain.mjs` (key parts):

```
const animal = {
  eats: true,
  describe() { return `${this.name} eats: ${this.eats}`; },
};
const rabbit = Object.create(animal);   // rabbit.[[Prototype]] = animal
rabbit.name = 'Rabbit';
rabbit.jumps = true;

console.log(Object.getPrototypeOf(rabbit) === animal);         // true
console.log(rabbit.eats, rabbit.describe());                   // found on animal
console.log('own keys:', Object.keys(rabbit));
console.log('hasOwn eats?', Object.hasOwn(rabbit, 'eats'), '| "eats" in rabbit?', 'eats' in rabbit);

const seen = [];
for (const k in rabbit) seen.push(k);                           // for...in walks the chain
console.log('for...in:', seen);

rabbit.eats = false;                                            // creates an OWN property
console.log('rabbit.eats', rabbit.eats, '| animal.eats', animal.eats);
delete rabbit.eats;
console.log('after delete, rabbit.eats', rabbit.eats);

console.log('rabbit :', chain(rabbit));   // chain() walks getPrototypeOf until null
console.log('[]     :', chain([]));
console.log('fn     :', chain(function f() {}));
console.log('Map    :', chain(new Map()));

const a = [1], b = [2];
console.log('shared push?', a.push === b.push, a.push === Array.prototype.push);

const dict = Object.create(null);         // no prototype at all
dict.toString = 'just data';
console.log('proto of dict:', Object.getPrototypeOf(dict), '| "hasOwnProperty" in dict:', 'hasOwnProperty' in
dict);

const base = { hello() { return 'hi from base'; } };
const late = {};
Object.setPrototypeOf(late, base);        // works, but slow — prefer Object.create
console.log(late.hello());

const lit = { __proto__: base, own: 1 };  // literal __proto__ key sets the prototype
console.log(lit.hello(), Object.keys(lit));
console.log('missing ->', rabbit.nope);
```

Output:

```
true
true Rabbit eats: true
own keys: [ 'name', 'jumps' ]
hasOwn eats? false | "eats" in rabbit? true
for...in: [ 'name', 'jumps', 'eats', 'describe' ]
rabbit.eats false | animal.eats true
after delete, rabbit.eats true
rabbit : animal -> Object.prototype -> null
[]     : Array.prototype -> Object.prototype -> null
fn     : Function.prototype -> Object.prototype -> null
Map    : Map.prototype -> Object.prototype -> null
shared push? true true
proto of dict: null | "hasOwnProperty" in dict: false
hi from base
hi from base [ 'own' ]
missing -> undefined
```

Checking properties — which tool?

Check	Looks at own props	Looks up the chain	Notes
`Object.hasOwn(o, k)`	✓	✗	Preferred (ES2022); works on `Object.create(null)` objects
`o.hasOwnProperty(k)`	✓	✗	Fails on null-prototype objects, can be shadowed
`k in o`	✓	✓	Includes non-enumerable and inherited
`Object.keys(o)`	✓ (enumerable, string keys)	✗	
`for (k in o)`	✓	✓ (enumerable only)	Why `for…in` is risky for objects with prototypes — see Chapter 3

Why it matters

- **Memory**: a thousand arrays share one `push` function.
- **Built-ins are extendable** (though you shouldn't — see mistakes): every array method lives on `Array.prototype`.
- `Object.create(null)` gives a "pure dictionary" whose keys can't collide with `toString`, `constructor` or `__proto__` — useful when keys come from user input. (Often a `Map` is better still; see Chapter 8.)

Common mistakes

- **Modifying built-in prototypes** (`Array.prototype.last = …`) — "monkey-patching". It can clash with future standard methods; this really happened: the planned `Array.prototype.contains` became `includes`, and `flatten` became `flat`, because the old MooTools library (a popular JavaScript library of the late 2000s) had already patched incompatible versions onto millions of websites.
- **Calling `Object.setPrototypeOf` on hot objects** (objects used very frequently) — engines optimize objects by their shape (an internal record of which properties an object has); changing the prototype after creation de-optimizes property access. MDN explicitly warns about this.
- **Confusing `Fn.prototype` with `Object.getPrototypeOf(Fn)`** — the first is the prototype *for instances*; the second is `Function.prototype`.
- **Prototype pollution** — merging untrusted JSON like `{"__proto__": {"isAdmin": true}}` into objects with a naive deep-merge (copying nested properties recursively) can modify `Object.prototype` for the whole program. Use null-prototype objects, `Map`, or libraries that skip `__proto__`/`constructor` keys. (Security is covered in Chapter 14.)

In the interview

30-second answer: "Every object has a hidden `[[Prototype]]` link. When a property isn't found on the object, the engine walks that chain until it finds it or hits `null`. Writing a property creates an own property, shadowing the inherited one. `Object.create(proto)` makes an object with a given prototype, and constructors and classes set it up via `Fn.prototype`. Methods are shared through the chain and run with `this` bound to the original receiver (the object the method was called on)."

- *Q: What's the difference between `__proto__` and `prototype`?* A: `__proto__` (legacy accessor) is an object's own prototype link; `prototype` is a property of functions naming the object that `new` will link instances to.
- *Q: What's at the end of every chain?* A: `null` (usually right after `Object.prototype`).
- *Q: How do you make an object with no prototype?* A: `Object.create(null)` or `{ __proto__: null }`.

Self-check

1. `const p = { x: 1 }; const c = Object.create(p); c.x = 2; p.x` → `1` — assignment made an own property on `c`.

2. Does `Object.keys` include inherited properties? → No.
3. Where does `[].map` live? → `Array.prototype.map`.
4. What does `Object.getPrototypeOf(Object.prototype)` return? → `null`.

TL;DR

Objects delegate missing property reads up a chain of `[[Prototype]]` links ending in `null`. Writes stay on the object. Use `Object.create`, `Object.getPrototypeOf`, `Object.hasOwn`; don't patch built-ins.

9.4 Constructor functions and `new`

What it is

Before ES2015, the way to make many similar objects was a **constructor function**: an ordinary function, by convention Capitalized, called with `new`. Per-instance data is assigned to `this`; shared methods are put on `Constructor.prototype`. Classes (§9.5) are built on exactly this pattern, so understanding it explains classes.

How `new` works

When you write `new Fn(args)`, the engine:

1. Creates a new empty object.
2. Sets its `[[Prototype]]` to `Fn.prototype`.
3. Calls `Fn` with `this` bound to that object and the given arguments.
4. If `Fn` explicitly returns an **object**, that object is the result; otherwise (no return, or a primitive return) the new object from step 1 is the result.

Every `Fn.prototype` starts with one property, `constructor`, pointing back to `Fn` — which is how `instance.constructor` finds its maker.

`code/04_constructor_functions.mjs`:

```
function Account(owner, balance = 0) {
  this.owner = owner;       // own, per-instance data
  this.balance = balance;
}
Account.prototype.deposit = function (amount) {  // one shared copy for all instances
  this.balance += amount;
  return this;
};
Account.prototype.toString = function () { return `${this.owner}: $${this.balance}`; };

const acc = new Account('Ravi', 100);
acc.deposit(50).deposit(25);
console.log(String(acc));
console.log('proto link  :', Object.getPrototypeOf(acc) === Account.prototype);
console.log('constructor :', acc.constructor === Account, Account.prototype.constructor === Account);
console.log('own keys    :', Object.keys(acc));

// What `new` does — written out by hand
function myNew(Ctor, ...args) {
  const obj = Object.create(Ctor.prototype);        // 1+2. new object linked to Ctor.prototype
  const result = Ctor.apply(obj, args);              // 3. run constructor with this = obj
  return result !== null && (typeof result === 'object' || typeof result === 'function')
    ? result                                         // 4a. if it returned an object, use that
    : obj;                                           // 4b. otherwise the new object
}
const acc2 = myNew(Account, 'Mei', 10);
console.log('myNew       :', String(acc2), acc2 instanceof Account);

// Forgetting `new` — in strict code this is undefined, so assigning throws
try {
  Account('Oops');
} catch (e) {
  console.log('no new      :', e.constructor.name, '-', e.message);
}

// Guard with new.target
function Safe(name) {
  if (!new.target) return new Safe(name);
  this.name = name;
}
console.log('new.target  :', Safe('guarded').name);

// Inheritance the old way
function Savings(owner, balance, rate) {
  Account.call(this, owner, balance);                // "super" call
  this.rate = rate;
}
Savings.prototype = Object.create(Account.prototype);  // link the prototypes
Savings.prototype.constructor = Savings;               // repair constructor
Savings.prototype.addInterest = function () { return this.deposit(this.balance * this.rate); };
const s = new Savings('Lena', 1000, 0.05);
console.log('old-style   :', String(s.addInterest()), s instanceof Savings, s instanceof Account);
```

Output:

```
Ravi: $175
proto link  : true
constructor : true true
own keys    : [ 'owner', 'balance' ]
myNew       : Mei: $10 true
no new      : TypeError - Cannot set properties of undefined (setting 'owner')
new.target  : guarded
old-style   : Lena: $1050 true true
```

`new.target` is a meta-property: inside a function called with `new`, it refers to the constructor that `new` was applied to; in a plain call it is `undefined`.

Common mistakes

- **Defining methods inside the constructor** (`this.deposit = function …`) — every instance gets its own copy. Works, but wastes memory and defeats sharing. Put methods on the prototype (classes do this automatically).
- **Forgetting `new`** — in sloppy code, `this` becomes the global object and you silently create global variables. In strict code you get a `TypeError`. Classes refuse to be called without `new`.
- **Replacing `Fn.prototype` and forgetting `constructor`** — after `Savings.prototype = Object.create(...)`, `s.constructor` would report `Account` until repaired.

Self-check

1. Which object becomes `[[Prototype]]` of `new Fn()`? → `Fn.prototype`.
2. A constructor ends with `return 42;` — what does `new` give you? → The new object (primitives are ignored).
3. A constructor ends with `return { a: 1 };` → `{ a: 1 }`, and it is **not** linked to `Fn.prototype`.

TL;DR

`new` = create object → link to `Fn.prototype` → run `Fn` with that `this` → return it (unless `Fn` returns another object). Shared methods go on `Fn.prototype`.

9.5 Classes

What it is

A **class** is a template for creating objects. In JavaScript, `class` declares a special kind of function (the constructor) and fills its `prototype` with methods — the same arrangement as §9.4 — but with shorter syntax, safer defaults, and features that are impossible with plain functions: truly private members, static blocks and `super`. People often call classes "syntactic sugar" (nicer syntax for something you could already write another way), and for methods and inheritance that is mostly right; for private fields it isn't.

Class features at a glance

Feature	Syntax	Lives on	Added in
Constructor	`constructor(a) { … }`	the class function itself	ES2015
Method	`m() { … }`	`Class.prototype` (non-enumerable)	ES2015
Getter / setter	`get x() {}` / `set x(v) {}`	`Class.prototype` (accessor property)	ES2015
Static method	`static m() {}`	the class function	ES2015
Computed name	`[expr]() {}`	prototype or class	ES2015
Public instance field	`x = 0;`	each instance (own, enumerable)	ES2022
Static field	`static x = 0;`	the class function	ES2022
Private field / method	`#x = 0;` `#m() {}`	hidden per-instance slot	ES2022
Private static	`static #x`, `static #m()`	hidden slot on the class	ES2022
Static initialization block	`static { … }`	runs once at definition time	ES2022
Private "brand check"	`#x in obj`	—	ES2022

Terms:

- **Field** — a property declared in the class body and created automatically on every instance (or on the class, if `static`). Its initializer runs once per instance, with `this` = the instance.

- **Private member** — a name starting with `#`. It can be used only by code *lexically inside* the class body. It's not a property: it doesn't appear in `Object.keys`, `JSON.stringify`, `for…in`, or proxies (`Proxy` objects that intercept property access), and accessing it from outside is a **syntax error** at parse time (when the engine first reads the code, before running it).
- **Accessor** — a getter/setter pair that looks like a property from the outside but runs code (defined in Chapter 5).
- **Static member** — belongs to the class itself, not instances: factory methods (methods that build and return new instances, e.g. `Temperature.fromFahrenheit`), counters, constants, caches.

`code/05_classes.mjs`:

```
class Temperature {
  unit = 'C';                       // public instance field
  #celsius;                         // private instance field
  static instances = 0;             // static field
  static #absoluteZero = -273.15;   // private static field

  static {                          // static initialization block — runs once
    Temperature.freezing = new Temperature(0);
    Temperature.instances = 0;      // don't count the helper instance
  }

  constructor(celsius) {
    Temperature.#check(celsius);
    this.#celsius = celsius;
    Temperature.instances++;
  }

  get fahrenheit() { return this.#celsius * 9 / 5 + 32; }
  set fahrenheit(f) {
    const c = (f - 32) * 5 / 9;
    Temperature.#check(c);
    this.#celsius = c;
  }
  get celsius() { return this.#celsius; }   // read-only from outside: no setter

  #format(v) { return v.toFixed(1); }       // private method
  toString() { return `${this.#format(this.#celsius)}°C`; }

  static #check(c) {
    if (c < Temperature.#absoluteZero) throw new RangeError(`below absolute zero: ${c}`);
  }
  static fromFahrenheit(f) { const t = new Temperature(0); t.fahrenheit = f; return t; }
  static isTemperature(obj) { return #celsius in obj; }   // brand check
}

const t = new Temperature(25);
console.log(String(t), '=', t.fahrenheit, '°F');
t.fahrenheit = 212;
console.log('after set  :', String(t), '| unit field:', t.unit);
console.log('static     :', Temperature.instances, String(Temperature.freezing),
String(Temperature.fromFahrenheit(-40)));
console.log('brand check:', Temperature.isTemperature(t), Temperature.isTemperature({ celsius: 1 }));
console.log('own keys   :', Object.keys(t), '(private fields are invisible)');
try { new Temperature(-300); } catch (e) { console.log('validation :', e.name, e.message); }
// t.#celsius  -> SyntaxError at parse time if written outside the class
```

Output (first part):

```
25.0°C = 77 °F
after set  : 100.0°C | unit field: C
static     : 1 0.0°C -40.0°C
brand check: true false
own keys   : [ 'unit' ] (private fields are invisible)
validation : RangeError below absolute zero: -300
```

Why does `static` print `1`? The static block created `freezing` (counter 1) and then reset the counter to 0; `new Temperature(25)` made it 1. JavaScript evaluates arguments left to right, so `Temperature.instances`

was read (1) *before* `fromFahrenheit(-40)` ran and bumped it to 2. Order of evaluation matters whenever expressions have side effects (changes beyond producing a value, like bumping a counter).

Facts about classes that trip people up

The same file continues:

```
console.log('typeof class        :', typeof Temperature);
console.log('methods on prototype:', Object.getOwnPropertyNames(Temperature.prototype));
console.log('methods enumerable? :', Object.keys(Temperature.prototype).length > 0);
try { Temperature(1); } catch (e) { console.log('call without new    :', e.message); }

try { new Later(); } catch (e) { console.log('TDZ                 :', e.name, '-', e.message); }
class Later {}

class Clicker {
  count = 0;
  method() { return ++this.count; }
  arrow = () => ++this.count;   // one function per instance, this captured forever
}
const c = new Clicker();
const { method, arrow } = c;
try { method(); } catch (e) { console.log('detached method     :', e.message); }
console.log('detached arrow      :', arrow(), arrow());
console.log('arrow on prototype? :', 'arrow' in Clicker.prototype, '| own?', Object.hasOwn(c, 'arrow'));

const key = 'shout';
const Loud = class { [key]() { return 'HEY'; } static [Symbol.hasInstance](x) { return x === 42; } };
console.log('computed name       :', new Loud().shout(), '| 42 instanceof Loud:', 42 instanceof Loud);

const desc = Object.getOwnPropertyDescriptor(Temperature.prototype, 'fahrenheit');
console.log('accessor descriptor :', typeof desc.get, typeof desc.set, 'enumerable:', desc.enumerable);
```

Output:

```
typeof class        : function
methods on prototype: [ 'constructor', 'fahrenheit', 'celsius', 'toString' ]
methods enumerable? : false
call without new    : Class constructor Temperature cannot be invoked without 'new'
TDZ                 : ReferenceError - Cannot access 'Later' before initialization
detached method     : Cannot read properties of undefined (reading 'count')
detached arrow      : 1 2
arrow on prototype? : false | own? true
computed name       : HEY | 42 instanceof Loud: true
accessor descriptor : function function enumerable: false
```

Fact	Consequence
A class is a function (`typeof` → `'function'`)	It has `prototype`, `name`, `length` like any function
Class bodies are always strict mode	Detached methods get `this === undefined`, not the global object
Methods are non-enumerable	`for…in` over an instance shows only data fields — nicer than §9.4
Must be called with `new`	Calling it plainly throws `TypeError`
Declarations are hoisted into the **TDZ** (temporal dead zone, Chapter 2)	You can't use a class before its `class` line runs — unlike function declarations
Private methods (`#format`) and the `#check` static are not on the prototype	`getOwnPropertyNames` doesn't list them
Arrow-function **fields** are own properties created per instance	Great for callbacks (`this` is fixed), but each instance gets a new function and subclasses can't call them via `super`
`static [Symbol.hasInstance]` customizes `instanceof`	Rarely needed; shown only so you recognize it

Getters, setters and private fields together

The `Temperature` class shows the standard pattern for **encapsulation** (hiding internal state and exposing a controlled interface): store the real value in a private field, expose a getter, and validate in the setter or the constructor. `celsius` has a getter but no setter, so it's read-only from outside — assigning `t.celsius = 5` is silently ignored in sloppy code and throws `TypeError` in strict code.

Before ES2022, "private" meant a naming convention (`this._celsius`) or a closure/`WeakMap` trick. The convention is still common in older code — but `_x` is fully public.

Static initialization blocks

`static { … }` runs once, when the class definition is evaluated, with `this` = the class. Use it for setup that needs statements (loops, `try/catch`, several related statics) or access to private statics — things a single `static x = expr` can't express. A class may have several; they run in order with static field initializers.

When not to use a class

- For a single object (configuration, a namespace of functions), an object literal or a module (Chapter 10) is simpler.
- For a function that just transforms data, a plain function is simpler and easier to test.
- React function components (UI pieces written as plain functions in the React library) replaced class components for new code (see the React handbook).

Common mistakes

- Using `this.x` in a field initializer that depends on a *later* field — fields initialize top to bottom.
- Expecting `#private` fields to survive `structuredClone`, `JSON.stringify`, or `{...spread}` — none of them copy private fields.
- Using `Object.keys(instance)` to "see" private data in debugging — use the DevTools/Node inspector (the built-in debuggers that let you pause code and look at values), which can display private fields.
- Writing `class A { method = function () {} }` thinking it's a method — it's a per-instance field.

In the interview

> **30-second answer:** "A JavaScript class is a constructor function plus a prototype with its methods, created with nicer syntax. It's strict, must be called with `new`, its methods are non-enumerable,

and it's in the TDZ until defined. ES2022 added public and private fields, private methods with `#`, static blocks and `#x in obj` checks. Private fields are enforced by the language, unlike the old `_underscore` convention."

- *Q: Are classes hoisted?* A: The binding is hoisted but uninitialized (TDZ), so using it early throws `ReferenceError`.
- *Q: Method vs arrow field — which for event handlers?* A: An arrow field keeps `this` bound without `bind`, at the cost of one function per instance and no `super` access; a method is shared on the prototype.
- *Q: How private is `#x` really?* A: Truly inaccessible from outside the class body; not reflected by any object API. Only code in the class body (including static methods handling other instances) can read it.

Self-check

1. Where does a class method live? → On `Class.prototype`.
2. Where does a public field live? → On each instance, as an own enumerable property.
3. `class A { #x = 1; static has(o) { return #x in o; } } A.has({})` → `false`.
4. What does `typeof class {}` return? → `'function'`.

TL;DR

`class` = constructor + prototype methods with safer defaults (strict, `new`-only, non-enumerable methods). ES2022 adds fields, `#private`, `static {}` and brand checks. Use getters/setters with private fields for encapsulation.

9.6 Inheritance with `extends` and `super`

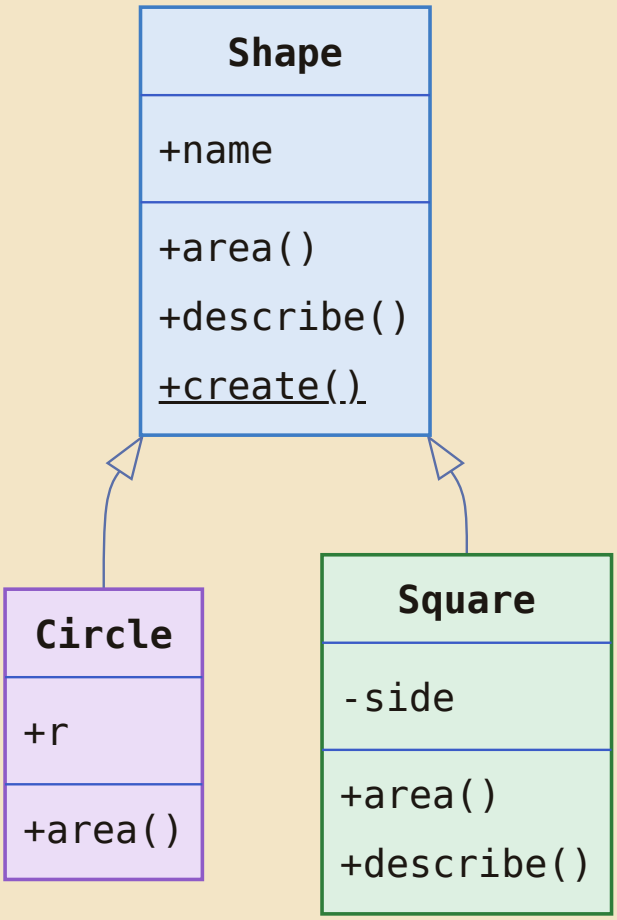
What it is

`class Child extends Parent` makes `Child` a specialized version of `Parent`: instances of `Child` get `Parent`'s methods through the prototype chain, and `Child` can add or **override** (replace) methods. `super` is how the child reaches the parent: `super(...)` calls the parent's constructor, and `super.method()` calls the parent's version of a method.

How it works

`extends` links **two** prototype chains:

1. `Child.prototype.[[Prototype]] = Parent.prototype` — instances inherit instance methods.
2. `Child.[[Prototype]] = Parent` — the class itself inherits static methods.



Rules for a derived constructor:

- It **must call** `super(...)` **before using** `this` — until then, the object doesn't exist (in a derived class, the *parent* constructor creates the object). Using `this` first throws `ReferenceError`.
- If you don't write a constructor, the default one is `constructor(...args) { super(...args); }`.
- **Initialization order**: `super()` runs the parent's field initializers and constructor; only then are the child's fields initialized; then the rest of the child's constructor body runs.

`code/06_inheritance.mjs`:

```
class Shape {
  constructor(name) {
    if (new.target === Shape) throw new TypeError('Shape is abstract');
    this.name = name;
  }
  area() { throw new Error(`${this.constructor.name} must implement area()`); }
  describe() { return `${this.name} with area ${this.area().toFixed(2)}`; }
  static create(...args) { return new this(...args); } // `this` = the class it's called on
}

class Circle extends Shape {
  constructor(r) {
    super('circle');          // must run before touching `this`
    this.r = r;
  }
  area() { return Math.PI * this.r ** 2; }
}

class Square extends Shape {
  #side;
  constructor(side) { super('square'); this.#side = side; }
  area() { return this.#side ** 2; }
  describe() { return super.describe() + ' (all sides equal)'; } // extend parent behaviour
}

const shapes = [new Circle(1), Square.create(3)];
for (const s of shapes) console.log(s.describe());

try { new Shape('x'); } catch (e) { console.log('abstract   :', e.message); }
class Broken extends Shape { constructor() { super('broken'); } }
try { new Broken().describe(); } catch (e) { console.log('no area   :', e.message); }
class NoSuper extends Shape { constructor() { this.x = 1; super('x'); } }
try { new NoSuper(); } catch (e) { console.log('this first :', e.name, '-', e.message); }

console.log('instance chain:', Object.getPrototypeOf(Circle.prototype) === Shape.prototype);
console.log('static chain  :', Object.getPrototypeOf(Circle) === Shape, '-> Circle.create exists:', typeof
Circle.create);
```

Output:

```
circle with area 3.14
square with area 9.00 (all sides equal)
abstract   : Shape is abstract
no area    : Broken must implement area()
this first : ReferenceError - Must call super constructor in derived class before accessing 'this' or returning
from derived constructor
instance chain: true
static chain  : true -> Circle.create exists: function
```

What to notice:

- **Polymorphism** — `describe()` is written once in `Shape` but calls `this.area()`, which runs the *subclass's* `area`. That's dynamic dispatch (choosing which method to run at call time, based on the actual object) through the prototype chain.
- **Abstract classes** — JavaScript has no `abstract` keyword. The `new.target === Shape` check and a throwing `area()` imitate one. (TypeScript adds real `abstract` checks at compile time — Chapter 16.)
- `static create` uses `new this(...)`, so `Square.create(3)` builds a `Square` — static methods are inherited and `this` is the class they're called on.
- A private field (`#side`) is **not** inherited in the sense of being accessible: `Circle` code could not read `Square`'s `#side`, and even a subclass of `Square` couldn't. Private means "this class body only". Use a normal property or a protected-by-convention name ("protected" in other languages means visible to subclasses but not outsiders) if subclasses need it.

Subclassing built-ins and initialization order

```
class Stack extends Array {
  peek() { return this.at(-1); }
}
const st = Stack.from([1, 2, 3]);
st.push(4);
console.log('Stack         :', st.peek(), st.length, Array.isArray(st), st.map((x) => x * 2) instanceof Stack);

class HttpError extends Error {
  constructor(status, message) { super(message); this.name = 'HttpError'; this.status = status; }
}
const he = new HttpError(404, 'Not Found');
console.log('Error subclass:', he instanceof Error, he.name, he.status, he.message);

class Queue { get [Symbol.toStringTag]() { return 'Queue'; } }
console.log('toStringTag   :', Object.prototype.toString.call(new Queue()));

class A { a = console.log('  A field') ?? 1; constructor() { console.log('  A constructor'); } }
class B extends A { b = console.log('  B field') ?? 2; constructor() { console.log('  B before super'); super();
console.log('  B after super'); } }
console.log('init order:'); new B();
```

Output:

```
instanceof    : true true true false
isPrototypeOf : true
fooled?       : true (instanceof only checks the chain)
Stack         : 4 4 true true
Error subclass: true HttpError 404 Not Found
toStringTag   : [object Queue]
init order:
  B before super
  A field
  A constructor
  B field
  B after super
```

(The first three lines come from the `instanceof` section, explained in §9.7.)

- `class … extends Array` gives real arrays: `length` updates automatically, `Array.isArray` is `true`, and methods like `map` return a `Stack` (through the `Symbol.species` mechanism). With old constructor functions, subclassing `Array` properly was impossible.
- Subclassing `Error` is the standard way to make custom error types — Chapter 11 builds a full hierarchy.
- The init-order output shows why a parent constructor must not call an overridable method that relies on child fields: when `A`'s constructor runs, `B`'s fields don't exist yet.

`super` in object literals

`super` also works inside **concise methods** of object literals, referring to the object's prototype: `const child = { __proto__: parent, hi() { return super.hi() + '!'; } }`. It is bound to the object the method was *defined* in (its "home object"), not to `this` — so copying such a method to another object doesn't change what `super` means.

Common mistakes

- Forgetting `super()` in a derived constructor, or calling it after using `this`.
- Deep hierarchies (`Animal → Mammal → Dog → ServiceDog → …`) — each level couples you to (makes your code depend on the details of) every level above. See §9.9.
- Overriding a method and forgetting to call `super.method()` when the parent's behaviour is still needed (cleanup, validation).
- Calling overridable methods from a parent constructor (child fields aren't initialized yet).

In the interview

> **30-second answer:** "`extends` links `Child.prototype` to `Parent.prototype` for instance methods and `Child` to `Parent` for statics. A derived constructor must call `super()` before using `this`, because the parent constructs the object. `super.method()` calls the parent's implementation. Built-ins like `Array`, `Error` and `Map` can be subclassed properly with class syntax."

- *Q: What happens if a derived class has no constructor?* A: A default one forwards all arguments to `super(...)`.
- *Q: Can you extend `null` or an expression?* A: `extends` accepts any expression that evaluates to a constructor (or `null`) — that's what makes mixin factories possible (§9.8).
- *Q: Order of initialization?* A: Parent fields → parent constructor body → child fields → rest of the child constructor.

Self-check

1. Why must `super()` come before `this` in a derived class? → The parent constructor creates and initializes the object.
2. How does a static method get inherited? → `Child.[[Prototype]]` is `Parent`.
3. Can a subclass read the parent's `#secret`? → No.

TL;DR

`extends` links both instance and static chains; `super()` first, then `this`; `super.m()` reaches the parent. Built-ins subclass cleanly. Keep hierarchies shallow.

9.7 `instanceof` and type checks

What it is

`obj instanceof C` asks: **"does `C.prototype` appear anywhere in `obj`'s prototype chain?"** It says nothing about how the object was actually created. `C.prototype.isPrototypeOf(obj)` asks the same question the other way around.

```
const c = shapes[0];
console.log('instanceof    :', c instanceof Circle, c instanceof Shape, c instanceof Object, c instanceof Square);
console.log('isPrototypeOf :', Shape.prototype.isPrototypeOf(c));
const fake = Object.create(Circle.prototype);
console.log('fooled?       :', fake instanceof Circle, '(instanceof only checks the chain)');
```

Output (shown above): `true true true false`, `true`, `true`.

`fake` never ran the `Circle` constructor (it has no `r`), yet `instanceof` says it is a `Circle`.

Choosing a type check

Question	Use	Pitfall
Is it a primitive of type T?	`typeof x === 'string'` etc.	`typeof null === 'object'` (Chapter 2)
Is it an array?	`Array.isArray(x)`	`instanceof Array` fails for arrays from another realm (iframe, `vm` context)
Is it an error?	`Error.isError(x)` (ES2026; Node 24 has it) or `x instanceof Error`	Same cross-realm issue for `instanceof`
Was it made by my class, really?	`#brand in x` (private brand check)	Only works inside the class body
Is it "like" a type (has the methods I need)?	**duck typing**: `typeof x?.then === 'function'`	Can't distinguish unrelated objects with the same method names
Built-in tag	`Object.prototype.toString.call(x)` → `"[object Date]"`	Can be customized via `Symbol.toStringTag`

A **realm** is one global environment — each browser tab, iframe or Node `vm` context has its own `Array`, `Object`, `Error` constructors. An array from an iframe has the *iframe's* `Array.prototype` in its chain, so `instanceof Array` in the parent page returns `false`. That's why `Array.isArray` exists.

Duck typing ("if it walks like a duck and quacks like a duck…") means checking for capabilities rather than ancestry. It's how promises interoperate: anything with a `then` method (a *thenable*) is treated like a promise by `await` (Chapter 12).

Self-check

1. Can `instanceof` be `true` for an object the constructor never ran on? → Yes (`Object.create(C.prototype)`, or `Object.setPrototypeOf`).
2. Most robust array check? → `Array.isArray`.
3. `Object.create(null) instanceof Object`? → `false` — `Object.prototype` isn't in its chain.

TL;DR

`instanceof` = "is `C.prototype` in the chain?". Prefer `Array.isArray`, `Error.isError`, `#brand in obj` or duck typing when correctness matters.

9.8 Mixins

What it is

JavaScript classes have **single inheritance**: one `extends`. A **mixin** is a reusable bundle of methods that you "mix into" a class that already has a parent, so unrelated classes can share a capability (serializable — can be turned into text such as JSON; observable — others can subscribe to its changes; timestamped) without a common ancestor.

There are two common styles:

1. **Object mixins** — copy methods onto a prototype with `Object.assign(Target.prototype, mixinA, mixinB)`.
2. **Subclass-factory mixins** — a function that takes a base class and returns `class extends Base { … }`. Because `extends` accepts any expression, you can stack them: `class Duck extends CanSwim(CanFly(Animal)) {}`.

`code/07_mixins_composition.mjs` (first part):

```
// 1) Object.assign mixin — copy methods onto a prototype
const Serializable = { toJSONString() { return JSON.stringify(this); } };
const Timestamped = { touch() { this.updatedAt = '2026-01-01T00:00:00Z'; return this; } };

class Note { constructor(text) { this.text = text; } }
Object.assign(Note.prototype, Serializable, Timestamped);
console.log('assign mixin :', new Note('hi').touch().toJSONString());

// 2) Subclass-factory mixins — functions that take a class and return a subclass
const CanFly = (Base) => class extends Base { fly() { return `${this.name} flies`; } };
const CanSwim = (Base) => class extends Base { swim() { return `${this.name} swims`; } };
class Animal { constructor(name) { this.name = name; } }
class Duck extends CanSwim(CanFly(Animal)) {}
const d = new Duck('Donald');
console.log('factory mixin:', d.fly(), '|', d.swim(), '| instanceof Animal:', d instanceof Animal);
```

Output:

```
assign mixin : {"text":"hi","updatedAt":"2026-01-01T00:00:00Z"}
factory mixin: Donald flies | Donald swims | instanceof Animal: true
```

Trade-offs

	`Object.assign` mixin	Subclass-factory mixin
Can use `super`	✗ (copied methods keep their original home object)	✓
Can use private `#` fields	✗	✓ (inside the factory class)
Shows up in `instanceof`	✗	only for the base class (each call creates a new anonymous class)
Name clashes	later mixin silently overwrites earlier	nearer subclass overrides farther
Copies getters correctly	✗ — `Object.assign` *invokes* getters and copies the value	✓

Common mistakes

- Two mixins defining the same method name — the last one wins silently.
- Mixing state-heavy behaviour: mixins that assume fields exist on the target (`this.items`) create hidden coupling. Document the "requires" contract (the written promise of what each side must provide).

Real-world examples

Libraries such as Lit (web components — custom HTML tags you define yourself) use subclass-factory mixins; Node's `EventEmitter` (a built-in class that lets objects emit named events and register listeners) is often inherited or mixed in for "observable" objects (Chapter 15).

TL;DR

Mixins add capabilities across unrelated classes. Prefer subclass factories (support `super` and private fields); watch for name clashes.

9.9 Composition vs inheritance

What it is

- **Inheritance** models an **is-a** relationship: a `Circle` *is a* `Shape`. Behaviour is received from a parent.

- **Composition** models a **has-a** relationship: a `Car` *has an* `Engine`. Behaviour is obtained by holding other objects (or functions) and **delegating** calls to them.

The classic design advice "favour composition over inheritance" (from the *Design Patterns* book, 1994) applies strongly in JavaScript, where functions and object literals make composition cheap. For the general design principle, see the LLD (low-level design: designing classes and their relationships) handbook's design principles chapter.

The problem it solves

Hierarchies break when real categories overlap. Say `Duck extends Bird` gives every duck `fly()`. Now add a `RoboDuck` that flies and quacks but can't swim, and a `RubberDuck` that only floats. With inheritance you override methods to throw "not supported", which violates the promise the parent made (the Liskov substitution principle, LLD chapter 4). With composition, each object simply gets the capabilities it has. This is often called the **gorilla/banana problem**: you wanted a banana, but inheritance gave you the gorilla holding it and the whole jungle.

`code/07_mixins_composition.mjs` (second part):

```
// 3) Composition — objects HAVE capabilities instead of BEING a subtype
const flyer = (state) => ({ fly: () => `${state.name} flies at ${state.speed} km/h` });
const quacker = (state) => ({ quack: () => `${state.name}: quack` });
function makeRobotDuck(name) {
  const state = { name, speed: 40 };
  return Object.freeze({ ...flyer(state), ...quacker(state) }); // no swim — robots sink
}
const robo = makeRobotDuck('RoboDuck');
console.log('composition  :', robo.fly(), '|', robo.quack(), '| can swim?', 'swim' in robo);

// 4) Composition with classes — delegate to a collaborator
class Engine { start() { return 'engine started'; } }
class Car {
  #engine;
  constructor(engine = new Engine()) { this.#engine = engine; }
  start() { return `car: ${this.#engine.start()}`; }   // delegation
}
class ElectricMotor { start() { return 'motor humming'; } }
console.log('delegation   :', new Car().start(), '|', new Car(new ElectricMotor()).start());

// 5) OLOO (objects linked to other objects) — pure prototype delegation, no classes
const Widget = { init(w) { this.width = w; return this; }, render() { return `widget ${this.width}px`; } };
const ButtonW = Object.create(Widget);
ButtonW.setup = function (w, label) { this.init(w); this.label = label; return this; };
ButtonW.render = function () { return `${Widget.render.call(this)} [${this.label}]`; };
console.log('OLOO         :', Object.create(ButtonW).setup(120, 'OK').render());
```

Output:

```
composition  : RoboDuck flies at 40 km/h | RoboDuck: quack | can swim? false
delegation   : car: engine started | car: motor humming
OLOO         : widget 120px [OK]
```

- The **factory function** style (`makeRobotDuck`) builds objects from small capability functions that close over shared `state` — private by closure, no `this` at all, so nothing can be "lost".
- The `Car` style injects a collaborator through the constructor (**dependency injection**) — swapping `Engine` for `ElectricMotor` needed no subclass, and tests can pass a fake engine.
- **OLOO** ("objects linked to other objects", a term from *You Don't Know JS*) uses `Object.create` directly: objects delegate to other objects with no constructors or classes at all. It shows that "inheritance" in JavaScript is really delegation along the prototype chain.

Trade-offs

	Inheritance (`extends`)	Composition
Relationship	is-a	has-a / can-do
Reuse	automatic, all-or-nothing	pick exactly the parts you need
Coupling (how much one piece depends on another's details)	tight — child depends on parent internals and init order	loose — depends on a small interface (the set of methods it promises)
Changing behaviour at runtime	hard	easy (swap the collaborator)
Testing	need the whole hierarchy	inject fakes
Best when	a true, stable taxonomy (a classification tree of kinds and sub-kinds: `HttpError extends Error`, UI base classes a framework requires)	most application code

When not to use composition

When a framework or platform expects a subclass (custom `Error`s, web components `extends HTMLElement`, `extends EventEmitter`), use inheritance — one shallow level.

In the interview

> **30-second answer:** "Inheritance is is-a and couples you to the parent's implementation; composition is has-a and assembles behaviour from small parts, so it adapts better when requirements cut across a hierarchy. In JavaScript I use `extends` for shallow, framework-mandated or genuinely taxonomic cases like custom errors, and composition — factory functions, injected collaborators, or mixins — for most app code."

- *Q: What's the fragile base class problem?* A: A change in a parent class (a new method, a changed call order) silently breaks subclasses that depended on its internals.
- *Q: How would you share "logging" across unrelated classes?* A: Inject a logger object (composition) or use a mixin; don't create a `Loggable` base class they all must extend.

Self-check

1. "A `Car` has an `Engine`" — which approach? → Composition.
2. Why is a factory function immune to lost-`this` bugs? → Its methods use closed-over variables, not `this`.
3. When is inheritance the right call in JS? → Shallow, stable is-a relationships or when a platform requires it (errors, custom elements).

TL;DR

Default to composition (factories, delegation, injection); use one shallow level of `extends` where the platform or a real taxonomy demands it.

Chapter 9 TL;DR

- **`this`** is set by the call site: `new` > `call/apply/bind` > `obj.m()` > plain call (`undefined` in strict mode). **Arrows** take `this` from where they're written and can't be rebound.
- **`call`** / **`apply`** invoke now with a chosen `this`; **`bind`** returns a permanently bound function (also good for partial application).

- Every object has a `[[Prototype]]`; missing reads walk the **prototype chain** to `null`. Writes create own properties (shadowing). Use `Object.create`, `Object.getPrototypeOf`, `Object.hasOwn`.
- `new Fn()` creates an object linked to `Fn.prototype`, runs `Fn` with it as `this`, returns it.
- **Classes** wrap that pattern: strict, `new`-only, non-enumerable methods, TDZ. ES2022 adds **fields**, `#private` members, **static blocks** and `#x in obj`.
- `extends` / `super`: `super()` before `this`; both instance and static methods are inherited; built-ins (`Array`, `Error`) subclass properly.
- `instanceof` only checks the chain — prefer `Array.isArray`, `Error.isError`, brand checks or duck typing when it matters.
- **Mixins** share capabilities across unrelated classes; **composition** (has-a) usually beats deep inheritance (is-a).

Next: organize all this code into files and packages — Chapter 10: Modules and Tooling.

Chapter 10 — Modules and Tooling

A real JavaScript program is hundreds of files and thousands of third-party packages (bundles of reusable code written by other people and downloaded into your project). **Modules** let each file keep its own private scope and share only what it chooses to export. **Tooling** — npm, bundlers, transpilers, linters and formatters, each explained in its own section below — turns those files and packages into something a browser or server can run, and keeps the code consistent. This chapter covers the language feature first (ES modules), then how Node runs both module systems, then the everyday tool chain.

What you'll learn

1. Why modules exist and how the module systems evolved (scripts → IIFE → CommonJS → ES modules).
2. Every `import` / `export` form; default vs named exports; barrels; side-effect imports.
3. How ES modules load: the three phases, hoisting, live bindings, singletons, circular imports.
4. Dynamic `import()`, top-level `await`, `import.meta`, JSON modules with import attributes.
5. Modules in the browser: `<script type="module">`, bare specifiers, import maps.
6. CommonJS vs ESM in Node: how Node decides, interop, `require(esm)`, `createRequire`, package `"exports"`.
7. npm and `package.json`: dependencies, scripts, `npx`, `node --run`, how `node_modules` resolution works.
8. Semantic versioning, version ranges and lockfiles (with a runnable range checker).
9. Bundlers (Vite, esbuild, webpack, Rollup/Rolldown): tree shaking, code splitting, source maps, dev servers.
10. Transpilers and polyfills; linters and formatters (ESLint, Prettier).

Before you start. You should know functions and closures (Chapter 4) — the pre-module "module pattern" is a closure (a function that keeps access to the variables around where it was created). Promises and `await` are covered properly in Chapter 12; here we only need that `await` pauses until a promise (an object standing for a result that arrives later) settles (finishes with a value or an error).

All runnable code is in `code/` — a small multi-file ES module project. Files whose first line is `// tools_run: skip` are helper modules imported by the numbered programs. Outputs are real (Node 24). **Commands for npm, bundlers and linters are shown as not-run snippets** (they need the network or a full project); their output is described, not pasted.

10.1 Why modules?

What it is

A **module** is a file whose top-level variables are private to that file. It explicitly **exports** the values it wants to share and explicitly **imports** what it needs from other modules. The set of modules connected by imports is the **module graph**; the first module you run is the **entry point**.

Analogy: departments in a company. Each department (module) has its own internal files; it publishes a few official forms (exports); other departments must request those forms by name (imports). Nobody rummages through anybody else's drawers.

The problem it solves

Early web pages loaded several `<script>` files that all shared **one global scope**. Any `var` or function at the top of any file became a global, so:

- **Name collisions** — two libraries both defining `$` or `init` silently overwrote each other.
- **Hidden dependencies** — `app.js` only worked if `utils.js` had been loaded first; script order in HTML was the dependency system.
- **No encapsulation** (keeping a piece of code's internals hidden behind a small public surface) — anything could change anything.

How the solutions evolved

Era	Approach	How it works	Still seen?
~2000s	Global scripts + namespaces (one global object holding everything, to reduce name clashes)	`var MyApp = MyApp || {}; MyApp.utils = …`	Legacy pages
~2008+	**IIFE / module pattern**	A function called immediately; a closure hides private state and returns a public object (Chapter 4)	Output of old bundlers
2009	**CommonJS** (Node)	`require()` / `module.exports`, synchronous (each file is fully loaded before the next line runs), one file = one module	Very common in Node packages
2011	AMD (RequireJS), later UMD	Asynchronous (loads in the background and calls back when ready) `define()` for browsers; UMD wraps a file to work as AMD, CommonJS or a global	Old libraries
ES2015	**ES modules (ESM)**	`import` / `export` built into the language; static; async-friendly	**The standard** — browsers and Node

ESM is the only one that is part of the language itself, works natively in browsers and Node, and can be statically analyzed (understood by reading the code, without running it) by tools. Use it for new code.

> **TL;DR**
>
> Modules give each file a private scope and explicit imports/exports. ES modules (ES2015) are the standard; CommonJS remains common in Node.

10.2 `export` and `import` syntax

Export forms

A module can have any number of **named exports** and at most one **default export**.

`code/math.mjs` — named exports:

```
// tools_run: skip — helper module imported by other examples
export const TAU = 2 * Math.PI;                // named export (declaration form)
export function add(a, b) { return a + b; }
function square(x) { return x * x; }
function cube(x) { return x ** 3; }
export { square, cube as toThePowerOfThree };   // export list, with renaming
console.log('[math.mjs] evaluated');            // runs once, however many importers
```

`code/logger.mjs` — a default export plus a named one:

```
export default function log(level, msg) {       // the default export
  return `[${level.toUpperCase()}] ${msg}`;
}
export const LEVELS = ['debug', 'info', 'warn', 'error'];
```

`code/barrel.mjs` — re-exports (a **barrel** is a file that gathers several modules' exports into one import point):

```
export * from './math.mjs';                          // re-export all named exports
export { default as log, LEVELS } from './logger.mjs'; // re-export a default under a name
export * as counterNS from './counter.mjs';          // re-export a namespace (ES2020)
```

Export form	Example
Declaration	`export const x = 1;` `export function f() {}` `export class C {}`
List (after declaring)	`export { a, b };`
Rename	`export { a as alpha };`
Default (expression)	`export default 42;` `export default { … };`
Default (declaration, may be anonymous)	`export default function () {}` `export default class {}`
Name something the default	`export { x as default };`
Re-export	`export { a } from './m.mjs';` `export * from './m.mjs';` `export * as ns from './m.mjs';`

`export *` does **not** re-export the default export — re-export it by name, as `barrel.mjs` does.

Import forms

`code/01_import_export.mjs`:

```
// Every static import form. Imports are hoisted: they load before any code here runs.
import './side_effect.mjs';                      // side-effect only
import log, { LEVELS } from './logger.mjs';      // default + named in one statement
import { add, square as sq, TAU } from './math.mjs'; // named, with renaming
import * as MathNS from './math.mjs';            // namespace object
import { toThePowerOfThree, counterNS, log as log2 } from './barrel.mjs';

console.log('named   :', add(2, 3), sq(4), TAU.toFixed(4));
console.log('default :', log('info', 'server started'), '| levels:', LEVELS.join(','));
console.log('ns keys :', Object.keys(MathNS));
console.log('barrel  :', toThePowerOfThree(2), typeof counterNS.increment, log2 === log);
console.log('polyfill:', globalThis.polyfilled);
```

Output:

```
[side_effect.mjs] patched globalThis
[math.mjs] evaluated
named   : 5 16 6.2832
default : [INFO] server started | levels: debug,info,warn,error
ns keys : [ 'TAU', 'add', 'square', 'toThePowerOfThree' ]
barrel  : 8 function true
polyfill: true
```

Import form	What you get
`import x from './m.mjs'`	the default export, under any name you choose
`import { a, b } from './m.mjs'`	named exports (names must match)
`import { a as z } from './m.mjs'`	named export, renamed locally
`import * as ns from './m.mjs'`	a **namespace object** with every named export as a property (the default is `ns.default`)
`import x, { a } from './m.mjs'`	default + named together
`import './m.mjs'`	runs the module for its side effects (things it does on load besides exporting — e.g. polyfills, which add missing built-in features; CSS in bundlers; registering elements)

Notice the output order: `side_effect.mjs` and `math.mjs` printed **before** any line of the importing file ran, even though `math.mjs` is imported by three statements. Imports are processed before the module body, and every module is evaluated exactly once.

A **module specifier** is the string after `from`:

- **Relative** — `'./math.mjs'`, `'../lib/x.js'`. In browsers and Node ESM you must include the file extension; there's no automatic `.js` guessing (CommonJS and bundlers do guess).
- **Absolute URL** — `'https://cdn.example.com/lib.js'` (browsers; a CDN is a network of servers that delivers files from close to the user — see System Design Fundamentals).
- **Bare** — `'react'`, `'lodash-es'`: a package name. Node resolves it in `node_modules` (the folder where installed packages live); browsers need an import map or a bundler (§10.7).
- **Built-in** — `'node:fs'`, `'node:path'` in Node. The `node:` prefix makes it unambiguous that this is a core module.

Default vs named exports

	Named exports	Default export
Importer's name	Must match (rename with `as`)	Anything — and different files may call it different things
Refactoring (restructuring code without changing behaviour) / IDE (code editor such as VS Code) auto-import	Rename-safe, auto-import finds it	Weaker: the name isn't tied to the export
Tree shaking (removing unused code from the final build, §10.11)	Fine-grained	Works, but a default *object* of many functions is all-or-nothing
Typical use	Utility modules, libraries	One main thing per file: a React component, a class, a config

Many teams prefer named exports for consistency (the `eslint-plugin-import` package even has an `import/no-default-export` rule to enforce it); frameworks such as Next.js (a popular React framework for full websites) require default exports for page files. Either is fine — be consistent within a project.

Common mistakes

- `import { default } from …` — `default` is a reserved word; write `import x from …` or `import { default as x } from …`.
- Forgetting the extension (`'./math'`) in native ESM → `ERR_MODULE_NOT_FOUND` in Node.
- Big barrels in application code: importing one function from `index.js` can make tools and test runners (programs that find and run your automated tests) load every re-exported module. Import from the specific file in hot paths (code that runs very often or on startup).
- Mixing up `export default const x = 1` (syntax error — `export default` takes an expression, or a function/class declaration) with `export const x = 1`.

Self-check

1. How many default exports can a module have? → At most one.
2. Does `export * from './m.mjs'` include `m`'s default? → No.
3. What's in `ns` after `import * as ns`? → All named exports as read-only properties, plus `default` if there is one.

TL;DR

Named exports: many per module, name must match. Default: one, importer picks the name. `import * as ns` gives a namespace object; `import './x'` runs side effects only. Always write file extensions in native ESM.

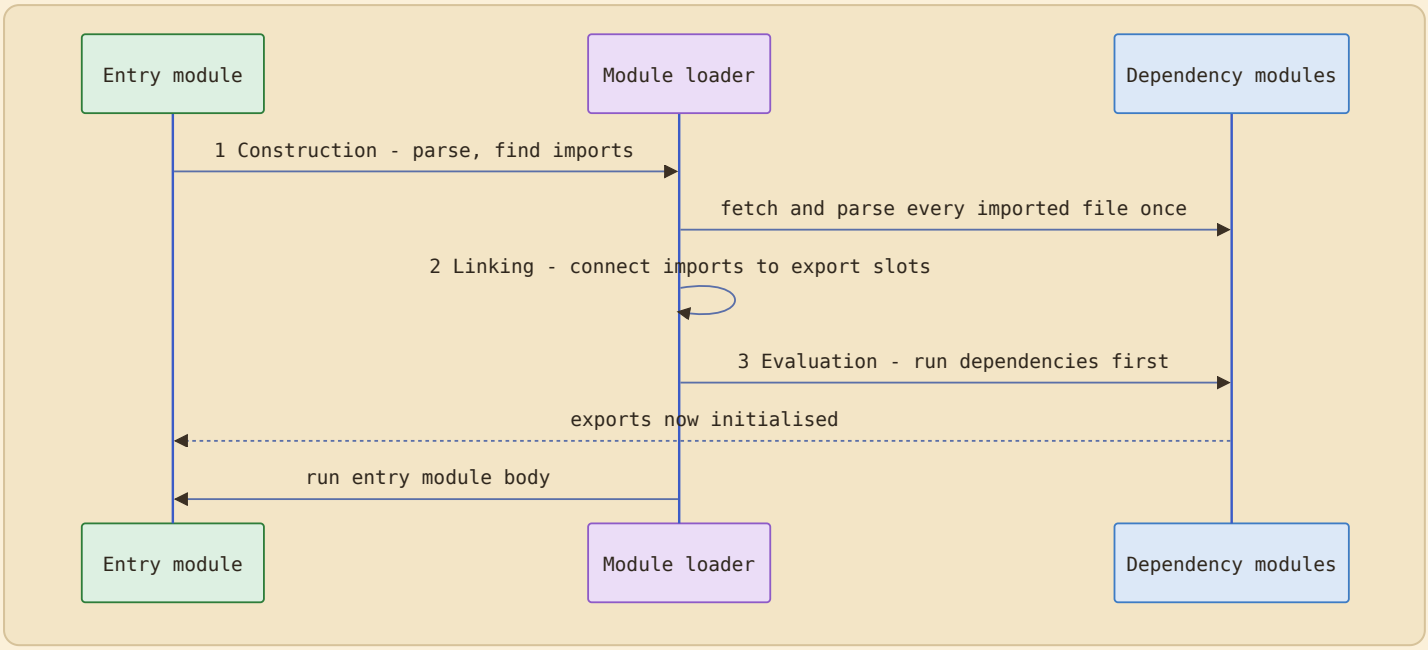
10.3 How ES modules load

What it is

ES modules are **static**: `import` and `export` may only appear at the top level of a module (not inside `if` or functions), and specifiers must be string literals. That lets the engine read the whole dependency graph *before running any code*.

How it works — three phases

1. **Construction (parse)** — starting at the entry point, fetch each file, parse it, find its `import` statements, fetch those, and so on until the whole graph is known. Nothing executes yet. Each module URL is recorded in a **module map**, so a module is fetched once no matter how many times it's imported.
2. **Linking (instantiation)** — create each module's environment and wire every import to the matching export's variable slot. Imports are *references to the exporter's bindings*, not copies. Missing named exports fail here, before any code runs (`SyntaxError: The requested module … does not provide an export named …`).
3. **Evaluation** — run each module's top-level code exactly once, **dependencies first** (a depth-first, post-order walk: children before parents, siblings in the order they were imported).



Consequences you can observe:

Property	Meaning
Hoisting	Imports are resolved before the body runs, wherever the `import` line appears in the file.
Singletons	Each module is evaluated once per program (per realm — one separate global environment, such as a browser tab or iframe); every importer shares the same instance and state.
Live bindings	An import is a live, **read-only** view of the exporter's variable. If the exporter changes it, importers see the change.
Strict mode	Module code is always strict (no `'use strict'` needed).
Own scope	Top-level `const` / `let` / `function` are module-scoped, not globals. `this` at top level is `undefined`.
Deferred	In browsers, module scripts don't block HTML parsing (§10.7).

`code/counter.mjs` and `code/02_live_bindings.mjs`:

```
// counter.mjs
export let count = 0;
export function increment() { count++; }
```

```
import { count, increment } from './counter.mjs';
import * as ns from './math.mjs';
import * as again from './math.mjs';     // same module -> evaluated once, same namespace object

console.log('before     :', count);
increment(); increment();
console.log('after x2   :', count, '(the import sees the new value)');

try {
  count = 100;                           // imports are read-only
} catch (e) {
  console.log('assign     :', e.name, '-', e.message);
}

console.log('same ns?   :', ns === again);
console.log('ns tag     :', Object.prototype.toString.call(ns));
console.log('ns sealed  :', Object.isSealed(ns), '| frozen:', Object.isFrozen(ns));
try { ns.add = null; } catch (e) { console.log('ns write   :', e.name, '-', e.message); }

// Module scope: strict mode, `this` is undefined, top-level bindings are not globals
console.log('this       :', this);
const topLevel = 1;
console.log('global?    :', 'topLevel' in globalThis);
try { undeclared = 5; } catch (e) { console.log('strict     :', e.name, '-', e.message); }
```

Output:

```
[math.mjs] evaluated
before     : 0
after x2   : 2 (the import sees the new value)
assign     : TypeError - Assignment to constant variable.
same ns?   : true
ns tag     : [object Module]
ns sealed  : true | frozen: false
ns write   : TypeError - Cannot assign to read only property 'add' of object '[object Module]'
this       : undefined
global?    : false
strict     : ReferenceError - undeclared is not defined
```

- `count` changed from 0 to 2 in the importer — a CommonJS `const { count } = require(...)` would have copied `0` and stayed `0`.
- Assigning to an import throws as if it were a `const`, even though the exporter declared it with `let`. Only the exporting module can change it (here via `increment`).
- The namespace object is sealed and its properties are non-writable; it reports "not frozen" only because its properties stay live (their values can still change from inside the module).

Circular imports

A **circular dependency** is when A imports B and B (directly or indirectly) imports A. ESM allows it: while evaluating, the loader notices the cycle and runs one module before the other has finished. The rule: **top-level code must not use an imported `let`/`const`/`class` binding that hasn't been initialized yet** — it's in its temporal dead zone (TDZ). Using imports inside functions that run later is fine, and function declarations are initialized during linking, so they're usable early.

`code/07_circular_imports.mjs` loads two tiny cycles:

```
// cyc_a.mjs
import { b } from './cyc_b.mjs';
export const a = 'A';
console.log('[cyc_a] sees', b);
```

```
// cyc_b.mjs
import { a } from './cyc_a.mjs';
export const b = `B knows ${a}`; // a in TDZ
```

```
// cyc_safe_a.mjs                          // cyc_safe_b.mjs
import { describeB } from './cyc_safe_b.mjs'; import { nameA } from './cyc_safe_a.mjs';
export function nameA() { return 'A'; }       export function describeB() { return `B calls ${nameA()}`; }
export const result = describeB();
```

```
try {
  await import('./cyc_a.mjs');
} catch (e) {
  console.log('unsafe cycle:', e.name, '-', e.message);
}
const { result } = await import('./cyc_safe_a.mjs');
console.log('safe cycle  :', result);
```

Output:

```
unsafe cycle: ReferenceError - Cannot access 'a' before initialization
safe cycle  : B calls A
```

Starting from `cyc_a`, the loader evaluates its dependency `cyc_b` first; `cyc_b`'s top level reads `a`, but `cyc_a` hasn't run yet. In the safe version, `nameA` is a hoisted function declaration, so `describeB()` can call it. Best fix in real code: **break the cycle** — move the shared piece into a third module that both import.

In the interview

30-second answer: "ES modules are static: the loader parses the whole graph, links imports to exports, then evaluates each module once, dependencies first. Imports are hoisted, read-only live bindings — not copies — and modules are singletons in strict mode with their own scope. That static structure is what enables tree shaking. Circular imports work as long as top-level code doesn't touch a binding that's still in its TDZ."

- *Q: Difference between ESM live bindings and CommonJS exports?* A: CommonJS `require` returns the `module.exports` object; destructuring copies values at that moment. ESM imports stay connected to the exporter's variable.
- *Q: Why can't you put `import` inside an `if`?* A: Static imports must be known before execution; use `import()` for conditional loading.

Self-check

1. Two files import `config.mjs`. How many times does its top-level code run? → Once.
2. Can an importer reassign an imported binding? → No — `TypeError`.
3. In which phase is a missing named export reported? → Linking, before any code runs.

TL;DR

Parse the graph → link bindings → evaluate once, dependencies first. Imports are hoisted, live and read-only; modules are strict singletons. Avoid top-level use of bindings in cycles.

10.4 Dynamic `import()`

What it is

`import(specifier)` (ES2020) looks like a function call but is special syntax. It can appear anywhere — inside `if`, functions, event handlers — accepts a computed string, and returns a **promise** that resolves to the module's namespace object.

Why it matters

- **Lazy loading / code splitting** — load a heavy chart library only when the user opens the chart tab. Bundlers turn each `import()` into a separate file (**chunk**) downloaded on demand (§10.11).
- **Conditional loading** — pick a module by platform, feature flag (a switch that turns a feature on or off) or locale (language + region setting) (`import(\`./locales/${lang}.mjs`)`).
- **From CommonJS** — the only way for a CommonJS file to load an ES module that uses top-level `await`.

`code/03_dynamic_import.mjs`:

```
import * as staticMath from './math.mjs';

const verbose = process.env.VERBOSE !== '0';   // decide at runtime whether to load the logger
if (verbose) {
  const { default: log } = await import('./logger.mjs');  // default export is the "default" key
  console.log(log('debug', 'logger loaded lazily'));
}

const name = 'math';
const dyn = await import(`./${name}.mjs`);                // computed specifier
console.log('same instance as static import?', dyn === staticMath);

const [m, l] = await Promise.all([import('./math.mjs'), import('./logger.mjs')]);  // in parallel
console.log('parallel :', m.add(1, 1), l.LEVELS.length);

try {
  await import('./does-not-exist.mjs');                   // failure = rejected promise
} catch (e) {
  console.log('missing  :', e.code);
}

const { basename } = await import('node:path');
console.log('node:path:', basename('/a/b/report.pdf'));
```

Output:

```
[math.mjs] evaluated
[DEBUG] logger loaded lazily
same instance as static import? true
parallel : 2 4
missing  : ERR_MODULE_NOT_FOUND
node:path: report.pdf
```

Trade-offs

	Static `import`	**Dynamic** `import()`
Where	top level only	anywhere
Specifier	string literal	any expression
Timing	before the module body runs	when that line runs (async)
Tree shaking / analysis	full	limited — tools can't know a fully computed path
Errors	whole graph fails to load	a catchable rejected promise

Common mistakes

- Forgetting that the default export is under `.default`: `const mod = await import('./x.mjs'); mod.default(...)`.
- Fully dynamic paths (`import(userInput)`) — bundlers can't pre-build them, and it's a security risk if the input isn't trusted.

TL;DR

`import()` loads modules on demand and returns a promise of the namespace. Use it for code splitting and conditional loading; keep static imports for everything else.

10.5 Top-level `await`

What it is

Since ES2022, `await` may be used at the **top level of an ES module** (not in classic scripts, and not in CommonJS). A module that awaits is an **async module**: modules that import it wait until it finishes evaluating before they run.

`code/slow_config.mjs` and `code/04_top_level_await.mjs`:

```
// slow_config.mjs
console.log('[slow_config] start loading');
const loadSettings = () => new Promise((resolve) => setTimeout(() => resolve({ theme: 'dark', retries: 3 }), 50));
export const settings = await loadSettings();    // importers wait until this resolves
console.log('[slow_config] done');
```

```
// 04_top_level_await.mjs
console.log('main body starts (after all static imports finished)');
import { settings } from './slow_config.mjs';
console.log('settings:', settings);

const results = await Promise.all([1, 2, 3].map(async (n) => n * 10));
console.log('awaited at top level:', results);
```

Output:

```
[slow_config] start loading
[slow_config] done
main body starts (after all static imports finished)
settings: { theme: 'dark', retries: 3 }
awaited at top level: [ 10, 20, 30 ]
```

The `import` line is written *after* the first `console.log`, yet the dependency ran completely first — imports are hoisted, and the importer waited for `slow_config`'s `await`.

Why it matters / real-world examples

- Loading configuration, a WebAssembly module (fast compiled code, often from C or Rust, that browsers and Node can run), or opening a database connection before exporting it.
- Scripts and CLIs (command-line programs): no more wrapping everything in `(async () => { … })()`.
- Fallbacks: `let lib; try { lib = await import('fast-lib'); } catch { lib = await import('./slow-fallback.mjs'); }`.

When not to use it

- **In widely imported library modules** — every importer (and their importers) is delayed, and a slow `await` blocks the app's startup. Prefer exporting an async function (`export async function connect()`), and let the application decide when to wait.
- **Modules loaded via `require()`** — Node throws `ERR_REQUIRE_ASYNC_MODULE` if the ESM graph contains top-level `await` (§10.8).

TL;DR

Top-level `await` works in ES modules only; importers wait for it. Great for entry points and scripts, risky in shared libraries.

10.6 `import.meta` and JSON modules

What it is

- `import.meta` is an object holding metadata (data about the module itself, rather than its contents) about the *current* module, filled in by the host (browser or Node). The standard part is `import.meta.url` (the module's full URL) and `import.meta.resolve(specifier)` (turn a specifier into a URL relative to this module).
- **Import attributes** (ES2025) add a `with { … }` clause to an import that tells the host how to interpret the file. The first standardized use is **JSON modules**: `import data from './data.json' with { type: 'json' }`. The attribute is required so a server can't trick a page into *executing* a file it expected to be data.

`code/05_import_meta_and_json.mjs` (with `code/data.json`):

```
import config from './data.json' with { type: 'json' };
import path from 'node:path';

console.log('json     :', config.app, config.version, config.features);
console.log('url      :', import.meta.url.startsWith('file://'), path.basename(import.meta.url));
console.log('filename :', path.basename(import.meta.filename));   // Node 20.11+
console.log('dirname  :', path.basename(import.meta.dirname));
console.log('resolve  :', path.basename(import.meta.resolve('./math.mjs')));
console.log('is main? :', import.meta.main);                     // Node 24.2+, early-stage feature
```

Output:

```
json     : notes 2.1.0 [ 'search', 'tags' ]
url      : true 05_import_meta_and_json.mjs
filename : 05_import_meta_and_json.mjs
dirname  : code
resolve  : math.mjs
is main? : true
```

(Only base names are printed so the output doesn't depend on where the book lives on disk.)

Property	Where	Replaces
`import.meta.url`	browsers + Node	—
`import.meta.resolve()`	browsers + Node (20.6+ unflagged — usable without a special command-line switch)	`require.resolve`
`import.meta.filename` / `.dirname`	Node 20.11+	CommonJS `__filename` / `__dirname`
`import.meta.main`	Node 24.2+ (early development)	`require.main === module`

A JSON module's default export is the parsed value; it has no named exports. It is evaluated once, so all importers share (and could mutate) the same object — treat it as read-only. For files outside the module graph (user uploads, data read at runtime), use `fs` or `fetch` + `JSON.parse` instead (Chapter 5, Chapter 15).

Before import attributes existed, there was an earlier `assert { type: 'json' }` syntax; it was replaced by `with` and removed from V8/Node, so update old code that uses `assert`.

TL;DR

`import.meta.url` identifies the current module (Node adds `filename`/`dirname`). Import JSON with `with { type: 'json' }` (ES2025).

10.7 Modules in the browser

How it works

```
<script type="module" src="/js/main.js"></script>
<script type="module">
  import { add } from './js/math.js';
  document.title = `2 + 3 = ${add(2, 3)}`;
</script>
```

(HTML — not run.) Differences from a classic `<script>`:

	Classic `<script>`	`<script type="module">`
Scope	top-level `var`/functions become globals	module scope
Strict mode	opt-in	always
Top-level `this`	`window`	`undefined`
Loading	blocks HTML parsing unless `defer`/`async`	**deferred by default** (runs after parsing, in order); `async` also allowed
Executes	every time it's included	once per URL
Cross-origin (from a different domain)	loads without CORS	must be served with CORS (Cross-Origin Resource Sharing: headers in which a server says which other sites may use its files) headers
`file://` pages	work	blocked by browsers — use a local dev server (a small web server on your own machine for development)
`import` / `export` / top-level `await`	✗	✓

`<script nomodule>` runs only in browsers that *don't* understand modules; it was used to ship a fallback bundle to old browsers and is rarely needed today — all evergreen browsers (browsers that update themselves automatically) have supported modules since about 2018.

Bare specifiers and import maps

Browsers can't resolve `import { h } from 'preact'` on their own — there is no `node_modules` on the web. An **import map** (supported in all major browsers since 2023) tells the browser what bare names mean:

```
<script type="importmap">
{
  "imports": {
    "preact": "https://cdn.jsdelivr.net/npm/preact@10/dist/preact.module.js",
    "utils/": "/js/utils/"
  }
}
</script>
<script type="module">
  import { h } from 'preact';
  import { slugify } from 'utils/strings.js';
</script>
```

(HTML — not run.) Import maps are useful for small apps and prototypes without a build step (an automated pass that processes source files before release). Larger apps still use a bundler (§10.11) for tree shaking, minification (shrinking code by removing spaces and shortening names) and fewer network requests. The browser side of the platform (fetching, `<script>` loading, CORS) is covered in Chapter 14.

TL;DR

`<script type="module">` = module scope, strict, deferred, once per URL, CORS required, no `file://`. Import maps let browsers resolve bare package names.

10.8 CommonJS vs ES modules in Node

What it is

CommonJS (CJS) is Node's original module system: `require(path)` synchronously loads and runs a file and returns its `module.exports` object. Node wraps every CJS file in a function — `(function (exports, require, module, __filename, __dirname) { … })` — which is where those five "globals" come from. It's still everywhere on npm, so you must be able to read, write and mix both systems.

How Node decides which system a file uses

1. `.mjs` → always ESM. `.cjs` → always CommonJS.
2. `.js` → look at the nearest `package.json`: `"type": "module"` → ESM; `"type": "commonjs"` → CJS.
3. `.js` with **no** `"type"` field → CommonJS, *unless* the file contains ESM-only syntax (`import`/`export`, top-level `await`); since Node 22.7 Node then re-runs it as ESM (**syntax detection**). This is a convenience — set `"type"` explicitly in real projects so nothing is guessed.

This book's examples use `.mjs`/`.cjs` so there's never any doubt.

Side-by-side comparison

	CommonJS	ES modules
Syntax	`const x = require('./x')` / `module.exports = …` / `exports.a = …`	`import` / `export`
Loading	synchronous, at runtime, can be inside `if`	static graph, async-capable; `import()` for dynamic
Exports	a single object (copied values when destructured)	live, read-only bindings
Top-level `await`	✗	✓
Strict mode	opt-in	always
Top-level `this`	`module.exports`	`undefined`
`__dirname` / `__filename`	✓	`import.meta.dirname` / `.filename`
File extensions	optional (`require('./x')` finds `x.js`, `x.json`, `x/index.js`)	required for relative paths
JSON	`require('./data.json')`	`import d from './data.json' with { type: 'json' }`
Cache	`require.cache` (can be deleted to reload)	internal module map (no official reload)
Tree shaking	poor (dynamic)	good (static)
Browsers	✗ (needs a bundler)	✓ native

Interop

- **ESM importing CJS** — works. The **default import** is `module.exports`. **Named imports** also work when Node can detect them by statically scanning the CJS source (it recognizes patterns like `exports.name = …` and `module.exports = { name, … }`).

- **CJS requiring ESM** — `require(esm)` is enabled by default since Node 22.12 / 20.19 (no warning since 23.5 / 22.13; officially stable in 25.4). It returns the module namespace object. It **fails with `ERR_REQUIRE_ASYNC_MODULE`** if the ESM graph uses top-level `await` — use `await import()` then.
- **`createRequire(import.meta.url)`** gives an ES module a real `require` function — handy for loading CJS-only files or using `require.resolve`.

`code/legacy.cjs`, `code/uses_esm.cjs` and `code/06_commonjs_interop.mjs`:

```
// legacy.cjs — CommonJS
const path = require('node:path');
console.log('[legacy.cjs] this === module.exports:', this === module.exports, '| __filename:',
path.basename(__filename));
let calls = 0;
function greet(name) { calls++; return `hello ${name} (call ${calls})`; }
module.exports = { greet, get calls() { return calls; } };
module.exports.version = '1.0.0';
```

```
// uses_esm.cjs — CommonJS requiring an ES module
const math = require('./math.mjs');
module.exports = { sum: math.add(20, 22), keys: Object.keys(math) };
```

```
// 06_commonjs_interop.mjs
import legacy from './legacy.cjs';                  // default import = module.exports
import { greet } from './legacy.cjs';               // named import (detected by static analysis)
import { createRequire } from 'node:module';

console.log('default  :', legacy.greet('ESM'), '| version', legacy.version);
console.log('named    :', greet('named'));

const require = createRequire(import.meta.url);     // a require() usable inside ESM
const again = require('./legacy.cjs');
console.log('cached   :', again === legacy, '| calls so far', again.calls);

const fromCjs = require('./uses_esm.cjs');          // CJS that itself require()s an ESM file
console.log('require(esm):', fromCjs);

console.log('cache key:', Object.keys(require.cache).some((k) => k.endsWith('legacy.cjs')));
```

Output:

```
[legacy.cjs] this === module.exports: true | __filename: legacy.cjs
default  : hello ESM (call 1) | version 1.0.0
named    : hello named (call 2)
cached   : true | calls so far 2
[math.mjs] evaluated
require(esm): { sum: 42, keys: [ 'TAU', 'add', 'square', 'toThePowerOfThree' ] }
cache key: true
```

- `legacy.cjs` ran once; the ESM import and the later `require` share one cached instance (the call counter continued from 1 to 2).
- `version` was added *after* the `module.exports = {…}` assignment, so static analysis didn't list it as a named export — but it's on the default import. When in doubt with CJS packages, use the default import.

Common mistakes

- `exports = { a }` (reassigning the `exports` shortcut) exports nothing — only `module.exports = …` or `exports.a = …` work.
- `ReferenceError: require is not defined in ES module scope` — you're in ESM; use `import` or `createRequire`.
- `SyntaxError: Cannot use import statement outside a module` — Node treated the file as CJS (a `"type": "commonjs"` somewhere up the tree, or a `.cjs` file); rename to `.mjs` or set `"type": "module"`.
- `__dirname is not defined` in ESM — use `import.meta.dirname`.
- The **dual package hazard**: a package that ships both CJS and ESM builds may get loaded *twice* (once per system) — two copies of its state, and `instanceof` checks that fail across them.

Package entry points: `"exports"`

A package's `package.json` can declare exactly which files outsiders may import and which file each module system gets:

```
{
  "name": "tiny-date",
  "type": "module",
  "exports": {
    ".": { "import": "./dist/index.js", "require": "./dist/index.cjs" },
    "./format": "./dist/format.js",
    "./package.json": "./package.json"
  }
}
```

(Not run.) With `"exports"` present, `import 'tiny-date/dist/secret.js'` fails with `ERR_PACKAGE_PATH_NOT_EXPORTED` — the package's internals are encapsulated. The older `"main"` field only named a single entry file and exposed everything. (`"imports"`, with keys starting `#`, defines private aliases *inside* a package, e.g. `import db from '#db'`.)

In the interview

30-second answer: "CommonJS is Node's original synchronous `require` / `module.exports` system; ES modules are the language standard with static `import` / `export`, live bindings, top-level `await` and browser support. Node picks the system by extension (`.mjs` / `.cjs`) or the `type` field in `package.json`. ESM can import CJS (default = `module.exports`), and modern Node can `require()` ESM unless it uses top-level `await`. New code should be ESM."

- *Q: Why can't `require()` load an ESM file with top-level await?* A: `require` is synchronous and must return immediately; an async module can't finish synchronously.
- *Q: What does `"exports"` do?* A: Defines the public entry points and conditional builds (`import` vs `require`), blocking deep imports of internal files.

Self-check

1. A `.js` file in a folder whose `package.json` has `"type": "module"` — which system? → ESM.
2. How do you get `__dirname` in ESM? → `import.meta.dirname` (Node 20.11+), or `path.dirname(fileURLToPath(import.meta.url))` on older versions.
3. What does `import x from './legacy.cjs'` give you? → The CJS `module.exports` value.

TL;DR

`.mjs` / `.cjs` or `"type"` decides. CJS: sync `require`, copied values, `__dirname`. ESM: static, live, strict, TLA. They interoperate; prefer ESM for new code and declare entry points with `"exports"`.

10.9 npm and `package.json`

What it is

- **npm** is three things: the **registry** (npmjs.com, millions of public packages), the **CLI** (command-line interface — the `npm` command, installed with Node) and the company that runs them.
- A **package** is a folder with a `package.json` file. A package you depend on is a **dependency**; the dependencies of your dependencies are **transitive dependencies**.
- **`package.json`** is the manifest of your project: its name, version, module type, scripts and dependencies.

Alternatives with the same registry and a compatible `package.json`: **pnpm** (content-addressed store (each file stored once, named by its contents), strict `node_modules`, saves disk space), **Yarn**, and **Bun** (a runtime with a built-in package manager). Pick one per project and commit its lockfile (the file recording exact installed versions, §10.10).

A typical `package.json`

```
{
  "name": "notes-app",
  "version": "2.1.0",
  "private": true,
  "type": "module",
  "engines": { "node": ">=22" },
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "test": "node --test",
    "lint": "eslint .",
    "format": "prettier --write .",
    "check": "npm run lint && npm test"
  },
  "dependencies": {
    "date-fns": "^4.1.0"
  },
  "devDependencies": {
    "eslint": "^10.0.0",
    "prettier": "^3.3.0",
    "vite": "^8.0.0"
  }
}
```

(Not run; version numbers are examples.)

Field	Meaning
`name`, `version`	Identity. Required if you publish. `version` follows semver (§10.10).
`private: true`	Prevents accidental `npm publish`.
`type`	`"module"` → `.js` files are ESM.
`main` / `exports`	Entry points for people importing your package (§10.8).
`scripts`	Named commands: `npm run <name>`; `npm test` and `npm start` are shortcuts.
`dependencies`	Needed at runtime (installed for anyone who installs your package).
`devDependencies`	Needed only to develop/build/test (linters, bundlers, test runners).
`peerDependencies`	"I work with *your* copy of X" — plugins (add-ons that extend another tool) declare the host they plug into (e.g., a React component library declares `react`). npm 7+ installs them automatically.
`optionalDependencies`	Install failures are tolerated.
`engines`	Supported Node versions (advisory unless enforced).
`bin`	Command-line executables the package provides.
`files`	What to include when publishing.

Everyday commands (not run)

```
npm init -y                    # create package.json with defaults
npm install                    # install everything in package.json (and update the lockfile if needed)
npm install date-fns           # add a runtime dependency (saved as "^<latest>")
npm install -D eslint prettier # add dev dependencies (--save-dev)
npm uninstall date-fns         # remove
npm ci                         # clean, exact install from package-lock.json (CI servers)
npm run build                  # run a script; npm test / npm start need no "run"
npm run test -- --watch        # pass extra args to the script after --
npx eslint .                   # run a package's binary (local first, else download temporarily)
npm outdated                   # which dependencies have newer versions
npm update                     # upgrade within the allowed ranges
npm audit                      # check for known vulnerabilities (npm audit fix to patch)
npm ls react                   # why / where is a package in the tree
npm version patch              # bump version 2.1.0 -> 2.1.1, commit and tag
node --run test                # Node 22+: run a script without npm's startup cost
node --watch server.mjs        # restart on file change (stable since Node 22)
```

How scripts find tools: `npm run` (and `node --run`) put `./node_modules/.bin` on the `PATH` (the list of folders your shell searches for commands), so a script can say `eslint .` without a global install. `npm run build` also runs `prebuild` and `postbuild` scripts if they exist; `node --run` deliberately skips those pre/post hooks and package-manager-specific environment variables in exchange for speed.

How `node_modules` resolution works

For a bare specifier like `import 'date-fns'` in `/app/src/utils/x.mjs`, Node looks for `node_modules/date-fns` in `/app/src/utils`, then `/app/src`, then `/app`, then `/`, stopping at the first match, then reads that package's `package.json` (`exports`, else `main`). npm **hoists** (flattens) most transitive dependencies to the top-level `node_modules` to share them; if two packages need incompatible versions, one gets a nested copy.

A consequence called **phantom dependencies**: your code can import a package you never declared, because a dependency happened to hoist it. It works until that dependency changes. pnpm's stricter layout prevents this.

Common mistakes

- Installing tools globally (`npm install -g`) so the project silently depends on your machine. Use devDependencies + scripts/`npx`.
- Putting build tools in `dependencies` (bloats production installs) or runtime libs in `devDependencies` (breaks production).
- Committing `node_modules` (never) or *not* committing the lockfile (always commit it).
- Running `npx some-package-name` with a typo — it may download and run a look-alike package. Check names.

TL;DR

`package.json` = identity + type + scripts + dependencies. `dependencies` for runtime, `devDependencies` for tooling. Run tools through scripts or `npx`. `node_modules` is searched upward from the importing file.

10.10 Semantic versioning, ranges and lockfiles

What it is

Semantic versioning (semver) gives version numbers a meaning: `MAJOR.MINOR.PATCH`.

Bump	When	Example	Should upgrading break you?
MAJOR	incompatible API (the functions and options a package exposes) change	3.4.1 → 4.0.0	May — read the changelog (the list of changes in each release)
MINOR	new feature, backward compatible	3.4.1 → 3.5.0	No
PATCH	backward-compatible bug fix	3.4.1 → 3.4.2	No
Pre-release	unstable preview	4.0.0-beta.2	Anything goes

Special rule: `0.x` **versions are "initial development"** — anything may change at any time, so tools treat the *minor* number of a `0.x` version like a major.

Version ranges in `package.json`

Range	Means	Allows
`1.2.3`	exactly	1.2.3 only
`^1.2.3` (caret — npm's default)	same **major**	≥1.2.3 <2.0.0
`^0.2.3`	same **minor** when major is 0	≥0.2.3 <0.3.0
`^0.0.3`	exact patch when major and minor are 0	≥0.0.3 <0.0.4
`~1.2.3` (tilde)	same **minor**	≥1.2.3 <1.3.0
`1.x` / `1` / `1.*`	any 1.y.z	≥1.0.0 <2.0.0
`>=1.2 <1.5`	explicit comparators	as written
`1.2 - 1.4`	hyphen range	≥1.2.0 ≤1.4.x
`^1 || ^2`	either range	
`*` / `latest`	anything	avoid

Caret's rule in one sentence: **don't change the left-most non-zero number.**

`code/08_semver.mjs` is a tiny model of these rules (it ignores pre-release tags and `||`/hyphen ranges; real projects use the `semver` package that npm itself uses):

```
function toBounds(range) {
  const op = range.match(/^[\^~]/)?.[0] ?? '';
  const parts = range.slice(op.length).split('.');
  const given = parts.filter((p) => p !== 'x' && p !== '*').length;       // how many numbers were given
  const [M = 0, m = 0, p = 0] = parts.map((x) => (x === 'x' || x === '*' ? 0 : Number(x)));
  const low = [M, m, p];
  let high;
  if (op === '^') {                    // don't change the left-most non-zero part
    if (M > 0 || given === 1) high = [M + 1, 0, 0];
    else if (m > 0 || given === 2) high = [0, m + 1, 0];
    else high = [0, 0, p + 1];
  } else if (op === '~') {             // allow patch changes (or minor, if only major given)
    high = given === 1 ? [M + 1, 0, 0] : [M, m + 1, 0];
  } else {                             // exact or x-range
    high = given === 3 ? [M, m, p + 1] : given === 2 ? [M, m + 1, 0] : given === 1 ? [M + 1, 0, 0] : [Infinity, 0,
0];
  }
  return { low, high };
}
// ... satisfies(version, range) checks low <= version < high
```

Output:

```
^1.2.3  >=1.2.3 <2.0.0        matches: 1.2.3, 1.2.9, 1.3.0, 1.9.9
~1.2.3  >=1.2.3 <1.3.0        matches: 1.2.3, 1.2.9
1.2.3   >=1.2.3 <1.2.4        matches: 1.2.3
1.x     >=1.0.0 <2.0.0        matches: 1.2.3, 1.2.9, 1.3.0, 1.9.9
^0.2.3  >=0.2.3 <0.3.0        matches: 0.2.5
^0.0.3  >=0.0.3 <0.0.4        matches: 0.0.3
~1      >=1.0.0 <2.0.0        matches: 1.2.3, 1.2.9, 1.3.0, 1.9.9
```

Lockfiles

A range like `^4.1.0` means two developers installing a month apart could get 4.1.0 and 4.3.2 — and every transitive dependency could differ too. A **lockfile** (`package-lock.json` for npm, `pnpm-lock.yaml`, `yarn.lock`, `bun.lock`) records the **exact** version, download URL and integrity hash (a fingerprint of the file's contents, used to detect tampering) of *every* package in the tree.

	npm install	**npm ci**
Reads	`package.json` + lockfile	lockfile (must match `package.json`, else error)
May change the lockfile	yes	never
Existing `node_modules`	reused/updated	deleted first
Use for	development, adding packages	CI servers (continuous integration — machines that automatically build and test every change), deploy builds, reproducible installs

Rules: **commit the lockfile**, use `npm ci` in automated builds, and upgrade deliberately (`npm outdated` → `npm update` or `npm install pkg@latest` → run tests). Tools like Dependabot and Renovate open pull requests (proposed code changes for review, on sites like GitHub) for updates.

Real-world examples — why this matters

- **left-pad (2016)**: an 11-line package was unpublished and thousands of builds broke; npm then restricted unpublishing.
- **Supply-chain attacks** (attacks that sneak malicious code into something you depend on, rather than attacking you directly): compromised maintainers have published malicious patch versions of popular packages (for example `event-stream` in 2018, and several hijacked packages in 2025). Caret ranges + no lockfile = you install the bad version automatically. Lockfiles, `npm ci`, `npm audit`, and reviewing new dependencies reduce the risk.

In the interview

30-second answer: "Semver is MAJOR.MINOR.PATCH — breaking, feature, fix. `^` allows everything that doesn't change the left-most non-zero number, `~` allows patch updates. Ranges keep you flexible, but the lockfile pins the exact tree so everyone and CI install the same thing — and `npm ci` installs exactly from it."

- *Q: `^0.3.1` allows 0.4.0?* A: No — for `0.x`, caret locks the minor: ≥0.3.1 <0.4.0.
- *Q: Should a library commit its lockfile?* A: Yes for its own development/CI reproducibility, but consumers ignore it — they resolve the library's ranges themselves.

Self-check

1. Largest version `~2.4.1` allows? → Anything <2.5.0.
2. Which command never modifies the lockfile? → `npm ci`.
3. A library fixes a bug without changing its API — which number bumps? → PATCH.

TL;DR

MAJOR.MINOR.PATCH; `^` = same left-most non-zero, `~` = same minor. Commit the lockfile, use `npm ci` in CI, upgrade on purpose.

10.11 Bundlers

What it is

A **bundler** reads your entry file, follows every `import` (including CSS, images and packages in `node_modules`) to build the module graph, and writes a few optimized output files (**bundles**) for the browser. Along the way it can transpile, minify and split code.

The problem it solves

Native ESM works in browsers, but shipping a large app as hundreds of unoptimized files means many requests, no dead-code removal, un-minified code, and no way to import from `node_modules` with bare names or to import CSS/images/TypeScript/JSX. Bundlers solve all of these.

What a bundler does

Feature	Meaning
Resolution	Turns `'react'` into a real file in `node_modules`; handles extensions, `exports`, aliases.
Tree shaking	Removes exports nobody imports (dead-code elimination). Depends on ESM's static structure; packages mark safe-to-drop files with `"sideEffects": false`.
Code splitting	Each dynamic `import()` becomes a separate **chunk** loaded on demand; shared code goes into common chunks.
Minification	Removes whitespace/comments, shortens local names — smaller downloads.
Transforms	TypeScript, JSX, modern syntax → target syntax (via a transpiler, §10.12); CSS modules; asset hashing (`app.3f9a1c.js`) for long-term caching (letting browsers keep files for a long time, because a changed file gets a new name — see System Design Fundamentals).
Source maps	`.map` files that map minified output back to your original source, so DevTools and error stacks show real file names and line numbers (Chapter 11).
Dev server + HMR	Serves the app locally and, with **Hot Module Replacement**, swaps changed modules in the running page without a full reload.

The main tools (2026)

Tool	What it is	Typical use
Vite	Dev server that serves native ESM on demand (very fast start) + production build. Vite 8 (March 2026) uses **Rolldown**, a Rust bundler, for builds; earlier versions used esbuild + Rollup.	Default choice for new front-end apps (React, Vue, Svelte templates)
esbuild	Extremely fast bundler/transpiler written in Go; minimal plugin API.	Libraries, scripts, tools built on top of it (this book's runner uses it)
webpack (v5)	The long-time standard; hugely configurable, loaders and plugins for everything.	Existing large apps, custom pipelines, Module Federation (loading parts of an app from separately deployed builds at runtime)
Rollup / Rolldown	Rollup pioneered ESM tree shaking; Rolldown is its Rust-based successor with a largely compatible plugin API.	Building libraries with clean ESM output
Turbopack	Rust bundler from Vercel; the default in Next.js since version 16.	Next.js apps
Parcel	Zero-configuration bundler.	Quick projects

A Vite workflow (not run):

```
npm create vite@latest my-app -- --template vanilla   # scaffold a project
cd my-app && npm install
npm run dev        # dev server with HMR, e.g. http://localhost:5173
npm run build      # production bundle in dist/
npm run preview    # serve dist/ locally to check the build
```

An esbuild one-liner (not run):

```
npx esbuild src/main.js --bundle --minify --sourcemap --format=esm --outdir=dist
```

Trade-offs

Option	Pros	Cons
No build (native ESM + import maps)	Zero tooling, instant	No tree shaking/minify, many requests, no JSX/TS
Vite	Fast, sensible defaults, big plugin ecosystem	Dev (unbundled) and prod (bundled) differ slightly
webpack	Maximum control	Slower, configuration-heavy
esbuild directly	Blazing fast, simple	Fewer features (no HMR dev server of its own, limited code-splitting options)

When not to use a bundler

Node servers and CLIs usually run source files directly (Node understands ESM, and strips TypeScript types natively since 23.6/22.18). Bundle server code only for special cases (serverless cold starts (the delay when a serverless platform starts a fresh copy of your function), single-file distribution).

Common mistakes

- Importing a whole library (`import _ from 'lodash'`) when you use one function — prefer ESM packages and named imports (`import { debounce } from 'lodash-es'`) so tree shaking works.
- Deploying without source maps (unreadable production errors) — or publishing them publicly when the source is secret (upload them to your error tracker, a service such as Sentry that collects crashes

from users, instead).
- Assuming dev behaviour equals production — always run `build` + `preview` before shipping.

TL;DR

Bundlers build the module graph and emit optimized bundles: resolve, transform, tree shake, split, minify, source-map. Use Vite for apps, esbuild/Rollup-style tools for libraries, webpack where a project already has it.

10.12 Transpilers and polyfills

What it is

- A **transpiler** (source-to-source compiler) rewrites code into different JavaScript: newer syntax → older syntax for old browsers, TypeScript → JavaScript (removing types), JSX → function calls. Examples: **Babel** (JS, plugin-based), **TypeScript's** `tsc`, **SWC** (Rust), **esbuild** (Go).
- A **polyfill** is code that *adds a missing API at runtime* (e.g., defining `Array.prototype.at` if the browser lacks it). Libraries: **core-js**.

The key distinction: **syntax** (`?.`, `class`, `#private`) can't be polyfilled — it must be transpiled, because an old parser would reject the file. **APIs** (`Array.prototype.findLast`, `Object.groupBy`, `structuredClone`) can be polyfilled.

```
// Input (modern)                    // Transpiled for an old target (roughly)
const city = user?.address?.city ?? 'n/a';
                                     var _user$address;
                                     var city = (_user$address = user == null ? void 0 : user.address) == null
                                       ? void 0 : _user$address.city;
                                     city = city != null ? city : 'n/a';
```

(Illustrative, not run — exact output varies by tool.)

How targets are chosen

Tools read a **browserslist** query (in `package.json` or `.browserslistrc`), for example `"> 0.5%, last 2 versions, not dead"` or `"baseline widely available"` (features that have worked in all major browsers for at least about two and a half years), and only transform what those browsers don't support. Vite's default build target is a set of modern "baseline widely available" browsers, so very little is transpiled. Targeting only evergreen browsers keeps bundles smaller and faster.

TypeScript note

TypeScript is covered in Chapter 16. Most bundlers (esbuild, Vite, SWC) **strip types without checking them** for speed; run `tsc --noEmit` separately (in CI or your editor) to actually type-check.

TL;DR

Transpile syntax, polyfill APIs. Configure targets with browserslist; modern targets mean less transformation. Bundlers usually strip TS types; type-check separately.

10.13 Linters and formatters

What it is

- A **linter** statically analyzes code (without running it) to find likely **bugs** and bad patterns: unused variables, use-before-define, `==` where `===` was meant, unreachable code, missing `await`, rules of React hooks. **ESLint** is the standard for JavaScript (Biome and Oxlint are faster Rust-based alternatives).

- A **formatter** rewrites code **layout** — indentation, quotes, semicolons, line wrapping — to one consistent style so reviews focus on logic. **Prettier** is the standard.

They're complementary: let the formatter own style and the linter own correctness. ESLint deprecated its formatting-related core rules (v8.53, 2023) for exactly this reason; `eslint-config-prettier` turns off any remaining rules that would fight Prettier.

ESLint's flat config

Since ESLint 9, the configuration file is `eslint.config.js` (a JavaScript module exporting an array of config objects — the "flat config"). ESLint 10 (February 2026) removed the old `.eslintrc.*` format entirely.

```
// eslint.config.js  (not run)
import js from '@eslint/js';
import globals from 'globals';
import prettier from 'eslint-config-prettier';

export default [
  js.configs.recommended,                    // ESLint's recommended rule set
  {
    files: ['**/*.{js,mjs}'],
    languageOptions: {
      ecmaVersion: 'latest',
      sourceType: 'module',
      globals: { ...globals.browser, ...globals.node },
    },
    rules: {
      'no-unused-vars': ['error', { argsIgnorePattern: '^_' }],
      eqeqeq: ['error', 'always'],           // forbid == / !=
      'prefer-const': 'warn',
    },
  },
  { ignores: ['dist/**', 'coverage/**'] },
  prettier,                                  // last: disable style rules that conflict with Prettier
];
```

Each object can target files with `files` and set language options, plugins and rules. Rule severity is `"off"`, `"warn"` or `"error"` (or 0/1/2). Later objects override earlier ones for matching files.

Commands (not run):

```
npm init @eslint/config@latest     # interactive setup that writes eslint.config.js
npx eslint .                       # lint the project; exits non-zero on errors (fails CI)
npx eslint . --fix                 # auto-fix what can be fixed safely
npx prettier . --check             # report unformatted files (CI)
npx prettier . --write             # format files in place
```

Typical ESLint output for a file with problems looks like (illustrative):

```
src/cart.js
   3:7   error    'total' is assigned a value but never used  no-unused-vars
  12:15  error    Expected '===' and instead saw '=='          eqeqeq

✖ 2 problems (2 errors, 0 warnings)
```

A minimal Prettier config, `.prettierrc.json`: `{ "singleQuote": true, "semi": true, "printWidth": 100 }`. Prettier is deliberately "opinionated": it has few options, to end style debates.

Wiring it into the workflow

1. **Editor** — ESLint and Prettier extensions show problems as you type and format on save.
2. **Pre-commit hook** — tools like husky + lint-staged run the linter/formatter on staged files (the changes you've marked with `git add` for the next commit) before each commit.
3. **CI** — `npm run lint` and `prettier --check` fail the build if anything slipped through.

Common mistakes

- Letting ESLint and Prettier fight over formatting (two tools rewriting the same lines) — use `eslint-config-prettier`.
- Turning every warning off to "get green" — fix the root cause or disable a rule for one line with a comment explaining why (`// eslint-disable-next-line no-console -- CLI output`).
- Following old tutorials with `.eslintrc.json` — that format no longer works in ESLint 10.

TL;DR

ESLint finds bugs (flat config `eslint.config.js`); Prettier formats. Run both in the editor, pre-commit and CI; let Prettier own style.

10.14 Putting it together — a modern project

```
notes-app/
├─ package.json          # "type": "module", scripts, deps
├─ package-lock.json     # committed
├─ eslint.config.js
├─ .prettierrc.json
├─ index.html            # <script type="module" src="/src/main.js">
├─ src/
│  ├─ main.js            # entry: imports app, starts it
│  ├─ app.js
│  ├─ features/notes/    # feature folders, named exports
│  └─ lib/               # shared helpers
├─ test/                 # node --test (Chapter 16)
└─ dist/                 # build output (git-ignored)
```

Daily loop: `npm run dev` (Vite + HMR) → edit → the editor lints/formats on save → `npm test` → commit (pre-commit hook lints staged files) → CI runs `npm ci`, `npm run lint`, `npm test`, `npm run build` → deploy `dist/`.

Chapter 10 TL;DR

- **Modules** give each file its own scope; **ES modules** (`import`/`export`) are the standard in browsers and Node.
- **Named exports** (many, names must match) vs **default** (one, importer names it); `import * as ns`; `export … from` for barrels.
- ESM loads in three phases (**parse → link → evaluate**): imports are **hoisted**, **live, read-only bindings**; modules are **strict singletons**; cycles work if top-level code avoids TDZ bindings.
- `import()` loads on demand (code splitting); **top-level** `await` makes importers wait — fine in apps, risky in libraries.
- `import.meta.url` / `dirname` / `filename`; JSON via `with { type: 'json' }` (ES2025).
- Browsers: `<script type="module">` is deferred, strict, CORS-bound; **import maps** resolve bare names.
- Node: `.mjs` / `.cjs` / `"type"` choose ESM vs CommonJS; ESM imports CJS (default = `module.exports`); `require(esm)` works unless top-level `await`; `"exports"` defines a package's public entry points.
- **npm**: `dependencies` vs `devDependencies`, scripts, `npx`, `node --run`; **semver** `^`/`~`; **commit the lockfile** and use `npm ci` in CI.
- **Bundlers** (Vite, esbuild, webpack, Rolldown): tree shaking, code splitting, minification, source maps, HMR. **Transpile syntax, polyfill APIs.**
- **ESLint** (flat config) finds bugs; **Prettier** formats; run both in editor, hooks and CI.

Next: when things go wrong — Chapter 11: Errors and Debugging.

Chapter 11 — Errors and Debugging

Every program fails sometimes: users type letters into number fields, networks drop, files go missing, and programmers make mistakes. JavaScript reports problems with **error objects** that are **thrown** and can be **caught**. This chapter shows how errors work, how to design your own, and how to find bugs quickly with the console, stack traces and real debuggers — in the browser and in Node (**Node.js**, the program that runs JavaScript outside the browser, on servers and laptops).

What you'll learn

1. The built-in error types, what each one means, and what every error object contains.
2. `throw`, `try` / `catch` / `finally` — exact control flow, rethrowing, and the traps.
3. Custom error classes and a clean error-handling layer.
4. Error chaining with `cause` (ES2022 — the 2022 edition of ECMAScript, the official JavaScript standard; a new edition comes out every year) and `AggregateError`.
5. Errors in asynchronous code (the preview; the full story is in Chapter 12).
6. Guaranteed cleanup with `finally` and the new `using` declaration.
7. The console toolbox beyond `console.log`.
8. Reading stack traces; source maps.
9. Debugging with browser DevTools (the developer-tools panel built into every browser) and the Node inspector; a systematic debugging method.
10. A gallery of the most common runtime errors, with their exact messages and causes.

Before you start. Custom errors use classes and `extends` (Chapter 9). Type coercion, which causes many silent bugs, is in Chapter 2.

All runnable code is in `code/`. Outputs are real (Node 24, V8 engine — V8 is Google's JavaScript **engine**, the program that actually reads and runs JS inside Chrome and Node). Error **messages** are not standardized — Firefox and Safari word them differently — but error **types** are.

11.1 Errors and the built-in error types

What it is

An **error** (also called an **exception**) is an object that describes something that went wrong. When code **throws** an error, normal execution stops immediately, and the engine unwinds the call stack looking for a `catch` that will handle it. If none is found, the error is **uncaught**: the browser logs it to the console (and the rest of that script task stops), and Node prints it and exits the process with a non-zero **exit code** (the number a program hands back to the operating system when it ends: `0` means success, anything else means failure).

Analogy: a fire alarm. Someone pulls it (`throw`); everyone stops what they're doing; the alarm travels up floor by floor until someone responsible (`catch`) deals with it. If nobody does, the building is evacuated (the program crashes).

Terms:

- **Runtime error** — thrown while code runs (`TypeError`, `RangeError`, …).
- **Parse-time (syntax) error** — the engine can't even read the code; *nothing* in that script/module runs.
- **Call stack** — the list of functions currently running, newest on top (Chapter 12 shows it in the event loop).

The built-in types

Type	Thrown when…	Typical example
`Error`	Generic base type — use for your own errors or subclass it	`throw new Error('config missing')`
`TypeError`	A value is the wrong type for an operation	`null.length`, calling a non-function, assigning to a `const`
`ReferenceError`	A name doesn't exist, or is used in its TDZ (temporal dead zone: the lines before its `let`/`const` declaration)	typo in a variable name; `let` used before its line
`SyntaxError`	Code (or JSON) can't be parsed	`JSON.parse('{bad}')`, `new Function('1 +')`
`RangeError`	A number is outside the allowed range	`new Array(-1)`, `toFixed(101)`, stack overflow (too many nested calls)
`URIError`	Malformed URI (web address) component	`decodeURIComponent('%')`
`EvalError`	Not thrown by modern engines; exists for backward compatibility	—
`AggregateError` (ES2021)	Several errors need reporting together	`Promise.any` when every promise rejects
`SuppressedError` (ES2027)	An error happened while disposing after another error	`using` declarations (§11.6)

All of them inherit from `Error` (`TypeError.prototype.[[Prototype]] === Error.prototype`), so `err instanceof Error` is `true` for every built-in error. Environments add their own: browsers have `DOMException` (the error type browser APIs throw, e.g., `AbortError`, `NotAllowedError`), Node adds `code` properties like `'ENOENT'` ("Error NO ENTry" — file or folder not found) to system errors.

`code/01_error_types.mjs` triggers each one for real:

```
const cases = {
  TypeError:      () => null.length,
  ReferenceError: () => notDeclaredAnywhere + 1,
  SyntaxError:    () => JSON.parse('{ bad json }'),
  RangeError:     () => new Array(-1),
  URIError:       () => decodeURIComponent('%'),
  EvalError:      () => { throw new EvalError('never thrown by modern engines — only kept for compatibility'); },
  AggregateError: () => { throw new AggregateError([new Error('a'), new TypeError('b')], 'several things failed');
},
};

for (const fn of Object.values(cases)) {
  try {
    fn();
  } catch (err) {
    console.log(`${err.name.padEnd(15)} | instanceof Error: ${err instanceof Error} | ${err.message}`);
  }
}

const e = new TypeError('price must be a number', { cause: { got: 'ten' } });
console.log('\nname   :', e.name);
console.log('message:', e.message);
console.log('cause  :', e.cause);
console.log('stack  :', e.stack.split('\n')[0], '... (+', e.stack.split('\n').length - 1, 'frame lines)');
console.log('String :', String(e));

console.log('\nproto chain:', Object.getPrototypeOf(TypeError) === Error,
Object.getPrototypeOf(RangeError.prototype) === Error.prototype);

try {
  await Promise.any([Promise.reject(new Error('mirror 1 down')), Promise.reject(new Error('mirror 2 down'))]);
} catch (agg) {
  console.log('\nPromise.any ->', agg.name, '|', agg.message, '|', agg.errors.map((x) => x.message));
}

console.log('\nError.isError:', Error.isError(e), Error.isError({ name: 'Error', message: 'fake' }));

try {
  new Function('return 1 +');   // parsing happens here
} catch (err) {
  console.log('parse-time  :', err.name, '-', err.message);
}
```

Output:

```
TypeError       | instanceof Error: true | Cannot read properties of null (reading 'length')
ReferenceError  | instanceof Error: true | notDeclaredAnywhere is not defined
SyntaxError     | instanceof Error: true | Expected property name or '}' in JSON at position 2 (line 1 column 3)
RangeError      | instanceof Error: true | Invalid array length
URIError        | instanceof Error: true | URI malformed
EvalError       | instanceof Error: true | never thrown by modern engines — only kept for compatibility
AggregateError  | instanceof Error: true | several things failed

name   : TypeError
message: price must be a number
cause  : { got: 'ten' }
stack  : TypeError: price must be a number ... (+ 4 frame lines)
String : TypeError: price must be a number

proto chain: true true

Promise.any -> AggregateError | All promises were rejected | [ 'mirror 1 down', 'mirror 2 down' ]

Error.isError: true false
parse-time  : SyntaxError - Unexpected token '}'
```

Anatomy of an error object

Property	Standard?	Meaning
`name`	✓ (inherited from the prototype)	The type, e.g. `'TypeError'`
`message`	✓	Human-readable description
`cause`	✓ ES2022	The underlying reason, passed as `new Error(msg, { cause })` (§11.4)
`errors`	✓ on `AggregateError`	Array of the individual errors
`stack`	de-facto (every engine; a standard is proposed)	Text of the call stack at creation time
`code`	Node / libraries	Machine-readable code, e.g. `'ENOENT'`, `'ERR_MODULE_NOT_FOUND'`

`String(err)` (and `err.toString()`) gives `"Name: message"`. The stack is captured when the error object is **created** (with `new`), not when it's thrown.

`Error.isError(value)` (ES2026; available in Node 24 and recent browser versions — check support if you target older ones) is the reliable check for "is this a genuine error object?" — unlike `instanceof Error` it works for errors from other **realms** (a realm is a separate JavaScript global environment with its own copy of built-ins like `Error` — e.g., an iframe, which is a page embedded inside another page, or a Node `vm` sandbox) and can't be fooled by an object with a hand-made prototype.

Self-check

1. `undefinedFn()` where `undefinedFn` was never declared — which error? → `ReferenceError` (name not found). If it's declared but holds `undefined`, it's a `TypeError` (not a function).
2. Which property holds the underlying error in a wrapped error? → `cause`.
3. When is a `SyntaxError` in your source file reported? → When the file is parsed, before any of it runs.

TL;DR

Errors are objects with `name`, `message`, `stack` (and optional `cause`). Know the core types: `TypeError` (wrong type), `ReferenceError` (unknown name/TDZ), `SyntaxError` (unparseable), `RangeError` (out of range), `AggregateError` (many).

11.2 `throw`, `try`, `catch`, `finally`

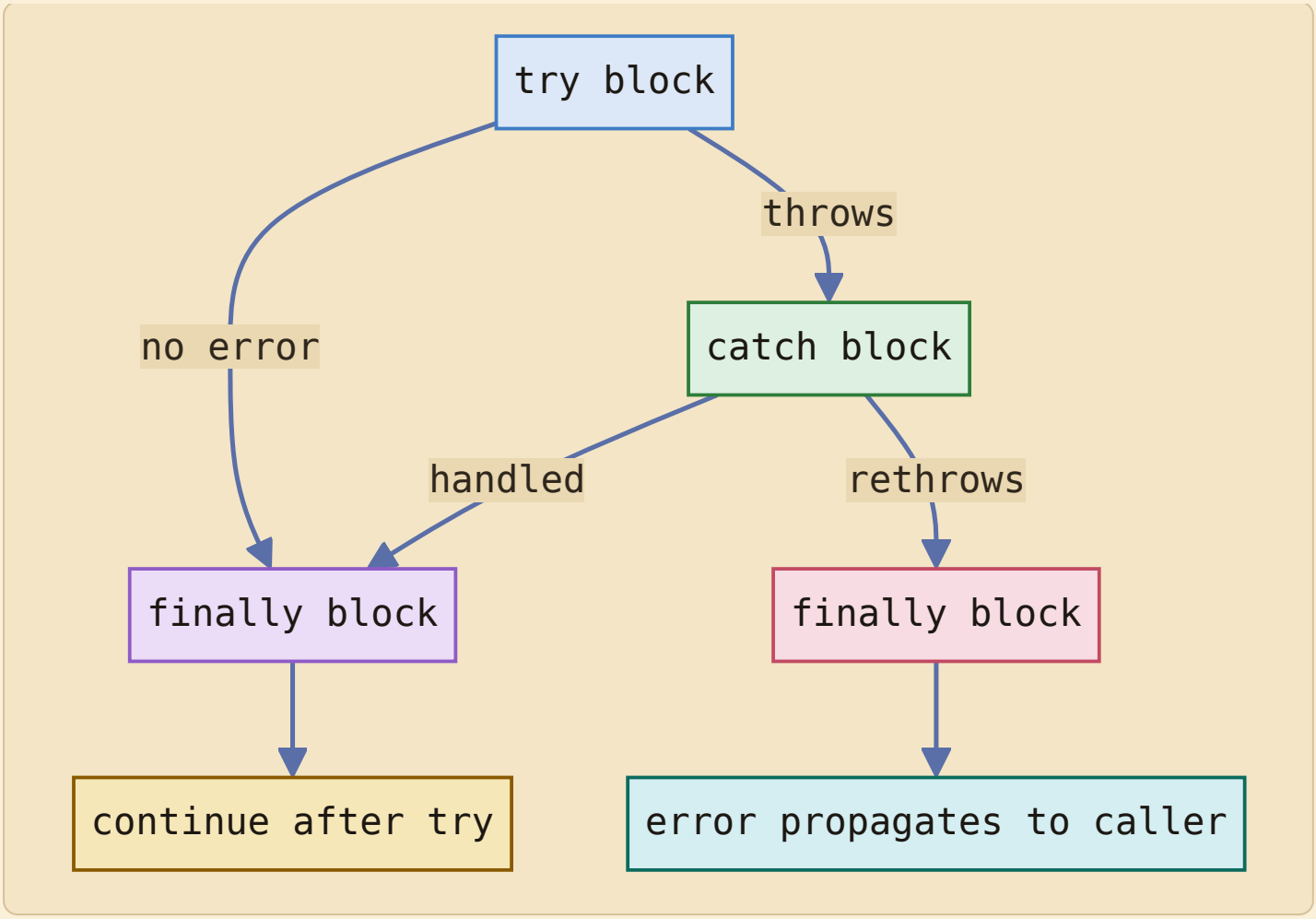
What it is

- `throw expression` — raises an error; execution jumps to the nearest enclosing `catch`.
- `try { … }` — code that might throw.
- `catch (err) { … }` — runs only if something in `try` threw; `err` is the thrown value.
- `finally { … }` — always runs after `try`/`catch`, whether the block succeeded, threw, or `return`ed. Used for cleanup.

A `try` must have a `catch`, a `finally`, or both.

How it works

1. Run the `try` block.
2. If it throws, abandon the rest of `try`, and run `catch` with the error.
3. Then run `finally` (always).
4. If `catch` itself throws (or rethrows), `finally` still runs, then the new error propagates outward.
5. If nothing catches it anywhere up the stack, it's an uncaught error.



`code/02_try_catch_finally.mjs`:

```
function parseAge(text) {
  const log = [];
  try {
    log.push('try start');
    const age = Number.parseInt(text, 10);
    if (Number.isNaN(age)) throw new TypeError(`not a number: "${text}"`);
    if (age < 0) throw new RangeError(`negative age: ${age}`);
    log.push('try end');
    return { age, log };
  } catch (err) {
    log.push(`catch ${err.name}`);
    return { age: null, log };
  } finally {
    log.push('finally');       // runs on success, on error, and even after return
  }
}
for (const input of ['42', 'abc', '-3']) {
  const { age, log } = parseAge(input);
  console.log(input.padEnd(4), '->', String(age).padEnd(5), log.join(' > '));
}

// finally's return OVERRIDES the try's return (and swallows errors) — avoid!
function sneaky() {
  try { throw new Error('lost forever'); }
  finally { return 'finally wins'; }  // eslint: no-unsafe-finally
}
console.log('\nsneaky():', sneaky());

// Optional catch binding (ES2019): omit (err) when you don't need it
function isValidJson(s) {
  try { JSON.parse(s); return true; } catch { return false; }
}
console.log('isValidJson:', isValidJson('{"a":1}'), isValidJson('{a:1}'));

// Catch only what you understand; rethrow the rest
function loadUser(raw) {
  try {
    return JSON.parse(raw).user.name;
  } catch (err) {
    if (err instanceof SyntaxError) return '(invalid JSON — using guest)';
    throw err;                     // a TypeError here means a real bug: don't hide it
  }
}
console.log('rethrow 1  :', loadUser('{oops'));
try { loadUser('{"nouser":true}'); } catch (err) { console.log('rethrow 2  :', err.name, '-', err.message); }

// You can throw anything — but only Error objects carry a stack
for (const thrown of ['a string', 42, { code: 'E_BAD' }, new Error('real error')]) {
  try { throw thrown; } catch (err) {
    console.log('threw', typeof err === 'object' ? err.constructor.name : typeof err, '| has stack:', typeof
err?.stack === 'string');
  }
}

// try/catch is synchronous: it cannot catch an error thrown later by a callback
try {
  setTimeout(() => {
    try { throw new Error('from timer'); } catch (err) { console.log('\ntimer error caught INSIDE the callback:',
err.message); }
  }, 0);
} catch {
  console.log('never printed');
}
console.log('\n(outer try block already finished)');
```

Output:

```
42   -> 42    try start > try end > finally
abc  -> null  try start > catch TypeError > finally
-3   -> null  try start > catch RangeError > finally

sneaky(): finally wins
isValidJson: true false
rethrow 1  : (invalid JSON — using guest)
rethrow 2  : TypeError - Cannot read properties of undefined (reading 'name')
threw string | has stack: false
threw number | has stack: false
threw Object | has stack: false
threw Error | has stack: true

(outer try block already finished)

timer error caught INSIDE the callback: from timer
```

What to notice:

- `finally` ran in all three cases — even though `try` and `catch` both `return`. The return value is computed first, then `finally` runs, then the function actually returns.
- A `return` (or `throw`) *inside* `finally` replaces whatever `try`/`catch` was doing — the error `'lost forever'` vanished. ESLint's `no-unsafe-finally` rule flags this (**ESLint** is a *linter* — a tool that scans your code for likely mistakes without running it).
- **Catch narrowly, rethrow the rest.** `loadUser` expects bad JSON (a `SyntaxError`) and recovers; a `TypeError` means the data shape is wrong — a real bug that should surface.
- `catch` receives *whatever* was thrown. Throwing strings or plain objects loses the stack trace and breaks `instanceof Error` checks. **Always throw `Error` objects** (or subclasses).
- The `catch` variable is block-scoped to the `catch` block.
- `try/catch` only covers code that runs *synchronously inside it* (synchronously = right now, line after line, before anything else gets a turn). The timer callback (a **callback** is a function you hand to other code so it can call you back later) ran later, on a fresh call stack, after the outer `try` was long gone.

When to use try/catch — and when not

Use it	Don't use it
At boundaries where failure is expected: parsing input, `JSON.parse`, network/file I/O (input/output), third-party code (libraries you didn't write)	To replace simple checks — `if (user?.name)` is cheaper and clearer than catching a `TypeError`
In a top-level handler that turns errors into a response/UI message and logs them	Wrapping every function "just in case" — it hides bugs
To add context and rethrow (§11.4)	To silently swallow errors: `catch {}` with no comment or handling

Performance note: in modern engines a `try` block that doesn't throw costs essentially nothing; *throwing* is relatively expensive (capturing the stack), so don't use exceptions for normal control flow like "item not found in a loop".

In the interview

30-second answer: "`throw` raises an error and unwinds the stack to the nearest `catch`. `finally` always runs — after success, error, or even `return` — so it's for cleanup; but a `return` inside `finally` overrides everything and swallows errors. `try/catch` is synchronous, so it can't catch errors from callbacks that run later — for promises you `await` inside `try`, or use `.catch()`. Always throw `Error` objects so you get a stack trace."

- *Q: What does `(() => { try { return 1 } finally { return 2 } })()` return?* A: `2`.
- *Q: Does `finally` run if `catch` rethrows?* A: Yes, then the error continues outward.

- *Q: Why is `catch (e) {}` dangerous?* A: It hides every error, including programming bugs, making failures silent and hard to trace.

Self-check

1. Can you have `try { } finally { }` without `catch`? → Yes; errors still propagate, but cleanup runs.
2. What is `err` in `catch (err)` after `throw 42`? → The number `42` (no stack).
3. Why didn't the outer `catch` see the timer's error? → The callback ran later, after the `try` finished.

TL;DR

`try` → `catch` on error → `finally` always. Throw `Error` objects, catch narrowly, rethrow what you can't handle, never `return` from `finally`, and remember try/catch is synchronous.

11.3 Custom errors

What it is

A **custom error** is a subclass of `Error` for a specific failure in *your* domain: `ValidationError`, `NotFoundError`, `PaymentDeclinedError`. It lets callers react to *kinds* of failure with `instanceof` (or a `code`), and carry extra data (which field was invalid, what HTTP status to return — the **HTTP status** is the 3-digit code a web server sends with every response, e.g. 200 OK, 400 bad request, 404 not found, 500 server error; see System Design Fundamentals).

How it works

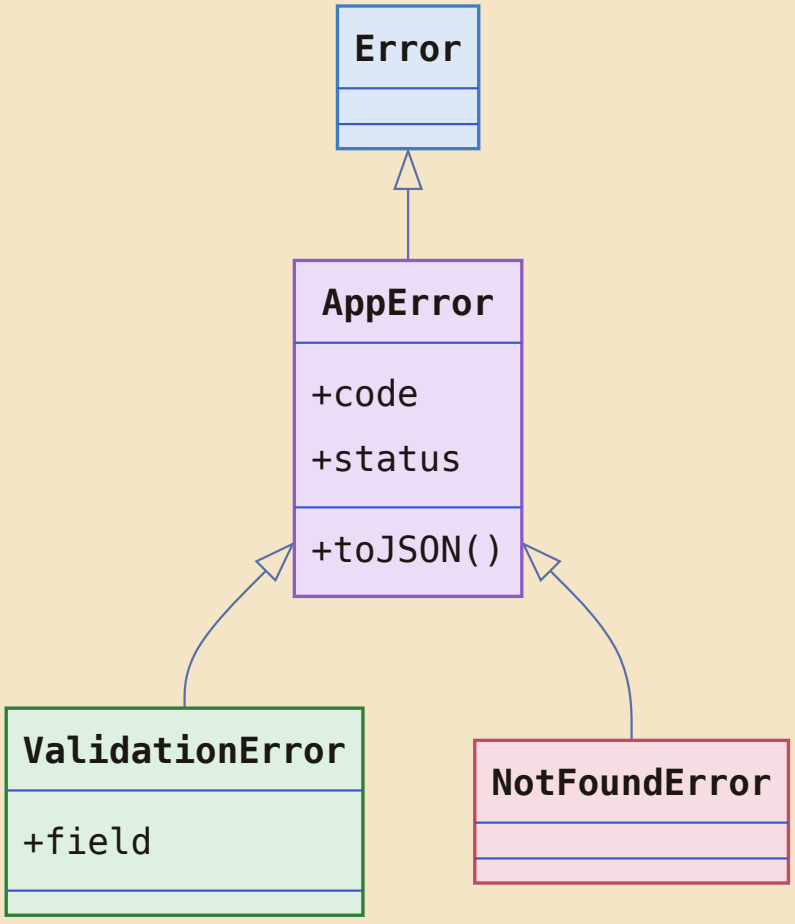
1. `class MyError extends Error`.
2. Call `super(message, options)` so `message`, `stack` and `cause` are set up.
3. Set `this.name` (otherwise it prints as plain `Error`). Using `new.target.name` sets it automatically for every subclass.
4. Add fields (`code`, `status`, `field`…).
5. Handle errors in **one place** (a top-level handler, an Express error middleware — Express is a popular Node web-server library, and *middleware* is a function every request passes through — or a React error boundary, a component that catches errors from the UI below it) that maps error types to responses, and treats *unknown* errors as 500s without leaking their details to users.

`code/03_custom_errors.mjs`:

```
class AppError extends Error {
  constructor(message, options = {}) {
    super(message, { cause: options.cause });   // pass cause up to Error
    this.name = new.target.name;                  // 'ValidationError', 'NotFoundError', ...
    this.code = options.code ?? 'APP_ERROR';
    this.status = options.status ?? 500;
  }
  toJSON() { return { name: this.name, code: this.code, message: this.message }; }
}

class ValidationError extends AppError {
  constructor(field, message) {
    super(message, { code: 'VALIDATION', status: 400 });
    this.field = field;
  }
}
class NotFoundError extends AppError {
  constructor(what, id) { super(`${what} ${id} not found`, { code: 'NOT_FOUND', status: 404 }); }
}

function placeOrder(order) {
  if (!Number.isInteger(order.qty) || order.qty < 1) throw new ValidationError('qty', 'qty must be a positive
integer');
  if (order.productId !== 'p1') throw new NotFoundError('Product', order.productId);
  return { ok: true };
}

// A single place that turns errors into responses
function handle(order) {
  try {
    return { status: 200, body: placeOrder(order) };
  } catch (err) {
    if (err instanceof AppError) return { status: err.status, body: err.toJSON() };
    return { status: 500, body: { message: 'internal error' } };   // unknown -> don't leak details
  }
}
console.log(handle({ productId: 'p1', qty: 2 }));
console.log(handle({ productId: 'p1', qty: 0 }));
console.log(handle({ productId: 'p9', qty: 1 }));

const v = new ValidationError('email', 'bad email');
console.log('\ninstanceof  :', v instanceof ValidationError, v instanceof AppError, v instanceof Error);
console.log('name/field  :', v.name, v.field);
console.log('String(v)   :', String(v));
console.log('stack line 1:', v.stack.split('\n')[0]);
console.log('JSON        :', JSON.stringify(v));
console.log('plain Error JSON:', JSON.stringify(new Error('x')), '(message/stack are non-enumerable own props)');
```

Output:

```
{ status: 200, body: { ok: true } }
{
  status: 400,
  body: {
    name: 'ValidationError',
    code: 'VALIDATION',
    message: 'qty must be a positive integer'
  }
}
{
  status: 404,
  body: {
    name: 'NotFoundError',
    code: 'NOT_FOUND',
    message: 'Product p9 not found'
  }
}

instanceof  : true true true
name/field  : ValidationError email
String(v)   : ValidationError: bad email
stack line 1: ValidationError: bad email
JSON        : {"name":"ValidationError","code":"VALIDATION","message":"bad email"}
plain Error JSON: {} (message/stack are non-enumerable own props)
```



Why the stack's first line already says `ValidationError`: V8 builds the `stack` string lazily (only when first needed) from the error's `name` the first time it's read — here after `this.name` was set. Don't rely on that everywhere: set `name` in the constructor so all engines agree.

`JSON.stringify(new Error('x'))` gives `{}` because `message` and `stack` are **non-enumerable** own properties (hidden from `for…in`, `Object.keys` and `JSON.stringify`), and `name` lives on the prototype. That's why `AppError` defines `toJSON()` (which `JSON.stringify` calls automatically if it exists) — useful for logs

and API responses (an **API** — application programming interface — is the set of operations a program or server offers for other code to call) (but never send `stack` to end users).

Error codes vs classes

Approach	Pros	Cons
Subclasses + `instanceof`	Rich, typed, hierarchical (`catch` a whole family)	Breaks across realms/duplicated packages (two copies of the class)
`code` string (`err.code === 'ENOENT'`)	Survives serialization (turning data into text to store or send), logs, process boundaries; Node's own convention	No hierarchy; typos go unnoticed
Both (as above)	`instanceof` inside the app, `code` in logs and API bodies	A little more code

Common mistakes

- Forgetting `super(message)` → `ReferenceError` when you touch `this` (Chapter 9).
- Not setting `name` → logs say `Error: …` for everything.
- Leaking internals: sending `err.message` / `err.stack` of *unexpected* errors to clients can reveal SQL (database query text), file paths or secrets. Log them server-side; send a generic message.
- Over-engineering: dozens of error classes nobody catches differently. Create a class only when some caller will handle it differently.

Real-world examples

Node's system errors carry `code` (`ENOENT`, `EADDRINUSE` — the network port is already taken, `ECONNREFUSED` — the other machine refused the connection); `fetch` rejects with a `TypeError` on network failure and an `AbortError` `DOMException` when aborted; HTTP libraries throw errors with a `status`/`response`; ORMs (object-relational mappers — libraries that let you work with database rows as JavaScript objects) throw `UniqueConstraintError`-style subclasses.

Self-check

1. Why set `this.name` in a custom error? → So `String(err)`, stacks and logs show the right type.
2. Why does `JSON.stringify(err)` often print `{}`? → `message`/`stack` are non-enumerable; `name` is inherited.
3. What should a top-level handler do with an unknown error? → Log full details internally; return a generic message (e.g., 500).

TL;DR

`class X extends Error`, call `super(message, { cause })`, set `name`, add `code`/data, handle all errors in one place, and never leak internals of unexpected errors.

11.4 Error chaining with `cause`

What it is

When you catch a low-level error and throw a higher-level one ("could not load config"), you want to **add context without losing the original**. Since ES2022, every error constructor accepts an options object: `new Error(message, { cause })`. The original is stored in `err.cause`.

Before `cause`, people either lost the original (bad for debugging) or concatenated messages into unreadable strings.

`code/04_error_cause.mjs`:

```
function readConfigText(name) {
  const err = new Error(`ENOENT: no such file or directory, open '${name}'`);   // pretend fs failed
  err.code = 'ENOENT';
  throw err;
}

function loadConfig(name) {
  try {
    return JSON.parse(readConfigText(name));
  } catch (err) {
    throw new Error(`Could not load config "${name}"`, { cause: err }); // add context, keep original
  }
}

function startServer() {
  try {
    loadConfig('server.json');
  } catch (err) {
    throw new Error('Server failed to start', { cause: err });
  }
}

try {
  startServer();
} catch (err) {
  let depth = 0;
  for (let e = err; e; e = e.cause) {                                    // walk the cause chain
    console.log(`${'  '.repeat(depth++)}${depth > 1 ? 'caused by: ' : ''}${e.message}${e.code ? ` [${e.code}]` :
''}`);
  }
  console.log('\nroot cause code:', rootCause(err).code);
}

function rootCause(e) { while (e.cause instanceof Error) e = e.cause; return e; }

import { inspect } from 'node:util';
const shown = inspect(new Error('outer', { cause: new Error('inner') }));
console.log('\nutil.inspect shows cause?', shown.includes('[cause]: Error: inner'));
```

Output:

```
Server failed to start
  caused by: Could not load config "server.json"
    caused by: ENOENT: no such file or directory, open 'server.json' [ENOENT]

root cause code: ENOENT

util.inspect shows cause? true
```

Each layer adds *what it was trying to do*; the root cause keeps *what actually failed*. Node's `console.log`/`console.error` and browser DevTools print the `cause` chain automatically.

`AggregateError` is the sibling for "several independent failures": its `errors` array holds each one (e.g., `Promise.any` where every mirror failed, or validation that reports all bad fields at once).

TL;DR

Wrap with `new Error('what I was doing', { cause: err })`; walk `.cause` to the root. Use `AggregateError` for many errors at once.

11.5 Errors in asynchronous code (preview)

The problem it solves

Asynchronous code (timers, network, files) finishes *later*, after the `try` block around the call has already exited. Promises (objects that stand for a result that will arrive later) carry errors as

rejections (the promise's "it failed" outcome) instead. The event loop (the engine's mechanism for running queued work one piece at a time) and promises are explained in Chapter 12; here are the error-handling rules you need now.

`code/05_async_errors.mjs`:

```
const fetchUser = async (id) => {
  await null;                                    // simulate async work
  if (id !== 1) throw new Error(`user ${id} not found`);
  return { id, name: 'Ada' };
};

// 1) await inside try/catch — the normal way
try {
  await fetchUser(2);
} catch (err) {
  console.log('await + try/catch :', err.message);
}

// 2) .catch() on the promise
await fetchUser(3).catch((err) => console.log('.catch()          :', err.message));

// 3) Promise.allSettled — collect successes and failures together
const results = await Promise.allSettled([fetchUser(1), fetchUser(4)]);
console.log('allSettled        :', results.map((r) => (r.status === 'fulfilled' ? r.value.name :
r.reason.message)));

// 4) Forgetting to await: the try/catch finishes before the promise rejects
process.on('unhandledRejection', (reason) => {
  console.log('unhandledRejection:', reason.message, '(Node would crash here without this handler)');
});
try {
  fetchUser(5);                                  // no await!
} catch {
  console.log('never printed');
}
console.log('after the try     : the rejection has not happened yet');
await new Promise((r) => setTimeout(r, 10));      // let the event loop report it
```

Output:

```
await + try/catch : user 2 not found
.catch()          : user 3 not found
allSettled        : [ 'Ada', 'user 4 not found' ]
after the try     : the rejection has not happened yet
unhandledRejection: user 5 not found (Node would crash here without this handler)
```

Rules:

1. Inside an `async` function, `throw` becomes a rejected promise, and `await` of a rejected promise throws — so `try { await … } catch` works naturally.
2. **Always `await` (or `return`, or `.catch()`) every promise.** A forgotten promise's rejection escapes every `try`. ESLint (with type info, `@typescript-eslint/no-floating-promises`) can flag this — a *floating* promise is one nobody awaits or handles.
3. An **unhandled rejection** — a rejected promise nobody handles — crashes Node by default (since Node 15) and logs "Uncaught (in promise)" in browsers.

Last-resort global handlers

Environment	Synchronous uncaught error	Unhandled promise rejection
Browser	`window.addEventListener('error', e => …)`	`window.addEventListener('unhandledrejection', e => …)`
Node	`process.on('uncaughtException', err => …)`	`process.on('unhandledRejection', reason => …)`

Use them to **log and report** (e.g., to an error-tracking service such as Sentry — a hosted tool that collects crashes from real users), not to keep going as if nothing happened. After an

`uncaughtException`, a Node process is in an unknown state — log, then exit and let a process manager (a supervisor program such as PM2, systemd or Kubernetes that keeps your app running) restart it.

TL;DR

`await` inside `try`, or `.catch()` every promise; never leave a promise floating. Global handlers are for logging, not recovery.

11.6 Guaranteed cleanup: `finally` and `using`

What it is

Resources — open files, database connections, locks (flags that give one piece of code exclusive use of something), timers, subscriptions — must be released even if something fails. `finally` has always done this. **Explicit Resource Management** (the `using` declaration) is a newer feature that makes cleanup automatic: an object with a `[Symbol.dispose]()` method (a method whose name is a built-in *symbol* — a unique key the language reserves for this purpose), declared with `using`, is disposed when the enclosing block exits, however it exits. It reached TC39 stage 4 in 2025 (TC39 is the committee that designs JavaScript; stage 4 means "finished, will be in the next standard") and is slated for ES2027; Node 24 and Chromium-based browsers (Chromium is the open-source core of Chrome and Edge; Chrome 134+) already support it — check your targets before relying on it in browsers. It mirrors C#'s `using`, Python's `with` and Java's try-with-resources.

`code/09_using_and_cleanup.mjs`:

```
function openFile(name) {
  console.log(`open  ${name}`);
  return {
    name,
    read() { if (name === 'bad.txt') throw new Error(`cannot read ${name}`); return `contents of ${name}`; },
    [Symbol.dispose]() { console.log(`close ${name}`); },   // called automatically by `using`
  };
}

// Classic pattern: try/finally
function classic(name) {
  const f = openFile(name);
  try { return f.read(); }
  finally { f[Symbol.dispose](); }
}
console.log('classic ->', classic('a.txt'));

// Same thing with `using`: disposed when the block exits, in reverse order, even on error
function modern(...names) {
  using first = openFile(names[0]);
  using second = openFile(names[1]);
  return [first.read(), second.read()];
}
console.log('modern  ->', modern('a.txt', 'b.txt'));
try {
  modern('c.txt', 'bad.txt');
} catch (err) {
  console.log('modern error ->', err.message, '(both files were still closed)');
}

// If disposal ALSO throws, you get a SuppressedError holding both errors
function doubleFailure() {
  using r = { [Symbol.dispose]() { throw new Error('dispose failed'); } };
  throw new Error('body failed');
}
try { doubleFailure(); } catch (err) {
  console.log(`${err.name}: error="${err.error.message}" suppressed="${err.suppressed.message}"`);
}
```

Output:

```
open  a.txt
close a.txt
classic -> contents of a.txt
open  a.txt
open  b.txt
close b.txt
close a.txt
modern  -> [ 'contents of a.txt', 'contents of b.txt' ]
open  c.txt
open  bad.txt
close bad.txt
close c.txt
modern error -> cannot read bad.txt (both files were still closed)
SuppressedError: error="dispose failed" suppressed="body failed"
```

- Resources are disposed in **reverse** order of declaration (last opened, first closed) — like a stack.
- `await using` does the same with `[Symbol.asyncDispose]()` for async cleanup (closing a DB connection).
- If both the body and the disposal throw, the result is a **`SuppressedError`**: `.error` is the disposal error, `.suppressed` the one it replaced — nothing is silently lost.

TL;DR

Release resources in `finally`, or with `using` + `[Symbol.dispose]` where supported. Disposal runs in reverse order, even on errors.

11.7 The console toolbox

What it is

`console` is the built-in logging object in browsers and Node. `console.log` is only the start; the other methods save a lot of time when debugging. In the browser they print to the DevTools **Console** panel (with filtering by level); in Node, `log/info/debug/table/dir/group/count/time` write to **stdout** (standard output, the normal output stream of a program) and `warn/error/trace/assert` to **stderr** (standard error, a separate stream meant for problems, so it can be redirected on its own).

`code/06_console_methods.mjs`:

```
console.log('log   : plain message', { id: 7 }, [1, 2]);
console.info('info  : alias of log in Node');
console.debug('debug : alias of log in Node (hidden by default in browser DevTools "Verbose" level)');
console.log('format: %s is %d years, %o', 'Ada', 36, { lang: 'js' });

const users = [
  { id: 1, name: 'Ada', role: 'admin' },
  { id: 2, name: 'Linus', role: 'dev' },
];
console.table(users);
console.table(users, ['name']);           // choose columns

const deep = { a: { b: { c: { d: { e: 'deep' } } } } };
console.log('default depth:', deep);
console.dir(deep, { depth: null });          // print everything

console.group('group: checkout');
console.log('validate cart');
console.group('payment');
console.log('charge card');
console.groupEnd();
console.groupEnd();

for (const k of ['a', 'b', 'a', 'a']) console.count(k);
console.countReset('a');
console.count('a');

console.assert(1 + 1 === 2, 'not printed: assertion holds');
console.assert(0.1 + 0.2 === 0.3, 'assertion failed: floating point strikes again'); // prints, does NOT throw

const label = 'sum loop';
console.time(label);
let s = 0; for (let i = 0; i < 1e6; i++) s += i;
console.timeEnd(label);                      // prints elapsed time (varies per run)

function inner() { console.trace('trace: how did we get here?'); }
inner();
```

Output (the timing varies per run; the trace's long file paths are shortened here):

```
log   : plain message { id: 7 } [ 1, 2 ]
info  : alias of log in Node
debug : alias of log in Node (hidden by default in browser DevTools "Verbose" level)
format: Ada is 36 years, { lang: 'js' }
┌─────────┬────┬─────────┬─────────┐
│ (index) │ id │ name    │ role    │
├─────────┼────┼─────────┼─────────┤
│ 0       │ 1  │ 'Ada'   │ 'admin' │
│ 1       │ 2  │ 'Linus' │ 'dev'   │
└─────────┴────┴─────────┴─────────┘

┌─────────┬─────────┐
│ (index) │ name    │
├─────────┼─────────┤
│ 0       │ 'Ada'   │
│ 1       │ 'Linus' │
└─────────┴─────────┘
default depth: { a: { b: { c: [Object] } } }
{
  a: {
    b: { c: { d: { e: 'deep' } } }
  }
}
group: checkout
  validate cart
  payment
    charge card
a: 1
b: 1
a: 2
a: 3
a: 1
Assertion failed: assertion failed: floating point strikes again
sum loop: 1.871ms
Trace: trace: how did we get here?
    at inner (file:///…/11-errors-and-debugging/code/06_console_methods.mjs:38:28)
    at file:///…/11-errors-and-debugging/code/06_console_methods.mjs:39:1
    at ModuleJob.run (node:internal/modules/esm/module_job:377:25)
    …
```

Method	Use it for
`log` / `info` / `debug`	General output; in browsers they're separate filter levels
`warn` / `error`	Problems — shown in yellow/red in DevTools, go to stderr in Node, include a stack in browsers
`%s %d %i %f %o %O %c`	Format specifiers (string, integer, float, object; `%c` applies CSS styling in browsers)
`table(data, cols?)`	Arrays of objects → readable grid
`dir(obj, { depth })`	Full object inspection (Node's `console.log` stops at depth 2 and shows `[Object]`)
`group` / `groupCollapsed` / `groupEnd`	Indent related messages
`count(label)` / `countReset`	"How many times did this run?"
`assert(cond, msg)`	Log only when `cond` is false — **does not throw**
`time` / `timeLog` / `timeEnd`	Quick timing (use `performance.now()` or a profiler for serious measurement)
`trace(msg)`	Print the current stack without throwing

Common mistakes

- **Logging objects in the browser and reading them later**: DevTools shows a *live* reference; when you expand it, you see the object's *current* state, not the state at log time. Log a snapshot:

`console.log(structuredClone(obj))` (a built-in that makes a deep copy) or `JSON.stringify(obj)`.

- Leaving debug logs in production code (the live version real users run) — use a lint rule (`no-console`) or a logger with levels (a logging library where each message is tagged debug/info/warn/error and you choose which levels to keep).
- `console.log('user: ' + user)` → `user: [object Object]`. Pass objects as separate arguments: `console.log('user:', user)`.

TL;DR

Use `table`, `dir`, `group`, `count`, `assert`, `time`, `trace` — and remember browser logs of objects are live references.

11.8 Reading stack traces and source maps

What it is

A **stack trace** lists the chain of function calls active when the error was created — **newest first**. Each line ("frame") shows the function name and the location `file:line:column`. Reading it top-down tells you *where* it failed; reading bottom-up tells you *how you got there*.

`code/07_stack_traces.mjs` (key parts):

```
function validate(cart) { if (cart.items.length === 0) throw new Error('empty cart'); }
function checkout(cart) { validate(cart); }
function handleClick() { checkout({ items: [] }); }

try {
  handleClick();
} catch (err) {
  // shorten absolute file paths so the output is the same on every machine
  const pretty = err.stack.replace(/file:\/\/\S+\//g, '').replace(/\(.*[\\/](\S+?:\d+:\d+)\)/g, '($1)');
  console.log(pretty.split('\n').slice(0, 5).join('\n'));
}
console.log('\nError.stackTraceLimit default:', Error.stackTraceLimit);
// ... Error.captureStackTrace(obj) attaches a .stack to any object
```

Output:

```
Error: empty cart
    at validate (07_stack_traces.mjs:4:62)
    at checkout (07_stack_traces.mjs:5:27)
    at handleClick (07_stack_traces.mjs:6:26)
    at 07_stack_traces.mjs:9:3

Error.stackTraceLimit default: 10
frames mention makeTimeout? true
captureStackTrace on plain object: string 07_stack_traces.mjs
```

How to read it:

1. **Line 1** = `name: message` — *what* happened.
2. **First frame** = where it was thrown (`validate`, line 4, column 62).
3. **Following frames** = callers. The first frame that's in *your* code (not a library or `node:internal`) is usually where to start looking.
4. **Anonymous frames** (`at 07_stack_traces.mjs:9:3`) are top-level code or unnamed functions — naming your functions makes traces easier to read.

V8-specific helpers (Node, Chrome, Edge — not standard): `Error.stackTraceLimit` (frames kept, default 10; raise it when debugging deep stacks, or run Node with `--stack-trace-limit=50`), and `Error.captureStackTrace(obj, fn?)`, which attaches a stack to any object (optionally hiding frames above `fn`). Async stack traces: V8 stitches together frames across `await` points (you'll see `at async functionName`), so `async/await` code has far more useful traces than old callback code.

Source maps

Production JavaScript is usually bundled (many files merged into a few) and minified (whitespace removed and names shortened to make the file smaller) (Chapter 10), so a raw trace says something like `at e (app.3f9a1c.js:1:48213)` — useless. A **source map** (`app.js.map`) maps each position in the generated file back to the original file, line and column. DevTools applies source maps automatically when it finds a `//# sourceMappingURL=` comment; Node applies them to stack traces with `node --enable-source-maps`. Error-tracking services let you upload maps privately so production traces are readable without publishing your source.

TL;DR

Read the first line for *what*, the first frame in your code for *where*, the rest for *how*. Use source maps for bundled code; name functions for clearer traces.

11.9 Debugging with DevTools and the Node inspector

What it is

A **debugger** lets you **pause** a running program at a chosen line, **inspect** every variable in scope, and **step** through the code one line at a time. It's far faster than adding `console.log`s and re-running when the problem is "why is this value wrong here?". Browsers ship one in **DevTools** (F12, or Cmd+Option+I on macOS); Node exposes the same Chrome DevTools protocol (the message format DevTools uses to control a running program) through its **inspector**.

Core concepts

Concept	What it does
Breakpoint	Pauses execution when a line is about to run. Set by clicking a line number (Sources panel in Chrome, Debugger in Firefox) or with a `debugger;` statement (ignored when no debugger is attached).
Conditional breakpoint	Pauses only when an expression is true (`item.qty > 100`) — essential inside loops.
Logpoint	Logs a message without pausing and without editing the code.
Pause on exceptions	Stops at the line that throws (optionally including caught exceptions) — shows the state at the moment of failure.
Step over (F10)	Run the current line, including any functions it calls, and stop at the next line.
Step into (F11)	Enter the function called on this line.
Step out (Shift+F11)	Finish the current function and stop in its caller.
Resume (F8)	Continue until the next breakpoint.
Call Stack pane	The frames (as in a stack trace) — click one to inspect that caller's variables.
Scope pane	Local, closure and global variables for the selected frame (including closures from Chapter 4 and private `#fields`).
Watch expressions	Expressions re-evaluated at every pause.
Console while paused	Evaluate any expression in the paused scope, even change variables.

(Keyboard shortcuts are Chrome DevTools' defaults; VS Code uses F10/F11/Shift+F11 too, with F5 to continue.)

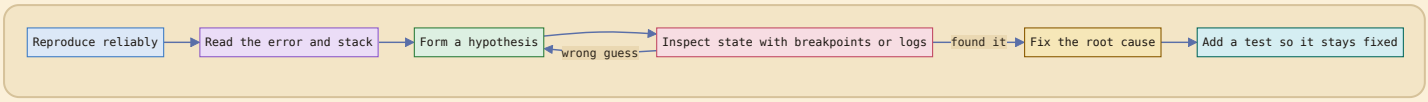
Browser-only extras: **DOM breakpoints** (the DOM — Document Object Model — is the browser's tree of objects representing the page; these pause when an element's subtree/attributes change), **event listener breakpoints** (pause on any `click`), **XHR/fetch breakpoints** (XHR = `XMLHttpRequest`, the older API for HTTP requests; these pause when a URL is requested), the **Network** panel (see every request, status and response — the first stop for "the API call failed" bugs), **Performance** and **Memory** panels (profiling — measuring where time goes — and heap snapshots, a dump of every object in memory, for finding memory leaks: memory that is never freed because something still references it), and an **ignore list** that hides framework/library frames when stepping.

Debugging Node (not run — interactive)

```
node --inspect app.mjs          # start with the inspector on 127.0.0.1:9229
node --inspect-brk app.mjs      # same, but pause on the first line (so you can set breakpoints)
# then open chrome://inspect in Chrome -> "Open dedicated DevTools for Node"
node inspect app.mjs            # built-in terminal debugger: c(ont), n(ext), s(tep), o(ut), repl, bt
node --enable-source-maps dist/app.js   # readable traces for transpiled code
node --watch app.mjs            # restart automatically on save while you fix things
```

In **VS Code**, open a "JavaScript Debug Terminal" and run the program normally — breakpoints set in the editor just work, for Node scripts, test runners and dev servers alike. Never expose `--inspect` on a public network interface (an address other machines can reach, unlike `127.0.0.1`, which only your own computer can): anyone who can connect can run code in your process.

A systematic method



1. **Reproduce** the bug reliably, with the smallest input that triggers it.
2. **Read** the actual error message and stack (really read it).
3. **Hypothesize** what's wrong, then **inspect** the values that would confirm or refute it — breakpoint/`debugger` or targeted, labelled logs.
4. **Narrow down** by halving: is the value already wrong at the midpoint? (`git bisect` does the same across commits — it binary-searches your Git history, the saved snapshots of your code, to find the change that introduced the bug.)
5. **Fix the root cause**, not the symptom.
6. **Add a test** that fails without the fix (Chapter 16).

Explaining the code line by line to someone — or to a rubber duck — works surprisingly often, because it forces you to state your assumptions.

Worked example

`code/10_debugging_walkthrough.mjs`: a cart total that comes out wrong.

```
import assert from 'node:assert/strict';

// Data as it arrives from an HTML form or query string: everything is a string.
const cart = [
  { name: 'Book', price: '12.50', qty: '2' },
  { name: 'Pen', price: '1.25', qty: '4' },
];

// BUGGY version
function totalBuggy(items) {
  return items.reduce((sum, item) => sum + item.price * item.qty, '');
}
console.log('symptom       :', totalBuggy(cart), '(expected 30)');

// Inspect the intermediate values — labelled and structured
function totalTraced(items) {
  return items.reduce((sum, item, i) => {
    const next = sum + item.price * item.qty;
    console.log(`  step ${i}:`, { sum, typeofSum: typeof sum, line: item.price * item.qty, next });
    // debugger;   // <- with DevTools / `node inspect` attached, execution would pause here
    return next;
  }, '');
}
console.log('inspect       :');
totalTraced(cart);

// Fix: numeric initial value + explicit conversion at the boundary.
function total(items) {
  return items.reduce((sum, item) => sum + Number(item.price) * Number(item.qty), 0);
}
console.log('fixed         :', total(cart));

assert.equal(total(cart), 30);
assert.equal(total([]), 0);
assert.throws(() => assert.equal(totalBuggy(cart), 30), assert.AssertionError);
console.log('tests         : passed');
```

Output:

```
symptom       : 255 (expected 30)
inspect       :
  step 0: { sum: '', typeofSum: 'string', line: 25, next: '25' }
  step 1: { sum: '25', typeofSum: 'string', line: 5, next: '255' }
fixed         : 30
tests         : passed
```

The inspection shows `typeofSum: 'string'` from the very first step: the initial value `''` made `+` concatenate (`'25' + 5` → `'255'`), while `*` had quietly converted the strings to numbers — a coercion rule from Chapter 2. No error was thrown at all, which is why these bugs are hard: **the most dangerous bugs are wrong values, not exceptions.** Converting input at the boundary (`Number(...)`) and adding tests prevents the whole class of bug. TypeScript (JavaScript plus type annotations that a compiler checks before the code runs) would also have flagged `string * string` (Chapter 16).

In the interview

30-second answer: "I reproduce the bug first, read the stack trace, then use breakpoints — often conditional ones or 'pause on exceptions' — to inspect the actual state instead of guessing. I step over/into to follow the flow and check the Network panel for API issues. For Node I use `--inspect` with Chrome DevTools or VS Code's debug terminal. Once fixed, I add a test that reproduces the bug."

- *Q: What's a logpoint?* A: A breakpoint that logs an expression instead of pausing — logging without editing code.
- *Q: How do you debug minified production errors?* A: Source maps, uploaded to the error tracker or loaded in DevTools.

- *Q: What does the `debugger` statement do without DevTools open?* A: Nothing.

> **TL;DR**

Pause, inspect, step: breakpoints (conditional, logpoints, pause-on-exceptions), Scope/Watch/Call Stack panes, Network panel. Node: `--inspect`/`--inspect-brk` + `chrome://inspect` or VS Code. Reproduce → hypothesize → inspect → fix root cause → add a test.

11.10 Common runtime errors — a field guide

`code/08_common_runtime_errors.mjs` triggers twenty everyday mistakes and prints V8's exact messages (what you'll see in Node, Chrome and Edge):

```
const gallery = [
  ['call a non-function', () => { const user = {}; user.getName(); }],
  ['property of undefined', () => { const res = {}; return res.data.items; }],
  ['property of null', () => { const el = null; return el.textContent; }],
  ['set property on undefined', () => { let cfg; cfg.debug = true; }],
  ['undeclared variable', () => totl * 2],
  ['TDZ (use before let)', () => { const f = () => x; f(); let x = 1; }],
  ['reassign const', () => { const n = 1; n = 2; }],
  ['not iterable', () => { const [a] = {}; }],
  ['spread non-iterable', () => [...123]],
  ['JSON.parse(undefined)', () => JSON.parse(undefined)],
  ['JSON.parse(HTML)', () => JSON.parse('<!DOCTYPE html>')],
  ['class without new', () => { class Cart {} Cart(); }],
  ['reduce empty array', () => [].reduce((a, b) => a + b)],
  ['BigInt + Number', () => 1n + 1],
  ['bad array length', () => new Array(2 ** 32)],
  ['toFixed digits', () => (1.5).toFixed(101)],
  ['infinite recursion', () => { const f = () => f(); f(); }],
  ['circular JSON', () => { const a = {}; a.self = a; JSON.stringify(a); }],
  ['frozen object (strict)', () => { const o = Object.freeze({ a: 1 }); o.a = 2; }],
  ['in on a primitive', () => 'x' in 'hello'],
];
for (const [what, fn] of gallery) {
  try { fn(); console.log(`${what.padEnd(26)} (no error)`); }
  catch (e) { console.log(`${what.padEnd(26)} ${e.name}: ${e.message.split('\n')[0]}`); }
}
```

Output:

```
call a non-function        TypeError: user.getName is not a function
property of undefined      TypeError: Cannot read properties of undefined (reading 'items')
property of null           TypeError: Cannot read properties of null (reading 'textContent')
set property on undefined  TypeError: Cannot set properties of undefined (setting 'debug')
undeclared variable        ReferenceError: totl is not defined
TDZ (use before let)       ReferenceError: Cannot access 'x' before initialization
reassign const             TypeError: Assignment to constant variable.
not iterable               TypeError: {} is not iterable
spread non-iterable        TypeError: 123 is not iterable
JSON.parse(undefined)      SyntaxError: "undefined" is not valid JSON
JSON.parse(HTML)           SyntaxError: Unexpected token '<', "<!DOCTYPE html>" is not valid JSON
class without new          TypeError: Class constructor Cart cannot be invoked without 'new'
reduce empty array         TypeError: Reduce of empty array with no initial value
BigInt + Number            TypeError: Cannot mix BigInt and other types, use explicit conversions
bad array length           RangeError: Invalid array length
toFixed digits             RangeError: toFixed() digits argument must be between 0 and 100
infinite recursion         RangeError: Maximum call stack size exceeded
circular JSON              TypeError: Converting circular structure to JSON
frozen object (strict)     TypeError: Cannot assign to read only property 'a' of object '#<Object>'
in on a primitive          TypeError: Cannot use 'in' operator to search for 'x' in hello
```

What each usually means in real code

Message (V8)	Usual real-world cause	Fix
`x is not a function`	Typo in a method name; wrong import (default vs named, Chapter 10); calling a property that holds data; a library version without that method	Check the name and the import; `console.log(typeof x)`
`Cannot read properties of undefined (reading 'y')`	Data not loaded yet (async), wrong shape of an API response, array index out of range, missing `return`	Find *which* value is `undefined` (the part before `.y`); use `?.` and defaults only if absence is legitimate
`Cannot set properties of undefined`	Writing into a nested object that doesn't exist	Create it first, or `obj.cfg ??= {}`
`x is not defined`	Typo, missing import, variable in another scope, browser-only global used in Node (`window`, `document`)	Declare/import it; check the environment
`Cannot access 'x' before initialization`	`let`/`const`/`class` used before its line (TDZ), often via circular imports (two modules importing each other)	Reorder code; break import cycles
`Assignment to constant variable.`	Reassigning a `const` or an imported binding	Use `let`, or mutate instead of reassign
`… is not iterable` (iterable = loopable with `for…of`)	`for…of` / spread / destructuring on a plain object, `undefined`, or a number	Use `Object.entries(obj)`, check for `undefined`
`"undefined" is not valid JSON` / `Unexpected token '<' … is not valid JSON`	Parsing a missing value; the server returned an HTML error page instead of JSON	Check `response.ok` and the content type (`Content-Type` header) before `response.json()` (Chapter 14)
`Class constructor X cannot be invoked without 'new'`	Calling a class like a function	Add `new`
`Reduce of empty array with no initial value`	`reduce` on a possibly empty array	Always pass an initial value
`Cannot mix BigInt and other types`	`bigint + number`	Convert explicitly: `BigInt(n)` or `Number(b)`
`Maximum call stack size exceeded`	Infinite or too-deep recursion (a function calling itself); a setter that assigns to itself; two functions calling each other	Fix the base case (the input where recursion stops); iterate instead
`Converting circular structure to JSON`	Serializing objects with back-references (DOM nodes, parent links, some library objects)	Pick the fields you need, or use a replacer (a filter function passed to `JSON.stringify`)
`Cannot assign to read only property`	Writing to a frozen object or a getter-only accessor in strict code (strict mode: always on in modules and classes)	Copy instead of mutating: `{ ...obj, a: 2 }`
`Cannot use 'in' operator to search for …`	`in` on a string/number (often an un-parsed JSON string)	Parse first; `in` works on objects only

Other messages worth recognizing: `Unexpected token` / `Unexpected end of input` (syntax: a missing bracket or comma), `await is only valid in async functions and the top level bodies of modules`, `Cannot use import statement outside a module` (Chapter 10), and `Failed to fetch` (network/CORS failure in browsers — CORS is the browser rule that blocks reading another site's responses unless that site allows it; a `TypeError` — Chapter 14).

Self-check

1. `Cannot read properties of undefined (reading 'map')` — what is `undefined`? → The value *before* `.map`, not `map` itself.
2. `JSON.parse` fails with `Unexpected token '<'` after a `fetch` — likely cause? → The server returned HTML (an error page or `index.html`), not JSON.
3. Why does `[].reduce((a, b) => a + b)` throw but `[].reduce((a, b) => a + b, 0)` doesn't? → With no initial value, reduce needs at least one element to start from.

TL;DR

Most runtime errors are `TypeError`s from `undefined`/`null` or wrong types. Read the message literally: it names the value and the property. Find where the bad value came from, not just where it exploded.

Chapter 11 TL;DR

- Errors are objects (`name`, `message`, `stack`, optional `cause`); core types: `TypeError`, `ReferenceError`, `SyntaxError`, `RangeError`, `URIError`, `AggregateError`, plus `SuppressedError` for `using`. `Error.isError` (ES2026) is the robust check.
- `try` → `catch` → `finally`: finally always runs; never `return` from it. Catch narrowly, rethrow the rest, always throw `Error` objects. try/catch is synchronous.
- **Custom errors**: `extends Error`, `super(msg, { cause })`, set `name`, add `code`/data, handle centrally, don't leak internals.
- `cause` chains context onto the original error; walk `.cause` to the root.
- **Async**: `await` inside `try` or `.catch()` every promise; global `error`/`unhandledrejection` (browser) and `uncaughtException`/`unhandledRejection` (Node) handlers are for logging.
- **Cleanup**: `finally`, or `using` + `Symbol.dispose` (ES2027, Node 24).
- **Console**: `table`, `dir`, `group`, `count`, `assert`, `time`, `trace`; browser object logs are live.
- **Stack traces**: newest frame first; source maps for bundled code.
- **Debuggers**: breakpoints (conditional, logpoints, pause on exceptions), step over/into/out, Scope/Watch/Call Stack, Network panel; Node `--inspect` + `chrome://inspect` or VS Code.
- **Method**: reproduce → read → hypothesize → inspect → fix root cause → add a test.

Next: the event loop, promises and `async`/`await` — Chapter 12: Asynchronous JavaScript.

Chapter 12 — Asynchronous JavaScript

JavaScript runs your code on **one thread**, yet a web page can download data, animate, and respond to clicks at the same time, and a Node server (Node.js runs JavaScript outside the browser) can handle thousands of connections. The trick is **asynchrony**: slow work is handed to the host (browser or Node), and your code is called back later, when the result is ready. The piece that decides *when* each callback (a function you hand over to be run later) runs is the **event loop**. This chapter explains the event loop precisely — precisely enough to predict the output of tricky ordering puzzles — and then the three generations of async code built on top of it: callbacks, promises, and `async`/`await`.

What you'll learn

1. Why JavaScript is single-threaded and what "blocking" means.
2. The event loop: call stack, task queue, microtask queue, and rendering — with diagrams.
3. Seven ordering puzzles, each run for real in Node **and** Chrome.
4. Node's own additions: `process.nextTick`, `setImmediate`, and the loop phases.
5. Timers: `setTimeout`, `setInterval`, and their guarantees (and non-guarantees).
6. Callbacks, error-first callbacks, and why "callback hell" led to promises.
7. Promises: states, chaining, error flow, `withResolvers`, `Promise.try`.
8. The combinators `all`, `allSettled`, `race`, `any`.
9. `async`/`await`: sequential vs parallel, loops, error handling.
10. Cancellation with `AbortController` and `AbortSignal`.
11. Async iteration: `for await…of`, async generators, `Array.fromAsync`.
12. Production patterns: retry with backoff, concurrency limits, stale responses.

Before you start. You need functions and closures (Chapter 4), and generators/iterators (Chapter 8) for §12.12. Errors and `try`/`catch` are covered in Chapter 11 — §11.5 previewed async errors; this chapter completes the story. Top-level `await` in modules is explained in Chapter 10.

All runnable code is in `code/`. Outputs are real (Node 24). Where timing matters, programs print rounded or yes/no values so the output is stable between runs.

12.1 One thread, and why blocking hurts

What it is

A **thread** is one sequence of instructions being executed. JavaScript gives your code exactly one: at any moment, at most one line of *your* JavaScript is running. **Synchronous** code runs start to finish without pausing. **Asynchronous** code starts an operation now and handles its result later, through a callback.

Analogy: a restaurant with one chef. A good chef doesn't stand watching the oven for 20 minutes — they put the dish in, set a timer (asynchronous), and cook other orders. When the timer rings, they come back to that dish. A bad chef who stares at the oven (blocking) makes every other customer wait.

Terms:

- **Blocking** — code that keeps the thread busy, so nothing else (other callbacks, clicks, rendering — drawing the page on screen) can run.
- **Non-blocking I/O** — input/output (network, disk) performed by the host in the background; JavaScript only runs when the result arrives.
- **Host / runtime** — the environment around the engine: the browser (Web APIs — the extra features a browser provides, such as timers, `fetch` and the page itself) or Node (libuv, its C library for async I/O). The engine (V8, Google's JavaScript engine inside Chrome and Node) runs JavaScript; the host does the waiting.

Why it matters

In a browser, the same thread that runs your JavaScript also handles user input and (usually) page rendering. A 200 ms synchronous loop freezes scrolling, typing and animation for 200 ms. In Node, a blocking handler stalls *every* request the server is handling.

`code/01_single_thread.mjs`:

```
const start = Date.now();
setTimeout(() => {
  const waited = Date.now() - start;
  console.log(`timer asked for 0 ms, ran after >= 200 ms: ${waited >= 200}`);
}, 0);

console.log('busy loop starts');
while (Date.now() - start < 200) { /* burn CPU: nothing else can run */ }
console.log('busy loop ends (the timer is still waiting)');

// Asynchronous work does NOT block: the callback is queued and we carry on.
import { readFile } from 'node:fs/promises';
readFile(new URL(import.meta.url)).then((buf) => console.log('file read finished,', buf.length > 0 ? 'got bytes' :
'empty'));
console.log('readFile started - main script keeps going');
```

Output:

```
busy loop starts
busy loop ends (the timer is still waiting)
readFile started - main script keeps going
timer asked for 0 ms, ran after >= 200 ms: true
file read finished, got bytes
```

The 0 ms timer could not interrupt the loop. JavaScript callbacks are never pre-empted (interrupted part-way through to let something else run): a callback runs to completion before anything else starts (**run-to-completion**). That rule is a gift — you never need locks (guards that stop two threads from changing the same data at the same moment) around your variables — and a responsibility: keep each piece of work short.

When you really need parallel CPU work

Heavy computation (image processing, big parsing) belongs off the main thread (the single thread that runs your page's JavaScript and its UI): **Web Workers** in the browser (Chapter 14) or `worker_threads` in Node (Chapter 15). Workers are separate threads with separate memory that talk by messages.

TL;DR

One thread runs your JavaScript; the host waits for I/O and timers in the background. Long synchronous work blocks everything, including 0 ms timers and rendering.

12.2 The event loop

What it is

The **event loop** is the host's scheduler. It repeatedly picks a piece of waiting work and runs it on the single JavaScript thread. The parts:

Part	What it holds	Examples
Call stack	Functions currently executing, newest on top	Your main script, a callback in progress
Host APIs (Web APIs / libuv)	Operations in progress *outside* JavaScript	A timer counting down, an HTTP request (a message asking a web server for data), a file read
Task queue (a.k.a. macrotask / callback queue)	Callbacks ready to run, one per loop turn	Timer callbacks, I/O callbacks, `message` events, user events like `click`
Microtask queue	Short follow-up jobs that must run "as soon as the current code finishes"	Promise `then` / `catch` / `finally` callbacks, code after `await`, `queueMicrotask`, `MutationObserver` (watches the page for changes) callbacks
Rendering (browsers)	Style (which CSS applies), layout (sizes and positions), paint (drawing pixels) — typically up to ~60 times a second	`requestAnimationFrame` callbacks run just before paint

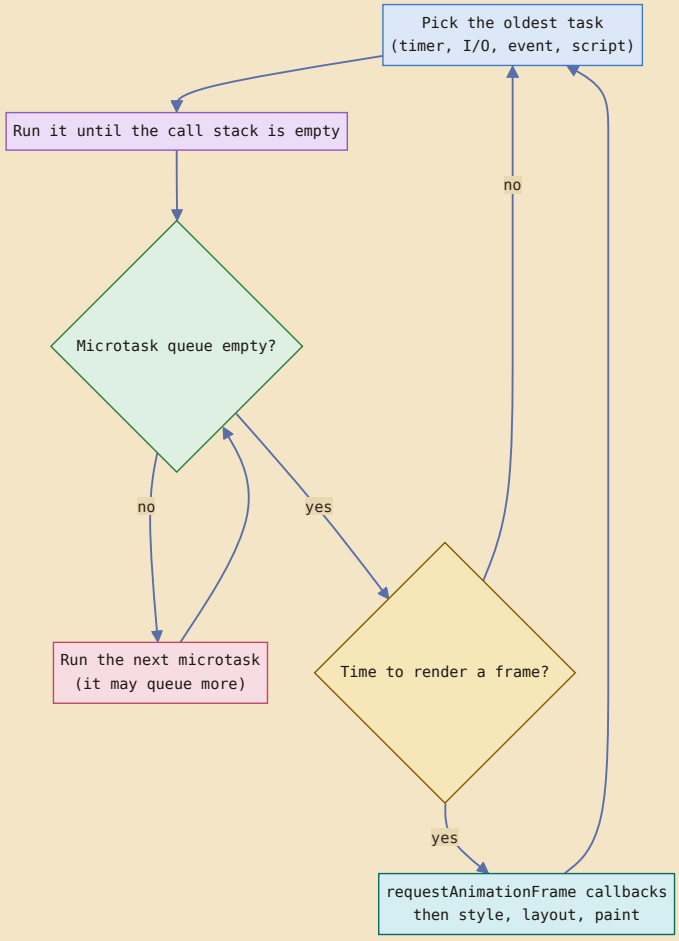
The HTML specification (section "Event loops") defines the browser algorithm; ECMAScript (the official name of the JavaScript language standard) itself only defines **jobs** (promise reactions) and leaves scheduling to the host. The spec actually allows several task queues (for example, user-input tasks may be prioritised over timers), but for predicting order you can treat tasks as one FIFO (first in, first out — like a queue at a shop) queue per source.

How it works

One turn of the browser event loop, simplified from the HTML spec:

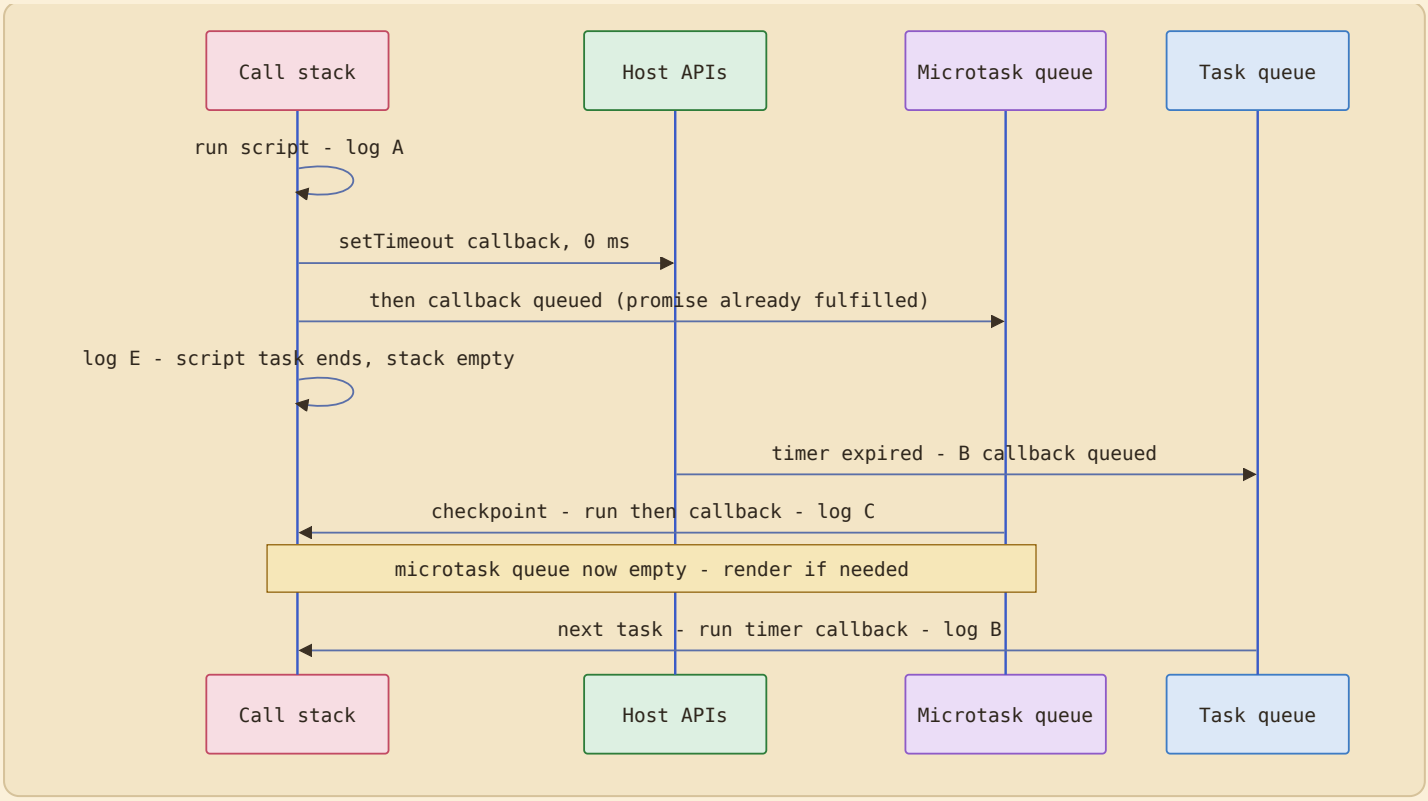
1. **Run one task** — take the oldest runnable task and run its callback until the call stack is empty.
2. **Perform a microtask checkpoint** — run microtasks until the microtask queue is **empty**, including any microtasks queued *by* microtasks.
3. **Maybe render** — if it's time for a new frame (one fresh picture of the page on screen) (and something changed): run `requestAnimationFrame` callbacks, recalculate style and layout, paint.
4. Go to step 1 (or sleep until a task arrives).

The initial script is itself a task, so step 2 happens right after your top-level code finishes. Microtasks also run whenever the call stack becomes empty during a task — for example between two event listeners when the user physically clicks (but not when you call `el.click()` from code, because your code is still on the stack).



The same flow as a sequence over time, for this code:

```
console.log('A');
setTimeout(() => console.log('B timeout'), 0);
Promise.resolve().then(() => console.log('C then'));
console.log('E');
```



Output: `A`, `E`, `C then`, `B timeout`.

Why microtasks exist

A promise callback should run **after** the current synchronous code (so behaviour is consistent — callbacks are always asynchronous) but **before** anything else gets a chance to observe inconsistent state (no other task, no render in between). Microtasks give exactly that "at the end of the current job" timing.

Common mistakes

- **Thinking `setTimeout(fn, 0)` runs "immediately".** It runs in a *later task*, after every pending microtask and possibly after a render.
- **Microtask starvation.** A microtask that always queues another microtask (e.g. a recursive `.then` loop) never lets the loop reach step 3 or step 1 — the page freezes exactly like a `while(true)`. Use `setTimeout` (a task) to yield (hand the thread back so other work can run) when chunking long work (splitting it into small pieces).
- **Expecting `await` to yield to rendering.** `await` only schedules a microtask; the browser can't paint in between. To let the page update during a long job, split it across tasks (`setTimeout`, `scheduler.yield()` where supported, or `requestAnimationFrame`).

TL;DR

Loop: one task → **all** microtasks → maybe render → repeat. Promises and `await` continuations are microtasks; timers, I/O and events are tasks.

12.3 Ordering puzzles

Interviews love "what does this print?" questions. With the rules from §12.2 you can answer any of them:

1. Run all synchronous code first (including promise **executors** — the function you pass to `new Promise` — and code before the first `await`).
2. Then drain the microtask queue in FIFO order, adding new microtasks to the end as they're created.

3. Then take the next task (timers in order of due time; equal times in creation order), run it, drain microtasks again, and so on.

`code/02_ordering_puzzles.mjs` runs seven puzzles. Each one logs into an array, the program waits for all queues to drain, then prints the order. (Try predicting each before reading the answer.)

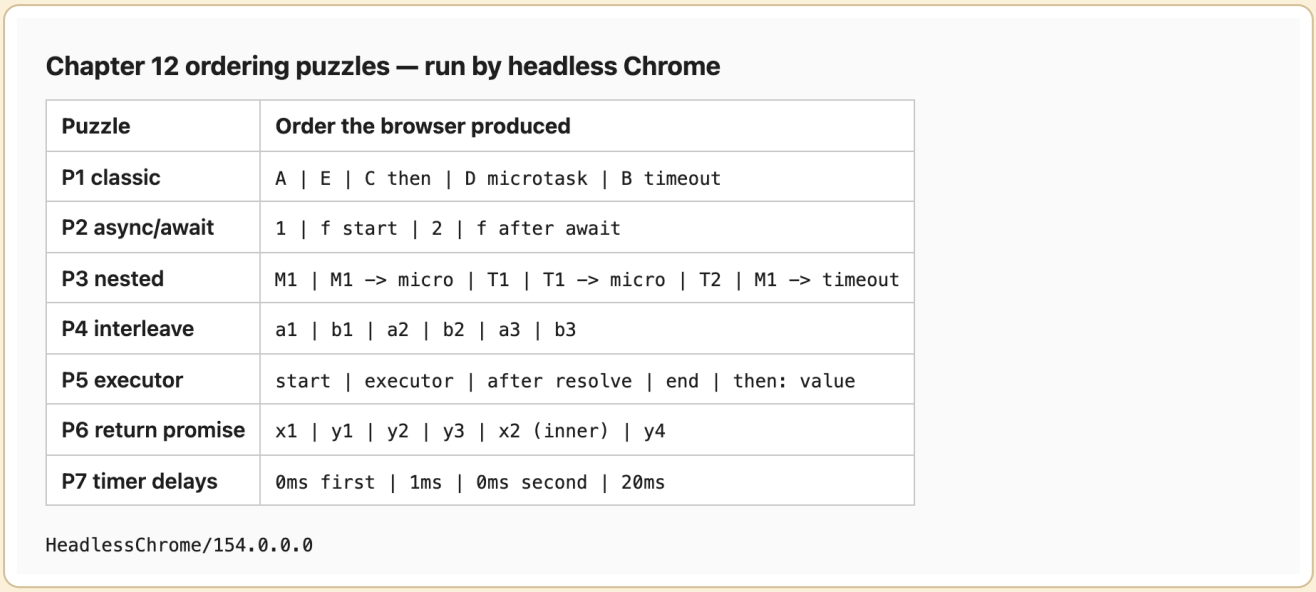
```
// P1 - the classic: sync code, then ALL microtasks, then the next task
puzzle('P1 classic', (log) => {
  log('A');
  setTimeout(() => log('B timeout'), 0);
  Promise.resolve().then(() => log('C then'));
  queueMicrotask(() => log('D microtask'));
  log('E');
});

// P2 - async/await: code before the first await runs synchronously
puzzle('P2 async/await', (log) => {
  async function f() {
    log('f start');
    await null;             // everything after this line is a microtask
    log('f after await');
  }
  log('1');
  f();
  log('2');
});

// P3 - microtasks queued BY microtasks still run before the next task
puzzle('P3 nested', (log) => {
  setTimeout(() => {
    log('T1');
    Promise.resolve().then(() => log('T1 -> micro'));
  }, 0);
  setTimeout(() => log('T2'), 0);
  Promise.resolve().then(() => {
    log('M1');
    setTimeout(() => log('M1 -> timeout'), 0);
    Promise.resolve().then(() => log('M1 -> micro'));
  });
});

// P4 - two promise chains interleave one step at a time
puzzle('P4 interleave', (log) => {
  Promise.resolve().then(() => log('a1')).then(() => log('a2')).then(() => log('a3'));
  Promise.resolve().then(() => log('b1')).then(() => log('b2')).then(() => log('b3'));
});

// P5 - the executor of new Promise runs synchronously; then() callbacks do not
puzzle('P5 executor', (log) => {
  log('start');
  const p = new Promise((resolve) => {
    log('executor');
    resolve('value');
    log('after resolve');   // resolve() does not stop the function
  });
  p.then((v) => log(`then: ${v}`));
  log('end');
});

// P6 - returning a promise from then() costs extra microtask ticks
puzzle('P6 return promise', (log) => {
  Promise.resolve()
    .then(() => { log('x1'); return Promise.resolve('inner'); }) // adopts: +2 ticks
    .then((v) => log(`x2 (${v})`));
  Promise.resolve()
    .then(() => log('y1'))
    .then(() => log('y2'))
    .then(() => log('y3'))
    .then(() => log('y4'));
});

// P7 - timers with different delays: order is by due time, then by creation
puzzle('P7 timer delays', (log) => {
  setTimeout(() => log('20ms'), 20);
  setTimeout(() => log('0ms first'), 0);
  setTimeout(() => log('1ms'), 1);
```

```
  setTimeout(() => log('0ms second'), 0);
});
// ... the harness runs each puzzle, waits 50 ms, prints the collected order
```

Output (Node 24):

```
P1 classic         -> A | E | C then | D microtask | B timeout
P2 async/await     -> 1 | f start | 2 | f after await
P3 nested          -> M1 | M1 -> micro | T1 | T1 -> micro | T2 | M1 -> timeout
P4 interleave      -> a1 | b1 | a2 | b2 | a3 | b3
P5 executor        -> start | executor | after resolve | end | then: value
P6 return promise  -> x1 | y1 | y2 | y3 | x2 (inner) | y4
P7 timer delays    -> 0ms first | 1ms | 0ms second | 20ms
```

The same puzzles, pasted into a web page (`code/puzzles_browser.html`) and run by headless Chrome (Chrome running without a visible window, driven by a program), give the identical order — promise and timer ordering is standardized, not a Node quirk:



Chapter 12 ordering puzzles — run by headless Chrome

Puzzle	Order the browser produced
P1 classic	A | E | C then | D microtask | B timeout
P2 async/await	1 | f start | 2 | f after await
P3 nested	M1 | M1 -> micro | T1 | T1 -> micro | T2 | M1 -> timeout
P4 interleave	a1 | b1 | a2 | b2 | a3 | b3
P5 executor	start | executor | after resolve | end | then: value
P6 return promise	x1 | y1 | y2 | y3 | x2 (inner) | y4
P7 timer delays	0ms first | 1ms | 0ms second | 20ms

HeadlessChrome/154.0.0.0

Figure — the ordering puzzles run in Chrome 154 (screenshot of the page after the script finished).

The explanations

Puzzle	Why
P1	`A`, `E` are synchronous. `C` and `D` are microtasks, run in the order queued. `B` is a task, so it waits for the microtask queue to empty.
P2	Calling an async function runs its body synchronously **up to the first** `await`. The rest is a microtask, so `2` prints before `f after await`.
P3	First the script's microtask `M1` runs; it queues a timer (behind T1 and T2) and a microtask, which runs immediately in the same checkpoint. Then task T1 runs; its microtask runs **before** task T2. Finally the timer queued by M1.
P4	Each `.then` in a chain is only queued when the previous one settles, so two chains alternate: a1, b1, a2, b2…
P5	The executor passed to `new Promise` runs synchronously. `resolve()` doesn't return or throw — later lines still run. `then` callbacks are always async, even on an already-resolved promise.
P6	When a `then` callback returns a promise (a **thenable**), the outer promise must *adopt* it: one microtask to call `inner.then(...)`, another for that reaction to fire, then the next `then`. So `x2` lands after `y3`. Returning a plain value would have printed `x2` right after `y1`.
P7	Timers fire in order of due time; equal due times in creation order. Node (and current Chrome) treat `0` like `1` ms here, so `1ms` sits between the two `0ms` timers — don't write code that depends on such fine differences.

P6 is why `return await somePromise` inside an `async` function used to be discouraged "for performance"; the difference is a couple of microtasks and doesn't matter in practice — and `return await` inside `try` is actually *required* to catch the rejection (§12.9).

In the interview

> **30-second answer:** "Synchronous code runs first. Then the engine drains the whole microtask queue — promise callbacks, code after `await`, `queueMicrotask` — including microtasks added along the way. Only then does the event loop take the next task, like a `setTimeout` callback or an event, and after each task it drains microtasks again. So promises always beat `setTimeout(0)`."

- *Q: Does `await` of a non-promise pause?* A: Yes — `await 5` still defers the rest of the function to a microtask.
- *Q: Why does P6's `x2` come late?* A: Adopting a returned thenable takes two extra microtask ticks.
- *Q: Can a microtask starve rendering?* A: Yes; the loop can't render or run tasks until the microtask queue is empty.

Self-check

1. `setTimeout(a, 0); queueMicrotask(b); Promise.resolve().then(c);` → order? → `b`, `c`, `a`.
2. What prints first: code inside `new Promise(executor)` or code after the `new Promise(...)` line? → The executor (it's synchronous).
3. A microtask queues a timer and another microtask. Which runs first? → The microtask.

TL;DR

Sync → all microtasks (FIFO, including new ones) → one task → all microtasks → … Executors and code before the first `await` are synchronous.

12.4 Node's event loop: `nextTick`, `setImmediate` and phases

What it is

Node's loop is implemented by **libuv** and cycles through **phases**, each with its own queue:

Phase	Runs
timers	Expired `setTimeout` / `setInterval` callbacks
pending callbacks	Some deferred system-level callbacks (e.g. certain TCP — the protocol under most network connections — errors)
poll	Waits for and runs I/O callbacks (files, sockets — a program's end of a network connection)
check	`setImmediate` callbacks
close callbacks	`'close'` events, e.g. `socket.on('close')`

Between every callback, Node drains two queues: first the **`process.nextTick` queue**, then the **promise microtask queue** (that's the order in CommonJS — Node's original `require`/`module.exports` module system; see the ES-module twist below — ES modules are the standard `import`/`export` system, also called ESM).

`code/03_node_queues.mjs`:

```
import { readFile } from 'node:fs';

const log = [];
setTimeout(() => log.push('timeout'), 0);
setImmediate(() => log.push('immediate'));
Promise.resolve().then(() => log.push('promise'));
process.nextTick(() => log.push('nextTick'));
log.push('sync');
setTimeout(() => {
  console.log('main module :', log.join(' | '));
  console.log('  (timeout vs immediate at top level can swap between runs - see text)');

  // Inside an I/O callback the order of immediate vs timeout IS guaranteed.
  readFile(new URL(import.meta.url), () => {
    const io = [];
    setTimeout(() => io.push('timeout'), 0);
    setImmediate(() => io.push('immediate'));
    process.nextTick(() => io.push('nextTick'));
    Promise.resolve().then(() => io.push('promise'));
    setTimeout(() => console.log('in I/O cb  :', io.join(' | ')), 20);
  });
}, 30);
```

Output:

```
main module : sync | promise | nextTick | immediate | timeout
  (timeout vs immediate at top level can swap between runs - see text)
in I/O cb  : nextTick | promise | immediate | timeout
```

And the CommonJS version, `code/03b_node_queues_cjs.js`:

```
const log = [];
Promise.resolve().then(() => log.push('promise'));
process.nextTick(() => log.push('nextTick'));
log.push('sync');
setTimeout(() => console.log('CommonJS main:', log.join(' | ')), 10);
```

Output:

```
CommonJS main: sync | nextTick | promise
```

Reading the results

1. **`nextTick` vs promises.** Normally `nextTick` callbacks run before promise callbacks (the I/O callback and the CommonJS script show this). But an ES module's top level is itself evaluated inside promise machinery, so its promise microtasks drain before Node gets to the `nextTick` queue — hence `promise | nextTick` in the `.mjs` main module. Don't build logic on this difference.
2. **`setImmediate` vs `setTimeout(0)` at top level** is **not deterministic** (not guaranteed to give the same result every run): it depends on whether 1 ms has already passed when the loop first checks timers. This run printed `immediate` first; another run may not.
3. **Inside an I/O callback** it *is* deterministic: after the poll phase comes the check phase, so `setImmediate` always wins over a new `setTimeout(0)`.

When to use which

Want	Use
Run after current code, before anything else (portable)	`queueMicrotask(fn)` or `Promise.resolve().then(fn)`
Node-only "right after this operation", e.g. emit an event after the constructor returns	`process.nextTick(fn)` (Node docs recommend `queueMicrotask` for most new code)
Yield to I/O, then continue	`setImmediate(fn)`
Run after a delay	`setTimeout(fn, ms)`

> **Common mistake:** recursive `process.nextTick` starves I/O exactly like recursive microtasks do. The architecture of Node (libuv, thread pool — a fixed set of helper threads that do blocking work such as file reads) is covered in Chapter 15.

TL;DR

Node phases: timers → pending → poll (I/O) → check (`setImmediate`) → close. Between callbacks: `nextTick` queue, then promises (ESM top level flips these). `setImmediate` vs `setTimeout(0)` is only guaranteed inside I/O callbacks.

12.5 Timers

What it is

Function	Does	Cancel with
`setTimeout(fn, ms, ...args)`	Run `fn(...args)` once, **at least** `ms` later	`clearTimeout(id)`
`setInterval(fn, ms, ...args)`	Run repeatedly, roughly every `ms`	`clearInterval(id)`
`queueMicrotask(fn)`	Run as a microtask	—
`requestAnimationFrame(fn)` (browser)	Run before the next paint (Ch. 14)	`cancelAnimationFrame(id)`
`setImmediate(fn)` (Node)	Run in the check phase	`clearImmediate(id)`

The returned **handle** is a number in browsers and a `Timeout` object in Node (which also has `.unref()` to let the process exit while the timer is pending, and `.refresh()`).

How it works

1. The timer is registered with the host; your code continues.
2. When the delay has elapsed, the callback is put in the task queue.
3. It runs when the loop reaches it — possibly much later if the thread is busy.

The delay is therefore a **minimum**, never a guarantee. Other rules (HTML spec + practice):

- Negative delays become `0`.
- After 5 levels of nested timers (a timer scheduling a timer scheduling…), browsers clamp delays below 4 ms up to **4 ms**.
- Browsers throttle (deliberately slow down) timers in background tabs (often to once per second or less) to save battery.
- Delays above $2^{31}-1$ ms (about 24.8 days) overflow (exceed the largest number a 32-bit integer slot can hold); Node prints a warning and uses 1 ms.

`code/04_timers.mjs`:

```js
import { setTimeout as sleep } from 'node:timers/promises';

// 1) Extra arguments are passed to the callback; the return value is a handle
const t = setTimeout((a, b) => console.log('never runs', a, b), 50, 'x', 'y');
clearTimeout(t);
console.log('cleared a timer before it fired');

// 2) setInterval until a condition, then clearInterval
let ticks = 0;
const id = setInterval(() => {
  ticks += 1;
  console.log('tick', ticks);
  if (ticks === 3) clearInterval(id);
}, 10);
await sleep(60);

// 3) The delay is a MINIMUM: a busy main thread delays the callback
const t0 = performance.now();
setTimeout(() => console.log(`asked 10 ms, got >= 40 ms: ${performance.now() - t0 >= 40}`), 10);
while (performance.now() - t0 < 40) {}
await sleep(20);

// 4) Recursive setTimeout: the gap is measured from the END of each run
let n = 0;
function poll() {
  n += 1;
  if (n < 3) setTimeout(poll, 5);
  else console.log('recursive setTimeout ran', n, 'times');
}
poll();
await sleep(40);

// 5) Too-large delays overflow: > 2^31-1 ms (~24.8 days) becomes 1 ms
const big = setTimeout(() => console.log('huge delay fired immediately!'), 2 ** 31);
await sleep(20);
clearTimeout(big);

// 6) Promise-based sleep
const s = performance.now();
await sleep(15);
console.log(`awaited sleep(15): >= 15 ms: ${performance.now() - s >= 14.5}`);
```

Output (the three `(node:…)` lines are the warning Node writes to stderr):

```
cleared a timer before it fired
tick 1
tick 2
tick 3
asked 10 ms, got >= 40 ms: true
recursive setTimeout ran 3 times
(node:51319) TimeoutOverflowWarning: 2147483648 does not fit into a 32-bit signed integer.
Timeout duration was set to 1.
(Use `node --trace-warnings ...` to show where the warning was created)
huge delay fired immediately!
awaited sleep(15): >= 15 ms: true
```

`setInterval` vs recursive `setTimeout`

	`setInterval(fn, 100)`	recursive `setTimeout(fn, 100)`
Spacing	Start-to-start; if `fn` takes 80 ms only ~20 ms idle	End-to-start; always ≥ 100 ms of idle between runs
Slow or async work	Runs can bunch up or overlap (an async `fn` isn't awaited)	Next run scheduled only after this one finishes
Change the delay each time (backoff)	Awkward	Easy

For polling a server (asking it again and again whether something changed), prefer recursive `setTimeout` (after the previous request completes).

Common mistakes

- Passing the *result* of a call: `setTimeout(save(), 1000)` runs `save` now. Pass a function: `setTimeout(save, 1000)` or `() => save()`.
- Forgetting to clear intervals (e.g. when a component unmounts — is removed from the page in a UI framework such as React) → memory leaks (memory that can never be freed) and duplicate work.
- Using timers to "wait until something is ready" — listen for the actual event or await the actual promise instead.
- A method passed as callback loses `this`: `setTimeout(obj.method, 10)` — use an arrow or `bind` (Chapter 9).

TL;DR

Timer delays are minimums; the callback runs as a task when the thread is free. Clear what you schedule. Prefer recursive `setTimeout` for polling; `node:timers/promises` gives an awaitable `setTimeout`.

12.6 Callbacks and callback hell

What it is

A **callback** is a function you pass to another function, to be called later with the result. It's the oldest async style in JavaScript (event handlers, `setTimeout`, Node's original APIs). Node standardized **error-first callbacks**: `callback(err, result)` — `err` is `null` on success.

`code/05_callbacks.mjs` — key parts:

```
// 1) Error-first callbacks (the Node convention): cb(err, result)
readFile(new URL('./missing.txt', import.meta.url), 'utf8', (err, text) => {
  if (err) { console.log('error-first:', err.code); return; }
  console.log(text);
});

// 2) Callback hell: each step needs the previous result
db.getUser(1, (err, user) => {
  if (err) return console.log(err.message);
  db.getOrders(user, (err, orders) => {
    if (err) return console.log(err.message);
    db.getItems(orders[0], (err, items) => {
      if (err) return console.log(err.message);
      console.log('pyramid result:', user.name, orders[0].id, items);
    });
  });
});

// 3) Inversion of control: a buggy library calls your callback twice
function buggyCharge(amount, cb) { cb(null, 'ok'); cb(null, 'ok'); }
let charges = 0;
buggyCharge(10, () => { charges += 1; });
console.log('callback-style charges:', charges);

// A promise can settle only once, so the second call is ignored
const charged = new Promise((resolve) => buggyCharge(10, (_e, r) => resolve(r)));
let promiseCharges = 0;
await charged.then(() => { promiseCharges += 1; });
console.log('promise-style charges :', promiseCharges);

// 4) Zalgo: a callback that is sometimes sync, sometimes async
function getValue(key, cb) {
  if (cache.has(key)) cb(cache.get(key));          // sync!
  else setTimeout(() => cb(42), 0);                // async
}
// ...
```

Output:

```
callback-style charges: 2
promise-style charges : 1
error-first: ENOENT
pyramid result: Ada 7 [ 'book', 'pen' ]
zalgo order: cached cb | after call (a) | after call (b) | fetched cb
```

The problems callbacks have

Problem	What goes wrong	How promises fix it
Callback hell ("pyramid of doom")	Each dependent step nests deeper; error checks repeat at every level	Flat `.then` chains or linear `await` code
Error handling	`try/catch` can't catch errors thrown in a later callback; every level must check `err`	One `.catch` / one `try` catches errors from any step
Inversion of control	You trust someone else's code to call you once, not zero or twice	A promise settles exactly once
"Releasing Zalgo"	A callback that's sometimes sync and sometimes async makes ordering unpredictable (see the output: the cached callback ran *before* "after call")	`then` callbacks are always async
Composition	Running things in parallel and waiting for all is hand-written counting code	`Promise.all` and friends

Callbacks remain perfect for **repeated events** (clicks, stream data — a stream delivers data in chunks over time) — a promise can only deliver one value. Named functions and early `return`s flatten a pyramid

a bit, but the real fix is promises.

TL;DR

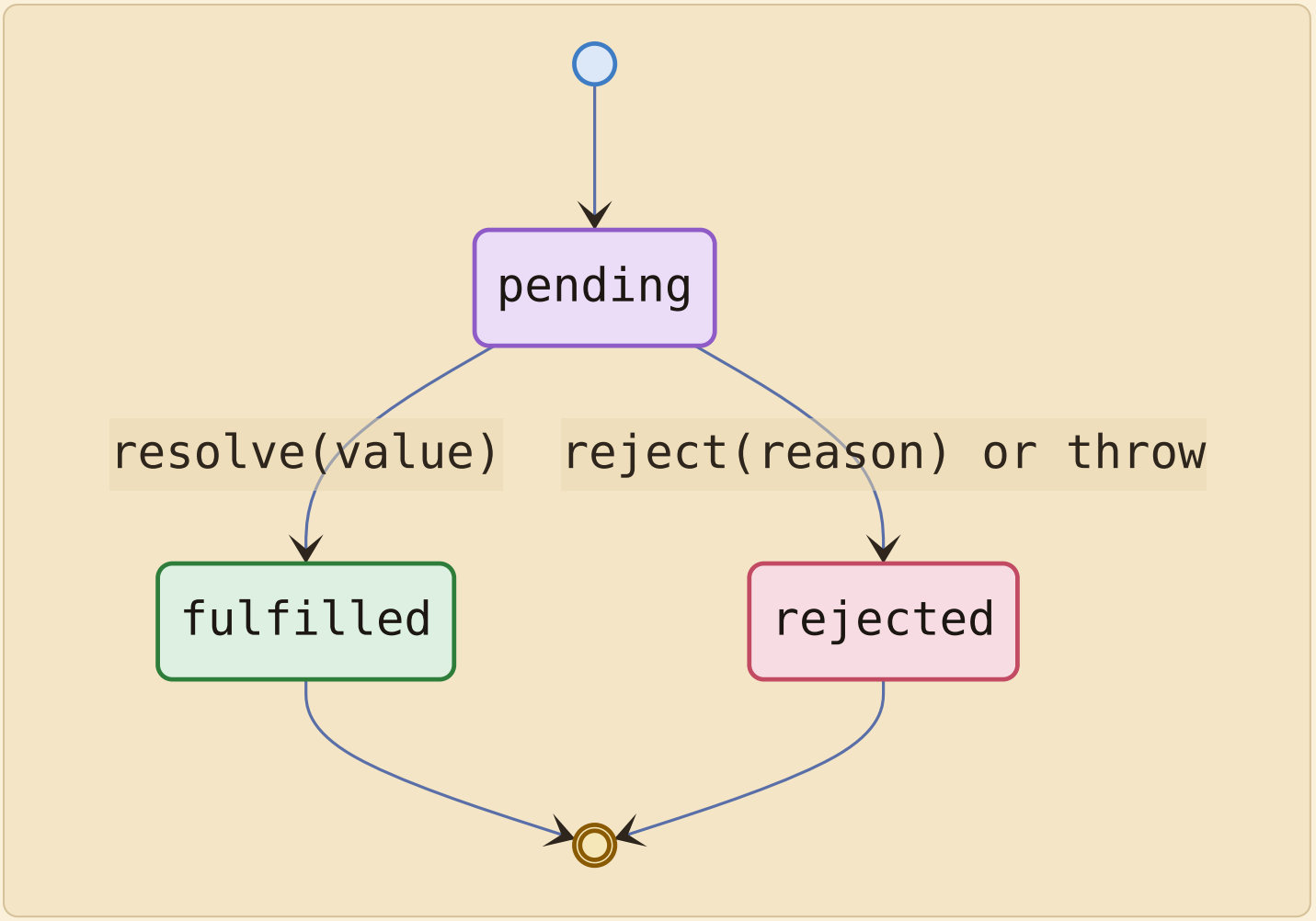
Callbacks: pass a function to be called later; Node uses `cb(err, result)`. They nest badly, can't use `try/catch`, and depend on trusting the caller — promises solve all of these. Keep callbacks for multi-fire events.

12.7 Promises

What it is

A **promise** is an object representing a value that will be available later — or an error explaining why not. Analogy: a pager a restaurant hands you. You can go sit down (your code continues); when the pager buzzes you either get your food (fulfilled) or an apology (rejected). It buzzes exactly once.

States:



- **pending** — not finished yet.
- **fulfilled** — finished with a value.
- **rejected** — failed with a reason (normally an `Error`).
- **settled** — fulfilled or rejected; a settled promise never changes again.
- **resolved** — "locked in": either settled, or following another promise. (A promise resolved with a pending promise is still pending.)

How it works

1. `new Promise((resolve, reject) => { … })` — the **executor** runs immediately and synchronously; it starts the work and later calls `resolve` or `reject`. If it throws, the promise is rejected.

2. `promise.then(onFulfilled, onRejected)` registers reactions and returns a **new** promise.
3. When the promise settles, reactions are queued as microtasks.
4. The new promise from `then` settles with whatever the reaction **returns** (a value → fulfilled; a promise → adopt its outcome) or **throws** (→ rejected).
5. A rejection skips `then(onFulfilled)` reactions until it meets a `catch` (or a `then` with `onRejected`). After `catch` returns normally, the chain is fulfilled again.
6. `finally(fn)` runs `fn` with no argument either way and passes the original outcome through (unless `fn` throws or returns a rejected promise).

`code/06_promises.mjs`:

```
const later = (value, ms, fail = false) =>
  new Promise((resolve, reject) => setTimeout(() => (fail ? reject(new Error(value)) : resolve(value)), ms));

const p = later('done', 10);
console.log('right away :', p);      // Node shows the state
await p;
console.log('after await:', p);

// Chaining: each then() returns a NEW promise
const result = await Promise.resolve(2)
  .then((x) => x * 10)               // return a value -> next then gets it
  .then((x) => later(x + 1, 5))      // return a promise -> chain waits for it
  .then((x) => { throw new Error(`boom at ${x}`); })
  .then(() => console.log('skipped because the chain is rejected'))
  .catch((err) => { console.log('caught:', err.message); return 'recovered'; })
  .finally(() => console.log('finally: runs either way, sees no value'));
console.log('chain result:', result);

// Mistake: forgetting to return inside then() breaks the chain
await Promise.resolve(1)
  .then((x) => { later(x + 1, 5); })  // no return!
  .then((v) => console.log('forgot return ->', v));

// resolve/reject only count once
const once = new Promise((resolve, reject) => { resolve('first'); resolve('second'); reject(new Error('ignored')); });
console.log('settles once:', await once);

// Promise.resolve adopts thenables; a promise is passed through unchanged
const thenable = { then(onOk) { onOk('from a thenable'); } };
console.log('thenable   :', await Promise.resolve(thenable));
console.log('same obj   :', Promise.resolve(p) === p);

// Promisify a callback API
const readFileP = promisify(readFile);
const text = await readFileP(new URL(import.meta.url), 'utf8');
console.log('promisify  :', text.split('\n')[0]);

// Promise.withResolvers (ES2024): resolve from outside the executor
const { promise, resolve } = Promise.withResolvers();
setTimeout(() => resolve('resolved from outside'), 5);
console.log('withResolvers:', await promise);

// Promise.try (ES2025): run a function that may throw synchronously OR return a promise
const risky = (x) => { if (x < 0) throw new RangeError('negative'); return later(x * 2, 1); };
for (const x of [3, -1]) {
  await Promise.try(risky, x).then((v) => console.log('Promise.try ok  :', v), (e) => console.log('Promise.try fail:', e.message));
}
```

Output:

```
right away : Promise { <pending> }
after await: Promise { 'done' }
caught: boom at 21
finally: runs either way, sees no value
chain result: recovered
forgot return -> undefined
settles once: first
thenable   : from a thenable
same obj   : true
promisify  : import { promisify } from 'node:util';
withResolvers: resolved from outside
Promise.try ok  : 6
Promise.try fail: negative
```

Key API

API	Meaning
`new Promise(executor)`	Wrap a callback API. Rarely needed otherwise (**explicit-construction anti-pattern** — an anti-pattern is a common habit that causes problems: wrapping something that already returns a promise).
`Promise.resolve(v)` / `Promise.reject(e)`	Already-settled promise (`resolve` adopts thenables, returns `v` itself if it's a native promise)
`.then(ok, fail)` / `.catch(fail)` / `.finally(fn)`	Reactions; each returns a new promise
`Promise.withResolvers()` (ES2024, the 2024 edition of the JS standard)	`{ promise, resolve, reject }` — handy when resolve is triggered by an event elsewhere
`Promise.try(fn, ...args)` (ES2025)	Call `fn`; a sync throw becomes a rejection, a returned value/promise is adopted
`util.promisify(fn)` (Node)	Turn an error-first callback function into a promise-returning one. Most Node APIs also have promise versions: `node:fs/promises`, `node:timers/promises`.

A **thenable** is any object with a `then` method; promises from different libraries interoperate through this duck-typed protocol (judged by which methods an object has, not by its class) (Chapter 9 — duck typing).

Common mistakes

- **Forgetting** `return` in a `then` callback (see output: `undefined`, and the inner promise's errors become unhandled).
- **Nesting** `then` inside `then` instead of returning — recreates callback hell.
- `.then(ok, fail)` **vs** `.then(ok).catch(fail)`**:** the first form does **not** catch errors thrown inside `ok`; the second does.
- **No** `catch` **at the end** → an unhandled rejection (crashes Node, logs in browsers) — §11.5.
- **Thinking promises are cancellable or lazy** (lazy = not starting until someone asks for the result). A promise represents work that has *already started*; use `AbortController` (§12.11) to cancel the work.

Self-check

1. What does `.then(() => 5)` return? → A new promise fulfilled with 5.
2. `Promise.resolve().then(() => { throw new Error('x') }).then(() => 1).catch(e => 2)` → fulfils with? → `2`.
3. What does `finally`'s callback receive? → Nothing; it can't see the value or reason.
4. Is the executor synchronous? → Yes.

TL;DR

A promise is a one-time, eventually-settled value: pending → fulfilled/rejected. `then` returns a new promise whose outcome is the callback's return value or thrown error. Always return inside `then`, end

chains with `catch`.

12.8 Promise combinators: `all`, `allSettled`, `race`, `any`

What it is

Static methods (called on `Promise` itself, not on a promise instance) that take an iterable (anything `for…of` can loop over, such as an array) of promises (or plain values) and return one promise.

Method	Fulfils when	Rejects when	Result	Added
`Promise.all`	**all** fulfil	**first** rejection (fail-fast)	array of values in **input order**	ES2015
`Promise.allSettled`	all settle (never rejects)	never	array of `{status, value}` / `{status, reason}`	ES2020
`Promise.race`	first to **settle** is fulfilled	first to settle is rejected	that value/reason	ES2015
`Promise.any`	first to **fulfil**	**all** reject	value, or `AggregateError` with `.errors`	ES2021

`code/07_combinators.mjs`:

```
console.log('all       :', await Promise.all([later('slow', 30), later('fast', 10), 42]), at());
t0 = performance.now();
await Promise.all([later('ok', 30), later('db down', 10, true)])
  .catch((e) => console.log('all fails :', e.message, 'after', at()));

const settled = await Promise.allSettled([later('ok', 5), later('bad', 5, true)]);
console.log('allSettled:', settled.map((r) => (r.status === 'fulfilled' ? `ok:${r.value}` :
`err:${r.reason.message}`)));

console.log('race      :', await Promise.race([later('A', 20), later('B', 10)]));
await Promise.race([later('A', 20), later('B failed', 10, true)]).catch((e) => console.log('race err  :',
e.message));

console.log('any       :', await Promise.any([later('mirror1 down', 5, true), later('mirror2', 15)]));
await Promise.any([later('x', 5, true), later('y', 5, true)])
  .catch((e) => console.log('any fails :', e.name, e.errors.map((x) => x.message)));

console.log('empty     :', await Promise.all([]), (await Promise.allSettled([])).length);
await Promise.any([]).catch((e) => console.log('any([])   :', e.name));
console.log('race([]) stays pending forever - never await it');

const withTimeout = (promise, ms) =>
  Promise.race([promise, new Promise((_, rej) => setTimeout(() => rej(new Error(`timed out after ${ms} ms`)),
ms))]);
await withTimeout(later('slow api', 100), 20).catch((e) => console.log('timeout   :', e.message));
```

Output:

```
all       : [ 'slow', 'fast', 42 ] 30 ms
all fails : db down after 10 ms
allSettled: [ 'ok:ok', 'err:bad' ]
race      : B
race err  : B failed
any       : mirror2
any fails : AggregateError [ 'x', 'y' ]
empty     : [] 0
any([])   : AggregateError
race([]) stays pending forever - never await it
timeout   : timed out after 20 ms
```

Notes:

- `all` took 30 ms (the slowest), not 40 — the promises ran concurrently (their waiting overlapped in time). The results are in input order even though `fast` finished first. Plain values like `42` are allowed.
- On failure, `all` rejects after 10 ms, but the other operation **keeps running** — combinators don't cancel anything. Pair them with an `AbortSignal` (§12.11) if you need that.
- `withTimeout` via `race` is a classic interview answer; in real code prefer `AbortSignal.timeout(ms)`, which also stops the underlying work (e.g. the HTTP request).

Choosing

Situation	Use
Load user, settings and feed; the page needs all three	`all`
Send 20 notifications; report which failed	`allSettled`
Fastest of several mirrors / CDNs (content delivery networks — servers worldwide holding copies of files)	`any`
A result or a timeout, whichever comes first	`race` (or `AbortSignal.timeout`)

In the interview

30-second answer: "`all` waits for everything and fails fast on the first rejection; `allSettled` waits for everything and never rejects; `race` settles with the first promise to settle, success or failure; `any` takes the first success and only rejects — with an `AggregateError` — if all fail. None of them cancel the losers."

- *Q: Implement `Promise.all`.* A: Return a new promise; keep a results array and a counter; for each input, `Promise.resolve(item).then(v => { results[i] = v; if (++done === n) resolve(results) }, reject)`; resolve immediately if the input is empty. (Chapter 17 has runnable polyfills — a polyfill is code that implements a built-in feature yourself, for environments that lack it.)
- *Q: `Promise.all` with one rejection — do the others stop?* A: No; their results are simply ignored.

TL;DR

`all` (all or first failure), `allSettled` (everything, never rejects), `race` (first settled), `any` (first success, else `AggregateError`). They don't cancel.

12.9 `async` / `await`

What it is

Syntax (ES2017) that lets you write promise-based code as if it were synchronous.

- An `async function` always returns a promise: a `return v` fulfils it with `v`; a `throw` rejects it.
- `await expr` pauses the async function until the promise settles, then gives back its value — or **throws** its rejection reason. Pausing doesn't block the thread: the function's remaining code is resumed later as a microtask, and meanwhile the caller continues.
- `await` is allowed only inside async functions and at the **top level of modules** (Chapter 10).

Under the hood an async function is like a generator (a function that can pause at `yield` and resume later — Chapter 8) that yields promises, driven by a runner that resumes it when each promise settles.

`code/08_async_await.mjs`:

```
// 1) An async function ALWAYS returns a promise
async function answer() { return 42; }
console.log('returns   :', answer() instanceof Promise, await answer());
async function fails() { throw new Error('nope'); }
console.log('throw ->  :', await fails().catch((e) => `rejected (${e.message})`));

// 2) Sequential vs parallel
let t = performance.now();
const a = await later('a', 30);
const b = await later('b', 30);
console.log('sequential:', a + b, round(performance.now() - t), 'ms');

t = performance.now();
const [c, d] = await Promise.all([later('c', 30), later('d', 30)]);
console.log('parallel  :', c + d, round(performance.now() - t), 'ms');

// 3) try/catch/finally with await
async function load(id) {
  try {
    return await later(`user ${id}`, 5, id < 0);   // "return await" keeps the error inside this try
  } catch (err) {
    return `fallback for ${err.message}`;
  } finally {
    console.log('  cleanup for', id);
  }
}
console.log('load(1)   :', await load(1));
console.log('load(-1)  :', await load(-1));

// 4) await in loops: for...of is sequential; forEach does NOT wait
const ids = [1, 2, 3];
const seen = [];
ids.forEach(async (id) => { await later(id, 12 - id * 3); seen.push(id); });
console.log('forEach done? seen =', seen, '(nothing yet!)');
await later(null, 20);
console.log('...later   seen =', seen, '(finish order, not input order)');
const inOrder = [];
for (const id of ids) inOrder.push(await later(id * 10, 2));
console.log('for...of  :', inOrder);
console.log('map+all   :', await Promise.all(ids.map((id) => later(id * 100, 5))));

// 5) Async arrow, async method, await of a non-promise
const double = async (x) => x * 2;
const obj = { async get() { return 'method'; } };
console.log('misc      :', await double(21), await obj.get(), await 'plain value');
```

Output:

```
returns   : true 42
throw ->  : rejected (nope)
sequential: ab 60 ms
parallel  : cd 30 ms
  cleanup for 1
load(1)   : user 1
  cleanup for -1
load(-1)  : fallback for user -1
forEach done? seen = [] (nothing yet!)
...later   seen = [ 3, 2, 1 ] (finish order, not input order)
for...of  : [ 10, 20, 30 ]
map+all   : [ 100, 200, 300 ]
misc      : 42 method plain value
```

Sequential vs parallel — the most common performance bug

Two independent `await`s in a row take the **sum** of their times (60 ms); starting both first and awaiting `Promise.all` takes the **max** (30 ms). Rule: `await` one after another only when step 2 needs step 1's result.

```
// Also parallel: start first, await later
const userP = getUser(id), postsP = getPosts(id);   // both requests in flight
const user = await userP, posts = await postsP;    // careful: if postsP rejects before
                                                    // userP settles, it's briefly "unhandled"
```

`Promise.all` is safer than the start-then-await form because it attaches handlers to every promise immediately.

Loops

Loop	Behaviour
`for…of` / `for` / `while` with `await` inside	One at a time, in order
`array.forEach(async …)`	Fires all, **doesn't wait**, ignores results and errors — avoid
`await Promise.all(array.map(async …))`	All at once, results in order
A concurrency-limited pool (§12.13)	At most N at a time

Error handling

- Wrap `await` in `try`/`catch`; the rejection reason arrives as the caught value.
- `return await p` inside `try` — without `await`, the function hands the promise back *before* it rejects, so your `catch` never sees the rejection and `finally` runs while the work is still in progress. Outside `try`, `return p` and `return await p` behave the same.
- Handle errors at the level that can do something about them (show a message, retry, fall back); let others propagate. Custom error classes and `cause` chaining are in Chapter 11.
- A promise you create but never await can reject with nobody listening → `unhandledrejection`.

Trade-offs: `async/await` vs `.then` chains

	`async` / `await`	`.then` chains
Readability	Reads like sync code; normal `if`, loops, `try`	Fine for short linear pipelines
Errors	`try`/`catch`	`.catch`
Parallelism	Must remember `Promise.all`	Same
Where it works	Async functions and module top level	Anywhere

They're the same machinery; mix freely (e.g. `await fetch(url).then(r => r.json())`).

In the interview

30-second answer: "`async` functions always return promises; `await` suspends just that function until the promise settles, resuming it as a microtask, so the thread stays free. Rejections become exceptions, so `try/catch` works. The classic mistake is awaiting independent calls one by one — use `Promise.all` — and using `forEach` with an async callback, which doesn't wait."

- *Q: What does `async function f() {}; f()` return?* A: A promise fulfilled with `undefined`.
- *Q: Does code after an `await` run synchronously if the promise is already resolved?* A: No, it's always deferred to a microtask.
- *Q: Why `return await` inside `try`?* A: So the rejection is caught by that `try`'s `catch`.

Self-check

1. Three independent 100 ms requests awaited one after another take…? → ~300 ms; with `Promise.all` ~100 ms.
2. Can you use `await` in a regular function? → No (SyntaxError); only in async functions or module top level.

3. Why did `seen` end up `[3, 2, 1]`? → `forEach` started all three at once; they finished in order of their delays.

TL;DR

`async` → returns a promise; `await` → unwraps it or throws. Independent work → `Promise.all`. Sequential loop → `for…of`; never `forEach(async)`. `try { return await … } catch`.

12.10 Error handling across async code — a checklist

Chapter 11 introduced the rules (§11.5); now that you know the event loop, here is why they hold and how to apply them.

Situation	Where the error goes	Handle with
`throw` inside an `async` function	Rejects its returned promise	`try/catch` around `await f()`, or `f().catch()`
`throw` inside a `then` callback	Rejects the promise `then` returned	A later `.catch`
`throw` inside a `setTimeout` / event callback	Uncaught error in *that task* — no enclosing `try` exists any more	`try/catch` **inside** the callback; global `error` handler for logging
Rejection nobody handles	`unhandledrejection` event (browser) / crash (Node ≥ 15)	Always `await` / `return` / `.catch`
One of many parallel operations fails	`Promise.all` rejects with the first; others ignored	`allSettled` if you need every outcome
Error in an error-first callback	Passed as the first argument	`if (err) …` first thing

Rules of thumb:

1. Every promise has an owner who awaits it, returns it, or attaches `.catch`.
2. Catch where you can **act** (retry, fallback UI, HTTP 500 response); otherwise let it propagate, adding context with `new Error(msg, { cause })`.
3. Don't mix styles in one function: a callback API inside an async function should be promisified first, or its errors escape your `try`.
4. Global handlers (`window.onunhandledrejection`, `process.on('unhandledRejection')`) are for logging and reporting, not recovery.

TL;DR

try/catch works across `await` because rejections are re-thrown on resumption. Errors in plain callbacks belong to their own task. Leave no promise floating.

12.11 Cancellation: `AbortController` and `AbortSignal`

What it is

Promises can't be cancelled, but the *work* behind them can. The standard mechanism (from the DOM spec — the standard for the browser's page APIs — also in Node) is:

- **`AbortController`** — the object held by whoever decides to cancel; `controller.abort(reason?)`.
- **`AbortSignal`** — `controller.signal`, handed to the operation; it has `.aborted`, `.reason`, an `abort` event and `throwIfAborted()`.

Analogy: the controller is the "cancel order" button at the till; the signal is the ticket the kitchen watches.

API	Purpose	Support
`new AbortController()` / `.abort(reason)`	Manual cancel	All modern browsers, Node ≥ 15
`AbortSignal.timeout(ms)`	Signal that aborts with a `TimeoutError` `DOMException` (browser-API error type) after `ms`	Baseline (works in all major browsers) since 2022 (Node ≥ 17.3)
`AbortSignal.any([s1, s2])`	Aborts when any input aborts	Baseline since 2024 (Node ≥ 20.3)
`AbortSignal.abort(reason)`	Already-aborted signal	Modern browsers, Node
`{ signal }` option	Accepted by `fetch`, `addEventListener`, Node `timers/promises`, `fs.readFile`, streams, `events.on`…	—

`code/09_abort.mjs`:

```
// 1) Making your own function cancellable with an AbortSignal
function delay(ms, { signal } = {}) {
  return new Promise((resolve, reject) => {
    signal?.throwIfAborted();                       // already aborted? fail fast
    const id = setTimeout(() => { signal?.removeEventListener('abort', onAbort); resolve(`waited ${ms} ms`); },
ms);
    function onAbort() { clearTimeout(id); reject(signal.reason); }
    signal?.addEventListener('abort', onAbort, { once: true });
  });
}

const controller = new AbortController();
setTimeout(() => controller.abort(), 20);
try {
  await delay(1000, { signal: controller.signal });
} catch (err) {
  console.log('manual abort  :', err.name, '-', err.message);
}
console.log('signal state  :', controller.signal.aborted);

// 2) Custom reason
const c2 = new AbortController();
c2.abort(new Error('user left the page'));
await delay(10, { signal: c2.signal }).catch((e) => console.log('custom reason :', e.message));

// 3) AbortSignal.timeout(ms) - aborts with a TimeoutError
await delay(1000, { signal: AbortSignal.timeout(15) }).catch((e) => console.log('timeout       :', e.name));

// 4) AbortSignal.any - abort if ANY of several signals aborts
const userCancel = new AbortController();
const combined = AbortSignal.any([userCancel.signal, AbortSignal.timeout(5000)]);
setTimeout(() => userCancel.abort(), 10);
await delay(1000, { signal: combined }).catch((e) => console.log('any()         :', e.name));

// 5) Built-in APIs accept signals too
await sleep(1000, null, { signal: AbortSignal.timeout(10) }).catch((e) => console.log('node sleep    :', e.name));

const target = new EventTarget();
const listeners = new AbortController();
target.addEventListener('ping', () => console.log('ping heard'), { signal: listeners.signal });
target.dispatchEvent(new Event('ping'));
listeners.abort();                                  // removes the listener
target.dispatchEvent(new Event('ping'));
console.log('listener removed by abort()');
```

Output:

```
manual abort  : AbortError - This operation was aborted
signal state  : true
custom reason : user left the page
timeout       : TimeoutError
any()         : AbortError
node sleep    : AbortError
ping heard
listener removed by abort()
```

How to write abortable functions

1. Accept `{ signal }` in an options object.
2. Check `signal?.throwIfAborted()` at the start (and between steps of long work).
3. Listen for `abort` once; on abort, undo/stop the work and reject with `signal.reason`.
4. Remove the listener when the work finishes normally (avoid leaks on long-lived signals).
5. Pass the signal down to every API you call (`fetch(url, { signal })`).

Callers distinguish a deliberate cancel from a real failure by `err.name === 'AbortError'` (or `'TimeoutError'`) and usually ignore it silently. Fetch with abort and timeouts against a real server is in Chapter 14.

Real-world examples

- Search-as-you-type: abort the previous request when a new keystroke arrives.
- React effects (code React runs after a component appears on screen): abort the fetch in the effect's cleanup function when the component unmounts.
- Removing a group of event listeners at once: one controller, `{ signal }` on each `addEventListener`, one `abort()`.

TL;DR

`AbortController` cancels, `AbortSignal` carries the cancel to the work. `AbortSignal.timeout` and `AbortSignal.any` cover timeouts and combined cancel. Rejections have `name` `AbortError`/`TimeoutError`.

12.12 Async iteration: `for await…of` and async generators

What it is

Chapter 8 showed the (sync) **iterable protocol**: `[Symbol.iterator]()` returns an iterator whose `next()` returns `{ value, done }`. The **async iterable protocol** (ES2018) is the same, except the method is `[Symbol.asyncIterator]()` and `next()` returns a **promise** of `{ value, done }`. You consume it with **`for await (const x of source)`**, and you produce one easily with an **async generator** (`async function*`), which can both `await` and `yield`.

Use it for data that arrives in pieces over time: paginated APIs (APIs that return a big result one page at a time), file and network streams (Node streams and web `ReadableStream`s are async iterable), event streams.

`code/10_async_iteration.mjs`:

```
// 1) An async generator: yields values that arrive over time
async function* countdown(from, ms) {
  for (let i = from; i > 0; i--) {
    await new Promise((r) => setTimeout(r, ms));
    yield i;
  }
}
for await (const n of countdown(3, 5)) console.log('countdown', n);

// 2) Paginated API -> one flat async stream
const pages = { 1: { items: ['a', 'b'], next: 2 }, 2: { items: ['c'], next: 3 }, 3: { items: ['d', 'e'], next:
null } };
const fetchPage = async (n) => { await null; return pages[n]; };
async function* allItems() {
  let page = 1;
  while (page) {
    const { items, next } = await fetchPage(page);
    yield* items;                  // delegate to the array's iterator
    page = next;
  }
}
const collected = [];
for await (const item of allItems()) {
  collected.push(item);
  if (collected.length === 4) break;   // break calls return() -> generator stops fetching
}
console.log('first 4 items :', collected);

// 3) Array.fromAsync (ES2026) collects an async iterable into an array
console.log('fromAsync     :', await Array.fromAsync(allItems()));

// 4) for await also works on an array of promises (awaits each in order)
const ps = [Promise.resolve('p1'), new Promise((r) => setTimeout(() => r('p2'), 5))];
for await (const v of ps) console.log('for await over promises:', v);

// 5) The protocol by hand
const ticker = {
  [Symbol.asyncIterator]() {
    let i = 0;
    return { next: async () => (i < 2 ? { value: `tick${i++}`, done: false } : { value: undefined, done: true })
};
  },
};
for await (const tk of ticker) console.log('manual iterator:', tk);

// 6) Events as an async stream (Node events.on)
const bus = new EventEmitter();
const ac = new AbortController();
setTimeout(() => { bus.emit('msg', 'hello'); bus.emit('msg', 'world'); ac.abort(); }, 5);
try {
  for await (const [text] of on(bus, 'msg', { signal: ac.signal })) console.log('event:', text);
} catch (e) {
  console.log('stream ended  :', e.name);
}
```

Output:

```
countdown 3
countdown 2
countdown 1
first 4 items : [ 'a', 'b', 'c', 'd' ]
fromAsync     : [ 'a', 'b', 'c', 'd', 'e' ]
for await over promises: p1
for await over promises: p2
manual iterator: tick0
manual iterator: tick1
event: hello
event: world
stream ended  : AbortError
```

Notes

- `for await` is allowed wherever `await` is (async functions, module top level).
- `break`, `return` or a thrown error inside the loop calls the iterator's `return()`, so an async generator's `finally` blocks run — useful for closing connections.
- Items are processed **one at a time**. For an array of independent promises, `Promise.all` is usually better; `for await` over such an array also can't report a rejection until the loop reaches it.
- `Array.fromAsync` is part of ES2026 and supported in Node 22+ and current browsers.

> **TL;DR**
>
> Async iterables deliver `{value, done}` promises; `for await…of` consumes them; `async function*` produces them. Great for pagination and streams; `break` cleans up.

12.13 Real-world async patterns

`code/11_patterns.mjs` implements three helpers that appear in almost every production codebase (code that runs the real, live service).

```
// 1) Retry with exponential backoff
async function retry(fn, { tries = 4, base = 5 } = {}) {
  for (let attempt = 1; ; attempt++) {
    try {
      return await fn(attempt);
    } catch (err) {
      if (attempt >= tries) throw new Error(`gave up after ${attempt} tries`, { cause: err });
      const wait = base * 2 ** (attempt - 1);   // 5, 10, 20 ... (add random jitter in production)
      console.log(`  attempt ${attempt} failed (${err.message}); retrying in ${wait} ms`);
      await sleep(wait);
    }
  }
}

// 2) Run many tasks but at most `limit` at once
async function mapLimit(items, limit, fn) {
  const results = new Array(items.length);
  let next = 0, running = 0, peak = 0;
  async function worker() {
    while (next < items.length) {
      const i = next++;                         // safe: no other code runs between read and write
      running++; peak = Math.max(peak, running);
      results[i] = await fn(items[i]);
      running--;
    }
  }
  await Promise.all(Array.from({ length: limit }, worker));
  return { results, peak };
}

// 3) Race condition: an older, slower response arrives last and overwrites newer data
let latest = 0;
await Promise.all(['ja', 'jav'].map(async (q) => {
  const ticket = ++latest;
  const r = await search(q);
  if (ticket === latest) shown = r;               // ignore answers to old requests
}));
```

Output:

```
  attempt 1 failed (503); retrying in 5 ms
  attempt 2 failed (503); retrying in 10 ms
retry result: payload
mapLimit    : [
   1,  4,  9, 16,
  25, 36, 49
] peak concurrency = 3
naive shows : results for "ja" <- stale!
guarded     : results for "jav"
```

Pattern	Problem it solves	Production notes
Retry + exponential backoff (double the wait after each failure)	Transient (temporary) failures (HTTP 503 "service unavailable", network blips)	Add random **jitter** so many clients don't retry in sync; retry only idempotent operations (safe to repeat — doing them twice equals doing them once; see System Design Fundamentals); cap attempts and total time; respect `Retry-After` headers
Concurrency limit	10,000 requests at once would overload the server or hit rate limits (caps on requests per second) / file-descriptor limits (caps on open files and sockets)	Libraries: `p-limit`, `p-map`; the worker-pool shape above is enough for scripts
Last-request-wins	Responses arrive out of order (search-as-you-type), stale data overwrites fresh — a **race condition** (the result depends on which operation happens to finish first)	Better still: abort the old request (§12.11); React Query and similar libraries do this for you
Timeout	A request that never answers	`AbortSignal.timeout(ms)`
Debounce / throttle (wait until events stop / run at most once per interval)	Too many events (keystrokes, scroll)	Implemented in Chapter 17

Note `mapLimit`'s `next++`: because of run-to-completion, no other worker can run between reading and incrementing `next`, so no lock is needed — a direct benefit of the single thread.

TL;DR

Retry with backoff + jitter for transient errors; cap concurrency with a worker pool; guard against stale responses with request IDs or abort; time out everything that talks to the network.

Chapter 12 TL;DR

- **One thread**, run-to-completion; the host does the waiting. Long sync work blocks timers, input and rendering.
- **Event loop:** one task → drain **all** microtasks → maybe render (rAF = `requestAnimationFrame` → style → layout → paint) → repeat. Promise reactions, code after `await`, `queueMicrotask` = microtasks; timers, I/O, events = tasks.
- **Puzzles:** sync first (including executors and code before the first `await`), then microtasks FIFO, then tasks. Returning a promise from `then` costs 2 extra ticks.
- **Node:** phases timers → poll → check (`setImmediate`) → close; `nextTick` before promises (except at ESM top level); `setImmediate` vs `setTimeout(0)` only ordered inside I/O callbacks.
- **Timers** are minimum delays; nested clamp 4 ms; overflow above $2^{31}-1$ ms; prefer recursive `setTimeout` for polling.
- **Callbacks** → callback hell, no `try/catch`, inversion of control, Zalgo. Keep them for repeated events.

- **Promises:** pending → fulfilled/rejected, settle once; `then` returns a new promise; always `return` and end with `catch`. `withResolvers` (ES2024), `Promise.try` (ES2025).
- **Combinators:** `all`, `allSettled`, `race`, `any` — none cancel.
- **async/await:** returns a promise; `await` throws rejections; use `Promise.all` for independent work; no `forEach(async)`; `return await` inside `try`.
- **Cancel** with `AbortController` / `AbortSignal.timeout` / `AbortSignal.any`.
- **Async iteration:** `for await…of`, `async function*`, `Array.fromAsync` (ES2026).
- **Patterns:** retry with backoff + jitter, concurrency limits, last-request-wins.

Next: the page itself — Chapter 13: DOM and Events.

Chapter 13 — The DOM and Events

A web page, as JavaScript sees it, is a tree of objects called the **DOM**. Every interactive feature on the web — a dropdown opening, a todo item ticked off, a form refusing an invalid email — is JavaScript finding nodes in that tree, changing them, and reacting to **events** such as clicks and key presses. This chapter teaches the DOM API (the set of objects and functions JavaScript uses to read and change the page) from the ground up, the event system in full (including the capture/bubble path — the down-then-up route an event travels through the page — that trips up many developers), forms, and the performance rules that keep pages smooth.

What you'll learn

1. What the DOM is: nodes, elements, the tree, and how HTML (the tag-based markup language web pages are written in) becomes a DOM.
2. Selecting elements and walking the tree.
3. Creating, inserting, moving and removing nodes; `textContent` vs `innerHTML`; templates and fragments.
4. Attributes vs properties; `classList`, `dataset` and styles.
5. Events: listeners, the event object, listener options, custom events.
6. Propagation: capture, target and bubble phases; stopping propagation.
7. Event delegation — one listener for many elements.
8. Default actions, `preventDefault`, passive listeners.
9. Forms: `submit`, `FormData`, the Constraint Validation API, `input` vs `change`.
10. Watching the DOM with `MutationObserver`.
11. Rendering performance: reflow, repaint and layout thrashing (measured in Chrome).
12. When scripts run: `DOMContentLoaded`, `load`, `defer`.

Before you start. You'll use closures (Chapter 4) and `this` in handlers (Chapter 9 §9.1). Event listeners run as tasks on the event loop (Chapter 12). The HTML and CSS (the language that styles a page) side — semantic elements (tags chosen for meaning, like `<nav>` or `<button>`), forms markup, selectors (patterns like `.title` or `#app` that pick out elements) — is in the HTML & CSS Handbook (`../../../html-css/`).

How the code runs. Node has no DOM, so the `.mjs` programs in `code/` start with `import '../../../../tools_dom.mjs';`, which creates a **jsdom** document (a JavaScript implementation of the DOM standard used by test tools) and exposes `document`, `window` etc. as globals (variables available everywhere without importing). jsdom implements the DOM and events faithfully but has **no layout engine** (the part of a browser that works out sizes and positions — so no sizes, no painting). For those parts, small `.html` pages were run in real headless Chrome (Chrome running with no visible window, controlled by a script) and screenshotted after a scripted interaction. All outputs are real.

13.1 The DOM tree

What it is

The **Document Object Model (DOM)** is the browser's live, in-memory representation of a page: a **tree** of **nodes**, with an API (defined by the WHATWG — the browser makers' group that maintains the web standards — DOM and HTML standards) for reading and changing it. Change the tree and the browser updates the screen.

Analogy: the HTML file is the architect's blueprint; the DOM is the actual building. JavaScript renovates the building, not the blueprint — "View Source" still shows the original HTML, while DevTools' Elements panel shows the current DOM.

Terms:

- **Node** — any item in the tree. Main types: **element** (`<li>`, nodeType 1), **text** (3), **comment** (8), **document** (9), **document fragment** (11).
- **Element** — a node created from a tag; has attributes and can have children.

- **Parent / child / sibling / ancestor / descendant** — family relations in the tree.
- `document` — the root object; `document.documentElement` is `<html>`, plus `document.head` and `document.body`.

How it works

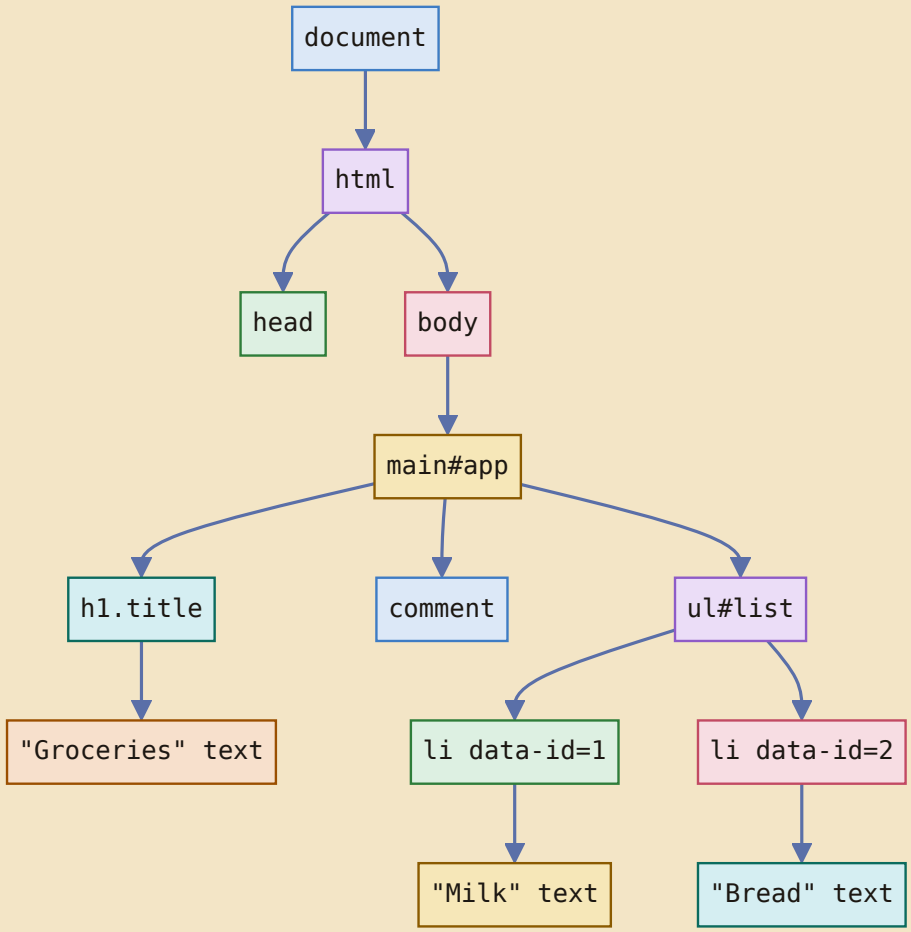
1. The browser's HTML **parser** (the component that reads text and understands its structure) reads the HTML text and builds the tree, fixing errors in standard ways (unclosed tags, missing `<tbody>`).
2. Whitespace between tags becomes **text nodes** — they exist even though they show nothing.
3. Scripts query and change the tree through the DOM API; the changes are live everywhere.

`code/01_dom_tree.mjs` — key parts:

```
document.body.innerHTML = `
<main id="app">
  <h1 class="title">Groceries</h1>
  <!-- a comment node -->
  <ul id="list">
    <li data-id="1">Milk</li>
    <li data-id="2" class="done">Bread</li>
  </ul>
</main>`;

function show(node, depth = 0) {           // recursive walk over childNodes
  // prints #text / #comment / <tag> ELEMENT with indentation
  for (const child of node.childNodes) show(child, depth + 1);
}
show(document.querySelector('#app'));

const main = document.getElementById('app');
console.log('\nchildNodes (all nodes):', main.childNodes.length, '| children (elements only):',
main.children.length);
// ...
const items = document.getElementById('list').children;   // live HTMLCollection
console.log('items before:', items.length);
document.getElementById('list').insertAdjacentHTML('beforeend', '<li data-id="3">Eggs</li>');
console.log('items after :', items.length);

const parsed = new DOMParser().parseFromString('<table><tr><td>cell</table><p>unclosed', 'text/html');
console.log('\nparser auto-inserted <tbody>:', parsed.querySelector('table > tbody > tr') !== null);
console.log('parsed body:', parsed.body.innerHTML);
```

Output:

```
<main> ELEMENT
  #text (whitespace)
  <h1> ELEMENT
    #text "Groceries"
  #text (whitespace)
  #comment "a comment node"
  #text (whitespace)
  <ul> ELEMENT
    #text (whitespace)
    <li> ELEMENT
      #text "Milk"
    #text (whitespace)
    <li> ELEMENT
      #text "Bread"
    #text (whitespace)
  #text (whitespace)

childNodes (all nodes): 7 | children (elements only): 2
document.documentElement: HTML | head: HEAD | body: BODY
nodeType of document: 9 === Node.DOCUMENT_NODE: true
items before: 2
items after : 3

parser auto-inserted <tbody>: true
parsed body: <table><tbody><tr><td>cell</td></tr></tbody></table><p>unclosed</p>
```



The tree for the snippet above (whitespace text nodes omitted).

Common mistakes

- Using `childNodes` / `firstChild` / `nextSibling` and getting whitespace text nodes. Prefer the **element-only** versions: `children`, `firstElementChild`, `nextElementSibling`.
- Assuming the DOM equals the HTML source — the parser inserts, moves and closes elements.

- Running a script in `<head>` that queries `document.body` elements before they're parsed (§13.12).

TL;DR

The DOM is a live tree of nodes (elements, text, comments) built by the parser from HTML. Use element-only navigation properties to skip whitespace text nodes.

13.2 Selecting and traversing

What it is

Method	Returns	Notes
`document.getElementById(id)`	Element or `null`	Fastest; id without `#`
`el.querySelector(css)`	First match or `null`	Any CSS selector; searches descendants of `el`
`el.querySelectorAll(css)`	**Static** `NodeList` (snapshot)	Has `forEach`; spread to an array for `map` / `filter`
`el.getElementsByClassName(c)` / `getElementsByTagName(t)`	**Live** `HTMLCollection`	Updates automatically; no `forEach`
`document.forms.name.elements.field`	Form controls by `name`	See §13.9
`el.closest(css)`	Nearest ancestor **or itself** matching, or `null`	Key to event delegation
`el.matches(css)`	`true` / `false`	"Does this element match?"
`parent.contains(node)`	`true` / `false`	Includes `parent` itself

Traversal properties: `parentElement`, `children`, `firstElementChild`, `lastElementChild`, `previousElementSibling`, `nextElementSibling` (and the node-level `parentNode`, `childNodes`, `firstChild`, `nextSibling`…).

`code/02_select_traverse.mjs` — key parts:

```
console.log('querySelector       :', document.querySelector('.links a').textContent);        // first match
console.log('querySelectorAll    :', [...document.querySelectorAll('.links a')].map((a) => a.textContent));
console.log('attribute selector  :', document.querySelector('[data-track]').textContent);
console.log(':not + :nth-child   :', [...document.querySelectorAll('li:not(:first-child) a')].map((a) =>
a.textContent));
console.log('no match            :', document.querySelector('.nope'), document.querySelectorAll('.nope').length);

const staticList = document.querySelectorAll('li');          // snapshot
const liveList = document.getElementsByTagName('li');         // live
document.querySelector('ul').append(document.createElement('li'));
console.log('\nafter adding an <li>: static', staticList.length, '| live', liveList.length);

const blog = document.querySelector('a[href="/blog"]');
console.log('closest("nav")      :', blog.closest('nav').id);
console.log('matches(":not(.x)") :', blog.matches('a:not(.active)'));
const li = blog.parentElement;
console.log('nextSibling (a node):', JSON.stringify(li.nextSibling.textContent), '<- whitespace text node');

try { document.querySelector('a[href=/]'); } catch (e) { console.log('\nbad selector        :', e.name); }
try { document.querySelector('#123'); } catch (e) { console.log('id starting w/ digit:', e.name, '-> use
getElementById("123") or CSS.escape'); }
```

Output:

```
getElementById      : menu
querySelector       : Home
querySelectorAll    : [ 'Home', 'Blog', 'About' ]
attribute selector  : About
:not + :nth-child   : [ 'Blog', 'About' ]
scoped query        : 3
forms collection    : email
no match            : null 0

after adding an <li>: static 3 | live 4
NodeList has forEach: function | HTMLCollection has forEach: undefined

parentElement       : LI
closest("nav")      : menu
closest(".missing") : null
matches(":not(.x)") : true
previousElementSibling: Home
nextElementSibling  : About
nextSibling (a node): "\n    " <- whitespace text node
firstElementChild   : Home
contains            : true

bad selector        : SyntaxError
id starting w/ digit: SyntaxError -> use getElementById("123") or CSS.escape
```

Common mistakes

- `document.querySelector('.item').classList.add(…)` when nothing matches → `TypeError: Cannot read properties of null`. Check for `null`, or use optional chaining (`?.`, which returns `undefined` instead of throwing when the left side is `null`/`undefined`) when absence is fine.
- Calling `.map` on a `NodeList` — it has `forEach` but not `map`; use `[...list]` or `Array.from(list)`.
- Looping over a **live** collection while removing items from it — indexes shift and items are skipped. Loop over a static copy.
- Unquoted attribute values containing `/` or starting with digits in selectors → `SyntaxError`. Quote them: `a[href="/"]`.

Self-check

1. Which selection method returns a live collection? → `getElementsBy…`. `querySelectorAll` is static.
2. `el.closest('li')` when `el` itself is an `<li>`? → Returns `el`.
3. How do you get the next *element*, skipping text? → `nextElementSibling`.

TL;DR

`querySelector(All)` with CSS selectors for almost everything; `getElementById` for ids; `closest`/`matches`/`contains` for relationships. `querySelectorAll` is a static snapshot; `getElementsBy*` collections are live.

13.3 Creating, inserting and removing nodes

What it is

Task	API
Create	`document.createElement('li')`, `document.createTextNode('hi')`, `document.createDocumentFragment()`
Insert inside	`parent.append(...nodesOrStrings)`, `parent.prepend(...)`, `parent.replaceChildren(...)` (also empties)
Insert beside	`el.before(...)`, `el.after(...)`
Remove / replace	`el.remove()`, `el.replaceWith(...)`
Insert HTML string	`el.insertAdjacentHTML('beforebegin' / 'afterbegin' / 'beforeend' / 'afterend', html)`
Copy	`el.cloneNode(true)` (deep = with all descendants)
Text	`el.textContent = '...'` (safe, never parsed)
Markup	`el.innerHTML = '...'` / `el.outerHTML` (parsed as HTML)

Older equivalents you'll see in legacy code (older code that is still in use): `appendChild`, `insertBefore`, `removeChild`, `replaceChild` (they accept only nodes, not strings).

`code/03_create_modify.mjs` — key parts:

```
const li = document.createElement('li');
li.textContent = 'appended';
list.append(li, 'a text node too');          // append accepts nodes AND strings
// ... prepend, after
list.lastChild.remove();                       // the text node
list.querySelector('li:nth-child(3)').replaceWith(/* new li "replaced" */);
document.getElementById('out').append(list.firstElementChild);   // appending an existing node MOVES it

const userInput = '<img src=x onerror="alert(1)">';
box.textContent = userInput;                    // shown as text, safe
console.log('3) textContent ->', box.innerHTML);
box.innerHTML = '<b>bold</b> and <i>italic</i>'; // parsed as HTML (only for trusted strings!)

p.insertAdjacentHTML('beforebegin', '<span>bb</span>');  // + afterbegin, beforeend, afterend

const frag = document.createDocumentFragment();
for (const name of ['Ada', 'Grace', 'Linus']) {
  const item = document.createElement('li');
  item.textContent = name;
  frag.append(item);
}
list.replaceChildren(frag);                     // one insertion

const tpl = document.getElementById('card');    // <template id="card"><article>...</article></template>
for (const pr of products) {
  const node = tpl.content.cloneNode(true);     // deep copy of the template's content
  node.querySelector('h2').textContent = pr.name;
  node.querySelector('.price').textContent = `$${pr.price}`;
  grid.append(node);
}
```

Output:

```
1) <li>prepended</li><li>after first</li><li>first</li><li>appended</li>a text node too
2) <li>after first</li><li>replaced</li><li>appended</li> | out: <li>prepended</li>
3) textContent -> &lt;img src=x onerror="alert(1)"&gt;
   innerHTML   -> 2 elements; textContent = "bold and italic"
   outerHTML   -> <div id="out"><b>bold</b> and <i>italic</i></div>
4) <span>bb</span><p><span>ab</span>P<span>be</span></p><span>ae</span>
5) <li>Ada</li><li>Grace</li><li>Linus</li> | fragment is now empty: 0
6) <article class="card"><h2>Lamp</h2><p class="price">$25</p></article><article class="card"><h2>Desk</h2><p
class="price">$180</p></article>
7) innerText in jsdom: not implemented (use textContent)
```

`textContent` **vs** `innerText` **vs** `innerHTML`

Property	Reads	Writes	Cost / safety
`textContent`	All text of all descendants, including hidden elements and `<script>` text	Replaces children with one text node	Fast; **safe** for user data
`innerText`	Only *rendered* text (respects CSS `display:none`, line breaks)	Like textContent but `\n` → ` `	Needs layout → slower; not in jsdom
`innerHTML`	Serialized HTML of the children (turned back into a string)	**Parses** the string as HTML and replaces children	**XSS risk** (cross-site scripting — attacker code injected into your page) with untrusted strings; destroys existing child nodes and their listeners

Line 3 of the output shows why `textContent` is safe: the "image" became harmless text (`<img …`). Assigning the same string to `innerHTML` would create a real `<img>` whose `onerror` runs attacker code — the **XSS** attack covered in Chapter 14 §14.10.

Why fragments and templates

- A **`DocumentFragment`** is a lightweight, parentless container. Appending it moves its children in one operation (the fragment ends up empty — see line 5), so the page does one update instead of N.
- A **`<template>`** holds inert markup (not rendered, scripts don't run, images don't load). `template.content.cloneNode(true)` stamps out copies — this is how frameworks-free apps and Web Components (the browser's built-in way to define your own reusable HTML tags) render lists. Fill clones with `textContent`, not string concatenation.

Common mistakes

- `el.innerHTML += '<li>…</li>'` in a loop: re-parses the whole content each time and wipes listeners and input state on existing children. Use `append` or `insertAdjacentHTML('beforeend', …)`.
- Forgetting that appending an existing node **moves** it (line 2). Clone it if you want a copy.
- `cloneNode` doesn't copy listeners added with `addEventListener`.

TL;DR

`createElement` + `textContent` + `append`/`before`/`after`/`remove`/`replaceWith`. Use `textContent` for data, `innerHTML` only for trusted markup. Batch many inserts with a fragment; stamp repeated markup from a `<template>`.

13.4 Attributes, properties, classes, data and styles

What it is

- An **attribute** is what's written in the HTML (`<input value="1">`); read/write with `getAttribute`, `setAttribute`, `hasAttribute`, `removeAttribute`, `toggleAttribute`. Always strings.

- A **property** is a field on the JavaScript element object (`input.value`). Most standard attributes are **reflected** as properties with the same name, but properties can be typed (booleans, numbers), normalized (absolute URLs — the full address including `https://` and the domain) and can hold **live state** that the attribute doesn't.

`code/04_attributes_properties.mjs` — key parts:

```
const qty = document.getElementById('qty');      // <input id="qty" type="number" value="1">
qty.value = '5';                                 // what the user "typed"
console.log('value property       :', qty.value, '| value attribute:', qty.getAttribute('value'), '|
defaultValue:', qty.defaultValue);
console.log('valueAsNumber        :', qty.valueAsNumber, typeof qty.valueAsNumber);
console.log('href property        :', link.href, '(resolved, absolute)');
console.log('href attribute       :', link.getAttribute('href'));

agree.checked = false;                           // <input type="checkbox" checked>
console.log('checked prop / attr :', agree.checked, '/', agree.hasAttribute('checked'));
buy.setAttribute('disabled', 'false');           // still disabled!
console.log('disabled="false"    :', buy.disabled);

buy.classList.add('loading');
buy.classList.remove('primary');
buy.classList.toggle('active');                  // add if missing
buy.classList.toggle('active', false);           // force off
buy.classList.replace('btn', 'button');

console.log('dataset             :', { ...buy.dataset });     // data-product-id="42" data-in-stock="true"
buy.dataset.variantColor = 'blue';               // -> data-variant-color="blue"

buy.style.backgroundColor = 'yellow';            // camelCase property
buy.style.setProperty('--accent', 'teal');       // custom properties need setProperty
console.log('computed color      :', getComputedStyle(buy).color);
```

Output:

```
value property       : 5 | value attribute: 1 | defaultValue: 1
valueAsNumber        : 5 number
href property        : http://localhost/docs?page=2 (resolved, absolute)
href attribute       : /docs?page=2
checked prop / attr : false / true
disabled="false"    : true
after remove         : false
className            : btn primary
classList            : [ 'button', 'loading' ] | contains loading: true
dataset              : { productId: '42', inStock: 'true' }
inStock is a string : string true
new attribute        : blue
style attribute      : color: red; background-color: yellow; --accent: teal;
computed color       : rgb(255, 0, 0)
attributes           : [
  'id',
  'class',
  'data-product-id',
  'data-in-stock',
  'style',
  'data-variant-color'
]
```

Key rules

Rule	Example
For form controls, the **attribute is the initial/default value**; the **property is the current value**	`value` / `defaultValue`, `checked` / `defaultChecked`
Boolean attributes: present = true, whatever the value	`disabled="false"` is still disabled; use `el.disabled = false` or `removeAttribute`
Some names differ	`class` → `className` / `classList`; `for` → `htmlFor`
URL properties are absolute	`a.href`, `img.src`
`data-*` → `el.dataset.camelCase` (words joined, later ones capitalised), always strings	`data-in-stock` → `dataset.inStock === 'true'`
`el.style` = **inline** style only; `getComputedStyle(el)` = final values from all CSS	Computed values are resolved (`rgb(255, 0, 0)`)

Prefer toggling **classes** over setting many inline styles: the look stays in CSS, and one class change is one style recalculation. Inline `style` is best for truly dynamic values (positions, a progress width), ideally through CSS custom properties (CSS variables, named with a leading `--`): `el.style.setProperty('--progress', '40%')`.

Self-check

1. User types "7" into `<input value="1">`. `getAttribute('value')`? → `"1"`; `input.value` is `"7"`.
2. How do you un-disable a button? → `btn.disabled = false` (or `removeAttribute('disabled')`), not `setAttribute('disabled', 'false')`.
3. `data-user-id="9"` → how to read it as a number? → `Number(el.dataset.userId)`.

TL;DR

Attributes = HTML text (strings, initial values); properties = live typed state. Use `classList` for classes, `dataset` for `data-*`, `style` for inline styles and `getComputedStyle` to read final styles.

13.5 Events and listeners

What it is

An **event** is a notification that something happened — a click, a key press, a form submission, a resource finishing loading. An **event listener** (handler) is a function registered to run when a given event type reaches a given **target** (element, `document`, `window`, or any `EventTarget` — the built-in base type for every object that can receive events).

Analogy: a doorbell. `addEventListener('click', fn)` wires a chime to the button; when someone presses it, every wired chime rings in the order it was installed.

How it works

1. `target.addEventListener(type, listener, options?)` registers.
2. The event occurs (user action, network, or `target.dispatchEvent(event)` from code).
3. The browser builds an **event object** and sends it along a path through the tree (§13.6), calling matching listeners with it.
4. User-generated events arrive as **tasks** on the event loop; `dispatchEvent` calls listeners **synchronously**, before it returns.

The event object's most-used members:

Member	Meaning
`type`	`'click'`, `'keydown'`, …
`target`	The element where the event originated (the deepest one clicked)
`currentTarget`	The element whose listener is running now (= `this` in a non-arrow listener)
`eventPhase`	1 capture, 2 at target, 3 bubble
`preventDefault()` / `defaultPrevented`	Cancel the browser's default action (§13.8)
`stopPropagation()` / `stopImmediatePropagation()`	Stop the journey (§13.6)
`isTrusted`	`true` only for real user/browser-generated events
`key`, `code`, `ctrlKey`, `shiftKey`, `altKey`, `metaKey`	Keyboard info (`key` = character/meaning, `code` = physical key)
`clientX/Y`, `button`	Mouse/pointer position and button
`detail`	Data on a `CustomEvent`

`code/05_events_basics.mjs` — key parts:

```
function onClick(event) {
  console.log('1) type:', event.type, '| target:', event.target.id, '| currentTarget:', event.currentTarget.id,
    '| this === currentTarget:', this === event.currentTarget, '| isTrusted:', event.isTrusted);
}
btn.addEventListener('click', onClick);
btn.click();                                    // synthetic click (isTrusted false)

btn.addEventListener('click', () => console.log('2) second listener'));
btn.addEventListener('click', onClick);         // duplicate, ignored
btn.click();

btn.removeEventListener('click', () => {});     // different function: does nothing
btn.removeEventListener('click', onClick);
console.log('3) after remove:'); btn.click();

b2.addEventListener('click', () => console.log('4) once fired'), { once: true });
b2.click(); b2.click();
const ac = new AbortController();
b2.addEventListener('click', () => console.log('   A'), { signal: ac.signal });
b2.addEventListener('click', () => console.log('   B'), { signal: ac.signal });
b2.click(); ac.abort(); b2.click();

b2.onclick = () => console.log('5) handler 1');
b2.onclick = () => console.log('5) handler 2 replaced 1');
b2.click();

input.addEventListener('keydown', (e) => console.log('6) key:', e.key, '| code:', e.code, '| ctrl:', e.ctrlKey));
input.dispatchEvent(new KeyboardEvent('keydown', { key: 's', code: 'KeyS', ctrlKey: true, bubbles: true }));

document.addEventListener('cart:add', (e) => console.log('7) custom event detail:', e.detail));
btn.dispatchEvent(new CustomEvent('cart:add', { detail: { sku: 'A-1', qty: 2 }, bubbles: true }));

console.log('8) before dispatch');
btn.addEventListener('ping', () => console.log('   listener'));
btn.dispatchEvent(new Event('ping'));
console.log('   after dispatch');
```

Output:

```
1) type: click | target: save | currentTarget: save | this === currentTarget: true | isTrusted: false
1) type: click | target: save | currentTarget: save | this === currentTarget: true | isTrusted: false
2) second listener
3) after remove:
2) second listener
4) once fired
   A
   B
   (both removed by one abort)
5) handler 2 replaced 1
6) key: s | code: KeyS | ctrl: true
   input event, value = Ada
7) custom event detail: { sku: 'A-1', qty: 2 }
8) before dispatch
   listener
   after dispatch
```

Listener options

Option	Effect
`capture: true`	Run during the capture phase (§13.6). Must match when removing.
`once: true`	Auto-remove after the first call
`passive: true`	Promise not to call `preventDefault()` — lets the browser scroll without waiting (§13.8)
`signal: abortSignal`	Remove the listener when the signal aborts — great for removing many at once (Ch. 12 §12.11)

`addEventListener` vs `on…` properties vs inline attributes

Style	Multiple handlers?	Notes
`el.addEventListener('click', fn)`	Yes	Options, removable — the standard choice
`el.onclick = fn`	No — assigning replaces (line 5)	Fine for quick scripts
`<button onclick="doIt()">`	—	Mixes JS into HTML, runs code from a string, blocked by a strict CSP (Content Security Policy — a rule list saying which scripts a page may run) (Ch. 14) — avoid

Common mistakes

- `addEventListener('click', handle())` — calls `handle` immediately and registers its return value. Pass the function.
- Removing with a *new* arrow function (line 3's no-op): keep a reference, or use `{ signal }`.
- Arrow function listeners don't get `this = element`; use `event.currentTarget` — it works for both styles.
- Listening for `'onclick'` — the event type has no `on` prefix.

TL;DR

`addEventListener(type, fn, { once, passive, signal, capture })`. The event object tells you `target` (origin), `currentTarget` (listener owner), and lets you cancel defaults or stop propagation. `dispatchEvent` + `CustomEvent` for your own events.

13.6 Propagation: capture, target, bubble

What it is

When you click a button inside nested boxes, you've also clicked each box, the body, the document and the window. The DOM delivers the event along that **propagation path** in three phases:

1. **Capture phase** — from `window` down through each ancestor toward the target; only listeners registered with `{ capture: true }` run.
2. **Target phase** — at the target itself; listeners run in registration order (capture ones first in modern browsers).
3. **Bubble phase** — back up from the target's parent to `window`; normal listeners run. Most events bubble; some don't.

Analogy: a letter delivered down the chain of command (capture), read by the addressee (target), then the reply copied back up to every manager (bubble).

The page `code/propagation.html` registers a capture and a bubble listener on `window`, `document`, `#outer`, `#inner` and the button, then clicks the button once:

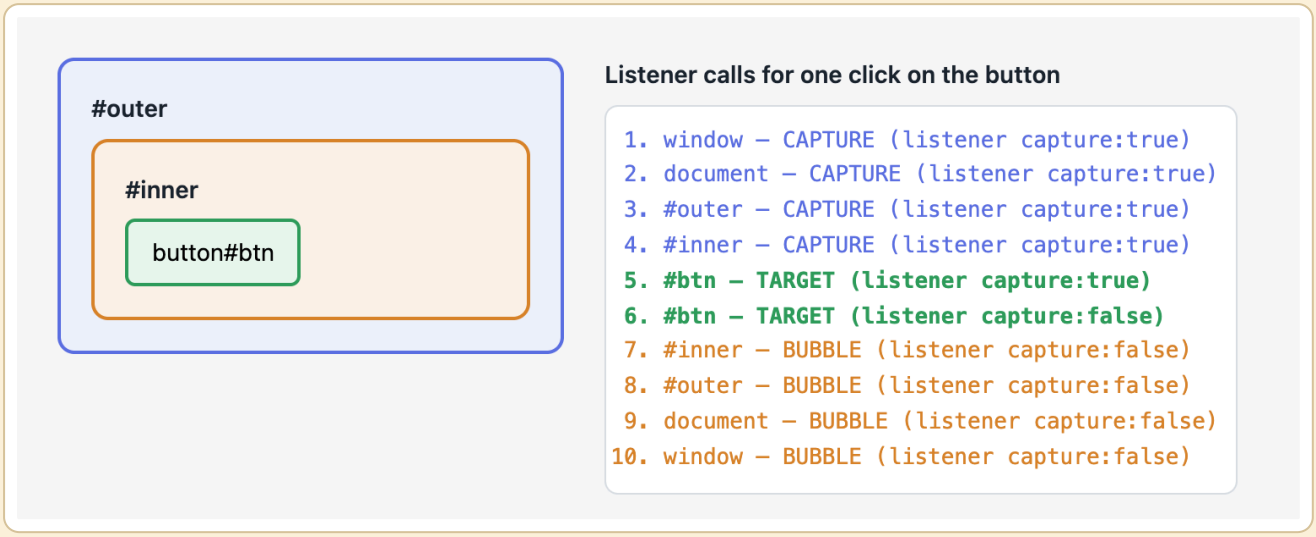


Figure — one click, ten listener calls: capture (blue) goes down, target (green), bubble (orange) goes up. Screenshot from headless Chrome.

`code/06_propagation.mjs` shows the same path in jsdom plus the ways to stop it:

```
for (const el of [window, document, ...els]) {
  const name = el === window ? 'window' : el === document ? 'document' : el.id;
  el.addEventListener('click', (e) => log.push(`${name}:${PHASE[e.eventPhase]}`), { capture: true });
  el.addEventListener('click', (e) => log.push(`${name}:${PHASE[e.eventPhase]}`));
}
document.getElementById('btn').click();

const stop = (e) => e.stopPropagation();
els[1].addEventListener('click', stop);          // on #inner, bubble phase
els[2].click();

btn.addEventListener('click', (e) => { log.push('btn:first-extra'); e.stopImmediatePropagation(); });
btn.addEventListener('click', () => log.push('btn:NEVER'));
btn.click();

document.body.addEventListener('focus', () => seen.push('focus (bubble listener)'));
document.body.addEventListener('focusin', () => seen.push('focusin'));
input.focus();
```

Output:

```
1) full path:
   window:CAPTURE > document:CAPTURE > outer:CAPTURE > inner:CAPTURE > btn:TARGET > btn:TARGET > inner:BUBBLE >
outer:BUBBLE > document:BUBBLE > window:BUBBLE
2) stopPropagation at inner:
   window:CAPTURE > document:CAPTURE > outer:CAPTURE > inner:CAPTURE > btn:TARGET > btn:TARGET > inner:BUBBLE
3) stopImmediatePropagation:
   window:CAPTURE > document:CAPTURE > outer:CAPTURE > inner:CAPTURE > btn:TARGET > btn:TARGET > btn:first-extra
4) new Event() defaults to x: bubbles=false
   body heard: [ 'focusin' ] <- focus does not bubble, focusin does
```

Stopping

Call	Effect
`stopPropagation()`	Finish the listeners on the current element, then stop travelling
`stopImmediatePropagation()`	Also skip the remaining listeners on the current element
`preventDefault()`	Unrelated to propagation: cancels the default action only

Events that don't bubble

`focus` / `blur` (an element gaining / losing **focus**, i.e. becoming / no longer being the one that receives keyboard input; use `focusin` / `focusout` to delegate), `mouseenter` / `mouseleave` (use `mouseover` / `mouseout`), `load` / `error` on elements like `<img>`, `scroll` on elements. Events you create with `new Event(type)` **don't bubble unless you pass** `{ bubbles: true }` (line 4) — a common reason a custom event "isn't heard" by a parent.

When not to stop propagation

`stopPropagation` is a blunt tool: it also blocks analytics listeners (code that records what users do), "click outside to close" handlers on `document`, and framework root listeners (React — a popular UI library — attaches its listeners at the root container). Prefer checking `event.target` in the outer handler, or `defaultPrevented` as a signal.

In the interview

30-second answer: "Events travel in three phases: capture from window down to the target, the target phase, then bubbling back up. `addEventListener` listens in the bubble phase by default; `{capture: true}` switches to capture. `stopPropagation` halts the journey, `stopImmediatePropagation` also skips other listeners on the same element, and `preventDefault` is separate — it only cancels the browser's default action."

- *Q: `target` vs `currentTarget`?* A: `target` is where the event started; `currentTarget` is the element whose listener is running.
- *Q: When would you use capture?* A: To see an event before children can stop it — e.g. a global "close menus on any click" or focus tracking.

TL;DR

Capture (down, `{capture:true}`) → target → bubble (up, default). Most events bubble; `focus`, `mouseenter`, element `load` don't. Stop sparingly.

13.7 Event delegation

What it is

Event delegation means putting **one** listener on a common ancestor instead of one per child, and using `event.target` (usually with `closest`) to find which child was acted on. It works because events bubble.

The problem it solves

- A list of 1,000 rows with 1,000 listeners costs memory and setup time.
- Items added later (from `fetch` — the browser function for network requests — or from the user) would need listeners attached each time — and removed ones should be cleaned up.
- With delegation, new items are handled automatically.

How it works

1. Attach a listener to a stable container (`ul`, `table`, a component root — the outermost element of a UI component — or `document`).
2. In the handler, `const item = event.target.closest('selector')`.
3. If `item` is `null` (click was elsewhere) — or outside the container (`container.contains(item)`) — ignore it.
4. Read what you need from the item (`item.dataset.id`) and act.

`code/07_delegation.mjs`:

```
list.addEventListener('click', (event) => {
  const del = event.target.closest('button.del');
  if (del && list.contains(del)) {
    const li = del.closest('li');
    console.log('delete item', li.dataset.id);
    li.remove();
    return;
  }
  const li = event.target.closest('li');
  if (li) {
    li.classList.toggle('done');
    console.log('toggle item', li.dataset.id, '-> done =', li.classList.contains('done'));
  }
});

let nextId = 3;
document.getElementById('add').addEventListener('click', () => {
  list.insertAdjacentHTML('beforeend', `<li data-id="${nextId}"><span>New task ${nextId}</span> <button
class="del">x</button></li>`);
  nextId += 1;
});

list.querySelector('li[data-id="1"] span').click();      // click on the text (a child of li)
document.getElementById('add').click();                   // adds item 3 AFTER the listener was attached
list.querySelector('li[data-id="3"] span').click();      // still handled - no new listener needed
list.querySelector('li[data-id="2"] .del').click();
```

Output:

```
toggle item 1 -> done = true
toggle item 3 -> done = true
delete item 2
remaining: [ '1(done)', '3(done)' ]
listeners attached to items: 0, to the list: 1
```

The same idea as a real page, `code/todo_delegation.html`: one `click` listener on the `<ul>`, one `submit` listener on the form (with `preventDefault`), and new items built with `textContent` — note the task typed as `Call <b>mom</b>` is shown literally, not as bold HTML. The script adds two tasks, ticks two and deletes one, then the page was screenshotted:

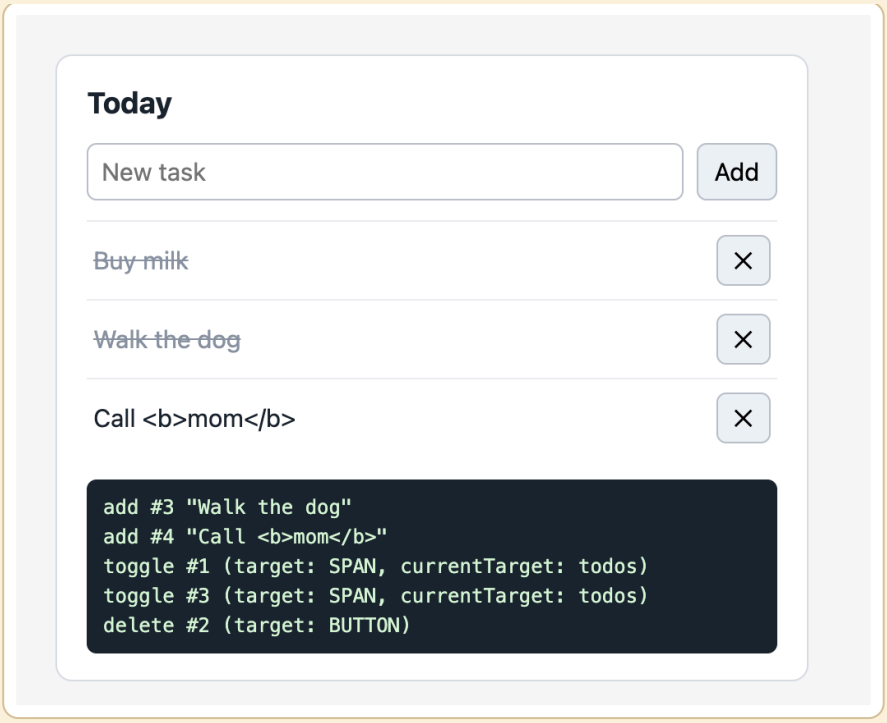


Figure — event delegation in Chrome: `target` is the inner `SPAN` / `BUTTON`, `currentTarget` is always the list.

Why `closest` and not `event.target` directly

`event.target` is the **deepest** element clicked — the `<span>` inside the `<li>`, or an `<svg>` icon inside a button. `closest('li')` climbs to the element you actually care about.

Trade-offs

	Delegation	Listener per element
Listeners	One	N
Dynamic items	Handled automatically	Must attach/detach
Non-bubbling events	Needs the bubbling twin (`focusin`) or capture	Works directly
Handler complexity	Must route by selector	Simple

TL;DR

One listener on the container + `event.target.closest(selector)` handles all current and future children. Use it for lists, tables and menus.

13.8 Default actions and `preventDefault`

What it is

Many events have a **default action** the browser performs after listeners run: a link navigates, a form submits (and reloads the page), a checkbox toggles, a key types a character, a wheel scrolls, a right-click opens the context menu. `event.preventDefault()` cancels it, if the event is **cancelable** (marked as allowed to be cancelled — most user-input events are). `dispatchEvent` returns `false` when a listener cancelled it.

Event	Default action	Typical reason to prevent
`submit` on a form	Send the form and navigate	Validate and send with `fetch` instead
`click` on `<a href>`	Navigate	Client-side routing (switching views and URL without a page reload; History API, Ch. 14)
`keydown`	Insert the character / browser shortcut	Custom shortcut like Ctrl+S
`contextmenu`	Show the browser menu	Custom context menu
`dragover`	Refuse drops	Allow dropping files

(Output lines for `preventDefault` are in the forms program, §13.9.)

Passive listeners

For `touchstart`, `touchmove` and `wheel`, the browser must wait for your listener to finish before scrolling, in case you call `preventDefault()`. That can make scrolling janky (stuttering instead of smooth). `{ passive: true }` promises you won't, so scrolling starts immediately. Browsers other than Safari already default `passive` to `true` for these events when the listener is on `window`, `document` or `body`; calling `preventDefault()` inside a passive listener is ignored (with a console warning).

Common mistakes

- Using `return false` in an `addEventListener` handler — it does nothing there (it only works in `on…` properties).
- Preventing the default of a link or button and forgetting keyboard and screen-reader users (people who use software that reads the page aloud) — use real `<button>`s for actions and `<a>` for navigation.
- `preventDefault()` does **not** stop propagation, and `stopPropagation()` does **not** prevent the default.

TL;DR

`preventDefault()` cancels the browser's built-in response (navigation, submit, typing, scrolling). Use passive listeners for scroll/touch.

13.9 Forms

What it is

Forms are where most user data enters a web app. The DOM gives you:

- `form.elements` — controls by `name` or index; `form.elements.email`.
- `submit` **event** on the form (fires for Enter in a field and for submit buttons) — listen here, not to the button's `click`.
- `FormData` — reads every successful control (named, enabled, checked where relevant) into key/value entries; sends directly as a `fetch` body.
- The **Constraint Validation API** — HTML attributes (`required`, `type="email"`, `min`, `max`, `minlength`, `pattern`) produce a `validity` object; `checkValidity()`, `reportValidity()`, `setCustomValidity(msg)`.
- `input` fires on every change of value; `change` fires when the change is committed (on blur for text, immediately for checkboxes/selects — a select is a dropdown).

`code/08_forms.mjs` — key parts:

```
age.value = '7';
console.log('1) form valid?', form.checkValidity());
for (const el of [email, age]) {
  const v = el.validity;
  console.log(`   ${el.name}: valid=${v.valid} valueMissing=${v.valueMissing} rangeUnderflow=${v.rangeUnderflow}
msg="${el.validationMessage}"`);
}
email.value = 'not-an-email';
console.log('   typeMismatch for "not-an-email":', email.validity.typeMismatch);

email.value = 'ada@example.com';
email.setCustomValidity(email.value.endsWith('@example.com') ? 'Please use your work email' : '');
console.log('2) customError:', email.validity.customError, '|', email.validationMessage);
email.setCustomValidity('');                 // '' clears the custom error

form.addEventListener('submit', (event) => {
  event.preventDefault();                     // stay on the page, send with fetch instead
  const data = new FormData(form);
  console.log('3) submit prevented:', event.defaultPrevented);
  console.log('   entries   :', JSON.stringify([...data.entries()]));
  console.log('   getAll    :', data.getAll('topics'));
  console.log('   as object :', JSON.stringify(Object.fromEntries(data)), '<- keeps only the LAST topic');
  console.log('   submitter :', event.submitter?.textContent);
});
form.elements.topics[1].checked = true;
form.requestSubmit(form.querySelector('button'));  // like clicking the button: validates, fires submit

age.addEventListener('input', () => log.push(`input(${age.value})`));
age.addEventListener('change', () => log.push(`change(${age.value})`));
// ... two input events, then one change event

const a = document.createElement('a'); a.href = '/leave';
a.addEventListener('click', (e) => e.preventDefault());
console.log('5) link click not cancelled?', a.dispatchEvent(new MouseEvent('click', { cancelable: true })));
cb.addEventListener('click', (e) => e.preventDefault());
cb.click();
console.log('   checkbox still checked after prevented click:', cb.checked);
```

Output:

```
1) form valid? false
   email: valid=false valueMissing=true rangeUnderflow=false msg="Constraints not satisfied"
   age: valid=false valueMissing=false rangeUnderflow=true msg="Constraints not satisfied"
   typeMismatch for "not-an-email": true
2) customError: true | Please use your work email
   now valid? true
3) submit prevented: true
   entries   : [["email","ada@example.com"],["age","36"],["topics","js"],["topics","css"],["plan","pro"],
["bio","Hello"]]
   getAll    : [ 'js', 'css' ]
   as object : {"email":"ada@example.com","age":"36","topics":"css","plan":"pro","bio":"Hello"} <- keeps only the
LAST topic
   submitter : Sign up
4) input(4) input(42) change(42)
5) link click not cancelled? false
   checkbox still checked after prevented click: true
```

(jsdom's `validationMessage` is a generic "Constraints not satisfied"; browsers give localized messages — in the user's language — like "Please fill out this field.")

The `validity` flags

Flag	Set when	Caused by
`valueMissing`	Empty required field	`required`
`typeMismatch`	Not a valid email/URL	`type="email"`, `type="url"`
`patternMismatch`	Regex (regular expression, a text pattern) doesn't match whole value	`pattern`
`tooShort` / `tooLong`	Length out of range (after user edits)	`minlength` / `maxlength`
`rangeUnderflow` / `rangeOverflow` / `stepMismatch`	Number/date out of range	`min` / `max` / `step`
`badInput`	Browser can't convert the input (letters in a number field)	input type
`customError`	`setCustomValidity('msg')` was called with a non-empty message	your code
`valid`	None of the above	—

The page `code/form_validation.html` uses `novalidate` (turning off the browser's own bubbles) and renders its own accessible messages (usable by everyone, including people on screen readers) from these flags — `aria-invalid` on the field and `aria-describedby` pointing to the message (ARIA attributes are extra HTML attributes that describe state and relationships to assistive technology). One delegated `input` listener on the form gives live feedback; the `submit` handler checks everything. After a scripted submit with a personal email, age 9 and no password:

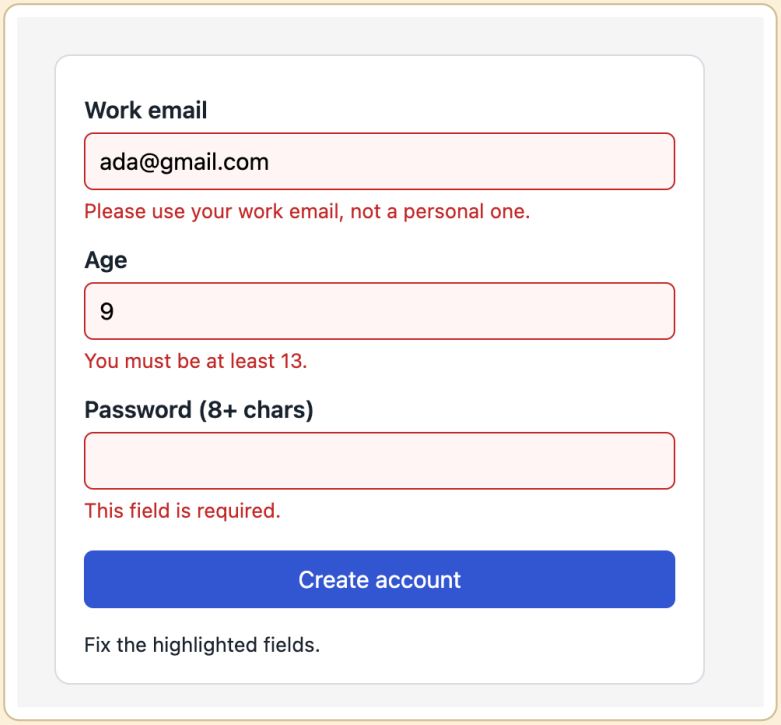


Figure — custom validation messages driven by the Constraint Validation API (headless Chrome).

Common mistakes

- Listening to the submit button's `click` → misses Enter-key submits. Listen to `submit` on the form.
- `Object.fromEntries(new FormData(form))` for multi-value fields (checkbox groups, `<select multiple>`) — keeps only the last value; use `getAll`.
- Client-side validation (checks running in the user's browser) as security. It's UX (user experience) only; **the server must validate everything again**.

- Forgetting `name` attributes — unnamed controls are left out of `FormData`.
- Calling `form.submit()` — it skips validation *and* the `submit` event. `requestSubmit()` behaves like a real submit.

TL;DR

Listen to `submit`, `preventDefault`, read with `FormData`, validate with `validity`/`setCustomValidity` for UX, and validate again on the server. `input` = every keystroke, `change` = committed value.

13.10 Watching the DOM: `MutationObserver`

What it is

`MutationObserver` calls you back when the DOM changes — children added/removed, attributes changed, text edited. Records are **batched** (collected and handed over together) and delivered as a **microtask** (a small queued job the engine runs before handling the next event), after the current script finishes (Ch. 12 §12.2). It replaced the old, slow "mutation events".

`code/09_mutation_observer.mjs`:

```
const observer = new MutationObserver((records) => {
  console.log(`observer got ${records.length} records:`,
    records.map((r) => r.type === 'childList' ? `childList +${r.addedNodes.length} -${r.removedNodes.length}` :
`${r.type}:${r.attributeName}`));
});
observer.observe(feed, { childList: true, attributes: true, subtree: true });

for (let i = 1; i <= 3; i++) feed.insertAdjacentHTML('beforeend', `<li>post ${i}</li>`);
feed.firstElementChild.remove();
feed.setAttribute('data-count', '2');
console.log('sync code done - no callback yet (it is a microtask)');
await null;
observer.disconnect();
```

Output:

```
sync code done - no callback yet (it is a microtask)
observer got 5 records: [
  'childList +1 -0',
  'childList +1 -0',
  'childList +1 -0',
  'childList +0 -1',
  'attributes:data-count'
]
done
```

Uses: reacting to DOM changes made by third-party code (scripts you didn't write, such as ads or chat widgets), auto-initialising widgets (self-contained UI pieces) when they appear, test utilities that wait for elements (Testing Library — a popular UI-testing toolkit — and its `waitFor`/`findBy*` use it). Don't use it to watch your own changes — you already know when you make them. Observers for size and visibility (`ResizeObserver`, `IntersectionObserver`) are in Chapter 14 §14.9.

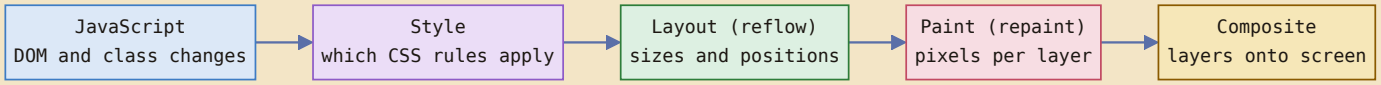
TL;DR

`new MutationObserver(cb).observe(node, { childList, attributes, subtree, characterData })`; batched microtask callbacks; `disconnect()` when done.

13.11 Rendering performance: reflow, repaint and layout thrashing

What it is

After DOM or style changes, the browser's **rendering pipeline** works out what to draw:



- **Reflow / layout** — computing geometry. Changing width, height, font size, adding elements, etc. dirties layout (marks it as needing recalculation) — often for the whole subtree (an element plus everything inside it) or document.
- **Repaint** — redrawing pixels without geometry changes (color, background, shadow).
- **Composite** — assembling layers (separately drawn sheets of the page that the GPU, the graphics chip, stacks together); changing only `transform` and `opacity` can often skip layout and paint entirely, which is why they're the go-to properties for animation.

Normally the browser **batches** all your changes and runs the pipeline once, just before the next frame (the next screen refresh, usually 60 per second; §12.2 step 3). But if you **read** a geometry property (`offsetWidth`, `offsetTop`, `clientHeight`, `scrollTop`, `getBoundingClientRect()`, `getComputedStyle(...)` values) after a write, the browser must run layout **synchronously** to give you a correct answer — a **forced synchronous layout**. Doing write → read → write → read in a loop forces a layout on every iteration: **layout thrashing**.

Measured

The page `code/reflow.html` widens 2,000 boxes by 10 px two ways, timing the median (middle value when sorted) of 7 runs in headless Chrome:

```
// Interleaved (thrashing): each read forces a layout because of the previous write
for (const b of boxes) { const w = b.offsetWidth; b.style.width = (w + 10) + 'px'; }

// Batched: all reads (one layout), then all writes (one layout before the next frame)
const ws = boxes.map((b) => b.offsetWidth);
boxes.forEach((b, i) => { b.style.width = (ws[i] + 10) + 'px'; });
```

Resize 2,000 boxes to 10 px wider than they are (median of 7 runs)

Strategy	Pattern	Time
Interleaved (thrashing)	`for each box: read offsetWidth, then write style.width`	346.3 ms
Batched reads, then writes	`widths = boxes.map(read); boxes.forEach(write)`	1.5 ms

Interleaving forced 2000 synchronous layouts; batching needed one. Here that was about 231x slower (Chrome, this machine - your numbers will differ). First box: 110px (was 100px).

Figure — the same result, two orders of magnitude (roughly 100×) apart (one run in headless Chrome on the author's machine; exact numbers vary, the gap doesn't).

How to keep rendering fast

1. **Batch reads, then writes.** Never interleave them in loops. Libraries like fastdom formalize this.
2. **Change classes, not many individual styles**; let CSS do the rest.
3. **Build off-DOM** (fragment/template) and insert once (§13.3).
4. **Animate `transform` and `opacity`** instead of `top` / `left` / `width`, and use `requestAnimationFrame` for JS-driven animation (Ch. 14 §14.5).

5. **Debounce/throttle** (run only after events stop / at most once per interval) high-frequency handlers (`scroll`, `resize`, `input`) and use passive listeners.
6. **Virtualize** very long lists (render only visible rows) and consider CSS `content-visibility: auto` for long off-screen sections.
7. **Measure** with the DevTools Performance panel — purple "Layout" blocks with a "Forced reflow" warning point straight at the culprit.

In the interview

> **30-second answer:** "The browser recalculates style, layout, paint and composite. It batches DOM writes until the next frame, but reading geometry like `offsetWidth` right after a write forces a synchronous layout. Doing that in a loop is layout thrashing. The fix is to batch all reads, then all writes, animate with `transform` / `opacity`, and use `requestAnimationFrame`."

- *Q: Reflow vs repaint?* A: Reflow recomputes geometry (expensive, cascades); repaint redraws pixels without geometry changes.
- *Q: Which properties avoid layout?* A: `transform` and `opacity` typically only need compositing.

TL;DR

Writes are cheap until you read layout. Batch reads then writes; animate `transform`/`opacity`; measure in DevTools.

13.12 When scripts run: `DOMContentLoaded`, `load` and `defer`

A script can only find elements that have already been parsed. The page lifecycle:

Moment	Event	Meaning
HTML fully parsed, DOM ready, deferred scripts run	`DOMContentLoaded` on `document`	Safe to query any element
Everything loaded (images, styles, iframes — pages embedded in the page)	`load` on `window`	Needed only for image sizes etc.
User leaves / tab hidden	`pagehide`, `visibilitychange`	Save state; `beforeunload` only to warn about unsaved changes

How `<script>` loading affects this:

Tag	Blocks parsing?	Runs when	Order
`<script src>` in `<head>`	Yes (download + run)	Immediately	Document order
`<script defer src>`	No	After parsing, before `DOMContentLoaded`	Document order
`<script async src>`	No (pauses only to run)	As soon as downloaded	Unpredictable
`<script type="module">`	No	Deferred by default	Document order

Recommended: put scripts in `<head>` with `defer` (or use `type="module"`, which is deferred automatically — Chapter 10 §10.7). Then the DOM is ready when your code runs, with no `DOMContentLoaded` wrapper needed. Use `async` for independent scripts like analytics.

```
<!-- (HTML — not run) -->
<head>
  <script src="app.js" defer></script>
</head>
```

TL;DR

Use `defer` or modules so scripts run after parsing. `DOMContentLoaded` = DOM ready; `load` = all resources loaded.

Chapter 13 TL;DR

- **DOM** = live tree of nodes (element, text, comment) built by the parser; use element-only navigation (`children`, `nextElementSibling`).
- **Select** with `querySelector(All)` / `getElementById`; relate with `closest`, `matches`, `contains`. `querySelectorAll` is static; `getElementsBy*` are live.
- **Modify** with `createElement`, `append`/`prepend`/`before`/`after`/`replaceWith`/`remove`, `replaceChildren`. `textContent` for data; `innerHTML` only for trusted markup. Fragments and `<template>` for bulk rendering.
- **Attributes vs properties**: attributes are initial HTML strings; properties are live typed state. `classList`, `dataset`, `style`, `getComputedStyle`.
- **Events**: `addEventListener(type, fn, { once, passive, signal, capture })`; `target` vs `currentTarget`; `CustomEvent` + `dispatchEvent` (synchronous).
- **Propagation**: capture ↓, target, bubble ↑; `stopPropagation` sparingly; `focus`/`mouseenter` don't bubble; `new Event` needs `{ bubbles: true }`.
- **Delegation**: one listener on the container + `event.target.closest()`.
- `preventDefault` cancels default actions; passive listeners for scroll/touch.
- **Forms**: listen to `submit`, use `FormData` (`getAll` for multi-values), the `validity` API and `setCustomValidity`; `requestSubmit()` not `submit()`; re-validate on the server.
- **MutationObserver**: batched microtask callbacks for DOM changes.
- **Performance**: batch reads then writes (thrashing measured ~230× slower), animate `transform`/`opacity`, `requestAnimationFrame`.
- **Loading**: `defer`/modules; `DOMContentLoaded` vs `load`.

Next: talking to servers, storing data, and keeping pages safe — Chapter 14: Browser APIs and Security.

Chapter 14 — Browser APIs and Security

Beyond the DOM, browsers give JavaScript a large toolbox: talking to servers (`fetch`), remembering things (Web Storage, cookies, IndexedDB), scheduling visual updates (`requestAnimationFrame`), managing URLs and history, running code on other threads (separate lines of execution that run at the same time — Web Workers), and watching elements (observers). The same power makes web pages a target, so the second half of this chapter covers the security model — the same-origin policy and CORS — and the three attacks every web developer must be able to prevent: XSS (cross-site scripting), CSRF (cross-site request forgery), and the defence-in-depth (an extra protective layer in case another one fails) header CSP (Content Security Policy).

What you'll learn

1. `fetch`: requests, responses, JSON, headers, HTTP vs network errors, timeouts.
2. The same-origin policy and CORS, including preflight requests (demonstrated in Chrome).
3. `localStorage` and `sessionStorage`.
4. Cookies and IndexedDB, and how to choose a storage mechanism.
5. Timers vs `requestAnimationFrame` for smooth animation.
6. `URL` and `URLSearchParams`.
7. The History API and client-side routing.
8. Web Workers for CPU-heavy work.
9. `IntersectionObserver` and `ResizeObserver`.
10. XSS and how to sanitize; CSRF and how to block it; Content Security Policy.

Before you start. Promises and `async`/`await` (Chapter 12), and DOM/events (Chapter 13). JSON is covered in Chapter 5; error classes in Chapter 11.

How the code runs. Programs in `code/` run in Node 24. `fetch` demos start a real **`node:http` server inside the same script** on a free local port (a numbered "door" on a machine that one network program listens on), so nothing touches the internet. Storage, URL and History demos use jsdom (`tools_dom.mjs`, a JavaScript implementation of the browser DOM for Node), which implements `localStorage`, `document.cookie` and `history`. Features that need a real browser — CORS enforcement, CSP, Web Workers, observers, IndexedDB — are small pages served by `code/10_browser_demo_server.mjs` on `localhost:4101` (`localhost` = your own computer; this one is the "site") and `localhost:4102` (a second origin, the "API"), run in headless Chrome (Chrome with no visible window, driven by a script) and screenshotted.

14.1 `fetch` and HTTP

What it is

`fetch(url, options?)` sends an HTTP (HyperText Transfer Protocol — the request/response language browsers and web servers speak; see System Design Fundamentals) request and returns a promise for a **`Response`**. It's built into browsers and, since Node 18, into Node (same API). It replaced the callback-based `XMLHttpRequest` (XHR).

HTTP in one paragraph: a **request** has a **method** (`GET` read, `POST` create, `PUT`/`PATCH` update, `DELETE` remove), a URL, **headers** (metadata — data about the request — like `Content-Type`, the body's format, and `Authorization`, the caller's credentials) and optionally a **body**. The **response** has a **status code** (2xx success, 3xx redirect — "look elsewhere", 4xx client error such as 404 Not Found or 401 Unauthorized, 5xx server error), headers and a body.

How it works

1. `fetch` resolves as soon as the **headers** arrive — with a `Response`, whatever the status code.
2. You read the body with one of `res.json()`, `res.text()`, `res.blob()` (a Blob is raw binary data such as an image), `res.arrayBuffer()`, `res.formData()` — each returns another promise, and a body can be read **once**.

3. `fetch` **rejects only on network failure** (DNS — the system that turns names like `example.com` into IP addresses — failing, refused connection, CORS block, abort). A 404 or 500 is a successful HTTP exchange: check `res.ok` (status 200–299).

`code/01_fetch.mjs` — the server part:

```
const server = http.createServer(async (req, res) => {
  const url = new URL(req.url, 'http://localhost');
  const send = (status, body, headers = {}) => {
    res.writeHead(status, { 'Content-Type': 'application/json', ...headers });
    res.end(JSON.stringify(body));
  };
  if (req.method === 'GET' && url.pathname === '/api/users') return send(200, users, { 'X-Total-Count':
String(users.length) });
  if (req.method === 'POST' && url.pathname === '/api/users') { /* read body, push, send(201, user, { Location })
*/ }
  if (url.pathname === '/api/slow') return setTimeout(() => send(200, { ok: true }), 2000);
  if (url.pathname === '/api/crash') return send(500, { error: 'database unavailable' });
  if (url.pathname === '/page') { res.writeHead(200, { 'Content-Type': 'text/html' }); return res.end('<!doctype
html><h1>Not JSON</h1>'); }
  send(404, { error: `no route ${req.method} ${url.pathname}` });
});
await new Promise((r) => server.listen(0, '127.0.0.1', r));   // port 0 = any free port
const BASE = `http://127.0.0.1:${server.address().port}`;
```

The client part:

```
// 1) GET + JSON
const res = await fetch(`${BASE}/api/users`);
console.log('1) status:', res.status, res.statusText, '| ok:', res.ok, '| type:', res.headers.get('content-type'),
'| total:', res.headers.get('x-total-count'));
console.log('   body  :', await res.json());

// 2) POST JSON
const created = await fetch(`${BASE}/api/users`, {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ name: 'Grace' }),
});
console.log('2) POST ->', created.status, created.headers.get('location'), await created.json());

// 3) HTTP errors do NOT reject: check res.ok yourself
const notFound = await fetch(`${BASE}/api/nope`);
console.log('3) 404 resolved, ok =', notFound.ok, '| body:', (await notFound.json()).error);

// 4) A reusable helper that turns HTTP errors into exceptions
class HttpError extends Error {
  constructor(res, body) { super(`HTTP ${res.status} for ${res.url}`); this.name = 'HttpError'; this.status =
res.status; this.body = body; }
}
async function getJSON(path, init) {
  const r = await fetch(`${BASE}${path}`, init);
  const isJson = r.headers.get('content-type')?.includes('application/json');
  const body = isJson ? await r.json() : await r.text();
  if (!r.ok) throw new HttpError(r, body);
  if (!isJson) throw new TypeError(`expected JSON, got ${r.headers.get('content-type')}`);
  return body;
}

// 5) Network failure DOES reject (TypeError) - a port where nothing listens
await fetch(`http://127.0.0.1:${deadPort}/`).catch((e) => console.log('5) network error:', e.name, '-', e.message,
'| cause:', e.cause?.code));

// 6) Timeout / cancel with AbortSignal
await fetch(`${BASE}/api/slow`, { signal: AbortSignal.timeout(100) })
  .catch((e) => console.log('6) aborted:', e.name, /* ... */));

// 7) A body can be read only once; clone() first if you need it twice
const r7 = await fetch(`${BASE}/api/users`);
const copy = r7.clone();
await r7.json();
await r7.text().catch((e) => console.log('7) second read:', e.name, '-', e.message));

// 8) Query strings, other methods, Request/Headers objects
const q = new URL('/api/nope', BASE);
q.searchParams.set('search', 'Ada Lovelace & co');
const req = new Request(`${BASE}/api/users`, { method: 'DELETE', headers: new Headers({ Authorization: 'Bearer
demo-token' }) });
```

Output:

```
1) status: 200 OK | ok: true | type: application/json | total: 1
   body  : [ { id: 1, name: 'Ada' } ]
2) POST -> 201 /api/users/2 { id: 2, name: 'Grace' }
3) 404 resolved, ok = false | body: no route GET /api/nope
4) /api/users -> 2 users
4) /api/crash -> HttpError | 500 database unavailable
4) /page -> TypeError |  expected JSON, got text/html
5) network error: TypeError - fetch failed | cause: ECONNREFUSED
6) aborted: TimeoutError after ~100 ms
7) second read: TypeError - Body is unusable: Body has already been read
   clone still readable: 2 users
8) url: /api/nope?search=Ada+Lovelace+%26+co
   Request: DELETE Bearer demo-token -> 404
```

(Browsers word the network error as `TypeError: Failed to fetch` (Chrome) or `NetworkError when attempting to fetch resource` (Firefox); Node says `fetch failed` and puts the system error in `cause`.)

Useful `fetch` options

Option	Values	Purpose
`method`	`'GET'` (default), `'POST'`, …	HTTP method
`headers`	object or `Headers`	e.g. `Content-Type`, `Authorization`, `Accept`
`body`	string, `FormData`, `URLSearchParams`, `Blob`, `ArrayBuffer`, stream	Not allowed with GET/HEAD. `FormData` sets a multipart (several parts, e.g. fields plus files) `Content-Type` for you — don't set it yourself
`signal`	`AbortSignal`	Cancel / timeout (§12.11)
`credentials`	`'same-origin'` (default), `'include'`, `'omit'`	Whether cookies are sent (§14.2)
`mode`	`'cors'` (default), `'same-origin'`, `'no-cors'`	CORS behaviour
`cache`	`'default'`, `'no-store'`, `'reload'`, …	HTTP cache use (the browser's saved copies of earlier responses)
`keepalive`	`true`	Let a small request outlive the page (analytics — usage tracking — sent as the page closes)

Common mistakes

- Not checking `res.ok` → your code parses an error page, or treats a 500 as data.
- Calling `res.json()` on a non-JSON response → `SyntaxError: Unexpected token '<'` (an HTML error page) — Chapter 11 §11.10.
- Forgetting `Content-Type: application/json` when sending JSON — many servers won't parse the body.
- Building query strings by concatenation (`'?q=' + text`) — `&` or `#` in the text breaks the URL. Use `URLSearchParams` (§14.6).
- No timeout: a hung server leaves your UI spinning forever. Always pass a `signal`.
- Putting secrets (API keys — secret strings that identify your app to a paid service) in front-end code (code that runs in the user's browser) — anything shipped to the browser is public. Call your own backend (your server-side code), which holds the key.

In the interview

30-second answer: "`fetch` returns a promise for a `Response` that resolves when headers arrive, even for 404 or 500 — it only rejects on network errors or aborts, so you check `res.ok`. You read the body once with `json()` or `text()`. For sending JSON you set `Content-Type` and `JSON.stringify` the body. I always wrap it in a helper that throws on non-2xx and use `AbortSignal.timeout` so requests can't hang."

- *Q: Does `fetch` send cookies?* A: For same-origin requests, yes (default `credentials: 'same-origin'`); cross-origin only with `credentials: 'include'` and the right CORS headers.
- *Q: `fetch` vs axios?* A: axios (a popular HTTP-client library) adds conveniences (auto JSON, throw on non-2xx, interceptors — hooks that run on every request or response); `fetch` is built in and a small wrapper covers most needs.

TL;DR

`fetch` → `Response`; check `res.ok`; read the body once; HTTP errors resolve, network errors reject; send JSON with a header + `JSON.stringify`; always time out.

14.2 The same-origin policy and CORS

What it is

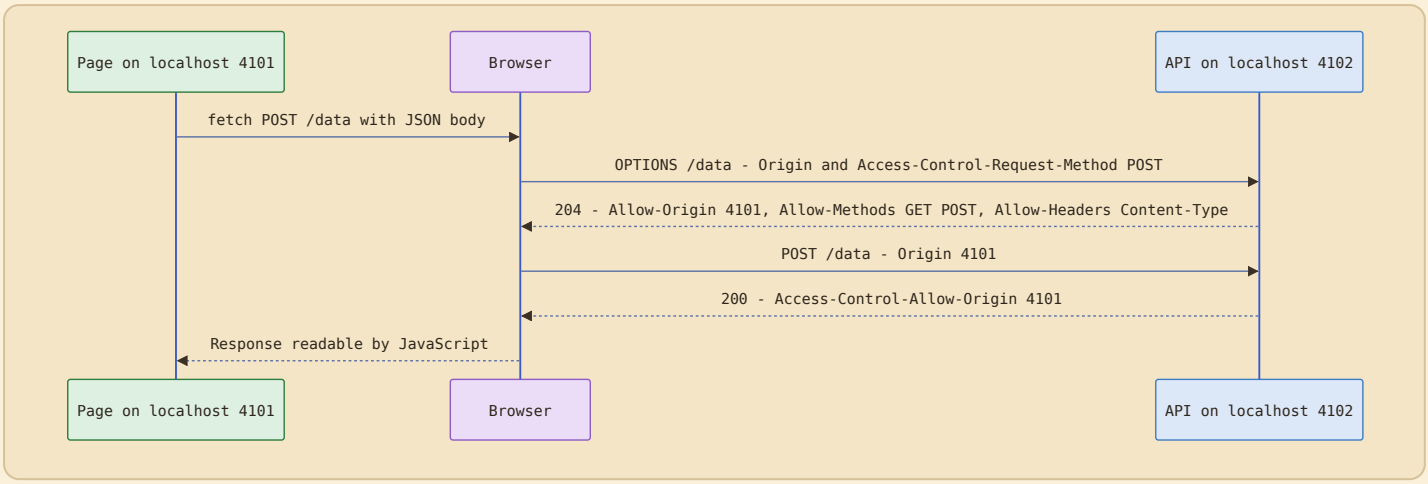
An **origin** is the triple **scheme + host + port** (scheme = `http` or `https`; host = the domain name; port = the number after `:`, 443 by default for HTTPS): `https://shop.example.com:443`. `http://localhost:4101` and `http://localhost:4102` are different origins (different port), as are `http://` vs `https://` and `a.example.com` vs `b.example.com`.

The **same-origin policy (SOP)** is the browser's core security rule: a page may **send** requests anywhere, but its JavaScript may **read responses** only from its own origin. Without it, any website you visit could `fetch('https://your-bank.example/account')` — your browser would attach your bank cookies — and read your balance.

CORS (Cross-Origin Resource Sharing) is the controlled exception: a server can declare, with response headers, which other origins may read its responses.

How it works

1. The page calls `fetch('http://localhost:4102/public')` from origin `http://localhost:4101`.
2. The browser adds an `Origin: http://localhost:4101` request header.
3. **Simple requests** (GET/HEAD/POST with only "safelisted" headers, and `Content-Type` limited to `text/plain`, `multipart/form-data` or `application/x-www-form-urlencoded`) are sent directly.
4. Anything else — `PUT`/`DELETE`, a JSON `Content-Type`, an `Authorization` header — first triggers a **preflight**: an automatic `OPTIONS` request asking "may I?" with `Access-Control-Request-Method`/`-Headers`. The real request is sent only if the answer allows it.
5. When the response arrives, the browser checks `Access-Control-Allow-Origin`. If it matches the page's origin (or is `*`), JavaScript gets the response; otherwise `fetch` rejects with a `TypeError` and the details go only to the DevTools console.



The CORS part of the API server in `code/10_browser_demo_server.mjs`:

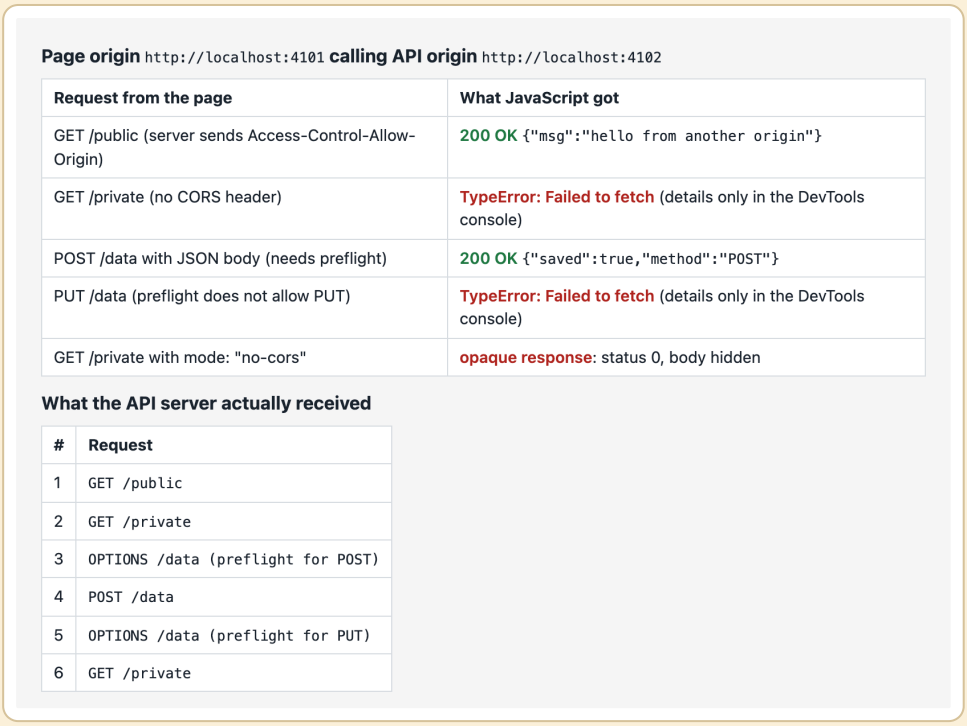
```
const allow = { 'Access-Control-Allow-Origin': SITE, Vary: 'Origin' };   // SITE = 'http://localhost:4101'
if (req.method === 'OPTIONS') {                          // preflight: allow POST with a JSON body, but not PUT
  return res.writeHead(204, { ...allow, 'Access-Control-Allow-Methods': 'GET, POST',
    'Access-Control-Allow-Headers': 'Content-Type', 'Access-Control-Max-Age': '600' }).end();
}
if (path === '/public') return json(200, { msg: 'hello from another origin' }, allow);
if (path === '/private') return json(200, { secret: 'no CORS header on this one' });   // browser will hide it
if (path === '/data') return json(200, { saved: true, method: req.method }, allow);
```

Run without arguments, the script inspects those headers with Node's `fetch` — which does **not** enforce CORS (only browsers do):

Output:

```
preflight -> 204 {
  'access-control-allow-headers': 'Content-Type',
  'access-control-allow-methods': 'GET, POST',
  'access-control-allow-origin': 'http://localhost:4101',
  'access-control-max-age': '600'
}
/public  ACAO: http://localhost:4101
/private ACAO: null | Node still reads it: no CORS header on this one
csp.html CSP : default-src 'self'; script-src 'self'; img-src 'self'; style-src 'self' 'unsafe-inline'
```

In Chrome, the page `code/cors.html` (served from `:4101`) makes five requests to `:4102`, then asks the API what it received:



Page origin `http://localhost:4101` **calling API origin** `http://localhost:4102`

Request from the page	What JavaScript got
GET /public (server sends Access-Control-Allow-Origin)	**200 OK** `{"msg":"hello from another origin"}`
GET /private (no CORS header)	**TypeError: Failed to fetch** (details only in the DevTools console)
POST /data with JSON body (needs preflight)	**200 OK** `{"saved":true,"method":"POST"}`
PUT /data (preflight does not allow PUT)	**TypeError: Failed to fetch** (details only in the DevTools console)
GET /private with mode: "no-cors"	**opaque response**: status 0, body hidden

What the API server actually received

#	Request
1	GET /public
2	GET /private
3	OPTIONS /data (preflight for POST)
4	POST /data
5	OPTIONS /data (preflight for PUT)
6	GET /private

Figure — CORS as enforced by Chrome, and what actually reached the server.

What the screenshot proves:

- `/private` **reached the server** (log line 2) — CORS doesn't stop requests, it stops the page **reading** the response. That's why CORS is not a defence against CSRF (§14.11).
- The JSON `POST` caused an automatic `OPTIONS` preflight first (lines 3–4).
- For `PUT`, the preflight said no, so the real `PUT` was **never sent** (line 5 has no follow-up).
- `mode: 'no-cors'` gives an **opaque** response: status 0, unreadable body — useful only for fire-and-forget requests.

CORS headers

Header (response)	Meaning
`Access-Control-Allow-Origin`	`*` or one exact origin (not a list — echo back an allow-listed (approved) `Origin` and add `Vary: Origin`, which tells caches the answer differs per origin)
`Access-Control-Allow-Methods` / `-Headers`	Preflight answer: allowed methods and request headers
`Access-Control-Allow-Credentials: true`	Allow cookies with `credentials: 'include'`; then `Allow-Origin` **can't** be `*`
`Access-Control-Max-Age`	Seconds the browser may cache the preflight answer
`Access-Control-Expose-Headers`	Non-standard response headers JS may read (e.g. `X-Total-Count`)

Common mistakes

- "Fixing" CORS in the front end — impossible; the **server** must send the headers (or proxy the API through your own origin — a proxy is a server that forwards requests on your behalf — e.g. the dev-server proxy in Vite, a popular front-end build tool).
- `Access-Control-Allow-Origin: *` plus credentials — browsers reject it.
- Reflecting any `Origin` back with credentials allowed — effectively disables the SOP for your API. Use an allow-list.
- Thinking a CORS error means the server is down — it usually means the server answered without the right headers.

In the interview

> **30-second answer:** "The same-origin policy lets a page send requests anywhere but only read responses from its own scheme-host-port. CORS lets a server opt in with `Access-Control-Allow-Origin`. Non-simple requests — JSON bodies, custom headers, PUT/DELETE — first get an OPTIONS preflight. CORS is enforced by the browser, not the server, and it protects reading, not sending."

- *Q: Why does Postman/curl (tools for sending HTTP requests by hand) work but the browser fails?* A: Only browsers enforce CORS.
- *Q: Is CORS a security feature for the server?* A: No, it's a relaxation of the browser's SOP; servers still need authentication (checking who the caller is) and CSRF protection.

TL;DR

Origin = scheme + host + port. SOP blocks reading cross-origin responses; CORS headers relax it per origin. Non-simple requests get an OPTIONS preflight. Browsers enforce it; the server configures it.

14.3 Web Storage: `localStorage` and `sessionStorage`

What it is

Two simple **key–value stores** (each value is saved and looked up under a name, its key) of **strings**, per origin, with a synchronous API (calls finish before returning, no promises):

- `localStorage` — persists until cleared (by your code or the user); shared by all tabs of the origin.
- `sessionStorage` — per **tab** (and origin); cleared when the tab closes; survives reloads.

API: `setItem(key, value)`, `getItem(key)` (→ `null` if missing), `removeItem(key)`, `clear()`, `key(i)`, `length`. Changes made in one tab fire a `storage` **event** in the **other** tabs of the same origin (not the one that wrote) — handy for syncing logout or theme across tabs.

`code/03_storage.mjs` — key parts:

```
localStorage.setItem('theme', 'dark');
localStorage.count = 5;                              // property style works but is discouraged
console.log('   numbers become strings:', typeof localStorage.getItem('count'),
JSON.stringify(localStorage.getItem('count')));
localStorage.setItem('user', { name: 'Ada' });       // WRONG: stores "[object Object]"

const store = {
  get(key, fallback = null) {
    try { const raw = localStorage.getItem(key); return raw === null ? fallback : JSON.parse(raw); }
    catch { return fallback; }                           // corrupt value or storage disabled
  },
  set(key, value) {
    try { localStorage.setItem(key, JSON.stringify(value)); return true; }
    catch (e) { console.log('   set failed:', e.name); return false; }  // QuotaExceededError, private mode...
  },
};
store.set('cart', { items: [{ sku: 'A-1', qty: 2 }], updated: '2026-09-28' });
// ... enumerate, removeItem
store.set('huge', 'x'.repeat(6_000_000));            // over the ~5 MB quota
sessionStorage.setItem('wizardStep', '2');
```

Output (lines 1–6):

```
1) getItem      : dark | missing -> null
   numbers become strings: string "5"
   object stored raw     : [object Object]
2) JSON round trip: { sku: 'A-1', qty: 2 } | corrupt -> fallback
3) length: 5 | keys: [ 'theme', 'count', 'user', 'cart', 'bad' ]
   after removeItem: 4
   set failed: QuotaExceededError
5) sessionStorage: 2 | separate from localStorage: null
6) after clear(): 0
```

Trade-offs

✓ Good for	✗ Not for
UI preferences (theme, collapsed panels), drafts, small caches	**Secrets and auth tokens** (strings proving a user is logged in) — any script on the page (including an XSS payload — the attacker's injected code — or a compromised npm package, i.e. a hacked library) can read it
Data that must survive reloads, a few KB–MB	Large data (quota ~5 MB per origin; use IndexedDB)
	Hot paths (code that runs very often) — it's **synchronous** and blocks the main thread on disk access
	Data for the server — it's never sent automatically (unlike cookies)

Common mistakes

- Storing objects without `JSON.stringify` → `"[object Object]"`.
- Not wrapping in `try`/`catch`: quota errors, and storage can be disabled (some privacy modes, sandboxed iframes — embedded pages with restricted permissions) so even *reading* can throw.
- Expecting `sessionStorage` to be shared across tabs — it isn't (a duplicated tab gets a copy).

TL;DR

String key–value storage per origin; `localStorage` persists, `sessionStorage` is per tab. JSON + try/catch; never store secrets.

14.4 Cookies and IndexedDB — choosing storage

Cookies

A **cookie** is a small (≈4 KB) name=value pair the browser stores per site and **sends automatically with every matching HTTP request** in the `Cookie` header. Servers set them with `Set-Cookie`; JavaScript can read/write non-HttpOnly cookies (ones without the `HttpOnly` flag, which hides a cookie from scripts) through the awkward `document.cookie` string.

```
document.cookie = 'lang=en; Path=/; Max-Age=86400; SameSite=Lax';
document.cookie = 'consent=yes; Path=/';
document.cookie = 'session=secret; HttpOnly';        // IGNORED: HttpOnly cookies can only be set by the server
console.log('7) document.cookie:', JSON.stringify(document.cookie));
const cookies = Object.fromEntries(document.cookie.split('; ').map((c) => c.split('=').map(decodeURIComponent)));
document.cookie = 'lang=; Path=/; Max-Age=0';         // delete = expire it
```

Output (end of `03_storage.mjs`):

```
7) document.cookie: "lang=en; consent=yes"
   parsed: { lang: 'en', consent: 'yes' }
   after delete: "consent=yes"
```

Assigning to `document.cookie` adds or updates **one** cookie; reading returns all visible ones as one string, without their attributes.

Attribute	Meaning
`Expires=<date>` / `Max-Age=<seconds>`	Lifetime; without either it's a **session cookie** (deleted when the browser session ends)
`Domain` / `Path`	Which hosts/paths receive it (omit `Domain` = this host only, the safest)
`Secure`	Only sent over HTTPS (encrypted HTTP) (browsers treat `localhost` as secure for development)
`HttpOnly`	Invisible to JavaScript — protects session cookies from theft by XSS
`SameSite=Strict` / `Lax` / `None`	Whether it's sent on **cross-site** requests (§14.11). `None` requires `Secure`. Chrome and Edge treat a missing attribute as `Lax`; set it explicitly
`__Host-` name prefix	Browser enforces `Secure`, `Path=/`, no `Domain` — hardest to tamper with

For session IDs (random strings the server uses to recognise a logged-in user) the standard recipe is: `Set-Cookie: __Host-sid=…; HttpOnly; Secure; SameSite=Lax; Path=/`. (A newer asynchronous `cookieStore` API exists; check browser support before using it.)

IndexedDB

IndexedDB is the browser's real database: asynchronous, transactional (a group of changes either all happen or none do), storing **structured-cloneable** values (ones the browser's built-in deep-copy algorithm can copy: objects, arrays, `Date`, `Map`, `Blob`, typed arrays) in **object stores** (roughly, tables) with **keys** and **indexes** (extra lookup structures for searching fast by a field), with quotas typically in the hundreds of MB to GB range. Its raw API is event-based and verbose, so most apps use a thin promise wrapper (like the small `req2p` below, or the `idb` library) or a higher-level library (Dexie).

The page `code/idb.html` — key parts:

```
const req2p = (req) => new Promise((resolve, reject) => { req.onsuccess = () => resolve(req.result); req.onerror =
() => reject(req.error); });

const open = indexedDB.open('notes-demo', 1);
open.onupgradeneeded = () => {                              // runs when the version number increases
  const store = open.result.createObjectStore('notes', { keyPath: 'id', autoIncrement: true });
  store.createIndex('byTag', 'tag');
};
const db = await req2p(open);

const tx = db.transaction('notes', 'readwrite');           // all writes commit together, or none
for (const n of notesToAdd) tx.objectStore('notes').add(n);
await new Promise((r) => (tx.oncomplete = r));

const ro = db.transaction('notes').objectStore('notes');   // read-only transaction
await req2p(ro.get(2));
await req2p(ro.index('byTag').getAll('home'));             // query by index
await req2p(ro.getAll(IDBKeyRange.bound(2, 3)));           // key range
```

IndexedDB: a notes database in the browser

```
upgradeneeded: created store "notes" + index "byTag"
added 3 notes (Dates are stored as real Date objects — structured clone)
count      : 3
get(2)     : Ship chapter 14 | created is a Date: true
tag="home" : Buy milk, Fix bike
ids 2..3   : 2:Ship chapter 14, 3:Fix bike
storage.estimate(): quota is about 11 GB for this origin (varies by browser and disk)
```

Figure — IndexedDB in headless Chrome. The quota comes from `navigator.storage.estimate()` and depends on the browser and free disk space.

Choosing

	Cookies	localStorage	sessionStorage	IndexedDB
Capacity	~4 KB each	~5 MB/origin	~5 MB/origin	Large (share of disk)
API	String; sync	Sync	Sync	Async, transactions
Data types	Strings	Strings	Strings	Structured clone (objects, Dates, Blobs…)
Sent to server	**Automatically**	No	No	No
Lifetime	Expiry / session	Until cleared	Tab	Until cleared (may be evicted — deleted by the browser — under storage pressure unless `navigator.storage.persist()` is granted)
Readable by JS	Unless `HttpOnly`	Yes	Yes	Yes
Typical use	Session ID, CSRF, consent	Preferences	Wizard progress	Offline data, caches, large datasets

A fifth option, the **Cache API**, stores whole HTTP responses; service workers (background scripts that sit between a page and the network) use it to make apps work offline (§14.8).

> **TL;DR**
>
> Cookies = tiny, auto-sent, the right place for `HttpOnly` session IDs. Web Storage = small, sync, strings. IndexedDB = async database for big or structured data.

14.5 Timers and `requestAnimationFrame`

What it is

`requestAnimationFrame(callback)` (**rAF**) asks the browser to run `callback(timestamp)` **just before the next repaint** — once per frame, matching the display's refresh rate (how many times per second the screen redraws; 60 Hz → every ~16.7 ms, 120 Hz → ~8.3 ms). It's the right scheduler for anything visual. Timers (`setTimeout`/`setInterval`, Chapter 12 §12.5) are for everything else.

	`setTimeout` / `setInterval`	`requestAnimationFrame`
When	After a delay, as a task	Right before paint, in the render step of the event loop
Synced to display	No (can run twice per frame or miss frames → jank, visible stutter)	Yes
Background tabs	Throttled (≥1 s typically)	**Paused** — saves battery
Callback argument	Your args	High-resolution frame timestamp
Cancel	`clearTimeout`	`cancelAnimationFrame(id)`

`code/06_raf.mjs` (jsdom simulates 60 fps):

```
function animate(el, to, duration) {
  return new Promise((resolve) => {
    let start;
    const frames = [];
    function step(now) {                        // now = frame timestamp (ms), same clock as performance.now()
      start ??= now;
      const t = Math.min((now - start) / duration, 1);
      const eased = 1 - (1 - t) ** 3;           // ease-out cubic
      el.style.width = `${Math.round(eased * to)}px`;
      frames.push(el.style.width);
      if (t < 1) requestAnimationFrame(step);
      else resolve(frames);
    }
    requestAnimationFrame(step);
  });
}
const frames = await animate(bar, 300, 200);

// Coalesce many events into one update per frame (e.g. scroll or pointermove)
let scheduled = false;
function onPointerMove() {
  events++;
  if (scheduled) return;
  scheduled = true;
  requestAnimationFrame(() => { scheduled = false; updates++; /* read layout, then write styles */ });
}
```

Output:

```
1) about 12 frames in 200 ms at 60 fps: true
   first widths: 0px, 69px, 130px, 178px ... last: 300px
2) cancelled a frame callback
3) 50 events -> 1 DOM update(s)
```

The intermediate widths depend on real frame timing, so they differ slightly from run to run; the first (0px) and last (300px) values do not.

Rules

1. **Base progress on time, not frame count** — frame rates vary (60/120/144 Hz, dropped frames). Compute `t = elapsed / duration` from the timestamp.
2. rAF runs **once**; call it again inside the callback to continue.
3. Use it to **coalesce** bursts of events (`scroll`, `pointermove`, `resize`) into one DOM update per frame — the third output line.
4. Prefer **CSS transitions/animations** or the Web Animations API (`el.animate()`) for simple effects; they can run on the compositor thread (a separate browser thread that stacks finished page layers onto the screen). Use rAF when JavaScript must compute each frame (games, canvas — an element you draw pixels on from JS — physics, drag).
5. Animate `transform`/`opacity` where possible (Chapter 13 §13.11).

> **TL;DR**
>
> rAF = "run before the next paint", once per frame, paused in background tabs. Time-based progress, re-request each frame, coalesce events through it.

14.6 `URL` and `URLSearchParams`

What it is

`new URL(input, base?)` parses a URL into parts; `url.searchParams` is a `URLSearchParams` for the query string (the `?key=value&…` part). Both handle **percent-encoding** (writing unsafe characters as `%` plus a hex code, e.g. a space as `%20`) correctly, so you never glue URLs together with `+`.

`code/04_url.mjs` — key parts:

```
const u = new URL('https://shop.example.com:8443/search/results?q=red+shoes&size=9&size=10#reviews');
console.log('1) parts:', { protocol: u.protocol, host: u.host, hostname: u.hostname, port: u.port, pathname:
u.pathname, search: u.search, hash: u.hash, origin: u.origin });
const p = u.searchParams;
p.set('page', '2'); p.append('size', '11'); p.delete('q'); p.sort();
console.log('3)', new URL('../img/logo.png', 'https://example.com/docs/guide/').href);
const search = new URLSearchParams({ q: 'Tom & Jerry', tag: 'c++', emoji: '✓' });

function safeHttpUrl(input) {
  try {
    const url = new URL(input);
    return ['http:', 'https:'].includes(url.protocol) ? url.href : null;
  } catch { return null; }
}
```

Output:

```
1) parts: {
  protocol: 'https:',
  host: 'shop.example.com:8443',
  hostname: 'shop.example.com',
  port: '8443',
  pathname: '/search/results',
  search: '?q=red+shoes&size=9&size=10',
  hash: '#reviews',
  origin: 'https://shop.example.com:8443'
}
2) q = red shoes | size = 9 | getAll = [ '9', '10' ] | has page: false
   now: ?page=2&size=9&size=10&size=11 | entries: [
  [ 'page', '2' ],
  [ 'size', '9' ],
  [ 'size', '10' ],
  [ 'size', '11' ]
]
3) https://example.com/docs/img/logo.png
   https://example.com/api?x=1
4) q=Tom+%26+Jerry&tag=c%2B%2B&emoji=%E2%9C%93
   manual concat would break: q=Tom & Jerry <- the & starts a new parameter
   encodeURIComponent: Tom%20%26%20Jerry%2F%3F | encodeURI keeps / and ?: /a%20b/?x=1&y=2
5) https://ada.dev          -> https://ada.dev/
5) javascript:alert(1)      -> null
5) not a url                -> null
5) data:text/html,<b>x</b>  -> null
   URL.canParse: true false
6) { a: '1', b: 'two' } | k=v1&k=v2
```

Notes

- `new URL('relative')` **without a base throws** `TypeError: Invalid URL`; `URL.canParse()` checks without throwing.
- `URLSearchParams` encodes spaces as `+` (form encoding); `encodeURIComponent` uses `%20`. Both decode correctly on servers.
- `get` returns the first value; `getAll` all of them — repeated keys are normal (`?size=9&size=10`).
- In the browser, `location` is URL-like (`location.pathname`, `location.search`), and `new URL(location.href)` gives you a mutable (changeable) copy.
- **Validate user-supplied URLs** by parsing and allow-listing the protocol (item 5). Blocking `javascript:` URLs prevents a classic XSS (§14.10).

TL;DR

Parse with `new URL`, build queries with `URLSearchParams`, resolve relative URLs with a base, and allow-list protocols for user links.

14.7 The History API

What it is

The **History API** lets a page change the URL in the address bar and add entries to the back/forward list **without loading a new page** — the foundation of **single-page application (SPA)** routing — an SPA loads one HTML page and swaps views with JavaScript; *routing* maps each URL to a view (React Router, Vue Router).

API	Does
`history.pushState(state, '', url)`	Add a new entry and change the URL (same origin only). No request, no `popstate`
`history.replaceState(state, '', url)`	Change the current entry (no new entry) — e.g. update query params for filters
`history.state`	The state object of the current entry (structured-cloned)
`history.back()` / `forward()` / `go(n)`	Navigate; async
`popstate` event on `window`	Fires when the user (or code) moves back/forward to an entry — **re-render from `location`**

`code/05_history.mjs` — a 30-line router:

```
const routes = [
  [/^\/$/, () => 'Home page'],
  [/^\/products$/, () => 'All products'],
  [/^\/products\/(\d+)$/, (id) => `Product #${id}`],
];
function render() { /* match location.pathname against routes, write the view with textContent */ }

// Intercept same-origin link clicks (delegation) and navigate without reloading
document.addEventListener('click', (e) => {
  const a = e.target.closest('a');
  if (!a || a.origin !== location.origin || e.ctrlKey || e.metaKey) return;   // let the browser handle it
  e.preventDefault();
  history.pushState({ from: location.pathname }, '', a.href);                  // new history entry, no request
  render();
});
// Back/forward buttons fire popstate: re-render from the URL
window.addEventListener('popstate', () => { render(); show('popstate (back/fwd)'); });

render(); show('initial');
document.querySelector('a[href="/products"]').click(); show('click Products');
document.querySelector('a[href="/products/42"]').click(); show('click Lamp');
history.replaceState({ tab: 'reviews' }, '', '/products/42?tab=reviews'); show('replaceState (no new entry)');
history.back();
```

Output:

```
initial                     /             | Home page | state: null | length: 1
click Products              /products     | All products | state: {"from":"/"} | length: 2
click Lamp                  /products/42  | Product #42 | state: {"from":"/products"} | length: 3
replaceState (no new entry) /products/42  | Product #42 | state: {"tab":"reviews"} | length: 3
popstate (back/fwd)         /products     | All products | state: {"from":"/"} | length: 3
external link: same origin? false -> not intercepted
```

Must-knows for SPA routing

1. **The server must serve your app for every route** (a "fallback" to `index.html`) — otherwise a reload or shared link to `/products/42` gets a 404.
2. Respect modifier keys (Ctrl, Cmd, Shift — e.g. Ctrl+click opens a new tab), `target="_blank"` and external links (let the browser do its normal thing).
3. Manage **focus and scroll** after navigation (move focus to the new heading; restore scroll on back) — screen-reader users otherwise don't notice the page changed.
4. Put UI state that should be shareable (filters, tabs, search) in the URL (`replaceState` + `URLSearchParams`), not only in memory.
5. Hash routing (`#/products`) + the `hashchange` event is the older approach that needs no server config.

A newer **Navigation API** (`navigation.navigate()`, a single `navigate` event for all navigations) simplifies routers; check browser support before relying on it.

TL;DR

`pushState` / `replaceState` change the URL without loading; `popstate` fires on back/forward — re-render from `location`. Configure the server fallback; handle focus.

14.8 Web Workers

What it is

A **Web Worker** runs a script on a **separate background thread** with its own event loop and global scope (`self` — its top-level object, like `window` on a page). It has no access to the DOM, `window` or `document`, but it has `fetch`, timers, IndexedDB, `crypto` (built-in cryptography: hashing, random numbers, encryption) and more. The page and the worker communicate only by **messages**:

```
// main page
const w = new Worker('prime_worker.js');
w.onmessage = (e) => show(e.data.count);
w.postMessage(5_000_000);
```

```
// prime_worker.js
self.onmessage = (e) => {
  const count = countPrimes(e.data);
  self.postMessage({ upTo: e.data, count });
};
```

Messages are copied with the **structured clone algorithm** (plain objects, arrays, `Date`, `Map`, `Set`, `Blob`, typed arrays… but **not** functions, DOM nodes or class prototypes). Big binary buffers (blocks of raw bytes, `ArrayBuffer`) can be **transferred** instead of copied (`postMessage(buf, [buf])` — the sender loses access, zero-copy: ownership moves, no bytes are duplicated).

Why it matters — measured

`code/worker.html` counts primes up to 5 million twice: once on the main thread, once in `code/prime_worker.js`. A 10 ms `setInterval` tick counter measures whether the page could do anything else meanwhile:

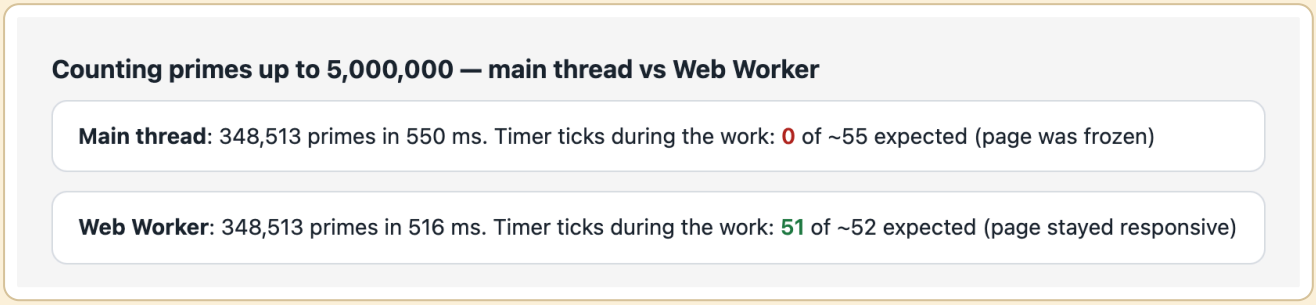


Figure — same work, same time; only the worker leaves the main thread free (headless Chrome).

The worker doesn't make the computation faster — it moves it off the thread that handles clicks, scrolling and painting. Node has the same model in `worker_threads`; `code/07_workers_node.mjs` shows the message passing and structured-clone rules:

```
const worker = new Worker(workerSource, { eval: true });
const sample = { when: new Date(0), tags: new Map([['a', 1]]) };   // Date and Map survive structured cloning
const ticker = setInterval(() => ticks.push('tick'), 5);             // the main thread stays responsive
const result = await new Promise((resolve, reject) => {
  worker.once('message', resolve);
  worker.once('error', reject);
  worker.postMessage({ upTo: 3_000_000, sample });
});
try { worker.postMessage({ fn: () => 1 }); } catch (e) { console.log('clone error   :', e.name); }
```

Output:

```
worker result : { upTo: 3000000, count: 216816, sampleIsCopy: true, mapSize: 1 }
main thread kept running while the worker computed: true
clone error   : DataCloneError
structuredClone keeps Date/Map: true
```

Kinds of workers

Kind	Purpose
Dedicated `Worker`	Background computation for one page (this section). `new Worker(url, { type: 'module' })` allows `import`
`SharedWorker`	One worker shared by several tabs of the same origin
Service worker	A network proxy between the page and the server — offline support, caching, push notifications (PWAs — progressive web apps, websites that can be installed and work offline like native apps). Event-driven, not for computation
Worklets	Tiny specialized scripts (audio, CSS paint)

When not to use it

For I/O (`fetch` is already asynchronous) or small tasks — message copying and worker start-up cost more than they save. Use it for parsing big files, image/audio processing, compression, cryptography, syntax highlighting, search indexes (precomputed lookup tables for fast text search). Libraries like Comlink make the message passing feel like function calls.

TL;DR

Workers = real parallel threads without DOM access, talking via `postMessage` (structured clone or transfer). Use them to keep CPU-heavy work off the main thread.

14.9 `IntersectionObserver` and `ResizeObserver`

What it is

Observers are callbacks the browser calls **when something changes**, instead of you polling in `scroll` or `resize` handlers (which fire constantly and force layout reads).

Observer	Tells you when…	Typical uses
`IntersectionObserver`	An element enters/leaves the viewport (the visible part of the page) (or a scroll container), by a given fraction	Lazy loading (loading things only when about to be seen), infinite scroll, "seen" analytics, starting animations
`ResizeObserver`	An element's size changes (for any reason, not just window resize)	Responsive components (that adapt their layout to available space), charts that redraw to their container
`MutationObserver`	The DOM changes	Chapter 13 §13.10

`code/observers.html` — key parts:

```
const io = new IntersectionObserver((entries, obs) => {
  for (const e of entries) {
    if (!e.isIntersecting) continue;
    e.target.classList.add('loaded');
    e.target.textContent = 'item ' + e.target.dataset.i + ' loaded';
    obs.unobserve(e.target);                    // load once
  }
}, { root: scroller, rootMargin: '20px' });     // root: null would mean the viewport
scroller.querySelectorAll('.item').forEach((el) => io.observe(el));

const ro = new ResizeObserver((entries) => {
  for (const e of entries) {
    const { inlineSize, blockSize } = e.contentBoxSize[0];
    log('RO callback: content box ' + Math.round(inlineSize) + ' x ' + Math.round(blockSize) + ' px');
  }
});
ro.observe(box);
// scripted: scrollTop = 300, then 600; then box.style.width = '240px'
```

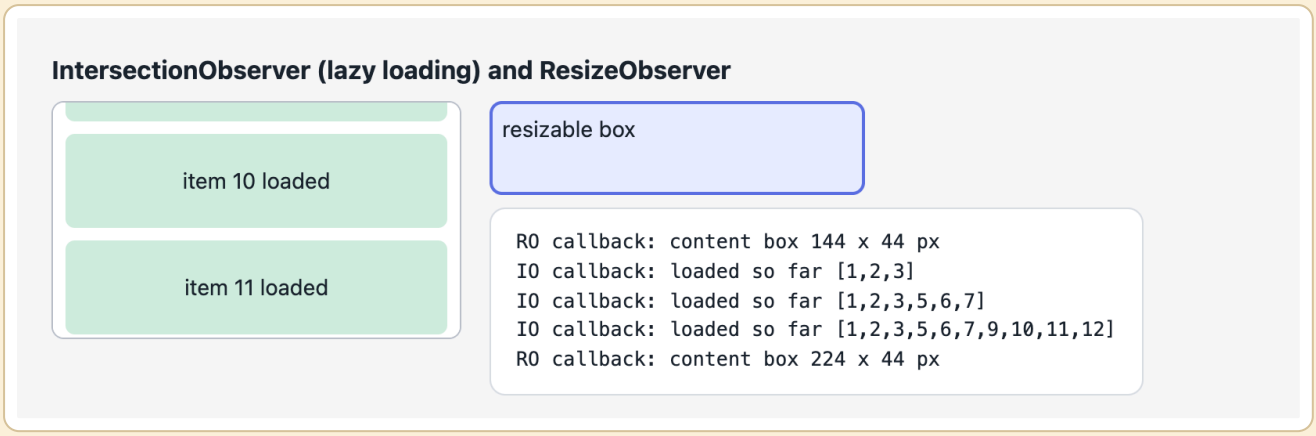


Figure — observers in headless Chrome. Items 4 and 8 were never loaded: the two programmatic scroll jumps skipped past them, exactly as a fast user scroll would.

Notes:

- Both observers call back **once initially** for each observed element (the first RO and IO lines), then on changes.
- Options: `root` (container or `null` = viewport), `rootMargin` (grow/shrink the root box, e.g. `'200px'` to start loading early), `threshold` (`0` = any pixel, `1` = fully visible, or an array).
- For images and iframes, prefer the native `loading="lazy"` attribute; use IO for custom cases.
- Always `unobserve` / `disconnect` when done (e.g. in a component's cleanup).
- A `ResizeObserver` callback that changes the observed element's size can loop; the browser stops after one pass and reports "ResizeObserver loop completed with undelivered notifications".

TL;DR

`IntersectionObserver` for visibility, `ResizeObserver` for element size — no scroll/resize polling, no forced layouts. Disconnect when done.

14.10 Cross-site scripting (XSS) and sanitizing

What it is

Cross-site scripting (XSS) happens when an attacker gets **their JavaScript to run in your page**, with your page's privileges: reading data shown to the user, calling your APIs as the user, reading non-HttpOnly cookies and `localStorage`, changing the page (fake login forms). It has appeared in every edition of the OWASP Top 10 (a widely used ranking of the most critical web-application security risks, published by the OWASP non-profit) — as its own category until 2017, and folded into "Injection" since the 2021 edition.

Type	How the payload arrives
Stored	Saved on the server (a comment, a profile name) and shown to other users
Reflected	Echoed from the request (a search term in the URL) into the response
DOM-based	Client-side code moves untrusted data (URL, `postMessage`, API data) into a dangerous **sink** like `innerHTML`

How it works — seen in a real browser

`code/xss.html` renders the same "user comment" twice. The comment contains `<img src="does-not-exist.png" onerror="…">`: the image fails to load, so the browser runs the `onerror` attribute as code.

```
document.getElementById('safe').textContent = comment;   // shown as text
document.getElementById('unsafe').innerHTML = comment;   // parsed: the onerror handler runs
```

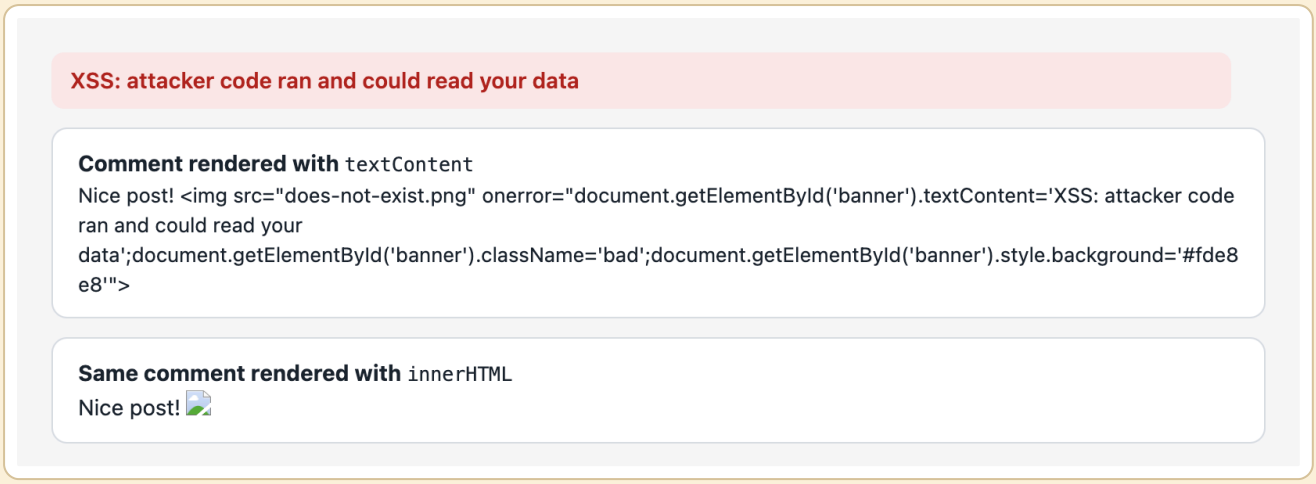


Figure — one line of `innerHTML` was enough for the attacker's code to take over the page (headless Chrome).

`code/08_xss.mjs` takes the same payload apart in jsdom:

```
box.innerHTML = comment;
console.log('1) innerHTML created <img> with onerror handler:', img?.getAttribute('onerror'));
box.textContent = comment;
const escapeHtml = (s) => String(s).replace(/[&<>"']/g, (ch) => ({ '&': '&amp;', '<': '&lt;', '>': '&gt;', '"': '&quot;', "'": '&#39;' })[ch]);
box.innerHTML = `<p class="comment">${escapeHtml(comment)}</p>`;
const website = 'javascript:alert(document.cookie)';
a.href = escapeHtml(website);                        // escaping does nothing useful here
```

Output:

```
1) innerHTML created <img> with onerror handler: stealCookies()
   <script> inserted via innerHTML exists but never runs: true (the <img onerror> is what attackers use instead)
2) textContent -> elements created: 0 | shown literally: Nice post! <img src="x...
3) escaped: &lt;img src=x onerror=&quot;go()&quot;&gt;
   elements inside <p>: 0
4) href after escaping : javascript:alert(document.cookie) <- still runs script when clicked
   href after allow-list: #
5) sinks to avoid with untrusted data:
   - el.innerHTML = s
   - el.outerHTML = s
   - el.insertAdjacentHTML(pos, s)
   - document.write(s)
   - eval(s) / new Function(s)
   - setTimeout('code string')
   - el.setAttribute("on...", s)
   - a.href = s (javascript:)
```

How to prevent it (in priority order)

1. **Treat all external data as text.** Use `textContent`, `setAttribute` for non-event attributes, and `createElement` — the DOM APIs from Chapter 13 §13.3. Frameworks do this by default: React escapes `{value}` in JSX (React's HTML-like syntax inside JavaScript); the danger is in escape hatches (deliberate ways to bypass a safety feature) (`dangerouslySetInnerHTML`, Vue's `v-html`).
2. **Escape for the context** (escaping = rewriting special characters, like `<` → `<`, so they're shown, not interpreted) when you build HTML strings (server templates): HTML body, attribute, URL and JavaScript contexts each need different encoding (item 4: HTML-escaping doesn't make a `javascript:` URL safe). Use a template engine (a library that fills HTML templates with data) that auto-escapes.
3. **Validate URLs** by parsing and allow-listing `http:`/`https:` (§14.6).
4. **Sanitize** (strip out every dangerous part) **when you must render user HTML** (rich-text comments, Markdown output — Markdown is a lightweight text format that converts to HTML): run it through a well-maintained sanitizer such as **DOMPurify** (`el.innerHTML = DOMPurify.sanitize(html)`), which removes scripts, event-handler attributes and dangerous URLs. The standard **HTML Sanitizer API** (`el.setHTML()`) is still of limited availability across browsers (per MDN, Mozilla's web documentation), so use it only with a fallback. Never write your own regex-based sanitizer.
5. **Defence in depth**: a strict **CSP** (§14.12) so injected code doesn't run even if a bug slips through; `HttpOnly` cookies so sessions can't be stolen; **Trusted Types** (a CSP feature that makes dangerous sinks reject plain strings; check browser support).

In the interview

30-second answer: "XSS is attacker-controlled script running in my origin — stored, reflected or DOM-based. The root cause is untrusted data reaching a sink that parses HTML or code, like `innerHTML`, `eval` or a `javascript:` URL. I prevent it by rendering data as text (`textContent`, framework escaping), context-aware escaping on the server, URL protocol allow-lists, DOMPurify when HTML is truly needed, and a strict CSP plus HttpOnly cookies as backup."

- *Q: Does a `<script>` injected with `innerHTML` run?* A: No — but `<img onerror>`, `<svg onload>` and similar event-handler attributes do, so that's no protection.
- *Q: Is React immune?* A: Mostly for text; not for `dangerouslySetInnerHTML`, `href={userUrl}` with `javascript:` (React warns and newer versions block it), or server-rendered HTML you inject.

TL;DR

XSS = untrusted data parsed as code. Render as text, escape by context, allow-list URL protocols, sanitize with DOMPurify, add CSP and HttpOnly.

14.11 Cross-site request forgery (CSRF)

What it is

CSRF tricks a logged-in user's browser into sending a request they didn't intend — for example, a hidden form on `evil.example` that auto-submits `POST https://bank.example/transfer`. Unless the cookie is protected by `SameSite`, the browser attaches the bank's cookies automatically, so the bank sees a valid session. The attacker can't **read** the response (SOP), but for a state-changing request (one that modifies data, like a transfer) they don't need to.

How it's prevented

Defence	How it helps
`SameSite` cookies (`Lax` or `Strict`)	The browser doesn't send the cookie on cross-site sub-requests and form POSTs. `Lax` still sends it on top-level GET navigations (clicking a link), so **GET must never change state**
CSRF token (synchronizer token)	The server gives the page a random secret (in the HTML or a JSON response); state-changing requests must echo it in a header or form field. `evil.example` can't read your page, so it can't know the token
Check `Origin` / `Sec-Fetch-Site` headers	Browsers send the request's origin; reject unexpected ones on state-changing requests
Require a custom header or JSON `Content-Type`	Forces a CORS preflight for cross-origin requests, which your server won't approve
Re-authentication	For critical actions (change password, large transfer)

Use at least two: `SameSite` + token (or origin check). `code/09_csrf_cookies_headers.mjs` implements session cookies + a token on a local server; since Node's `fetch` has no cookie jar (storage that keeps cookies between requests), the script plays the browser and sends the `Cookie` header by hand:

```
if (req.method === 'POST' && req.url === '/login') {
  const sid = randomBytes(16).toString('hex');
  const csrf = randomBytes(16).toString('hex');
  sessions.set(sid, { user: 'ada', csrf });
  return reply(200, { csrf }, {                          // token goes to the page, cookie to the browser
    'Set-Cookie': `sid=${sid}; HttpOnly; Secure; SameSite=Lax; Path=/; Max-Age=3600`,
  });
}
if (req.method === 'POST' && req.url === '/transfer') {
  if (!session) return reply(401, { error: 'not logged in' });
  if (!same(req.headers['x-csrf-token'], session.csrf)) return reply(403, { error: 'CSRF token missing or wrong'
});
  return reply(200, { ok: true, by: session.user });
}
// ...
const forged = await fetch(`${BASE}/transfer`, { method: 'POST', headers: { Cookie: cookie } });
const legit = await fetch(`${BASE}/transfer`, { method: 'POST', headers: { Cookie: cookie, 'X-CSRF-Token': csrf }
});
const noCookie = await fetch(`${BASE}/transfer`, { method: 'POST', headers: { 'X-CSRF-Token': csrf } });
```

Output:

```
1) Set-Cookie: sid=<random>; HttpOnly; Secure; SameSite=Lax; Path=/; Max-Age=3600
   CSP       : default-src 'self'; script-src 'self'; object-src 'none'; base-uri 'self'; frame-ancestors 'none'
2) forged request  -> 403 CSRF token missing or wrong
3) legit request   -> 200 { ok: true, by: 'ada' }
4) without cookie  -> 401 not logged in
```

Line 2 is a forged request that still carried the cookie (as it would with `SameSite=None`, or on an old browser): the missing token stops it. Line 4 is what `SameSite=Lax` does to a cross-site POST in a modern browser: no cookie, so no session. The token is compared with `crypto.timingSafeEqual` so response timing doesn't leak how many characters matched.

Common mistakes

- Changing state on `GET` (`/logout`, `/delete?id=5`) — an `<img src>` can trigger it.
- Believing CORS prevents CSRF — CORS controls reading; the forged request is still sent (the CORS screenshot in §14.2 shows `/private` reaching the server).
- Storing the session token in `localStorage` and sending it in an `Authorization` header avoids CSRF but makes the token readable by any XSS — a trade-off, not a free win. `HttpOnly` cookie + `SameSite` + CSRF token is the common recommendation (OWASP CSRF Prevention Cheat Sheet).

TL;DR

CSRF abuses automatically-sent cookies. Defend with `SameSite` cookies, CSRF tokens (or origin checks), and never change state on GET.

14.12 Content Security Policy (CSP)

What it is

CSP is an HTTP response header (`Content-Security-Policy`) that tells the browser **which sources of scripts, styles, images, frames etc. are allowed** on this page. Anything else is blocked and reported. It's **defence in depth**: if an XSS bug lets an attacker inject markup, CSP stops the injected code from running.

Common directives:

Directive	Controls	Example
`default-src`	Fallback for all fetch directives	`'self'`
`script-src`	Scripts (`'unsafe-inline'` would re-allow inline code — avoid; a *nonce* is a random one-time token)	`'self'` or `'nonce-r4nd0m'`
`script-src-attr`	Inline event-handler attributes (`onclick=…`)	falls back to `script-src`
`style-src`, `img-src`, `connect-src` (fetch/WebSocket — a persistent two-way connection), `font-src`, `frame-src`	Each resource type	`connect-src 'self' https://api.example.com`
`object-src 'none'`	Plugins (`<object>`, `<embed>`)	Always `'none'`
`base-uri 'self'`	Stops `<base>` tag hijacking relative URLs	
`frame-ancestors 'none'`	Who may embed you in an iframe — **clickjacking** (hiding your page in an invisible frame to trick clicks) protection (supersedes the older `X-Frame-Options` header)	
`report-to` / `report-uri`	Where to send violation reports	

Seen in a real browser

`code/csp.html` is served by the demo server with `Content-Security-Policy: default-src 'self'; script-src 'self'; img-src 'self'; style-src 'self' 'unsafe-inline'`. It contains an inline `<script>` that would replace a message, and loads `code/csp_app.js`, which injects an attacker's comment through `innerHTML` and logs `securitypolicyviolation` events:

```
// csp_app.js (allowed: same origin)
document.addEventListener('securitypolicyviolation', (e) => {
  log.textContent += `blocked ${e.effectiveDirective.padEnd(15)} ${e.blockedURI || '(inline)'}\n`;
});
document.getElementById('comment').innerHTML =
  'Great article! <img src="https://evil.example/pixel.png" onerror="document.body.style.background=\'red\'">';
```

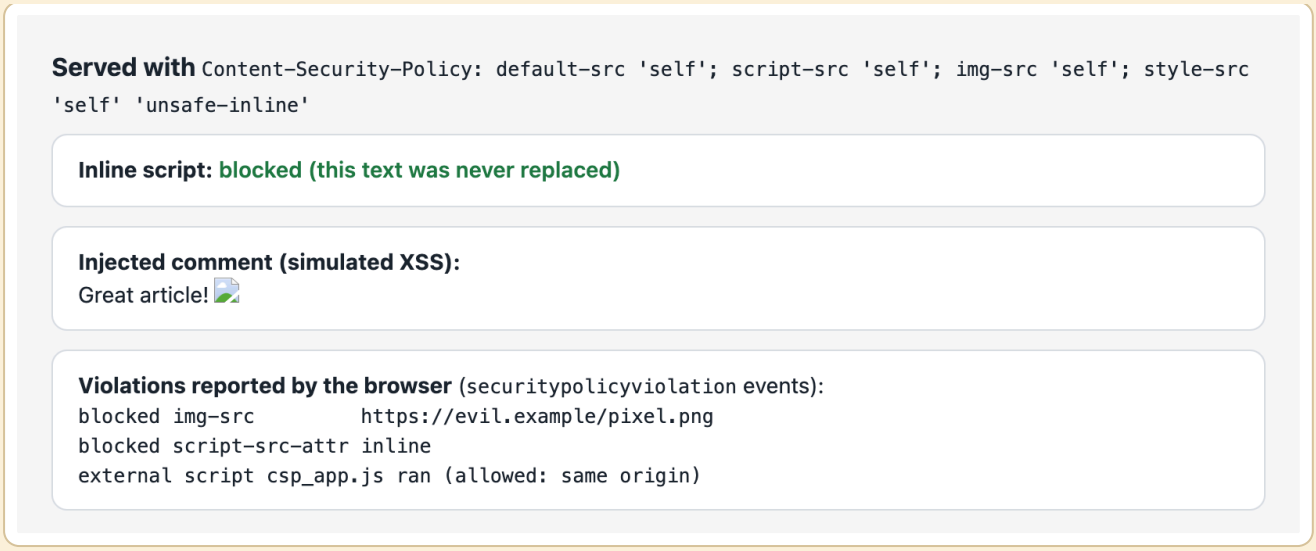


Figure — the same kind of injection that took over the page in §14.10 is neutralized: the external image and the inline `onerror` handler were both blocked (headless Chrome).

(The inline-script violation isn't in the log because it happened before `csp_app.js` registered its listener; it appears in the DevTools console.)

How to adopt CSP

1. Move inline scripts and `on…=` attributes into external files and `addEventListener` (good practice anyway — Chapter 13 §13.5).
2. Start with `Content-Security-Policy-Report-Only` to collect violations without breaking the site.
3. For pages that need inline scripts (e.g. server-rendered data), use a per-response random **nonce** (`script-src 'nonce-…'`) — with `'strict-dynamic'` scripts loaded by trusted scripts are allowed too. This "strict CSP" is what Google recommends over long domain allow-lists, which are often bypassable.
4. Enforce, keep reporting, and review violations.

Other security headers worth sending

Header	Purpose
`Strict-Transport-Security: max-age=31536000; includeSubDomains`	HTTPS only (HSTS: HTTP Strict Transport Security)
`X-Content-Type-Options: nosniff`	Don't guess MIME types (labels like `text/html` naming a file's format; stops some script injection via uploads)
`Referrer-Policy: strict-origin-when-cross-origin`	Don't leak full URLs (the *referrer*, the page a request came from) to other sites (this is also the browser default today)
`Permissions-Policy: camera=(), geolocation=()`	Turn off powerful features you don't use
`Cross-Origin-Opener-Policy: same-origin`	Isolate your window from cross-origin popups

Security checklist for front-end code

Risk	Rule
XSS	Data as text; escape by context; DOMPurify for HTML; allow-list URL protocols; strict CSP
Session theft	`HttpOnly; Secure; SameSite` cookies; never tokens in URLs
CSRF	SameSite + CSRF token or origin check; no state change on GET
Secrets	No API keys or secrets in front-end bundles
Dependencies	`npm audit` (scans installed packages for known vulnerabilities), lockfiles (files pinning exact dependency versions), few and well-maintained packages (Chapter 10); Subresource Integrity (`integrity=`, a hash the browser checks so a tampered file is refused) for third-party CDN (content delivery network) scripts
Clickjacking	`frame-ancestors 'none'` or `'self'`
Validation	Client-side validation is UX; the server validates everything
Prototype pollution (attacker data adding properties to `Object.prototype`, affecting every object)	Don't deep-merge untrusted JSON into objects (Chapter 9)

TL;DR

CSP limits where scripts/styles/images may come from and blocks inline code, so injected markup can't execute. Roll out in report-only mode, prefer nonces + `'strict-dynamic'`, add HSTS, `nosniff` and `frame-ancestors`.

Chapter 14 TL;DR

- **fetch**: resolves on any HTTP status (check `res.ok`), rejects on network failure/abort; read the body once; JSON needs `Content-Type` + `JSON.stringify`; time out with `AbortSignal.timeout`.
- **SOP & CORS**: origin = scheme + host + port. Pages can send anywhere but read only same-origin responses unless the server sends `Access-Control-Allow-Origin`. Non-simple requests get an `OPTIONS` preflight. Browsers enforce it.
- **Storage**: cookies (tiny, auto-sent; `HttpOnly; Secure; SameSite` for sessions), `localStorage` (persistent strings, ~5 MB, sync, never secrets), `sessionStorage` (per tab), IndexedDB (async database, structured data, large quota).
- **rAF** for visual updates: once per frame, time-based progress, paused in background tabs.
- **URL/URLSearchParams** for parsing, building, encoding and validating URLs.
- **History API**: `pushState`/`replaceState` + `popstate` = SPA routing; server fallback to `index.html`.
- **Web Workers**: parallel threads without DOM; `postMessage` with structured clone; keep CPU-heavy work off the main thread.
- **Observers**: `IntersectionObserver` (visibility, lazy loading), `ResizeObserver` (element size).
- **XSS**: data as text, context-aware escaping, URL allow-lists, DOMPurify, CSP, HttpOnly.
- **CSRF**: SameSite cookies + CSRF tokens/origin checks; no state changes on GET; CORS is not a CSRF defence.
- **CSP** and security headers are defence in depth.

Next: JavaScript on the server — Chapter 15: Node.js Basics.

Chapter 15 — Node.js Basics

Until now most of this book has treated JavaScript as a language, with the browser as its natural home. **Node.js** takes the same language out of the browser and puts it on servers, laptops and build machines — every `npm install` (npm is Node's package manager, the tool that downloads libraries), every bundler run (a bundler merges many source files into a few for the browser), every `node --test` in this book already used it. This chapter explains what Node adds on top of the language and how to use its core modules (the modules built into Node itself) to do real work: files, paths, configuration, events, streams, HTTP, and child processes.

What you'll learn

1. What Node.js is: V8 + libuv + core modules, and how it differs from a browser.
2. Where the event loop and the libuv thread pool fit, why blocking hurts, and how worker threads help.
3. Modules in Node in one page (the details are in Chapter 10).
4. Files with `node:fs/promises`, paths and file URLs with `node:path` / `node:url`.
5. `process`: arguments, environment variables, `.env` files, exit codes, signals.
6. `Buffer` — raw bytes.
7. `EventEmitter` — the event pattern behind most Node APIs.
8. Streams: readable, writable, transform, `pipeline`, backpressure.
9. Building an HTTP server with `node:http` and calling it from the same script.
10. Child processes and npm scripts (`node --run`).
11. Node vs Deno vs Bun, and the everyday command-line flags (options such as `--watch` typed after a command).

Before you start. You should know promises and `async`/`await` (Chapter 12) — almost every Node API in this chapter is used with `await`. Node's event-loop phases, `process.nextTick` and `setImmediate` are already explained in §12.4; module systems (ESM vs CommonJS, `package.json`, npm) in Chapter 10; the Node inspector in §11.9. This chapter links to them rather than repeating them.

All runnable code is in `code/`. Every program was run with **Node 24.11.1** and the output shown is real. Programs that touch the disk use a fresh temporary folder (`os.tmpdir()`) and delete it at the end; the HTTP example listens only on `127.0.0.1` on a free port chosen by the operating system, so nothing leaves your machine.

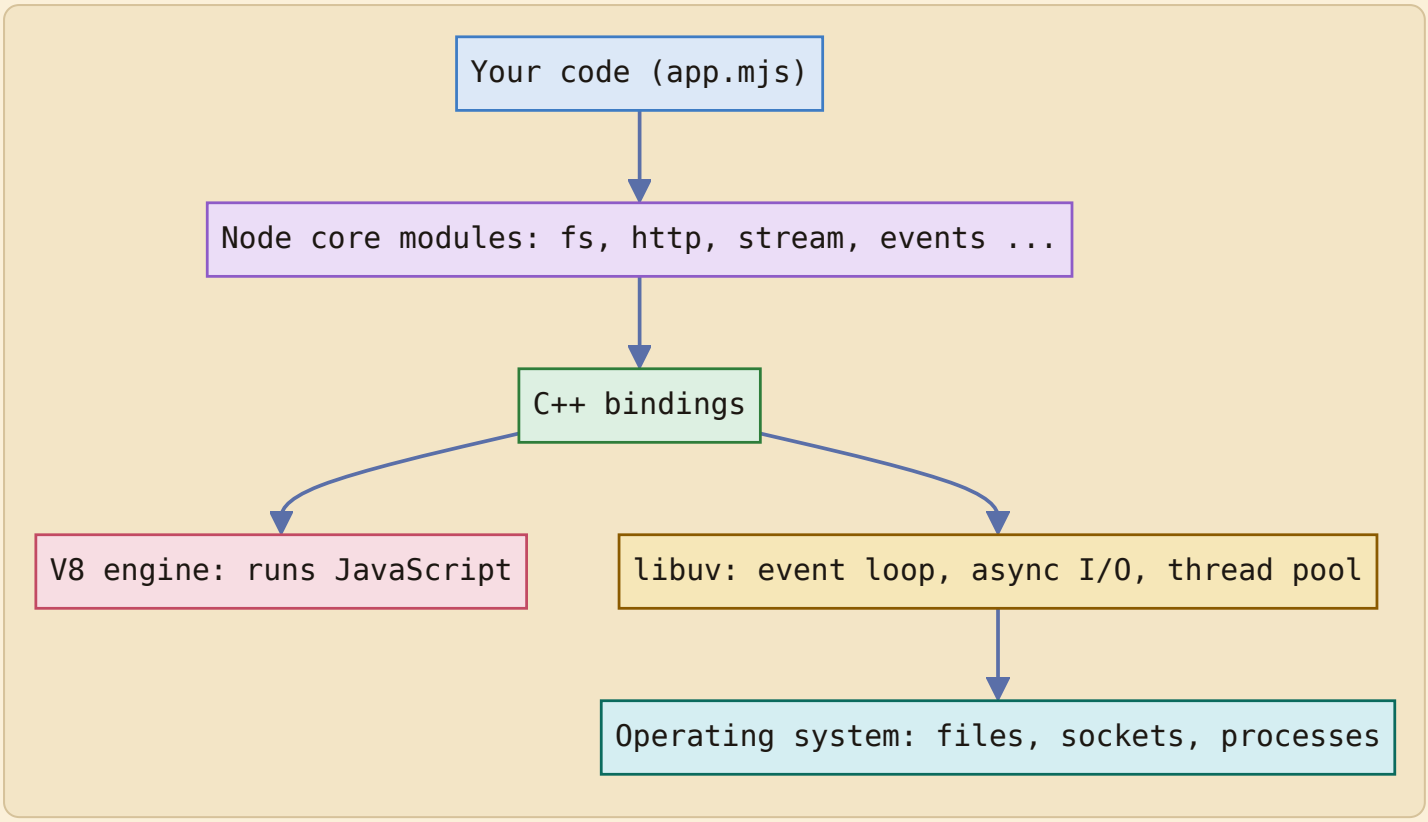
15.1 What Node.js is

What it is

Node.js is a **JavaScript runtime**: a program that runs JavaScript outside a browser. It was created by Ryan Dahl in 2009. It combines three things:

Part	What it does	Written in
V8	Google's JavaScript engine (the same one in Chrome) — parses, compiles and runs your JS (§1.3).	C++
libuv	A C library that gives Node its **event loop**, asynchronous file and network I/O (input/output), timers, a **thread pool** (a fixed set of helper threads for work that would otherwise block), and child processes (other programs started from Node) — on Linux, macOS and Windows alike.	C
Core modules + bindings	The JavaScript APIs you import (`node:fs`, `node:http`, `node:stream` …) and the C++ glue ("bindings") that connects them to V8 and libuv.	JS + C++

Analogy: V8 is the engine of a car; libuv is the gearbox and wheels that connect the engine to the road (the operating system); core modules are the dashboard controls you actually touch.



Why it matters

- **One language on both sides** — front-end (code running in the browser) and back-end (code running on the server) share code, types and validation.
- **Great at I/O-heavy work** — one process juggles thousands of connections because it never waits idly for a disk or socket (a program's end of a network connection); it registers a callback and moves on (Chapter 12).
- **It runs the JavaScript tool chain** — npm, Vite (a build tool and dev server), ESLint (a linter that flags likely bugs), Prettier (a code formatter) and the JavaScript-based TypeScript 5.x compiler (TypeScript adds checked types to JavaScript) all run on Node, even for front-end projects.

Node vs the browser

Both run the same language (ECMAScript, the official name of the JavaScript standard), but the **host APIs** (the extra objects and functions the environment provides) differ:

	Browser	Node.js
Global object	`window` (also `globalThis`)	`global` (also `globalThis`)
DOM (`document`, elements)	Yes	No (use jsdom for tests, Chapter 13)
File system, processes, raw sockets	No (sandboxed — locked inside a restricted environment)	Yes (`node:fs`, `node:child_process`, `node:net`)
Security model	Sandbox + same-origin policy	Full access of the user who started it (optional permission model, §15.12)
Modules	ES modules (`import`/`export`)	ES modules **and** CommonJS (Node's original `require` system)
Shared web APIs	`fetch`, `URL`, `AbortController`, `TextEncoder`, `structuredClone`, `crypto.subtle`, `Blob`, `WebSocket`, timers	The same — Node implements them too
You control the version?	No — the user's browser decides	Yes — you choose the Node version

The last row matters: in Node you can use every feature your version supports without transpiling (converting new syntax into older syntax that more environments understand).

`code/01_node_runtime.mjs` checks these differences:

```
import { builtinModules } from 'node:module';

console.log('typeof window   :', typeof window);        // no DOM in Node
console.log('typeof process  :', typeof process);       // Node-only global
console.log('globalThis === global:', globalThis === global);
console.log('has V8 & libuv  :', 'v8' in process.versions, 'uv' in process.versions);
const webGlobals = ['fetch', 'URL', 'AbortController', 'TextEncoder', 'structuredClone',
  'setTimeout', 'queueMicrotask', 'crypto', 'Blob', 'WebSocket'];
console.log('web APIs present:', webGlobals.filter((n) => n in globalThis).length, 'of', webGlobals.length);
console.log('this file       :', import.meta.filename.split('/').pop());
console.log('typeof __dirname:', typeof __dirname);
// ...
```

Output:

```
typeof window   : undefined
typeof document : undefined
typeof process  : object
globalThis === global: true
node major      : 24
has V8 & libuv  : true true
web APIs present: 10 of 10
this file       : 01_node_runtime.mjs
dirname ends    : 15-nodejs-basics/code
typeof __dirname: undefined
built-ins exist : true
built-in count >50: true
```

Note `typeof __dirname` is `undefined`: that variable exists only in CommonJS files. ES modules use `import.meta.dirname` and `import.meta.filename` (Node 20.11+) instead (§10.6).

Versions and releases

- Up to Node 26, a new **major version** (the first number of a version like `24.11.1`; a change there may break existing code) ships every six months (April and October). **Even-numbered** majors become **LTS** (Long-Term Support) in October of their release year, with about 30 months of fixes; odd-numbered majors are short-lived "Current" releases.

- **From Node 27 (2027)** the project moves to **one major per year** (April), and **every** major becomes LTS each October — no more odd/even split.
- **Node 24** was released in May 2025, became LTS in October 2025 and is supported until April 2028 (V8 13.6, npm 11). Production apps (the live systems real users depend on) should run an **Active or Maintenance LTS** version.

Check yours with `node --version` (`node -v`). Tools such as **nvm**, **fnm** or **Volta** let you install several versions and switch per project (an `.nvmrc` file pins one).

Real-world examples

Netflix, PayPal, LinkedIn and Walmart have written publicly about moving web servers or API layers to Node. Typical uses: REST/GraphQL APIs (two common styles of web API: REST uses URLs plus HTTP methods, GraphQL uses one endpoint with a query language), backend-for-frontend servers (a server shaped around one specific UI), server-side rendering (building the HTML on the server) for React (Next.js, React Handbook ch13), real-time apps (chat, WebSockets — persistent two-way connections), CLIs (command-line interface programs), and build tooling.

When not to use it

Long CPU-bound computations (limited by processor speed rather than by waiting — video encoding, heavy numeric work, ML (machine learning) training) are a poor fit for the main thread — Node shines when the work is *waiting* on I/O, not *computing*. Use worker threads (§15.2), a native module (an add-on compiled from C/C++ or Rust), or a language built for it (Go, Rust, Java, Python with NumPy).

TL;DR

Node = V8 (runs JS) + libuv (event loop, async I/O, thread pool) + core modules. Same language as the browser, different host APIs: no DOM, but files, processes and sockets. Run an LTS version in production (Node 24 today; Node 26 becomes LTS in October 2026).

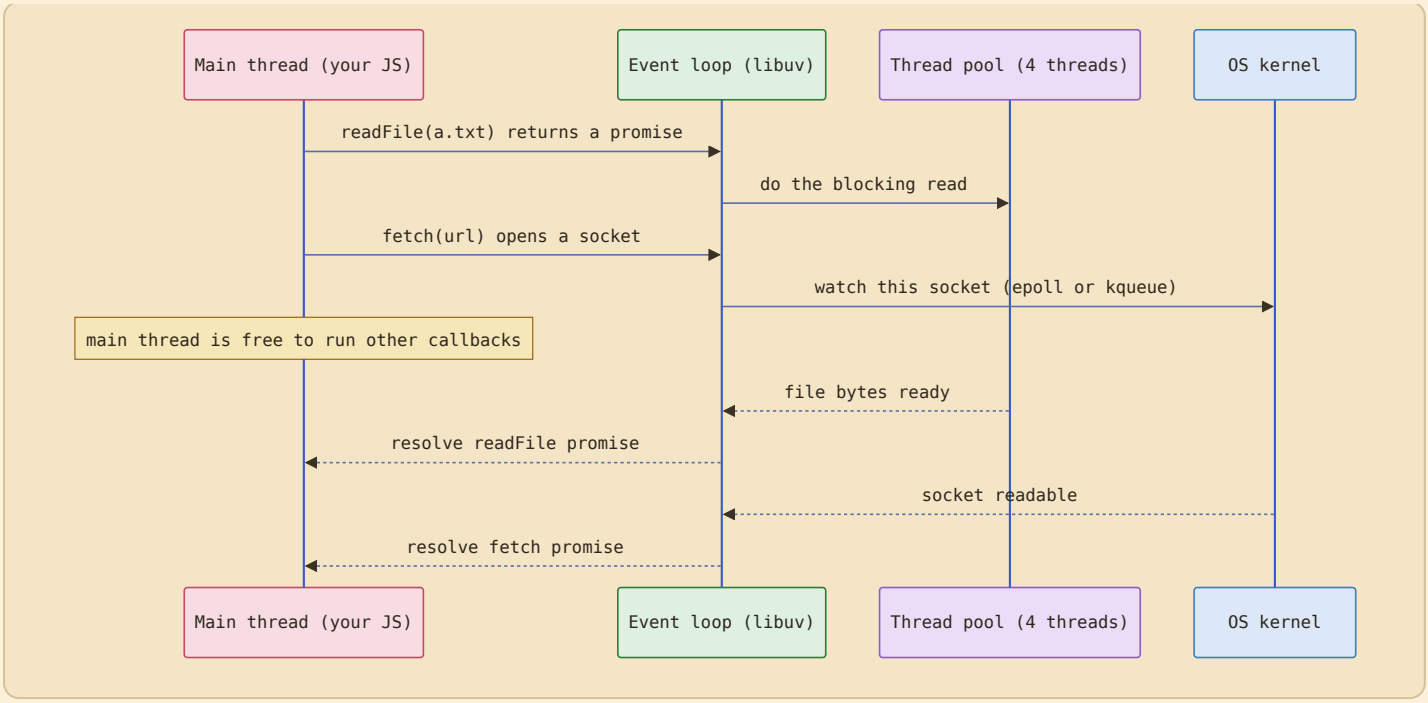
15.2 The event loop, the thread pool and blocking

What it is

Your JavaScript runs on **one main thread**. Node's **event loop** (from libuv) is the loop that, over and over, picks the next ready callback — a timer that expired, a socket with data, a finished file read — and runs it on that thread. The loop's phases (timers → pending → poll → check → close) and the `process.nextTick` / promise microtask queues are covered with runnable ordering puzzles in §12.4.

What this chapter adds is **how the I/O actually gets done** while your thread is free:

1. **Network I/O** (TCP — the protocol under most internet connections — HTTP, and pipes, channels that connect one process's output to another's input) uses the operating system's own asynchronous notification mechanism — **epoll** on Linux, **kqueue** on macOS/BSD, **IOCP** on Windows. No extra threads are needed: the OS tells libuv "this socket has data", and libuv queues your callback.
2. **Work the OS cannot do asynchronously** goes to the **libuv thread pool**: most `fs` operations, `dns.lookup()` (turning a host name into an IP address), CPU-heavy `crypto` functions (`pbkdf2`, `scrypt` — deliberately slow password-hashing functions — and `randomBytes`), and `zlib` compression. The pool has **4 threads by default**; set the environment variable (a named setting the OS hands to a program, §15.6) `UV_THREADPOOL_SIZE` (up to 1024) *before* Node starts to change it.
3. When a pool thread finishes, it puts the result on the event loop's queue; your callback (or the promise continuation) then runs on the **main thread**.



Why it matters — blocking the loop

Because every callback shares one thread, **one slow synchronous piece of code delays everything**: other HTTP requests, timers, health checks (regular "are you alive?" requests from a monitor or load balancer). A 200 ms `JSON.parse` of a huge payload means every other user waits 200 ms. Typical culprits:

- `*Sync` APIs (`readFileSync`, `execSync`) inside request handlers;
- huge `JSON.parse` / `JSON.stringify`;
- regular expressions with catastrophic backtracking — patterns that take exponentially long on certain inputs ("ReDoS", regular-expression denial of service);
- tight loops over large arrays; synchronous crypto (`pbkdf2Sync`).

Worker threads

`node:worker_threads` runs JavaScript on additional threads, each with its own V8 instance and event loop. They communicate by **message passing** (`postMessage`, copied with the structured-clone algorithm) or by sharing a `SharedArrayBuffer` (a block of memory several threads can read and write). Use them for **CPU-bound** work; they do not help with I/O, which is already asynchronous.

`code/02_blocking_and_workers.mjs`:

```
function busyFor(ms) {                  // simulate CPU-bound work
  const end = Date.now() + ms;
  while (Date.now() < end) {}
}

// 1) A 10 ms timer cannot fire while the main thread is busy for 200 ms.
const start = Date.now();
await new Promise((resolve) => {
  setTimeout(() => {
    console.log('timer asked for 10 ms, fired after >= 200 ms:', Date.now() - start >= 200);
    resolve();
  }, 10);
  busyFor(200);
});

// 2) Move the CPU work to a worker thread; the main thread stays responsive.
const workerCode = `
  const { parentPort, workerData } = require('node:worker_threads');
  const fib = (n) => (n < 2 ? n : fib(n - 1) + fib(n - 2));
  parentPort.postMessage(fib(workerData));
`;
let ticks = 0;
const ticker = setInterval(() => ticks++, 5);          // proves the loop is free
const result = await new Promise((resolve, reject) => {
  const w = new Worker(workerCode, { eval: true, workerData: 32 });
  w.once('message', resolve);
  w.once('error', reject);
});
clearInterval(ticker);
console.log('fib(32) from worker:', result);
console.log('main thread kept ticking meanwhile:', ticks > 0);
```

Output:

```
timer asked for 10 ms, fired after >= 200 ms: true
fib(32) from worker: 2178309
main thread kept ticking meanwhile: true
```

(`eval: true` lets the demo keep the worker's code in a string; real projects pass a file path such as `new Worker(new URL('./worker.mjs', import.meta.url))`. Creating a worker costs milliseconds and memory, so real apps keep a **pool** of workers — libraries like Piscina do this.)

Scaling options compared

Tool	What it gives you	Shares memory?	Use for
Async I/O (default)	Thousands of concurrent I/O operations on one thread	—	Almost all server work
`worker_threads`	Extra JS threads in the same process	Only `SharedArrayBuffer`	CPU-heavy tasks (image resize, parsing, hashing)
`child_process` (§15.11)	Separate OS processes, any program	No	Running other programs, isolation
`cluster` / multiple processes	Several Node processes sharing one port	No	Using all CPU cores (a core is one independent processor inside the chip) for an HTTP server (often done by a process manager, container orchestrator such as Kubernetes, or load balancer — a component that spreads requests across servers — instead; see System Design Fundamentals)

Common mistakes

- Reading "single-threaded" as "one thing at a time": JS runs on one thread, but I/O proceeds in parallel in the OS and the thread pool.
- `*Sync` functions in request handlers (fine in CLIs and one-off startup code).
- Expecting workers to speed up I/O (`fetch`, database queries) — they only add overhead.
- Saturating the 4-thread pool with slow `fs` or `pbkdf2` calls, so unrelated `fs` calls queue behind them.

In the interview

Q: Is Node single-threaded? "Your JavaScript runs on a single main thread with an event loop. Network I/O is handled asynchronously by the OS (epoll/kqueue/IOCP), and file system, DNS lookup, some crypto and zlib work runs on libuv's thread pool (4 threads by default). CPU-heavy JS can be moved to worker threads."

- *How would you find what blocks the event loop?* Measure event-loop delay (`perf_hooks.monitorEventLoopDelay()`), then profile (record where the time goes) with `node --cpu-prof` or the inspector (§11.9) to see which function holds the thread.
- *Why can `readFile` be slow under load when CPU is idle?* The thread pool may be saturated; raise `UV_THREADPOOL_SIZE` or reduce concurrent blocking work.
- *Worker thread vs child process?* Worker: same process, cheaper, can share memory, JS only. Child: separate process, any executable (any runnable program), full isolation, more overhead.

Self-check

1. Which of these use the thread pool: `fs.readFile`, an HTTP request, `crypto.pbkdf2`, `setTimeout`? → `fs.readFile` and `crypto.pbkdf2`. HTTP uses OS async I/O; timers are managed by the loop itself.
2. A handler runs a 300 ms synchronous loop. What happens to a second request that arrives meanwhile? → It waits until the loop finishes; its latency (time from request to response) grows by up to 300 ms.
3. How do you give a worker its input? → `workerData` at creation or `postMessage` later.

TL;DR

One JS thread + event loop; the OS does network I/O asynchronously; libuv's thread pool (default 4) does files, DNS lookup, crypto and zlib. Never block the main thread in a server — move CPU work to worker threads.

15.3 Modules in Node — the short version

Everything about module syntax and loading is in Chapter 10. The rules you need while writing Node code:

Question	Answer
Is my `.js` file ESM or CommonJS?	`.mjs` → ESM, `.cjs` → CommonJS; `.js` follows the nearest `package.json` `"type"` (`"module"` → ESM). Without a `"type"`, Node detects ESM syntax automatically (Node 22.7+). §10.8
How do I import a built-in?	Prefer the **`node:` prefix**: `import { readFile } from 'node:fs/promises'`. It is unambiguous (no npm package can shadow it — take its place by having the same name) and required for some newer modules such as `node:test` and `node:sqlite`.
Where is `__dirname`?	CommonJS only. In ESM use `import.meta.dirname` / `import.meta.filename`.
Can ESM use a CommonJS package?	Yes: `import pkg from 'cjs-pkg'` (default import = `module.exports`).
Can CommonJS `require()` an ES module?	Yes in Node 22.12+ / 20.19+, unless the module uses top-level `await`.
Loading JSON?	`import data from './x.json' with { type: 'json' }`, or read and `JSON.parse` it.

Most core modules offer several API styles. Pick the promise one in new code:

Module	Promise API	Callback API	Sync API
File system	`node:fs/promises`	`node:fs` (`fs.readFile(p, cb)`)	`node:fs` (`fs.readFileSync`)
Streams helpers	`node:stream/promises`	`node:stream`	—
Timers	`node:timers/promises` (`await setTimeout(100)`)	global `setTimeout`	—
DNS	`node:dns/promises`	`node:dns`	—
Child processes	`util.promisify(execFile)`	`node:child_process`	`execFileSync`

`util.promisify(fn)` turns any Node-style callback function (`(err, value) => …` as the last argument) into a promise-returning one.

15.4 Files with `node:fs/promises`

What it is

`node:fs` is Node's file-system module: read, write, list, copy, move, delete files and folders, and inspect their metadata. **`node:fs/promises`** is the same functionality returning promises, so it reads naturally with `await`.

How it works — the core operations

Task	Function	Notes
Read a whole file	`readFile(path, 'utf8')`	Without an encoding (the rule for turning bytes into text, e.g. UTF-8) you get a `Buffer` (raw bytes, §15.7)
Write (create/replace)	`writeFile(path, data)`	`{ flag: 'wx' }` = fail if the file exists
Append	`appendFile(path, data)`	Creates the file if missing
Create folders	`mkdir(path, { recursive: true })`	Like `mkdir -p`; no error if it exists
List a folder	`readdir(path, { withFileTypes: true })`	`Dirent` objects with `isFile()`/`isDirectory()`; `{ recursive: true }` walks subfolders (Node 20.1+)
Match patterns	`glob('**/*.txt', { cwd })` (glob = wildcard file pattern; cwd = the folder to search from)	Async iterator (Node 22+)
Metadata	`stat(path)`	size, `mtime` (last-modified time), `isFile()`, `isDirectory()`
Copy / move	`copyFile(a, b)`, `rename(a, b)`, `cp(a, b, { recursive: true })`	`rename` across disks may fail — copy then delete
Delete	`rm(path, { recursive: true, force: true })`	Like `rm -rf`; `unlink` for a single file
Temp folder	`mkdtemp(join(tmpdir(), 'prefix-'))`	Unique name each call
Low-level	`open(path, flags)` → `FileHandle`	`read`, `write`, `close`; always close in `finally`

`code/03_fs_promises.mjs` (key parts):

```
import { mkdtemp, writeFile, appendFile, readFile, readdir, stat, rename,
  copyFile, rm, open, mkdir, access, constants, glob } from 'node:fs/promises';
import { tmpdir } from 'node:os';
import { join } from 'node:path';

const dir = await mkdtemp(join(tmpdir(), 'notes-'));   // unique temp folder
try {
  const file = join(dir, 'todo.txt');
  await writeFile(file, 'buy milk\n');
  await appendFile(file, 'write chapter 15\n');
  const text = await readFile(file, 'utf8');
  console.log('read:', JSON.stringify(text));

  const buf = await readFile(file);
  console.log('as Buffer:', buf.constructor.name, buf.length, 'bytes, first byte', buf[0]);
  // ... write config.json, JSON.parse it back

  await mkdir(join(dir, 'archive/2026'), { recursive: true });
  await copyFile(file, join(dir, 'archive/2026/todo-copy.txt'));
  await rename(join(dir, 'config.json'), join(dir, 'settings.json'));

  const entries = await readdir(dir, { withFileTypes: true });
  console.log('entries:', entries.map((e) => `${e.name}${e.isDirectory() ? '/' : ''}`).sort().join(', '));
  const all = await readdir(dir, { recursive: true });
  console.log('recursive:', all.sort().join(', '));
  const txt = [];
  for await (const p of glob('**/*.txt', { cwd: dir })) txt.push(p);
  console.log('glob *.txt:', txt.sort().join(', '));

  // ... stat(file); open(file, 'r') -> FileHandle, read 8 bytes, close() in finally

  try { await readFile(join(dir, 'nope.txt'), 'utf8'); }
  catch (err) { console.log('error code:', err.code, '| syscall:', err.syscall); }
  // ... access() and the 'wx' flag
} finally {
  await rm(dir, { recursive: true, force: true });
  console.log('cleaned up');
}
```

Output:

```
read: "buy milk\nwrite chapter 15\n"
as Buffer: Buffer 26 bytes, first byte 98
config.port: 8080
entries: archive/, settings.json, todo.txt
recursive: archive, archive/2026, archive/2026/todo-copy.txt, settings.json, todo.txt
glob *.txt: archive/2026/todo-copy.txt, todo.txt
stat: isFile true | size 26 | mtime is Date true
first 8 bytes: "buy milk"
error code: ENOENT | syscall: open
exists & readable: true
wx on existing file -> EEXIST
cleaned up
```

What to notice:

- `readFile` without `'utf8'` returned a `Buffer`; byte 98 is the letter `b`.
- File-system errors are ordinary `Error`s with a string `code`: `ENOENT` (no such file), `EEXIST` (already exists), `EACCES`/`EPERM` (permission), `EISDIR`, `ENOTDIR`, `EMFILE` (too many open files). Branch on `err.code`, never on the message.
- `try … finally` removes the temp folder even if something throws.

Common mistakes

- **Check-then-act races.** `if (await exists(f)) await readFile(f)` can fail anyway — the file may vanish between the two calls (a "TOCTOU" bug: time of check vs time of use). Just try the operation and handle `ENOENT`; use flag `'wx'` for "create only if new".

- **Relative paths** (paths not starting from the root, like `data/x.json`) **resolve against** `process.cwd()`, the folder you launched Node from, *not* the folder of the source file. Build paths from `import.meta.dirname` when you mean "next to this file".
- **Reading huge files whole.** `readFile` loads everything into memory; a 5 GB log will crash or stall. Use streams (§15.9).
- `*Sync` **calls in servers** (§15.2).
- **Forgetting that writes are not atomic** (atomic = happens completely or not at all). A crash in the middle of `writeFile` can leave a half-written file. For important files, write to a temp file in the same folder and then `rename` it over the original (rename is atomic on the same file system).
- **Not closing** `FileHandle`**s** — leaks file descriptors (the OS's numbered handles for open files and sockets) until `EMFILE`.

Real-world examples

Static-site generators read Markdown (a lightweight text format) and write HTML; bundlers write `dist/` (the conventional output folder); loggers append to files; scaffolding CLIs (tools that generate a starter project) copy template folders with `cp(..., { recursive: true })`.

TL;DR

Use `node:fs/promises` with `await`. Pass `'utf8'` to get strings. Handle errors by `err.code`. Build paths from `import.meta.dirname`, not the cwd. Don't check-then-act; don't `readFile` huge files; clean up in `finally`.

15.5 Paths and file URLs: `node:path`, `node:url`

What it is

`node:path` builds and splits file-system paths correctly for the current operating system (`/` on Linux/macOS, `\` on Windows). String concatenation (`dir + '/' + name`) breaks on Windows, doubles slashes and ignores `..`.

How it works

Function	Does	Example result
`join(...parts)`	Glue segments, normalize `..` and `.`	`join('/srv','app','..','data')` → `/srv/data`
`resolve(...parts)`	Make an **absolute** path, processing right to left until absolute; falls back to `process.cwd()`	`resolve('/srv/app','logs')` → `/srv/app/logs`
`basename(p, ext?)`	Last segment	`price.test.mjs`
`extname(p)`	Last extension only	`.mjs`
`dirname(p)`	Everything before the last segment	`/home/ana/.../utils`
`parse(p)` / `format(obj)`	Split into / build from `{ root, dir, base, name, ext }`	
`relative(from, to)`	Path from one location to another	`../public/logo.png`
`normalize(p)`	Clean up `//`, `.` and `..`	`/a/c/d`
`isAbsolute(p)`	Starts from the root?	
`sep`, `delimiter`	`/` or `\`; `:` or `;` (for `PATH`)	
`path.posix`, `path.win32`	Force one platform's rules	

ES modules identify files by **URL** (`file:///home/ana/a.txt`), so **`node:url`** gives `fileURLToPath(url)` and `pathToFileURL(path)` — they handle percent-encoding (a space becomes `%20`) and Windows drive letters correctly. `new URL('./data.json', import.meta.url)` is another common way to point at a sibling file; most `fs` functions accept URL objects directly.

`code/04_path_and_urls.mjs` (excerpt):

```
import path from 'node:path';
import { fileURLToPath, pathToFileURL } from 'node:url';

const p = '/home/ana/projects/shop/src/utils/price.test.mjs';
console.log('basename :', path.basename(p));
console.log('  no ext :', path.basename(p, '.mjs'));
console.log('extname  :', path.extname(p));
console.log('parse    :', path.parse(p));
console.log('join     :', path.join('/srv', 'app', '..', 'data', './users.json'));
console.log('resolve abs wins:', path.resolve('/srv/app', '/etc', 'hosts'));
console.log('relative :', path.relative('/srv/app/src', '/srv/app/public/logo.png'));
console.log('win32 join:', path.win32.join('C:\\Users', 'ana', 'file.txt'));
console.log('fileURLToPath:', fileURLToPath(new URL('file:///home/ana/my%20notes/a.txt')));

// Security: stop "../" escaping a base folder (path traversal)
function safeJoin(base, userPath) {
  const full = path.resolve(base, userPath);
  if (full !== base && !full.startsWith(base + path.sep)) throw new Error(`blocked: ${userPath}`);
  return full;
}
console.log('safe  :', safeJoin('/srv/uploads', 'cat.png'));
try { safeJoin('/srv/uploads', '../../etc/passwd'); } catch (e) { console.log('unsafe:', e.message); }
```

Output:

```
basename : price.test.mjs
  no ext : price.test
extname  : .mjs
dirname  : /home/ana/projects/shop/src/utils
parse    : {
  root: '/',
  dir: '/home/ana/projects/shop/src/utils',
  base: 'price.test.mjs',
  ext: '.mjs',
  name: 'price.test'
}
join     : /srv/data/users.json
...
resolve abs wins: /etc/hosts
relative : ../public/logo.png
...
win32 join: C:\Users\ana\file.txt
...
fileURLToPath: /home/ana/my notes/a.txt
...
safe  : /srv/uploads/cat.png
unsafe: blocked: ../../etc/passwd
```

Common mistakes

- `join` vs `resolve`: `join` just glues (result may be relative); `resolve` always returns an absolute path and **restarts at any absolute segment** (`resolve('/srv/app', '/etc', 'hosts')` → `/etc/hosts`). Passing user input to `resolve` can therefore jump anywhere — hence `safeJoin`.
- **Path traversal**: serving `join(publicDir, req.url)` lets `GET /../../etc/passwd` read system files. Resolve, then check the result is still inside the base folder (the `+ path.sep` stops `/srv/uploads-evil` from passing a plain `startsWith('/srv/uploads')`).
- `extname('archive.tar.gz')` is `.gz`, not `.tar.gz`.
- Using `new URL(import.meta.url).pathname` as a path — it stays percent-encoded (`my%20notes`) and is wrong on Windows. Use `fileURLToPath` or `import.meta.dirname`.

TL;DR

Never concatenate paths by hand: `path.join` to combine, `path.resolve` for absolute paths, `basename/extname/dirname/parse` to split. Convert module URLs with `fileURLToPath`. Validate user-supplied paths against traversal.

15.6 `process`: arguments, environment and exit codes

What it is

`process` is a global object describing the running Node program: its command-line arguments (`argv`), environment variables (`env`), working directory (`cwd()`), standard streams (`stdin`, `stdout`, `stderr` — the default input, output and error channels every program gets), process id (the number the OS uses to identify it), exit code, and lifecycle events.

Command-line arguments

`process.argv` is an array: `[pathToNode, pathToScript, ...yourArgs]`, so user arguments start at index 2. For anything beyond one or two positional values (arguments without a `--name` in front) use the built-in `util.parseArgs` (stable since Node 20) instead of hand-parsing:

```
import { parseArgs } from 'node:util';
const { values, positionals } = parseArgs({
  args: ['--port', '8080', '-v', 'build', 'src/'],   // omit args to read process.argv
  options: {
    port: { type: 'string', default: '3000' },
    verbose: { type: 'boolean', short: 'v' },
    env: { type: 'string', default: 'development' },
  },
  allowPositionals: true,
});
```

Unknown flags throw an error by default (`strict: true`), which is what a CLI usually wants. For big CLIs with sub-commands and help text, libraries such as Commander or yargs add more.

Environment variables

Environment variables are key–value strings the operating system passes to every process (`PATH`, `HOME`, and your own like `DATABASE_URL`). They are the standard way to configure one codebase differently per environment (development, test, production) and to inject secrets without committing them (saving them into Git, the version-control system) — the "config in the environment" rule of the Twelve-Factor App methodology (a widely cited checklist for building cloud services).

Key facts:

1. `process.env.X` is **always a string or `undefined`**. Assigning a number stores its string form. `"false"` and `"0"` are *truthy* strings (treated as true by `if`) — compare explicitly.
2. **`.env` files**: Node 20.6+ can load them itself — `node --env-file=.env app.mjs` (stable in Node 24.10+; `--env-file-if-exists` for optional files), or `process.loadEnvFile(path)` at runtime. Variables already set in the real environment win over the file. The older `dotenv` npm package does the same job.
3. **Never commit `.env` files with secrets**; commit a `.env.example` with dummy values and add `.env` to `.gitignore` (the file listing what Git must not track).
4. `NODE_ENV=production` is a convention many libraries read to switch off dev-only checks. It is not special to Node itself.
5. Read and **validate** configuration once at startup and fail fast with a clear message; don't sprinkle `process.env` reads all over the code.

`code/05_process_and_env.mjs` (excerpt):

```
process.env.MAX_ITEMS = 25;                        // assigned a number...
console.log('typeof MAX_ITEMS:', typeof process.env.MAX_ITEMS);  // ...stored as a string
const maxItems = Number.parseInt(process.env.MAX_ITEMS ?? '10', 10);
const debug = process.env.DEBUG === 'true';         // "false" is a truthy string!

writeFileSync(envFile, 'DB_HOST=localhost\nDB_PORT=5432\n# comment\nGREETING="hello world"\n');
process.loadEnvFile(envFile);

function loadConfig(env) {
  const port = Number(env.PORT ?? 3000);
  if (!Number.isInteger(port) || port <= 0) throw new Error(`Invalid PORT: ${env.PORT}`);
  return Object.freeze({ port, dbHost: env.DB_HOST ?? 'localhost', isProd: env.NODE_ENV === 'production' });
}

process.on('exit', (code) => console.log(`exit event, code ${code}`));
process.exitCode = 0;
```

Output:

```
argv[0] is node: true
user args      : []
parsed values  : [Object: null prototype] {
  port: '8080',
  verbose: true,
  env: 'development'
}
positionals    : [ 'build', 'src/' ]
typeof MAX_ITEMS: string
parsed number  : 26
DEBUG flag     : false | truthy "false"? true
from .env      : localhost 5432 hello world
config         : { port: 8080, dbHost: 'localhost', isProd: true }
config error   : Invalid PORT: abc
...
exit event, code 0
```

(`[Object: null prototype]` means `parseArgs` returns an object created with `Object.create(null)`, so a flag named `--constructor` can't collide with inherited properties.)

Exit codes, signals and graceful shutdown

- A process ends when the event loop has nothing left to do. **Exit code 0** means success; anything else means failure — shells (command-line interpreters such as bash), CI systems (continuous integration: servers that build and test every change automatically) and `npm test` rely on it.
- Prefer **`process.exitCode = 1`** and let Node finish naturally. **`process.exit(1)`** stops immediately and can cut off pending asynchronous work such as writes to a pipe.
- An uncaught exception or unhandled promise rejection ends the process with code 1 (Node 15+ for rejections).
- Signals (short notifications the operating system sends to a process): **Ctrl+C** sends `SIGINT`; container orchestrators and process managers send **`SIGTERM`** before killing a process. Handle them to shut down cleanly:

```
// Not run - pattern for a long-running server (see server in §15.10)
process.once('SIGTERM', async () => {
  console.log('SIGTERM: stop accepting requests, finish in-flight ones');
  server.close();                   // stop new connections
  await db.end();                   // close pools, flush logs
  process.exitCode = 0;             // let the loop drain
  setTimeout(() => process.exit(1), 10_000).unref();   // hard deadline
});
```

`.unref()` tells Node "don't keep the process alive just for this timer".

Other `process` members worth knowing

Member	Use
`process.cwd()` / `process.chdir()`	Current working directory
`process.platform`, `process.arch`	`'linux'`, `'darwin'`, `'win32'`; `'x64'`, `'arm64'`
`process.versions`	Versions of Node, V8, libuv, OpenSSL …
`process.stdin/stdout/stderr`	Standard streams (streams, §15.9); `console.log` writes to `stdout`, `console.error` to `stderr`
`process.hrtime.bigint()`	Nanosecond timer for measuring durations (`performance.now()` also works)
`process.memoryUsage()`	Heap (memory holding JS objects) and RSS (all memory the process holds) numbers for diagnosing leaks
`process.on('uncaughtException' / 'unhandledRejection')`	Last-resort logging — log and exit; don't try to continue (Chapter 11)

Common mistakes

- `if (process.env.FEATURE)` when the value is `"false"` — true!
- Reading `process.env.PORT` as a number without converting it.
- Calling `process.exit()` right after `console.log` to a pipe or file — output may be lost.
- Logging the whole `process.env` (it contains secrets).

TL;DR

`process.argv` (args from index 2; parse with `util.parseArgs`), `process.env` (strings only; load `.env` with `--env-file`; validate at startup), `process.exitCode` for failure, handle `SIGTERM`/`SIGINT` for graceful shutdown.

15.7 `Buffer`: raw bytes

What it is

A **`Buffer`** is a fixed-size chunk of raw bytes. It is Node's subclass of the standard `Uint8Array` (a typed array whose items are bytes, 0–255), with extra helpers for encodings. You meet Buffers whenever data is not (yet) text: file contents read without an encoding, network packets, stream chunks, images, hashes.

An **encoding** is the rule for turning text into bytes and back: `utf8` (default, variable length: 1–4 bytes per character), `base64` (bytes written using 64 safe text characters), `hex` (two characters 0–9/a–f per byte), `latin1`, `utf16le`.

`code/10_buffers.mjs`:

```
const text = 'héllo 👋';
const buf = Buffer.from(text, 'utf8');
console.log('string length (UTF-16 units):', text.length);
console.log('byte length (UTF-8)          :', buf.length, '=', Buffer.byteLength(text));
console.log('bytes (hex)                  :', buf.toString('hex'));
console.log('base64                       :', buf.toString('base64'));
// ... decode base64 back to text; buf instanceof Uint8Array

const whole = Buffer.from('abcdef');
const part = whole.subarray(0, 3);
part[0] = 0x7a;                                  // 'z'
console.log('subarray shares memory       :', whole.toString());
console.log('cut mid-character            :', JSON.stringify(buf.subarray(0, 2).toString('utf8')));
const joined = Buffer.concat([Buffer.from('Node '), Buffer.from('24')]);
console.log('concat                       :', joined.toString(), '| equals:', joined.equals(Buffer.from('Node
24')));
console.log('alloc(4) is zero-filled      :', Buffer.alloc(4));
```

Output:

```
string length (UTF-16 units): 8
byte length (UTF-8)          : 11 = 11
bytes (hex)                  : 68c3a96c6c6f20f09f918b
base64                       : aMOpbGxvIPCfkYs=
back to text                 : héllo 👋
is a Uint8Array              : true
subarray shares memory       : zbcdef
cut mid-character            : "h�"
concat                       : Node 24 | equals: true
alloc(4) is zero-filled      : <Buffer 00 00 00 00>
```

What to notice: `é` takes 2 bytes and the emoji 4, so 8 string units become 11 bytes (string length vs code points is explained in §6.3). Cutting in the middle of `é` produced `�` — which is exactly why stream code must not decode each chunk separately (use `setEncoding('utf8')`, `TextDecoder` with `{ stream: true }`, or collect bytes and `Buffer.concat` first).

Common mistakes

- Using `str.length` as the byte size for `Content-Length` (the HTTP header giving the body's size in bytes) — use `Buffer.byteLength(str)`.
- `Buffer.allocUnsafe(n)` returns memory that may contain old data; use `Buffer.alloc` unless you will overwrite every byte immediately.
- Forgetting `subarray` (and the older `slice`) share memory with the original.
- Treating base64 as encryption — it is only an encoding, anyone can decode it.

TL;DR

Buffer = bytes (a `Uint8Array`). Convert with `Buffer.from(str, enc)` and `buf.toString(enc)`. Byte length ≠ string length. Don't decode UTF-8 chunk by chunk.

15.8 Events: `EventEmitter`

What it is

`EventEmitter` (from `node:events`) is Node's implementation of the **observer / publish-subscribe** pattern: an object keeps a list of **listeners** per **event name**; calling `emit(name, ...args)` calls each listener with those arguments. Servers, sockets, streams, child processes and `process` itself are all EventEmitters — so learning it once explains most of Node's APIs.

Analogy: a radio station. Listeners tune in to a channel (`on('news', fn)`); the station broadcasts (`emit('news', story)`); it doesn't know or care who is listening.

The browser has the same idea as `EventTarget` + `addEventListener` (Chapter 13); Node also supports `EventTarget`, but core modules use `EventEmitter`.

How it works

Method	Meaning
`on(name, fn)` / `addListener`	Add a listener (appended)
`once(name, fn)`	Listener removed after its first call
`prependListener(name, fn)`	Add to the **front** of the list
`off(name, fn)` / `removeListener`	Remove — needs the **same function reference**
`emit(name, ...args)`	Call listeners **synchronously, in order**; returns `true` if there were any
`listenerCount(name)`, `removeAllListeners(name)`	Inspect / clear
`events.once(emitter, name)`	Promise that resolves with the event's arguments array (rejects on `'error'`)
`events.on(emitter, name, { signal })`	Async iterator over repeated events

`code/06_event_emitter.mjs` (excerpt):

```
import { EventEmitter, once, on } from 'node:events';

const bus = new EventEmitter();
function onOrder(order) { console.log('  handler A got order', order.id); }
bus.on('order', onOrder);
bus.once('order', (o) => console.log('  once-handler fires only the first time', o.id));
bus.prependListener('order', () => console.log('  prepended handler runs first'));
console.log('emit #1 ->', bus.emit('order', { id: 1 }));
console.log('emit #2');
bus.emit('order', { id: 2 });
bus.off('order', onOrder);
console.log('listeners left:', bus.listenerCount('order'));

class Kettle extends EventEmitter {          // subclassing: a class that "is" an emitter
  #temp = 20;
  heat(step) {
    this.#temp += step;
    this.emit('temp', this.#temp);
    if (this.#temp >= 100) this.emit('boiled');
  }
}

const e3 = new EventEmitter();
try { e3.emit('error', new Error('disk full')); }        // no 'error' listener -> throws
catch (err) { console.log('unhandled error event threw:', err.message); }

const [name, port] = await once(e4, 'ready');            // promise helper
for await (const [n] of on(e4, 'tick', { signal: ac.signal })) { /* ... */ }
```

Output:

```
  prepended handler runs first
  handler A got order 1
  once-handler fires only the first time 1
emit #1 -> true
emit #2
  prepended handler runs first
  handler A got order 2
listeners left: 1
emit with no listeners -> false
before emit
  (listener runs)
after emit
  kettle at 40
  kettle at 80
  kettle: boiled!
unhandled error event threw: disk full
handled error event: disk full
once() resolved with: db 42
  tick 1
  ...
on() loop ended by AbortError
default max listeners: 10
```

What to notice:

1. The listeners printed **before** `emit #1 -> true` — `emit` is synchronous. It does not schedule anything; asynchrony only happens if a listener itself does something async.
2. The `'error'` event is special: **emitting `'error'` with no listener throws** the error (and would crash the process if uncaught). Always attach an `'error'` listener to emitters you create or receive (streams, sockets, child processes).
3. More than 10 listeners for one event prints a `MaxListenersExceededWarning` — usually a sign you add a listener per request and never remove it (a leak). Raise the limit with `setMaxListeners(n)` only if the number is intentional.

Trade-offs

Pattern	Good for	Weak at
EventEmitter	Many events over time, many listeners, decoupling (parts don't need to know about each other)	Error handling is manual; control flow is harder to follow
Promise	One result, once	Repeated events
Async iterator (`events.on`, streams)	A sequence you process in order with `for await`	Fan-out (delivering the same data to many consumers)
Callback	Legacy APIs	Composition

When not to use it

For a single "done or failed" result, return a promise. An app-wide emitter becomes a hidden global dependency; across processes or machines use a message broker (a server that stores and passes messages between programs: Kafka, RabbitMQ, Redis streams).

Common mistakes

- `off('x', () => …)` with a new arrow function — it's a different reference, nothing is removed.
- Throwing inside a listener: the exception propagates out of `emit()` to the emitter's caller.
- Using an `async` listener and forgetting its rejections are not caught by the emitter (option: `new EventEmitter({ captureRejections: true })` routes them to `'error'`).
- Missing `'error'` listeners.

In the interview

Q: What is EventEmitter and is `emit` async? "It's Node's observer pattern: `on` registers listeners per event name, `emit` calls them synchronously in registration order and returns whether any existed. Streams, HTTP servers and sockets extend it. An `'error'` event without a listener throws."

- *How to wait for one event with await?* `const [arg] = await events.once(emitter, 'name')`.
- *How to avoid listener leaks?* Remove listeners when done (`off`, `once`, or an `AbortSignal` passed via `{ signal }` in `events.on`/`once`), and watch for `MaxListenersExceededWarning`.

Self-check

1. Output order of `console.log('A'); em.emit('x'); console.log('C')` with a listener printing `B`? → A, B, C.
2. Why must `off` receive the original function? → Listeners are matched by reference.
3. What happens on `emit('error', err)` with no listener? → `err` is thrown.

TL;DR

`on`/`once`/`off`/`emit`; emit is synchronous and ordered; always handle `'error'`; use `events.once` / `events.on` to bridge to promises and `for await`.

15.9 Streams

What it is

A **stream** is data that arrives (or leaves) **piece by piece over time** instead of all at once. Each piece is a **chunk** (a `Buffer` or string, or any value in **object mode**). Analogy: drinking through a straw from a jug instead of pouring the whole jug into your mouth — memory use stays small no matter how big the jug.

Node has four stream types (all EventEmitters):

Type	Direction	Examples
Readable	Source you read from	`fs.createReadStream`, HTTP request (server side), `process.stdin`, `Readable.from(iterable)`
Writable	Destination you write to	`fs.createWriteStream`, HTTP response, `process.stdout`
Duplex	Both, independent sides	TCP socket
Transform	Duplex whose output is computed from input	`zlib.createGzip()`, `crypto` hashes/ciphers, your own parsers

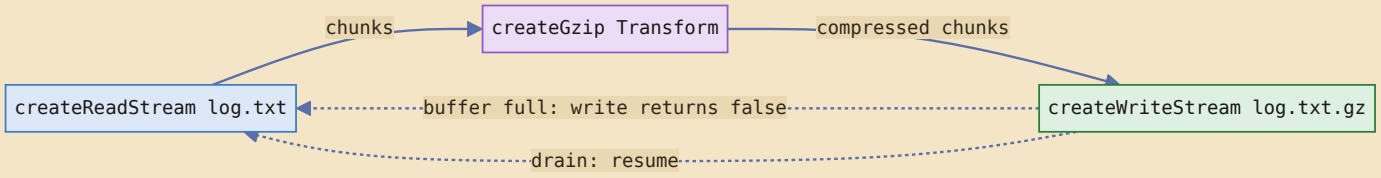
Why it matters

Memory (a 10 GB file needs only a few KB of buffer, not 10 GB), **time to first byte** (how soon the receiver gets anything — send a response before all of it exists), and **composition** of small stages (read → decompress → parse → write), like Unix pipes (`cmd1 | cmd2` in a shell).

How it works

1. A Readable produces chunks into an internal buffer; a consumer takes them (`for await`, `'data'` events, or piping).
2. A Writable accepts chunks with `write(chunk)`. Each has an internal buffer limited by `highWaterMark` (64 KiB — 1 KiB = 1024 bytes — for byte streams since Node 22 — it was 16 KiB before — and 16 objects in object mode).
3. **Backpressure**: when the Writable's buffer passes `highWaterMark`, `write()` returns `false` — "slow down". The producer should stop and wait for the `'drain'` event. Ignoring this makes memory grow without limit when the source is faster than the destination (e.g. reading a fast disk, writing to a slow network client).

4. `pipeline(src, ...transforms, dest)` from `node:stream/promises` connects the stages, **handles backpressure automatically**, **forwards errors from any stage**, **destroys all streams** on failure, and returns a promise. Prefer it over the older `src.pipe(dest)`, which does not propagate errors or clean up.
5. Readables are **async iterables**, so `for await (const chunk of readable)` works (and respects backpressure).



`code/07_streams.mjs` (key parts):

```
// imports: Readable, Writable, Transform (node:stream), pipeline (node:stream/promises),
// createReadStream/createWriteStream (node:fs), createGzip/createGunzip (node:zlib), createInterface
(node:readline)
// 2) A Transform (uppercase) and a Writable (collector)
const upper = () => new Transform({
  transform(chunk, _enc, done) { done(null, chunk.toString().toUpperCase()); },
});
function collector(out) {
  return new Writable({ write(chunk, _enc, done) { out.push(chunk.toString()); done(); } });
}
const out = [];
await pipeline(Readable.from(['hello ', 'streams']), upper(), collector(out));

// 3) Chunks do NOT align with lines: keep the incomplete tail between chunks
function splitLines() {
  let rest = '';
  return new Transform({
    readableObjectMode: true,
    transform(chunk, _enc, done) {
      const parts = (rest + chunk).split('\n');
      rest = parts.pop();
      for (const line of parts) this.push(line);
      done();
    },
    flush(done) { if (rest) this.push(rest); done(); },
  });
}

// 4) 50,000-line file -> gzip -> gunzip, constant memory
function* logLines() { for (let i = 1; i <= 50_000; i++) yield `line ${i} status=ok\n`; }
await pipeline(Readable.from(logLines()), createWriteStream(src));
await pipeline(createReadStream(src), createGzip(), createWriteStream(src + '.gz'));
await pipeline(createReadStream(src + '.gz'), createGunzip(), createWriteStream(src + '.copy'));

// 5) Line by line: for await (const line of createInterface({ input: createReadStream(src) })) ...
// 6) pipeline(createReadStream('missing.txt'), createGzip(), ...) -> the promise rejects

// 7) Backpressure by hand: write() -> false -> wait for 'drain'
const slow = new Writable({ highWaterMark: 16, write(_c, _e, done) { setTimeout(done, 1); } });
for (let i = 0; i < 5; i++) {
  const ok = slow.write('0123456789');
  if (!ok) { falses++; await new Promise((r) => slow.once('drain', r)); }
}
```

Output:

```
1) read chunks: [ 'a', 'b', 'c' ]
2) pipeline result: HELLO STREAMS
3) lines: [ 'id,name', '1,Ana', '2,Bob', '3,Cy' ]
4) original bytes: 1038894 | gzip smaller: true
   round trip identical: true
5) readline counted 50000 lines; last = "line 50000 status=ok"
6) pipeline rejected: ENOENT
7) write() returned false 2 times; waited for drain each time: true
```

What to notice:

- In part 3 the input chunks were `'…2,Bo'` and `'b\n3,Cy'` — the name `Bob` was split across chunks, and the line splitter stitched it back. Chunk boundaries are arbitrary; never assume a chunk is a line or a JSON object.
- The 1 MB file compressed to under a fifth of its size, and the round trip was byte-identical — without ever holding the file in memory.
- The missing file error (`ENOENT`) surfaced as a rejected `pipeline` promise, catchable with `try/catch`.
- In part 7, the first write (10 bytes) fit under the 16-byte mark; the second pushed the buffer to 20 bytes, so `write()` said `false`, and we waited for `'drain'`.

Web Streams

Node also implements the standard **Web Streams API** (`ReadableStream`, `WritableStream`, `TransformStream`) used by `fetch` — `response.body` is a web `ReadableStream`. Convert with `Readable.fromWeb(webStream)` and `Readable.toWeb(nodeStream)`. Use Node streams with Node APIs, web streams with `fetch` and cross-runtime code.

Trade-offs

Approach	Memory	Simplicity	Use when
`readFile` / `res.text()`	Whole payload	Simplest	Small data (config, JSON under a few MB)
`for await` over a stream	One chunk	Simple	Processing sequentially
`pipeline` of stages	One chunk per stage	Moderate	Copy/transform/compress; server responses
Hand-written `write`/`drain`	One chunk	Error-prone	Rarely — let `pipeline` do it

Common mistakes

- `src.pipe(dest)` without error handlers — an error in `src` leaves `dest` open and leaks file descriptors. Use `pipeline`.
- Ignoring the return value of `write()` in a loop (unbounded memory).
- Decoding each chunk as UTF-8 separately (§15.7).
- Mixing `'data'` events and `for await` on the same stream.
- Forgetting `flush` in a Transform that keeps state (the last partial line is lost).

In the interview

Q: What is backpressure? "When a consumer is slower than the producer, data piles up in memory. Node streams signal it: `write()` returns false once the buffer passes `highWaterMark`, and the producer should pause until `'drain'`. `pipeline()` and `pipe()` do this automatically; `pipeline` also propagates errors and cleans up every stream."

- *Why stream a file download instead of `readFile` + `res.end`?* Constant memory per request and the first bytes reach the client sooner.
- *How do you read a big file line by line?* `readline.createInterface({ input: createReadStream(f) })` with `for await`.

Self-check

1. Which stream type is `zlib.createGzip()`? → Transform.

2. What does `write()` returning `false` mean? → The internal buffer is above `highWaterMark`; wait for `'drain'`.
3. Name two advantages of `pipeline` over `pipe`. → Error propagation and cleanup (destroying all streams); returns a promise.

TL;DR

Streams move data in chunks: Readable, Writable, Duplex, Transform. Backpressure = `write()` false → wait for `'drain'`. Use `stream/promises` `pipeline` to connect stages safely, `for await` to consume, `readline` for lines.

15.10 Building an HTTP server with `node:http`

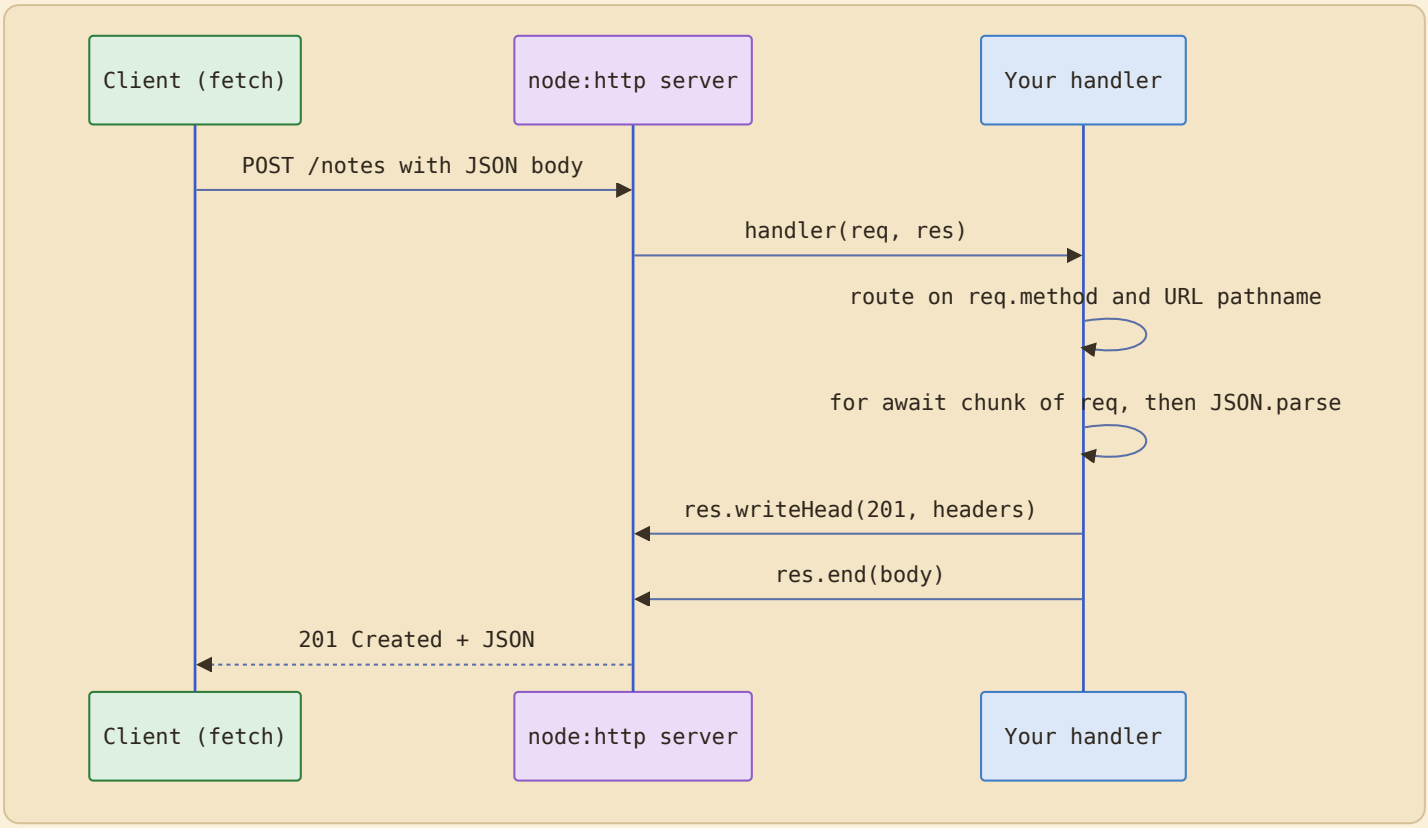
What it is

`node:http` implements HTTP/1.1 clients and servers. `http.createServer(handler)` returns a server; `handler(req, res)` runs for every request:

- `req` (`IncomingMessage`) — a **Readable stream** of the request body, plus `req.method`, `req.url` (path + query string), `req.headers` (lower-cased names).
- `res` (`ServerResponse`) — a **Writable stream**: set status and headers (`res.writeHead(status, headers)` or `res.statusCode` + `res.setHeader`), write the body (`res.write`), and finish with `res.end()`.

Frameworks such as **Express**, **Fastify**, **Koa** and **Hono** wrap exactly this API, adding routing (picking a handler by method and path), middleware (functions every request passes through), body parsing and validation. Knowing the raw layer makes those frameworks — and their performance and error behaviour — understandable. (HTTP concepts: methods, status codes, headers, CORS are covered from the browser side in Chapter 14.)

How a request flows



Example — a JSON API and its client in one script

`code/08_http_server.mjs` starts a server on **port 0** (the OS picks a free port), calls it with the global `fetch`, then shuts it down:

```
import { createServer } from 'node:http';
import { once } from 'node:events';
const notes = [{ id: 1, text: 'learn streams' }];

async function readJson(req, limit = 1_000_000) {
  let body = '';
  for await (const chunk of req) {                  // req is a Readable stream
    body += chunk;
    if (body.length > limit) throw Object.assign(new Error('Payload too large'), { status: 413 });
  }
  try { return JSON.parse(body); }
  catch { throw Object.assign(new Error('Invalid JSON'), { status: 400 }); }
}

function send(res, status, data) {
  const body = JSON.stringify(data);
  res.writeHead(status, { 'Content-Type': 'application/json', 'Content-Length': Buffer.byteLength(body) });
  res.end(body);
}

const server = createServer(async (req, res) => {
  const url = new URL(req.url, `http://${req.headers.host}`);
  try {
    if (req.method === 'GET' && url.pathname === '/health') return send(res, 200, { ok: true });
    if (req.method === 'GET' && url.pathname === '/notes') {
      const q = url.searchParams.get('q') ?? '';
      return send(res, 200, notes.filter((n) => n.text.includes(q)));
    }
    if (req.method === 'POST' && url.pathname === '/notes') {
      const { text } = await readJson(req);
      if (typeof text !== 'string' || !text.trim()) return send(res, 422, { error: 'text is required' });
      const note = { id: notes.length + 1, text: text.trim() };
      notes.push(note);
      res.setHeader('Location', `/notes/${note.id}`);
      return send(res, 201, note);
    }
    if (url.pathname === '/stream') {               // stream a response in chunks
      res.writeHead(200, { 'Content-Type': 'text/plain' });
      for (let i = 1; i <= 3; i++) res.write(`chunk ${i}\n`);
      return res.end();
    }
    send(res, 404, { error: `No route for ${req.method} ${url.pathname}` });
  } catch (err) {
    send(res, err.status ?? 500, { error: err.status ? err.message : 'Internal Server Error' });
  }
});

server.listen(0, '127.0.0.1');                      // port 0 = let the OS pick a free port
await once(server, 'listening');
const { port } = server.address();
// ... call() wraps fetch(`http://127.0.0.1:${port}${path}`) and logs status + body
await call('GET', '/health');
await call('GET', '/notes');
const created = await call('POST', '/notes', { text: '  write chapter 15 ' });
console.log('  Location header:', created.headers.get('location'));
await call('POST', '/notes', { text: '' });
await call('POST', '/notes', '{not json');
await call('GET', '/notes?q=chapter');
await call('GET', '/stream');
await call('DELETE', '/notes/1');

server.close();                                     // stop accepting new connections
server.closeAllConnections();                       // drop idle keep-alive sockets (Node 18.2+)
await once(server, 'close');
console.log('server closed');
```

Output:

```
listening on a free port: true
GET /health -> 200 { ok: true }
GET /notes -> 200 [ { id: 1, text: 'learn streams' } ]
POST /notes -> 201 { id: 2, text: 'write chapter 15' }
  Location header: /notes/2
POST /notes -> 422 { error: 'text is required' }
POST /notes -> 400 { error: 'Invalid JSON' }
GET /notes?q=chapter -> 200 [ { id: 2, text: 'write chapter 15' } ]
GET /stream -> 200 "chunk 1\nchunk 2\nchunk 3\n"
DELETE /notes/1 -> 404 { error: 'No route for DELETE /notes/1' }
server closed
```

How each part works:

1. **Routing** is `if` statements on `req.method` and `url.pathname`; `new URL(req.url, base)` parses path and query safely.
2. **Reading the body**: `req` is a stream, so `for await` collects chunks; the size limit protects memory (a real server would also check `Content-Type`).
3. **Status codes**: 200, 201 Created (+ `Location`), 400 bad JSON, 404 unknown route, 413 too large, 422 failed validation, 500 unexpected — with internal details hidden.
4. **`closeAllConnections()`**: `server.close()` stops *new* connections but waits for existing ones; `fetch` keeps connections alive (keep-alive: reusing one connection for several requests), so without it `'close'` would wait for idle sockets to time out.
5. **Port 0 + `127.0.0.1`**: no port conflicts, nothing reachable from other machines — the standard pattern in test suites.

Production concerns (what frameworks and platforms add)

Concern	Raw `node:http`	Typical solution
Routing with params (`/notes/:id`)	Manual	Framework router (Express, Fastify, Hono)
Body parsing, size limits, validation	Manual	Framework + schema (a description of the expected data shape: Zod, JSON Schema)
Error handling	`try/catch` per handler	Central error middleware
HTTPS/TLS (TLS is the encryption under HTTPS)	`node:https` with certs (certificates proving the server's identity)	Usually terminated (decrypted) at a reverse proxy — a server in front of your app that forwards traffic — or load balancer (nginx, cloud LB)
HTTP/2	`node:http2`	Proxy or `http2` module
Timeouts	`server.requestTimeout` (default 300 s), `headersTimeout`, `keepAliveTimeout`	Tune per workload
Graceful shutdown	`close()` + `SIGTERM` handler (§15.6)	Same, plus health checks
Using all cores	One process = one core for JS	Several processes behind a load balancer / container replicas

Common mistakes

- Forgetting `res.end()` — the client hangs until a timeout.
- Setting headers after the body started (`ERR_HTTP_HEADERS_SENT`).
- Not handling a rejected promise inside an async handler — Node doesn't catch it for you; the request hangs and an unhandled rejection crashes the process. Wrap handlers in `try/catch` (as above).
- Unlimited body size (memory exhaustion attack — sending huge bodies until the server runs out of memory).

- Using `str.length` for `Content-Length` with non-ASCII text (characters beyond basic English, which take more than one byte).
- Returning stack traces to clients.

In the interview

Q: How does a Node HTTP server handle many concurrent requests on one thread? "Each request is an event; the handler runs briefly, and whenever it awaits I/O (database, another API) the thread serves other requests. As long as handlers don't do long synchronous work, thousands of connections share one thread."

- *How do you shut down gracefully?* On `SIGTERM`: stop accepting (`server.close()`), finish in-flight requests, close idle keep-alive sockets, close DB pools, exit with a deadline.
- *Express vs Fastify?* Express: minimal, huge ecosystem (plugins and libraries built around it), v5 (2024) handles rejected promises from async handlers. Fastify: schema-based validation/serialization, faster JSON handling, plugin system.

Self-check

1. How do you get the query parameter `q` from `req.url`? → `new URL(req.url, 'http://x').searchParams.get('q')`.
2. What status fits "JSON parsed but `text` missing"? → 422 (or 400).

TL;DR

`createServer((req, res) => …)`: `req` = readable stream + method/url/headers; `res` = writable stream; always `res.end()`. Parse URLs with `new URL`, limit bodies, map errors to status codes, close gracefully. Frameworks add routing and validation on top of this.

15.11 Child processes and npm scripts

What it is

`node:child_process` starts other programs (git, ffmpeg, a Python script, another Node process) as separate operating-system processes.

Function	Shell?	Output	Use for
`execFile(file, args)`	No	Buffered (collected in memory, handed over at the end)	Run a program with arguments — **safe default**
`spawn(cmd, args)`	No (unless `shell: true`)	Streams (`child.stdout`)	Long-running or large output
`exec(commandString)`	**Yes**	Buffered	Quick shell one-liners with **trusted** input only
`fork(modulePath)`	No	Streams + IPC (inter-process communication) channel (`send`/`'message'`)	Another Node script that talks to the parent
`*Sync` versions	—	Blocks	Build scripts and CLIs, never servers

Security: `exec(\`convert ${userFile}\`)` passes a string to a shell, so a file name like `x; rm -rf ~` becomes two commands — **command injection**. `execFile` / `spawn` pass arguments as an array with no shell interpretation.

`code/09_child_process_and_scripts.mjs` also builds a throw-away project and runs its `package.json` scripts with **`node --run`** (Node 22+), which is a faster, simpler alternative to `npm run` (§10.9 covers npm, scripts, `npx` and the pre/post hook differences):

```
const execFileP = promisify(execFile);
const { stdout } = await execFileP(process.execPath, ['-e', 'console.log(6 * 7)']);
console.log('child said:', stdout.trim());

const child = spawn(process.execPath, ['-e', 'console.log("working"); process.exit(3)']);
child.stdout.on('data', (d) => console.log('spawn stdout:', d.toString().trim()));
const [code] = await new Promise((r) => child.on('close', (...a) => r(a)));
console.log('spawn exit code:', code);

await writeFile(join(dir, 'package.json'), JSON.stringify({
  name: 'demo', private: true, type: 'module',
  scripts: { hello: 'node hello.js', greet: 'node hello.js' },
}, null, 2));
await writeFile(join(dir, 'hello.js'), 'console.log(`hello from ${process.argv[2] ?? "script"}`)');
const r1 = await execFileP(process.execPath, ['--run', 'hello'], { cwd: dir });
const r2 = await execFileP(process.execPath, ['--run', 'greet', '--', 'Ana'], { cwd: dir });
```

Output:

```
child said: 42
spawn stdout: working
spawn exit code: 3
node --run hello -> hello from script
node --run greet -- Ana -> hello from Ana
```

`process.execPath` is the absolute path of the running `node` binary (the compiled program file) — safer than assuming `node` is on the `PATH` (the list of folders the shell searches for programs). Arguments after `--` are passed to the script.

npm scripts in a Node project

A typical back-end `package.json` `scripts` block:

```
{
  "scripts": {
    "start": "node src/server.mjs",
    "dev": "node --watch --env-file=.env src/server.mjs",
    "test": "node --test",
    "typecheck": "tsc --noEmit",
    "lint": "eslint ."
  }
}
```

(Not run.) `npm start`, `npm test`, `npm run dev` or `node --run dev`. Tests and type-checking are the subject of Chapter 16.

> **TL;DR**

Use `execFile`/`spawn` with argument arrays (no shell); `exec` only with trusted strings. `node --run <script>` runs `package.json` scripts quickly; `npm run` adds pre/post hooks.

15.12 Everyday Node flags, and Node vs Deno vs Bun

Command-line flags worth knowing (Node 24)

Flag	What it does	Status
`node --watch app.mjs`	Restart on file changes	Stable
`node --env-file=.env app.mjs`	Load environment variables from a file	Stable (24.10+)
`node --run <script>`	Run a `package.json` script	Added 22
`node --test`	Run tests (Chapter 16)	Stable (20+)
`node app.ts`	Run TypeScript by **stripping types** (deleting the type annotations, without checking them) (§16.9)	On by default since 23.6; stable in 24.12
`node --inspect` / `--inspect-brk`	Attach Chrome DevTools / VS Code debugger (§11.9)	Stable
`node --cpu-prof`, `--heap-prof`	Write CPU / heap profiles	Stable
`node --permission --allow-fs-read=./data app.mjs`	**Permission model**: deny file-system reads and writes, child processes, workers, native addons … unless explicitly allowed	Stable since 23.5 / 22.13
`node -e "code"` / `node -p "expr"`	Evaluate / evaluate-and-print	Stable

The permission model is defence in depth — it limits what a compromised dependency (a third-party package your project uses) can do — but the docs are explicit that it is not a sandbox against deliberately malicious code.

Node vs Deno vs Bun

Two newer runtimes also run JavaScript and TypeScript on servers. All three run most npm packages today.

	Node.js	Deno	Bun
First release	2009	2020 (1.0); **Deno 2** in Oct 2024 added full npm / `package.json` compatibility	2023 (1.0, September)
Creator / owner	Ryan Dahl; now OpenJS Foundation	Ryan Dahl; Deno Land Inc.	Jarred Sumner / Oven; **acquired by Anthropic in December 2025**, still MIT-licensed (a permissive open-source licence)
Engine	V8	V8 (runtime written in Rust)	**JavaScriptCore** (Safari's engine; runtime written in Zig)
TypeScript	Type stripping built in (erasable syntax only); type-check with `tsc`	Built in; `deno check` type-checks	Built in (transpiles, no type check)
Security default	Full access (opt-in `--permission`)	**Deny by default** — `--allow-net`, `--allow-read` …	Full access
Built-in tools	test runner, watch mode, `--run`	formatter, linter, test runner, bundler-ish `compile`, package manager	package manager (fast `bun install`), bundler, test runner (Jest-like)
Strength	Maturity, ecosystem, LTS policy, hosting support everywhere	Secure defaults, web-standard APIs, all-in-one tooling	Speed of install and startup, all-in-one tooling

Which to choose? Node for production by default — largest ecosystem, predictable LTS, supported by every host and observability tool (monitoring, logging and tracing services). Deno for secure-by-default permissions and integrated tooling. Bun when install/startup speed matters; many teams use `bun install` or Bun's test runner alongside a Node production runtime. Check native add-ons and less common Node APIs before switching.

TL;DR

Learn `--watch`, `--env-file`, `--run`, `--test`, `--inspect`, and running `.ts` directly. Node is the safe default runtime; Deno (secure-by-default, V8) and Bun (fast, JavaScriptCore) are credible alternatives that run most npm code.

Chapter 15 TL;DR

- **Node.js** = V8 + libuv + core modules. No DOM; files, processes, sockets instead. Run an **LTS** release (Node 24 today).
- **One JS thread**: network I/O is OS-async; fs, DNS lookup, crypto and zlib use the **libuv thread pool** (4 threads). Never block the loop; use **worker threads** for CPU work.
- Import built-ins with `node:`; ESM uses `import.meta.dirname`; details in Chapter 10.
- `node:fs/promises`: `readFile(p,'utf8')`, `writeFile`, `mkdir({recursive})`, `readdir`, `stat`, `rm({recursive,force})`; errors by `err.code`; don't check-then-act.
- `node:path`: `join`, `resolve`, `basename/extname/dirname/parse`; `fileURLToPath`; guard against **path traversal**.
- `process`: `argv` (+ `util.parseArgs`), `env` **is strings only**, `--env-file`, validate config at startup, `process.exitCode`, handle `SIGTERM`.
- **Buffer** = bytes; byte length ≠ string length.
- **EventEmitter**: `on/once/off/emit`; emit is synchronous; always handle `'error'`; `events.once`/`events.on` for promises.
- **Streams**: Readable/Writable/Duplex/Transform; **backpressure** (`write()` false → `'drain'`); use `pipeline` from `stream/promises`.

- **`node:http`**: `req` readable + `res` writable; always `res.end()`; limit bodies; map errors to statuses; port 0 in tests; graceful shutdown.
- **`child_process`**: `execFile`/`spawn` with arrays (no shell injection); `node --run` for scripts.
- **Deno** (secure by default) and **Bun** (fast, JavaScriptCore, owned by Anthropic) are alternatives; Node remains the default.

Next: prove your code works and catch type errors before running it — Chapter 16: Testing and TypeScript.

Chapter 16 — Testing and TypeScript

Two habits separate code that *seems* to work from code you can change without fear. **Automated tests** prove behaviour by running your code and checking the results. **Static types** (TypeScript) catch whole classes of mistakes — typos, wrong argument types, forgotten `null` checks — before the code runs at all. They overlap only a little: types tell you *the shapes fit*; tests tell you *the answers are right*. This chapter teaches both with tools that ship with Node 24 and the TypeScript compiler (the program that checks TypeScript and can translate it to JavaScript).

What you'll learn

1. Why we test, kinds of tests, and the test-runner landscape (node:test, Vitest, Jest, Playwright).
2. Assertions with `node:assert/strict`.
3. Writing and running tests with `node:test`: `test`/`describe`/`it`, hooks, subtests, skip/todo/only, reporters, watch mode.
4. Async tests and the classic failures (wrong expectation, timeout, missing `await`).
5. Test doubles: fakes, stubs, spies, `mock.fn`, `mock.method`, fake timers, module mocks.
6. Snapshot tests and code coverage.
7. TypeScript: what it is, `tsc`, `tsconfig.json`, TypeScript 7.
8. Running `.ts` files directly with Node (type stripping — deleting the type annotations so plain JavaScript remains) and its rules.
9. Types: primitives, arrays, tuples, object types, interfaces, unions, literals, `any` vs `unknown`.
10. Narrowing, discriminated unions and exhaustive checks.
11. Generics and utility types.
12. Classes, `as const`, `satisfies`, enums and their alternatives.
13. Tests written in TypeScript, and JSDoc types (types written inside specially formatted `/** … */` comments) for plain JavaScript.

Before you start. You need functions (Chapter 4), classes (Chapter 9), modules and `package.json` scripts (Chapter 10), errors (Chapter 11), and `async`/`await` (Chapter 12). Testing React components (Testing Library) and TypeScript with React are in the React Handbook: ch11 and ch12.

The code. Everything is in `code/` — a tiny project with its own `package.json` (the project's manifest: name, scripts, dependencies — here `"type": "module"`, scripts `test`, `coverage`, `typecheck`) and `tsconfig.json`. The modules under test live in `code/src/`. All outputs are real: **Node 24.11.1** and **TypeScript 7.0.2** (`tsc`). Test files are ordinary scripts too — running `node cart.test.mjs` executes its tests and exits with code 1 if any fail — so the book's runner executes them like every other program. Files that fail *on purpose* (`demo_failing.mjs`, `ts_errors.ts`, `jsdoc_errors.mjs`) or need a special flag (`module_mock_demo.mjs`) are marked `// tools_run: skip`, and the exact command to run them is in their first line and in the text. Timings in test output (the `(0.42ms)` parts) vary from run to run.

16.1 Why test, and what kinds of tests exist

What it is

An **automated test** is code that runs other code with known inputs and **asserts** (checks) that the result is what you expect. A **test runner** finds test files, runs them, and reports passes and failures. A **test suite** is a group of related tests.

Analogy: a checklist a pilot runs before every flight. It is boring, fast, and catches the one thing you'd otherwise forget.

Why it matters

- **Regression safety** (a *regression* is something that used to work and broke): after a refactor (restructuring code without changing what it does) or upgrade, the suite tells you in seconds whether something that used to work broke.

- **Design feedback**: code that's hard to test (hidden global state — data any code can change — or hard-wired network calls) is usually hard to reuse.
- **Executable documentation** and a **CI gate** (CI, continuous integration, is a server that builds and tests every change automatically): the test command's exit code (0 = success) decides whether a pull request (a proposed change submitted for review) can merge (be added to the main code).

Kinds of tests

Kind	What it exercises	Speed	Typical tool (JS)
Unit	One function/class/module in isolation; dependencies replaced by doubles (stand-ins, §16.5)	Milliseconds	node:test, Vitest, Jest
Integration	Real pieces together (HTTP handler + a real database in a container — a lightweight packaged environment, e.g. Docker)	Tens of ms to seconds	Same runners + Testcontainers (a library that starts such containers for tests)
End-to-end (E2E)	The whole app through its real interface (a browser clicking through pages)	Seconds	Playwright, Cypress
Static checks	No execution: types, lint rules (automatic checks for suspicious code)	Fast	TypeScript, ESLint

The **test pyramid** advises many unit tests, fewer integration tests and a handful of E2E tests, because higher layers are slower and flakier (a *flaky* test sometimes passes and sometimes fails with no code change). The **testing trophy** (popular in front-end) makes integration tests the fattest layer, for the most confidence per test. Both agree: static checks at the bottom, few E2E tests, and test **behaviour**, not implementation details.

Test runners in 2026

Runner	Strengths	Notes
`node:test` (built in)	No dependencies, fast, runs TypeScript via type stripping; mocks, fake timers, snapshots, coverage built in	Stable since Node 20; used here
Vitest	Jest-compatible API, Vite's (a popular build tool's) config and transforms, browser mode	Common for Vite / React projects
Jest	Very large ecosystem (plugins and add-ons)	Older projects; ESM (`import`/`export` modules) historically needed extra config
Playwright Test	Real browsers (Chromium, Firefox, WebKit — the engines behind Chrome, Firefox and Safari), auto-waiting, tracing (recording each step for later inspection)	E2E and component tests
Bun test / Deno test	Built into those runtimes (§15.12)	Jest-like APIs

The concepts (describe/it, hooks, assertions, mocks, snapshots) are nearly identical across all of them; §16.7 has a translation table.

> **TL;DR**

Tests run your code and check results automatically. Unit tests are fast and isolated; integration tests check real pieces together; E2E tests drive the whole app. Test behaviour, keep most tests fast, and let CI run them on every change.

16.2 Assertions with `node:assert/strict`

What it is

An **assertion** is a check that throws an `AssertionError` when it fails. A test *fails* exactly when something inside it throws. `node:assert` is Node's assertion module; import the **strict** variant, `node:assert/strict`, in which `equal` means `===`-style comparison (actually `Object.is`) and `deepEqual` also compares types and prototypes. (The legacy, non-strict mode uses `==` — avoid it.)

Assertion	Passes when
`equal(actual, expected)`	`Object.is(actual, expected)` — same primitive value or **same object reference**
`deepEqual(actual, expected)`	Same structure and values, recursively, **type-strict** (`1` ≠ `'1'`), same prototypes
`notEqual` / `notDeepEqual`	The opposites
`ok(value)`	`value` is truthy (counts as true in an `if`)
`match(string, regex)` / `doesNotMatch`	Regex (regular expression, a text pattern) matches / doesn't
`throws(fn, expected?)`	`fn()` throws; `expected` can be an error class, a regex for the message, a validation object `{ name, message }`, or a function
`rejects(promiseOrFn, expected?)`	The promise **rejects** — must be `await`ed
`doesNotThrow` / `fail(msg)`	Rarely needed (just call the code) / always fails
`partialDeepStrictEqual(actual, expected)`	`actual` contains at least the properties in `expected` (newer; check your Node version)

`code/01_assert_basics.mjs` runs each assertion and prints PASS or FAIL:

```
import assert from 'node:assert/strict';

function check(label, fn) {
  try { fn(); console.log('PASS', label); }
  catch (err) { console.log('FAIL', label, '->', err.code, '|', err.message.split('\n')[0]); }
}

check('equal uses Object.is', () => assert.equal(0.1 + 0.2, 0.3));
check('equal numbers', () => assert.equal(2 + 2, 4));
check('equal objects (reference!)', () => assert.equal({ a: 1 }, { a: 1 }));
check('deepEqual objects', () => assert.deepEqual({ a: 1, tags: ['x'] }, { a: 1, tags: ['x'] }));
check('deepEqual is type-strict', () => assert.deepEqual({ n: 1 }, { n: '1' }));
// ... ok([].length), match, throws with { name, message }, notDeepEqual
check('throws but nothing thrown', () => assert.throws(() => 42));
check('float with tolerance', () => assert.ok(Math.abs(0.1 + 0.2 - 0.3) < Number.EPSILON));

try {
  assert.deepEqual({ id: 7, items: ['pen', 'ink'] }, { id: 7, items: ['pen', 'paper'] });
} catch (err) {
  console.log('--- AssertionError message ---\n' + err.message);
}

await assert.rejects(Promise.reject(new Error('timeout')), /timeout/);
console.log('PASS rejects');
```

Output:

```
FAIL equal uses Object.is -> ERR_ASSERTION | Expected values to be strictly equal:
PASS equal numbers
FAIL equal objects (reference!) -> ERR_ASSERTION | Values have same structure but are not reference-equal:
PASS deepEqual objects
FAIL deepEqual is type-strict -> ERR_ASSERTION | Expected values to be strictly deep-equal:
FAIL ok (truthy) -> ERR_ASSERTION | The expression evaluated to a falsy value:
PASS match regex
PASS throws with class + message
FAIL throws but nothing thrown -> ERR_ASSERTION | Missing expected exception.
PASS notDeepEqual
PASS float with tolerance
--- AssertionError message ---
Expected values to be strictly deep-equal:
+ actual - expected

  {
    id: 7,
    items: [
      'pen',
+     'ink'
-     'paper'
    ]
  }

PASS rejects
```

What to notice:

- `0.1 + 0.2` is not `0.3` in floating point (the approximate way computers store decimal numbers) (§2 Number) — compare floats with a tolerance, or keep money in integer cents (as the examples below do).
- `equal` on two separate objects fails even with identical contents; use `deepEqual` for structure.
- The diff (`+ actual - expected`) is what makes a failing test quick to diagnose — prefer one `deepEqual` on a whole object over many `equal`s on single fields.

TL;DR

Use `node:assert/strict`. `equal` for primitives/same reference, `deepEqual` for structures, `throws`/`await rejects` for errors, `match` for strings. A test fails when anything in it throws.

16.3 Writing and running tests with `node:test`

What it is

`node:test` is Node's built-in test runner (stable since Node 20). You import functions to *declare* tests; Node runs them and reports results. The `node:` prefix is mandatory for this module.

Function	Purpose
`test(name, [options], fn)` / alias `it`	One test. `fn` may be `async`; it receives a **test context** `t`
`describe(name, fn)` / alias `suite`	Group tests (can nest)
`before`/`after`, `beforeEach`/`afterEach`	Run once around the suite / around every test in it (and nested suites)
`t.test(name, fn)`	A **subtest** inside a test (`await` it)
`test.skip`, `test.todo`, `test.only` (+ `{ skip, todo, only, timeout }` options)	Skip; mark as not yet written (runs but failures don't count); run only these (with `--test-only`)
`t.mock`, `t.assert`, `t.plan(n)`, `t.signal`, `t.diagnostic(msg)`	Context helpers: auto-restoring mocks (§16.5), assertions incl. snapshots (§16.6), expected assertion count, an `AbortSignal` for cancellation, extra output

How it works

1. You run `node --test` (usually via `npm test`). With no arguments Node searches the current folder recursively (into every subfolder, skipping `node_modules` — the folder where installed packages live) for files named like `*.test.{js,mjs,cjs}`, `*-test.*`, `*_test.*`, `test-*.*`, `test.*`, or anything inside a `test/` folder — and the same patterns with `.ts/.mts/.cts`.
2. Each file runs in **its own child process** (a separate program Node starts) by default (isolation: one file's globals or mocks can't leak into another; `--test-isolation=none` runs them in one process for speed).
3. Inside a file, tests run **in order**; `describe` callbacks run immediately to collect tests; hooks wrap them.
4. The **reporter** prints results: `spec` (default, the tree below), `tap`, `dot`, `junit` (CI dashboards), `lcov` (coverage) — standard machine-readable report formats; `--test-reporter-destination` writes to a file.
5. The **exit code** is 1 if any test failed — what CI checks.

Useful commands:

```
node --test                              # all matching files under the cwd
node --test test/cart.test.mjs           # specific files or globs
node --test --watch                      # re-run on file changes
node --test --test-name-pattern="Cart"   # only tests whose name matches a regex
node --test --test-reporter=dot          # compact output (--test-only: run only test.only)
node cart.test.mjs                       # a single file also runs as a plain script
```

Example — a unit test suite

The module under test, `code/src/cart.mjs`, is a shopping cart that stores prices in integer cents, plus `applyDiscount`, `formatPrice` and an async `checkout`:

```
export class Cart {
  #items = new Map();                                  // sku -> { sku, price, qty }
  add(sku, price, qty = 1) {
    if (typeof sku !== 'string' || sku === '') throw new TypeError('sku must be a non-empty string');
    if (!Number.isInteger(price) || price < 0) throw new RangeError('price must be integer cents >= 0');
    if (!Number.isInteger(qty) || qty < 1) throw new RangeError('qty must be an integer >= 1');
    // ... merge into the existing line for this sku; return this
  }
  // remove(sku), get count(), lines() (copies), subtotal()
}
export function applyDiscount(cents, percent) {
  if (percent < 0 || percent > 100) throw new RangeError(`bad percent: ${percent}`);
  return Math.round(cents * (100 - percent) / 100);
}
export function formatPrice(cents, currency = 'USD', locale = 'en-US') { /* Intl.NumberFormat */ }
export async function checkout(cart, { priceService, now = () => new Date() }) { /* §16.5 */ }
```

`code/cart.test.mjs`:

```
import { describe, it, test, before, beforeEach, afterEach, after } from 'node:test';
import assert from 'node:assert/strict';
import { Cart, applyDiscount, formatPrice } from './src/cart.mjs';

describe('Cart', () => {
  let cart;
  beforeEach(() => { cart = new Cart(); });          // fresh object for every test

  it('starts empty', () => {
    assert.equal(cart.count, 0);
    assert.equal(cart.subtotal(), 0);
  });

  it('adds items and merges the same sku', () => {
    // Arrange - Act - Assert
    cart.add('pen', 150, 2).add('ink', 900).add('pen', 150);
    assert.equal(cart.count, 4);
    assert.deepEqual(cart.lines(), [
      { sku: 'pen', price: 150, qty: 3 },
      { sku: 'ink', price: 900, qty: 1 },
    ]);
    assert.equal(cart.subtotal(), 1350);
  });
  // ... 'removes items'

  describe('validation', () => {
    const bad = [
      ['', 100, 1, TypeError],
      ['pen', -1, 1, RangeError],
      ['pen', 1.5, 1, RangeError],
      ['pen', 100, 0, RangeError],
    ];
    for (const [sku, price, qty, ErrorType] of bad) {     // table-driven tests
      it(`rejects add(${JSON.stringify(sku)}, ${price}, ${qty})`, () => {
        assert.throws(() => cart.add(sku, price, qty), ErrorType);
      });
    }
  });

  // ... 'lines() returns copies, not internal state'
});

describe('applyDiscount', () => {
  it('takes a percentage off and rounds to cents', () => {
    assert.equal(applyDiscount(1000, 10), 900);
    assert.equal(applyDiscount(999, 15), 849);         // 849.15 -> 849
    assert.equal(applyDiscount(1000, 0), 1000);
    assert.equal(applyDiscount(1000, 100), 0);
  });
  it('rejects percentages outside 0-100', () => {
    assert.throws(() => applyDiscount(1000, 101), { name: 'RangeError', message: 'bad percent: 101' });
  });
});

test('formatPrice', async (t) => {                     // subtests with t.test
  await t.test('US dollars', () => assert.equal(formatPrice(123456), '$1,234.56'));
  await t.test('euros in Germany', () => assert.equal(formatPrice(123456, 'EUR', 'de-DE'), '1.234,56 €'));
});

test.skip('multi-currency carts', () => {});           // reported as skipped
test.todo('coupon codes');                               // reported as todo

describe('hooks order', () => {       // before/beforeEach/afterEach push to a log; after() prints it
  // ... it('one'), it('two')
});
```

Output of `node --test cart.test.mjs`:

```
# hooks ran: before > beforeEach > test1 > afterEach > beforeEach > test2 > afterEach
▶ Cart
  ✔ starts empty (0.425292ms)
  ✔ adds items and merges the same sku (0.421917ms)
  ✔ removes items (0.091708ms)
  ▶ validation
    ✔ rejects add("", 100, 1) (0.228958ms)
    ...
  ✔ validation (0.595709ms)
  ✔ lines() returns copies, not internal state (0.081625ms)
✔ Cart (2.060667ms)
...
﹣ multi-currency carts (0.020625ms) # SKIP
✔ coupon codes (0.016958ms) # TODO
...
ℹ tests 17
ℹ suites 4
ℹ pass 15
ℹ fail 0
...
ℹ skipped 1
ℹ todo 1
```

What each part shows:

1. **Arrange–Act–Assert (AAA)**: set up, do one action, check. One behaviour per test, named as a sentence so the report reads like a specification.
2. `beforeEach` **creates a fresh** `Cart`, so tests don't depend on each other's leftovers.
3. **Table-driven tests**: one named test per row of bad inputs — cheap to extend, each failure reported separately.
4. **Edge cases** (unusual or boundary inputs) are explicit: zero, 100 %, rounding (`849.15 → 849`), mutation of returned data.
5. The `'1.234,56 €'` expectation contains a **non-breaking space** (a special space character that stops a line break there) before the euro sign — locale output (formatting that depends on language and region settings) is a classic source of surprising failures.
6. The `after` hook's `console.log` printed at the very top: each file runs in a child process whose stdout is printed before the formatted results. Use `t.diagnostic()` for messages inside the report.

Common mistakes

- Shared mutable state between tests (a module-level `cart`) — order-dependent, flaky.
- Testing implementation details (`#items`) instead of public behaviour — every refactor breaks tests.
- Leaving `test.only` or `{ only: true }` in committed code. Node runs *all* tests anyway unless `--test-only` is passed (it prints "'only' and 'runOnly' require the --test-only command-line option"), but Jest and Vitest really do skip the others — a whole suite can silently stop running.
- Many assertions checking unrelated behaviours in one test — the first failure hides the rest.
- Tests with no assertion at all (they always pass). `t.plan(n)` can enforce a count when callbacks are involved.

TL;DR

`import { test, describe, it, beforeEach } from 'node:test'`, run `node --test`. One behaviour per test, AAA, fresh fixtures (the prepared objects and data a test starts from) in `beforeEach`, table-driven tests for many inputs, `skip`/`todo` for known gaps. Exit code 1 on failure.

16.4 Async tests and how tests fail

How async tests work

If a test returns a promise (any `async` function does), the runner **waits**: fulfilled = pass, rejected = fail. So `await` everything inside a test, especially `await assert.rejects(...)`. A test also fails if it

exceeds its `timeout` option (default: none — set one for code that might hang).

Example — three ways to fail

`code/demo_failing.mjs` (skip-marked; run `node --test demo_failing.mjs`):

```
test('a wrong expectation', () => {
  assert.equal(applyDiscount(999, 15), 850);          // real answer is 849
});

test('a test that hangs', { timeout: 50 }, async () => {
  await new Promise((r) => setTimeout(r, 1000));
});

test('forgot to await the rejection', async () => {
  assert.rejects(Promise.resolve('no error'));          // missing await -> failure escapes the test
});
```

Output (stack trace shortened with `…`):

```
✖ a wrong expectation (0.612875ms)
✖ a test that hangs (51.228334ms)
✔ forgot to await the rejection (0.396875ms)
ℹ Error: Test "forgot to await the rejection" at demo_failing.mjs:14:1 generated asynchronous activity after the
test ended. This activity created the error "AssertionError [ERR_ASSERTION]: Missing expected rejection." and
would have caused the test to fail, but instead triggered an unhandledRejection event.
ℹ tests 3
...
ℹ pass 1
ℹ fail 1
ℹ cancelled 1
...

✖ failing tests:

test at demo_failing.mjs:6:1
✖ a wrong expectation (0.612875ms)
  AssertionError [ERR_ASSERTION]: Expected values to be strictly equal:

  849 !== 850

      at TestContext.<anonymous> (file:///…/16-testing-and-typescript/code/demo_failing.mjs:7:10)
      …
    code: 'ERR_ASSERTION',
    actual: 849,
    expected: 850,
    ...
  }

test at demo_failing.mjs:10:1
✖ a test that hangs (51.228334ms)
  'test timed out after 50ms'
exit=1
```

How to read it:

1. **Wrong expectation** — actual vs expected plus the assertion's file:line. Decide whether the code or the test is wrong (here the test: 999 × 0.85 = 849.15).
2. **Timeout** — reported as *cancelled*; usually a promise that never settles or a server never closed.
3. **Missing `await`** — the dangerous one: marked **✔ passed** because the test finished before the assertion's promise rejected; Node only printed a warning that a busy CI log hides. *Every* `assert.rejects` and async call in a test gets `await` (`@typescript-eslint/no-floating-promises` catches this).

TL;DR

Make tests `async` and `await` everything; set `timeout` for code that could hang; a missing `await` can make a failing test look green.

16.5 Test doubles and mocking

What it is

A **test double** is a stand-in for a real dependency (network API, database, clock, random numbers) so the unit under test runs fast, deterministically (same result every run), and offline. Terms you'll meet:

Double	What it is	Example
Dummy	Passed but never used	`{}` for an unused options argument
Stub	Returns canned answers	`getDiscount: async () => ({ percent: 10 })`
Spy	Records how it was called (and may call through)	`mock.fn()` → `.mock.calls`
Mock	A spy with expectations about calls, verified in the test	"called once with 500"
Fake	A working lightweight implementation	An in-memory repository (an object that loads and saves records) instead of Postgres (a popular SQL database)

In everyday speech people say "mock" for all of these.

Why it matters — and dependency injection

The easiest code to test takes its dependencies **as parameters** (**dependency injection**) instead of importing and calling them directly. `checkout` receives `priceService` and `now`:

```
export async function checkout(cart, { priceService, now = () => new Date() }) {
  if (cart.count === 0) throw new Error('cart is empty');
  const { percent } = await priceService.getDiscount(cart.subtotal());
  return { total: applyDiscount(cart.subtotal(), percent), at: now().toISOString() };
}
```

Production passes the real HTTP client and the real clock; tests pass a stub and a fixed clock — no mocking library needed.

The `node:test` mocking API

API	Does
`mock.fn(impl?)`	A spy function; `fn.mock.calls` (each `{ arguments, result, error, this }`), `fn.mock.callCount()`
`fn.mock.mockImplementation(f)` / `mockImplementationOnce(f)`	Change behaviour (for all / the next call)
`mock.method(obj, 'name', impl?)`	Replace a method on an existing object, keep a spy on it
`mock.getter` / `setter` / `property`	Replace accessors / a property value
`t.mock.*`	Same, but **auto-restored when the test ends** — prefer it over the global `mock` (which needs `mock.restoreAll()`)
`t.mock.timers.enable({ apis: ['setTimeout', 'Date', …], now })` + `tick(ms)`	**Fake timers** — control `setTimeout`/`setInterval`/`setImmediate`/`Date` without waiting
`t.mock.module(specifier, { namedExports, defaultExport })`	Replace what `import` returns (needs `--experimental-test-module-mocks`)

`code/checkout.test.mjs`:

```
import { describe, it, mock } from 'node:test';
import assert from 'node:assert/strict';
import { Cart, checkout } from './src/cart.mjs';

const fixedNow = () => new Date('2026-01-15T10:00:00Z');   // an injected, fixed clock

describe('checkout (async + dependency injection)', () => {
  it('applies the discount returned by the price service (fake object)', async () => {
    const fakePriceService = { getDiscount: async () => ({ percent: 10 }) };
    const cart = new Cart().add('pen', 1000);
    const receipt = await checkout(cart, { priceService: fakePriceService, now: fixedNow });
    assert.deepEqual(receipt, { total: 900, at: '2026-01-15T10:00:00.000Z' });
  });
  // ... 'rejects an empty cart', 'propagates service failures' (await assert.rejects)
  it('calls the service once with the subtotal (mock.fn spy)', async () => {
    const getDiscount = mock.fn(async () => ({ percent: 0 }));
    await checkout(new Cart().add('ink', 250, 2), { priceService: { getDiscount }, now: fixedNow });
    assert.equal(getDiscount.mock.callCount(), 1);
    assert.deepEqual(getDiscount.mock.calls[0].arguments, [500]);
  });
});

describe('mock.method and t.mock', () => {
  const api = { async fetchUser(id) { throw new Error('real network call - must not run in tests'); } };
  it('replaces a method for one test; t.mock restores it automatically', async (t) => {
    const m = t.mock.method(api, 'fetchUser', async (id) => ({ id, name: 'Test User' }));
    assert.deepEqual(await api.fetchUser(7), { id: 7, name: 'Test User' });
    assert.equal(m.mock.callCount(), 1);
  });
  // ... 'after the previous test the real method is back' (assert.rejects on the real one)
  it('mockImplementationOnce: fail first, then succeed (retry logic)', async (t) => {
    const flaky = t.mock.fn(async () => 'ok');
    flaky.mock.mockImplementationOnce(async () => { throw new Error('ECONNRESET'); });
    assert.equal(await withRetry(flaky), 'ok');      // withRetry: try up to 3 times
    assert.equal(flaky.mock.callCount(), 2);
  });
});

describe('fake timers', () => {
  function debounce(fn, ms) {
    let id;
    return (...args) => { clearTimeout(id); id = setTimeout(() => fn(...args), ms); };
  }
  it('debounce calls once, after the quiet period - without really waiting', (t) => {
    t.mock.timers.enable({ apis: ['setTimeout'] });
    const save = t.mock.fn();
    const debouncedSave = debounce(save, 300);
    debouncedSave('a'); debouncedSave('ab'); debouncedSave('abc');
    t.mock.timers.tick(299);
    assert.equal(save.mock.callCount(), 0);
    t.mock.timers.tick(1);
    assert.equal(save.mock.callCount(), 1);
    assert.deepEqual(save.mock.calls[0].arguments, ['abc']);
  });
  // ... 'Date can be frozen too': enable({ apis: ['Date'], now: new Date('2030-05-01T00:00:00Z') })
});
```

Output of `node --test checkout.test.mjs`:

```
▶ checkout (async + dependency injection)
  ✔ applies the discount returned by the price service (fake object) (1.343458ms)
  ...
✔ checkout (async + dependency injection) (2.544709ms)
▶ mock.method and t.mock
  ...
▶ fake timers
  ✔ debounce calls once, after the quiet period - without really waiting (0.290834ms)
  ✔ Date can be frozen too (0.12175ms)
✔ fake timers (0.520417ms)
ℹ tests 9
ℹ suites 3
ℹ pass 9
ℹ fail 0
...
```

The debounce test checks a 300 ms delay in under half a millisecond: fake timers move time only when you call `tick`. (Debounce itself is built in Chapter 17.)

Module mocking

When code imports a dependency directly (no injection), you can replace the **module** itself. `code/module_mock_demo.mjs` mocks the built-in `node:os` that `src/greeting.mjs` imports:

```
test('greeting uses the mocked os module', async (t) => {
  t.mock.module('node:os', { namedExports: { userInfo: () => ({ username: 'ana' }) } });
  const { greeting } = await import('./src/greeting.mjs');   // import AFTER mocking
  assert.equal(greeting(), 'Hello, ana!');
});
```

Output of `node --test --experimental-test-module-mocks module_mock_demo.mjs`:

```
(node:77997) ExperimentalWarning: Module mocking is an experimental feature and might change at any time
(Use `node --trace-warnings ...` to show where the warning was created)
✔ greeting uses the mocked os module (2.704792ms)
ℹ tests 1
...
ℹ pass 1
ℹ fail 0
...
```

Module mocking is still **experimental** in Node 24 (hence the flag and warning), and the module must be imported *after* the mock is set up — hence the dynamic `import()` (Vitest's `vi.mock` and Jest's `jest.mock` hoist — automatically move — mocks above imports for you). Prefer dependency injection; use module mocks when you can't change the code.

When not to mock

- **Don't mock what you own and can run cheaply** (pure functions — no side effects, same input always gives the same output — and your own small modules) — use the real thing.
- **Don't mock the unit under test** — you would be testing the mock.
- Too many mocks give tests that pass while production fails (doubles drift from reality) — balance with integration tests against real (containerized) databases and **contract tests** (tests checking that both sides of an API still agree on its request/response format) for external APIs.
- For HTTP, prefer a real local server (§15.10's port-0 pattern) or an interception library (one that catches outgoing requests and answers them: MSW, undici `MockAgent`) over hand-mocking `fetch`.

In the interview

Q: Stub vs mock vs fake? "A stub returns canned data; a mock (or spy) records calls so you can assert on them; a fake is a simple working implementation, like an in-memory DB. I prefer injecting dependencies so tests can pass fakes without a mocking framework."

- *How do you test code using `setTimeout` or `Date.now()`?* Fake timers (`t.mock.timers`), or inject a clock function.

- *What makes tests flaky?* Real time, real network, shared state between tests, test order, unawaited promises, locale/timezone differences.

> **TL;DR**

Inject dependencies and pass stubs/fakes; use `t.mock.fn` / `t.mock.method` for spies (auto-restored); fake timers for time; module mocks as a last resort. Don't over-mock.

16.6 Snapshot tests and code coverage

Snapshot testing

A **snapshot test** saves a function's output the first time and fails later if it changes — good for large, stable outputs (a receipt, a CLI help screen) that are tedious to write by hand. In Node, `t.assert.snapshot(value)` is **stable since Node 23.4**.

`code/snapshot.test.mjs`:

```
function receiptText(cart) {
  const rows = cart.lines().map((l) => `${l.sku.padEnd(6)} x${l.qty}  ${(l.price * l.qty /
100).toFixed(2).padStart(7)}`);
  return [...rows, '-'.repeat(20), `TOTAL ${(cart.subtotal() / 100).toFixed(2).padStart(13)}`].join('\n');
}
test('receipt layout', (t) => {
  const cart = new Cart().add('pen', 150, 3).add('ink', 900);
  t.assert.snapshot(receiptText(cart));
});
```

Workflow:

1. First run: `node --test --test-update-snapshots snapshot.test.mjs` writes `snapshot.test.mjs.snapshot`:

   ```
   exports[`receipt layout 1`] = `
   "pen    x3     4.50\\nink    x1     9.00\\n--------------------\\nTOTAL         13.50"
   `;
   ```

2. Commit the `.snapshot` file. Later runs compare against it; without the file, the test fails with "Missing snapshots can be generated by rerunning the command with the --test-update-snapshots flag".
3. When the output changes **on purpose**, rerun with `--test-update-snapshots` and review the diff (line-by-line comparison of old and new) in code review (teammates reading a change before it's merged).

The danger: updating snapshots without reading them. Keep them small and review them like code.

Code coverage

Coverage measures which lines, branches (each possible path through an `if`/`else`, `switch` or `?:`) and functions your tests executed. Output of `node --test --experimental-test-coverage` in `code/` (all test files):

```
ℹ start of coverage report
ℹ ---------------------------------------------------------
ℹ file      | line % | branch % | funcs % | uncovered lines
ℹ ---------------------------------------------------------
ℹ src       |        |          |         |
ℹ  cart.mjs | 100.00 |   100.00 |   90.91 |
ℹ  money.ts | 100.00 |    88.89 |  100.00 |
ℹ ---------------------------------------------------------
ℹ all files | 100.00 |    97.14 |   93.33 |
ℹ ---------------------------------------------------------
ℹ end of coverage report
```

(Test files are excluded by default.) `cart.mjs` shows 90.91 % of functions: the default clock `now = () => new Date()` never ran, because every test injected its own — coverage points at code no test

exercises. Flags: `--test-coverage-include`/`-exclude`, `--test-coverage-lines=80` (fail below a threshold), `--test-reporter=lcov` for CI services. The flag still says "experimental" in Node 24.

Coverage is a **floor, not a goal**: 100 % means every line *ran*, not that results were *checked* — use it to find untested areas, don't chase the number.

> **TL;DR**
>
> Snapshots record output and flag changes — commit and review them. Coverage (`--experimental-test-coverage`) shows unexecuted code; treat it as a hint, not a target.

16.7 Good tests, and other runners

What makes a good test (F.I.R.S.T. and more)

Property	Meaning
Fast	Milliseconds; slow suites stop being run
Independent/Isolated	No shared state, any order
Repeatable	Same result every run: fixed clock, seeded randomness (random numbers generated from a fixed starting value, so they repeat), no real network
Self-validating	Pass/fail by assertions, not by a human reading logs
Timely	Written with (or before — **TDD**, test-driven development: red → green → refactor) the code
Behaviour-focused	Tests public behaviour; survives refactoring
Readable	Named as sentences; AAA structure; a failure message says what broke

Same ideas in Vitest and Jest

Concept	node:test	Vitest	Jest
Import	`from 'node:test'` + `node:assert/strict`	`from 'vitest'` (or globals)	globals
Test / group	`test`/`it`, `describe`	same	same
Assert equal	`assert.equal(a, b)`	`expect(a).toBe(b)`	same
Deep equal	`assert.deepEqual`	`expect(a).toEqual(b)` / `toStrictEqual`	same
Throws	`assert.throws(fn, Err)`	`expect(fn).toThrow(Err)`	same
Rejects	`await assert.rejects(p)`	`await expect(p).rejects.toThrow()`	same
Spy	`t.mock.fn()`	`vi.fn()`	`jest.fn()`
Method mock	`t.mock.method(o, 'm')`	`vi.spyOn(o, 'm')`	`jest.spyOn`
Fake timers	`t.mock.timers.enable()` / `tick`	`vi.useFakeTimers()` / `vi.advanceTimersByTime`	`jest.useFakeTimers()`
Module mock	`t.mock.module` (flag)	`vi.mock()` (hoisted)	`jest.mock()` (hoisted)
Snapshot	`t.assert.snapshot`	`toMatchSnapshot()`	same
Run	`node --test`	`vitest`	`jest`

TL;DR

Fast, isolated, repeatable, behaviour-focused, readable tests. The API names change between runners; the concepts don't.

16.8 TypeScript: what it is and how it's checked

What it is

TypeScript (TS) is JavaScript plus a **static type system** (rules about what kind of value each variable may hold, checked without running the code), developed by Microsoft since 2012. You add **type annotations** (`let pages: number`), and the **type checker** (the compiler `tsc`, or your editor using it) verifies that values are used consistently — *before* the code runs. The types are then **erased**: what executes is ordinary JavaScript. TS has **no runtime** of its own (nothing of TypeScript exists while the program runs).

Key terms:

- **Static typing**: types are checked by analyzing code, not by running it (JavaScript itself is dynamically typed — §2).
- **Type inference**: TS works out many types by itself (`let pages = 30` is `number`), so you annotate mainly function parameters and public APIs.
- **Structural typing**: compatibility is decided by **shape**, not by name. Any object with `{ id: number; name: string }` is a valid `User`, whether or not it was declared as one ("duck typing" checked at compile time).
- **Gradual typing**: you can adopt TS file by file; `any` switches checking off for a value.
- **Soundness**: TS is deliberately not fully sound — `any`, assertions and some index operations let wrong types through. Types are strong evidence, not proof.

Analogy: a spell-checker for program structure. It underlines mistakes as you type; the printed letter (the JavaScript) contains no red lines.

Why it matters

Catches typos, wrong argument orders, missing `null` checks and forgotten cases at edit time; powers accurate autocomplete (the editor suggesting names as you type) and safe renames; documents APIs. It is the industry default for serious front-end and Node projects (React, Angular, Vue and Next.js all support it first-class).

Trade-offs

Benefit	Cost
Bugs caught before running; editor support	A check step and a `tsconfig.json`; a learning curve for advanced types
Self-documenting APIs	Types lie if you misuse `any` / `as`, and never validate **runtime** data (API JSON still needs a check or a schema library — one that validates data against a described shape — such as Zod)

`tsc` and TypeScript 7

`tsc` is the TypeScript compiler. Its two jobs are **type-checking** and **emitting** JavaScript (writing out `.js` files). Today, emitting is often left to faster tools (esbuild, Vite, SWC — very fast compilers and build tools written in Go or Rust, or Node's type stripping, §16.9) and `tsc` is run only for checking: `tsc --noEmit`.

TypeScript 7.0 (released 8 July 2026) is a **native port of the compiler written in Go** (a rewrite in Go, a compiled language, instead of JavaScript); Microsoft reports full builds roughly 8–12× faster than TypeScript 6, with the same `typescript` npm package and `tsc` command. It also changes defaults and removes legacy options:

In TypeScript 7	Details
New defaults	`strict: true`, `module: esnext`, `target`: the current stable ECMAScript version, `types: []` (you list `@types` packages — community-written type definitions for JavaScript libraries — explicitly), `noUncheckedSideEffectImports: true`
Removed	`target: es5`, `downlevelIteration`, `moduleResolution: node` / `node10` / `classic`, `module: amd/umd/systemjs/none`, `baseUrl` (use `paths`)
Not yet	A stable programmatic (callable from your own code; JavaScript) API — expected in 7.1; tools that call the compiler's API may lag

Everyday commands (not run — the project in `code/` already has TypeScript available):

```
npm install -D typescript @types/node   # compiler + Node's type definitions
npx tsc --init                          # create a tsconfig.json
npx tsc --noEmit                        # type-check the project (what CI runs)
npx tsc --noEmit --watch                # re-check on every save
```

`tsconfig.json`

The project's `code/tsconfig.json`:

```
{
  "compilerOptions": {
    "target": "es2024",
    "lib": ["es2024", "dom"],
    "module": "nodenext",
    "strict": true,
    "noEmit": true,
    "noUncheckedIndexedAccess": true,
    "allowImportingTsExtensions": true,
    "erasableSyntaxOnly": true,
    "verbatimModuleSyntax": true,
    "allowJs": true,
    "checkJs": false
  },
  "include": ["*.ts", "src/*.ts", "15_jsdoc.mjs"],
  "exclude": ["ts_errors.ts", "no_typecheck_demo.ts", "*.test.ts"]
}
```

Option	Why
`target`, `lib`	Output syntax and available built-in APIs (`dom` provides `console`, as `@types/node` isn't installed)
`module: nodenext`	Follow Node's real module rules (reads `"type"` from `package.json`)
`strict`	`strictNullChecks`, `noImplicitAny`, … — always on (default in TS 7)
`noEmit`	Check only; something else runs the code
`noUncheckedIndexedAccess`	`arr[i]` and `record[key]` are `T | undefined` — catches out-of-range reads
`allowImportingTsExtensions`	Allow `import './shapes.ts'` (Node requires the real extension)
`erasableSyntaxOnly`	Reject TS features that need code generation (enum — a set of named constants; namespaces with code; parameter properties) — exactly what Node can't strip (TS 5.8+)
`verbatimModuleSyntax`	Force `import type` for type-only imports, so removing types never changes which modules load

Running the checker on the whole project:

```
$ tsc --noEmit
$ echo $?
0
```

No output and exit code 0 means no type errors.

TL;DR

TypeScript = JavaScript + static types that are checked by `tsc` and erased before running. Use `strict`, check with `tsc --noEmit`. TypeScript 7 is the Go-native, ~10× faster compiler with stricter defaults.

16.9 Running TypeScript: Node's type stripping and other tools

What it is

Since **Node 23.6** (and backported — also added — to the older 22.18 line), `node file.ts` works with no flags: Node **strips the type annotations** — replacing them with whitespace, so line and column numbers stay correct without source maps (files mapping generated code back to the original lines) — and runs the remaining JavaScript. The feature stopped printing an experimental warning in Node 24.3 and was declared **stable in Node 24.12**.

Stripping is not checking. `code/no_typecheck_demo.ts`:

```ts
const price: number = 5;
const qty: number = '2';                  // type error: string is not assignable to number
const total: number = price + qty;        // JS concatenates: 5 + '2' -> '52'
console.log('total =', JSON.stringify(total), '| typeof', typeof total);
```

Output of `node no_typecheck_demo.ts`, then `tsc` on the same file:

```
total = "52" | typeof string
no_typecheck_demo.ts(4,7): error TS2322: Type 'string' is not assignable to type 'number'.
```

Node ran code whose `number` annotation holds the string `"52"`. **Always run** `tsc --noEmit` (editor, pre-commit — a hook that runs before every Git commit — and CI) as well.

The rules for Node's type stripping

1. **Only erasable syntax.** Things that generate JavaScript are rejected with `ERR_UNSUPPORTED_TYPESCRIPT_SYNTAX`: `enum`, `namespace`/`module` blocks containing code, **parameter properties** (`constructor(public x: number)`), `import x = require()` aliases, and legacy decorators (`@name` annotations placed on classes and members). (`--experimental-transform-types` enables them, with a warning.) Turn on `erasableSyntaxOnly` so the editor flags them.
2. **Write real file extensions** in relative imports: `import { area } from './shapes.ts'`.
3. **Use** `import type` (or inline `type` modifiers) for imports that are only types (`verbatimModuleSyntax` enforces it).
4. `tsconfig.json` **is ignored** by Node — no `paths` aliases (short import names mapped to folders), no `target` downlevelling (rewriting new syntax for older engines).
5. **Not in** `node_modules`: Node refuses to strip TypeScript files inside dependencies; packages must publish JavaScript (+ `.d.ts` type declarations).
6. `.ts` follows the same ESM/CommonJS rules as `.js`; `.mts`/`.cts` force one.

A first draft of `11_narrowing.ts` used a parameter property and hit rule 1:

```
class HttpError extends Error { constructor(public status: number) { super(`HTTP ${status}`); } }
                                                ^^^^^^^^^^^^^^^
SyntaxError [ERR_UNSUPPORTED_TYPESCRIPT_SYNTAX]: TypeScript parameter property is not supported in strip-only mode
```

`tsc` with `erasableSyntaxOnly` reports it as `TS1294`. Fix: declare the field and assign it in the constructor.

`code/14_node_runs_ts.ts` follows all the rules:

```
import type { Shape } from './shapes.ts';
import { area } from './shapes.ts';

const shapes: Shape[] = [
  { kind: 'rect', w: 2, h: 5 },
  { kind: 'circle', r: 1 },
];
const total: number = shapes.reduce((sum, s) => sum + area(s), 0);
console.log('total area:', total.toFixed(3));
console.log('shapes:', shapes.map((s): string => s.kind).join(', '));
```

Output of `node 14_node_runs_ts.ts`:

```
total area: 13.142
shapes: rect, circle
```

Other ways to run TypeScript

Tool	What it does	Type-checks?
`node file.ts`	Strips erasable types	No
`tsx` (npm)	esbuild-powered runner; supports all TS syntax and `tsconfig` paths	No
`esbuild` / `Vite` / `SWC`	Transpile (convert to plain JavaScript) or bundle (merge into a few files) TS for browsers and Node (this book's runner uses esbuild)	No
`tsc`	Checks, and can emit `.js` + `.d.ts` (needed when publishing libraries)	**Yes**
`Deno` / `Bun`	Run TS natively (§15.12); `deno check` type-checks	Deno: on request

Typical setup: *run* with Node/tsx/Vite, *check* with `tsc --noEmit`, *publish libraries* with `tsc` emitting declarations.

TL;DR

`node app.ts` works in Node 24 (stable type stripping) but never type-checks. Use only erasable syntax, `.ts` extensions in imports, `import type`, and keep `tsc --noEmit` in CI.

16.10 Everyday types

How it works

`code/10_ts_basics.ts` walks through the everyday types (key parts):

```ts
// 1) Primitive annotations - and inference
let title: string = 'Testing & TypeScript';
let pages = 30;                          // inferred: number
const lang = 'ts';                       // inferred: the LITERAL type "ts" (const)
// ... also bigint and null annotations

// 2) Arrays and tuples
const scores: number[] = [90, 72, 85];
const point: [x: number, y: number] = [3, 4];          // tuple: fixed length & types

// 3) Object types: type alias vs interface; optional and readonly properties
type User = {
  readonly id: number;
  name: string;
  email?: string;                        // optional: string | undefined
};
interface Product { sku: string; price: number }
interface Product { tags: string[] }     // interfaces MERGE (declaration merging)

// 4) Functions: parameter & return types, default params, void
function greet(user: User, greeting = 'Hello'): string { return `${greeting}, ${user.name}!`; }

// 5) Union and literal types: type Status = 'draft' | 'published' | 'archived'; let id: number | string

// 6) any vs unknown: unknown forces you to check before use
function parse(json: string): unknown { return JSON.parse(json); }
const data = parse('{"count": 3}');
if (typeof data === 'object' && data !== null && 'count' in data && typeof data.count === 'number') {
  console.log('6) count + 1 =', data.count + 1);
}

// 7) Type assertions and the non-null assertion - you override the checker (use sparingly)
const maybe = scores.find((n) => n > 80);           // number | undefined
console.log('7)', (maybe as number).toFixed(1), maybe!.toFixed(0));
// 8) ... types disappear at runtime
```

Output of `node 10_ts_basics.ts`:

```
1) string number ts 10000000000000000000n null
2) 3 Ana+Bo 5
3) (no email) office
4) Hello, Ana!
4) Hi, Bo!
5) [LIVE] [DRAFT] A-42
6) count + 1 = 4
7) 90.0 90
8) typeof User at runtime: undefined
```

The type vocabulary

Type	Meaning
`string`, `number`, `boolean`, `bigint`, `symbol`, `null`, `undefined`	The JS primitives (lower-case; `String` is the wrapper object)
`'draft'`, `42`, `true`	**Literal types** — exactly that value
`A | B`	**Union** — either
`A & B`	**Intersection** — both at once (combine object types)
`T[]` / `Array<T>`	Array
`[string, number]`	**Tuple** — fixed length, typed positions
`{ a: string; b?: number }`	Object type; `?` optional, `readonly` prevents reassignment
`(x: number) => string`	Function type
`any`	Opt out of checking — anything goes, errors spread silently
`unknown`	"Could be anything" — you must narrow it before use. **Use for untrusted input** (JSON, `catch (e)`)
`never`	No value possible (a function that always throws; the impossible branch of a switch)
`void`	A function returns nothing useful
`object`	Any non-primitive

`type` **vs** `interface`: both describe object shapes. `interface` can be **reopened** (declaration merging, used to extend library types) and `extends` others; `type` can also name unions, tuples and computed types (types derived from other types, §16.12). Many teams use `interface` for object shapes and `type` for the rest — consistency matters more than the choice.

Seeing the checker work

`code/ts_errors.ts` (skip-marked) contains nine deliberate mistakes:

```ts
type User = { id: number; name: string; email?: string };

const u: User = { id: 1, name: 'Ana', age: 30 };              // 1) excess property
u.id = '2';                                                   // 2) wrong type
console.log(u.email.toLowerCase());                           // 3) possibly undefined

function total(prices: number[]): number {
  return prices.reduce((a, b) => a + b, '');                  // 4) string is not number
}
total(['1.50', '2.00']);                                      // 5) string[] is not number[]

type Status = 'draft' | 'published';
const s: Status = 'publised';                                 // 6) typo caught

function len(x: string | number) {
  return x.length;                                            // 7) number has no length
}

class Point { constructor(private x: number) {} }             // 8) not erasable (Node can't strip)
enum Color { Red, Green }                                     // 9) not erasable either
```

Output of `tsc --ignoreConfig --noEmit --strict --target es2024 --erasableSyntaxOnly ts_errors.ts`:

```
ts_errors.ts(4,39): error TS2353: Object literal may only specify known properties, and 'age' does not exist in
type 'User'.
ts_errors.ts(5,1): error TS2322: Type 'string' is not assignable to type 'number'.
ts_errors.ts(6,13): error TS18048: 'u.email' is possibly 'undefined'.
ts_errors.ts(9,3): error TS2322: Type 'string' is not assignable to type 'number'.
...
ts_errors.ts(14,7): error TS2820: Type '"publised"' is not assignable to type 'Status'. Did you mean
'"published"'?
ts_errors.ts(17,12): error TS2339: Property 'length' does not exist on type 'string | number'.
  Property 'length' does not exist on type 'number'.
ts_errors.ts(20,27): error TS1294: This syntax is not allowed when 'erasableSyntaxOnly' is enabled.
...
```

Each line is `file(line,column): error TSxxxx: message` (`--ignoreConfig` is needed because TypeScript 7 won't combine a present `tsconfig.json` with file names on the command line). Error 3 is the famous one: with `strict`, "possibly `undefined`" catches `TypeError: Cannot read properties of undefined` (§11.10) before it happens. Error 1, the **excess property check**, applies only to object literals (objects written out inline with `{ … }`) written where a type is expected; an object stored in a variable first may carry extra properties (structural typing).

Common mistakes

- `any` to silence an error — it spreads to everything derived from it. Use `unknown` and narrow.
- Overusing `as` and `!` — when you're wrong, the runtime error comes back.
- Annotating everything instead of letting inference handle locals.
- Trusting types for external data: `(await res.json()) as User` checks nothing. Validate at the boundary (schema library or a hand-written guard).

TL;DR

Annotate parameters and public APIs; let inference do the rest. Unions and literal types model real states; `unknown` for untrusted data, avoid `any`; `strict` catches `undefined` bugs.

16.11 Narrowing

What it is

Narrowing is how TS refines a broad type (a union, `unknown`) to a more specific one inside a branch, based on the checks you write. The checks are ordinary JavaScript — TS just understands what they prove.

Check	Narrows by
`typeof x === 'string'`	Primitive type
`x === null`, `x !== undefined`, truthiness `if (x)`	Removing `null`/`undefined` (careful: truthiness also removes `0` and `''`)
`x instanceof Date`	Class
`'code' in x`	Property existence
`s.kind === 'circle'`	**Discriminant** property of a union
`function isCat(p): p is Cat`	Your own **type predicate**
`function assertX(v): asserts v is X`	**Assertion function** — narrows everything after the call

Discriminated unions and exhaustiveness

A **discriminated (tagged) union** is a union of object types that share one literal property (the **discriminant**, here `kind`). Checking it tells TS exactly which member you have. Assigning the leftover

to `never` in `default` makes the compiler **force you to handle every case**: add `{ kind: 'hexagon' }` to `Shape` and `area` stops compiling until you add a branch.

`code/11_narrowing.ts` (key parts):

```
type Shape =
  | { kind: 'circle'; radius: number }
  | { kind: 'rect'; width: number; height: number }
  | { kind: 'triangle'; base: number; height: number };

function area(s: Shape): number {
  switch (s.kind) {
    case 'circle': return Math.PI * s.radius ** 2;
    case 'rect': return s.width * s.height;
    case 'triangle': return (s.base * s.height) / 2;
    default: {
      const unreachable: never = s;        // compile error here if a new kind is not handled
      throw new Error(`unknown shape ${JSON.stringify(unreachable)}`);
    }
  }
}

type Result<T> = { ok: true; value: T } | { ok: false; error: string };   // parsePort(text): Result<number>

function isCat(pet: Cat | Dog): pet is Cat { return 'meow' in pet; }   // type predicate

function assertIsString(v: unknown, name: string): asserts v is string {
  if (typeof v !== 'string') throw new TypeError(`${name} must be a string`);
}
```

Output of `node 11_narrowing.ts`:

```
1) text of length 3 | number 3.14 | nothing
2) http 404 | code ENOENT | error x
3) circle=3.14, rect=6.00, triangle=10.00
4) port 8080
4) bad port "http"
5) meow woof
5) after assert: HELLO
6) [ 'Pune', 'unknown' ]
```

Real-world examples

Redux (a popular state-management library) actions (`{ type: 'added' } | { type: 'removed' }`, React Handbook ch6), API responses (`{ status: 'loading' } | { status: 'success'; data } | { status: 'error'; error }`), and `Result` types in libraries that avoid exceptions are all discriminated unions.

In the interview

Q: How do you make sure every case of a union is handled? "Model it as a discriminated union, switch on the discriminant, and assign the value to a `never`-typed variable in `default`. Adding a new member makes that assignment a compile error."

- *What is a type predicate?* A function returning `x is T`; when it returns true, TS narrows the argument to `T` in the caller.

Self-check

1. After `if (typeof x === 'string') return;`, what is `x` if its type was `string | number`? → `number`.
2. Why prefer `x !== undefined` over `if (x)` for a `number | undefined`? → Truthiness would also drop `0`.
3. What does `asserts v is string` do? → After the call returns, `v` is treated as `string`; the function must throw otherwise.

TL;DR

TS narrows through `typeof`, `instanceof`, `in`, equality, and discriminants. Model states as discriminated unions; use `never` for exhaustive switches; write type predicates and assertion functions for custom

checks.

16.12 Generics and utility types

What it is

A **generic** is a type parameter — a placeholder (`T`) filled in when the function or type is used. It lets one function work for many types **without losing** the specific type (as `any` would).

Analogy: a labelled storage box. A `Box<Book>` and a `Box<Plate>` are built the same way, but the label tells you what comes out.

`code/12_generics.ts` (key parts):

```ts
function first<T>(items: readonly T[]): T | undefined { return items[0]; }
const n = first([10, 20]);          // number | undefined  (T inferred)

function longest<T extends { length: number }>(a: T, b: T): T { /* ... */ }   // constraint

function pluck<T, K extends keyof T>(rows: T[], key: K): T[K][] {   // keyof + indexed access
  return rows.map((r) => r[key]);
}
pluck(users, 'name');               // string[]  -  pluck(users, 'nme') would not compile

class TypedCache<K, V> { /* set(k: K, v: V): this; get(k: K): V | undefined - wraps a Map<K, V> */ }

interface Todo { id: number; title: string; done: boolean; tags: string[] }
type TodoPatch = Partial<Omit<Todo, 'id'>>;
type ById = Record<number, Pick<Todo, 'id' | 'title'>>;

type Flags<T> = { [K in keyof T]: boolean };              // mapped type
type ElementOf<T> = T extends readonly (infer E)[] ? E : never;   // conditional type
```

Output of `node 12_generics.ts`:

```
1) 10 a
2) apple [ 1, 2, 3 ]
3) [ 'Ana', 'Bo' ] [ true, false ]
4) 2 3
5) true write ch16
5) [ 1, 'x' ] true
6) { title: true, done: false } urgent
```

Built-in utility types

Utility	Result
`Partial<T>` / `Required<T>`	All properties optional / required
`Readonly<T>`	All properties `readonly` (shallow)
`Pick<T, K>` / `Omit<T, K>`	Keep / drop some properties
`Record<K, V>`	Object with keys `K` and values `V`
`Exclude<U, X>` / `Extract<U, X>`	Remove / keep union members
`NonNullable<T>`	Remove `null` and `undefined`
`ReturnType<F>` / `Parameters<F>`	A function's return type / parameter tuple
`Awaited<T>`	The value a promise resolves to

Mapped types (`{ [K in keyof T]: … }`) transform every property; **conditional types** (`T extends X ? A : B`, with `infer`) pick types from other types. Utility types are built from them; application code rarely needs them.

Common mistakes

- Generic parameters that appear only once (`function f<T>(x: T): void`) — then `unknown` is simpler.
- `any[]` instead of `T[]` — loses the element type.
- Deep type gymnastics in application code — hard to read; keep types boring.

TL;DR

Generics (`<T>`) keep precise types across reusable code; constrain with `extends`, use `keyof` (the union of an object type's key names) / `T[K]` (the type of property `K`) for property-safe helpers. Know `Partial`, `Pick`, `Omit`, `Record`, `ReturnType`, `Awaited`.

16.13 Classes, `as const`, `satisfies`, and enums

`code/13_classes_and_modern_ts.ts` (key parts):

```ts
interface Account { deposit(cents: number): void; readonly balance: number }

class BankAccount implements Account {
  #balance = 0;                              // JS private field (runtime)
  private readonly history: number[] = [];    // TS private (compile-time only)
  protected owner: string;
  // ... constructor(owner), deposit(cents), get balance()
}

const ROLES = ['viewer', 'editor', 'admin'] as const;
type Role = (typeof ROLES)[number];           // 'viewer' | 'editor' | 'admin'

const LogLevel = { Debug: 0, Info: 1, Warn: 2, Error: 3 } as const;   // enum alternative

type Route = { path: string; auth: boolean };
const routes = {
  home: { path: '/', auth: false },
  admin: { path: '/admin', auth: true },
} satisfies Record<string, Route>;
// stock['ink'] on a Record<string, number> is number | undefined (noUncheckedIndexedAccess)
```

Output of `node 13_classes_and_modern_ts.ts`:

```
1) Ana's savings: 5000 | accounts: 1
2) viewer:false editor:true admin:true
3) false true
4) home,admin /admin
5) 0
```

What each feature is for:

Feature	Meaning	Note
`implements Account`	The class must match the interface's shape	Checked at compile time only
`private` / `protected` / `public`	Visibility **checked by TS only** — at runtime the property is ordinary	Use JS `#private` (§9) when privacy must hold at runtime
`readonly`	Can't be reassigned after construction	Compile time only
`abstract class`	Can't be instantiated; subclasses implement abstract members	Erasable
`as const`	Make a literal deeply `readonly` with literal types	Derive unions from data (`typeof ROLES[number]`) — one source of truth
`satisfies T` (TS 4.9+)	Check a value against `T` **without widening** it (widening = TS loosening a type to something broader, e.g. `'admin'` to `string`)	`routes.admin` stays known; with `: Record<string, Route>` it would be "any string key"
`enum`	Named constants that generate a runtime object	**Not erasable** — rejected by Node's type stripping. Prefer a union of literals or an `as const` object

> **TL;DR**
>
> TS classes add `implements`, `private/protected/readonly` (compile-time) on top of JS classes; use `#private` for runtime privacy. Prefer `as const` objects/unions to `enum`; use `satisfies` to validate config objects while keeping precise types.

16.14 Tests written in TypeScript

`node --test` also finds `*.test.ts` files and type-strips them, so TypeScript projects can use the built-in runner with **no build step** (no separate compile command before running). The module `code/src/money.ts` splits money without losing cents:

```ts
export type Currency = 'USD' | 'EUR' | 'INR';
export interface Money { readonly cents: number; readonly currency: Currency }
export function money(cents: number, currency: Currency = 'USD'): Money { /* validate, freeze */ }
export function add(a: Money, b: Money): Money { /* same currency only */ }
export function split(m: Money, n: number): Money[] {
  const base = Math.floor(m.cents / n);
  const extra = m.cents % n;
  return Array.from({ length: n }, (_, i) => money(base + (i < extra ? 1 : 0), m.currency));
}
```

`code/money.test.ts`:

```ts
import { describe, it } from 'node:test';
import assert from 'node:assert/strict';
import { money, add, split, type Money } from './src/money.ts';

describe('money', () => {
  it('adds amounts in the same currency', () => {
    assert.deepEqual(add(money(150), money(275)), { cents: 425, currency: 'USD' });
  });
  // ... 'refuses to mix currencies'
  it('splits without losing a cent', () => {
    const parts: Money[] = split(money(1000), 3);
    assert.deepEqual(parts.map((p) => p.cents), [334, 333, 333]);
    assert.equal(parts.reduce((t, p) => t + p.cents, 0), 1000);
  });
  it('values are immutable', () => {
    const m = money(5);
    assert.throws(() => { (m as { cents: number }).cents = 6; }, TypeError);
  });
});
```

Output of `node --test money.test.ts`:

```
▶ money
  ✔ adds amounts in the same currency (0.494792ms)
  ...
  ✔ values are immutable (0.055667ms)
✔ money (1.196625ms)
ℹ tests 4
...
ℹ pass 4
ℹ fail 0
...
```

"values are immutable" (cannot be changed) relies on `Object.freeze` plus module strict mode (the stricter JS mode, always on in modules) (assigning to a frozen property throws `TypeError`). The `as { cents: number }` cast steps around TS's (correct) `readonly` error so the test can prove the *runtime* guard works too.

Type-checking test files needs Node's type definitions. This project doesn't install `@types/node` (hence `tsconfig.json` excludes `*.test.ts`); checking the file alone shows the error a fresh project gets:

```
money.test.ts(2,30): error TS2591: Cannot find name 'node:test'. Do you need to install type definitions for node?
Try `npm i --save-dev @types/node` and then add 'node' to the types field in your tsconfig.
...
```

In a real Node project: `npm i -D @types/node` and `"types": ["node"]` in `compilerOptions` (listing it is required in TypeScript 7, where `types` defaults to `[]`).

The full suite in `code/` (`node --test` finds all four test files):

```
ℹ tests 31
ℹ suites 8
ℹ pass 29
ℹ fail 0
...
ℹ skipped 1
ℹ todo 1
```

16.15 JSDoc types: TypeScript checking for plain JavaScript

What it is

Without any `.ts` files, TypeScript can check **JavaScript** using types in **JSDoc comments** (`/** @param {number} x */`). Add `// @ts-check` at the top of a file (or `checkJs: true` in `tsconfig.json`) and the editor

and `tsc` check it with the same engine — nothing to compile or strip. Svelte (a UI framework) and webpack (a bundler) are well-known codebases typed this way.

JSDoc tag	Meaning
`@param {Type} name` / `@returns {Type}`	Parameter and return types
`@type {Type}`	A variable's type
`@typedef {object} Name` + `@property`	Declare a named object type
`@template T`	A generic type parameter
`@import` / `import('./x.js').Type`	Refer to types from other modules

`code/15_jsdoc.mjs`:

```
// @ts-check
/**
 * @typedef {object} LineItem
 * @property {string} sku
 * @property {number} price  unit price in cents
 * @property {number} [qty]  optional, defaults to 1
 */

/**
 * @param {LineItem[]} items
 * @returns {number} total in cents
 */
export function sum(items) {
  return items.reduce((t, i) => t + i.price * (i.qty ?? 1), 0);
}

/** @template T @param {T[]} xs @returns {T | undefined} */
export function last(xs) { return xs[xs.length - 1]; }

/** @type {Record<string, 'admin' | 'user'>} */
const roles = { ana: 'admin', bo: 'user' };
```

Output of `node 15_jsdoc.mjs`:

```
sum  : 1200
last : c
roles: { ana: 'admin', bo: 'user' }
```

The file is part of the project's `tsc --noEmit` check (clean). With mistakes — `code/jsdoc_errors.mjs`:

```
/**
 * @param {number} cents
 * @returns {string}
 */
function formatCents(cents) {
  return (cents / 100).toFixed(2);
}

formatCents('150');                       // string passed where number expected
/** @type {number} */
const count = formatCents(99);            // string result assigned to a number
console.log(count.toUpperCase());         // number has no toUpperCase
```

Output of `tsc --ignoreConfig --noEmit --allowJs --checkJs --strict --target es2024 jsdoc_errors.mjs`:

```
jsdoc_errors.mjs(10,13): error TS2345: Argument of type 'string' is not assignable to parameter of type 'number'.
jsdoc_errors.mjs(12,7): error TS2322: Type 'string' is not assignable to type 'number'.
jsdoc_errors.mjs(13,19): error TS2339: Property 'toUpperCase' does not exist on type 'number'.
```

Trade-offs

	`.ts` files	JSDoc + `@ts-check`
Syntax	Concise	Verbose comments
Build/strip step	Needed (or Node type stripping)	None
Advanced types	Everything	Most things (complex types can live in a `.d.ts` file)
Adoption	Rename files	Add a comment line per file — easiest first step for an existing JS codebase

TL;DR

`// @ts-check` + JSDoc annotations give TypeScript checking to plain `.js` files with zero build step — a gentle way to adopt types.

Chapter 16 TL;DR

- **Tests** run code and assert results; unit (fast, isolated) → integration → E2E (few). Test behaviour, not internals; exit code gates CI.
- `node:assert/strict`: `equal` (Object.is), `deepEqual` (structure, type-strict), `throws`, `await rejects`, `match`.
- `node:test`: `test`/`it`, `describe`, hooks, subtests, `skip`/`todo`/`only`, `timeout`; `node --test` (finds `*.test.*` and `test/`), `--watch`, reporters.
- **Async**: return/await promises; a missing `await` can make a failing test pass.
- **Doubles**: dependency injection + stubs/fakes first; `t.mock.fn`/`t.mock.method` spies (auto-restored); **fake timers** for time; module mocks need `--experimental-test-module-mocks`.
- **Snapshots** (`t.assert.snapshot`, `--test-update-snapshots`) — review them; **coverage** (`--experimental-test-coverage`) — a hint, not a goal.
- **TypeScript** = JS + static types, erased at runtime; structural typing; `strict`; check with `tsc --noEmit`. **TS 7** (July 2026) is the Go-native compiler, ~10× faster, stricter defaults, `types: []`.
- `node file.ts` works (stable type stripping in Node 24) but **never type-checks**. Erasable syntax only (no `enum`, parameter properties, namespaces with code), `.ts` extensions, `import type`.
- Types: literals, unions, tuples, `interface` vs `type`, `unknown` **over** `any`, `never`. **Narrowing** + **discriminated unions** with a `never` exhaustive check.
- **Generics** with constraints and `keyof`; utility types `Partial`, `Pick`, `Omit`, `Record`, `ReturnType`, `Awaited`.
- Classes: `implements`, TS-only `private`/`readonly` vs runtime `#private`; prefer `as const` objects to `enum`; `satisfies` checks without widening.
- `*.test.ts` run directly with `node --test`; type-checking them needs `@types/node`.
- **JSDoc + `// @ts-check`**: type-check plain JavaScript with no build step.

Next: put it all together for interviews — Chapter 17: JavaScript for Interviews.

Chapter 17 — JavaScript for Interviews

A JavaScript interview usually has three parts. A **coding round** asks you to solve an algorithm (a step-by-step method for solving a problem) problem (in JavaScript, so you need to know which built-ins are fast and which are traps). A **"build it yourself" round** asks you to write a utility that libraries normally give you — `debounce`, `Promise.all`, a deep clone. And a **language round** throws short snippets at you and asks "what does this print, and why?". This chapter prepares all three. It does not re-teach the language: every answer links to the section of this book that explains it fully.

What you'll learn

1. The JavaScript **DSA toolkit** (DSA = data structures and algorithms, §17.1): arrays as stacks and queues, `Map` / `Set`, sorting comparators and stability, number pitfalls (`0.1 + 0.2`, safe integers, integer division), building strings, `BigInt` — with a complexity table.
2. **Twenty-one coding patterns** — two pointers, sliding window, hashing, stack, prefix sums, intervals, heap (with a hand-written `MinHeap`, since JavaScript has none, plus two heaps for a running median and k-way merge), binary search, BFS (breadth-first search), DFS (depth-first search), dynamic programming with memoization, backtracking, linked lists, fast & slow pointers, trees, tries, topological sort and union-find, greedy, sort & search (quickselect, Dutch flag, merge sort), bit manipulation, math & geometry — each with two or three classic problems in runnable, asserted code.
3. **Implement-from-scratch classics** with tests: `debounce`, `throttle`, `curry`, `compose` / `pipe`, `memoize`, `flatten`, a `bind` polyfill (your own implementation of a built-in feature), deep clone (`structuredClone` vs manual), deep equal, an LRU (least recently used) cache, an `EventEmitter`, and `Promise.all` / `allSettled` / `race` / `any` polyfills.
4. **Fifty-three tricky questions** with real outputs: coercion (JavaScript's automatic conversion between types), equality, `typeof null`, `NaN`, hoisting and the TDZ, closures in loops, `this`, arrow vs regular functions, prototypes, classes, event-loop ordering puzzles, shallow vs deep copies, garbage collection (the engine automatically freeing memory nothing uses any more) and memory leaks.

Before you start. This chapter assumes the whole book up to Chapter 16. The tests use `node:assert/strict` and fake timers from `node:test`, explained in §16.2 and §16.5. The same patterns are solved in Python in the Python Handbook's Chapter 19: Python for Coding Interviews — handy for comparing; here we focus on what is specific to JavaScript.

All code is in `code/`. Every program checks itself with `assert` and exits with an error if any answer is wrong; outputs below are real, from Node 24.

File	Contents
`01_toolkit.mjs`	Arrays, queue, grids, Map/Set, sorting, numbers, strings, BigInt
`02_patterns_arrays.mjs`	Two pointers, sliding window, hashing, stack, prefix sums, intervals
`03_patterns_search_graphs_dp.mjs`	MinHeap, two heaps, k-way merge, binary search, BFS, DFS, DP, backtracking
`04_classics_functions.mjs`	debounce, throttle, curry, compose/pipe, memoize, flatten, bind/call
`05_classics_data.mjs`	structuredClone, deepClone, deepEqual, LRU cache, EventEmitter
`06_promise_polyfills.mjs`	all/allSettled/race/any, timeout, promisify, pool, retry
`07_qa_language.js`	Outputs for the language questions (CommonJS, sloppy mode — i.e. non-strict mode — on purpose)
`08_qa_async_and_memory.mjs`	Outputs for event-loop, strict-mode and memory questions
`09_patterns_lists_trees_tries.mjs`	Linked lists, fast & slow pointers, trees, tries (Patterns 13–16)
`10_patterns_graphs_greedy_bits_math.mjs`	Topological sort, union-find, greedy, quickselect/Dutch flag/merge sort, bits, math & geometry (Patterns 17–21)

17.1 The JavaScript DSA toolkit

What it is

DSA means *data structures and algorithms*. In an interview you have to turn an idea ("use a queue", "count with a hash map") into working code quickly. JavaScript gives you fewer ready-made structures than Java or Python — no priority queue (always hands back the smallest or most urgent item), no deque (double-ended queue: add/remove at both ends), no sorted map (keeps keys in order) — so you need to know what the built-ins really cost and how to fill the gaps.

> **Key term — hash map (hash table):** a structure that stores key → value pairs and finds a key almost instantly. A *hash function* turns the key into a number that picks a slot in an internal array, so a lookup doesn't scan every entry. JavaScript's `Map`, `Set` and plain objects are built on hash tables. Lookups are O(1) "on average" because two keys occasionally land in the same slot and need a little extra work.

Why it matters

Two JavaScript-specific traps sink otherwise-correct solutions: **hidden O(n) operations** (`shift`, `unshift`, `includes` on an array inside a loop) and **wrong answers from defaults** (`sort()` comparing numbers as text, floating-point arithmetic). Knowing the table below is the difference between an O(n) and an O(n²) solution.

> **Key term — Big-O notation (complexity):** a way to describe how the work (or memory) of an algorithm grows as the input size `n` grows, ignoring constant factors. O(1) = constant, same cost for any size; O(log n) = grows very slowly (halving each step); O(n) = linear, proportional to the input; O(n log n) = typical good sorting; O(n²) = quadratic, 10× more input means ~100× more work. Interviewers expect you to state both *time* and extra *space* complexity.

Complexity of the built-ins

Operation	Cost	Note
`arr[i]`, `arr[i] = x`, `arr.length`, `arr.at(-1)`	O(1)	
`push`, `pop`	O(1) amortized (averaged over many calls)	Use the end of an array as a **stack**
`shift`, `unshift`, `splice(i, …)`	O(n)	Every later element moves — avoid in loops
`indexOf`, `includes`, `find`, `some`	O(n)	Put values in a `Set` if you look them up repeatedly
`slice`, spread `[...a]`, `concat`, `Array.from`	O(k)	k = number of elements copied
`map`, `filter`, `reduce`, `forEach`	O(n)	Plus the callback's cost
`sort`, `toSorted`	O(n log n)	Stable (guaranteed since ES2019 — the 2019 edition of the JS standard); V8 uses TimSort (a hybrid of merge and insertion sort)
`Map` / `Set` `get`, `set`, `has`, `add`, `delete`	O(1) average	The spec only requires *sublinear* (growing slower than n) on average; engines use hash tables
Object property read/write, `in`, `Object.hasOwn`	O(1) average	
`Object.keys` / `entries`, `[...map]`	O(n)	
`str[i]`, `str.length`, `charCodeAt`	O(1)	Strings are immutable (can't be changed in place)
`str.slice`, `split`, `join`, `[...str]`	O(n)	
`JSON.stringify`, `structuredClone`	O(size)	
Heap push/pop (you write it, §17.2)	O(log n)	

Arrays as stack, queue and grid

From `code/01_toolkit.mjs`:

```
const stack = [];
stack.push(1); stack.push(2);
console.log('stack pop:', stack.pop(), 'peek:', stack.at(-1));   // O(1) at the end

const q = [1, 2, 3];
console.log('queue shift:', q.shift(), 'left:', q);             // shift is O(n)!

// A real O(1) queue: keep a head index instead of shifting
class Queue {
  #items = []; #head = 0;
  push(x) { this.#items.push(x); }
  shift() {
    if (this.#head === this.#items.length) return undefined;
    const x = this.#items[this.#head++];
    if (this.#head > 1024 && this.#head * 2 > this.#items.length) {   // compact occasionally
      this.#items = this.#items.slice(this.#head); this.#head = 0;
    }
    return x;
  }
  get size() { return this.#items.length - this.#head; }
}

// 2D grid: Array.from, NOT fill with an array (shared rows!)
const bad = new Array(2).fill(new Array(2).fill(0));
bad[0][0] = 9;
const grid = Array.from({ length: 2 }, () => new Array(2).fill(0));
grid[0][0] = 9;
```

Output:

```
stack pop: 2 peek: 1
queue shift: 1 left: [ 2, 3 ]
Queue: a 1
fill(array) shares rows: [[9,0],[9,0]]
Array.from rows are separate: [[9,0],[0,0]]
```

1. A **stack** (last in, first out) is just `push`/`pop`; peek with `at(-1)`.
2. A **queue** (first in, first out) with `shift()` is fine for small inputs, but in general each `shift` moves every element. Measured in Node 24: emptying a 10,000-element array with `shift` took 0.3 ms, a 100,000-element one 0.6 s, a 400,000-element one 10 s — quadratic growth (V8 has a shortcut for smaller arrays, not for big ones). For BFS on large graphs (a *graph* is a set of nodes joined by edges, like cities and roads) keep a **head index** — the `Queue` class above, or simply `for (let head = 0; head < queue.length; head++)` as in §17.2.
3. `fill(x)` puts the *same* value in every slot; with an array as `x`, every row is the same object. `Array.from({ length }, () => …)` calls the function once per row. (Sorting, copying and array methods are taught in Chapter 7.)

`Map` and `Set`

```
const freq = new Map();
for (const ch of 'banana') freq.set(ch, (freq.get(ch) ?? 0) + 1);
const visited = new Set();
visited.add(`${1},${2}`);                          // compound key as a string
new Set([[1, 2]]).has([1, 2]);                     // false: arrays compare by reference
const a = new Set([1, 2, 3]), b = new Set([2, 3, 4]);
a.union(b); a.intersection(b); a.difference(b);    // ES2025
```

Output:

```
freq: [ [ 'b', 1 ], [ 'a', 3 ], [ 'n', 2 ] ]
set: [ 3, 1, 2 ] has 2? true
visited has "1,2"? true | Set of arrays: false
union [ 1, 2, 3, 4 ] intersection [ 2, 3 ] difference [ 1 ]
```

- Prefer `Map` over a plain object for counting and lookups: any key type, no inherited keys like `constructor`, a real `size`, and insertion-order iteration (§8.1).
- `Map` and `Set` compare keys with **SameValueZero** (like `===`, except `NaN` equals `NaN`). Two arrays with equal contents are *different* keys, so encode coordinates as a string (`` `${r},${c}` ``) or a number (`r * cols + c`).
- The ES2025 set methods (`union`, `intersection`, `difference`, `symmetricDifference`, `isSubsetOf`, …) are in Node 22+ and all current browsers (§8.2).

Sorting: comparators and stability

```
[10, 9, 1, 100].sort();                          // compared as STRINGS
[10, 9, 1, 100].sort((x, y) => x - y);           // numeric
people.toSorted((p, r) => p.age - r.age);        // stable: ties keep input order
people.toSorted((p, r) => r.age - p.age || p.name.localeCompare(r.name));  // two keys
```

Output:

```
default sort: [ 1, 10, 100, 9 ]
numeric sort: [ 1, 9, 10, 100 ]
by age (stable): Bob Dee Ann Cid
age desc, name asc: Ann Cid Bob Dee
original untouched: Ann Bob Cid Dee
```

1. A **comparator** `(a, b) => number` returns negative if `a` goes first, positive if `b` goes first, `0` for a tie. `a - b` sorts numbers ascending; `b - a` descending.
2. **Multi-key sort**: chain comparisons with `||` — when the first difference is `0`, the second decides.
3. **Stability** means equal elements keep their original relative order (Bob before Dee, Ann before Cid above). It is guaranteed since ES2019, so you can sort by a secondary key first and the primary key second.
4. `sort` **mutates** (changes the array in place); `toSorted` (ES2023) returns a new array. In an interview, say which one you use — mutating the caller's input is a classic review comment.
5. Common mistake: returning a boolean (`(a, b) => a > b`). `true`/`false` become `1`/`0`, never negative, so the result is engine-dependent and wrong. Full details: §7.6.

Number pitfalls

```
0.1 + 0.2;                        // 0.30000000000000004
Number.MAX_SAFE_INTEGER;          // 9007199254740991 = 2**53 - 1
2 ** 53 + 1;                      // 9007199254740992 (can't be represented)
Math.trunc(-7 / 2); Math.floor(-7 / 2);   // -3 and -4
-7 % 3;                           // -1 (sign follows the dividend)
(2 ** 31 + 2) >> 1;               // -1073741823: bit operators use 32-bit ints
Math.max(...[]);                  // -Infinity
```

Output:

```
0.1 + 0.2 = 0.30000000000000004 | === 0.3? false
close(0.1+0.2, 0.3): true
MAX_SAFE_INTEGER: 9007199254740991 | 2**53 + 1 = 9007199254740992
isSafeInteger(2**53): false
7 / 2 = 3.5 | Math.trunc: 3 | -7/2 trunc: -3 floor: -4
-7 % 3 = -1 | true modulo: 2
mid without overflow worry: 5 | but >> breaks above 2**31: -1073741823
Infinity sentinels: Infinity -Infinity
Math.max(...[]) = -Infinity | parseInt("08") = 8 | Number("") = 0
NaN === NaN: false | Number.isNaN(NaN): true | isNaN("x"): true Number.isNaN("x"): false
```

Pitfall	What to do
All numbers are 64-bit floats (IEEE 754 doubles — the standard binary format for decimals, which can't store most fractions exactly), so `0.1 + 0.2 !== 0.3`	Compare with a tolerance (`Math.abs(a - b) < Number.EPSILON * scale`), or work in integers (cents, not dollars)
No integer division	`Math.trunc(a / b)` (towards zero) or `Math.floor(a / b)` (towards −∞) — they differ for negatives
`%` is a *remainder*, sign follows the left side	For array wrap-around use `((i % n) + n) % n`
Integers are exact only up to `2**53 − 1`	Check with `Number.isSafeInteger`; use `BigInt` beyond (e.g. factorials, large Fibonacci, 64-bit IDs)
Bitwise operators (`	`, `>>`, `~~`) convert to 32-bit signed ints
`Infinity` / `-Infinity`	Handy initial values for min/max; but `Math.max(...[])` is `-Infinity` and spreading a very large array (about 125,000+ elements in Node 24 on this machine — the limit depends on stack size) throws `RangeError` — use a loop or `reduce`
`isNaN('x')` coerces first	Use `Number.isNaN`

Background: §2.10 Numbers in depth.

Strings: immutable, so build with arrays

```
const parts = [];
for (let i = 0; i < 5; i++) parts.push(i);
parts.join('-');                          // '0-1-2-3-4'
'd'.charCodeAt(0) - 97;                   // 3: letter -> index 0..25
[...'hello'].reverse().join('');          // 'olleh'
```

- Strings can't be changed in place; `s[0] = 'x'` silently does nothing (throws in strict mode). To edit, convert to an array (`[...s]`), change it, `join('')`.
- Building with `+=` in a loop is fine in modern engines (V8 uses *ropes*, a tree of pieces, to avoid copying), but `push` + `join` is the clear, portable idiom (the usual way experienced programmers write it) and makes the O(n) cost obvious.
- A 26-slot counting array (`charCodeAt(0) - 97`) is faster than a `Map` for lowercase-letter problems.
- `[...s]` splits by **code point** (one Unicode character number, so emoji stay whole); `s.split('')` splits by UTF-16 code unit (16-bit pieces; some characters need two). See §6.5.

BigInt

```
const big = 2n ** 64n;                    // 18446744073709551616n, typeof 'bigint'
const factorial = (n) => (n <= 1n ? 1n : n * factorial(n - 1n));
factorial(25n);                           // 15511210043330985984000000n
7n / 2n;                                  // 3n (division truncates)
1n == 1; 1n === 1;                        // true, false
1n + 1;                                   // TypeError: can't mix BigInt and Number
JSON.stringify(1n);                       // TypeError
```

`BigInt` (ES2020) holds integers of any size — write a literal with an `n` suffix or call `BigInt(x)`. It has no fractions, no `Math.*` support, can't be mixed with numbers in arithmetic (convert explicitly), and is slower than numbers, so use it only when values may pass `2**53`.

In the interview

30-second answer — "What should I watch for when doing algorithms in JavaScript?" Arrays are stacks (`push` / `pop`); queues need a head index, not `shift`. `Map` / `Set` give O(1) lookups (string keys

for coordinates). Always pass `sort` a comparator; it mutates. Numbers are doubles: `Math.trunc` for integer division, `%` can be negative, exact only to $2^{53} - 1$ (then `BigInt`). No heap — write one.

- *Is `Array.prototype.sort` stable?* Yes, required since ES2019.
- *What does `sort()` do with no comparator?* Converts to strings and compares UTF-16 code units, with `undefined` last — so `[10, 9]` stays `[10, 9]`.
- *Why a `Map` rather than `{}` for counting?* Any key type, no prototype keys (`'constructor' in {}` is `true`), `size`, insertion order, optimised for frequent adds/removes.

Self-check

1. What is the cost of `arr.shift()`? → O(n): every remaining element moves one slot.
2. How do you create a 3×3 grid of zeros safely? → `Array.from({ length: 3 }, () => new Array(3).fill(0))`.
3. How do you sort strings alphabetically in a locale-aware way? → `arr.sort((a, b) => a.localeCompare(b))`.
4. Is `Number.isSafeInteger(2 ** 53)` true? → No — the largest safe integer is `2 ** 53 - 1`.

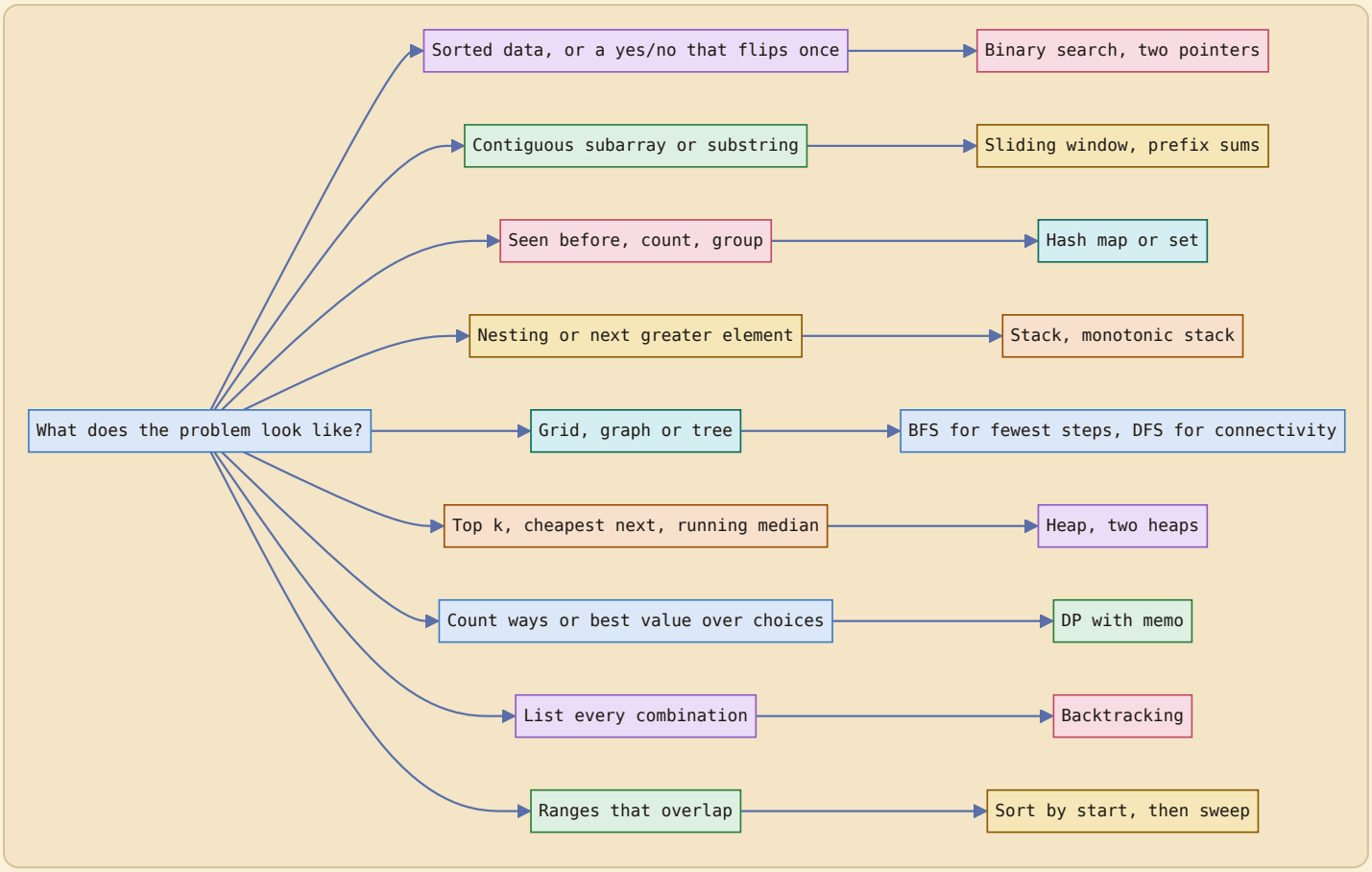
TL;DR

Stack = `push`/`pop`; queue = head index; grid = `Array.from`. `Map`/`Set` for O(1) lookups (string keys for pairs). Always give `sort` a comparator; it's stable; `toSorted` doesn't mutate. Floats: tolerance for equality, `Math.trunc` for division, safe up to $2^{53} - 1$, `BigInt` beyond.

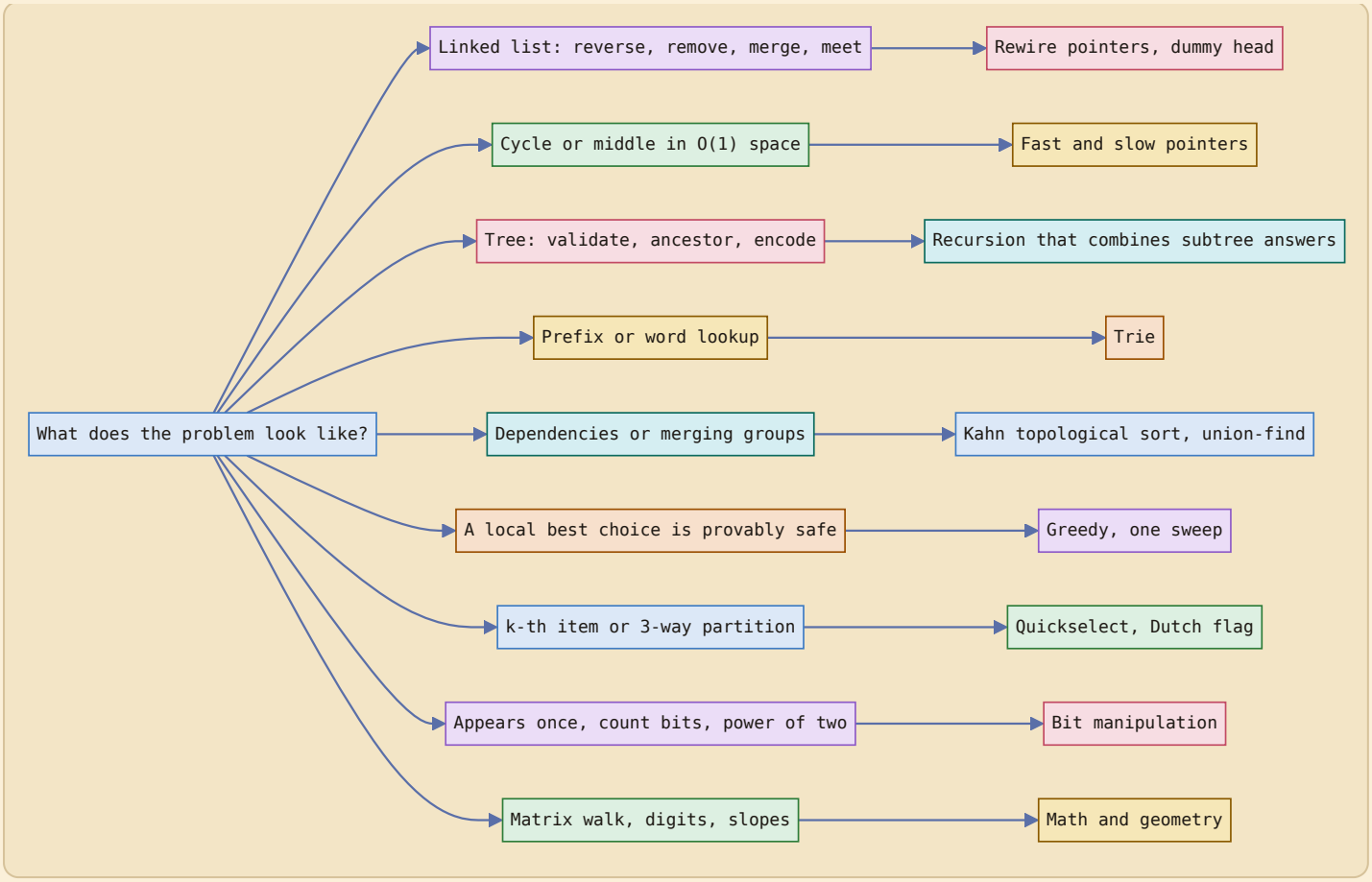
17.2 Coding patterns

What it is

A **pattern** is a reusable problem-solving shape. Most interview problems are one of about twenty patterns in disguise; recognising the pattern gives you the algorithm and its complexity almost for free. The decision chart below is a starting point, not a law.



Patterns 13–21 cover the remaining shapes — specific data structures and a few algorithm families:



For each pattern: the idea, when it applies, the code (with asserts), and its cost. The full programs are `code/02_patterns_arrays.mjs`, `code/03_patterns_search_graphs_dp.mjs`, `code/09_patterns_lists_trees_tries.mjs` and `code/10_patterns_graphs_greedy_bits_math.mjs`.

Pattern 1 — Two pointers

Idea: two indexes move towards each other (or in the same direction) so each element is looked at once. **Signal:** sorted array, pairs, palindromes (text that reads the same backwards), in-place partitioning (rearranging inside the same array, no copy). **Cost:** O(n) time, O(1) space.

```
function pairSum(sorted, target) {
  let lo = 0, hi = sorted.length - 1;
  while (lo < hi) {
    const s = sorted[lo] + sorted[hi];
    if (s === target) return [lo, hi];
    if (s < target) lo++; else hi--;      // too small: need a bigger left value
  }
  return null;
}
assert.deepEqual(pairSum([1, 3, 4, 6, 9], 12), [1, 4]);

function isPalindrome(s) {
  const t = s.toLowerCase().replace(/[^a-z0-9]/g, '');
  for (let i = 0, j = t.length - 1; i < j; i++, j--) if (t[i] !== t[j]) return false;
  return true;
}
assert.equal(isPalindrome('A man, a plan, a canal: Panama'), true);
```

Why it's correct: if the sum is too small, the smallest element can't be part of any answer with a *smaller* partner, so discarding it (`lo++`) loses nothing.

Pattern 2 — Sliding window

Idea: keep a window `[start, end]` over a sequence; grow the end, shrink the start when a rule breaks. **Signal:** "longest/shortest/max-sum **contiguous** (neighbouring, with no gaps) subarray or substring". **Cost:**

O(n) — each index enters and leaves the window once.

```
function longestUnique(s) {
  const last = new Map();                  // char -> last index seen
  let start = 0, best = 0;
  for (let end = 0; end < s.length; end++) {
    const ch = s[end];
    if (last.has(ch) && last.get(ch) >= start) start = last.get(ch) + 1;   // shrink
    last.set(ch, end);
    best = Math.max(best, end - start + 1);
  }
  return best;
}
assert.equal(longestUnique('abcabcbb'), 3);

function maxSumK(nums, k) {                // fixed-size window: add the new, drop the old
  let sum = 0;
  for (let i = 0; i < k; i++) sum += nums[i];
  let best = sum;
  for (let i = k; i < nums.length; i++) { sum += nums[i] - nums[i - k]; best = Math.max(best, sum); }
  return best;
}
assert.equal(maxSumK([2, 1, 5, 1, 3, 2], 3), 9);
```

Pattern 3 — Hashing

Idea: trade memory for time — remember what you've seen in a `Map`/`Set` so a later lookup is O(1). **Signal:** "find a pair/duplicate", "group by", "count". **Cost:** O(n) time, O(n) space.

```
function twoSum(nums, target) {
  const indexOf = new Map();
  for (let i = 0; i < nums.length; i++) {
    const need = target - nums[i];
    if (indexOf.has(need)) return [indexOf.get(need), i];   // check BEFORE inserting
    indexOf.set(nums[i], i);
  }
  return null;
}
assert.deepEqual(twoSum([2, 7, 11, 15], 9), [0, 1]);
assert.deepEqual(twoSum([3, 3], 6), [0, 1]);

function groupAnagrams(words) {
  const groups = new Map();
  for (const w of words) {
    const key = [...w].sort().join('');      // canonical form = the hash key
    if (!groups.has(key)) groups.set(key, []);
    groups.get(key).push(w);
  }
  return [...groups.values()];
}
```

Checking before inserting is what makes `[3, 3]` work: the first `3` is not paired with itself. (Anagrams are words with the same letters in a different order; sorting the letters gives every anagram the same *canonical* key.) (`Map.groupBy` from §8.7 does the grouping step in one call.)

Pattern 4 — Stack (and monotonic stack)

Idea: a stack remembers "things still open" — brackets, unfinished elements. A **monotonic stack** keeps its contents increasing or decreasing and answers "next greater/smaller element" questions. **Cost:** O(n) — every index is pushed and popped at most once.

```
function isBalanced(s) {
  const pairs = { ')': '(', ']': '[', '}': '{' };
  const st = [];
  for (const ch of s) {
    if ('([{'.includes(ch)) st.push(ch);
    else if (ch in pairs) { if (st.pop() !== pairs[ch]) return false; }
  }
  return st.length === 0;                  // leftovers = unclosed brackets
}

function dailyTemperatures(t) {            // days until a warmer day
  const ans = new Array(t.length).fill(0);
  const st = [];                           // indices whose temperatures are decreasing
  for (let i = 0; i < t.length; i++) {
    while (st.length && t[i] > t[st.at(-1)]) { const j = st.pop(); ans[j] = i - j; }
    st.push(i);
  }
  return ans;
}
assert.deepEqual(dailyTemperatures([73, 74, 75, 71, 69, 72, 76, 73]), [1, 1, 4, 2, 1, 1, 0, 0]);
```

Pattern 5 — Prefix sums

Idea: precompute running totals `p[i] = nums[0] + … + nums[i-1]`; then any range sum is one subtraction. Combined with a hash map it counts subarrays with a given sum, even with negative numbers (where a sliding window fails). **Cost:** O(n) setup, O(1) per query.

```
function prefix(nums) {
  const p = [0];
  for (const x of nums) p.push(p.at(-1) + x);
  return (i, j) => p[j + 1] - p[i];        // sum of nums[i..j] inclusive
}
assert.equal(prefix([3, 1, 4, 1, 5, 9])(2, 5), 19);

function subarraySum(nums, k) {            // how many subarrays sum to k?
  const count = new Map([[0, 1]]);         // the empty prefix
  let sum = 0, total = 0;
  for (const x of nums) {
    sum += x;
    total += count.get(sum - k) ?? 0;      // earlier prefixes that leave exactly k
    count.set(sum, (count.get(sum) ?? 0) + 1);
  }
  return total;
}
assert.equal(subarraySum([1, -1, 0], 0), 3);
```

Pattern 6 — Intervals

Idea: sort by start, then sweep once, merging or counting overlaps. **Signal:** meetings, bookings, ranges. **Cost:** O(n log n) for the sort.

```
function mergeIntervals(intervals) {
  const sorted = intervals.toSorted((a, b) => a[0] - b[0]);   // don't mutate the input
  const out = [];
  for (const [s, e] of sorted) {
    const last = out.at(-1);
    if (last && s <= last[1]) last[1] = Math.max(last[1], e);   // overlap: extend
    else out.push([s, e]);
  }
  return out;
}
assert.deepEqual(mergeIntervals([[8, 10], [1, 3], [2, 6], [15, 18]]), [[1, 6], [8, 10], [15, 18]]);

function minRooms(meetings) {              // sweep line: sorted starts vs sorted ends
  const starts = meetings.map(m => m[0]).sort((a, b) => a - b);
  const ends = meetings.map(m => m[1]).sort((a, b) => a - b);
  let rooms = 0, best = 0, j = 0;
  for (const s of starts) {
    while (ends[j] <= s) { j++; rooms--; }  // a meeting ending at s frees its room
    rooms++; best = Math.max(best, rooms);
  }
  return best;
}
assert.equal(minRooms([[0, 30], [5, 10], [15, 20]]), 2);
```

Output of the array-patterns program:

```
pairSum [ 1, 4 ] | longestUnique("abcabcbb") 3
twoSum [ 0, 1 ] | groupAnagrams [["eat","tea","ate"],["tan","nat"],["bat"]]
dailyTemperatures 1,1,4,2,1,1,0,0
subarraySum([1,1,1],2) 2 | merge [[1,6],[8,10],[15,18]] | rooms 2
all array-pattern asserts passed
```

Pattern 7 — Heap (write your own `MinHeap`)

What it is: a **binary heap** is a complete binary tree (a tree where each node has at most two children and every level is full except possibly the last, filled from the left — see Pattern 15) stored in an array where every parent is ≤ its children (a *min*-heap). The smallest item is always at index 0. For index `i`, the children are `2i+1` and `2i+2`, the parent is `(i−1)>>1`. **Signal:** "k largest/smallest", "merge k sorted lists", "median (middle value when sorted) of a stream", "always process the cheapest next" (Dijkstra's shortest-path algorithm, scheduling). **Cost:** `push`/`pop` O(log n), `peek` O(1).

JavaScript has **no built-in priority queue**, so interviewers expect you to write one (or at least describe it). Twenty-five lines are enough:

```
export class MinHeap {
  #a = [];
  #less;
  constructor(less = (x, y) => x < y) { this.#less = less; }   // custom ordering
  get size() { return this.#a.length; }
  peek() { return this.#a[0]; }
  push(x) {                                    // add at the end, sift UP
    const a = this.#a; a.push(x);
    let i = a.length - 1;
    while (i > 0) {
      const p = (i - 1) >> 1;
      if (!this.#less(a[i], a[p])) break;
      [a[i], a[p]] = [a[p], a[i]]; i = p;
    }
  }
  pop() {                                      // move the last to the root, sift DOWN
    const a = this.#a;
    if (a.length === 0) return undefined;
    const top = a[0], last = a.pop();
    if (a.length) {
      a[0] = last;
      let i = 0;
      for (;;) {
        const l = 2 * i + 1, r = l + 1;
        let m = i;
        if (l < a.length && this.#less(a[l], a[m])) m = l;
        if (r < a.length && this.#less(a[r], a[m])) m = r;
        if (m === i) break;
        [a[i], a[m]] = [a[m], a[i]]; i = m;
      }
    }
    return top;
  }
}

function kLargest(nums, k) {                   // min-heap of size k: O(n log k)
  const heap = new MinHeap();
  for (const x of nums) { heap.push(x); if (heap.size > k) heap.pop(); }
  const out = []; while (heap.size) out.push(heap.pop());
  return out.reverse();
}
assert.deepEqual(kLargest([3, 2, 1, 5, 6, 4], 2), [6, 5]);
```

The `less` parameter makes it a general **priority queue**: pass `(x, y) => x > y` for a max-heap, or compare a field. Dijkstra's shortest paths uses `[distance, node]` pairs:

```
function dijkstra(graph, src) {
  const dist = new Map([[src, 0]]);
  const pq = new MinHeap((x, y) => x[0] < y[0]);   // [distance, node]
  pq.push([0, src]);
  while (pq.size) {
    const [d, u] = pq.pop();
    if (d > dist.get(u)) continue;                  // stale entry: a shorter path was found
    for (const [v, w] of graph[u] ?? []) {
      const nd = d + w;
      if (nd < (dist.get(v) ?? Infinity)) { dist.set(v, nd); pq.push([nd, v]); }
    }
  }
  return Object.fromEntries(dist);
}
const roads = { A: [['B', 4], ['C', 1]], C: [['B', 2], ['D', 7]], B: [['D', 1]] };
assert.deepEqual(dijkstra(roads, 'A'), { A: 0, C: 1, B: 3, D: 4 });
```

Two heaps: the running median. To report the median of a stream after every new number, keep the smaller half in a **max-heap** (`low`) and the larger half in a **min-heap** (`high`), with `low` holding the same number of items or one more. The median is then `low`'s top, or the average of both tops. Each `add` is O(log n); `median` is O(1).

```
class MedianFinder {
  #low = new MinHeap((x, y) => x > y);        // max-heap: the smaller half
  #high = new MinHeap();                      // min-heap: the larger half
  add(x) {
    if (this.#low.size === 0 || x <= this.#low.peek()) this.#low.push(x); else this.#high.push(x);
    if (this.#low.size > this.#high.size + 1) this.#high.push(this.#low.pop());   // rebalance
    else if (this.#high.size > this.#low.size) this.#low.push(this.#high.pop());
  }
  median() {
    return this.#low.size > this.#high.size ? this.#low.peek() : (this.#low.peek() + this.#high.peek()) / 2;
  }
}
const mf = new MedianFinder();
assert.deepEqual([5, 15, 1, 3].map(x => { mf.add(x); return mf.median(); }), [5, 10, 5, 4]);
```

k-way merge. To merge k sorted lists, keep one "front" item per list in a min-heap; pop the smallest, then push the next item from the same list. With N items in total the cost is O(N log k) — better than concatenating and sorting (O(N log N)) when k is small.

```
function mergeKSorted(lists) {
  const pq = new MinHeap((x, y) => x[0] < y[0]);    // [value, list index, position]
  lists.forEach((l, i) => { if (l.length) pq.push([l[0], i, 0]); });
  const out = [];
  while (pq.size) {
    const [v, i, j] = pq.pop();
    out.push(v);
    if (j + 1 < lists[i].length) pq.push([lists[i][j + 1], i, j + 1]);   // refill from the same list
  }
  return out;
}
assert.deepEqual(mergeKSorted([[1, 4, 7], [2, 5, 8], [0, 3, 6, 9], []]), [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]);
```

Shortcut worth knowing: if you only need to sort once, `toSorted` is simpler and also O(n log n). A heap wins when items arrive over time or you need only the top k.

Pattern 8 — Binary search

Idea: halve the search space each step. **Signal:** sorted data, or a yes/no condition that flips once as a number grows ("the smallest speed that works"). **Cost:** O(log n) steps.

```
function lowerBound(a, target) {            // first index with a[i] >= target
  let lo = 0, hi = a.length;                // half-open range [lo, hi)
  while (lo < hi) {
    const mid = lo + Math.floor((hi - lo) / 2);
    if (a[mid] < target) lo = mid + 1; else hi = mid;
  }
  return lo;                                 // a.length if every value is smaller
}
assert.equal(lowerBound([1, 3, 3, 5, 8], 3), 1);
assert.equal(lowerBound([1, 3, 3, 5, 8], 4), 3);

function minEatingSpeed(piles, hours) {     // binary search on the ANSWER
  let lo = 1, hi = Math.max(...piles);
  const ok = (k) => piles.reduce((t, p) => t + Math.ceil(p / k), 0) <= hours;
  while (lo < hi) { const mid = (lo + hi) >> 1; if (ok(mid)) hi = mid; else lo = mid + 1; }
  return lo;
}
assert.equal(minEatingSpeed([3, 6, 7, 11], 8), 4);
```

Use one template (half-open `[lo, hi)`, `lo < hi`, `hi = mid`) and you avoid the off-by-one bugs (being wrong by exactly one index) that make binary search famous. JavaScript has no `bisect` built-in; `indexOf` / `findIndex` are linear.

Pattern 9 — Breadth-first search (BFS)

Idea: explore in rings — all nodes 1 step away, then 2 steps, … using a queue. The first time you reach a node is by a **shortest path** (in steps). **Signal:** "minimum number of moves", unweighted shortest path,

level-order traversal. **Cost:** O(V + E) (V = number of vertices/nodes, E = number of edges; for a grid, O(rows × cols)).

```
function shortestPath(grid, [sr, sc], [tr, tc]) {
  const R = grid.length, C = grid[0].length;
  const dist = Array.from({ length: R }, () => new Array(C).fill(-1));   // -1 = unvisited
  const queue = [[sr, sc]]; dist[sr][sc] = 0;
  for (let head = 0; head < queue.length; head++) {       // index instead of shift()
    const [r, c] = queue[head];
    if (r === tr && c === tc) return dist[r][c];
    for (const [dr, dc] of [[1, 0], [-1, 0], [0, 1], [0, -1]]) {
      const nr = r + dr, nc = c + dc;
      if (nr >= 0 && nr < R && nc >= 0 && nc < C && grid[nr][nc] !== '#' && dist[nr][nc] === -1) {
        dist[nr][nc] = dist[r][c] + 1; queue.push([nr, nc]);   // mark when ENQUEUED
      }
    }
  }
  return -1;
}
assert.equal(shortestPath(['..#.', '..#.', '....'], [0, 0], [0, 3]), 7);
```

Mark a cell visited when you **enqueue** it (add it to the queue), not when you dequeue it (take it out) — otherwise the same cell enters the queue many times.

Pattern 10 — Depth-first search (DFS)

Idea: go as deep as possible, then back up. **Signal:** connectivity ("number of islands"), cycle detection, topological order (an ordering where every dependency comes first — Pattern 17), exploring every path. **Cost:** O(V + E).

```
function numIslands(rows) {
  const g = rows.map(r => [...r]);          // copy: don't destroy the caller's grid
  let count = 0;
  for (let r = 0; r < g.length; r++) for (let c = 0; c < g[0].length; c++) {
    if (g[r][c] !== '1') continue;
    count++;
    const st = [[r, c]]; g[r][c] = '0';      // explicit stack = iterative DFS
    while (st.length) {
      const [x, y] = st.pop();
      for (const [nx, ny] of [[x + 1, y], [x - 1, y], [x, y + 1], [x, y - 1]]) {
        if (g[nx]?.[ny] === '1') { g[nx][ny] = '0'; st.push([nx, ny]); }   // ?. handles edges
      }
    }
  }
  return count;
}
assert.equal(numIslands(['11000', '11000', '00100', '00011']), 3);

function topoSort(n, edges) {               // recursive DFS with three colours
  const adj = Array.from({ length: n }, () => []);
  for (const [u, v] of edges) adj[u].push(v);
  const state = new Array(n).fill(0);       // 0 new, 1 on the current path, 2 done
  const order = [];
  const visit = (u) => {
    if (state[u] === 1) throw new Error('cycle');
    if (state[u] === 2) return;
    state[u] = 1;
    for (const v of adj[u]) visit(v);
    state[u] = 2; order.push(u);            // post-order
  };
  for (let u = 0; u < n; u++) visit(u);
  return order.reverse();
}
assert.deepEqual(topoSort(4, [[0, 1], [0, 2], [1, 3], [2, 3]]), [0, 2, 1, 3]);
```

JavaScript-specific: recursion depth is limited (roughly ten thousand frames — one stack frame per active call — in V8, depending on frame size), and there's no tail-call optimisation (reusing the

current frame when a function's very last action is a call) in V8. For inputs that can be deep — a 1000×1000 grid of land — use the iterative version with an explicit stack.

Pattern 11 — Dynamic programming with memoization

What it is: **dynamic programming (DP)** solves a problem by combining answers to smaller versions of the same problem, storing each answer so it is computed once. **Top-down** = recursion + a cache (**memoization**); **bottom-up** = fill a table in order. **Signal:** "count the ways", "minimum cost", "longest …", when the plain recursion repeats the same calls. **Cost:** (number of states) × (work per state).

```
function climb(n, memo = new Map()) {       // ways to climb n stairs taking 1 or 2 steps
  if (n <= 1) return 1;
  if (memo.has(n)) return memo.get(n);
  const v = climb(n - 1, memo) + climb(n - 2, memo);
  memo.set(n, v);
  return v;
}
assert.equal(climb(70), 308061521170129);   // exact: below 2**53
assert.equal(Number.isSafeInteger(climb(80)), false);   // too big: use BigInt

function coinChange(coins, amount) {        // bottom-up: fewest coins for each amount
  const dp = new Array(amount + 1).fill(Infinity); dp[0] = 0;
  for (let a = 1; a <= amount; a++)
    for (const c of coins) if (c <= a) dp[a] = Math.min(dp[a], dp[a - c] + 1);
  return dp[amount] === Infinity ? -1 : dp[amount];
}
assert.equal(coinChange([1, 2, 5], 11), 3);

function lcs(a, b) {                        // longest common subsequence: 2D table
  const dp = Array.from({ length: a.length + 1 }, () => new Array(b.length + 1).fill(0));
  for (let i = 1; i <= a.length; i++) for (let j = 1; j <= b.length; j++)
    dp[i][j] = a[i - 1] === b[j - 1] ? dp[i - 1][j - 1] + 1 : Math.max(dp[i - 1][j], dp[i][j - 1]);
  return dp[a.length][b.length];
}
assert.equal(lcs('abcde', 'ace'), 3);
```

Without the memo, `climb(70)` would make about 6 × 10^{14} calls; with it, about 140. Note the JavaScript twist: the answer for 80 stairs is larger than `2**53`, so a real solution would switch to `BigInt` (§17.1). For multi-argument states, use a string key (`` `${i},${j}` ``) or a 2D array.

Pattern 12 — Backtracking

Idea: build a solution one choice at a time; after exploring a choice, **undo** it and try the next (choose → explore → un-choose). Prune (stop exploring) branches that can't succeed. **Signal:** "list all subsets/permutations (orderings)/combinations", puzzles (N-Queens, Sudoku). **Cost:** exponential (doubling or worse with every extra item) — 2^n subsets, n! permutations — so inputs are small.

```
function subsets(nums) {
  const out = [], cur = [];
  const go = (i) => {
    if (i === nums.length) { out.push([...cur]); return; }   // COPY the current path
    cur.push(nums[i]); go(i + 1); cur.pop();                 // choose, explore, un-choose
    go(i + 1);                                               // skip nums[i]
  };
  go(0);
  return out;
}
assert.deepEqual(subsets([1, 2]), [[1, 2], [1], [2], []]);

function nQueens(n) {                        // pruning with Sets of attacked lines
  const cols = new Set(), d1 = new Set(), d2 = new Set();
  let count = 0;
  const place = (r) => {
    if (r === n) { count++; return; }
    for (let c = 0; c < n; c++) {
      if (cols.has(c) || d1.has(r - c) || d2.has(r + c)) continue;
      cols.add(c); d1.add(r - c); d2.add(r + c);
      place(r + 1);
      cols.delete(c); d1.delete(r - c); d2.delete(r + c);
    }
  };
  place(0);
  return count;
}
assert.equal(nQueens(8), 92);
```

The most common JavaScript bug here is `out.push(cur)` without copying: every entry would be the *same* array, which ends up empty. (`permutations` with a `used` array is in the code file.)

Output of the search/graph/DP program:

```
heap drain 1,2,3,5,8,9 | kLargest [ 6, 5 ] | dijkstra {"A":0,"B":3,"C":1,"D":4}
running medians 5,10,5,4 | k-way merge 0,1,2,3,4,5,6,7,8,9
lowerBound 4 -> 3 | minEatingSpeed 4
maze steps 7 | islands 3 | topo 0,2,1,3
climb(70) 308061521170129 | climb(80) safe? false | coinChange 11 3 | lcs 3
subsets [[1,2],[1],[2],[]] | perms of 3: 6 | 8-queens 92
all search/graph/DP asserts passed
```

Pattern 13 — Linked lists

What it is: a **linked list** is a chain of nodes; each node holds a value and a `next` reference to the following node (`null` at the end). Unlike an array there is no index: reaching the k-th node means walking k steps, but inserting or removing a node you already hold is O(1) — you only rewire references. JavaScript has no built-in linked list, so problems hand you a `{ val, next }` shape. **Signal:**

"reverse", "remove the k-th from the end", "merge", "where do two lists meet". **Cost:** usually O(n) time, O(1) extra space.

```
class ListNode { constructor(val, next = null) { this.val = val; this.next = next; } }

function reverseList(head) {                // flip every arrow while walking
  let prev = null, cur = head;
  while (cur) {
    const next = cur.next;                  // save before overwriting
    cur.next = prev;
    prev = cur; cur = next;
  }
  return prev;                              // the old tail is the new head
}
assert.deepEqual(toArray(reverseList(fromArray([1, 2, 3, 4]))), [4, 3, 2, 1]);

function removeKthFromEnd(head, k) {        // one pass: `lead` runs k nodes ahead
  const dummy = new ListNode(0, head);      // dummy head: removing the head needs no special case
  let lead = dummy, trail = dummy;
  for (let i = 0; i < k; i++) lead = lead.next;
  while (lead.next) { lead = lead.next; trail = trail.next; }
  trail.next = trail.next.next;             // trail sits just before the node to remove
  return dummy.next;
}
assert.deepEqual(toArray(removeKthFromEnd(fromArray([1, 2, 3, 4, 5]), 2)), [1, 2, 3, 5]);

function intersection(a, b) {               // first node shared by two lists, or null
  let p = a, q = b;
  while (p !== q) { p = p ? p.next : b; q = q ? q.next : a; }
  return p;
}
```

Why `intersection` works: pointer `p` walks list A then list B, `q` walks B then A. Both travel `lenA + lenB` steps, so they line up and reach the shared node at the same moment — or both reach `null` together if there is none. It compares **nodes** with `!==`, not values: two lists that merely contain equal numbers do not intersect. The **dummy head** trick (a fake node before the real head) appears in almost every list problem because it removes the "what if I change the head?" branch. `fromArray`/`toArray` helpers are in the code file.

Pattern 14 — Fast & slow pointers

Idea: two pointers move through a sequence at different speeds — slow one step, fast two steps. If the sequence loops, fast eventually laps slow and they meet; if it ends, fast reaches the end when slow is halfway. **Signal:** "is there a cycle / where does it start", "middle of a list", any "does repeating this step loop forever?" question. **Cost:** O(n) time, **O(1) space** — the point, since a `Set` of visited nodes would also work but costs O(n).

```
function cycleStart(head) {                  // Floyd's algorithm
  let slow = head, fast = head;
  while (fast && fast.next) {
    slow = slow.next; fast = fast.next.next;
    if (slow === fast) {                     // met inside the loop
      let p = head;
      while (p !== slow) { p = p.next; slow = slow.next; }
      return p;                              // they meet again at the loop's first node
    }
  }
  return null;                               // fast fell off the end: no cycle
}

function middle(head) {                      // second middle for even lengths
  let slow = head, fast = head;
  while (fast && fast.next) { slow = slow.next; fast = fast.next.next; }
  return slow;
}
assert.equal(middle(fromArray([1, 2, 3, 4, 5])).val, 3);

function isHappy(n) {                        // n -> sum of squared digits; reaches 1 or loops
  const next = (x) => { let s = 0; while (x > 0) { const d = x % 10; s += d * d; x = Math.floor(x / 10); } return
s; };
  let slow = n, fast = next(n);
  while (fast !== 1 && slow !== fast) { slow = next(slow); fast = next(next(fast)); }
  return fast === 1;
}
assert.equal(isHappy(19), true);             // 19 -> 82 -> 68 -> 100 -> 1
assert.equal(isHappy(2), false);
```

Why the second phase finds the loop's start: say the start is `a` steps from the head and the pointers meet `b` steps into a loop of length `c`. Fast walked twice as far, so `2(a + b) = a + b + k·c` for some whole number `k`, which gives `a = k·c − b`. Walking `a` more steps from the meeting point therefore lands exactly on the loop's start — the same place a pointer starting at the head reaches after `a` steps. `isHappy` shows the pattern without any list: the "next" function defines an implicit chain.

Pattern 15 — Trees

What it is: a **binary tree** is a set of nodes where each node has at most two children (`left`, `right`); the top node is the **root**, nodes without children are **leaves**. A **binary search tree (BST)** adds a rule: every value in the left subtree is smaller than the node, every value in the right subtree larger. Most tree answers are a short recursive function that asks each subtree a question and combines the two answers. **Signal:** "validate", "lowest common ancestor", "path sum", "rebuild / encode a tree".

Cost: O(n) time, O(h) stack for a tree of height `h` (h = log n when balanced — both sides of every node about equally deep — n when it degenerates into a line).

```
class TreeNode { constructor(val, left = null, right = null) { this.val = val; this.left = left; this.right = right; } }

function isValidBST(node, lo = -Infinity, hi = Infinity) {   // pass the allowed range DOWN
  if (!node) return true;
  if (node.val <= lo || node.val >= hi) return false;
  return isValidBST(node.left, lo, node.val) && isValidBST(node.right, node.val, hi);
}
assert.equal(isValidBST(fromLevels([5, 1, 7, null, null, 4, 8])), false);   // 4 is right of 5

function lca(root, p, q) {                  // lowest common ancestor of values p and q
  if (!root || root.val === p || root.val === q) return root;
  const left = lca(root.left, p, q), right = lca(root.right, p, q);
  return left && right ? root : left ?? right;   // found on both sides: this node splits them
}
const family = fromLevels([3, 5, 1, 6, 2, 0, 8, null, null, 7, 4]);
assert.equal(lca(family, 7, 4).val, 2);

function serialize(root) {                  // preorder, '#' marks an empty child
  const out = [];
  const go = (n) => { if (!n) { out.push('#'); return; } out.push(n.val); go(n.left); go(n.right); };
  go(root);
  return out.join(',');
}
function deserialize(text) {
  const tokens = text.split(',');
  let i = 0;
  const go = () => {
    const t = tokens[i++];
    if (t === '#') return null;
    const node = new TreeNode(Number(t));
    node.left = go(); node.right = go();      // same order as serialize
    return node;
  };
  return go();
}
assert.equal(serialize(fromLevels([1, 2, 3, null, null, 4, 5])), '1,2,#,#,3,4,#,#,5,#,#');
```

The classic BST bug is comparing a node only with its parent: in `[5, 1, 7, null, null, 4, 8]`, 4 is a correct left child of 7 but sits in 5's *right* subtree, so the tree is invalid. Passing the range `(lo, hi)` down catches it. Serialization (turning the tree into a string) works because preorder (visit a node, then its left subtree, then its right) plus explicit `#` markers fixes the shape: the reader always knows whether the next token is a left child, a right child or an empty slot. (`fromLevels`, which builds a tree from a level-order array with `null` gaps, is in the code file; deep, unbalanced trees hit the recursion-depth limit from Pattern 10 — switch to an explicit stack.)

Pattern 16 — Tries

What it is: a **trie** (prefix tree, pronounced "try") stores strings one character per level; words that share a prefix share the same path from the root, and a flag marks "a word ends here". Looking up a word or prefix of length `L` costs O(L), no matter how many words are stored. **Signal:** "autocomplete",

"starts with", "search with a wildcard", "find many words in a grid". **Cost:** O(L) per insert/search; memory O(total characters).

```
class Trie {
  #root = { next: new Map(), end: false };
  insert(word) {
    let n = this.#root;
    for (const ch of word) {
      if (!n.next.has(ch)) n.next.set(ch, { next: new Map(), end: false });
      n = n.next.get(ch);
    }
    n.end = true;                          // "a word finishes here"
  }
  #walk(prefix) {
    let n = this.#root;
    for (const ch of prefix) { n = n.next.get(ch); if (!n) return null; }
    return n;
  }
  search(word) { return this.#walk(word)?.end === true; }
  startsWith(prefix) { return this.#walk(prefix) !== null; }
  match(pattern) {                         // '.' matches any one letter
    const go = (n, i) => {
      if (i === pattern.length) return n.end;
      if (pattern[i] !== '.') { const nx = n.next.get(pattern[i]); return nx ? go(nx, i + 1) : false; }
      for (const nx of n.next.values()) if (go(nx, i + 1)) return true;   // try every branch
      return false;
    };
    return go(this.#root, 0);
  }
}
// words: car, card, care, cat, dog
assert.equal(trie.search('ca'), false);      // a prefix, not a stored word
assert.equal(trie.match('c.r.'), true);      // card, care
```

The third classic, **word search on a board** (find every listed word that can be traced through neighbouring cells, each cell used once per word), combines a trie with Pattern 12's backtracking: one DFS from each cell walks the board and the trie together, and stops as soon as the letters so far are not a prefix of any word. That prune is what makes it fast compared with searching for each word separately.

```
function findWords(board, words) {
  const root = {};
  for (const w of words) { let n = root; for (const ch of w) n = n[ch] ??= {}; n.$word = w; }
  const grid = board.map(row => [...row]);   // copy: cells are marked while exploring
  const R = grid.length, C = grid[0].length, found = [];
  const dfs = (r, c, node) => {
    const ch = grid[r][c], next = node[ch];
    if (!next) return;                       // no word continues this way: prune
    if (next.$word) { found.push(next.$word); delete next.$word; }   // report once
    grid[r][c] = '#';                        // choose
    for (const [dr, dc] of [[1, 0], [-1, 0], [0, 1], [0, -1]]) {
      const nr = r + dr, nc = c + dc;
      if (nr >= 0 && nr < R && nc >= 0 && nc < C && grid[nr][nc] !== '#') dfs(nr, nc, next);
    }
    grid[r][c] = ch;                         // un-choose
  };
  for (let r = 0; r < R; r++) for (let c = 0; c < C; c++) dfs(r, c, root);
  return found.sort();
}
assert.deepEqual(findWords(['cats', 'ored', 'waxy'], ['cat', 'cats', 'car', 'core', 'cow', 'dog', 'red', 'wax',
'tax']),
  ['car', 'cat', 'cats', 'core', 'cow', 'red', 'wax']);
```

JavaScript notes: a `Map` per node is the clean choice; a plain object (`n[ch] ??= {}`) is shorter, but then a stored key such as `$word` must never collide with a letter. For lowercase-only input, a 26-slot array per node is the fastest variant (§17.1).

Output of the list/tree/trie program:

```
reverse 4,3,2,1 | remove 2nd from end 1,2,3,5 | intersection at 8
cycle starts at 3 | middle of 1..4 3 | happy 19? true | happy 2? false
validBST true false | lca(7,4) 2 | serialized 1,2,#,#,3,4,#,#,5,#,#
trie search car true | prefix ca true | match c.r. true | board words car,cat,cats,core,cow,red,wax
all list/tree/trie asserts passed
```

Pattern 17 — Topological sort and union-find

What it is: two graph tools that plain BFS/DFS (Patterns 9–10) don't cover directly. A **topological order** lists the nodes of a directed graph (one whose edges point one way, like "A before B") so every edge goes from earlier to later ("take prerequisites first"); it exists only if the graph has no cycle. **Kahn's algorithm** builds it with BFS: repeatedly take a node whose **in-degree** (number of incoming edges still unprocessed) is 0. A **union-find** (disjoint-set union) keeps a collection of groups and answers "are x and y in the same group?" and "merge their groups" in almost constant time. **Signal:** "order of tasks with dependencies", "can all courses be finished" → topological sort; "number of connected groups", "does this edge create a cycle", merging accounts or friend circles → union-find. **Cost:** Kahn O(V + E); union-find O(α(n)) per operation, where α (the inverse Ackermann function) is below 5 for any real input.

```
function courseOrder(n, prereqs) {           // prereqs: [course, mustTakeFirst]
  const adj = Array.from({ length: n }, () => []);
  const indeg = new Array(n).fill(0);
  for (const [c, p] of prereqs) { adj[p].push(c); indeg[c]++; }
  const order = [];
  for (let i = 0; i < n; i++) if (indeg[i] === 0) order.push(i);
  for (let h = 0; h < order.length; h++)     // `order` doubles as the queue
    for (const v of adj[order[h]]) if (--indeg[v] === 0) order.push(v);
  return order.length === n ? order : null;  // null = cycle: impossible schedule
}
assert.deepEqual(courseOrder(4, [[1, 0], [2, 0], [3, 1], [3, 2]]), [0, 1, 2, 3]);
assert.equal(courseOrder(2, [[0, 1], [1, 0]]), null);

class UnionFind {
  constructor(n) {
    this.parent = Array.from({ length: n }, (_, i) => i);   // everyone starts alone
    this.size = new Array(n).fill(1);
    this.sets = n;
  }
  find(x) {                                  // follow parents to the group's root
    while (this.parent[x] !== x) {
      this.parent[x] = this.parent[this.parent[x]];        // path halving: flatten as we go
      x = this.parent[x];
    }
    return x;
  }
  union(a, b) {
    let ra = this.find(a), rb = this.find(b);
    if (ra === rb) return false;             // already in the same group
    if (this.size[ra] < this.size[rb]) [ra, rb] = [rb, ra];
    this.parent[rb] = ra;                    // hang the smaller tree under the bigger one
    this.size[ra] += this.size[rb];
    this.sets--;
    return true;
  }
}
const countComponents = (n, edges) => { const uf = new UnionFind(n); for (const [a, b] of edges) uf.union(a, b);
return uf.sets; };
assert.equal(countComponents(5, [[0, 1], [1, 2], [3, 4]]), 2);

function redundantEdge(edges) {              // first edge that closes a cycle (nodes 1..n)
  const uf = new UnionFind(edges.length + 1);
  for (const e of edges) if (!uf.union(e[0], e[1])) return e;
  return null;
}
assert.deepEqual(redundantEdge([[1, 2], [2, 3], [3, 4], [1, 4], [1, 5]]), [1, 4]);
```

Kahn vs the DFS topological sort in Pattern 10: both are O(V + E). Kahn detects a cycle by counting ("fewer than n nodes came out") instead of by recursion colours, needs no recursion (no depth limit), and processes the graph in "waves" — handy when asked for the minimum number of semesters. Union-find answers connectivity for edges that arrive one at a time, where rerunning BFS after each edge would cost O(V + E) each time. The two optimisations (path halving and union by size) are what make it fast; without them a chain of unions can make `find` O(n).

Pattern 18 — Greedy

What it is: a **greedy** algorithm makes the choice that looks best right now and never revisits it. It is fast (usually one pass, often after a sort) but only correct when you can argue that a locally best choice never blocks a globally best answer — so in the interview, always say *why* the greedy choice is safe, or give a counterexample (one input where it gives the wrong answer) and switch to DP (Pattern 11). **Signal:** "minimum number of …", "can you reach …", scheduling, and problems where a sort plus one sweep feels natural. **Cost:** O(n) or O(n log n) with a sort.

```
function canJump(nums) {                    // nums[i] = max jump length from i
  let reach = 0;                            // farthest index reachable so far
  for (let i = 0; i < nums.length; i++) {
    if (i > reach) return false;            // stuck before i
    reach = Math.max(reach, i + nums[i]);
  }
  return true;
}
assert.equal(canJump([3, 2, 1, 0, 4]), false);

function gasStation(gas, cost) {            // start index for a full circle, or -1
  let total = 0, tank = 0, start = 0;
  for (let i = 0; i < gas.length; i++) {
    const d = gas[i] - cost[i];
    total += d; tank += d;
    if (tank < 0) { start = i + 1; tank = 0; }   // nobody in start..i can be the answer
  }
  return total >= 0 ? start : -1;
}
assert.equal(gasStation([1, 2, 3, 4, 5], [3, 4, 5, 1, 2]), 3);

function candies(ratings) {                 // everyone >= 1; higher rating than a neighbour => more
  const n = ratings.length, c = new Array(n).fill(1);
  for (let i = 1; i < n; i++) if (ratings[i] > ratings[i - 1]) c[i] = c[i - 1] + 1;
  for (let i = n - 2; i >= 0; i--) if (ratings[i] > ratings[i + 1]) c[i] = Math.max(c[i], c[i + 1] + 1);
  return c.reduce((a, b) => a + b, 0);
}
assert.equal(candies([1, 3, 4, 5, 2]), 11);  // 1, 2, 3, 4, 1
```

The safety arguments, in one line each. **Jump game:** every index up to `reach` is reachable, so extending `reach` from each of them loses nothing. **Gas station:** if the tank goes negative at `i` after starting at `s`, every station between `s` and `i` arrived with at least 0 fuel from `s`, so starting there is no better — skip to `i + 1`; and if total gas covers total cost, the last surviving start completes the loop. **Candies:** the left-to-right pass satisfies every "better than my left neighbour" rule with the fewest candies, the right-to-left pass does the same for right neighbours, and `Math.max` keeps both. When greedy fails — coin change with coins `[1, 3, 4]` and amount 6 (greedy picks 4+1+1, the best is 3+3) — use DP. (Pattern 6's meeting-rooms and merge-intervals are greedy too: sort, then sweep.)

Pattern 19 — Sort & search: quickselect, Dutch flag, merge sort

Idea: some questions are really about the sorting algorithms themselves: find the k-th item *without* fully sorting, partition in place, or sort stably. **Quickselect** partitions like quicksort (a **pivot** value; smaller items to its left, the rest to its right) but then continues into only the side that contains the answer. **Signal:** "k-th largest/smallest", "median", "sort 0/1/2 in one pass", "sort a linked list", "count inversions" (pairs that are in the wrong order). **Cost:** quickselect O(n) average (O(n²) worst, made unlikely by a random pivot); Dutch flag O(n) one pass, O(1) space; merge sort O(n log n) always, O(n) extra space, **stable**.

```
function kthLargest(nums, k) {                // quickselect
  const a = [...nums], target = a.length - k;   // k-th largest = index n-k ascending
  let lo = 0, hi = a.length - 1;
  for (;;) {
    const r = lo + Math.floor(Math.random() * (hi - lo + 1));
    [a[r], a[hi]] = [a[hi], a[r]];            // random pivot, moved to the end
    const pivot = a[hi];
    let store = lo;                           // Lomuto partition
    for (let i = lo; i < hi; i++) if (a[i] < pivot) { [a[i], a[store]] = [a[store], a[i]]; store++; }
    [a[store], a[hi]] = [a[hi], a[store]];    // the pivot is now in its final sorted place
    if (store === target) return a[store];
    if (store < target) lo = store + 1; else hi = store - 1;   // keep ONE side
  }
}
assert.equal(kthLargest([3, 2, 1, 5, 6, 4], 2), 5);

function sortColors(a) {                      // Dutch national flag: 0s, 1s, 2s in one pass
  let lo = 0, mid = 0, hi = a.length - 1;     // [0,lo)=0s  [lo,mid)=1s  [mid,hi]=unknown  (hi,end]=2s
  while (mid <= hi) {
    if (a[mid] === 0) { [a[lo], a[mid]] = [a[mid], a[lo]]; lo++; mid++; }
    else if (a[mid] === 1) mid++;
    else { [a[mid], a[hi]] = [a[hi], a[mid]]; hi--; }   // don't move mid: new a[mid] is unchecked
  }
  return a;
}
assert.deepEqual(sortColors([2, 0, 2, 1, 1, 0]), [0, 0, 1, 1, 2, 2]);

function mergeSort(a, cmp = (x, y) => x - y) {
  if (a.length <= 1) return a;
  const mid = a.length >> 1;
  const L = mergeSort(a.slice(0, mid), cmp), R = mergeSort(a.slice(mid), cmp);
  const out = [];
  let i = 0, j = 0;
  while (i < L.length && j < R.length) out.push(cmp(L[i], R[j]) <= 0 ? L[i++] : R[j++]);   // <= : stable
  return out.concat(L.slice(i), R.slice(j));
}
const people = [['ann', 30], ['bob', 25], ['cy', 30], ['di', 25]];
assert.deepEqual(mergeSort(people, (x, y) => x[1] - y[1]).map(p => p[0]), ['bob', 'di', 'ann', 'cy']);
```

Quickselect vs a heap (Pattern 7): quickselect is O(n) on average but needs the whole array in memory and rearranges a copy; a size-k heap is O(n log k) but works on a stream. The code file also checks `kthLargest` against `toSorted` on 200 random arrays — a cheap way to test any randomised algorithm. Merge sort's merge step is the same idea as "merge two sorted lists" and, on a linked list, needs no extra array (find the middle with Pattern 14, sort both halves, merge). In everyday JavaScript, `toSorted` is already a stable O(n log n) sort (§17.1); write these only when the question is about the algorithm.

Pattern 20 — Bit manipulation

What it is: treating a number as its binary digits (**bits**) and using the bitwise operators: `&` (and), `|` (or), `^` (xor: 1 when the bits differ), `~` (not), `<<` (shift left = ×2), `>>` (shift right keeping the sign), `>>>` (shift right filling with 0). **Signal:** "without extra space, find the one that appears once", "count 1-bits", "power of two", subsets as bitmasks (an integer whose bits mark which items are chosen). **Cost:** O(1) per operation. **The JavaScript twist:** numbers are 64-bit floats, but every bitwise operator first converts to a **32-bit signed integer**, so results can come out negative and values above

2^{32} lose their high bits; `x >>> 0` reinterprets the 32 bits as unsigned (no sign bit, so never negative).

```
const singleNumber = (nums) => nums.reduce((acc, x) => acc ^ x, 0);   // x ^ x = 0, x ^ 0 = x
assert.equal(singleNumber([4, 1, 2, 1, 2]), 4);

function countBits(n) {                     // number of 1-bits for every i in 0..n
  const bits = new Array(n + 1).fill(0);
  for (let i = 1; i <= n; i++) bits[i] = bits[i >> 1] + (i & 1);   // DP: drop the last bit
  return bits;
}
assert.deepEqual(countBits(5), [0, 1, 1, 2, 1, 2]);

// swap every pair of neighbouring bits: 0xAAAAAAAA = odd positions, 0x55555555 = even
const swapOddEvenBits = (x) => (((x & 0xAAAAAAAA) >>> 1) | ((x & 0x55555555) << 1)) >>> 0;
assert.equal(swapOddEvenBits(0b10110), 0b101001);
assert.equal(swapOddEvenBits(0x40000000), 0x80000000);   // 2147483648, thanks to >>> 0
assert.equal(0x40000000 << 1, -2147483648);              // the trap without it

const isPowerOfTwo = (n) => n > 0 && (n & (n - 1)) === 0;   // n & (n-1) clears the lowest 1-bit
```

`singleNumber` works in any order because xor is commutative and associative (neither the order nor the grouping of the operations changes the result): the pairs cancel and only the lone value is left, with O(1) space instead of a `Set`. `countBits` is a tiny DP: `i >> 1` is `i` without its last bit (already computed) and `i & 1` is that last bit. In the swap, the final `>>> 0` matters: shifting a 1 into bit 31 makes the signed result negative. Because the file asserts against a slow bit-by-bit version on edge values (`0`, `0x80000000`, `0xFFFFFFFF`), the sign bug would be caught. For numbers beyond 32 bits, `BigInt` supports the same operators (`5n & 3n`), except `>>>`.

Pattern 21 — Math & geometry

Idea: simulation and number facts rather than a data structure: walk a matrix in a given order, handle integer limits, compare slopes exactly. The skill is spotting the invariant (a fact that stays true at every step) and the edge cases. **Signal:** "spiral / rotate a matrix", "reverse digits … return 0 on overflow", "points on a line", gcd (greatest common divisor), modular arithmetic (arithmetic with remainders, like hours on a clock). **Cost:** problem-specific — here O(rows × cols), O(digits) and $O(n^2)$.

```
function spiral(m) {                        // peel layers with four shrinking walls
  const out = [];
  let top = 0, bottom = m.length - 1, left = 0, right = m[0].length - 1;
  while (top <= bottom && left <= right) {
    for (let c = left; c <= right; c++) out.push(m[top][c]);
    top++;
    for (let r = top; r <= bottom; r++) out.push(m[r][right]);
    right--;
    if (top <= bottom) { for (let c = right; c >= left; c--) out.push(m[bottom][c]); bottom--; }
    if (left <= right) { for (let r = bottom; r >= top; r--) out.push(m[r][left]); left++; }
  }
  return out;
}
assert.deepEqual(spiral([[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12]]), [1, 2, 3, 4, 8, 12, 11, 10, 9, 5, 6, 7]);

function reverseInt(x) {                    // 0 if the result leaves the 32-bit signed range
  const sign = x < 0 ? -1 : 1;
  let n = Math.abs(x), r = 0;
  while (n > 0) { r = r * 10 + (n % 10); n = Math.floor(n / 10); }
  r *= sign;
  return r < -(2 ** 31) || r > 2 ** 31 - 1 ? 0 : r;
}
assert.equal(reverseInt(-120), -21);
assert.equal(reverseInt(1534236469), 0);     // 9646324351 > 2^31 - 1

function maxPointsOnLine(points) {          // most points on one straight line
  if (points.length <= 2) return points.length;
  const gcd = (a, b) => (b === 0 ? Math.abs(a) : gcd(b, a % b));
  let best = 0;
  for (let i = 0; i < points.length; i++) {
    const byDir = new Map();
    let same = 1, most = 0;                  // `same` = the anchor plus exact duplicates
    for (let j = i + 1; j < points.length; j++) {
      let dx = points[j][0] - points[i][0], dy = points[j][1] - points[i][1];
      if (dx === 0 && dy === 0) { same++; continue; }
      const g = gcd(dx, dy);
      dx /= g; dy /= g;
      if (dx < 0 || (dx === 0 && dy < 0)) { dx = -dx; dy = -dy; }   // one sign per direction
      const key = `${dx}/${dy}`;
      const c = (byDir.get(key) ?? 0) + 1;
      byDir.set(key, c);
      most = Math.max(most, c);
    }
    best = Math.max(best, most + same);
  }
  return best;
}
assert.equal(maxPointsOnLine([[1, 1], [3, 2], [5, 3], [4, 1], [2, 3], [1, 4]]), 4);
```

The two `if`s in `spiral` stop a single remaining row or column from being walked twice (try a 3×1 matrix). In `reverseInt`, JavaScript is *easier* than languages with fixed-size integers: numbers are exact up to 2^{53}, so the intermediate value cannot overflow and one range check at the end is enough; in Java or C++ you must check *before* `r * 10`. In `maxPointsOnLine`, a floating-point slope `dy / dx` would be both imprecise (in the code file, `94911150 / 94911151 === 94911151 / 94911152` is `true` although the two directions differ, so three non-collinear points — not on one straight line — would be counted as one line) and infinite for vertical lines; the reduced integer pair as a `Map` key avoids both — the same "canonical key" idea as Pattern 3's anagram grouping.

Output of the graph/greedy/sort/bit/math program:

```
course order 0,1,2,3 | cyclic null | components 2 | redundant [2,3]
canJump true false | gas start 3 | candies 5 11
2nd largest 5 | colors 0,0,1,1,2,2 | mergeSort stable bob,di,ann,cy
single 4 | countBits(5) 0,1,1,2,1,2 | swap bits 22 -> 41 | swap 0x40000000 -> 2147483648
spiral 1,2,3,6,9,8,7,4,5 | reverse -21 0 | max collinear 4
all graph/greedy/sort/bit/math asserts passed
```

Summary table

#	Pattern	Signal words	Time	Extra space
1	Two pointers	sorted, pair, palindrome, in place	O(n)	O(1)
2	Sliding window	longest/shortest contiguous	O(n)	O(k)
3	Hashing	seen before, count, group	O(n)	O(n)
4	Stack / monotonic stack	brackets, next greater	O(n)	O(n)
5	Prefix sums	range sum, subarray sum = k	O(n) + O(1)/query	O(n)
6	Intervals	overlap, merge, rooms	O(n log n)	O(n)
7	Heap (two heaps, k-way merge)	top k, cheapest next, running median	O(n log k)	O(k)
8	Binary search	sorted, minimum value that works	O(log n) × check	O(1)
9	BFS	fewest steps, levels	O(V + E)	O(V)
10	DFS	connected, cycle, order	O(V + E)	O(V)
11	DP + memo	count ways, min cost, overlapping calls	states × transitions	states
12	Backtracking	all combinations, puzzles	exponential	O(depth)
13	Linked lists	reverse, k-th from end, merge, intersection	O(n)	O(1)
14	Fast & slow pointers	cycle, cycle start, middle, "does it loop?"	O(n)	O(1)
15	Trees	validate BST, ancestor, path, serialize	O(n)	O(h)
16	Tries	prefix, autocomplete, wildcard, words in a grid	O(L) per word	O(total chars)
17	Topological sort / union-find	dependencies, order; groups, redundant edge	O(V + E); ≈ O(1) per op	O(V)
18	Greedy	reach, minimum count, schedule (prove the choice)	O(n) or O(n log n)	O(1)–O(n)
19	Sort & search	k-th largest, 3-way partition, stable sort	O(n) avg; O(n log n)	O(1); O(n)
20	Bit manipulation	appears once, count bits, power of two, masks	O(n) or O(1)	O(1)
21	Math & geometry	spiral, overflow, collinear points, gcd	problem-specific	O(1)–O(n)

In the interview

30-second answer — "How do you approach a coding problem?" Restate it; agree on input/output and edge cases (empty, duplicates, negatives). State the brute force (the obvious try-everything solution) and its cost. Find the pattern — sorted → binary search/two pointers; contiguous → window; lookups → hash map; fewest steps → BFS — and state the better complexity before coding. Then test by hand with a normal and an edge case.

- *Why BFS rather than DFS for shortest path?* BFS reaches nodes in order of distance, so the first visit is the shortest; DFS may find a long path first.
- *Top-down or bottom-up DP?* Top-down is easier to derive and computes only needed states; bottom-up avoids recursion-depth limits (important in JavaScript) and allows rolling arrays (keeping only the last row or two of the table).

- *How do you implement a priority queue in JavaScript?* A binary heap over an array: push = append + sift up, pop = move last to root + sift down, both O(log n).
- *Why a dummy head in linked-list problems?* It gives every real node a predecessor, so removing or inserting at the head needs no special case.
- *How do you prove a greedy solution?* Show that the greedy choice can be swapped into any optimal answer without making it worse (an **exchange argument**), or that it keeps an invariant (jump game: everything up to `reach` is reachable). If you can't, try a small counterexample and fall back to DP.
- *Quickselect or heap for the k-th largest?* Quickselect: O(n) average, needs the whole array. Heap of size k: O(n log k), works on a stream and has no bad worst case.
- *Kahn's algorithm or DFS for topological order?* Both O(V + E). Kahn counts nodes to detect a cycle and needs no recursion; DFS uses three colours and post-order.

Self-check

1. Which pattern finds the length of the longest substring with at most two distinct letters? → Sliding window with a `Map` of counts.
2. Why does `subarraySum` start the map with `[0, 1]`? → So subarrays that start at index 0 (prefix sum exactly `k`) are counted.
3. What's wrong with marking BFS cells visited when you dequeue them? → Duplicates enter the queue; still correct but can blow up time and memory.
4. What's the parent index of heap node 6? → `(6 − 1) >> 1 = 2`.
5. In `removeKthFromEnd`, why start both pointers at a dummy node? → So that when k equals the list length, `trail` can stop *before* the head and remove it.
6. Why must `isValidBST` pass a range instead of comparing each node with its parent? → A node can be correct for its parent yet violate an ancestor further up (4 under 7 under 5's right side).
7. What does `union` return in `redundantEdge`, and why does `false` mean a cycle? → `false` = the two ends already had the same root, so a path between them existed and this edge closes a loop.
8. Why does `swapOddEvenBits` end with `>>> 0`? → Bitwise results are 32-bit **signed**; a 1 in bit 31 would print as a negative number. `>>> 0` reads the same bits as unsigned.

TL;DR

Recognise the pattern, state the complexity, then code. Two pointers/window/hashing/prefix sums are O(n) array tricks; stacks handle nesting; sort first for intervals; heaps (hand-written in JS) for top-k, Dijkstra, k-way merge and — two of them — a running median; binary search on data or on the answer; BFS for fewest steps, DFS (iterative for deep inputs) for connectivity; memoize overlapping recursion; backtrack with choose/explore/un-choose and copy results. Linked lists: rewire with a dummy head; fast & slow pointers find cycles and middles in O(1) space; trees: recursion that combines subtree answers (pass ranges down for BSTs); tries for prefixes; Kahn's algorithm for dependency order, union-find for growing groups; greedy only with a safety argument; quickselect for the k-th item, Dutch flag for 3-way partitions, merge sort when stability matters; bitwise ops are 32-bit signed (`>>> 0` for unsigned); in math problems use exact integer keys, not float slopes.

17.3 Implement-from-scratch classics

What it is

"Implement `X` without using the built-in" questions test whether you understand *how* everyday tools work: closures and timers (`debounce`, `throttle`), `this` and `apply` (`bind`), recursion and object types (deep clone, deep equal), `Map` ordering (LRU cache), and promises (`Promise.all`). Each implementation below is short, correct for the cases interviewers probe, and tested.

How the tests control time

`debounce` and `throttle` depend on time, and real timers make tests slow and flaky. `code/04_classics_functions.mjs` uses the **fake timers** from `node:test` (§16.5): `setTimeout` and `Date.now()` only move forward when the test calls `tick(ms)`.

```
import { mock } from 'node:test';
mock.timers.enable({ apis: ['setTimeout', 'Date'] });
const tick = (ms) => mock.timers.tick(ms);
```

Debounce

What it is: a **debounced** function waits until calls *stop* for `wait` ms, then runs once with the latest arguments. Analogy: a lift door that stays open while people keep walking in, and closes only after nobody has come for a few seconds. **Use it for:** search-as-you-type, auto-save, resize handling.

```
function debounce(fn, wait) {
  let timer = null;
  function debounced(...args) {
    clearTimeout(timer);                          // every call restarts the countdown
    timer = setTimeout(() => { timer = null; fn.apply(this, args); }, wait);
  }
  debounced.cancel = () => { clearTimeout(timer); timer = null; };
  return debounced;
}

const calls = [];
const search = debounce((q) => calls.push(q), 300);
search('j'); tick(100); search('ja'); tick(100); search('jav'); // user still typing
tick(299); assert.deepEqual(calls, []);            // 299 ms after the last key: nothing yet
tick(1);   assert.deepEqual(calls, ['jav']);       // fires once, with the LAST args
search('x'); search.cancel(); tick(1000);
assert.deepEqual(calls, ['jav']);                  // cancelled
```

Output: `debounce calls: [ 'jav' ]`

Interviewers check: (1) the timer lives in a **closure** shared by all calls; (2) `debounced` is a regular `function` forwarding `this` and arguments with `apply`, so it works as a method; (3) a `cancel` method (Lodash — a popular utility library — also offers `flush` and `leading`).

Throttle

What it is: a **throttled** function runs **at most once per** `wait` **ms**, however often it's called. Analogy: a bus that leaves every 10 minutes whether 1 or 50 people arrive. **Use it for:** scroll and mousemove handlers, rate-limited API calls (APIs that allow only so many calls per second). This version runs on

the **leading** edge (immediately) and also on the **trailing** edge (once more at the end with the latest arguments) so the final position is never lost.

```
function throttle(fn, wait) {
  let last = -Infinity, timer = null, pendingArgs = null, pendingThis = null;
  function throttled(...args) {
    const now = Date.now(), remaining = wait - (now - last);
    if (remaining <= 0) {                          // window open: run now (leading edge)
      clearTimeout(timer); timer = null;
      last = now; fn.apply(this, args);
    } else {                                       // inside window: remember the latest call
      pendingArgs = args; pendingThis = this;
      timer ??= setTimeout(() => {                 // trailing edge
        last = Date.now(); timer = null;
        fn.apply(pendingThis, pendingArgs);
      }, remaining);
    }
  }
  return throttled;
}

const calls = [], t0 = Date.now();
const onScroll = throttle((y) => calls.push(`${y}@${Date.now() - t0}`), 100);
for (let t = 0; t <= 250; t += 10) { onScroll(t); tick(10); }  // an event every 10 ms
for (let i = 0; i < 20; i++) tick(10);                          // then 200 quiet ms
assert.deepEqual(calls, ['0@0', '90@100', '190@200', '250@300']);
```

Output: `throttle calls: [ '0@0', '90@100', '190@200', '250@300' ]` — 26 events became 4 calls, exactly 100 ms apart, and the last one carries the final value `250`.

	Debounce	Throttle
Runs	Once, after the calls stop	Regularly, at most once per interval
During a long burst	Never (until it ends)	Every `wait` ms
Typical use	Search box, auto-save, validation	Scroll, drag, resize-driven layout, analytics

In the browser, visual updates tied to scrolling are often better done with `requestAnimationFrame` (once per frame) — see §14.5.

Curry

What it is: **currying** turns `f(a, b, c)` into a chain `f(a)(b)(c)`. The usual interview version also accepts several arguments at once. It relies on `fn.length` — the number of declared parameters (it doesn't count default or rest parameters).

```
function curry(fn) {
  return function curried(...args) {
    if (args.length >= fn.length) return fn.apply(this, args);     // enough: call it
    return (...more) => curried.apply(this, [...args, ...more]);   // else collect more
  };
}
const add3 = curry((a, b, c) => a + b + c);
assert.equal(add3(1)(2)(3), 6);
assert.equal(add3(1, 2)(3), 6);
const add1 = add3(1);                                  // partials are reusable
assert.equal(add1(10)(20), 31); assert.equal(add1(0)(0), 1);

// Variant: unknown number of arguments, ended by an empty call
const sum = (a) => (b) => (b === undefined ? a : sum(a + b));
assert.equal(sum(1)(2)(3)(), 6);
```

Reusability works because each call builds a **new** argument array instead of pushing into a shared one — a common bug.

Compose and pipe

Function composition chains functions so the output of one is the input of the next. `compose(f, g)(x)` = `f(g(x))` (right to left, as in maths); `pipe(f, g)(x)` = `g(f(x))` (left to right, as you read).

```
const compose = (...fns) => (x) => fns.reduceRight((acc, f) => f(acc), x);
const pipe = (...fns) => (x) => fns.reduce((acc, f) => f(acc), x);
const inc = (x) => x + 1, dbl = (x) => x * 2;
assert.equal(compose(inc, dbl)(5), 11);               // inc(dbl(5))
assert.equal(pipe(inc, dbl)(5), 12);                  // dbl(inc(5))
assert.equal(pipe()(7), 7);                           // no functions = identity
```

Memoize

Memoization caches a pure function's results by its arguments. Key design questions: how to build the key (here `JSON.stringify(args)`, overridable), and where the cache lives (a `Map`, which can grow forever — production code bounds it, e.g. with the LRU cache below).

```
function memoize(fn, keyOf = (...args) => JSON.stringify(args)) {
  const cache = new Map();
  return function (...args) {
    const key = keyOf(...args);
    if (cache.has(key)) return cache.get(key);        // has(): a cached undefined still counts
    const value = fn.apply(this, args);
    cache.set(key, value);
    return value;
  };
}
let runs = 0;
const slowSquare = memoize((n) => { runs++; return n * n; });
slowSquare(4); slowSquare(4); slowSquare(5);
assert.equal(runs, 2);
// Recursive functions must call the MEMOIZED version, or only the top call is cached
const fib = memoize((n) => (n < 2 ? n : fib(n - 1) + fib(n - 2)));
```

Output: `memoize runs: 2 | fib(50): 12586269025`. `JSON.stringify` keys can't tell apart some values (`undefined` and `null` in arrays both become `null`; objects with different key orders differ) — mention it and offer a custom `keyOf`, or a `WeakMap` (a `Map` whose object keys don't stop garbage collection) for single-object arguments.

Flatten

Flatten a nested array to a given depth, exactly like the built-in `arr.flat(depth)` (§7.5):

```
function flatten(arr, depth = Infinity) {
  const out = [];
  for (const x of arr) {
    if (Array.isArray(x) && depth > 0) out.push(...flatten(x, depth - 1));
    else out.push(x);
  }
  return out;
}
function flattenDeep(arr) {                    // iterative: no recursion limit
  const stack = [...arr], out = [];
  while (stack.length) {
    const x = stack.pop();
    if (Array.isArray(x)) stack.push(...x); else out.push(x);
  }
  return out.reverse();                        // we took items from the end
}
const nested = [1, [2, [3, [4]], 5]];
assert.deepEqual(flatten(nested, 1), nested.flat(1));   // [1, 2, [3, [4]], 5]
assert.deepEqual(flattenDeep(nested), nested.flat(Infinity));

function flattenObject(obj, prefix = '', out = {}) {     // frequent variant: dotted keys
  for (const [k, v] of Object.entries(obj)) {
    const key = prefix ? `${prefix}.${k}` : k;
    if (v && typeof v === 'object' && !Array.isArray(v)) flattenObject(v, key, out);
    else out[key] = v;
  }
  return out;
}
assert.deepEqual(flattenObject({ a: 1, b: { c: 2, d: { e: 3 } } }), { a: 1, 'b.c': 2, 'b.d.e': 3 });
```

A `bind` polyfill (and `call`)

A **polyfill** is code that implements a standard feature for environments that lack it (§10.12). `bind` must (1) fix `this`, (2) pre-fill arguments (partial application), and (3) when the bound function is used with `new`, ignore the bound `this` and construct normally. How `this` works is in §9.1–9.2.

```
Function.prototype.myBind = function (thisArg, ...preset) {
  const target = this;                                 // the function myBind was called on
  if (typeof target !== 'function') throw new TypeError('myBind must be called on a function');
  function bound(...later) {
    if (new.target) return new target(...preset, ...later);   // called with `new`
    return target.apply(thisArg, [...preset, ...later]);
  }
  return bound;
};
const user = { name: 'Ada' };
function greet(greeting, punct) { return `${greeting}, ${this.name}${punct}`; }
const hi = greet.myBind(user, 'Hi');
assert.equal(hi('!'), 'Hi, Ada!');
assert.equal(hi.call({ name: 'Bob' }, '?'), 'Hi, Ada?');   // bound this can't be overridden
function Point(x, y) { this.x = x; this.y = y; }
const p = new (Point.myBind(null, 1))(2);
assert.ok(p instanceof Point);                                 // { x: 1, y: 2 }

Function.prototype.myCall = function (thisArg, ...args) {
  const ctx = thisArg == null ? globalThis : Object(thisArg);   // sloppy-mode rule
  const key = Symbol('fn');                                     // unique: can't clash
  ctx[key] = this;                                              // call it AS A METHOD of ctx
  try { return ctx[key](...args); } finally { delete ctx[key]; }
};
```

Output: `myBind: Hi, Ada! | new bound -> {"x":1,"y":2}`

`myCall` shows the classic trick: calling a function as a method of `ctx` makes `this === ctx`. (The real `call` doesn't box primitives — wrap them in object form, like `Object(5)` — or default to `globalThis` in strict code — that's the sloppy-mode behaviour.) Adding methods to built-in prototypes is fine in an interview but **not** in real code — it can clash with future standard methods.

Deep clone: `structuredClone` vs manual

A **shallow copy** (`{ ...obj }`, `[...arr]`, `Object.assign`) copies the top level only; nested objects are still shared. A **deep copy** copies every level. The built-in is `structuredClone` (browsers since 2022, Node 17+), which uses the *structured clone algorithm* (the one `postMessage` uses).

From `code/05_classics_data.mjs`:

```
const original = {
  when: new Date(0), tags: new Set(['a']), meta: new Map([['k', { n: 1 }]]),
  nested: { list: [1, { deep: true }] },
};
original.self = original;                                  // a cycle
const copy = structuredClone(original);                    // Date, Set, Map, cycles: all fine
structuredClone({ f() {} });                               // throws DataCloneError
structuredClone(new Point(-3));                            // becomes a plain { x: -3 }
JSON.parse(JSON.stringify({ d: new Date(0), u: undefined, m: new Map([[1, 2]]), n: NaN }));
```

Output:

```
structuredClone keeps Date/Set/Map/cycles: 1970-01-01T00:00:00.000Z [ 'a' ] true
function -> DataCloneError
class instance -> plain object: false {"x":-3}
JSON clone: { d: '1970-01-01T00:00:00.000Z', m: {}, n: null }
```

	Spread / `Object.assign`	`JSON.parse(JSON.stringify(x))`	`structuredClone(x)`	Manual `deepClone`
Depth	Top level only	Deep	Deep	Deep
`Date`	Shared	Becomes a string	✓	✓
`Map` / `Set`	Shared	Become `{}`	✓	✓
`undefined`, `NaN`, `Infinity`	✓	Dropped / `null`	✓	✓
Cycles	✓ (shared)	Throws `TypeError`	✓	✓ (with a `WeakMap`)
Functions	Shared	Dropped	Throws `DataCloneError`	Shared (as written)
Class prototype	Lost (spread gives a plain object)	Lost	Lost	Kept (as written)
Getters, property descriptors (a property's hidden settings: writable, enumerable, getter/setter)	Values copied	Values copied	Values copied	Values copied

The manual version interviewers ask for — a recursive walk that handles primitives, `Date`, `RegExp`, `Map`, `Set`, arrays, objects with their prototype, and **cycles** via a `WeakMap` of already-copied objects:

```
function deepClone(value, seen = new WeakMap()) {
  if (value === null || typeof value !== 'object') return value;   // primitives & functions
  if (seen.has(value)) return seen.get(value);                     // cycles / shared refs
  if (value instanceof Date) return new Date(value);
  if (value instanceof RegExp) return new RegExp(value.source, value.flags);
  if (value instanceof Map) {
    const m = new Map(); seen.set(value, m);                       // register BEFORE recursing
    for (const [k, v] of value) m.set(deepClone(k, seen), deepClone(v, seen));
    return m;
  }
  if (value instanceof Set) {
    const s = new Set(); seen.set(value, s);
    for (const v of value) s.add(deepClone(v, seen));
    return s;
  }
  const out = Array.isArray(value) ? [] : Object.create(Object.getPrototypeOf(value));
  seen.set(value, out);
  for (const key of Reflect.ownKeys(value)) {                      // strings AND symbols
    const desc = Object.getOwnPropertyDescriptor(value, key);
    if (desc.enumerable) out[key] = deepClone(value[key], seen);
  }
  return out;
}
```

Output: `deepClone keeps class: true | shared ref kept: true` — the copy of `new Point(-3)` still has `norm()`, and an object referenced twice in the original is one object (referenced twice) in the copy.

Which to use? `structuredClone` for data; a manual clone only to keep class instances — where a `clone()` method is usually clearer.

Deep equal

Compare two values structurally. Decisions to state out loud: `NaN` equals `NaN`? (yes, via `Object.is`); `0` vs `-0`? (not equal with `Object.is` — say which you want); do prototypes have to match? (yes, so `[1, 2]` ≠ `{0: 1, 1: 2}`); does `{a: 1}` equal `{a: 1, b: undefined}`? (no — the key sets differ).

```
function deepEqual(a, b, seen = new WeakMap()) {
  if (Object.is(a, b)) return true;                          // handles NaN and -0 vs +0
  if (typeof a !== 'object' || typeof b !== 'object' || a === null || b === null) return false;
  if (Object.getPrototypeOf(a) !== Object.getPrototypeOf(b)) return false;
  if (seen.get(a) === b) return true;                        // already comparing this pair (cycles)
  seen.set(a, b);
  if (a instanceof Date) return a.getTime() === b.getTime();
  if (a instanceof RegExp) return String(a) === String(b);
  if (a instanceof Map) {
    if (a.size !== b.size) return false;
    for (const [k, v] of a) if (!b.has(k) || !deepEqual(v, b.get(k), seen)) return false;
    return true;
  }
  if (a instanceof Set) {
    if (a.size !== b.size) return false;
    for (const v of a) if (!b.has(v)) return false;          // primitive members only (simplification)
    return true;
  }
  const ka = Reflect.ownKeys(a), kb = Reflect.ownKeys(b);
  if (ka.length !== kb.length) return false;
  return ka.every((k) => Object.hasOwn(b, k) && deepEqual(a[k], b[k], seen));
}
assert.ok(deepEqual({ a: [1, { b: NaN }] }, { a: [1, { b: NaN }] }));
assert.ok(!deepEqual({ a: 1 }, { a: 1, b: undefined }));
assert.ok(!deepEqual([1, 2], { 0: 1, 1: 2 }));
{ const x = { v: 1 }; x.me = x; const y = { v: 1 }; y.me = y; assert.ok(deepEqual(x, y)); }
```

In tests you don't write this: `assert.deepStrictEqual` does it (§16.2).

LRU cache

An **LRU (least recently used) cache** holds at most `capacity` entries; when full, it evicts the entry that was used longest ago. The textbook design is a hash map plus a doubly linked list (each node links to both the previous and the next one). Caches in general — keeping copies of results so repeat requests are fast — are covered in System Design Fundamentals. In JavaScript a single `Map` is enough, because a `Map` iterates in insertion order: delete-and-reinsert moves a key to the "newest" end, and `map.keys().next().value` is the oldest. Both operations are O(1).

```
class LRUCache {
  #cap; #map = new Map();
  constructor(capacity) { this.#cap = capacity; }
  get(key) {
    if (!this.#map.has(key)) return -1;
    const v = this.#map.get(key);
    this.#map.delete(key); this.#map.set(key, v);           // move to "most recent" end
    return v;
  }
  put(key, value) {
    this.#map.delete(key);                                   // updating also refreshes recency
    this.#map.set(key, value);
    if (this.#map.size > this.#cap) {
      const oldest = this.#map.keys().next().value;          // first key = least recent
      this.#map.delete(oldest);
    }
  }
  keys() { return [...this.#map.keys()]; }
}
const lru = new LRUCache(2);
lru.put('a', 1); lru.put('b', 2);
assert.equal(lru.get('a'), 1);          // a is now most recent
lru.put('c', 3);                        // evicts b
assert.equal(lru.get('b'), -1);
lru.put('a', 10);                       // update moves a to the end
assert.deepEqual(lru.keys(), ['c', 'a']);
```

If the interviewer forbids the `Map` trick, build the linked list: nodes with `prev`/`next`, a `Map` from key to node, and dummy head/tail nodes so insert and remove never special-case the ends. The LLD (low-level design) handbook builds the linked-list version (in Java) with eviction policies (rules for what to throw out) and thread safety (staying correct when several threads use it at once): §20.3.

EventEmitter

The **publish/subscribe** pattern: code registers listeners for named events; `emit` calls them in registration order. Node's own `EventEmitter` is covered in §15.8; this is the interview version with `on`, `off`, `once` and `emit`.

```
class EventEmitter {
  #listeners = new Map();                                  // event -> array of fns
  on(event, fn) {
    if (!this.#listeners.has(event)) this.#listeners.set(event, []);
    this.#listeners.get(event).push(fn);
    return () => this.off(event, fn);                      // handy unsubscribe
  }
  off(event, fn) {
    const list = this.#listeners.get(event);
    if (!list) return this;
    const i = list.findIndex((l) => l === fn || l.original === fn);
    if (i !== -1) list.splice(i, 1);
    return this;
  }
  once(event, fn) {
    const wrapper = (...args) => { this.off(event, wrapper); fn.apply(this, args); };
    wrapper.original = fn;                                 // so off(event, fn) still works
    return this.on(event, wrapper);
  }
  emit(event, ...args) {
    const list = this.#listeners.get(event);
    if (!list || list.length === 0) {
      if (event === 'error') throw args[0];                // same rule as Node
      return false;
    }
    for (const fn of [...list]) fn.apply(this, args);      // copy: listeners may unsubscribe
    return true;
  }
  listenerCount(event) { return this.#listeners.get(event)?.length ?? 0; }
}
```

Output: `EventEmitter log: [ 'A:1', 'once:1', 'B:1', 'B:2' ]`

A good answer iterates over a **copy** of the listener list (a `once` listener removes itself during `emit`, which would otherwise skip the next one), lets `off` remove a `once` listener by the original function, throws on an unhandled `'error'` like Node, and returns an unsubscribe function.

`Promise.all`, `allSettled`, `race`, `any` polyfills

How the real combinators behave is in §12.8. From `code/06_promise_polyfills.mjs`:

```
function promiseAll(iterable) {
  return new Promise((resolve, reject) => {
    const items = [...iterable];                        // accept any iterable
    const results = new Array(items.length);
    let remaining = items.length;
    if (remaining === 0) return resolve(results);       // all([]) resolves immediately
    items.forEach((item, i) => {
      Promise.resolve(item).then((v) => {               // wrap non-promises too
        results[i] = v;                                 // by index, not push -> keeps order
        if (--remaining === 0) resolve(results);
      }, reject);                                       // first rejection wins; later ones ignored
    });
  });
}

function promiseAllSettled(iterable) {                  // never rejects
  return promiseAll([...iterable].map((item) =>
    Promise.resolve(item).then(
      (value) => ({ status: 'fulfilled', value }),
      (reason) => ({ status: 'rejected', reason }))));
}

function promiseRace(iterable) {                        // first to SETTLE, either way
  return new Promise((resolve, reject) => {
    for (const item of iterable) Promise.resolve(item).then(resolve, reject);
  });                                                   // race([]) stays pending forever
}

function promiseAny(iterable) {                         // first to FULFIL
  return new Promise((resolve, reject) => {
    const items = [...iterable], errors = new Array(items.length);
    let remaining = items.length;
    if (remaining === 0) return reject(new AggregateError([], 'All promises were rejected'));
    items.forEach((item, i) => Promise.resolve(item).then(resolve, (e) => {
      errors[i] = e;
      if (--remaining === 0) reject(new AggregateError(errors, 'All promises were rejected'));
    }));
  });
}
```

The tests run each polyfill next to the built-in:

```
const all = await promiseAll([sleep(30, 'slow'), sleep(25, 'fast'), 42]);
assert.deepEqual(all, ['slow', 'fast', 42]);            // input order, not finish order
assert.deepEqual(await promiseAll([]), []);
await assert.rejects(promiseAll([sleep(20, 'ok'), sleep(5, 'bad', true)]), /bad/);
assert.equal(await promiseRace([sleep(20, 'a'), sleep(5, 'b')]), 'b');
assert.equal(await promiseAny([sleep(5, 'e1', true), sleep(15, 'win')]), 'win');
```

Output:

```
all: [ 'slow', 'fast', 42 ] | parallel (under 55 ms)? true
allSettled: [ 1, 'rejected(x)', 'y' ]
race: b
any all-rejected: AggregateError [ 'p', 'q' ]
pool results: [ 't0', 't1', 't2', 't3', 't4' ] peak concurrency: 2
all promise asserts passed
```

Four things interviewers look for in `Promise.all`: results **by index** (not `push`, which orders by finish time); a **counter**, not `results.length` (a pre-sized array's length is already full); `Promise.resolve(item)` so plain values work; and an **empty input** that resolves immediately. A settled promise ignores later calls, so `reject` can be passed straight through.

The same file also builds on these: a **timeout** (`race` a promise against a rejecting timer), **`promisify`** (wrap an error-first callback API — Node ships `util.promisify`), a **concurrency-limited pool** (run at most *n* tasks at once, results in order — the peak is asserted to be 2), and **retry**. The pool core:

```
async function pool(tasks, limit) {
  const results = new Array(tasks.length);
  let next = 0;
  async function worker() {
    while (next < tasks.length) {
      const i = next++;                    // no race: nothing else runs between these lines
      results[i] = await tasks[i]();
    }
  }
  await Promise.all(Array.from({ length: Math.min(limit, tasks.length) }, worker));
  return results;
}
```

More production-ready patterns (retry with back-off, stale responses) are in §12.13.

Common mistakes

- Using arrow functions for `debounced`/`throttled`/`memoized` wrappers, which silently drops the caller's `this`.
- `Promise.all` with `results.push(v)` (wrong order) or `if (results.length === n)` (fires too early).
- Deep clone without a `seen` map — infinite recursion on the first cycle.
- `curry` that mutates a shared argument array, so partial functions interfere with each other.
- `EventEmitter.emit` iterating the live array while a `once` listener removes itself.

In the interview

30-second answer — "Implement debounce." Return a wrapper holding a timer in a closure; each call clears it and starts a new `wait`-ms timer; when it fires, call the original with the latest arguments and `this`. Add `cancel`; mention the leading-edge option.

- *Debounce vs throttle?* Debounce waits for quiet and runs once; throttle runs at a steady maximum rate during activity.
- *Why store `Promise.all` results by index?* Promises finish in any order; the result must match the input order.
- *Why is a `Map` enough for an LRU cache?* It preserves insertion order and supports O(1) delete/insert, so "move to newest" = delete + set and "oldest" = first key.

Self-check

1. What does `debounce` do with the arguments of calls that were superseded? → Nothing — only the last call's arguments are used.
2. What does `Promise.race([])` return? → A promise that never settles.
3. Why does `deepClone` call `seen.set` *before* copying children? → So a child that points back to the parent finds the copy instead of recursing forever.
4. What is `fn.length` for `(a, b = 2, ...rest) => {}`? → `1` — counting stops at the first default or rest parameter.

TL;DR

Closures hold state (`debounce` timer, `memoize` cache, `curry` arguments); forward `this` with `apply`; `bind` must handle `new`. `structuredClone` for data, manual clone for class instances, `WeakMap` for cycles. LRU = one `Map`. EventEmitter: copy the list before emitting. `Promise.all`: index, counter, `Promise.resolve`, empty input.

17.4 Fifty-three tricky interview questions

How to use this section

Each question shows the code, the **real output** (from `code/07_qa_language.js` — CommonJS in sloppy mode on purpose, so legacy behaviour is visible — and `code/08_qa_async_and_memory.mjs`), a short answer you can say out loud, and a link to the section that teaches the topic properly. Cover the answer, predict the output, then check.

17.4.1 Coercion and equality

Q1. What do `1 + '2'`, `'5' - 2`, `'3' * '4'`, `true + 1`, `[] + []`, `[] + {}` and `[1,2] + [3]` give?

```
Q1  1 + '2'                    "12"
Q1  '5' - 2                    3
Q1  '3' * '4'                  12
Q1  true + 1                   2
Q1  [] + []                    ""
Q1  [] + {}                    "[object Object]"
Q1  [1,2] + [3]                "1,23"
```

`+` first turns objects into primitives (arrays → `"1,2"`-style strings, plain objects → `"[object Object]"`); if **either** side is then a string it concatenates, otherwise it adds. `-`, `*`, `/` always convert to numbers. (At the start of a console line, `{} + []` is `0`: `{}` parses as an empty block and `+[]` is unary plus — the `+x` operator that converts `x` to a number.) → §2.5, §2.12

Q2. What do `null == undefined`, `null == 0`, `null >= 0`, `'' == 0` and `'0' == false` give?

```
Q2  null == undefined          true
Q2  null == 0                  false
Q2  null >= 0                  true
Q2  '' == 0                    true
Q2  '0' == false               true
```

`==` converts types first (booleans and strings become numbers), but `null` and `undefined` are `==` only to each other, so `null == 0` is false. Relational operators (`<`, `>`, `<=`, `>=`) have no such exception — `null` becomes `0`, so `null >= 0` is true. Use `===`; keep `x == null` for a deliberate "null or undefined" check. → §2.6

Q3. Why is `[] == ![]` true?

```
Q3  [] == ![]                  true
```

`![]` is `false` (every object is truthy). Now `[] == false`: the boolean becomes `0`, the array becomes `''` then `0`, and `0 == 0`. The same array is "equal to its own negation" — the best argument for `===`. → §2.6, §2.11

Q4. What is `typeof null`, and why?

```
Q4  typeof null                "object"
```

A bug from the first engine (1995): values carried a type tag, objects had tag 0, and `null` (the null pointer — a reference to nothing, stored as 0) also read as tag 0. Fixing it would break the web. Use `x === null`, and `x !== null && typeof x === 'object'` for "a real object". → §2.3

Q5. How does `NaN` behave?

```
Q5  NaN === NaN                false
Q5  [NaN].includes(NaN)        true
Q5  [NaN].indexOf(NaN)         -1
Q5  typeof NaN                 "number"
```

`NaN` (the result of failed numeric operations like `0 / 0`) is the only value not equal to itself. `indexOf` uses `===`, so it can't find it; `includes`, `Map` and `Set` use SameValueZero, where `NaN` equals

`NaN`. Test with `Number.isNaN`, not the global `isNaN`, which converts first (`isNaN('x')` is `true`). → §2.10

Q6. How does `Object.is` differ from `===`?

```
Q6  Object.is(-0, 0) / -0 === 0    "false / true"
```

Only in two cases: `Object.is(NaN, NaN)` is `true` and `Object.is(0, -0)` is `false`. React uses `Object.is` to decide whether state changed. → §2.6

Q7. What does `typeof` return for arrays, functions, classes and undeclared variables?

```
Q7  typeof [] / function / class   "object / function / function"
Q7  typeof undeclaredVar           "undefined"
```

Arrays are objects — use `Array.isArray`. Classes are functions underneath. `typeof` on a name that was never declared returns `'undefined'` instead of throwing (useful for feature detection — checking whether an API exists before using it) — but see Q16 for the TDZ exception. → §2.3

Q8. Which values are falsy? Is `[]` truthy? And `new Boolean(false)`?

```
Q8  Boolean([]), Boolean('0')      "true, true"
Q8  !!new Boolean(false)           true
```

Falsy: `false`, `0`, `-0`, `0n`, `''`, `null`, `undefined`, `NaN` (plus the legacy `document.all`). Everything else is truthy — `[]`, `{}`, `'0'`, `'false'`, even a `Boolean` object holding `false`. Test an empty array with `arr.length === 0`. → §2.11

Q9. What does `'b' + 'a' + +'a' + 'a'` give?

```
Q9  'b' + 'a' + +'a' + 'a'         "baNaNa"
```

`+'a'` is unary plus: it converts `'a'` to a number, which is `NaN`; concatenation turns it into the text `"NaN"`. → §2.5

Q10. Why does `['1', '7', '11'].map(parseInt)` give `[1, NaN, 3]`?

```
Q10 ['1','7','11'].map(parseInt)   [ 1, NaN, 3 ]
Q10 ['1','7','11'].map(Number)     [ 1, 7, 11 ]
```

`map` passes `(value, index, array)`, and `parseInt`'s second parameter is the **radix** (the number base, e.g. 10 for decimal, 2 for binary): `parseInt('1', 0)` (0 = default base 10 → 1), `parseInt('7', 1)` (invalid base → `NaN`), `parseInt('11', 2)` (binary → 3). Fix: `map(Number)` or `map((s) => parseInt(s, 10))`. → §7.5, §2.10

Q11. Why does `[10, 1, 2].sort()` give `[1, 10, 2]`?

```
Q11 [10, 1, 2].sort()              [ 1, 10, 2 ]
```

Without a comparator, `sort` compares elements as strings, and `'10' < '2'` because `'1' < '2'`. Pass `(a, b) => a - b`. → §7.6

Q12. What's the difference between `||` and `??`?

```
Q12 0 || 'x' / 0 ?? 'x'            "x / 0"
```

`a || b` falls back to `b` when `a` is **any falsy** value, so it wrongly replaces a legitimate `0`, `''` or `false`. `a ?? b` (nullish coalescing, ES2020) falls back only when `a` is `null` or `undefined`. Use `??` for defaults. → §2.7

Q13. Why is `{} === {}` false?

```
Q13 {} === {}                      false
```

Objects (including arrays and functions) are compared by **reference** — "is this the same object in memory?" — not by content. Two literals create two objects. To compare content, use a deep-equal (§17.3). → §2.4

Q14. Why is `0.1 + 0.2 !== 0.3`? And why is `Math.max() < Math.min()`?

```
Q14 0.1 + 0.2 === 0.3              false
Q14 Math.max() < Math.min()        true
```

0.1 and 0.2 have no exact binary (IEEE 754) form, so the sum is `0.30000000000000004` — compare with a tolerance or use integers (§17.1). With no arguments `Math.max()` is `-Infinity` (the identity for "max") and `Math.min()` is `Infinity`. → §2.10

17.4.2 Scope, hoisting and closures

Q15. What are the differences between `var`, `let` and `const`?

	`var`	`let`	`const`
Scope	Function	Block `{ }`	Block `{ }`
Before the declaration line	`undefined` (hoisted and initialised)	`ReferenceError` (TDZ)	`ReferenceError` (TDZ)
Redeclare in the same scope	Allowed	`SyntaxError`	`SyntaxError`
Reassign	Yes	Yes	No (`TypeError`)
At top level of a script	Becomes a property of `globalThis`	Doesn't	Doesn't

`const` stops *reassignment* of the variable, not mutation of the object it points to (Q44). Default to `const`, use `let` when you must reassign, avoid `var`. → §2.1

Q16. What happens when you read a variable before its declaration?

```
Q16 var read before declaration    "undefined"
Q16 let read before declaration    "ReferenceError: Cannot access 'tdz' before initialization"
Q16 typeof in the TDZ              "ReferenceError: Cannot access 'inTdz' before initialization"
```

All declarations are **hoisted** (registered on entering the scope); `var` is also initialised to `undefined`. `let`, `const` and `class` stay uninitialised until their line runs — the **temporal dead zone (TDZ)** — and any access throws, even `typeof`. → §2.2

Q17. Can you call a function before it's defined?

```
Q17 declaration hoisted            "works"
Q17 expression not hoisted         "TypeError: late is not a function"
```

A function **declaration** is hoisted with its body. A function **expression** in a `var` hoists only the variable (`undefined`), so calling it throws `TypeError`; with `let`/`const` it's a TDZ `ReferenceError`. → §4.2

Q18. What do these loops print?

```
for (var i = 0; i < 3; i++) setTimeout(() => out.var.push(i), 0);
for (let j = 0; j < 3; j++) setTimeout(() => out.let.push(j), 0);
for (var k = 0; k < 3; k++) ((k2) => setTimeout(() => out.fixed.push(k2), 0))(k);
```

```
Q18 closures in loops              {"var":[3,3,3],"let":[0,1,2],"fixed":[0,1,2]}
```

The callbacks run after the loop. `var` has **one** `i` for the whole function — `3` by then. `let` in a `for` header creates a **fresh binding per iteration**. The pre-ES2015 fix was an IIFE (immediately invoked function expression — a function defined and called on the spot) copying the value into a parameter. → §4.7

Q19. What is a closure? Show one.

```
const makeCounter = () => { let n = 0; return () => ++n; };
const c1 = makeCounter(), c2 = makeCounter();
c1(); c1();
```

```
Q19 counter via closure           "c1: 3, c2: 1"
```

A **closure** is a function plus the variables of the scope where it was created, kept alive and updatable after the outer function returns. Each `makeCounter()` call creates a new `n`, so `c1` and `c2` are independent. Uses: private state, factories, `debounce`/`memoize` (§17.3). → §4.7

Q20. What's wrong with `let a = b = 5`?

```
(function () { let a = b = 5; })();
```

```
Q20 let a = b = 5                 "outside: typeof a=undefined, typeof b=number"
```

It parses as `b = 5; let a = b;`. `a` is local, but `b` was never declared, so in sloppy mode the assignment creates an **accidental global**. In strict mode it throws `ReferenceError` (Q22). → §4.5

Q21. Is a `var` inside an `if` block visible outside it?

```
Q21 var in block leaks            "v=1, typeof l=undefined"
```

Yes: `var` ignores blocks and belongs to the whole function. `let`/`const` are confined to the block. → §4.5

Q22. What does strict mode change?

```
Q22/Q23 strict: lost this      TypeError: Cannot read properties of undefined (reading 'name')
Q22 strict: undeclared assign  ReferenceError: undeclaredThing is not defined
```

Strict mode (automatic in ES modules and class bodies) turns silent mistakes into errors: undeclared assignments and writes to read-only properties throw, `this` in a plain call is `undefined` instead of `globalThis`, and duplicate parameter names and `with` are syntax errors. → §1.8

17.4.3 `this`, arrow functions, `call`/`apply`/`bind`, `new`

Q23. Why does a method lose `this` when you pass it around?

```
const f = user.regular; f();
```

```
Q23 method call                   "Ada"
Q23 extracted method (sloppy)     undefined
```

`this` is decided by **how** a function is called. `user.regular()` sets `this = user`; a plain `f()` gives `undefined` in strict code (→ `TypeError`, Q22) or `globalThis` in sloppy code (→ `undefined` name). It bites with callbacks like `setTimeout(user.regular)`. Fix: `user.regular.bind(user)` or `() => user.regular()`. → §9.1

Q24. How do arrow functions differ from regular functions?

```
Q24 arrow in object literal       "object"
```

	Regular function	Arrow function
`this`	Set by the call	Taken from the surrounding scope (lexical); `call`/`bind` can't change it
`arguments`	Has its own	None (uses the outer one); use rest `...args`
`new`	Can be a constructor	`TypeError` — no `prototype`
Good for	Methods, constructors	Callbacks inside methods

So an arrow used as an object-literal *method* doesn't see the object: its `this` is the outer scope's — here CommonJS's `module.exports`; in an ES module, `undefined`. → §4.3

Q25. Inside a method, what is `this` in a nested regular function vs a nested arrow?

```
delayed() { return [function () { return this === globalThis; }(), (() => this.name)()]; }
```

```
Q25 inner function vs arrow        [ true, 'Ada' ]
```

The inner regular function is called plainly (default `this`: global object in sloppy mode, `undefined` in strict); the arrow inherits the method's `this` — why arrows suit callbacks like `setTimeout(() => this.save(), 100)` inside methods. → §9.1

Q26. What's the difference between `call`, `apply` and `bind`?

```
Q26 call / apply / bind            "Hi Lin! | Yo Lin? | Hey Lin."
```

All three set `this`. `call(thisArg, a, b)` calls now with arguments listed; `apply(thisArg, [a, b])` calls now with an array; `bind(thisArg, a)` **returns a new function** with `this` (and optionally leading arguments) fixed, to call later. Memory aid: **A**pply = **A**rray. → §9.2

Q27. What happens if you `bind` twice?

```
Q27 bind twice                     1
```

The first `bind` wins: a bound function ignores any later `this` from `bind`, `call` or `apply` (the only exception is `new`). → §9.2

Q28. What does `new` do? What if the constructor returns something?

```
Q28 new returns object?            "{\"b\":2}"
Q28 new returns primitive?         "{\"a\":1}"
```

`new F(args)`: (1) creates an empty object whose prototype is `F.prototype`; (2) calls `F` with `this` set to that object; (3) returns that object — **unless** `F` explicitly returns an *object*, which then replaces it. Returned primitives are ignored. → §9.4

17.4.4 Prototypes and classes

Q29. What's the difference between `prototype` and `__proto__`?

```
Q29 proto links                    "true true true"
Q29 own vs inherited               "true false true"
```

Every object has an internal `[[Prototype]]` link (read with `Object.getPrototypeOf`; `__proto__` is the legacy accessor — an old getter/setter property for the same link); missed lookups continue along this **prototype chain**. `F.prototype` is an ordinary property of **functions**: the object `new F()` installs as the instance's `[[Prototype]]`. So methods live on `Animal.prototype` (shared) and data on the instance (`Object.hasOwn(rex, 'speak')` is false, `'speak' in rex` is true). → §9.3

Q30. How does `instanceof` work? Can it be fooled?

```
Q30 instanceof after proto swap    true
```

`x instanceof F` walks `x`'s prototype chain looking for `F.prototype`. It doesn't check how `x` was created, so a plain object whose prototype was set to `Animal.prototype` passes. It also fails across realms (separate JavaScript environments with their own built-ins — an array from an iframe, a page embedded in another, isn't `instanceof Array` here — use `Array.isArray`). → §9.7

Q31. How are classes different from constructor functions?

```
Q31 class before declaration       "ReferenceError: Cannot access 'Later' before initialization"
Q31 class without new              "TypeError: Class constructor K cannot be invoked without 'new'"
```

Mostly syntax over prototypes, but: class declarations are in the TDZ until defined; calling without `new` throws; bodies are strict; methods are non-enumerable; and classes add real features — `#private`, `static` blocks, `super`. → §9.5

Q32. What is `Object.create(null)` for?

```
Q32 Object.create(null)          "undefined true"
```

It creates an object with **no prototype** — no inherited `toString`, `constructor` or `__proto__` — a safe "dictionary" where keys like `'constructor'` from user input can't collide with built-ins. A `Map` is usually the better choice today. → §9.3

Q33. How do `#private` fields compare with closure-based privacy?

```
class Wallet { #cents = 0; add(c) { this.#cents += c; return this.#cents; } }
```

```
Q33 #private vs closure          "10 undefined 0 {}"
```

`#cents` is truly private — invisible to `w.cents`, `Object.keys` and `JSON.stringify`, and a *syntax* error outside the class body. Closure privacy also hides state, but every instance needs its own method copies; `#` fields keep methods shared on the prototype. → §9.5

Q34. What happens if a subclass constructor uses `this` before `super()`?

```
Q34 this before super()          "ReferenceError: Must call super constructor in derived class before accessing
'this' or returning from derived constructor"
```

In a derived class (one that `extends` another) the **parent** constructor creates the object, so `this` doesn't exist until `super(...)` returns. → §9.6

17.4.5 The event loop and async code

For all of these, apply the rules from §12.2–12.3: run all synchronous code first; then drain the **microtask** queue (promise callbacks, code after `await`, `queueMicrotask`) completely; then run **one task** (timer, I/O, event); drain microtasks again; repeat.

Q35. The classic.

```
log('1');
setTimeout(() => log('2'), 0);
Promise.resolve().then(() => log('3'));
log('4');
```

```
Q35 classic                  1 4 3 2
```

`1` and `4` are synchronous. The `.then` callback is a microtask, so it runs as soon as the script finishes; the timer is a task and waits for the next loop turn — even with a 0 ms delay. → §12.2

Q36. `async`/`await` ordering.

```
async function a() { log('a1'); await b(); log('a2'); }
async function b() { log('b'); }
log('s1');
setTimeout(() => log('t'), 0);
a();
new Promise((res) => { log('p'); res(); }).then(() => log('then'));
log('s2');
```

```
Q36 async/await              s1 a1 b p s2 a2 then t
```

An `async` function runs **synchronously until its first** `await` (`a1`, `b`); the rest (`a2`) becomes a microtask. The promise **executor** is synchronous too (`p`). After `s2`, microtasks run in queue order — `a2`, then `then` — and the timer last. → §12.3, §12.9

Q37. `process.nextTick`, promises and `setImmediate` in Node.

```
setImmediate(() => q.push('immediate'));
Promise.resolve().then(() => q.push('then'));
process.nextTick(() => q.push('nextTick'));
q.push('sync');
```

```
Q37 CommonJS main                 sync nextTick then immediate
Q37 ESM (after await)          sync then nextTick immediate
```

Node normally drains `nextTick` **before** promise microtasks (the CommonJS result). Inside an ES module (here after a top-level `await`) the code already runs in a promise job, so promises drain first. `setImmediate` runs later, in the *check* phase. Don't rely on the `nextTick`/promise order; prefer `queueMicrotask`. → §12.4

Q38. When does a promise executor run, and what if you resolve twice?

```
Q38 executor + settle once     executor after first
```

The function passed to `new Promise(...)` runs **synchronously**, right away. A promise settles **once**: the first `resolve`/`reject` wins, later calls are ignored silently. → §12.7

Q39. Why doesn't this `try`/`catch` catch the error?

```
try { setTimeout(() => { throw new Error('boom'); }, 0); } catch { /* never runs */ }
try { await Promise.reject(new Error('rejected')); } catch (e) { /* runs */ }
```

```
Q39 errors in callbacks        uncaught: boom | await caught: rejected
```

`try` only covers code running *while it's on the stack*. The timer callback runs later, so its error is uncaught (the demo shows it via `process.on('uncaughtException')`; normally Node would crash). With `await`, the rejection is thrown back into the function at the `await`, so `try`/`catch` works. → §11.5, §12.10

Q40. Does `await` inside `forEach` wait?

```
[20, 10].forEach(async (ms) => { await wait(ms); log.push(ms); });
log.push('forEach returned');
for (const ms of [20, 10]) { await wait(ms); log.push(`for-of ${ms}`); }
```

```
Q40 await in forEach           forEach returned | 10 | 20 | for-of 20 | for-of 10
```

No. `forEach` ignores the promise each async callback returns, so it returns immediately and callbacks finish in timer order. Use `for…of` + `await` for sequential work, `await Promise.all(arr.map(async …))` for parallel. → §12.9

Q41. Two independent `await`s — how long do they take?

```
Q41 sequential vs parallel     sequential >= 80ms: true, parallel < 80ms: true
```

`await wait(40); await wait(40);` takes ~80 ms because the second doesn't start until the first finishes. Starting both and awaiting `Promise.all([...])` takes ~40 ms. Start independent work first, await later. → §12.9

Q42. `Promise.all` vs `allSettled` vs `race` vs `any`?

Combinator	Fulfils when	Rejects when
`all`	All fulfil (array of values, input order)	The first rejection
`allSettled`	All settle (array of `{status, value/reason}`)	Never
`race`	The first to settle fulfils	The first to settle rejects
`any`	The first fulfils	All reject (`AggregateError`)

Implementations and tests: §17.3. → §12.8

17.4.6 Objects, arrays and copies

Q43. Shallow vs deep copy — what does this print?

```
const a = { n: { v: 1 } }; const b = { ...a }; b.n.v = 2; a.n.v;
```

```
Q43 spread copy is shallow        2
```

Spread copies only the top level; `b.n` and `a.n` are the same nested object. For a deep copy use `structuredClone` (or the manual clone, §17.3). → §5.4

Q44. Can you change a `const` object? Does `Object.freeze` make it immutable?

```
Q44 const + mutate                2
Q44 freeze is shallow             "2 undefined"
```

`const` protects the **binding**, not the object. `Object.freeze` blocks adding, removing and changing *own* properties (silently in sloppy mode, `TypeError` in strict) but is **shallow** — nested objects stay mutable; deep-freeze by recursing. → §5.6

Q45. What happens when you use an object as an object key?

```
const o = {}; o[{}] = 'a'; o[{ id: 2 }] = 'b'; o[1] = 'c';
```

```
Q45 object keys become strings    "[\"1\",\"[object Object]\"] b"
```

Property keys are strings (or symbols). Both objects convert to the same string `"[object Object]"`, so the second assignment overwrites the first; `1` becomes `"1"`. Use a `Map` for object keys. → §5.2, §8.1

Q46. What do `delete arr[i]` and `arr.length = n` do?

```
Q46 array holes                   "3 [1,null,3] false"
Q46 length truncates              "[1]"
Q46 length of sparse              6
```

`delete` removes the element but leaves a **hole** — the length stays 3 (JSON shows the hole as `null`). Use `splice` to remove. Setting `length` smaller truncates the array (`arr.length = 0` empties it in place). Assigning `a[5]` on an empty array makes `length` 6 with five holes. → §7.3

Q47. Is JavaScript pass-by-value or pass-by-reference?

```
const o = { v: 1 };
(function (p) { p = { v: 99 }; })(o);   // reassign the parameter
(function (p) { p.v = 2; })(o);          // mutate through it
o.v;
```

```
Q47 reassign param                2
```

Always **by value** — but an object's value *is a reference* ("call by sharing"): mutating through the parameter changes the caller's object; reassigning it changes only the local variable. → §2.4

Q48. When should you use a `Map` instead of an object?

Use a `Map` for dynamic keys added and removed at runtime, non-string keys, frequent size checks and guaranteed insertion order; use a plain object for fixed-shape records (a user, a config) and JSON. Objects also have inherited keys (`'toString' in {}` is true) — a source of bugs with user-supplied keys. → §7.8, §8.1

Q49. What does `JSON.stringify` drop?

```
Q49 JSON drops                    "{\"n\":null,\"d\":\"1970-01-01T00:00:00.000Z\"}"
Q49 JSON array holes              "[null,null]"
```

Object properties holding `undefined`, a function or a symbol are **omitted** (in arrays they become `null`); `NaN` / `Infinity` → `null`; dates → ISO strings (the standard `2026-01-15T10:00:00.000Z` format); `Map` / `Set` → `{}`; `BigInt` and cycles throw. → §5.12

Q50. `for…in` vs `for…of`?

```
Q50 for...in vs for...of          "in: 0,1,extra | of: a,b"
```

`for…in` loops over **enumerable property keys** (strings; enumerable = visible to loops, which ordinary properties are), including inherited ones and extras like `extra` — meant for plain objects. `for…of` loops over the **values** of any *iterable*; use it for arrays. → §3.10

17.4.7 Garbage collection and memory leaks

Q51. How does JavaScript free memory?

Automatically, with a **garbage collector (GC)**. An object stays alive while it is **reachable** from a *root* (globals, the call stack, active closures, pending callbacks). Unreachable objects are freed even if they reference each other — **mark-and-sweep** marks everything reachable and sweeps the rest, so cycles are fine. V8 is **generational**: a small *young generation* is collected often and cheaply (most objects die young), survivors move to an *old generation* collected less often, much of it on helper threads. Normal code can't (and needn't) trigger GC. → §8.3

Q52. What causes memory leaks in JavaScript, and how do you fix them?

A **leak** is memory that is still reachable but no longer needed. The usual causes:

Cause	Example	Fix
Forgotten listeners and timers	`setInterval` or `addEventListener` never removed; the callback's closure keeps data alive	`clearInterval`, `removeEventListener`, `AbortController` signal, React effect cleanup
Unbounded caches and collections	A `Map` from request/DOM objects to data that only grows	Size limit (LRU), or a `WeakMap` keyed by the object
Closures capturing big data	A long-lived callback references a huge array it doesn't need	Copy only what's needed into the closure; set big references to `null`
Detached DOM nodes	A removed element still referenced from a JS variable or array	Drop the reference when the element is removed
Accidental globals	`x = …` without declaration (sloppy mode)	Strict mode / modules

The demo stores ~8 KB of data per short-lived "request" object in a `Map` and in a `WeakMap`, then forces a GC (only possible with a V8 flag, used here for the demonstration):

```
Q52 Map vs WeakMap cache          Map retains > 10 MB: true, WeakMap retains < 2 MB: true
```

The `Map` keeps every key object and its data alive forever; the `WeakMap` lets both be collected once the request object is unreachable. Find leaks with DevTools' **Memory** panel: take heap snapshots (a dump of every object in memory at that moment) before and after an action and compare what grew. → §4.7, §8.3, §11.9

Q53. What are `WeakMap`, `WeakRef` and `FinalizationRegistry` for?

```
Q53 WeakRef before                true
Q53 WeakRef after gc              collected
```

`WeakMap` / `WeakSet` hold keys **weakly** — an entry vanishes with its key, ideal for metadata or caches on objects you don't own. A `WeakRef` points to an object without keeping it alive (`deref()` → the object or `undefined`); `FinalizationRegistry` calls back some time after collection. GC timing is **not guaranteed**, so use these only for optional optimisations, never program logic. → §8.3

In the interview

30-second answer — "`this`, closures and the event loop in one breath?" `this` comes from the call site (method, plain call, `new`, `call` / `apply` / `bind`) except in arrows, which take it lexically. A closure is a function plus its creation scope's variables, kept alive with it. The event loop runs sync code, then all microtasks, then one task, repeat — so promise callbacks beat `setTimeout(0)`.

Self-check

1. `console.log([] + null + 1)`? → `"null1"`: `[]` becomes `''`, then string concatenation.
2. Why does `for (var i…) setTimeout(() => console.log(i))` print the final value? → One shared `var` binding; `let` creates one per iteration.
3. In `async function f() { console.log(1); await null; console.log(2); } f(); console.log(3);` — the order? → `1 3 2`.
4. `typeof typeof 1`? → `'string'` (`typeof 1` is the string `'number'`).
5. Does `Object.freeze` stop `obj.nested.x = 1`? → No, freeze is shallow.

TL;DR

Use `===`; know the falsy list, `typeof null`, `NaN` rules and how `+` concatenates. `var` is function-scoped and hoisted as `undefined`; `let` / `const` are block-scoped with a TDZ; `let` in loops gives one binding per iteration. `this` depends on the call; arrows inherit it; `bind` wins once; `new` returns the new object unless you return an object. Classes are prototypes plus TDZ, strictness and `#private`. Sync → microtasks → one task. Spread is shallow; `structuredClone` is deep. Leaks come from things that stay reachable — listeners, timers, caches, closures; `WeakMap` and cleanup fix them.

Chapter 17 TL;DR

- **Toolkit:** `push` / `pop` stack; head-index queue (no `shift` in hot loops); `Array.from` grids; `Map` / `Set` with string keys for pairs; `sort` needs a comparator, is stable, mutates (`toSorted` doesn't); floats — tolerance, `Math.trunc`, `%` sign, safe to $2^{53} - 1$, then `BigInt`; build strings with `join`.
- **Patterns (21):** two pointers, sliding window, hashing, stack/monotonic stack, prefix sums, intervals (sort + sweep), heap (hand-written `MinHeap`, O(log n) push/pop; two heaps for a running median; k-way merge), binary search (on data or the answer), BFS (fewest steps, mark on enqueue), DFS (iterative for deep inputs), DP with memo, backtracking (choose/explore/un-choose, copy results), linked lists (dummy head, reverse, k-th from end), fast & slow pointers (cycle start, middle), trees (validate with ranges, LCA, serialize), tries (prefix, wildcard, board words), Kahn's topological sort and union-find, greedy (with a proof), quickselect / Dutch flag / merge sort, bit manipulation (32-bit signed, `>>> 0`), math & geometry (spiral, overflow, gcd-reduced slopes).
- **Classics:** `debounce` (clear + restart a timer) vs `throttle` (at most once per interval, leading + trailing); `curry` via `fn.length`; `compose` = `reduceRight`, `pipe` = `reduce`; `memoize` with a `Map`; `flatten` recursive or with a stack; `bind` must support `new`; `structuredClone` vs manual `deepClone` with a `WeakMap`; `deepEqual` with `Object.is`; LRU = one `Map`; EventEmitter copies listeners before emitting; `Promise.all` = index + counter + `Promise.resolve` + empty case.
- **Q&A:** coercion and `==` rules, `typeof null`, `NaN`, `??` vs `||`; hoisting and TDZ, closures in loops, accidental globals, strict mode; `this` rules, arrows, `call` / `apply` / `bind`, `new`; prototype chain, `instanceof`, classes vs functions, `#private`; event-loop order (sync → microtasks → task), `nextTick`, errors in callbacks, `await` in `forEach`, parallel awaits; shallow vs deep copy, `freeze`, keys as strings, holes, call by sharing, JSON; GC by reachability, leaks and weak references.

Next: the cheat sheets and glossary — Chapter 18.

Chapter 18 — Cheat Sheets and Glossary

This last chapter squeezes Chapters 1–17 into pages you can scan in a minute. It has one **cheat sheet** per area, a **"which API for which job"** table, a verified **ECMAScript version table** (ES2015 → ES2026), a **"where to go next"** plan, an **index of the interview questions** in Chapter 17, and an alphabetical **glossary** of 700+ terms, each linked to the section that explains it properly.

Nothing here is new. If a line doesn't make sense, follow its link back to the full explanation. The terms, acronyms and tool names used across the book (from *API* and *bytecode* to *trie* and *Zod*) each have a one-line, plain-English definition in the glossary, so if a word on a sheet is new to you, look it up there first. References are written like **§12.2**, meaning Chapter 12, section 2.

What you'll learn: how to use this chapter as a revision tool. Read a sheet, cover the right-hand column, and try to say the answer before you look. The glossary is also a good last check before an interview: if you can't define a term in one sentence, follow its link.

Every concrete value quoted on the sheets (outputs, orderings, return values, errors thrown) is asserted by `code/cheatsheet_checks.mjs`, which runs on Node 24 with jsdom for the DOM sheet:

```
OK   18.1 syntax & types
OK   18.2 operators & coercion
OK   18.3 control flow
OK   18.4 functions/closures/this
OK   18.5 objects/Math/Date/JSON
OK   18.6 classes & prototypes
OK   18.7 strings & regex
OK   18.8 arrays
OK   18.9 Map/Set/iterators
OK   18.10 modules
OK   18.11 errors
OK   18.12 async & event loop
OK   18.13 DOM & events
OK   18.14 browser APIs
OK   18.15 Node core
OK   18.16 testing
      Node v24.11.1 - not yet available: Math.sumPrecise (ES2026), Uint8Array.fromBase64 (ES2026), Map#getOrInsert
(ES2026), Iterator.concat (ES2026), Temporal (ES2027)
OK   18.18 versions (Node 24 availability)
all 17 cheat-sheet checks passed
```

What's inside

#	Sheet	Built from
18.1	Syntax, variables and types	Ch 1, 2
18.2	Operators and the coercion table	Ch 2
18.3	Control flow and loops	Ch 3
18.4	Functions, scope, closures and `this`	Ch 4, 9
18.5	Objects, Math, Date and JSON	Ch 5
18.6	Classes and prototypes	Ch 9
18.7	Strings and regular expressions	Ch 6
18.8	Arrays: mutating vs non-mutating	Ch 7
18.9	Map, Set, iterators and generators	Ch 8
18.10	Modules and tooling	Ch 10
18.11	Errors and debugging	Ch 11
18.12	Async and the event loop (with diagram)	Ch 12
18.13	DOM and events	Ch 13
18.14	Browser APIs and security	Ch 14
18.15	Node.js core modules	Ch 15
18.16	Testing and TypeScript	Ch 16
18.17	Which API for which job	all
18.18	ECMAScript versions ES2015 → ES2026	Ch 1 + TC39
18.19	Where to go next	—
18.20	Interview question index	Ch 17
18.21	Glossary (A–Z)	all

18.1 Syntax, variables and types

Running code (§1.5): the browser console, `<script defer src="app.js">` or `<script type="module">` in a page, `node file.js` / `node file.mjs`, or the Node REPL (`node` with no file).

Basic rules (§1.6): case-sensitive; identifiers use letters, digits, `_` and `$` and can't start with a digit; statements end with `;` (automatic semicolon insertion exists, but never break a line straight after `return`). Comments: `//`, `/* */`, `/** JSDoc */` (§1.7).

Strict mode (§1.8) is on automatically in modules and class bodies, and opt-in with `"use strict"` at the top of a script or function. It turns silent mistakes into errors:

Mistake	Sloppy mode	Strict mode
`x = 1` with no declaration	creates a global	`ReferenceError`
`this` in a plain call `f()`	`globalThis`	`undefined`
Writing a frozen / read-only property	ignored	`TypeError`
`function f(a, a)`, `010`, `with`	allowed	`SyntaxError`

Declaring variables (§2.1, §2.2):

	`const` (default choice)	`let` (when you reassign)	`var` (legacy)
Scope	block `{}`	block `{}`	function
Reassign	✗ `TypeError`	✓	✓
Redeclare in same scope	✗ `SyntaxError`	✗	✓ silently
Read before its line	`ReferenceError` (TDZ)	`ReferenceError` (TDZ)	`undefined`
Top level of a classic script	not on `globalThis`	not on `globalThis`	becomes `globalThis.x`

`const` stops *reassignment*, not *mutation*: `const o = { n: 1 }; o.n = 2` is fine.

The eight types (§2.3): seven primitives plus objects.

Value	`typeof`	Remember
`42`, `3.14`, `NaN`, `Infinity`	`"number"`	one 64-bit float type for all numbers
`10n`	`"bigint"`	whole numbers of any size; can't mix with `number`
`'hi'`	`"string"`	immutable UTF-16 text
`true`	`"boolean"`	
`undefined`	`"undefined"`	"not set yet" — what the language gives you
`null`	**`"object"`**	a historic bug; test with `x === null`
`Symbol('id')`	`"symbol"`	unique keys
`{}`, `[]`, `new Date()`	`"object"`	arrays too — use `Array.isArray`
`function () {}`, `class {}`	`"function"`	functions are callable objects
an undeclared name	`"undefined"`	`typeof` never throws, except in the TDZ

Primitives vs objects (§2.4): primitives are copied by value and compared by value; objects are copied as a **reference** and compared by identity, so `{} === {}` is `false` and two variables can point at one object.

18.2 Operators and the coercion table

Arithmetic (§2.5):

Expression	Result	Why
`7 / 2`	`3.5`	`/` is always floating point; integer division is `Math.trunc(a / b)`
`-7 % 3`	`-1`	`%` is a remainder that keeps the **left** operand's sign; true modulo is `((a % n) + n) % n` → `2`
`2 ** 10`	`1024`	exponent (ES2016)
`n++` / `++n` (n = 5)	`5` / `7`	postfix returns the old value, prefix the new one
`1 + '2'`	`"12"`	`+` concatenates if either side is a string
`'5' - 2`, `'3' * '4'`	`3`, `12`	other arithmetic converts to numbers
`true + 1`	`2`	`true` → `1`
`[] + []` / `[] + {}` / `[1,2] + [3]`	`""` / `"[object Object]"` / `"1,23"`	objects become strings first

Equality (§2.6). Use `===`. The only common `==` idiom is `x == null` (true for `null` **or** `undefined`).

Expression	==	Rule behind it
`0 == ''`	`true`	string → number (`''` → `0`)
`0 == '0'`	`true`	string → number
`'' == '0'`	`false`	both strings: compared as text — so `==` is **not transitive**
`false == '0'`	`true`	boolean → number first: `0 == '0'`
`'2' == true`	`false`	`true` → `1`, then `2 == 1`
`null == undefined`	`true`	they equal each other…
`null == 0`	`false`	…and nothing else
`null >= 0`	`true`	relational operators turn `null` into `0`
`NaN == NaN`	`false`	`NaN` equals nothing, not even itself
`[1] == 1`, `[] == ![]`	`true`, `true`	object → primitive (`[]` → `''` → `0`)
`1n == 1`	`true`	BigInt vs number compares the maths value
`'10' < '9'` / `'10' < 9`	`true` / `false`	two strings compare by character code; otherwise numerically
`Object.is(NaN, NaN)` / `Object.is(0, -0)`	`true` / `false`	`===` except for those two cases

Conversions (§2.12):

Value	Number(v)	String(v)	Boolean(v)
`''` / `' '`	`0` / `0`	`''` / `' '`	`false` / `true`
`'42'` / `'4px'` / `'0x1f'`	`42` / `NaN` / `31`	same text	`true`
`null`	`0`	`'null'`	`false`
`undefined`	`NaN`	`'undefined'`	`false`
`true` / `false`	`1` / `0`	`'true'` / `'false'`	—
`[]` / `[7]` / `[1,2]`	`0` / `7` / `NaN`	`''` / `'7'` / `'1,2'`	`true` (all objects)
`{}`	`NaN`	`'[object Object]'`	`true`

`parseInt('08.5kg', 10)` → `8` (reads a prefix, **always pass the radix**) but `Number('08.5kg')` → `NaN` (all or nothing).

Falsy (§2.11) — the complete list: `false`, `0`, `-0`, `0n`, `''`, `null`, `undefined`, `NaN`. Everything else is truthy, including `[]`, `{}`, `'0'`, `'false'` and `new Boolean(false)`.

Logical operators (§2.7):

Operator	Returns	Example
`a && b`	`a` if falsy, else `b`	`0 && 'b'` → `0`; `'a' && 'b'` → `'b'`
`a || b`	`a` if truthy, else `b`	`0 || 'x'` → `'x'`
`a ?? b`	`a` unless `null`/`undefined`	`0 ?? 'x'` → `0`, `'' ?? 'x'` → `''`
`a?.b`, `a?.[k]`, `a?.()`	`undefined` if `a` is `null`/`undefined`	`null?.deep.prop` → `undefined` (short-circuits the whole chain)
`a ||= b`, `a &&= b`, `a ??= b`	assign only when falsy / truthy / nullish	`cfg.port ??= 8080`

Mixing `??` with `||` or `&&` without parentheses is a `SyntaxError`.

Numbers (§2.10): `0.1 + 0.2` is `0.30000000000000004`; integers are exact only up to `Number.MAX_SAFE_INTEGER` = $2^{53} - 1$ (so `2 ** 53 === 2 ** 53 + 1` is `true`); `1 / 0` is `Infinity`; `Math.max()` is `-Infinity`. Use `Number.isNaN(x)` (the global `isNaN('abc')` converts first and says `true`), `toFixed(2)` for display (`(123.456).toFixed(2)` → `'123.46'`), integer cents for money, `BigInt` beyond 2^{53}. Numeric separators: `1_000_000`. Bitwise operators (§2.8) work on 32-bit integers: `5 & 3` = 1, `5 | 3` = 7, `5 ^ 3` = 6, `~5` = −6.

Precedence in one breath (§2.9): member access/calls → postfix → unary (`!`, `typeof`, `await`) → `**` → `* / %` → `+ -` → shifts → `< > in instanceof` → equality → bitwise → `&&` → `|| ??` → `? :` → assignment and `=>` → comma. When unsure, add parentheses.

18.3 Control flow and loops

Need	Use	Watch out
Choose between code paths	`if / else if / else` (§3.2)	first truthy branch wins — order specific → general
Choose between two **values**	`cond ? a : b` (§3.3)	don't nest deeply
One value vs many fixed options	`switch` (§3.4)	compares with `===`; forgetting `break` **falls through**; `switch (1) { case 1: … case 2: … break }` runs both cases
Pure value → value mapping	lookup object or `Map`	object keys are strings
Count / index loop	`for (let i = 0; i < n; i++)` (§3.6)	`<` vs `<=` off-by-one
Unknown number of repeats	`while (cond)` (§3.7)	update the condition or it never ends
Run at least once	`do { } while (cond)` (§3.8)	`do {…} while (false)` runs once
Values of an array, string, Map, Set	`for (const v of it)` (§3.10)	plain objects aren't iterable — use `Object.entries(obj)`
Keys of a plain object	`for (const k in obj)` (§3.9)	keys are **strings** (`for…in` over `['a','b']` gives `'0'`, `'1'`), includes inherited ones — avoid on arrays
Index and value	`for (const [i, v] of arr.entries())`	
Leave a loop / skip one pass	`break` / `continue` (§3.11)	`forEach` can't be stopped — use `for…of`, `some`, `find`
Leave an **outer** loop	`outer: for (…) { for (…) { break outer; } }`	labels work with `continue outer` too

Closures in loops (§4.7): `for (var i…)` shares one `i`, so three saved callbacks all return `3`; `for (let i…)` makes a fresh binding per iteration, giving `0, 1, 2`.

18.4 Functions, scope, closures and `this`

Forms (§4.1–§4.3):

```
function add(a, b) { return a + b; }        // declaration: hoisted with its body
const sub = function (a, b) { return a - b; }; // expression: follows const (TDZ)
const sq = (x) => x * x;                    // arrow, implicit return
const mk = (x, y) => ({ x, y });            // returning an object literal needs ( )
(() => { /* IIFE: runs immediately */ })();
```

	Regular `function`	**Arrow `=>`**
`this`	set by the call site	lexical — taken from where it's written; `call`/`bind` can't change it
`arguments`	yes	no — use `...args`
`new`	allowed	`TypeError` (no `prototype`)
Best as	object methods, constructors	callbacks, especially inside methods

Parameters (§4.4): with `f = (a, b = 2, ...rest) => [a, b, rest]`, `f(1)` → `[1, 2, []]`; `f(1, undefined, 3, 4)` → `[1, 2, [3, 4]]`; `f(1, null)` → `[1, null, []]` — **defaults replace only `undefined`**. `fn.length` counts parameters before the first default/rest (`(a, b = 1, c) => 0` has length `1`). Use an options object for named parameters: `function f({ a = 1, b = 2 } = {})`.

Scope and hoisting (§4.5, §4.6):

Declaration	**Before its line**
`function f() {}`	callable
`var x`	`undefined`
`let` / `const` / `class`	`ReferenceError` (TDZ)
`var f = function…`	`undefined` → calling it is a `TypeError`

Lookups walk the **lexical scope chain** outward from where the code is *written*; an inner name shadows an outer one.

Closures (§4.7): a function keeps live access to the variables of the scope it was created in.

```
function counter() { let c = 0; return () => ++c; }
const c1 = counter(); c1(); c1();
const c2 = counter();
c1(); // 3   - c1 has its own c
c2(); // 1   - c2 has a separate c
```

Uses: private state, factories, `once`, `memoize`, event handlers, debounce/throttle (§17.3).

Higher-order functions (§4.8): take or return functions — `map/filter/reduce`, `pipe` (left → right, `reduce`), `compose` (right → left, `reduceRight`), currying `f(a)(b)(c)`, partial application. **Recursion** (§4.10): base case + smaller case; too deep → `RangeError: Maximum call stack size exceeded`. **Pure functions** (§4.11): same input → same output, no side effects.

`this` — decided at call time (§9.1), highest priority first:

Call site	**`this`**
`new Fn()`	the new object
`fn.call(obj, a, b)`, `fn.apply(obj, [a, b])`, `fn.bind(obj)()`	`obj`
`obj.fn()`	`obj` (the thing before the dot)
plain `fn()`	`undefined` in strict code (`globalThis` in sloppy scripts)
arrow function	whatever `this` was where the arrow was written

Checked facts: `user.hi()` → `'Ada'`, but `const lost = user.hi; lost()` → `this` is `undefined` (the method "loses" its object when passed around — fix with an arrow wrapper or `bind`). Binding twice keeps the **first** binding. `call` runs now with arguments listed, `apply` runs now with an array, `bind` returns a new function (§9.2).

`['1', '7', '11'].map(parseInt)` → `[1, NaN, 3]`, because `map` passes `(value, index)` and `parseInt` treats the index as the radix.

18.5 Objects, Math, Date and JSON

Objects (§5.1–§5.5):

```
const key = 'dyn';
const o = { b: 1, 2: 'two', a: 2, 1: 'one', [key]: 3 };   // computed key
Object.keys(o);            // ['1', '2', 'b', 'a', 'dyn'] - integer-like keys first, then insertion order
const { a, b: renamed, missing = 'def' } = o;            // 2, 1, 'def'
const merged = { ...{ x: 1, y: 1 }, ...{ y: 2 } };       // { x: 1, y: 2 } - later wins
```

Task	Code	Note
Read with a fixed / dynamic key	`o.name` / `o[expr]`	brackets for spaces, dashes, variables
Own key?	`Object.hasOwn(o, k)`	`'toString' in {}` is `true` (inherited); `Object.hasOwn({}, 'toString')` is `false`
Remove	`delete o.k`	
List	`Object.keys / values / entries`, rebuild with `Object.fromEntries`	own, enumerable, string keys
Shallow copy / merge	`{ ...a, ...b }`, `Object.assign({}, a, b)`	nested objects are **shared**
Deep copy	`structuredClone(o)`	handles Date, Map, Set, cycles; throws on functions
Lock	`Object.preventExtensions` < `Object.seal` < `Object.freeze`	all **shallow**: a frozen object's nested objects stay writable
Prototype	`Object.create(proto)`, `Object.create(null)` for a bare dictionary	§18.6
Getter / setter	`get total() {…}`, `set total(v) {…}` (§5.7)	keep them cheap

Descriptors (§5.6): `writable`, `enumerable`, `configurable` (+ `get`/`set`). Normal assignment makes them all `true`; `Object.defineProperty` defaults them to `false`.

Math (§5.10):

Call	Result
`Math.round(2.5)` / `Math.round(-2.5)`	`3` / `-2` (halves go toward +∞)
`Math.floor(-1.5)` / `Math.trunc(-1.5)` / `Math.ceil(1.1)`	`-2` / `-1` / `2`
`Math.max(...[3, 9, 1])`	`9` (spread — `max` takes separate arguments)
Random int in [min, max]	`Math.floor(Math.random() * (max - min + 1)) + min`
Secure random	`crypto.getRandomValues`, `crypto.randomUUID()` — never `Math.random`

Date (§5.11): a `Date` is milliseconds since 1970-01-01 UTC. Months are **0-based** (`new Date(2024, 0, 31)` is 31 January); `getDay()` is the weekday (0 = Sunday), `getDate()` the day of month; setters mutate and roll

over (setting month 1 on 31 January gives a date in March); a date-only ISO string like `'2024-03-10'` is parsed as **UTC midnight**; `new Date(0).toISOString()` → `'1970-01-01T00:00:00.000Z'`. Show dates with `Intl.DateTimeFormat` / `toLocaleDateString`. **Temporal** reached Stage 4 in March 2026 (it goes into ES2027); it is not in Node 24, so feature-detect it.

JSON (§5.12):

Input to `JSON.stringify`	Output
`{ u: undefined, f() {}, n: NaN, d: new Date(0), m: new Map([[1, 2]]) }`	`{"n":null,"d":"1970-01-01T00:00:00.000Z","m":{}}`
`[undefined, () => 1]`	`[null,null]`
`undefined` (top level)	`undefined` (not a string)
`{ big: 1n }` or an object with a cycle	throws `TypeError`
`JSON.stringify(v, null, 2)`	pretty-printed with 2-space indent

`JSON.parse` is strict: double-quoted keys and strings only, no trailing commas, no comments (`JSON.parse("{'a':1}")` throws `SyntaxError`). A **reviver** `(key, value) => …` can turn ISO strings back into `Date`s.

18.6 Classes and prototypes

```
class Animal {
  #secret = 's';                    // private field (ES2022)
  static count = 0;                 // static field
  static { Animal.ready = true; }   // static block, runs once
  constructor(name) { this.name = name; Animal.count++; }
  speak() { return `${this.name} makes a sound`; }   // on Animal.prototype
  get upper() { return this.name.toUpperCase(); }     // accessor
  static isAnimal(x) { return #secret in x; }        // brand check
}
class Dog extends Animal {
  speak() { return `${super.speak()} - woof`; }
}
new Dog('Rex').speak();  // 'Rex makes a sound - woof'
```

Fact	Checked result
`Object.keys(new Dog('Rex'))`	`['name']` — methods are non-enumerable, `#private` is invisible
`Object.getPrototypeOf(Dog.prototype) === Animal.prototype`	`true` — `extends` links the chains
`Animal('x')` without `new`	`TypeError`
Using `this` before `super()` in a subclass constructor	`ReferenceError`
`typeof class {}`	`'function'` — a class is a special function
`Object.getPrototypeOf(Object.prototype)`	`null` — the end of every chain
`'toString' in Object.create(null)`	`false` — no inherited keys

How lookup works (§9.3): reading a missing property walks the `[[Prototype]]` chain until it finds it or reaches `null`; writing always creates an **own** property (shadowing). `Fn.prototype` is the object that *instances* of `new Fn()` get as their prototype, not `Fn`'s own prototype. `__proto__` is a legacy accessor — use `Object.getPrototypeOf` / `Object.create`.

`new Fn(args)` (§9.4): 1. create an object linked to `Fn.prototype`; 2. run `Fn` with `this` = that object; 3. return it, unless `Fn` returns a different **object** (a returned primitive is ignored).

Type checks (§9.7): `typeof` for primitives, `Array.isArray` for arrays, `Error.isError` (ES2026) for errors, `x instanceof C` for "made by C" (walks the chain; breaks across realms), `#brand in x` inside a class, duck typing (`typeof x?.then === 'function'`) for "behaves like", `Object.prototype.toString.call(x)` → `'[object Date]'` for built-in tags.

Design (§9.8, §9.9): inheritance (is-a) only for stable taxonomies such as `HttpError extends Error`; composition (has-a) for most application code; mixins to share a capability between unrelated classes. TypeScript's `private` is compile-time only; JavaScript's `#private` is enforced at runtime.

18.7 Strings and regular expressions

Strings (§6.2–§6.6) are immutable UTF-16 primitives. Methods always return a **new** string.

Job	Method	Example → result
Interpolate / multi-line	template literal	`` `${1 + 1}` `` → `'2'`
Last char, out-of-range	`at(-1)`, `s[i]`, `charAt(i)`	`'Hello'.at(-1)` → `'o'`; `'Hello'[10]` → `undefined`; `charAt(10)` → `''`
Substring	`slice(start, end?)` (negatives OK)	`'Hello'.slice(-3)` → `'llo'`, `slice(1, 3)` → `'el'`
Search	`includes`, `startsWith`, `endsWith`, `indexOf` (−1 if absent)	
Replace first / all	`replace` / `replaceAll`	`'a-b-c'.replace('-', '+')` → `'a+b-c'`; `replaceAll` → `'a+b+c'`
Pad, repeat	`padStart`, `padEnd`, `repeat`	`'5'.padStart(3, '0')` → `'005'`
Trim	`trim`, `trimStart`, `trimEnd`	
Split / join	`split(sep)`, `arr.join(sep)`	`'a,b,,c'.split(',')` → `['a', 'b', '', 'c']`
Case	`toUpperCase`, `toLowerCase`, `toLocale…`	
Human ordering	`a.localeCompare(b)`, `Intl.Collator`	default `sort` gives `'Cab'` for `['b','a','C']`; with `localeCompare` → `'abC'`

Unicode (§6.5): `length` counts 16-bit code units — `'😀'.length` is `2`, `[...'😀'].length` is `1` (code points), and a family emoji `'👨‍👩‍👧'` is one grapheme to `Intl.Segmenter`. Normalize with `normalize('NFC')` before comparing accented text.

Regex syntax (§6.7):

Piece	Meaning
`.` `\d` `\w` `\s` (`\D` `\W` `\S`)	any char (not newline) / digit / word char / whitespace (and complements)
`[abc]` `[^abc]` `[a-z]`	class / negated class / range
`*` `+` `?` `{n,m}`	0+ / 1+ / 0–1 / n to m — **greedy**; add `?` for lazy
`^` `$` `\b`	start / end / word boundary
`( )` `(?<name> )` `(?: )`	capture / named capture / group only
`(?= )` `(?! )` `(?<= )` `(?<! )`	lookahead / negative lookahead / lookbehind / negative lookbehind
`\1`, `\k<name>`	backreference
`\p{L}` (with `u` or `v`)	any letter in any script
Flags	`g` all, `i` ignore case, `m` multiline anchors, `s` dot matches newline, `u` Unicode, `v` Unicode sets, `y` sticky, `d` indices

Regex methods (§6.8):

Call	Result
`/^\d{3}-\d{4}$/.test('555-1234')`	`true`
`'x'.match(/y/)`	`null` (not `[]`)
`'a1b22'.match(/\d+/g)`	`['1', '22']`
`'2024-03-10'.match(/(?<y>\d{4})-…/).groups.y`	`'2024'`
`'2024-03-10'.replace(/(\d+)-(\d+)-(\d+)/, '$3/$2/$1')`	`'10/03/2024'`
`[...'a1b2'.matchAll(/\d/g)].map(m => m.index)`	`[1, 3]`; `matchAll` without `g` throws `TypeError`
`'<b>x</b>'.replace(/<.+>/, '')` vs `/<.+?>/`	`''` (greedy) vs `'x</b>'` (lazy)
`/a/g.test('aa')` then `lastIndex`	`1` — a `g` regex remembers its position between calls
`new RegExp(RegExp.escape(userText))`	matches the text literally (ES2025)

Prefer plain string methods for fixed text and real parsers (`URL`, `JSON.parse`, `DOMParser`) for structured formats (§6.9).

18.8 Arrays: mutating vs non-mutating

✎ = changes the original array (§7.4).

Mutates ✎	Returns	Copying twin / alternative (leaves the original alone)
`push(x)` / `pop()`	new length / removed item	`[...arr, x]` / `arr.slice(0, -1)`
`unshift(x)` / `shift()`	new length / removed item	`[x, ...arr]` / `arr.slice(1)`
`splice(i, n, ...items)`	array of **removed** items	`toSpliced(i, n, ...items)` (ES2023)
`sort(cmp)`	the **same** array	`toSorted(cmp)` (ES2023)
`reverse()`	the same array	`toReversed()` (ES2023)
`arr[i] = x`	—	`with(i, x)` (ES2023; negative `i` OK, out of range → `RangeError`)
`fill(v, s, e)`, `copyWithin(t, s, e)`	the same array	`Array.from` / `map`
`arr.length = n`	truncates	`slice(0, n)`

Never mutated: `slice`, `concat`, `map`, `filter`, `reduce`, `flat`, `flatMap`, `find*`, `some`, `every`, `includes`, `indexOf`, `at`, `join`, `keys/values/entries`, spread.

Checked behaviours: `const s = arr.sort(); s === arr` → `true`; `[1].push(2)` → `2` (not the array); `[10, 1, 2].sort()` → `[1, 10, 2]` (default sort compares **strings**) but `.sort((a, b) => a - b)` → `[1, 2, 10]`; `[3, 1, 2].toSorted()` → `[1, 2, 3]` with the original unchanged.

Iteration methods (§7.5):

Want	Method	Checked example
Transform each	`map`	
Keep some	`filter`	
Fold to one value	`reduce(fn, init)`	`[1,2,3].reduce((s, x) => s + x, 0)` → `6`; `[].reduce(fn)` with no init → `TypeError`
First match / its index	`find` / `findIndex`	`find(x => x > 1)` → `2`; `findIndex(x => x > 5)` → `-1`
Last match	`findLast` / `findLastIndex` (ES2023)	`findLast(x => x > 1)` → `3`
Any / all	`some` / `every`	`[].every(…)` → `true` (vacuous truth)
Contains	`includes`	`[NaN].includes(NaN)` → `true`, `[NaN].indexOf(NaN)` → `-1`
Flatten	`flat(depth = 1)`, `flatMap`	`[1,[2,[3,[4]]]].flat()` → `[1,2,[3,[4]]]`; `flat(Infinity)` → `[1,2,3,4]`
Side effects only	`forEach` / `for…of`	`forEach` can't `break` or `await`

Sorting (§7.6): numbers `(a, b) => a - b`; text `a.localeCompare(b)`; several keys chained with `||`: `(p, q) => p.age - q.age || p.name.localeCompare(q.name)`. `sort` is stable (guaranteed since ES2019).

Creating and destructuring (§7.2, §7.7): `Array.from({ length: 3 }, (_, i) => i * 2)` → `[0, 2, 4]`; `Array.of(3)` → `[3]` but `new Array(3)` has length 3 and holes; `Array(2).fill([])` puts the **same** inner array in both slots. `[x, y] = [y, x]` swaps; `const [first, , third = 'd', ...others] = [1, 2, undefined, 4, 5]` → `1`, `'d'`, `[4, 5]`. `join` turns `null`/`undefined` into empty strings.

Immutable updates (React/Redux style): add `[...arr, x]`; insert `arr.toSpliced(i, 0, x)`; remove by id `arr.filter(t => t.id !== id)`; replace `arr.with(i, x)`; update one `arr.map(t => t.id === id ? { ...t, done: true } : t)`.

Which collection (§7.8): array for ordered lists, object for fixed-shape records, `Map` for dynamic keyed lookup, `Set` for uniqueness. **Typed arrays** (§7.9) hold fixed-size binary numbers over an

`ArrayBuffer`; values wrap (`new Uint8Array([256, 257])` → `[0, 1]`).

18.9 Map, Set, iterators and generators

`Map` (§8.1) — keys of any type, insertion order, `size`:

```
const m = new Map([[1, 'num'], ['1', 'str']]);   // 1 and '1' are different keys
m.set({}, 'obj').set(NaN, 'nan');                 // set() returns the map, so it chains
m.get(NaN);           // 'nan' - keys compare with SameValueZero
JSON.stringify(m);    // '{}'  - convert first: Object.fromEntries(m) or [...m]
for (const [k, v] of m) { /* insertion order */ }
```

`Set` (§8.2) — unique values, O(1) `has`: `[...new Set([3, 1, 3, 2, 1])]` → `[3, 1, 2]` (dedupe, first-seen order). ES2025 set algebra, with A = {1,2,3} and B = {2,3,4}:

Method	Result
`A.union(B)`	{1, 2, 3, 4}
`A.intersection(B)`	{2, 3}
`A.difference(B)`	{1}
`A.symmetricDifference(B)`	{1, 4}
`A.isSubsetOf(B)`, `isSupersetOf`, `isDisjointFrom`	booleans

Use	When
`Map`	dynamic dictionaries, counting, caches (an LRU cache is one `Map`), non-string keys
plain object	fixed-shape records, config, JSON
`Set`	membership tests, dedupe, visited sets in BFS/DFS
`WeakMap` / `WeakSet` (§8.3)	data attached to objects you don't own; entries vanish with the key object; no `size`, no iteration
`WeakRef` / `FinalizationRegistry`	advanced caches; GC timing is never guaranteed

Iteration protocol (§8.4): an **iterable** has a `[Symbol.iterator]()` method returning an **iterator**, an object whose `next()` returns `{ value, done }`. Arrays, strings, Maps, Sets, typed arrays, NodeLists and generators are iterable; plain objects are not (`[...{ a: 1 }]` → `TypeError`). Consumers: `for…of`, spread, array destructuring, `Array.from`, `new Map/Set`, `Promise.all`, `yield*`.

Generators (§8.5):

```
function* nat() { let n = 1; while (true) yield n++; }   // infinite, lazy
nat().next();                          // { value: 1, done: false }
const iterable = { *[Symbol.iterator]() { yield 1; yield 2; } };
[...iterable];                         // [1, 2]
function* two() { yield 'a'; return 'r'; }
[...two()];                            // ['a'] - the return value is not iterated
```

Iterator helpers (ES2025, §8.6) are lazy: `nat().filter(n => n % 2).map(n => n * n).take(3).toArray()` → `[1, 9, 25]`. Arrays need `.values()` first; `Iterator.from(x)` wraps custom iterators.

Grouping (ES2024, §8.7): `Object.groupBy([1,2,3,4], n => n % 2 ? 'odd' : 'even')` → `{ odd: [1, 3], even: [2, 4] }` (a null-prototype object); `Map.groupBy` when keys aren't strings.

Async iteration (§12.12): `async function*` + `for await (const page of pages())`; `await Array.fromAsync(pages())` (ES2026) collects them.

18.10 Modules and tooling

```
// math.mjs
export const PI = 3.14;                 // named export
export function area(r) { return PI * r * r; }
export default class Circle {}          // one default per module
// app.mjs
import Circle, { area, PI as pi } from './math.mjs';
import * as math from './math.mjs';     // namespace object: math.area, math.default
export { area } from './math.mjs';      // re-export (barrel file)
const { heavy } = await import('./heavy.mjs');  // dynamic import -> promise, code splitting
import data from './data.json' with { type: 'json' };  // JSON module (ES2025)
```

ES module fact (§10.3)	Consequence
Loads in 3 phases: parse → link → evaluate	imports are hoisted; a missing export fails before any code runs
Imports are **live, read-only bindings**	you see the exporter's updates; you can't assign to them
Each module is evaluated **once** (singleton)	module-level state is shared by every importer
Always strict; top-level `this` is `undefined`	checked by the sheet script
Top-level `await` allowed (§10.5)	importers wait for it
`import.meta.url`, `import.meta.dirname` / `filename` (Node) (§10.6)	replace CommonJS `__dirname` / `__filename`

CommonJS vs ESM in Node (§10.8): `.mjs` = ESM, `.cjs` = CommonJS, `.js` follows `"type"` in the nearest `package.json`. CommonJS uses `require()` / `module.exports`, loads synchronously, has `__dirname`. ESM can import CommonJS (default import = `module.exports`); since Node 22.12 / 20.19, CommonJS can `require()` an ES module unless it uses top-level `await` (`ERR_REQUIRE_ASYNC_MODULE`). **Browsers** (§10.7): `<script type="module">` is deferred, strict, needs CORS and a server (not `file://`); import maps resolve bare names.

npm (§10.9, §10.10):

Item	Meaning
`dependencies` / `devDependencies` / `peerDependencies`	needed at runtime / only to build and test / "use the host's copy"
`^1.2.3`	≥1.2.3 <2.0.0 (npm's default)
`~1.2.3`	≥1.2.3 <1.3.0
`^0.2.3`	≥0.2.3 <0.3.0 (caret is stricter below 1.0)
`package-lock.json`	commit it; CI runs `npm ci` (exact, clean install)
`npx tool`, `npm run x`, `node --run x`	run a package binary / a script

Tools (§10.11–§10.13): bundlers (Vite, esbuild, webpack, Rollup/Rolldown) do resolution, tree shaking, code splitting, minification, source maps and HMR. **Transpile syntax, polyfill APIs**: a transpiler rewrites new syntax for old engines; a polyfill adds missing built-ins. ESLint finds bugs (flat config `eslint.config.js`); Prettier formats.

18.11 Errors and debugging

Error type (§11.1)	Checked trigger
`TypeError`	`null.length`, calling a non-function, `1n + 1`, assigning to a `const`
`ReferenceError`	undeclared name, TDZ access
`SyntaxError`	`JSON.parse('{bad}')`, bad code in `eval` / `new Function`
`RangeError`	`new Array(-1)`, `(1).toFixed(101)`, stack overflow
`URIError`	`decodeURIComponent('%')`
`AggregateError`	`Promise.any` when all reject — `.errors` holds them

```
class ValidationError extends Error {                // §11.3
  constructor(msg, opts) { super(msg, opts); this.name = 'ValidationError'; this.code = 'E_VALID'; }
}
try {
  save();
} catch (err) {
  if (!(err instanceof ValidationError)) throw err;   // catch narrowly, rethrow the rest
  throw new Error('save failed', { cause: err });     // §11.4 error chaining
} finally {
  release();                                          // always runs; never return from it
}
```

Rules: always throw `Error` objects (they carry a `stack`); `String(err)` → `'ValidationError: save failed'`; a `return` in `try` still runs `finally` first; `catch {}` without a binding is legal (ES2019) but never swallow errors silently. **`try/catch` is synchronous**: it catches a rejected `await` inside an `async` function, but not an error thrown later in a timer callback (§11.5). Global last-resort handlers: browser `error` / `unhandledrejection` events; Node `uncaughtException` / `unhandledRejection` — log, then exit. `using` + `Symbol.dispose` gives automatic cleanup (§11.6; finished proposal, edition ES2027).

Debugging (§11.7–§11.9): `console.table`, `dir`, `group`, `count`, `time` / `timeEnd`, `trace`, `assert` (logs, doesn't throw). Stack traces list the newest frame first; source maps map bundled code back to your files. Breakpoints (conditional, logpoints, pause on exceptions), step over / into / out, Scope and Watch panes; `debugger;` statement; Node: `node --inspect` then `chrome://inspect`. Method: reproduce → read the message → hypothesize → inspect → fix the root cause → add a test. Common messages decoded: §11.10.

18.12 Async and the event loop

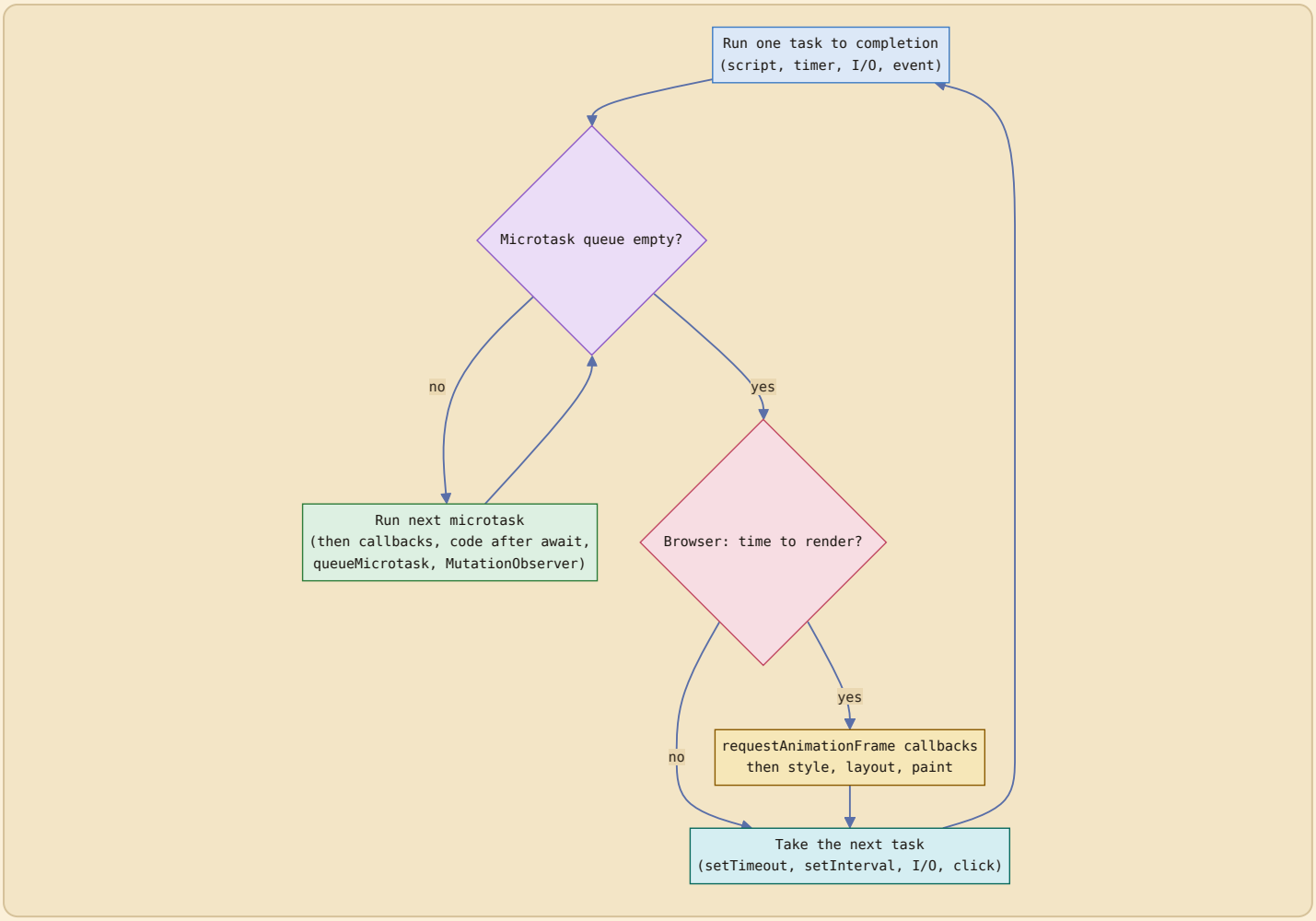


*Figure — one turn of the event loop: one task, then **all** microtasks, then maybe a render (§12.2).*

The ordering rule (§12.3): synchronous code first (including promise executors and an async function's body up to its first `await`), then microtasks in FIFO order, then the next task.

```
log('A');
setTimeout(() => log('B timeout'), 0);
Promise.resolve().then(() => log('C then')).then(() => log('D then2'));
queueMicrotask(() => log('M micro'));
new Promise((res) => { log('P executor'); res(); });
(async () => { log('F before await'); await null; log('F after await'); })();
log('E');
// A, P executor, F before await, E, C then, M micro, F after await, D then2, B timeout
```

Node extras (§12.4): phases timers → pending → poll (I/O) → check (`setImmediate`) → close. Inside a callback, `process.nextTick` runs before promise microtasks, and both run before `setImmediate` — checked order: `nextTick, promise, immediate`.

Promises (§12.7, §12.8): pending → fulfilled or rejected, settle **once**; `then` returns a new promise, so always `return` inside it and end the chain with `catch`.

Combinator	Fulfils when	Rejects when	Checked
`Promise.all`	all fulfil — values in **input order**	first rejection	`[ok(1, 5ms), ok(2, 1ms)]` → `[1, 2]`
`Promise.allSettled`	all settle	never	statuses `['fulfilled', 'rejected']`
`Promise.race`	first to **settle** fulfils	first to settle rejects	fastest wins
`Promise.any`	first to **fulfil**	all reject → `AggregateError`	skips a rejection
`Promise.withResolvers()` (ES2024)	`{ promise, resolve, reject }`		
`Promise.try(fn)` (ES2025)	a sync throw becomes a rejection		

None of the combinators cancel the losing operations.

async/await (§12.9): an `async` function always returns a promise; `await` unwraps it and **throws** on rejection (so `try/catch` works). Two independent awaits one after another take the sum of their times (checked: ~80 ms); `await Promise.all([...])` takes the longest (~40 ms). `arr.forEach(async …)` doesn't wait — use `for…of` (one at a time) or `Promise.all(arr.map(async …))` (all at once). Use `return await` inside `try` so the `catch` sees the rejection.

Timers (§12.5): delays are minimums; nested timers clamp to 4 ms in browsers; delays above $2^{31} - 1$ ms overflow; prefer recursive `setTimeout` to `setInterval` for polling. **Cancellation** (§12.11): `const ac = new AbortController(); fetch(url, { signal: ac.signal }); ac.abort()`; `AbortSignal.timeout(ms)`; `AbortSignal.any([...])`. **Patterns** (§12.13): retry with exponential backoff + jitter, concurrency limits, last-request-wins, debounce/throttle.

18.13 DOM and events

Task	API (§13.1–§13.4)
Find one / many	`document.getElementById('id')`, `el.querySelector(css)` / `querySelectorAll(css)` (static `NodeList`)
Live collections	`getElementsByTagName` / `ByClassName` — update automatically (checked: after appending an `<li>`, the live list has 2 items, the static one still 1)
Relatives	`el.closest(css)` (self or ancestor), `el.matches(css)`, `parent.contains(node)`, `children`, `nextElementSibling`
Create and insert	`document.createElement('li')`, `append`, `prepend`, `before`, `after`, `replaceWith`, `remove`, `replaceChildren`; `DocumentFragment` / `<template>` for bulk
Text vs HTML	`el.textContent = userText` (never parsed — `'<b>safe</b>'` shows as text); `innerHTML` only for trusted markup (XSS risk)
Classes, data, style	`classList.add/remove/toggle/contains`, `el.dataset.id` (from `data-id`, always a string), `el.style.x` (inline), `getComputedStyle(el)` (final)
Attribute vs property	`input.getAttribute('value')` = initial HTML value; `input.value` = current value

Events (§13.5–§13.8):

```
list.addEventListener('click', (e) => {          // one listener for all items (delegation)
  const item = e.target.closest('li');           // target = where it started
  if (!item || !list.contains(item)) return;
  console.log(item.dataset.id, e.currentTarget); // currentTarget = list (the listener's element)
}, { signal: controller.signal });                // options: once, passive, capture, signal
```

Fact	Checked
Propagation: capture (window → target), target, bubble (target → window)	order: `body capture`, `delegated 7`, `body bubble target=BUTTON current=BODY`
`new Event('ping')` doesn't bubble	parent heard 0; with `{ bubbles: true }` → 1
`{ once: true }`	handler ran once for two clicks
`e.preventDefault()`	cancels the default action (link navigation, form submit); `e.defaultPrevented` → `true`
`stopPropagation()`	stops the journey — use rarely
`dispatchEvent`	runs listeners **synchronously**

Forms (§13.9): listen for `submit`, call `preventDefault()`, read `new FormData(form)` (`getAll('t')` → `['a', 'b']` for repeated names), check `input.validity` / `setCustomValidity`, use `form.requestSubmit()`; always validate again on the server. **Watch changes**: `MutationObserver` (§13.10). **Performance** (§13.11): batch all layout reads, then all writes (interleaving them causes **layout thrashing**); animate `transform`/`opacity`; use `requestAnimationFrame`. **Script timing** (§13.12): `defer` and modules run after parsing, in order, before `DOMContentLoaded`; `async` runs whenever downloaded; `load` waits for images and styles.

18.14 Browser APIs and security

fetch (§14.1):

```
async function getJSON(url, { timeout = 5000 } = {}) {
  const res = await fetch(url, { signal: AbortSignal.timeout(timeout),
                                 headers: { Accept: 'application/json' } });
  if (!res.ok) throw new Error(`HTTP ${res.status}`);   // fetch resolves on 404/500!
  return res.json();                                    // the body can be read only once
}
await fetch('/api/items', { method: 'POST',
  headers: { 'Content-Type': 'application/json' }, body: JSON.stringify(item) });
```

`fetch` rejects only on network failure or abort; a 404 `Response` has `ok === false`. Reading a body twice throws `TypeError`.

URL (§14.6): `new URL('https://shop.example.com:8443/search?q=red shoes')` → `.origin` = `'https://shop.example.com:8443'`, `.searchParams.get('q')` = `'red shoes'`; after `set('page','3')` and `append('tag','a&b')` the search string is `'?q=red+shoes&page=3&tag=a%26b'` (encoding done for you). `new URL('../img/a.png', 'https://x.dev/app/page/')` resolves to `https://x.dev/app/img/a.png`; `new URL('not a url')` throws `TypeError` (a cheap validator). `encodeURIComponent('a b&c')` → `'a%20b%26c'`.

Storage (§14.3, §14.4):

	Cookie	localStorage	sessionStorage	IndexedDB
Size	~4 KB	~5 MB	~5 MB	large
API	strings, sync	strings, sync (`setItem('n', 42)` stores `'42'`; missing → `null`)	same, per tab	async, structured data
Sent to server	automatically	no	no	no
Use for	session ID (`HttpOnly; Secure; SameSite`)	preferences	wizard state	offline data, big caches

Never put tokens or secrets in `localStorage` — any script on the page can read it.

Other APIs: `requestAnimationFrame` for visual updates, once per frame, paused in background tabs (§14.5); History API `pushState` / `replaceState` + `popstate` for SPA routing (§14.7); Web Workers for CPU work off the main thread, talking via `postMessage` + structured clone (§14.8); `IntersectionObserver` (visibility, lazy loading) and `ResizeObserver` (element size) (§14.9). `structuredClone` copies Dates and Maps but throws `DataCloneError` on functions.

Security (§14.2, §14.10–§14.12):

Threat / rule	Remember
Origin	scheme + host + port; the browser lets a page *send* anywhere but *read* only same-origin responses
CORS	the **server** opts in with `Access-Control-Allow-Origin`; non-simple requests get an `OPTIONS` preflight; `*` can't be combined with credentials
XSS	treat data as text (`textContent`), escape by context (`<` → `<`), sanitize HTML with DOMPurify, allow-list URL schemes, CSP, `HttpOnly` cookies
CSRF	`SameSite` cookies + CSRF token or `Origin` check; GET must never change state; CORS is not a CSRF defence
CSP	`Content-Security-Policy: default-src 'self'; script-src 'self'; object-src 'none'; frame-ancestors 'none'` — blocks inline scripts and clickjacking
Other headers	HSTS, `X-Content-Type-Options: nosniff`, `Referrer-Policy`, `Permissions-Policy`
Supply chain	lockfiles, `npm audit`, few dependencies, Subresource Integrity for CDN scripts

---

18.15 Node.js core modules

Node = V8 + **libuv** (event loop, async I/O, a 4-thread pool for fs, DNS lookup, crypto, zlib) + core modules (§15.1, §15.2). Import built-ins with the `node:` prefix. Never block the loop; move CPU-heavy work to `worker_threads`.

Module	Everyday calls	Checked / remember
`node:fs/promises` (§15.4)	`readFile(p, 'utf8')`, `writeFile`, `appendFile`, `mkdir(p, { recursive: true })`, `readdir`, `stat`, `rm(p, { recursive: true, force: true })`	branch on `err.code` (`'ENOENT'`); don't check-then-act
`node:path` (§15.5)	`join`, `resolve`, `basename`, `extname`, `dirname`, `parse`, `relative`	`join('/srv','app','..','data')` → `/srv/data`; `extname('price.test.mjs')` → `.mjs`
`node:url`	`fileURLToPath(import.meta.url)`, `pathToFileURL`	or `import.meta.dirname`
`process` (§15.6)	`argv`, `env`, `exitCode`, `cwd()`, `on('SIGTERM')`	`process.env.X = 5` reads back `'5'` — env values are strings; `--env-file=.env`
`node:util`	`parseArgs`, `promisify`, `inspect`	`parseArgs` on `['--port','8080','file.txt']` → `port: '8080'`, positionals `['file.txt']`
`Buffer` (§15.7)	`Buffer.from(str)`, `.toString('hex' / 'base64')`	`Buffer.from('é').length` → `2` bytes, `'é'.length` → `1`
`node:events` (§15.8)	`on`, `once`, `off`, `emit`; `events.once(em, name)` → promise	`emit` is **synchronous**; an unhandled `'error'` event throws
`node:stream` (§15.9)	Readable, Writable, Duplex, Transform; `pipeline` from `node:stream/promises`	`pipeline` handles **backpressure** and errors
`node:http` (§15.10)	`createServer((req, res) => { … res.end() })`, `listen(0)` in tests	always end the response; limit body size
`node:child_process` (§15.11)	`execFile(cmd, args)`, `spawn`	pass argument arrays — no shell injection
`node:test`, `node:assert/strict`	§18.16	

Flags (§15.12): `--watch`, `--env-file`, `--run`, `--test`, `--inspect`, `--permission`; `node app.ts` strips types (no type check). **Alternatives**: Deno (secure by default, V8) and Bun (JavaScriptCore, fast tooling); Node stays the default choice.

18.16 Testing and TypeScript

```
// price.test.mjs - run with: node --test
import { test, describe, beforeEach } from 'node:test';
import assert from 'node:assert/strict';
import { applyDiscount } from './price.mjs';

describe('applyDiscount', () => {
  test('takes 10 percent off', () => assert.equal(applyDiscount(100, 10), 90));
  test('rejects negative percentages', () => assert.throws(() => applyDiscount(100, -1), RangeError));
  test('async loader rejects', async () => { await assert.rejects(load('bad'), /not found/); });
  test('calls the logger once', (t) => {
    const log = t.mock.fn();                    // spy, auto-restored
    applyDiscount(100, 10, { log });
    assert.equal(log.mock.callCount(), 1);
  });
});
```

Assertion (§16.2)	Passes when	Checked
`equal(a, b)`	`Object.is(a, b)` — same primitive or same object	`equal({a:1}, {a:1})` fails
`deepEqual(a, b)`	same structure, type-strict	`{a:1}` vs `{a:'1'}` fails
`throws(fn, Class | /regex/)`	`fn()` throws a match	
`await rejects(promise, …)`	the promise rejects — **must be awaited**	
`match(str, /re/)`	regex matches	

`node:test` (§16.3–§16.6): `test`/`it`, `describe`, hooks `before`/`beforeEach`/`afterEach`/`after`, `skip`/`todo`/`only`; `node --test` (finds `*.test.*`), `--watch`, `--test-name-pattern`; `t.mock.fn` / `t.mock.method` spies; fake timers `t.mock.timers.enable()` + `tick(ms)`; snapshots; `--experimental-test-coverage`. Test pyramid: many unit, some integration, few end-to-end (Playwright). Vitest/Jest use `expect(x).toBe(y)` (§16.7).

TypeScript (§16.8–§16.13):

```
type Shape = { kind: 'circle'; r: number } | { kind: 'rect'; w: number; h: number }; // discriminated union
function area(s: Shape): number {
  switch (s.kind) {                              // narrowing by discriminant
    case 'circle': return Math.PI * s.r ** 2;
    case 'rect': return s.w * s.h;
    default: { const never: never = s; return never; }   // exhaustive check
  }
}
function first<T>(xs: readonly T[]): T | undefined { return xs[0]; }   // generic
const ROLES = ['admin', 'user'] as const;            // readonly literal tuple
type Role = (typeof ROLES)[number];                  // 'admin' | 'user'
```

Concept	One line
Types are erased	nothing is checked at runtime — validate JSON with a schema (e.g. Zod)
Structural typing	shapes match, names don't matter
`unknown` vs `any`	`unknown` must be narrowed before use; `any` switches checking off
Narrowing	`typeof`, `instanceof`, `in`, discriminant property, type predicates `x is T`, `asserts` functions
Utility types	`Partial`, `Required`, `Readonly`, `Pick`, `Omit`, `Record`, `ReturnType`, `Parameters`, `Awaited`, `NonNullable`
`interface` vs `type`	both describe objects; `type` also does unions/tuples; interfaces merge
`satisfies`	checks a value against a type without widening it
Running	`tsc --noEmit` type-checks; `node file.ts` only strips erasable syntax (no `enum`, no parameter properties)
JSDoc	`// @ts-check` + `/** @param {number} x */` type-checks plain `.js` files (§16.15)

18.17 Which API for which job

Job	Reach for	Not	See
Default variable	`const` (`let` if reassigned)	`var`	§2.1
Compare values	`===`, `Object.is` for `NaN`/`-0`	`==` (except `x == null`)	§2.6
Default for a missing value	`??` / `??=`	`||` when `0` or `''` are valid	§2.7
Read possibly-missing nested data	`a?.b?.c`	long `&&` chains	§5.8
Money	integer cents, `Intl.NumberFormat` for display	floats	§2.10
Integers beyond 2^{53}	`BigInt`	`number`	§2.10
Parse user input to a number	`Number(s)` + `Number.isNaN`	`parseInt` without a radix	§2.12
Walk an array / string / Map	`for…of`	`for…in`	§3.12
Walk a plain object	`Object.entries(obj)` + `for…of`	`for…in` without `hasOwn`	§5.6
Transform / select / fold	`map` / `filter` / `reduce`	`forEach` + push	§7.5
Sort without touching the original	`toSorted(cmp)`	`sort()` on shared data	§7.4
Last element	`arr.at(-1)`	`arr[arr.length - 1]`	§7.3
Copy an object / array (top level)	`{ ...o }`, `[...a]`	`Object.assign` into the original	§5.4
Deep copy	`structuredClone`	`JSON.parse(JSON.stringify(x))`	§5.4
Dictionary with dynamic or non-string keys	`Map`	plain object	§8.1
Unique values, fast membership	`Set`	`array.includes` in a loop	§8.2
Metadata on objects you don't own	`WeakMap`	`Map` (leaks)	§8.3
Group items by a key	`Object.groupBy` / `Map.groupBy`	hand-written `reduce`	§8.7
Lazy or infinite sequence	generator + iterator helpers	building a huge array	§8.5
Text search / replace of fixed text	`includes`, `replaceAll`	regex	§6.6
Pattern matching	regex (`test`, `matchAll`, named groups)	manual char loops	§6.8
Count visible characters	`Intl.Segmenter`	`length`	§6.5
Format dates / numbers for people	`Intl.DateTimeFormat`, `toLocaleString`	string concatenation	§5.11
Callback that keeps `this`	arrow function	`const self = this`	§9.1
Truly private state	`#private` fields or closures	`_underscore` names	§9.5

Job	Reach for	Not	See
Share code between modules	ES modules (`import` / `export`)	globals, IIFEs	§10.2
Load code only when needed	`import()`	one giant bundle	§10.4
Reproducible installs	lockfile + `npm ci`	`npm install` in CI	§10.10
Report a failure	`throw new Error(msg, { cause })` / custom subclass	throwing strings	§11.3
Several independent async jobs	`await Promise.all([...])`	sequential awaits	§12.9
Report every outcome	`Promise.allSettled`	`all`	§12.8
Timeout or cancel	`AbortSignal.timeout(ms)`, `AbortController`	`Promise.race` alone (doesn't cancel)	§12.11
Wrap an old callback API	`util.promisify` or `new Promise` once	nesting callbacks	§12.7
Paged / streamed data	`async function*` + `for await…of`	loading everything first	§12.12
Insert user text into the page	`textContent`	`innerHTML`	§13.3
Handle events on many children	delegation with `closest`	a listener per item	§13.7
Remove many listeners at once	`{ signal }` + `controller.abort()`	tracking every function	§13.5
Animate	CSS transitions, or `requestAnimationFrame` changing `transform`	`setInterval` changing `top` / `left`	§14.5
React to scroll visibility / element size	`IntersectionObserver` / `ResizeObserver`	`scroll` / `resize` handlers with measurements	§14.9
HTTP request	`fetch` + check `res.ok`	`XMLHttpRequest`	§14.1
Build or parse URLs	`URL`, `URLSearchParams`	string concatenation	§14.6
Store a session	`HttpOnly; Secure; SameSite` cookie	`localStorage` token	§14.4
Large offline data	IndexedDB	`localStorage`	§14.4
Heavy computation in a page	Web Worker	a long loop on the main thread	§14.8
Heavy computation in Node	`worker_threads`	blocking the event loop	§15.2
Big files / HTTP bodies	streams + `pipeline`	`readFile` of gigabytes	§15.9
Run another program	`execFile` / `spawn` with an argument array	`exec` with user input	§15.11
Tests without dependencies	`node:test` + `node:assert/strict`	ad-hoc `console.log` checks	§16.3
Time-dependent tests	fake timers (`t.mock.timers`)	real `setTimeout` waits	§16.5
Types for untrusted data	`unknown` + narrowing / a schema	`any`	§16.10

Job	Reach for	Not	See
A fixed set of names in TS	union of literals or `as const` object	`enum` (not erasable)	§16.13

18.18 ECMAScript versions ES2015 → ES2026

Each edition is published in June and contains every proposal that reached **Stage 4** in the year before (§1.2). The years below were checked against TC39's list of finished proposals (github.com/tc39/proposals, "finished-proposals"), and the Node 24 column was checked by running the features in Node v24.11 (the `18.18` block of `cheatsheet_checks.mjs`).

Edition	Main features	Node 24
ES2015 (ES6)	`let` / `const`, arrow functions, classes, ES modules, promises, template literals, destructuring, default / rest / spread parameters, `Map` / `Set` / `WeakMap` / `WeakSet`, `Symbol`, iterators and generators, `for…of`, `Proxy` / `Reflect`, typed arrays, `Array.from` / `of`, `find` / `findIndex`, `Object.assign`, `String` `includes` / `startsWith` / `repeat`, `Number.isNaN` / `isInteger`, regex `u` and `y` flags	✓
ES2016	`Array.prototype.includes`, exponent operator `**`	✓
ES2017	`async` / `await`, `Object.values` / `entries`, `padStart` / `padEnd`, `Object.getOwnPropertyDescriptors`, trailing commas in parameter lists, `SharedArrayBuffer` and `Atomics`	✓
ES2018	object rest/spread `{ ...o }`, async iteration (`for await…of`, async generators), `Promise.prototype.finally`, regex `s` flag, named groups, lookbehind, `\p{…}` escapes	✓
ES2019	`flat` / `flatMap`, `Object.fromEntries`, `trimStart` / `trimEnd`, optional `catch` binding, `Symbol.prototype.description`, well-formed `JSON.stringify`; `sort` required to be stable	✓
ES2020	`BigInt`, `??`, `?.`, dynamic `import()`, `import.meta`, `globalThis`, `Promise.allSettled`, `String.prototype.matchAll`, defined `for…in` order	✓
ES2021	`replaceAll`, `Promise.any` + `AggregateError`, logical assignment `||=` `&&=` `??=`, numeric separators `1_000`, `WeakRef` / `FinalizationRegistry`	✓
ES2022	class fields, `#private` methods and fields, `static` blocks, `#x in obj`, top-level `await`, `.at()`, `Object.hasOwn`, error `cause`, regex `d` flag (match indices)	✓
ES2023	`toSorted` / `toReversed` / `toSpliced` / `with`, `findLast` / `findLastIndex`, hashbang `#!` lines, symbols as `WeakMap` keys	✓
ES2024	`Object.groupBy` / `Map.groupBy`, `Promise.withResolvers`, regex `v` flag, `isWellFormed` / `toWellFormed`, resizable `ArrayBuffer`, `ArrayBuffer.prototype.transfer`, `Atomics.waitAsync`	✓
ES2025	iterator helpers + `Iterator.from`, `Set` methods (`union`, `intersection`, …), `Promise.try`, `RegExp.escape`, regex pattern modifiers `(?i:…)`, duplicate named groups, import attributes + JSON modules, `Float16Array` / `Math.f16round`	✓
ES2026	`Array.fromAsync`, `Error.isError`, `JSON.parse` source text access (`JSON.rawJSON`), `Math.sumPrecise`, `Uint8Array` to/from base64 and hex, `Iterator.concat`, upsert (`Map.prototype.getOrInsert` / `getOrInsertComputed`)	partial: first three ✓; the rest ✗ (feature-detect)
Next: ES2027	finished but not yet published: **`Temporal`**, explicit resource management (`using`, `Symbol.dispose`), `Atomics.pause`, joint iteration (`Iterator.zip`)	`using` and `Atomics.pause` already work; Temporal and `Iterator.zip` don't

"Node 24 ✓" means Node's V8 ships the feature. For browsers, check MDN's compatibility tables or the Baseline badge before relying on anything from the last two editions. A transpiler can rewrite new *syntax*, but only a polyfill can add missing *APIs* (§10.12).

18.19 Where to go next

The course behind this book closes by inviting feedback and pointing to further courses. A list of places is more useful with an order, so here is a four-step study path built on what this book covered.

Step 1 — Lock in the core (1–2 weeks)

1. Re-read the TL;DR at the end of each chapter, then do each sheet in this chapter with the right-hand column covered.
2. Work through the ordering puzzles in §12.3 and the 53 questions in §17.4 until you can explain each answer out loud.
3. Rebuild from memory: `debounce`, `throttle`, `curry`, `memoize`, a `Promise.all` polyfill and an LRU cache (§17.3).

Step 2 — Build small, real things (2–4 weeks)

Pick projects that force you to combine chapters:

Project	Exercises
To-do list with filters, saved in `localStorage`	DOM, events, delegation, immutable array updates, storage (Ch 13, Ch 14)
Search-as-you-type against a public API	`fetch`, `AbortController`, debounce, last-request-wins, error states (Ch 12)
A command-line tool that summarizes a CSV or log file	`node:fs/promises`, streams, `parseArgs`, exit codes (Ch 15)
A tiny JSON REST API with tests	`node:http`, validation, `node:test`, TypeScript via type stripping (Ch 15, Ch 16)

Write tests for each one, run ESLint and Prettier, and put it in a git repository.

Step 3 — Choose a direction

Direction	Next topics	In this library
Front end	semantic HTML, CSS layout (flexbox, grid), accessibility, then React (components, state, effects, routing, data fetching)	HTML & CSS Handbook, React Handbook
Back end with Node	a framework (Express, Fastify or Hono), SQL and schema design, authentication, caching, queues	SQL Handbook, Database modeling
Types at scale	strict TypeScript, generics, declaration files, typing React components	§16.8, TypeScript with React
Interviews	DSA patterns in JS, machine-coding rounds, object-oriented and system design	Ch 17, LLD Handbook, DDIA notes

Step 4 — Keep up to date

- **MDN Web Docs** is the reference for every API in this book, with browser-compatibility tables.
- **The ECMAScript spec** (tc39.es/ecma262) and the TC39 proposals repository show what's coming; features at Stage 3 are usually safe to learn.
- **Node.js release notes and docs** (nodejs.org): use an even-numbered LTS line.
- **Books for depth**: *You Don't Know JS Yet* (scope, closures, `this`, types), *Eloquent JavaScript* (fundamentals with exercises), and javascript.info as a free, well-structured tutorial.
- Read other people's code: small, well-maintained npm packages are a good place to start.

18.20 Interview question index

All questions below are answered, with real output, in Chapter 17. Links go to the group that contains the question.

Coercion and equality → §17.4.1

Q	Topic	Q	Topic
1	`+` with strings, arrays and objects	8	falsy values, `[]` and `new Boolean(false)`
2	`null == undefined`, `null >= 0`, `'' == 0`	9	`'b' + 'a' + +'a' + 'a'`
3	why `[] == ![]` is true	10	`['1','7','11'].map(parseInt)`
4	`typeof null`	11	`[10, 1, 2].sort()`
5	how `NaN` behaves	12	`||` vs `??`
6	`Object.is` vs `===`	13	`{} === {}`
7	`typeof` for arrays, functions, classes, undeclared names	14	`0.1 + 0.2` and `Math.max() < Math.min()`

Scope, hoisting and closures → **§17.4.2**: Q15 `var` vs `let` vs `const` · Q16 reading before the declaration (TDZ) · Q17 calling a function before its definition · Q18 closures in loops · Q19 what a closure is · Q20 `let a = b = 5` · Q21 `var` leaking out of a block · Q22 what strict mode changes.

`this`, arrows, `call`/`apply`/`bind`, `new` → **§17.4.3**: Q23 a method losing `this` · Q24 arrow vs regular functions · Q25 nested function vs nested arrow · Q26 `call` vs `apply` vs `bind` · Q27 binding twice · Q28 what `new` does and constructor return values.

Prototypes and classes → **§17.4.4**: Q29 `prototype` vs `__proto__` · Q30 how `instanceof` works · Q31 classes vs constructor functions · Q32 `Object.create(null)` · Q33 `#private` vs closure privacy · Q34 `this` before `super()`.

Event loop and async → **§17.4.5**: Q35 the classic timer/promise order · Q36 `async`/`await` order · Q37 `nextTick`, promises, `setImmediate` · Q38 executor timing and resolving twice · Q39 why `try/catch` misses a timer's error · Q40 `await` inside `forEach` · Q41 two independent awaits · Q42 `all` vs `allSettled` vs `race` vs `any`.

Objects, arrays and copies → **§17.4.6**: Q43 shallow vs deep copy · Q44 changing a `const` object, `Object.freeze` · Q45 objects as object keys · Q46 `delete arr[i]` and `arr.length = n` · Q47 pass-by-value or by reference · Q48 `Map` vs object · Q49 what `JSON.stringify` drops · Q50 `for…in` vs `for…of`.

Memory → **§17.4.7**: Q51 how memory is freed · Q52 causes of leaks and fixes · Q53 `WeakMap`, `WeakRef`, `FinalizationRegistry`.

Also in Chapter 17: the complexity table of built-ins (§17.1), twenty-one coding patterns with a summary table (§17.2), and implement-from-scratch classics — debounce, throttle, curry, compose/pipe, memoize, flatten, `bind`, deep clone, deep equal, LRU cache, EventEmitter, promise combinator polyfills (§17.3).

Coding patterns at a glance → **§17.2**

#	Pattern	Reach for it when…	Classic problems in §17.2
1–3	Two pointers, sliding window, hashing	sorted pairs; contiguous ranges; "seen before"	pair sum, longest unique substring, two sum, anagram groups
4–6	Stack, prefix sums, intervals	nesting / next greater; range sums; overlaps	brackets, daily temperatures, subarray sum = k, merge, meeting rooms
7	Heap, two heaps, k-way merge	top k, cheapest next, running median	k largest, Dijkstra, median of a stream, merge k sorted lists
8–10	Binary search, BFS, DFS	sorted / monotone yes-no; fewest steps; connectivity	lower bound, eating speed, grid path, islands, DFS topo order
11–12	DP with memo, backtracking	overlapping subproblems; list every option	stairs, coin change, LCS, subsets, N-Queens
13	Linked lists	rewire `next` references	reverse, remove k-th from end, intersection
14	Fast & slow pointers	cycle or middle in O(1) space	cycle start, middle node, happy number
15	Trees	combine left/right subtree answers	validate BST, lowest common ancestor, serialize
16	Tries	prefix lookups, many words at once	prefix tree, wildcard match, words on a board
17	Topological sort, union-find	dependencies; merging groups	course order (Kahn), components, redundant edge
18	Greedy	a local choice you can prove safe	jump game, gas station, candies
19	Sort & search	k-th item, 3-way partition, stable sort	quickselect, Dutch flag, merge sort
20	Bit manipulation	xor cancelling, bit counts, masks (32-bit!)	single number, counting bits, swap odd/even bits
21	Math & geometry	matrix walks, integer limits, exact slopes	spiral order, reverse integer, max points on a line

---

18.21 Glossary (A–Z)

Each term has a one-line definition and a link to the section that explains it. Symbols and backticks are ignored when sorting.

A

- **AbortController** — An object whose `signal` is passed to `fetch`, listeners or timers; calling `abort()` cancels them all. §12.11
- `AbortSignal.any` — Combines several signals into one that aborts as soon as any of them does, e.g. a user's cancel button plus a timeout. §12.11
- **AbortSignal.timeout** — A ready-made signal that aborts itself after a number of milliseconds with a `TimeoutError`. §12.11
- **Abstract class** — A class meant only to be extended, never created directly; JavaScript has no `abstract` keyword (a constructor can throw if `new.target` is the base class), while TypeScript adds one. §9.6
- **Abstract syntax tree (AST)** — A tree-shaped description of your code's structure that a parser builds before the engine compiles or runs it; linters and transpilers work on it too. §1.3
- **Accessor property** — A property backed by a getter and/or setter function instead of a stored value. §5.7

- **Accidental global** — A variable that becomes a property of the global object because it was assigned without `let`/`const`/`var` in sloppy mode; strict mode turns it into a `ReferenceError`. §17.4
- **`addEventListener`** — Registers a function to run when an event of a given type reaches an element; accepts `once`, `passive`, `capture` and `signal` options. §13.5
- **AggregateError** — An error that wraps several errors in its `errors` array; `Promise.any` rejects with one when every input rejects. §11.1
- **AMD / UMD** — Older module formats: AMD loads modules asynchronously with `define()`; UMD wraps a file so it works as AMD, CommonJS or a global. §10.1
- **Amortized cost** — The average cost per operation over a long run: `push` is O(1) amortized even though the array sometimes has to grow and copy. §17.1
- **Anchor (regex)** — A position rather than a character: `^` start, `$` end, `\b` word boundary. §6.7
- **`any` (TS)** — The TypeScript type that switches checking off for a value; anything goes in and out, so prefer `unknown`. §16.10
- **API** — Application Programming Interface: the set of functions, objects and rules one piece of software offers for others to use, such as the DOM API or a web server's HTTP endpoints. §1.4
- **Argument** — The actual value you pass when calling a function; the parameter is the name that receives it. §4.1
- **`arguments`** — An array-like object holding every argument passed to a non-arrow function; replaced in modern code by rest parameters. §4.4
- **Arrange–Act–Assert (AAA)** — A layout for a test: set up the data, perform the one action under test, then check the result. §16.3
- **Array destructuring** — Unpacking values by position: `const [a, , c = 0, ...rest] = arr`. §7.7
- **Array-like** — An object with `length` and numeric indexes but no array methods (e.g. `arguments`, a `NodeList`); convert with `Array.from`. §4.4
- **`Array.from`** — Builds a real array from an iterable or array-like, with an optional mapping function. §7.2
- **`Array.fromAsync`** — Collects an async iterable (or iterable of promises) into an array, returning a promise (ES2026). §12.12
- **`Array.isArray`** — The reliable array test; works across realms, unlike `instanceof Array`. §7.1
- **ArrayBuffer** — A fixed block of raw bytes that typed arrays and `DataView` read and write. §7.9
- **Arrow function** — Short function syntax `(x) => x * 2` with lexical `this`, no `arguments` and no `new`. §4.3
- **`as const` (TS)** — Tells TypeScript to treat a literal as deeply read-only with its exact values (`'admin'` rather than `string`). §16.13
- **ASCII** — An old 7-bit character set covering 128 characters (English letters, digits, punctuation); its codes are the first 128 Unicode code points. §6.9
- **Assertion** — A check in a test that fails loudly when a value isn't what you expected (`assert.equal`, `assert.throws`). §16.2
- **Assertion function (TS)** — A TypeScript function declared `asserts v is T` that narrows its argument for the code after the call. §16.11
- **Assignment operators** — `=` plus compound forms such as `+=`, `**=`, `||=`, `&&=` and `??=`. §2.8
- **Associativity** — Which side groups first when operators of equal precedence meet: `-` is left-to-right (`8 - 3 - 1` is 4), while `**` and `=` are right-to-left. §2.9
- **`async` function** — A function that always returns a promise and may use `await` inside. §12.9
- **Async generator** — An `async function*` that yields values over time; consumed with `for await…of`. §12.12
- **Async iteration** — Walking a sequence whose items arrive asynchronously, using `Symbol.asyncIterator` and `for await…of`. §12.12
- **`async` script** — `<script async>` downloads in parallel and runs as soon as it arrives, in no guaranteed order; fine for independent scripts such as analytics. §13.12
- **Attribute (HTML)** — The text written in markup (`value="init"`); read with `getAttribute`. Often the initial value only. §13.4

- **Auto-boxing** — The engine briefly wraps a primitive in its object wrapper so that `'abc'.toUpperCase()` works. §6.4
- **Automatic Semicolon Insertion (ASI)** — The parser's rule for adding missing semicolons; it is why a line break straight after `return` returns `undefined`. §1.6
- `await` — Pauses an async function until a promise settles; gives the value or throws the rejection reason. §12.9

B

- **Babel** — A transpiler that rewrites modern JavaScript (and JSX/TypeScript) into syntax older engines understand. §10.12
- **Backpressure** — A stream's signal that the consumer is slower than the producer; `write()` returns `false` until `'drain'`. §15.9
- **Backreference** — `\1` or `\k<name>` in a regex: match the same text a group matched earlier. §6.7
- **Backtracking** — Building a solution step by step and undoing the last choice when it leads to a dead end; used to list subsets, permutations and N-Queens boards. §17.2
- **Bare specifier** — An import name without a path, such as `'lodash'`; Node finds it in `node_modules`, a browser needs an import map. §10.7
- **Barrel file** — A module that only re-exports other modules (`export * from './a.mjs'`) to offer one import path. §10.2
- **Base case** — The condition that stops a recursive function from calling itself again. §4.10
- **Base64** — A way to write arbitrary bytes using only 64 safe text characters (A–Z, a–z, 0–9, `+`, `/`), so binary data can travel inside text such as JSON or URLs. §15.7
- **Baseline** — A label on MDN and web.dev saying a web feature works in all major current browsers ("newly" or "widely" available). §18.18
- **Basic Multilingual Plane (BMP)** — The first 65,536 Unicode code points; characters outside it, like most emoji, need two UTF-16 code units (a surrogate pair). §6.5
- **BigInt** — A primitive for integers of any size, written `10n`; can't be mixed with `number` in arithmetic. §2.10
- **Big-O notation** — A way to describe how running time or memory grows with input size `n`, ignoring constants: O(1) constant, O(log n), O(n) linear, O(n log n), O(n²). §17.1
- **Binary search** — Finding a target in sorted data by repeatedly checking the middle and discarding half, taking O(log n) steps. §17.2
- **Binary search tree (BST)** — A binary tree where every left descendant is smaller and every right descendant larger than the node, so an in-order walk is sorted. §17.2
- **Binary tree** — A tree in which each node has at most two children, `left` and `right`. §17.2
- `bind` — Returns a new function with `this` (and optionally leading arguments) fixed permanently. §9.2
- **Binding** — The link between a name and the storage it refers to; `let` in a `for` loop creates a fresh binding each iteration. §4.7
- **Bitwise operators** — `& | ^ ~ << >> >>>`: operate on the 32-bit integer form of numbers. §2.8
- **Block** — Statements grouped in `{ }`, as in an `if` body or loop body; `let` and `const` are scoped to it. §1.6
- **Block scope** — Visibility limited to the nearest `{ }`; applies to `let`, `const` and `class`. §4.5
- **Blocking** — Code that keeps the single JavaScript thread busy (a long loop, synchronous file read), so nothing else, not even clicks or other requests, can run meanwhile. §12.1
- **Bound function** — The new function returned by `bind`, with `this` (and optionally some arguments) fixed permanently. §9.2
- **Brand check** — Testing whether an object was really made by a class by checking for one of its private fields: `#secret in obj`. §9.7
- **Breadth-first search (BFS)** — Exploring a graph or grid level by level using a queue; finds the fewest-steps path when all steps cost the same. §17.2
- **Breakpoint** — A marker in DevTools or an editor that pauses execution at a line so you can inspect variables; can be conditional. §11.9

- **browserslist** — A shared config listing which browsers a project supports; Babel, autoprefixer and other tools read it to decide what to transform. §10.12
- **Bubbling** — The phase in which an event travels from the target back up through its ancestors. §13.6
- **Buffer** — Node's type for raw bytes; its length is in bytes, not characters. §15.7
- **Built-in objects** — Objects the language provides everywhere, such as `Object`, `Array`, `Math`, `JSON`, `Date`, `Map` and `Promise`. §5.9
- **Bun** — A JavaScript runtime built on JavaScriptCore with its own package manager, bundler and test runner. §15.12
- **Bundler** — A tool that follows imports and produces optimized files for the browser (Vite, esbuild, webpack, Rollup). §10.11
- **Busy-waiting** — Burning CPU in a loop until a condition changes, instead of waiting for an event or timer; it freezes the page in JavaScript. §3.7
- **Bytecode** — A compact, low-level instruction format an engine's interpreter runs; V8's Ignition turns your parsed code into it. §1.3

C

- **Cache API** — A browser store of request/response pairs, used mostly by service workers for offline support. §14.4
- **`call` / `apply`** — Invoke a function right now with a chosen `this`; `call` takes listed arguments, `apply` an array. §9.2
- **Call site** — The place in the code where a function is called; for regular functions it decides what `this` is. §9.1
- **Call stack** — The stack of function calls currently running; the newest frame is on top. §12.2
- **Callback** — A function passed to another function to be called later or for each item. §4.8
- **Callback hell** — Deeply nested callbacks where each step depends on the previous one; hard to read and to handle errors. §12.6
- **camelCase** — Naming style where the first word is lower-case and later words start with a capital (`totalPrice`); the JavaScript convention for variables and functions. §2.1
- **Cancelable event** — An event whose default action `preventDefault()` can stop; `event.cancelable` tells you. §13.8
- **Capture phase** — The first event phase, travelling from `window` down to the target; listen with `{ capture: true }`. §13.6
- **Capturing group** — `( … )` in a regex: groups a sub-pattern and remembers its match as `$1`, `m[1]`. §6.7
- **Caret range (`^`)** — An npm version range allowing newer minor and patch releases of the same major (`^1.2.3` means at least 1.2.3, below 2.0.0). §10.10
- **`cause` (error)** — Option `new Error(msg, { cause })` that links an error to the one that caused it (ES2022). §11.4
- **Character class** — `[abc]`, `[a-z]` or shorthands like `\d`: matches one character from a set. §6.7
- **Child process** — Another OS program started from Node with `node:child_process` (`execFile`, `spawn`, `fork`). §15.11
- **Chunk** — One piece of data delivered by a stream (often a `Buffer` of up to 64 KB); also a separate file a bundler emits for code splitting. §15.9
- **CI (continuous integration)** — A server that automatically installs, lints, tests and builds every change pushed to a repository. §10.10
- **Circular dependency** — Two or more modules that import each other; ES modules allow it, but one side may see the other's bindings before they are initialised. §10.3
- **Class** — Syntax for a constructor plus prototype methods, with fields, `#private` members, statics and `extends`. §9.5
- **Class field** — A property declared in the class body (`count = 0`) and created on each instance (ES2022). §9.5
- **Classic script** — A normal `<script>` without `type="module"`: sloppy by default, top-level `var` becomes a global, and it blocks HTML parsing unless `defer` or `async` is set. §1.8
- **`classList`** — An element's list of CSS classes with `add`, `remove`, `toggle` and `contains`. §13.4

- **CLI** — Command-line interface: a program you run by typing commands in a terminal, such as `node`, `npm` or `tsc`. §10.9
- **Clickjacking** — Tricking a user into clicking something on your site hidden inside an invisible frame on another site; blocked with CSP `frame-ancestors`. §14.12
- `closest` — Returns the element itself or its nearest ancestor that matches a CSS selector, or `null`; the workhorse of event delegation. §13.2
- **Closure** — A function together with the variables of the scope where it was created, which it can still read and change. §4.7
- **Code point** — A Unicode character number (U+0000 to U+10FFFF); some need two UTF-16 code units. §6.5
- **Code splitting** — Breaking a bundle into chunks that load on demand, usually at each dynamic `import()`. §10.11
- **Code unit** — One 16-bit value of UTF-16; `length` and indexes count these. §6.5
- **Coercion** — Automatic type conversion done by operators such as `+`, `==` and `if`. §2.12
- **Combinator (promise)** — A function that turns several promises into one: `Promise.all`, `allSettled`, `race` and `any`. §12.8
- **Command injection** — An attack where user input passed to a shell is run as extra commands; avoided by `execFile`/`spawn` with an argument array. §15.11
- **CommonJS** — Node's original module system: `require()` and `module.exports`, loaded synchronously. §10.8
- **Comparator** — The function passed to `sort`: negative keeps `a` first, positive puts `b` first, 0 keeps their order. §7.6
- **Compile** — Translating source code into another form, such as machine code or bytecode, before it runs; JavaScript engines do this on the fly. §1.3
- **Composition** — Building behaviour by combining objects or functions (has-a) instead of inheriting (is-a). §9.9
- **Compound assignment** — An operator that combines an operation with assignment: `x += 1` means `x = x + 1`. §2.8
- **Computed property name** — An object key written as an expression in brackets: `{ [key]: value }`. §5.2
- **Concatenation** — Joining strings end to end, with `+`, `concat` or a template literal. §6.3
- **Concurrency limit** — Running many async jobs but only N at a time, so you don't flood a server or run out of file handles. §12.13
- **Conditional breakpoint** — A breakpoint that pauses only when an expression you give is truthy (e.g. `i === 500`). §11.9
- **Conditional statement** — Code that chooses what to run based on a condition: `if`, `else if`, `else` and `switch`. §3.1
- **Conditional types (TS)** — Types that pick one of two results based on a test, written `T extends U ? X : Y`. §16.12
- `const` — Declares a block-scoped binding that can't be reassigned (the value may still be mutated). §2.1
- **Constraint Validation API** — Built-in form validation from HTML attributes (`required`, `min`, `pattern`), read via `input.validity` and customised with `setCustomValidity`. §13.9
- **Constructor** — The function that sets up a new object: the `constructor` method of a class, or a function called with `new`. §9.5
- **Constructor function** — A regular function called with `new` to create objects; the pre-class way to make "types". §9.4
- **Content Security Policy (CSP)** — A response header that tells the browser which sources of scripts, styles and frames are allowed. §14.12
- **Content-Type / MIME type** — The header naming the format of a body, such as `application/json` or `text/html`; MIME types are those standard format names. §14.1
- **Cookie** — A small string the server sets and the browser sends back automatically with matching requests. §14.4
- **core-js** — A widely used library of polyfills for modern JavaScript built-ins. §10.12
- **CORS** — Cross-Origin Resource Sharing: headers by which a server allows pages from other origins to read its responses. §14.2

- **Coverage** — The share of code lines or branches your tests executed; a hint, not a goal. §16.6
- **CPU-bound / I/O-bound** — CPU-bound work is limited by computation (hashing, image resizing); I/O-bound work mostly waits for disks or networks. Node is great at the second, needs workers for the first. §15.2
- **CSRF** — Cross-site request forgery: another site makes the victim's browser send an authenticated request. §14.11
- **CSRF token** — A secret, unpredictable value the server puts in a form and checks on submit, proving the request came from its own page. §14.11
- **CSS** — Cascading Style Sheets: the language that controls how HTML looks (colours, layout, fonts). §1.1
- `currentTarget` — The element whose listener is running now, as opposed to `target`, where the event started. §13.5
- **Currying** — Turning `f(a, b, c)` into `f(a)(b)(c)` so arguments can be supplied one at a time. §4.8
- **CustomEvent** — An event you create with your own `detail` data and fire with `dispatchEvent`. §13.5

D

- `dataset` — Access to `data-*` attributes as camelCase string properties (`data-user-id` → `dataset.userId`). §13.4
- `Date` — A built-in object holding milliseconds since 1970-01-01 UTC, with local and UTC getters. §5.11
- **Daylight-saving time (DST)** — Seasonal clock changes that make some local days 23 or 25 hours long; a reason to store and compute dates in UTC. §5.11
- **Debounce** — Run a function only after calls have stopped for a given time (e.g. search-as-you-type). §17.3
- `debugger` — A statement that pauses execution when developer tools are attached. §11.9
- **Declaration** — Code that introduces a name: `let x`, `const y = 1`, `function f() {}` or `class C {}`. §2.1
- **Declarative code** — Code that says what result you want (`items.filter(isActive)`) rather than spelling out each step in a loop (imperative code). §7.5
- **Deep copy** — A copy in which nested objects are copied too, e.g. `structuredClone(obj)`. §5.4
- **Default action** — What the browser does on its own after an event, such as following a link or submitting a form; stopped with `preventDefault()`. §13.8
- **Default binding** — The `this` rule for a plain call `f()`: `undefined` in strict code, the global object in sloppy scripts. §9.1
- **Default export** — The single unnamed export of a module, imported under any name: `import x from './m.mjs'`. §10.2
- **Default parameter** — A value used when an argument is missing or `undefined`: `function f(n = 1)`. §4.4
- **Defence in depth** — Using several independent protections (escaping, CSP, `HttpOnly` cookies) so one failure doesn't expose you. §14.10
- `defer` — Script attribute: download in parallel, run after the HTML is parsed, in document order. §13.12
- `delete` — Operator that removes an own property from an object. §5.2
- **Deno** — A V8-based runtime with deny-by-default permissions, built-in TypeScript and tooling. §15.12
- **Deoptimization** — When the optimizing compiler's assumptions about your code (e.g. "this is always a number") turn out wrong, so the engine throws away the fast machine code and goes back to slower code. §1.3
- **Dependency** — A package your project uses, listed in `package.json`; transitive dependencies are the packages your dependencies use in turn. §10.9
- **Dependency injection** — Passing collaborators (clock, database, HTTP client) in as arguments so tests can swap them for fakes. §16.5
- **Depth-first search (DFS)** — Exploring a graph or tree by going as deep as possible down one path before backing up, using recursion or a stack. §17.2
- **Destructuring** — Syntax that unpacks properties or elements into variables, with renames and defaults. §5.5
- `devDependencies` — Packages needed only to build and test (linters, bundlers), not at runtime. §10.9

- **DevTools** — The developer tools built into browsers (F12): console, element inspector, debugger, network and performance panels. §1.5
- **Dijkstra's algorithm** — Finds the cheapest path from one node to all others in a graph with non-negative edge weights, using a priority queue. §17.2
- **Directive** — A string statement that changes how code is treated, such as `"use strict"` at the top of a script or function. §1.8
- **Discriminated union** — A TypeScript union whose members share a literal "kind" property used to narrow between them. §16.11
- `dispatchEvent` — Fires an event object at a node; its listeners run synchronously. §13.5
- **Document** — The root object of the DOM for one page (`document`); everything you select or create starts from it. §13.1
- **DocumentFragment** — A lightweight container for building many nodes off-page before inserting them at once. §13.3
- **DOM** — Document Object Model: the live tree of objects the browser builds from HTML and exposes to scripts. §13.1
- **DOM-based XSS** — XSS that happens entirely in the browser when script copies attacker-controlled data (often from the URL) into a dangerous sink like `innerHTML`. §14.10
- `DOMContentLoaded` — Event fired when the HTML is parsed and deferred scripts have run; images may still be loading. §13.12
- **DOMPurify** — A widely used library that sanitizes untrusted HTML before it goes into `innerHTML`. §14.10
- **dotAll flag** — The regex `s` flag: lets `.` match newline characters too. §6.7
- **Dual package hazard** — When both the CommonJS and ESM builds of one package get loaded, so the app ends up with two separate copies of its state. §10.8
- **Duck typing** — Checking whether a value has the methods you need rather than which class made it. §9.7
- **Dummy (test double)** — A placeholder argument a test passes only to fill a parameter; it is never actually used. §16.5
- **Duplex stream** — A stream that is both readable and writable, like a network socket. §15.9
- **Dutch national flag partition** — Sorting an array of three kinds of value (e.g. 0, 1, 2) in one pass with three pointers. §17.2
- **Dynamic import** — `import(specifier)`: loads a module at runtime and returns a promise for its namespace. §10.4
- **Dynamic programming (DP)** — Solving a problem by combining answers to overlapping smaller subproblems, storing each so it is computed once (memo top-down or table bottom-up). §17.2
- **Dynamic typing** — Types belong to values, not variables; a variable may hold a number and later a string. §1.1

E

- **Ecma International** — The standards organisation that publishes ECMAScript (as ECMA-262); its TC39 committee designs the language. §1.2
- **ECMAScript** — The official language standard (ECMA-262) that JavaScript engines implement; a new edition every June. §1.2
- **Electron** — A framework for desktop apps (VS Code, Slack) built from Chromium plus Node.js, written in HTML, CSS and JavaScript. §1.4
- **Element** — A DOM node that represents an HTML tag, such as `<li>` or `<button>`; it has attributes, children and methods like `querySelector`. §13.1
- **Encapsulation** — Hiding internal state so outside code can only use a public interface; achieved with closures, `#private` fields or module scope. §4.7
- **Encoding** — The rule that turns text into bytes and back; UTF-8 is the usual one (`'é'` is 2 bytes in UTF-8). §15.7
- **Endianness** — The byte order of multi-byte numbers in memory: little-endian stores the least significant byte first; `DataView` lets you choose. §7.9

- **End-to-end (E2E) test** — A test that drives the whole application through its real interface, often a browser. §16.1
- **Engine** — The program that parses, compiles and runs JavaScript: V8, SpiderMonkey, JavaScriptCore. §1.3
- **`ENOENT`** — Node's error code for "no such file or directory"; check `err.code` rather than the message. §15.4
- **Entry point** — The first module or file that starts an app (or that a bundler starts from) and from which all imports are followed. §10.3
- **Enum (TS)** — A TypeScript construct that creates a named set of constants as a real runtime object; not erasable, so plain unions or `as const` objects are often preferred. §16.13
- **Enumerable** — A property attribute; enumerable properties appear in `for…in`, `Object.keys` and spread. §5.6
- **Environment variable** — A named string setting given to a process by the OS or shell (e.g. `PORT=3000`), read in Node with `process.env`. §15.6
- **Erasable syntax (TS)** — TypeScript syntax that disappears when types are deleted (annotations, interfaces); `enum` and parameter properties are not erasable because they generate code. §16.9
- **Error-first callback** — Node's callback convention `(err, result) => …`, where `err` is checked first. §12.6
- **`Error.isError`** — Reliable check that a value is a genuine error object, even across realms (ES2026). §9.7
- **ES module (ESM)** — The standard module system: static `import` / `export`, strict mode, live bindings. §10.2
- **esbuild** — A very fast bundler and transpiler written in Go. §10.11
- **Escape sequence** — A backslash code inside a string literal for a character that is hard to type: `\n` newline, `\t` tab, `\'`, `\u{1F600}`. §6.2
- **ESLint** — A configurable linter that reports likely bugs and style problems in JavaScript and TypeScript. §10.13
- **Event** — An object describing something that happened (click, key press, load) delivered to listeners. §13.5
- **Event delegation** — One listener on a container handles events from many children using `event.target.closest()`. §13.7
- **Event listener** — A function registered for an event type on a target; the browser calls it with an event object when that event happens. §13.5
- **Event loop** — The host scheduler that runs one task, then all microtasks, then possibly a render, repeatedly. §12.2
- **Event object** — The object passed to a listener, describing what happened: `type`, `target`, `currentTarget`, key or mouse details, and methods like `preventDefault`. §13.5
- **EventEmitter** — Node's publish/subscribe class: `on`, `once`, `off` and synchronous `emit`. §15.8
- **Exception** — An error that has been thrown and interrupts normal flow until a `catch` handles it; uncaught, it ends the script or process. §11.1
- **Excess property check (TS)** — TypeScript's error when an object literal has a property its target type doesn't declare, usually a typo. §16.10
- **Execution context** — The engine's record for a running piece of code (its variables, scope chain and `this`); each call pushes one onto the call stack. §4.1
- **Executor** — The function passed to `new Promise((resolve, reject) => …)`; it runs immediately and synchronously. §12.7
- **Exit code** — The number a process returns to the OS when it ends: 0 for success, anything else for failure; set with `process.exitCode`. §15.6
- **Explicit binding** — Setting `this` yourself with `call`, `apply` or `bind`. §9.1
- **Explicit resource management** — The `using` / `await using` declarations plus `Symbol.dispose`, which clean up a resource automatically when its block ends (ES2027). §11.6
- **Exponential backoff** — Retrying a failed request with waits that double each time (100 ms, 200 ms, 400 ms …), usually plus random jitter. §12.13

- **Exponentiation operator** — `**`: raises to a power (`2 ** 10` is 1024). §2.5
- `export` — Makes a binding available to other modules, by name or as the default. §10.2
- **Express** — The most widely used Node.js web framework, adding routing and middleware on top of `node:http`. §15.10
- **Expression** — Code that produces a value (`2 + 3`, `f(x)`, `a ? b : c`). §1.6
- `extends` — Makes one class inherit from another (or from a built-in such as `Error`), linking their prototype chains. §9.6

F

- **Factory function** — A plain function that builds and returns a new object, an alternative to classes and `new`. §9.9
- **Fake (test double)** — A working but simplified stand-in, such as an in-memory database used in place of a real one. §16.5
- **Fake timers** — Test tool that replaces `setTimeout`/`Date` so you can move time forward instantly. §16.5
- **Fall-through** — In a `switch`, running on into the next `case` because a `break` (or `return`) is missing. §3.4
- **Falsy** — A value that becomes `false` in a boolean context: `false`, `0`, `-0`, `0n`, `''`, `null`, `undefined`, `NaN`. §2.11
- **Fast and slow pointers** — Two pointers moving through a list at different speeds; they meet if there is a cycle, and the slow one finds the middle. §17.2
- **Fastify / Hono / Koa** — Alternative Node.js web frameworks to Express: Fastify focuses on speed, Hono is tiny and runs on many runtimes, Koa is a minimal async-first design. §15.10
- **Feature detection** — Checking whether an API exists before using it (`if ('Temporal' in globalThis)`), instead of guessing from the browser name. §1.4
- `fetch` — The promise-based HTTP API in browsers and Node; resolves even for 4xx/5xx responses. §14.1
- **FinalizationRegistry** — Lets you ask for a callback some time after an object has been garbage-collected; timing is never guaranteed. §8.3
- `finally` — Block that runs after `try`/`catch` whatever happened, for cleanup. §11.2
- **First-class function** — Functions are values: stored in variables, passed as arguments and returned from other functions. §4.1
- **Fisher–Yates shuffle** — The correct way to shuffle an array fairly: walk from the end, swapping each item with a random earlier one (or itself). §7.6
- `flat` / `flatMap` — Flatten nested arrays by a depth / map then flatten one level. §7.5
- **Floating point** — The way computers store fractional numbers in binary with limited precision, which is why `0.1 + 0.2` isn't exactly `0.3`. §2.10
- `for await…of` — Loops over an async iterable, awaiting each value. §12.12
- **Forced synchronous layout** — When reading a layout value (like `offsetHeight`) after a style change makes the browser recalculate layout immediately; repeated, it causes layout thrashing. §13.11
- `for…in` — Loops over enumerable property names (strings), including inherited ones. §3.9
- **Formatter** — A tool that rewrites code to a consistent style (spacing, quotes, line breaks) without changing behaviour; Prettier is the common one. §10.13
- `for…of` — Loops over the values of any iterable (arrays, strings, Maps, Sets, generators). §3.10
- **FormData** — An object holding a form's field names and values; can be sent directly with `fetch`. §13.9
- **Framework** — A larger toolkit that shapes how a whole app is structured and calls your code (React, Angular, Express); a library is something your code calls. Ch 1
- **Fulfilled** — The state of a promise that finished successfully with a value. §12.7
- **Function** — A reusable, named (or anonymous) block of code that takes inputs (parameters), runs, and may return a value; in JavaScript it is also an object. §4.1
- **Function composition** — Combining functions so the output of one feeds the next: `compose(f, g)(x)` is `f(g(x))`; `pipe` runs left to right. §4.8
- **Function declaration** — `function name() {}` as a statement; hoisted with its body. §4.2

- **Function expression** — A function used as a value (`const f = function () {}`); follows its variable's hoisting. §4.2
- **Function scope** — The scope created by each function call; `var` declarations and parameters live here. §4.5

G

- **Garbage collection** — Automatic freeing of memory for objects no longer reachable from roots such as globals and the stack. §17.4.7
- **Generational garbage collection** — Collecting young objects often and old ones rarely, because most objects die young; V8 works this way. §17.4
- **Generator** — A `function*` that can pause at `yield`; calling it returns an iterator. §8.5
- **Generics (TS)** — Type parameters such as `<T>` that let one function or type work for many types safely. §16.12
- `getComputedStyle` — Returns the final CSS values the browser applied to an element after all stylesheets and inline styles. §13.4
- **Getter** — A `get` method that runs when a property is read. §5.7
- **Global object** — The object that holds the host's global variables (`window` in browsers, `global` in Node), always reachable as `globalThis`. §1.4
- **Global scope** — The outermost scope, visible to all code; names declared there can clash across scripts. §4.5
- `globalThis` — The standard name for the global object in every environment (ES2020). §1.4
- **Graceful shutdown** — Stopping a server cleanly on a signal like `SIGTERM`: stop accepting new work, finish what's in flight, close connections, then exit. §15.10
- **Gradual typing** — Adding static types to a codebase a little at a time; typed and untyped code coexist (TypeScript, JSDoc checking). §16.8
- **Graph** — A set of nodes connected by edges, possibly directed or weighted; maps, dependency lists and social networks are graphs. §17.2
- **Grapheme cluster** — What a reader sees as one character, possibly several code points (flags, family emoji). §6.5
- **Greedy algorithm** — Making the best-looking local choice at each step, which is correct only when you can prove those choices never hurt the final answer. §17.2
- **Greedy quantifier** — A regex repetition that matches as much as possible; add `?` to make it lazy. §6.7
- `groupBy` — `Object.groupBy` / `Map.groupBy` put items into buckets by a key function (ES2024). §8.7
- **Guard clause** — An early `return` (or `throw`) at the top of a function for invalid or special cases, so the main logic needn't be nested. §3.2

H

- **Hash map** — A structure that maps keys to values with average O(1) lookup by hashing the key; in JavaScript use `Map` or a plain object. §17.1
- **Hashbang** — A `#!/usr/bin/env node` first line that tells Unix shells which program runs the file; allowed by JavaScript since ES2023. §18.18
- `hasOwn` — `Object.hasOwn(obj, key)`: is this an own (not inherited) property? (ES2022) §5.2
- **Heap (data structure)** — A tree kept in an array so the smallest (min-heap) or largest (max-heap) item is always on top; push and pop are O(log n). §17.2
- **Heap (memory)** — The area of memory where objects live until the garbage collector frees them. §17.4
- **Higher-order function** — A function that takes or returns another function. §4.8
- **History API** — `pushState`, `replaceState` and `popstate` for changing the URL without reloading the page. §14.7
- **HMR** — Hot Module Replacement: a dev server swaps changed modules into the running page without a full reload. §10.11
- **Hoisting** — Declarations are registered before code runs: functions fully, `var` as `undefined`, `let`/`const`/`class` in the TDZ. §4.6

- **Hooks (testing)** — `before`, `beforeEach`, `afterEach` and `after`: functions a test runner calls around tests for setup and cleanup. §16.3
- **Host** — The environment around the engine that supplies extra APIs: a browser, Node.js, Deno, Bun. §1.4
- **HSTS** — HTTP Strict Transport Security: a response header telling the browser to use only HTTPS for this site from now on. §14.12
- **HTML** — HyperText Markup Language: the tag-based language that describes a web page's structure and content. §1.1
- **HTML Sanitizer API** — A newer browser API that cleans untrusted HTML of scripts and dangerous attributes, the built-in counterpart to DOMPurify. §14.10
- **HTTP** — HyperText Transfer Protocol: the request/response rules browsers and servers use to exchange data; HTTPS is HTTP encrypted with TLS. §14.1
- **HTTP header** — A `Name: value` line of metadata on a request or response, such as `Content-Type`, `Authorization` or `Set-Cookie`. §14.1
- **HTTP method** — The verb of a request: `GET` reads, `POST` creates, `PUT`/`PATCH` update, `DELETE` removes, `OPTIONS` asks what is allowed. §14.1
- **HTTP status code** — The three-digit number in a response: 2xx success, 3xx redirect, 4xx client error (404 not found), 5xx server error. §14.1
- `HttpOnly` — Cookie attribute that hides the cookie from JavaScript, protecting sessions from XSS theft. §14.4
- **HTTPS** — HTTP over an encrypted TLS connection, so nobody in between can read or change the traffic; required for `Secure` cookies. §14.12

I

- **Idempotent** — An operation that has the same effect whether it runs once or several times (e.g. "set status to paid"); only these are safe to retry blindly. §12.13
- **Identifier** — A name you give to a variable, function, class or parameter; letters, digits, `_` and `$`, not starting with a digit. §1.6
- **IEEE 754** — The 64-bit floating-point format behind every JavaScript `number`. §2.10
- **Ignition / Sparkplug / Maglev / TurboFan** — V8's tiers: Ignition interprets bytecode, Sparkplug and Maglev compile it quickly to machine code, TurboFan produces the most optimized code for hot functions. §1.3
- **IIFE** — Immediately Invoked Function Expression `(() => { … })()`: an old way to get a private scope. §4.9
- **Immutability** — Not changing data in place; producing new values instead (important in React and Redux). §7.4
- **Imperative code** — Code that spells out each step (counters, loops, pushes) to reach a result; contrast with declarative code. §7.5
- **Implicit binding** — The `this` rule for `obj.method()`: `this` is the object before the dot. §9.1
- **Implicit return** — An arrow function with an expression body returns that expression without `return`. §4.3
- `import` — Brings exported bindings from another module into this one. §10.2
- **Import attributes** — `with { type: 'json' }` on an import, telling the host how to load the module (ES2025). §10.6
- **Import map** — A browser `<script type="importmap">` that maps bare specifiers like `'lodash'` to URLs. §10.7
- `import.meta` — Per-module metadata such as `url`, `dirname`, `filename` and `resolve()`. §10.6
- **Impure function** — A function that has side effects or depends on outside state, so the same input may give a different result. §4.11
- **In-degree** — The number of edges pointing into a node of a directed graph; Kahn's topological sort starts from nodes with in-degree 0. §17.2
- **IndexedDB** — The browser's asynchronous, transactional database for large structured data. §14.4
- **Infinite loop** — A loop whose condition never becomes false, so it runs forever and freezes the page or process. §3.7

- **`Infinity`** — A special number bigger than any other, produced by `1 / 0` or overflow; `-Infinity` is its negative. §2.10
- **Inheritance** — One class extending another (`extends`) and reusing its behaviour through the prototype chain. §9.6
- **Inline style** — CSS set directly on one element via its `style` attribute or `el.style.x`; overrides most stylesheet rules. §13.4
- **`innerHTML`** — Gets or sets an element's children as an HTML string; setting it parses markup (XSS risk). §13.3
- **Inspector** — The debugging interface of an engine; `node --inspect` opens it so Chrome DevTools can attach to Node. §11.9
- **`instanceof`** — Tests whether `C.prototype` appears in an object's prototype chain. §9.7
- **Integer** — A whole number with no fractional part; JavaScript numbers hold integers exactly only up to $2^{53} - 1$. §2.10
- **Integration test** — A test that runs several real components together, such as a handler and a database. §16.1
- **Interface (TS)** — A named object type in TypeScript; can be extended and merged. §16.10
- **Interpolation** — Inserting values into a string, as with `${expr}` in a template literal. §6.3
- **Interpreter** — A program that runs code instruction by instruction without first compiling all of it to machine code; engines start this way and compile hot code later. §1.3
- **Intersection type (TS)** — `A & B`: a value that has all the properties of both types. §16.10
- **IntersectionObserver** — Reports when an element enters or leaves the viewport; used for lazy loading and infinite scroll. §14.9
- **`Intl`** — The internationalization namespace: `DateTimeFormat`, `NumberFormat`, `Collator`, `Segmenter` and more. §5.11
- **`Intl.NumberFormat`** — Formats numbers for people in a given locale, including currency and percentages (`€1,234.50`). §2.10
- **Invalid Date** — A `Date` whose time value is `NaN` because parsing failed; `isNaN(d)` or `Number.isNaN(d.getTime())` detects it. §5.11
- **Inversion of control** — Handing your callback to someone else's code and trusting it to call it once, at the right time; promises reduce that trust problem. §12.6
- **ISO 8601** — The international date-time text format `YYYY-MM-DDTHH:mm:ss.sssZ`; the safe format to send and parse. §5.11
- **Iterable** — An object with a `[Symbol.iterator]()` method, usable in `for…of` and spread. §8.4
- **Iterator** — An object with a `next()` method returning `{ value, done }`. §8.4
- **Iterator helpers** — Lazy `map`, `filter`, `take`, `drop`, `flatMap` and consuming `toArray`, `reduce` on iterators (ES2025). §8.6

J

- **JavaScriptCore** — Apple's JavaScript engine, used by Safari and Bun. §1.3
- **Jest** — A popular all-in-one JavaScript test runner with `expect` assertions, mocking and snapshots. §16.7
- **JIT compilation** — Just-in-time compilation: the engine compiles frequently run code to machine code while the program runs. §1.3
- **Jitter** — Random extra delay added to retries so many clients don't all retry at the same instant. §12.13
- **JSDoc** — Structured comments (`/** @param {number} x */`) that document code and can drive TypeScript checking. §16.15
- **jsdom** — A JavaScript implementation of the DOM that runs in Node, used to test browser code without a browser. §13.3
- **JSON** — A text data format with double-quoted keys and strings, numbers, booleans, `null`, arrays and objects. §5.12
- **JSON module** — A `.json` file imported directly with `with { type: 'json' }` (ES2025). §10.6
- **`JSON.parse`** — Turns JSON text into values; strict syntax; an optional reviver transforms values. §5.12

- **`JSON.stringify`** — Turns values into JSON text; drops functions and `undefined` in objects, throws on BigInt and cycles. §5.12
- **JSX** — HTML-like syntax inside JavaScript used by React (`<Button />`); a build tool compiles it into function calls. §10.11
- **JWT** — JSON Web Token: a signed, Base64-encoded JSON token often used to prove who a user is; its timestamps are in seconds, not milliseconds. §5.11

K

- **Kahn's algorithm** — Topological sort by repeatedly taking nodes with no remaining incoming edges (in-degree 0) from a queue. §17.2

L

- **Label** — A name before a loop (`outer:`) that `break outer` / `continue outer` can target. §3.11
- **`lastIndex`** — The position where a `g` or `y` regex will start its next search; it makes repeated `test` calls stateful. §6.8
- **Last-request-wins** — Ignoring or cancelling responses from older requests so only the latest one updates the UI (e.g. in search-as-you-type). §12.13
- **Layout thrashing** — Alternating DOM writes and layout reads so the browser recalculates layout again and again. §13.11
- **Lazy evaluation** — Computing values only when they're requested, as generators and iterator helpers do. §8.5
- **Lazy quantifier** — A regex repetition with a trailing `?` (`.+?`) that matches as little as possible. §6.7
- **`let`** — Declares a block-scoped variable that can be reassigned. §2.1
- **Lexical scope** — Scope decided by where code is written, not where it's called from. §4.5
- **Lexical `this`** — An arrow function's `this` is the `this` of the surrounding code. §4.3
- **Library** — Reusable code you call from your own program, such as Lodash or date-fns; contrast with a framework, which calls your code. §1.1
- **libuv** — The C library that gives Node its event loop, async I/O and thread pool. §15.1
- **Linked list** — A chain of nodes where each holds a value and a reference (`next`) to the following node; O(1) insert at a known spot, O(n) lookup. §17.2
- **Linter** — A tool that analyses source code for bugs and style problems without running it (ESLint). §10.13
- **Literal** — A value written directly in code: `42`, `'hi'`, `true`, `[1, 2]`, `{ a: 1 }`, `/re/`. §1.6
- **Literal type (TS)** — A type that allows only one exact value, such as `'admin'` or `42`; unions of them make safe string options. §16.10
- **Live binding** — An ES module import stays connected to the exporter's variable and sees updates. §10.3
- **Live collection** — A DOM collection (`getElementsByTagName`) that updates automatically when the document changes. §13.2
- **`load` event** — Fires when the page and all its images, stylesheets and frames have finished loading, after `DOMContentLoaded`. §13.12
- **Local time** — A date and time in the user's own time zone, as `getHours()` and `toString()` show it; contrast with UTC. §5.11
- **`localeCompare`** — Compares two strings by a language's alphabet rules, returning a negative, zero or positive number; use it to sort text. §6.6
- **Localhost (`127.0.0.1`)** — The address that always means "this same computer"; servers bound to it can't be reached from other machines. §15.10
- **`localStorage`** — Per-origin string key-value storage that survives reloads; synchronous, about 5 MB. §14.3
- **Lockfile** — `package-lock.json`: records the exact installed version of every dependency. §10.10
- **Logical assignment** — `a ||= b`, `a &&= b`, `a ??= b`: assign only when the condition holds (ES2021). §2.7

- **Logpoint** — A breakpoint that logs a message instead of pausing, so you can add logging without editing code. §11.9
- **Lone surrogate** — Half of a surrogate pair with no partner, which makes a string not well-formed; `isWellFormed` detects it. §6.5
- **Lookahead / lookbehind** — Regex assertions `(?=…)`, `(?!…)`, `(?<=…)`, `(?<!…)` that check surrounding text without consuming it. §6.7
- **LRU cache** — A fixed-size cache that throws out the Least Recently Used entry when full; a `Map` (which keeps insertion order) can implement it. §17.3
- **LTS** — Long-Term Support: a Node.js release line (the even-numbered ones) that gets fixes for about 30 months; use one in production. §15.1

M

- **Macrotask** — Another name for a task in the event loop (timers, I/O, events). §12.2
- **Main thread** — The single thread that runs your JavaScript, and in a browser also handles rendering and input; keep it free of long work. §12.1
- **Major / minor / patch** — The three parts of a semantic version `MAJOR.MINOR.PATCH`: breaking changes, new features that don't break anything, bug fixes. §10.10
- `Map` — A key-value collection with keys of any type, insertion order and a `size`. §8.1
- **Mapped types (TS)** — Types built by transforming every property of another type, e.g. `{ [K in keyof T]?: T[K] }` (how `Partial` works). §16.12
- **Mark-and-sweep** — Garbage collection by marking everything reachable from the roots, then freeing everything unmarked. §17.4
- `Math` — A namespace object with maths constants and functions (`round`, `max`, `random`). §5.10
- **MDN Web Docs** — Mozilla's free reference for JavaScript and web APIs, with browser-compatibility tables. §18.19
- **Memoization** — Caching a function's results by its arguments so repeated calls are instant. §4.7
- **Memory leak** — Memory that stays reachable by mistake (forgotten listeners, timers, growing caches, detached DOM nodes). §17.4.7
- **Merge sort** — Sorting by splitting the array in halves, sorting each recursively, and merging them; O(n log n) and stable. §17.2
- **Message passing** — Communicating between threads by sending copies of data (`postMessage`) instead of sharing variables. §14.8
- **Method** — A function stored as an object property and called through the object (`user.greet()`), where `this` is that object. §5.3
- **Method borrowing** — Calling one object's method on another object with `call`/`apply`, e.g. `Array.prototype.slice.call(arguments)`. §9.2
- **Method shorthand** — Writing `greet() { … }` inside an object literal instead of `greet: function () { … }`. §5.3
- **Microtask** — A job that runs right after the current task finishes: promise reactions, code after `await`, `queueMicrotask`. §12.2
- **Microtask queue** — The queue for promise callbacks, code after `await` and `queueMicrotask`; it is emptied completely after every task. §12.2
- **Microtask starvation** — When microtasks keep scheduling more microtasks, so the event loop never gets to timers, input or rendering. §12.2
- **Middleware** — A function in a web framework's request pipeline that can inspect or change the request and response, or pass control to the next one (logging, auth, parsing). §15.10
- **Minification** — Removing whitespace and shortening names so bundles download faster. §10.11
- **Mixin** — A reusable set of methods added to unrelated classes, by copying or by a subclass factory. §9.8
- **Mock** — A test double that records calls so the test can verify how it was used. §16.5
- **Module** — A file with its own scope that exports and imports bindings. §10.1
- **Module graph** — All the modules an app loads, linked by their imports, starting from the entry point. §10.3

- **Module pattern** — An IIFE that returns an object of public functions while keeping private state in a closure. §4.9
- **Module scope** — The private top-level scope of an ES module; its names are invisible to other files unless exported. §4.5
- **Module specifier** — The string after `from` in an import: relative (`'./a.js'`), absolute URL, or bare (`'lodash'`). §10.2
- **Monotonic stack** — A stack kept in increasing or decreasing order, used for "next greater element" style problems in O(n). §17.2
- **Multi-paradigm** — A language that supports several styles of programming; JavaScript allows procedural, object-oriented and functional code. §1.1
- **Mutating method** — An array method that changes the original array (`push`, `splice`, `sort`, `reverse`). §7.4
- **Mutation** — Changing an existing object or array in place (`o.n = 2`, `arr.push(x)`); everyone holding a reference sees the change. §2.4
- **MutationObserver** — Calls back (as a microtask) with a batch of changes when the observed DOM changes. §13.10

N

- **Named capture group** — `(?<name>…)` in a regex; its match appears as `match.groups.name`. §6.7
- **Named export** — An export referenced by its name: `export const x`, imported with `import { x }`. §10.2
- **Named function expression** — A function expression with its own name (`const f = function fact(n) {…}`); the name is visible inside for recursion and in stack traces. §4.2
- **Namespace import** — `import * as ns from './m.mjs'`: one object holding all of a module's exports. §10.2
- `NaN` — "Not a Number": a `number` value from failed maths; not equal to anything, including itself. §2.10
- **Narrowing (TS)** — Refining a broad type to a specific one via checks like `typeof`, `in` or a discriminant. §16.11
- **Navigation API** — A newer browser API (`navigation`) for single-page-app routing that intercepts all navigations, a cleaner successor to the History API. §14.7
- **NDJSON** — Newline-delimited JSON: one JSON value per line, handy for logs and streaming because each line parses on its own. §5.12
- `never` **(TS)** — The type with no values, used for code that can't be reached; assigning to it makes `switch` checks exhaustive. §16.11
- `new` — Creates an object linked to `Fn.prototype`, runs the constructor with it as `this` and returns it. §9.4
- `new` **binding** — The `this` rule for `new Fn()`: `this` is the brand-new object; it has the highest priority. §9.1
- `new.target` — Inside a function, tells whether it was called with `new` (and which constructor). §9.4
- **Node (DOM)** — Any item in the DOM tree: elements, text nodes, comments and the document itself. §13.1
- `node:` **prefix** — Writing built-in Node modules as `node:fs` or `node:path`, which makes it clear they aren't npm packages. §15.3
- `node_modules` — The folder where npm installs a project's packages; never edit or commit it. §10.9
- `node:test` — Node's built-in test runner with suites, hooks, mocks, snapshots and coverage. §16.3
- **Node.js** — A JavaScript runtime built from V8 and libuv, for servers, CLIs and tools. §15.1
- **NodeList** — A list of DOM nodes; `querySelectorAll` returns a static one that has `forEach`. §13.2
- **Non-blocking I/O** — Starting a disk or network operation and continuing with other work, getting a callback or promise when it finishes. §12.1
- **Non-capturing group** — `(?:…)`: groups part of a regex without remembering the match. §6.7
- **Nonce (CSP)** — A random value generated for each response and placed in both the CSP header and allowed `<script>` tags, so only those inline scripts run. §14.12
- **Normalization (Unicode)** — Converting text to a standard form (NFC) so visually identical strings, like a precomposed é and e plus an accent, compare equal. §6.5
- **npm** — Node's package manager and public registry. §10.9

- **`npm ci`** — Clean, exact install from the lockfile; the right command for CI and deploys. §10.10
- **`npm scripts`** — Commands named under `"scripts"` in `package.json` and run with `npm run name` or `node --run name`. §15.11
- **`npx`** — Runs a command from an npm package, using the local install or downloading it temporarily. §10.9
- **`null`** — A primitive meaning "deliberately empty"; `typeof null` is `'object'`. §2.3
- **Nullish** — The two "no value" values: `null` and `undefined`. §2.7
- **Nullish coalescing** — `a ?? b`: `b` only when `a` is `null` or `undefined` (ES2020). §2.7
- **Null-prototype object** — An object made with `Object.create(null)` that inherits nothing, so keys like `toString` or `__proto__` are safe to use as data. §8.7
- **`Number.EPSILON`** — The gap between 1 and the next double, about 2.2e-16; a tolerance for float comparisons. §2.10
- **`Number.MAX_SAFE_INTEGER`** — $2^{53} - 1$, the largest integer that numbers represent exactly. §2.10
- **Numeric separator** — Underscores inside number literals for readability: `1_000_000` (ES2021). §2.10

O

- **O(1), O(n), O(log n)** — Big-O labels for constant time, time growing in proportion to input size, and time growing with the logarithm of input size. §17.1
- **Object** — A collection of key–value properties; arrays, functions, dates and maps are all objects, and they are passed around by reference. §2.3
- **Object literal** — Creating an object directly with braces: `{ name: 'Ada', age: 36 }`. §5.1
- **`Object.create`** — Makes a new object with a chosen prototype, or none: `Object.create(null)`. §9.3
- **`Object.entries`** — Returns an object's own enumerable `[key, value]` pairs. §5.6
- **`Object.freeze`** — Makes an object's own properties read-only and blocks new ones; shallow. §5.6
- **`Object.is`** — Like `===` except `Object.is(NaN, NaN)` is true and `Object.is(0, -0)` is false. §2.6
- **Observer pattern** — A design where an object keeps a list of listeners and notifies them when something happens; `EventEmitter` and DOM events follow it (also called publish/subscribe). §15.8
- **Off-by-one error** — A loop or index that runs one step too many or too few, often `<=` used where `<` was meant. §3.6
- **OLOO** — "Objects linked to other objects": a style that skips classes and links plain objects with `Object.create`. §9.9
- **Operand** — A value an operator works on; in `a + b`, `a` and `b` are the operands. §2.5
- **Operator** — A symbol or keyword that computes a value from operands, such as `+`, `===`, `&&`, `typeof` or `?:`. §2.5
- **Optional catch binding** — `try { } catch { }` without naming the error (ES2019). §11.2
- **Optional chaining** — `a?.b`, `a?.[k]`, `a?.()`: stop and return `undefined` if the left side is `null`/`undefined`. §5.8
- **Options object** — Passing one object of named settings instead of many positional arguments: `connect({ host, port })`. §4.4
- **Origin** — The scheme, host and port of a URL; the unit of the same-origin policy. §14.2
- **Override** — Defining a method in a subclass with the same name as one in its parent so it replaces it; `super.method()` still reaches the parent's. §9.6
- **Own property** — A property stored directly on an object rather than inherited. §9.3

P

- **Package** — A folder of code with a `package.json`, published to a registry such as npm so others can install it. §10.9
- **`package.json`** — A project's manifest: name, version, scripts, dependencies, `type` and entry points. §10.9
- **`padStart` / `padEnd`** — Pad a string to a length with a fill string (`'5'.padStart(3, '0')` → `'005'`). §6.6
- **Parameter** — A name in a function's definition that receives a value (the argument) when the function is called. §4.1

- **Parameter properties (TS)** — TypeScript shorthand `constructor(private name: string)` that declares and assigns a field; not erasable. §16.9
- `parseInt` — Reads an integer from the start of a string, stopping at the first invalid character; always pass the radix (`parseInt(s, 10)`). §2.10
- **Partial application** — Fixing some of a function's arguments now and supplying the rest later. §4.8
- **PascalCase** — Naming style where every word starts with a capital (`ShoppingCart`); the convention for classes and constructors. §2.1
- **Passive listener** — A listener registered with `{ passive: true }`, promising not to call `preventDefault()`, so scrolling stays smooth. §13.8
- `path` **module** — Node's utilities for joining, resolving and splitting file paths. §15.5
- **Path traversal** — An attack using `../` in a user-supplied path to reach files outside an allowed folder; resolve the path and check its prefix. §15.5
- **Peer dependency** — A package a library expects the host app to install itself (e.g. a React plugin needs `react`), so only one copy exists. §10.9
- **Pending** — The starting state of a promise, before it is fulfilled or rejected. §12.7
- **Percent-encoding** — Writing unsafe URL characters as `%` plus hex bytes (`' '` → `%20`, `&` → `%26`); `URLSearchParams` and `encodeURIComponent` do it. §14.6
- **Permission model** — Node's `--permission` flag that denies file-system reads and writes, child processes, worker threads and more unless each is allowed (`--allow-fs-read=…`). §15.12
- **Phantom dependency** — A package your code imports that isn't in your own `package.json` and only works because some other package pulled it in. §10.9
- `pipeline` — Connects streams, handles backpressure and errors, and returns a promise (from `node:stream/promises`). §15.9
- **Playwright** — A tool from Microsoft for end-to-end testing that drives real Chromium, Firefox and WebKit browsers. §16.1
- **Poll phase** — The Node event-loop phase that waits for and runs I/O callbacks. §12.4
- **Polyfill** — Code that implements a missing built-in API for older environments. §10.12
- **Polymorphism** — Different classes offering the same method name so calling code can treat them alike (`shape.area()`), each doing its own thing. §9.6
- `popstate` — Event fired when the user moves back or forward through history entries. §14.7
- **Precedence** — The order in which operators are evaluated when parentheses don't decide it. §2.9
- **Pre-commit hook** — A script git runs before each commit (often lint and format) that can block a bad commit. §10.13
- **Prefix / postfix increment** — `++n` adds one and returns the new value; `n++` adds one but returns the old value. §2.5
- **Prefix sum** — An array where each entry is the total of all earlier values, letting you answer any range-sum question in O(1). §17.2
- **Preflight** — The `OPTIONS` request a browser sends before a non-simple cross-origin request to ask permission. §14.2
- **Prettier** — An opinionated code formatter. §10.13
- `preventDefault` — Cancels the browser's default action for an event, such as following a link or submitting a form. §13.8
- **Primitive** — An immutable value that isn't an object: number, bigint, string, boolean, undefined, null, symbol. §2.4
- **Priority queue** — A queue that always hands out the highest-priority (smallest or largest) item next; usually built on a heap. §17.1
- **Private field** — A `#name` class member only reachable inside the class body, enforced at runtime (ES2022). §9.5
- `process` — Node's global object for arguments, environment variables, exit codes and signals. §15.6
- `process.nextTick` — Node queue that runs before promise microtasks after the current operation. §12.4
- **Promise** — An object representing a value that will be available later; pending, then fulfilled or rejected. §12.7
- `Promise.all` — Waits for all promises; values in input order; rejects on the first rejection. §12.8

- **`Promise.allSettled`** — Waits for every promise and reports each outcome; never rejects. §12.8
- **`Promise.any`** — Fulfils with the first fulfilment; rejects with `AggregateError` if all reject. §12.8
- **`Promise.race`** — Settles like whichever input settles first. §12.8
- **Promisify** — Converting a callback-style function into one that returns a promise (`util.promisify`). §12.7
- **Property (DOM)** — The live JavaScript value on an element object (`input.value`), as opposed to the HTML attribute. §13.4
- **Property descriptor** — The settings of a property: `value`, `writable`, `enumerable`, `configurable`, or `get`/`set`. §5.6
- **Proposal (TC39)** — A suggested new language feature that moves through Stages 0–4, guided by a "champion" on the committee. §1.2
- **Prototype** — The object another object delegates missing property reads to, via its `[[Prototype]]` link. §9.3
- **Prototype chain** — The linked series of prototypes searched when a property isn't found on the object itself. §9.3
- **Prototype pollution** — An attack that writes to `Object.prototype` (e.g. via a `__proto__` key in merged JSON), affecting every object. §9.3
- **Proxy** — An object that wraps another and intercepts operations like get and set. §5.9
- **Pure function** — A function whose output depends only on its inputs and which has no side effects. §4.11

Q

- **`querySelector`** — Returns the first element matching a CSS selector, or `null`. §13.2
- **Queue** — A first-in, first-out (FIFO) list: add at the back, remove from the front. §7.3
- **`queueMicrotask`** — Schedules a function as a microtask. §12.5
- **Quickselect** — Finding the k-th smallest item in average O(n) by partitioning around a pivot and recursing into one side only. §17.2

R

- **Race condition** — A bug where the result depends on which of several async operations finishes first. §12.13
- **Radians** — The unit `Math.sin`, `cos` and friends use for angles: 180° is π radians. §5.10
- **Radix** — The base of a number system: 10 for decimal, 16 for hex, 2 for binary; the second argument of `parseInt` and `toString`. §2.10
- **`RangeError`** — Thrown when a number is outside an allowed range: `new Array(-1)`, `toFixed(101)`, or too-deep recursion. §11.1
- **Reachability** — An object is reachable if some chain of references leads to it from a root (globals, the stack); unreachable objects are garbage-collected. §17.4
- **React** — A popular JavaScript library for building user interfaces from components; it relies heavily on immutable updates and closures. §2.4
- **Realm** — A separate global environment (an iframe, a `vm` context) with its own copies of built-ins. §9.7
- **Recursion** — A function solving a problem by calling itself on a smaller version of it. §4.10
- **ReDoS** — Regular-expression denial of service: a badly written pattern that takes exponential time on crafted input and freezes the thread. §6.9
- **`reduce`** — Folds an array into one value by running an accumulator function; pass an initial value. §7.5
- **Redux** — A state-management library whose reducers must be pure functions that return new state instead of mutating the old one. §4.11
- **Reference** — The address an object variable holds; copying the variable copies the address, so both names point at the same object. §2.4
- **`ReferenceError`** — Thrown when you use a name that doesn't exist, or a `let`/`const` before its declaration (TDZ). §11.1

- **`Referrer-Policy` / `Permissions-Policy`** — Security headers: the first limits how much of the current URL is sent to other sites, the second turns off browser features (camera, geolocation) a page doesn't need. §14.12
- **Reflect** — A built-in object of methods mirroring basic object operations (`Reflect.get`, `Reflect.ownKeys`); handy inside `Proxy` handlers. §5.9
- **Reflected XSS** — XSS where the attack script comes from the request (e.g. a URL parameter) and the server echoes it straight back into the page. §14.10
- **Reflow** — The browser recalculating element positions and sizes (layout) after DOM or style changes. §13.11
- **`RegExp.escape`** — Escapes special regex characters so a string can be matched literally (ES2025). §6.9
- **Registry (npm)** — The online database (registry.npmjs.org) that npm downloads packages from and publishes them to. §10.9
- **Regression** — A bug in something that used to work; tests guard against regressions when code changes. §16.1
- **Regular expression** — A pattern for matching text, written `/pattern/flags` or with `new RegExp`. §6.7
- **Rejection** — A promise settling with an error (reason) instead of a value. §12.7
- **Remainder operator** — `%`: the remainder of division, with the sign of the left operand. §2.5
- **Rendering (event loop)** — The step where the browser runs `requestAnimationFrame` callbacks, then style, layout and paint. §12.2
- **Repaint** — Redrawing pixels after a visual change that doesn't affect layout. §13.11
- **REPL** — Read–Eval–Print Loop: an interactive prompt that runs each line you type and prints the result (`node` with no file, the browser console). §1.5
- **`replaceAll`** — Replaces every occurrence of a string or global regex (ES2021). §6.6
- **Replacer** — The optional second argument of `JSON.stringify`: a function or key list that changes or filters what is written. §5.12
- **Request / response** — The two halves of an HTTP exchange: the client sends a request (method, URL, headers, body); the server answers with a response (status, headers, body). §14.1
- **`requestAnimationFrame`** — Runs a callback right before the next paint; the right tool for animations. §14.5
- **ResizeObserver** — Reports when an element's size changes. §14.9
- **Rest parameter** — `...args` as the last parameter: collects remaining arguments into a real array. §4.4
- **Return value** — The value a function hands back with `return`; without one, a call gives `undefined`. §4.1
- **Reviver** — A function passed to `JSON.parse` to transform each parsed value. §5.12
- **Runtime** — The environment that runs your code (Node, a browser, Deno, Bun); also "at runtime" means while the program is running, as opposed to while compiling. §1.3
- **Run-to-completion** — Once a task starts, it runs until the call stack is empty; nothing interrupts it. §12.1

S

- **Same-origin policy** — Browser rule that a page may only read responses from its own origin unless CORS allows it. §14.2
- **Same-origin policy (SOP)** — The browser rule that a page may read responses only from its own origin unless the other server allows it with CORS. §14.2
- **`SameSite`** — Cookie attribute (`Strict`, `Lax`, `None`) controlling whether it's sent on cross-site requests. §14.11
- **SameValueZero** — The equality `Map`, `Set` and `includes` use: like `===`, except `NaN` equals `NaN`. §8.1
- **`satisfies`** — TypeScript operator that checks a value against a type without widening the value's own type. §16.13
- **Scope** — The region of code where a name is visible: global, module, function or block. §4.5
- **Scope chain** — The nested scopes searched outward when a variable is looked up. §4.5
- **Seal** — `Object.seal`: no properties can be added or removed, but values can change. §5.6

- **Semantic versioning** — `MAJOR.MINOR.PATCH`: breaking changes, new features, bug fixes. §10.10
- **Serialization** — Turning an in-memory value into a format that can be stored or sent (such as a JSON string); deserialization turns it back. §5.2
- **Service worker** — A background script that acts as a network proxy for offline support, caching and push. §14.8
- **Session cookie** — A cookie with no expiry date that disappears when the browser session ends; also loosely, the cookie holding a login session ID. §14.4
- `sessionStorage` — Like `localStorage` but scoped to one tab and cleared when it closes. §14.3
- `Set` — A collection of unique values in insertion order with O(1) average `has`. §8.2
- **Set methods** — `union`, `intersection`, `difference`, `symmetricDifference` and subset tests on Sets (ES2025). §8.2
- `setImmediate` — Node timer that runs in the check phase, after I/O callbacks. §12.4
- `setInterval` — Runs a function repeatedly roughly every N milliseconds until cleared. §12.5
- **Setter** — A `set` method that runs when a property is assigned. §5.7
- `setTimeout` — Runs a function once after at least N milliseconds, as a new task. §12.5
- **Settled** — A promise that is no longer pending: it has been either fulfilled or rejected, and never changes again. §12.7
- **Shadowing** — An inner variable or own property hiding one with the same name further out. §4.5
- **Shallow copy** — A copy of the top level only; nested objects are still shared. §5.4
- **SharedArrayBuffer / Atomics** — Memory that several workers can read and write at once, plus `Atomics` operations to update it safely without races. §15.2
- **Short-circuit evaluation** — `&&`, `||`, `??` and `?.` stop evaluating as soon as the result is known. §2.7
- **Side effect** — Anything a function does beyond returning a value: mutating inputs, I/O, logging, changing globals. §4.11
- `SIGTERM` / `SIGINT` — OS signals asking a process to stop: `SIGTERM` comes from tools such as Docker or `kill`, `SIGINT` from pressing Ctrl+C. §15.6
- **Simple request (CORS)** — A cross-origin `GET`, `HEAD` or `POST` with only basic headers and content types, sent without a preflight. §14.2
- **Single-page application (SPA)** — A web app that loads one page and updates the view with JavaScript, changing the URL with the History API instead of full page loads. §14.7
- **Single-threaded** — JavaScript runs one piece of your code at a time on one thread; waiting is handed to the host. §12.1
- **Singleton** — Something that exists only once; every importer of an ES module shares the same single instance of it. §10.3
- **Sink (XSS)** — An API that turns a string into HTML or code (`innerHTML`, `eval`, `document.write`); untrusted data must never reach it unescaped. §14.10
- **Sliding window** — Keeping a moving range `[left, right]` over an array or string and adjusting its ends, to solve contiguous-range problems in O(n). §17.2
- **Sloppy mode** — The default, non-strict mode of classic scripts, which silently allows mistakes strict mode reports. §1.8
- **Snapshot test** — A test that saves a serialized output and fails when it changes. §16.6
- **Soundness (TS)** — Whether a type system guarantees values really match their types; TypeScript is deliberately not fully sound (`any`, casts). §16.8
- **Source map** — A file mapping minified or transpiled code back to the original source for debugging. §11.8
- **Sparse array** — An array with holes, indexes that were never set, as made by `new Array(3)` or `delete arr[i]`. §7.3
- **SpiderMonkey** — Firefox's JavaScript engine, the first one ever written. §1.3
- `splice` — Mutating array method that removes and/or inserts items at an index and returns the removed items. §7.4
- **Spread syntax** — `...` expanding an iterable into elements or arguments, or an object's properties into another object. §5.4

- **Spy** — A test double that records its calls (`t.mock.fn()`). §16.5
- **Stable sort** — A sort that keeps items with equal keys in their original order; `Array.prototype.sort` has been stable since ES2019. §7.6
- **Stack** — A last-in, first-out (LIFO) list: `push` and `pop` at the same end. §7.3
- **Stack overflow** — The error when too many nested calls fill the call stack, usually recursion without a reachable base case: `RangeError: Maximum call stack size exceeded`. §4.10
- **Stack trace** — The list of function calls active when an error was created, newest first. §11.8
- **Stage (TC39)** — A proposal's maturity from 0 (idea) to 4 (finished, goes into the next edition). §1.2
- **Stale closure** — A callback still seeing old values because it closed over variables from an earlier render or call. §4.7
- **Statement** — A complete instruction such as a declaration, an `if` or a loop. §1.6
- **Static initialization block** — `static { … }` inside a class: code that runs once when the class is defined (ES2022). §9.5
- **Static member** — A method or field on the class itself, not on instances: `static create()`. §9.5
- **Static typing** — Checking types before the program runs, from annotations and inference; TypeScript adds it to JavaScript. §16.8
- **`stdout` / `stderr`** — A process's two output streams: normal output and error/diagnostic output; `console.log` writes to one, `console.error` to the other. §15.6
- **Step over / into / out** — Debugger controls: run the next line, enter the function being called, or run until the current function returns. §11.9
- **Sticky flag (`y`)** — A regex flag that only matches starting exactly at `lastIndex`, useful for writing tokenizers. §6.7
- **`stopPropagation`** — Stops an event from travelling to further elements. §13.6
- **Stored XSS** — XSS where the attacker's script is saved on the server (a comment, a profile name) and served to every visitor. §14.10
- **Stream** — Node's abstraction for data processed in chunks: Readable, Writable, Duplex, Transform. §15.9
- **Strict equality** — `===`: same type and value, no conversion. §2.6
- **Strict mode** — A restricted mode that turns silent mistakes into errors; automatic in modules and classes. §1.8
- **String** — An immutable sequence of UTF-16 code units. §6.1
- **`String.raw`** — A tag for template literals that keeps backslashes as written, handy for regex sources and Windows paths. §6.3
- **Structural typing** — TypeScript's rule that types are compatible if their shapes match, whatever their names. §16.8
- **Structured clone** — The algorithm that deep-copies values for `structuredClone`, `postMessage` and IndexedDB; handles Dates, Maps and cycles but not functions. §14.8
- **`structuredClone`** — Built-in deep copy that handles Dates, Maps, Sets and cycles, but not functions. §5.4
- **Stub** — A test double that returns canned answers. §16.5
- **`submit` event** — Fires on a `<form>` when it is submitted; listen for it (not button clicks) and call `preventDefault()` to handle it in JavaScript. §13.9
- **Subresource Integrity** — An `integrity` hash on a `<script>` so the browser refuses a tampered CDN file. §14.12
- **Subtest** — A test nested inside another test with `t.test()`, reported as part of its parent. §16.3
- **`super`** — Calls the parent class's constructor (`super()`) or methods (`super.m()`). §9.6
- **Supply-chain attack** — Malicious code slipped into a package you depend on (or its dependencies) so it runs in your app. §10.10
- **SuppressedError** — The error thrown when cleanup with `using` fails while another error is already in flight; it holds both. §11.6
- **Surrogate pair** — Two UTF-16 code units that together encode one character outside the Basic Multilingual Plane, like most emoji. §6.5

- **SWC** — A very fast transpiler written in Rust, used by Next.js and others to strip types and transform syntax. §10.12
- `switch` — Compares one value against `case` values with `===`; falls through without `break`. §3.4
- **Symbol** — A primitive creating unique identifiers, often used as property keys. §2.3
- `Symbol.dispose` — The well-known symbol naming the cleanup method that a `using` declaration calls automatically. §11.6
- `Symbol.iterator` — The well-known symbol naming the method that makes an object iterable. §8.4
- **Synchronous** — Code that runs step by step and blocks until each step finishes. §12.1
- **Syntax** — The grammar rules for how code must be written; breaking them gives a `SyntaxError` before anything runs. §1.6
- `SyntaxError` — Thrown when code or text can't be parsed: malformed source, bad `JSON.parse` input, an invalid regex. §11.1

T

- **Table-driven test** — One test body run over a table of inputs and expected outputs, so new cases are a single line. §16.3
- **Tagged template** — A function called with a template literal's pieces: `tag`Hi ${name}``. §6.3
- **Task** — A unit of work in the event loop: a script, timer callback, I/O callback or UI event. §12.2
- **TC39** — The Ecma committee that designs and maintains ECMAScript. §1.2
- **TCP** — Transmission Control Protocol: the reliable, ordered byte-stream connection that HTTP runs on top of. §15.2
- **Template literal** — A backtick string with `${}` interpolation and real line breaks. §6.3
- **Temporal** — The modern date-time API (Stage 4 in March 2026, planned for ES2027); not in Node 24. §5.11
- **Temporal Dead Zone (TDZ)** — The time between entering a scope and a `let`/`const`/`class` declaration running; access throws. §2.2
- **Terminal** — A text window where you type commands (the command line), such as `node app.js` or `npm install`. §1.5
- **Ternary operator** — `cond ? a : b`: chooses between two values. §3.3
- **Test double** — Any stand-in for a real dependency in a test: dummy, stub, spy, mock or fake. §16.5
- **Test pyramid** — The guideline to have many fast unit tests, fewer integration tests and only a few slow end-to-end tests. §16.1
- **Test runner** — The tool that finds tests, runs them and reports passes and failures: `node --test`, Vitest, Jest. §16.1
- **Test suite** — A group of related tests, or all of a project's tests together. §16.1
- **Test-driven development (TDD)** — Writing a failing test first, then the code that makes it pass, then tidying up (red, green, refactor). §16.7
- **Testing trophy** — An alternative to the test pyramid that puts most effort into integration tests, on top of static checks. §16.1
- **Text node** — A DOM node holding plain text inside an element, including the whitespace between tags. §13.1
- `textContent` — All text inside a node; setting it inserts plain text, never HTML. §13.3
- `then` — Registers callbacks on a promise and returns a new promise. §12.7
- **Thenable** — Any object with a `then` method; `await` and promise resolution treat it like a promise. §12.7
- `this` — The context object of a function call, set by how the function is called. §9.1
- **Thread** — An independent line of execution the CPU can run; JavaScript code runs on one, while the host may use others behind the scenes. §12.1
- **Thread pool** — libuv's worker threads (4 by default) that run file-system, DNS lookup, crypto and zlib jobs for Node. §15.2
- **Throttle** — Run a function at most once per interval during a burst of calls (e.g. scroll handlers). §17.3
- `throw` — Raises an exception, stopping normal flow until a `catch` handles it. §11.2

- **Tilde range (`~`)** — An npm version range that allows only patch updates (`~1.2.3` means at least 1.2.3, below 1.3.0). §10.10
- **Timers phase** — The Node event-loop phase that runs expired `setTimeout` and `setInterval` callbacks. §12.4
- **TOCTOU (check-then-act)** — "Time of check to time of use": a race where something changes between checking (does the file exist?) and acting on it; just try the action and handle the error. §15.4
- `toJSON` — A method `JSON.stringify` calls if present, letting an object choose its own JSON form (`Date` uses it to give an ISO string). §5.12
- **Top-level `await`** — Using `await` at the top level of an ES module; importers wait for it (ES2022). §10.5
- **Topological sort** — Ordering the nodes of a directed graph so every node comes after everything it depends on; impossible if there is a cycle. §17.2
- `toSorted` / `toReversed` / `toSpliced` / `with` — Non-mutating copies of `sort`, `reverse`, `splice` and index assignment (ES2023). §7.4
- **Transform stream** — A duplex stream that changes data as it passes through, like gzip compression. §15.9
- **Transitive dependency** — A package you don't install directly but get because one of your dependencies (or theirs) needs it. §10.9
- **Transpiler** — A tool that rewrites source code into another syntax (TypeScript or new JavaScript → older JavaScript). §10.12
- **Tree shaking** — Removing unused exports from a bundle, made possible by static ES module syntax. §10.11
- **Trie** — A tree of characters where each path from the root spells a prefix; fast for "starts with" lookups. §17.2
- **Trusted Types** — A browser feature that makes dangerous sinks like `innerHTML` accept only values produced by approved sanitizing policies. §14.10
- **Truthy** — Any value that is not falsy, including `[]`, `{}` and `'0'`. §2.11
- `try…catch` — Runs code and handles exceptions thrown synchronously inside it. §11.2
- `tsc` — The TypeScript compiler: it type-checks code and, unless given `--noEmit`, writes JavaScript output. §16.8
- `tsconfig.json` — The configuration file for a TypeScript project: which files to check and with which compiler options (e.g. `strict`). §16.8
- `tsx` — A popular command-line tool that runs TypeScript files directly in Node using esbuild. §16.9
- **Tuple (TS)** — A fixed-length array type with a type per position, e.g. `[string, number]`. §16.10
- **Two pointers** — Using two indexes that move through an array (often from both ends of sorted data) to solve pair problems in O(n). §17.2
- **Type annotation** — A TypeScript declaration of a value's type, such as `x: number`. §16.10
- **Type assertion (TS)** — `value as Type`: telling TypeScript to trust you about a type; it checks nothing at runtime. §16.8
- **Type checker** — The part of TypeScript that verifies values are used consistently with their types and reports errors before running. §16.8
- **Type erasure** — TypeScript types are deleted when code is compiled or stripped, so none of them exist at runtime. §16.8
- **Type inference** — TypeScript working out a type from how a value is created (`let n = 1` is `number`) so you needn't write it. §16.10
- **Type predicate** — A TypeScript return type `x is T` that narrows the argument when the function returns true. §16.11
- **Type stripping** — Node running `.ts` files by deleting type syntax without checking it. §16.9
- **Typed array** — An array-like view of fixed-size numbers over an `ArrayBuffer`, such as `Uint8Array`. §7.9
- `TypeError` — Thrown when a value is the wrong type for an operation: reading a property of `null`, calling a non-function, assigning to a `const`. §11.1
- `typeof` — Operator returning a string naming a value's type; `typeof null` is `'object'`. §2.3

- **TypeScript** — JavaScript plus static types that are checked by `tsc` and erased before running. §16.8

U

- **Unary operator** — An operator with one operand: `-x`, `+x` (converts to number), `!x`, `typeof x`, `++x`. §2.5
- `uncaughtException` — Node's `process` event for an error nobody caught; log it and exit, because the app may be in a broken state. §11.5
- `undefined` — A primitive meaning "no value assigned yet"; what missing properties and arguments give. §2.3
- **Unhandled rejection** — A rejected promise with no handler; reported by the browser and fatal by default in Node. §11.5
- **Unicode** — The standard that gives every character in every writing system a number (code point), plus encodings such as UTF-8 and UTF-16 to store them. §6.5
- **Unicode property escape** — `\p{…}` in a `u`/`v` regex, matching characters by Unicode property such as `\p{L}`. §6.9
- **Union type (TS)** — `A | B`: a value that may be any one of several types; narrow it before using type-specific features. §16.10
- **Union-find** — A structure (disjoint-set union) that tracks which items are in the same group and merges groups quickly. §17.2
- **Unit test** — A fast test of one function or module in isolation. §16.1
- **Unix epoch** — The moment 1970-01-01T00:00:00Z; JavaScript dates count milliseconds since it. §5.11
- `unknown` **(TS)** — The type-safe "anything" type: you must narrow it before using it. §16.10
- **Upsert (**`getOrInsert`**)** — Getting a key's value or inserting a default if it is missing, in one call: `map.getOrInsert(k, [])` (ES2026). §8.1
- `URIError` — Thrown when `encodeURIComponent` or `decodeURIComponent` gets malformed input, like `decodeURIComponent('%')`. §11.1
- **URL / URLSearchParams** — Built-ins for parsing, building and encoding URLs and query strings. §14.6
- `using` — Declaration that automatically calls `[Symbol.dispose]()` when the block ends (finished proposal, ES2027; works in Node 24). §11.6
- **UTC** — Coordinated Universal Time: the time-zone-free world clock; store and transmit dates in UTC and convert to local time only for display. §5.11
- **UTF-16** — The encoding JavaScript strings use: characters as one or two 16-bit code units. §6.5
- **UTF-8** — The most common text encoding on the web and in files: 1 to 4 bytes per code point, ASCII stays 1 byte. §6.5

V

- **V8** — Google's JavaScript engine, used in Chrome, Edge, Node.js and Deno. §1.3
- `var` — The original function-scoped, hoisted variable declaration; avoid in new code. §2.1
- **Variable** — A named box for a value, declared with `let`, `const` or `var`. §2.1
- **Virtualization (lists)** — Rendering only the rows currently visible in a long list and reusing them as you scroll, to keep the DOM small. §13.11
- **Visual Studio Code (VS Code)** — A free code editor from Microsoft with built-in JavaScript/TypeScript support, a terminal and a debugger. §1.5
- **Vite** — A dev server plus production bundler; the default choice for new front-end apps. §10.11
- **Vitest** — A fast test runner built on Vite with a Jest-compatible `expect` API. §16.7
- **Void operator** — `void expr` evaluates the expression and returns `undefined`. §2.9

W

- **Watch expression** — An expression the debugger re-evaluates and shows every time execution pauses. §11.9
- **WeakMap** — A map with object keys held weakly: entries disappear when the key object is garbage-collected. §8.3
- **WeakRef** — A weak reference to an object; `deref()` returns it or `undefined` once collected. §8.3
- **WeakSet** — A set of objects held weakly; used to tag objects without keeping them alive. §8.3

- **Web Streams API** — The standard `ReadableStream` / `WritableStream` / `TransformStream` classes, shared by browsers and Node (`fetch` bodies use them). §15.9
- **Web Worker** — A background thread with no DOM access that talks to the page via `postMessage`. §14.8
- **webpack** — A long-established, highly configurable bundler. §10.11
- **Well-known symbol** — A built-in `Symbol` the language uses as a hook, such as `Symbol.iterator`, `Symbol.asyncIterator` or `Symbol.dispose`. §8.4
- **Worker threads** — Node's `worker_threads` module for running CPU-heavy JavaScript in parallel. §15.2

X

- `X-Content-Type-Options: nosniff` — A header telling the browser to trust the declared `Content-Type` instead of guessing, so an uploaded text file can't be run as a script. §14.12
- **XOR** — "Exclusive or": a bit is 1 when exactly one input bit is 1 (`^` in JavaScript); `x ^ x` is 0, which helps find a single unpaired number. §2.8
- **XSS** — Cross-site scripting: attacker-controlled script runs in your page, usually through unescaped HTML. §14.10

Y

- `yield` — Pauses a generator and hands a value to the consumer; `yield*` delegates to another iterable. §8.5

Z

- **Zalgo** — A callback API that is sometimes synchronous and sometimes asynchronous, making ordering unpredictable. §12.6
- **Zero-width joiner (ZWJ)** — An invisible character (U+200D) that glues emoji into one picture, such as a family emoji made of several people. §6.5
- **Zod** — A TypeScript-first library for describing data shapes and validating untrusted input at runtime. §16.8

Chapter 18 TL;DR

- Use this chapter for **revision**: cover the answer column of a sheet, say the answer, then follow the § link for anything you missed.
- **Language core**: `const` by default, `===` always, `??` for defaults, eight types with `typeof null === 'object'`, eight falsy values, floats that aren't exact (`0.1 + 0.2`), closures that remember their scope, and `this` decided at the call site (arrows excepted).
- **Data**: shallow spread vs `structuredClone`; `Object.freeze` is shallow; `toSorted` / `with` instead of mutating shared arrays; `Map` / `Set` for keyed lookup and uniqueness; generators and iterator helpers for lazy sequences.
- **Async**: one task, then **all** microtasks, then render; sync code → microtasks → tasks; `Promise.all` for parallel work, `allSettled` for every outcome, `AbortController` to cancel.
- **Platform**: `textContent` over `innerHTML`, delegation with `closest`, check `res.ok` after `fetch`, `HttpOnly; Secure; SameSite` cookies for sessions, CSP against XSS; in Node, `node:` modules, streams with `pipeline`, never block the loop.
- **Quality**: `node:test` + `assert/strict`, `tsc --noEmit` for types, ESLint + Prettier, lockfile + `npm ci`.
- **Versions**: ES2015 was the big one; since then one edition every June. ES2026 is only partly in Node 24; Temporal and `using` belong to ES2027.

This is the end of the JavaScript Handbook. Next: the React Handbook, or back to Chapter 1 for a second pass.

Chapter 19 — Interview Question Bank

How to use this bank

This chapter collects the JavaScript questions interviewers ask most, worded the way they usually appear in LeetCode study lists (above all **30 Days of JavaScript**) and the big public question collections listed at the end. It covers the language (ES2025), the browser, Node 24 and basic TypeScript.

- ★ marks a question that shows up in many sources: expect it.
- Each question has a **short answer you can say out loud** in about 30 seconds, or a short code snippet plus a sentence or two. Answers are written in this book's own words.
- → **More** links to the section of this book that explains the topic in full. Chapter 17 already answers 53 tricky questions with real output, and §17.3 holds tested implementations of the coding classics; this bank points there instead of repeating them, and adds every common question that chapter lacks.
- Code answers are correct for Node 24 and current browsers. Outputs in the puzzle group were checked by running them.

How to drill. Cover the answer, say yours out loud (or type the code), then uncover and compare. Mark the ones you missed and come back to them the next day. Do the ★ questions first, then the coding tasks (groups 14 and 15) from memory, with a timer.

Terms used everywhere below. A **primitive** is a simple immutable value (a string, a number…). An **object** is a collection of properties held by **reference** (variables store a pointer to it, not a copy). A **callback** is a function you pass to another function so it can call it later. A **polyfill** is your own code that adds a built-in feature an engine lacks. The **event loop** is the mechanism that runs queued work once the current code finishes.

---

1. Types, values and coercion

Q1. ★ What are the data types in JavaScript? There are eight. Seven are **primitives**: `string`, `number` (64-bit floating point), `bigint` (whole numbers of any size), `boolean`, `undefined`, `null` and `symbol` (a unique identifier). The eighth is `object`, which includes arrays, functions, dates, maps and everything else built from objects. Primitives are immutable and compared by value; objects are mutable and compared by reference. → *More:* §2.3 Data types and `typeof`

Q2. ★ What is the difference between primitive and reference (non-primitive) values? A primitive is stored directly in the variable; copying the variable copies the value, and nothing can change it in place. An object lives on the heap (the memory area for dynamically created data) and the variable holds a reference to it; copying the variable copies the reference, so both names see the same object. That is why `{} === {}` is `false` (two different objects) but `a === a` is `true`. → *More:* §2.4 Primitives vs objects

Q3. ★ What is the difference between `==` and `===`? `===` (strict equality) is `true` only if both values have the same type and value, with no conversion. `==` (loose equality) first **coerces** (automatically converts) the operands to a common type using a set of rules: `null == undefined` is `true`, strings and booleans become numbers, objects become primitives. Use `===` always; the one idiom some teams allow is `x == null` to test for "null or undefined". → *More:* §2.6 Comparison operators and equality · §17.4.1 Q2–Q3

Q4. ★ What is the difference between `null`, `undefined` and an undeclared variable? How do you check each? `undefined` means "no value was ever assigned" (a declared variable, a missing property, a missing argument). `null` is a value a programmer assigns on purpose to mean "empty". An **undeclared** variable was never declared at all: reading it throws `ReferenceError`. Check with `x === undefined`, `x === null`, `x == null` (either), and `typeof x === 'undefined'`, which is the only test that does not throw for an undeclared name. → *More:* §2.3 Data types and `typeof`

Q5. ⋆ What does `typeof` return, and why is `typeof null` `"object"`? `typeof` returns one of `"string"`, `"number"`, `"bigint"`, `"boolean"`, `"undefined"`, `"symbol"`, `"object"` or `"function"`. Arrays give `"object"` (use `Array.isArray`), classes give `"function"`. `typeof null === "object"` is a mistake in the original 1995 implementation: values carried a type tag, the tag for objects was 0, and `null` (a zero pointer) was read as an object. It stays because changing it now would break existing websites. → *More:* §17.4.1 Q4 and Q7

Q6. ⋆ What is type coercion? What are its common pitfalls? Coercion is JavaScript converting a value to another type automatically. `+` with any string concatenates (`1 + '2'` is `'12'`), while `-`, `*`, `/` convert to numbers (`'5' - 2` is `3`). `if`, `!`, `&&`, `||` convert to boolean. Pitfalls: `==` surprises, `'' == 0` is `true`, `[] + {}` is `'[object Object]'`, form inputs are strings so `input.value + 1` concatenates. Convert explicitly with `Number()`, `String()`, `Boolean()`. → *More:* §2.12 Type conversion summary · §17.4.1 Q1

Q7. ⋆ Which values are falsy? Exactly eight: `false`, `0`, `-0`, `0n`, `''` (empty string), `null`, `undefined` and `NaN`. Everything else is truthy, including `'0'`, `'false'`, `[]`, `{}` and `new Boolean(false)` (an object). → *More:* §2.11 Booleans, truthy and falsy

Q8. ⋆ What is `NaN`? What is the difference between `isNaN` and `Number.isNaN`? `NaN` ("not a number") is the number value produced by a failed numeric operation, such as `Number('abc')` or `0 / 0`. It is the only value not equal to itself (`NaN === NaN` is `false`). The global `isNaN(x)` converts `x` to a number first, so `isNaN('abc')` is `true`; `Number.isNaN(x)` is `true` only for the actual `NaN` value. Prefer `Number.isNaN` or `Object.is(x, NaN)`. → *More:* §2.10 Numbers in depth · §17.4.1 Q5

Q9. What is the difference between `Object.is` and `===`? They agree except in two cases: `Object.is(NaN, NaN)` is `true` (but `NaN === NaN` is `false`), and `Object.is(0, -0)` is `false` (but `0 === -0` is `true`). React uses `Object.is` to decide whether state changed. → *More:* §17.4.1 Q6

Q10. ⋆ Why is `0.1 + 0.2 !== 0.3`? How do you compare decimals? Numbers are IEEE 754 binary floating point, and 0.1 and 0.2 have no exact binary form, so the sum is `0.30000000000000004`. Compare with a tolerance: `Math.abs(a - b) < Number.EPSILON` (or a tolerance that fits the scale), round for display with `toFixed`, and store money as integer cents (or `BigInt`). → *More:* §2.10 Numbers in depth · §17.4.1 Q14

Q11. What is `Number.MAX_SAFE_INTEGER`, and what is `BigInt`? `Number.MAX_SAFE_INTEGER` is $2^{53} - 1$ (9,007,199,254,740,991): the largest integer a `number` can hold exactly, after which `n + 1` may equal `n + 2`. `BigInt` (`123n`, `BigInt('123')`) stores integers of any size. You cannot mix `BigInt` and `number` in arithmetic (`1n + 1` throws `TypeError`), `BigInt` has no decimals, and `JSON.stringify` cannot serialize it without a custom replacer. → *More:* §2.10 Numbers in depth · §17.1

Q12. ⋆ What is the difference between `||` and `??`? What does `?.` do? `a || b` returns `b` when `a` is any falsy value, so `0 || 10` is `10`. `a ?? b` (nullish coalescing) returns `b` only when `a` is `null` or `undefined`, so `0 ?? 10` is `0` — right for defaults where `0` or `''` are valid. `a?.b` (optional chaining) returns `undefined` instead of throwing when `a` is `null`/`undefined`; it also works as `a?.[key]` and `fn?.()`. → *More:* §2.7 Logical operators, `??` and `?.` · §17.4.1 Q12

Q13. What do `"hello" || "world"` and `"foo" && "bar"` return? `"hello"` and `"bar"`. `||` and `&&` return one of their **operands**, not a boolean: `||` returns the first truthy operand (or the last one), `&&` returns the first falsy operand (or the last one). This is called short-circuit evaluation: the right side is not evaluated if the left side decides the result. → *More:* §2.7

Q14. What does `!!` do? It converts any value to its boolean: the first `!` converts and negates, the second negates back. `!!'text'` is `true`, `!!0` is `false`. `Boolean(x)` does the same and reads more clearly.

Q15. What are logical assignment operators? ES2021 added `a ||= b` (assign if `a` is falsy), `a &&= b` (assign if `a` is truthy) and `a ??= b` (assign if `a` is `null`/`undefined`). They short-circuit: if no assignment is needed, `b` is not evaluated and no write happens. Common use: `options.timeout ??= 5000`. → *More:* §2.8 Bitwise and assignment operators

Q16. How do you convert a string to a number? `Number` vs `parseInt` vs unary `+`? `Number(s)` and `+s` convert the whole string and give `NaN` if any part is not numeric (`Number('12px')` is `NaN`); empty or whitespace-only strings give `0`. `parseInt(s, 10)` reads digits from the start and stops at the first non-digit (`parseInt('12px', 10)` is `12`); always pass the radix (the base, 10). `parseFloat` does the same for decimals. → *More:* §2.12 Type conversion summary

Q17. How is an object converted to a primitive? When an object is used where a primitive is needed (`+`, `==`, template literals), the engine calls `obj[Symbol.toPrimitive](hint)` if defined; otherwise, for the `"number"` or `"default"` hint, it tries `valueOf()` then `toString()`, and for the `"string"` hint `toString()` first. Plain objects end up as `'[object Object]'`, arrays as their elements joined by commas (`[1, 2] + ''` is `'1,2'`), dates as a readable string with `+` but a timestamp with `-`.

```
const price = { [Symbol.toPrimitive]: (hint) => (hint === 'number' ? 42 : '$42') };
console.log(+price, `${price}`); // 42 '$42'
```

Q18. What are `Symbol`s used for? A symbol (`Symbol('id')`) is a unique, immutable primitive used as a property key that can never clash with another key, even one with the same description. Uses: hidden-ish metadata on objects you don't own, and **well-known symbols** that hook into language behaviour — `Symbol.iterator` (make an object iterable), `Symbol.toPrimitive`, `Symbol.asyncIterator`, `Symbol.dispose`. Symbol keys are skipped by `for…in`, `Object.keys` and `JSON.stringify`. `Symbol.for('k')` gives a shared, registry-wide symbol. → *More:* §2.3 · §8.4 Iterators

Q19. How can `'abc'.toUpperCase()` work if strings are primitives? When you access a property on a primitive, the engine temporarily wraps it in its **wrapper object** (`String`, `Number`, `Boolean`, `Symbol`, `BigInt`), calls the method, and discards the wrapper. That's also why `str.foo = 1` silently does nothing (in sloppy mode) or throws in strict mode. Never create wrappers yourself with `new String('x')`: they are objects, so they are truthy and `===` fails. → *More:* §6.4

Q20. How do you check whether a value is an array, a plain object, a number? `Array.isArray(v)` (works across iframes, unlike `instanceof Array`). Number: `typeof v === 'number' && Number.isFinite(v)`. Plain object: `v !== null && typeof v === 'object' && Object.getPrototypeOf(v) === Object.prototype` (or `=== null`). For a general type name, `Object.prototype.toString.call(v)` gives `'[object Date]'`, `'[object Map]'`, and so on. → *More:* §9.7 `instanceof` and type checks

---

2. Variables, hoisting, scope and closures

Q21. ⋆ What are the differences between `var`, `let` and `const`? `var` is **function-scoped** (visible in the whole function), can be re-declared, is hoisted and initialised to `undefined`, and at the top level creates a property on `window`. `let` and `const` are **block-scoped** (visible only inside the `{}` where they are declared), cannot be re-declared in the same scope, and sit in the temporal dead zone until their line runs. `const` must be initialised and cannot be reassigned — but the object it points to can still be changed. Default to `const`, use `let` when you reassign, avoid `var`. → *More:* §2.1 · §17.4.2 Q15

Q22. ⋆ What is hoisting? Before code in a scope runs, the engine registers all its declarations. That makes it look as if declarations were "hoisted" (moved) to the top. Function declarations are hoisted with their body, so you can call them before their line. `var` is hoisted and set to `undefined`. `let`, `const` and `class` are hoisted but left uninitialised (the TDZ), so reading them early throws. Function expressions and arrows follow the rule of the variable that holds them. → *More:* §2.2 · §4.6 Hoisting — the complete picture

Q23. ⋆ What is the temporal dead zone (TDZ)? The TDZ is the time between entering a scope and reaching a `let`/`const`/`class` declaration. During it the variable exists (it shadows any outer variable of the same name) but any access throws `ReferenceError: Cannot access 'x' before initialization`. Even `typeof x` throws inside the TDZ. It exists to catch use-before-define bugs.

```
let x = 'outer';
{ console.log(x); let x = 'inner'; } // ReferenceError, not 'outer'
```

→ *More:* §2.2 · §17.4.2 Q16

Q24. ⋆ What is the difference between `function foo() {}` and `const foo = function () {}`? The first is a **function declaration**: hoisted with its body, so it can be called anywhere in its scope. The second is a **function expression** assigned to a variable: it exists only after that line runs (before it, `foo` is in the TDZ for `const`/`let`, or `undefined` for `var`, giving `TypeError: foo is not a function`). Expressions can be anonymous or named; a name on an expression is visible only inside it (handy for recursion and stack traces). → *More:* §4.2 Declarations vs expressions

Q25. What is scope? What kinds are there? Scope is the region of code where a name is visible. JavaScript has **global** scope, **module** scope (top level of an ES module — not global), **function** scope, and **block** scope (any `{}` for `let`/`const`/`class`). When a name is used, the engine looks in the current scope, then the enclosing one, and so on up to global: this chain is the **scope chain**. If it is not found, reading throws `ReferenceError`. → *More:* §4.5 Scope

Q26. ⋆ What is lexical scope? Lexical (static) scope means a function's accessible variables are decided by **where it is written** in the source, not by where it is called. A function defined inside another can see the outer function's variables, wherever it is later called from. Closures are a consequence of lexical scope. (`this` is the exception: for regular functions it depends on the call.) → *More:* §4.5

Q27. What are shadowing and illegal shadowing? Shadowing is declaring a variable in an inner scope with the same name as an outer one; inside, the inner one hides the outer. **Illegal shadowing** is shadowing a `let`/`const` with a `var` in an inner block of the same function: because `var` belongs to the whole function, it would clash with the `let`, so it is a `SyntaxError`. Shadowing a `var` with a `let` in a block is fine.

```
function f() { let a = 1; { var a = 2; } } // SyntaxError: 'a' has already been declared
```

Q28. ⋆ What is a closure? Give a practical use. A closure is a function together with the variables of the scope it was created in. The function keeps access to those variables even after the outer function has returned, and each call of the outer function creates a fresh set. Uses: private state (counters, caches), function factories, `debounce`/`memoize`, event handlers that remember data, the module pattern.

```
function makeCounter() { let count = 0; return () => ++count; }
const next = makeCounter(); next(); next(); // 2 — `count` lives on in the closure
```

→ *More:* §4.7 Closures · §17.4.2 Q19

Q29. ⋆ What does this print, and how do you fix it? `for (var i = 0; i < 3; i++) setTimeout(() => console.log(i));` It prints `3 3 3`. `var` creates a single `i` for the whole function; the callbacks run after the loop, when `i` is already 3. Fix with `let i` (a `for` loop creates a fresh binding per iteration, so it prints `0 1 2`), or capture the value with an IIFE or `setTimeout(console.log, 0, i)`. → *More:* §17.4.2 Q18

Q30. How do you create private variables in JavaScript? Three ways: a **closure** (variables of a factory function, reachable only through the functions it returns); `#private` **class fields** (`#balance`, enforced by the language, ES2022); or a **module** (anything not exported is private to the file). A `WeakMap` keyed by instance is an older pattern. A leading underscore (`_x`) is only a naming convention. → *More:* §4.7 · §17.4.4 Q33

Q31. ⋆ What is an IIFE and why was it used? An IIFE (immediately invoked function expression) is a function that runs as soon as it is defined: `(function () { … })();` or `(() => { … })();`. Before `let`/`const` and modules, it was the way to create a private scope so variables didn't leak into the global scope, and to capture loop values. Today modules and block scope cover most uses; it's still handy for an `async` IIFE in old CommonJS code. → *More:* §4.9 IIFE

Q32. What is the module pattern? An IIFE that returns an object of public functions, while variables inside stay private through closure. It was the standard way to build modules before ES modules.

```
const bank = (() => { let balance = 0;
  return { deposit: (n) => (balance += n), get: () => balance }; })();
```

→ *More:* §4.9

Q33. ⋆ What is an execution context? What are the call stack and its phases? An execution context is the record the engine creates to run a piece of code: its variables (the **environment record**), the link to the outer scope, and the value of `this`. There is one global context, and a new one for each function call. Each has two phases: **creation** (declarations registered — this is hoisting — and `this` decided) and **execution** (code runs line by line). The **call stack** holds these contexts: a call pushes a context, a return pops it. A recursive function that never stops overflows it: `RangeError: Maximum call stack size exceeded`. → *More:* §4.6 · §12.1

Q34. ⋆ What is strict mode? How do you enable it? Strict mode is a stricter variant of JavaScript that turns silent mistakes into errors: assigning to an undeclared name throws instead of creating a global, `this` is `undefined` in a plain function call (not the global object), duplicate parameter names and `with` are forbidden, writing to read-only properties throws. Enable it with `'use strict';` at the top of a file or function. ES modules and class bodies are always strict. → *More:* §1.8 Strict mode · §17.4.2 Q22

Q35. What is `globalThis`? What problems do global variables cause? `globalThis` (ES2020) is the global object in every environment: `window` in a page, `self` in a worker, `global` in Node. Global variables are risky: any script can overwrite them, names collide between libraries, they stay in memory forever, and they make code hard to test. Keep state in modules, closures or passed-in parameters. → *More:* §4.5

Q36. What is wrong with `let a = b = 5;`? It is parsed as `let a = (b = 5)`. Only `a` is declared; `b` is assigned without a declaration, so in sloppy mode it becomes an accidental global variable, and in strict mode it throws `ReferenceError`. → *More:* §17.4.2 Q20

---

3. Functions and functional patterns

Q37. ⋆ How do arrow functions differ from regular functions? Arrow functions have no own `this`, `arguments`, `super` or `new.target`: they use the ones of the surrounding scope (lexical `this`). They cannot be called with `new` and have no `prototype`. They have a shorter syntax with implicit return for a single expression (wrap an object literal in parentheses: `() => ({ a: 1 })`). Use them for callbacks; use regular functions or methods for object methods that need `this`, and for constructors. → *More:* §4.3 Arrow functions · §17.4.3 Q24

Q38. What does "functions are first-class" mean? What is a higher-order function? First-class means functions are values: you can store them in variables, pass them as arguments, return them and put properties on them. A **higher-order function** takes a function as an argument and/or returns one: `map`, `filter`, `reduce`, `setTimeout`, `addEventListener`, and your own `debounce` or `memoize`. → *More:* §4.8 Higher-order functions and callbacks

Q39. What is a pure function? What are side effects? A pure function always returns the same output for the same input and has no side effects. A **side effect** is anything a function does besides returning a value: changing outside variables or its arguments, I/O (network, files, DOM, `console.log`), reading the clock or `Math.random()`. Pure functions are easy to test, cache (memoize) and run in parallel. **Referential transparency** means a call can be replaced by its result without changing the program — true for pure functions. → *More:* §4.11 Pure functions

Q40. What is a callback, and what is callback hell? A callback is a function passed to another function to be called later — when an event fires, a timer ends, or async work finishes. **Callback hell** is deeply nested callbacks for sequential async steps, each with its own error check, making code hard to read and error handling easy to forget. Promises and `async`/`await` flatten it. → *More:* §12.6 Callbacks and callback hell

Q41. ⋆ What is currying? Why is it useful? Currying turns a function of several arguments into a chain of functions taking one (or a few) arguments each: `add(1, 2, 3)` becomes `add(1)(2)(3)`. It lets you build specialised functions by fixing early arguments (`const tax = multiply(0.2)`), which suits `map` and composition. **Partial application** is related: fixing some arguments now (`fn.bind(null, a)`) and passing the rest later, without the one-at-a-time rule. → *More:* §17.3 Curry · Q255 below

Q42. What are `compose` and `pipe`? Both combine functions into one. `compose(f, g, h)(x)` is `f(g(h(x)))` — applies right to left, as in maths. `pipe(f, g, h)(x)` is `h(g(f(x)))` — left to right, the order you read. Implementation: `const pipe = (...fns) => (x) => fns.reduce((v, f) => f(v), x);` and `compose` uses `reduceRight`. → *More:* §17.3 Compose and pipe

Q43. ⋆ What is memoization? Memoization caches a function's results by its arguments so a repeat call returns the cached value instead of recomputing. It only makes sense for pure functions. The cache key is the tricky part (one primitive argument: use it directly; several: `JSON.stringify(args)` or a nested `Map`); unbounded caches leak memory, so cap them (LRU) or use a `WeakMap` for object keys. → *More:* §17.3 Memoize · Q257 below

Q44. ⋆ What is the difference between rest parameters and spread syntax? Both use `...`. **Rest** collects many values into one array: in a parameter list (`function f(...args)`) or a destructuring pattern (`const [first, ...others] = arr`). **Spread** expands one iterable or object into many: in a call (`Math.max(...nums)`), an array literal (`[...a, ...b]`) or an object literal (`{ ...defaults, ...opts }`). Rest must be last. → *More:* §4.4 Parameters

Q45. What is the `arguments` object, and how does it differ from rest parameters? `arguments` is an array-like object (indexed, with `length`, but no array methods) holding all arguments of a regular function call. Rest parameters give a real array, only of the arguments you didn't name, and also work in arrow functions (which have no `arguments` of their own). Prefer rest. → *More:* §4.4

Q46. How do default parameters work? When are they used? A default (`function f(x = 10)`) applies only when the argument is `undefined` (missing or passed as `undefined`); `null`, `0` and `''` do not trigger it. Defaults are evaluated at call time, left to right, and can refer to earlier parameters (`(a, b = a * 2)`). A parameter with a default (and everything after it) is not counted in `fn.length`. → *More:* §4.4

Q47. How do you find how many parameters a function expects? `fn.length` gives the number of parameters before the first one with a default or a rest parameter: `((a, b = 2, ...c) => 0).length` is `1`. The `curry` implementation uses it to know when enough arguments have arrived.

Q48. What is recursion? Does JavaScript have tail-call optimisation? Recursion is a function calling itself on a smaller input until a **base case** stops it. Each call uses a stack frame, so very deep recursion (around ten thousand frames, depending on the engine) throws `RangeError`. ES2015 specified **proper tail calls** (a call in return position reusing the frame), but only Safari's engine implements it; V8 (Chrome, Node) does not. For deep inputs, use a loop with an explicit stack. → *More:* §4.10 Recursion

Q49. Why should you avoid `eval` and `new Function`? Both run a string as code. `eval` runs it in the current scope, so it can read and change local variables; both are code-injection risks when the string contains user input, block engine optimisations, and are blocked by a strict Content Security Policy. Use `JSON.parse` for data and lookup tables or functions instead of building code from strings. → *More:* §14.12 CSP

Q50. What is a thunk? A thunk is a function that wraps a computation to delay it: `const thunk = () => expensive(a, b)`. Nothing runs until you call it. Async thunks take a callback (or return a promise) and are the idea behind lazy evaluation and the Redux `thunk` middleware (functions dispatched instead of plain action objects).

Q51. What is an anonymous function, and when do you use one? A function without a name, usually a function expression or arrow passed inline: `arr.map((x) => x * 2)`, `setTimeout(function () { … })`, an IIFE. Use it for short one-off callbacks. Name functions that are reused, recursive, or that you want readable in stack traces.

---

4. `this`, `call` / `apply` / `bind`, prototypes and classes

Q52. ⋆ How does `this` work in JavaScript? For regular functions, `this` is decided at **call time** by how the function is called, checked in this order: (1) `new Fn()` — the new object; (2) `fn.call/apply(obj)` or a function made by `bind(obj)` — `obj`; (3) `obj.method()` — `obj`, the object before the dot; (4) a plain call `fn()` — `undefined` in strict mode, the global object in sloppy mode. **Arrow functions** ignore all of this and use `this` from the scope where they were written. In event listeners (regular function), `this` is the element. → *More:* §9.1 `this` — the call-time context

Q53. ⋆ What is the difference between `call`, `apply` and `bind`? All three set `this` explicitly. `fn.call(obj, a, b)` calls immediately with arguments listed; `fn.apply(obj, [a, b])` calls immediately with arguments as an array; `fn.bind(obj, a)` does not call — it returns a new function with `this` (and optionally the first arguments) fixed permanently. Mnemonic: **a**pply = **a**rray; bind = later. → *More:* §9.2 · §17.4.3 Q26

Q54. ⋆ Why does a method lose `this` when passed as a callback? How do you fix it? `setTimeout(user.greet, 100)` passes only the function; when the timer calls it, it is a plain call, so `this` is `undefined` (or global). Fixes: `user.greet.bind(user)`, a wrapper arrow `() => user.greet()`, or define the method as an arrow class field (`greet = () => …`) so it captures the instance. → *More:* §17.4.3 Q23

Q55. What is `this` inside an arrow function defined as an object-literal property? It is **not** the object. An object literal does not create a scope, so the arrow takes `this` from the code around the literal — `undefined` at the top of a module, `window` in a classic script. Use method shorthand (`greet() { … }`) for methods. → *More:* §17.4.3 Q25 · Q226 below

Q56. What happens if you `bind` a function twice, or call a bound function with `new`? The first `bind` wins: `f.bind(a).bind(b)` still has `this === a` (the second bind can still add arguments). `call`/`apply` on a bound function cannot change its `this` either. But `new` beats `bind`: `new (Fn.bind(obj))()` creates a fresh object as `this`, ignoring `obj`. → *More:* §17.4.3 Q27

Q57. ⋆ Explain prototypal inheritance and the prototype chain. Every object has a hidden link, `[[Prototype]]`, to another object (or `null`). When you read a property the object doesn't have, the engine follows that link and looks there, then that object's prototype, and so on until `null`. That chain is how objects share methods: arrays get `map` from `Array.prototype`, which gets `hasOwnProperty` from `Object.prototype`. Writing a property always creates it on the object itself (shadowing the prototype's). Classes are built on exactly this mechanism. → *More:* §9.3 Prototypes and the prototype chain

Q58. ⋆ What is the difference between `prototype` and `__proto__`? `Fn.prototype` is a property of **functions** (constructors and classes): the object that becomes the prototype of instances created with `new Fn()`. `obj.__proto__` is the old accessor for an **object's own** prototype link; use `Object.getPrototypeOf(obj)` / `Object.setPrototypeOf` instead. So `new Fn().__proto__ === Fn.prototype`. → *More:* §17.4.4 Q29

Q59. ⋆ What does the `new` keyword do? `new Fn(args)` (1) creates an empty object whose prototype is `Fn.prototype`; (2) calls `Fn` with `this` set to that object; (3) returns that object — unless `Fn` explicitly returns another **object**, which then replaces it (returning a primitive is ignored). → *More:* §9.4 Constructor functions and `new` · §17.4.3 Q28 · Q265 below

Q60. Explain `function Person(){}`, `const p = Person()` and `const p = new Person()`. The first declares a function (meant as a constructor, by the capital letter convention). `Person()` is a plain call: it returns `undefined` (no `return`), so `p` is `undefined`, and any `this.name = …` inside writes to the global object (sloppy) or throws (strict). `new Person()` creates and returns a new object linked to `Person.prototype`.

Q61. What are the differences between ES6 classes and ES5 constructor functions? Classes are mostly cleaner syntax over the same prototype mechanism, with real differences: a class must be called with `new` (calling it plainly throws), class bodies are always strict mode, class declarations are in the TDZ until their line (not usable early like function declarations), methods are non-enumerable, and classes add `extends`/`super`, `static` members and blocks, fields and `#private` members. Subclassing built-ins like `Array` and `Error` works properly only with classes. → *More:* §9.5 Classes · §17.4.4 Q31

Q62. How does inheritance work with `extends` and `super`? How was it done before ES6? `class Dog extends Animal` links `Dog.prototype` to `Animal.prototype` (instance methods) and `Dog` to `Animal` (static methods). In the subclass constructor you must call `super(args)` before using `this`, because the parent creates the object. `super.method()` calls the parent's version. Before ES6: `Dog.prototype = Object.create(Animal.prototype); Dog.prototype.constructor = Dog;` and call `Animal.call(this, args)` in the `Dog` constructor. → *More:* §9.6 Inheritance with `extends` and `super` · §17.4.4 Q34

Q63. What are static members, private fields and static blocks? `static` members belong to the class itself, not instances: utility functions (`Math`-style helpers), factory methods (`User.fromJSON`), shared counters or caches. `#field` and `#method()` are **private**: accessible only inside the class body, a `SyntaxError` elsewhere; `#field in obj` checks whether an object has it. A `static { … }` block runs once when the class is defined, for complex static initialisation. → *More:* §9.5 Classes

Q64. What are getters and setters? Accessor properties that run a function when read (`get fullName() { … }`) or assigned (`set fullName(v) { … }`), while looking like plain properties to callers. Uses: computed values, validation on assignment, lazy initialisation, keeping a stable API while changing internals. A getter without a setter makes the property read-only (assignment throws in strict mode). → *More:* §5.7 Getters and setters

Q65. ⋆ How does `instanceof` work? `obj instanceof C` walks `obj`'s prototype chain and returns `true` if it meets `C.prototype`. It can be fooled: changing prototypes changes the answer, it fails across realms (an array from an iframe is not `instanceof Array` of your window — use `Array.isArray`), and a class can

override it with `static [Symbol.hasInstance]`. Primitives always give `false`. For errors, `Error.isError(v)` (ES2026, in Node 24) is the cross-realm check. → *More:* §9.7 · §17.4.4 Q30 · Q266 below

Q66. What are the ways to create an object? An object literal `{}`; `new` with a constructor function or class; `Object.create(proto)` (choose the prototype, or `null` for a bare dictionary); a factory function returning a literal; `Object.fromEntries(pairs)`; `structuredClone` or spread of an existing object. Literals and classes cover almost all real code. → *More:* §5.1

Q67. What is the difference between `Object.create()` and `Object.assign()`? `Object.create(proto, descriptors?)` makes a **new empty object whose prototype is `proto`** — it sets up inheritance, copying nothing. `Object.assign(target, ...sources)` **copies own enumerable properties** from sources into an existing target (a shallow copy, running setters on the target) and returns the target. `{ ...a, ...b }` is the modern form of `Object.assign({}, a, b)`. → *More:* §5.6 · §17.4.4 Q32

Q68. What is a mixin? Composition vs inheritance? A mixin is a set of methods copied into a class (or applied as a function `Base => class extends Base {…}`), adding behaviour without a deep inheritance chain. **Composition** builds objects from smaller parts they *have* ("has-a") rather than inheriting what they *are* ("is-a"); it avoids fragile deep hierarchies and is generally preferred. Use inheritance for a true, shallow "is-a" relationship. → *More:* §9.8 Mixins · §9.9 Composition vs inheritance

Q69. What are the pros and cons of extending built-in prototypes (e.g. adding `Array.prototype.last`)? Pro: convenient syntax everywhere. Cons: it changes every array in the program, including in libraries; names can clash with future standard methods (this is why the standard had to call it `flat` instead of `flatten` — a library had already added a different `flatten`); and `for…in` can expose enumerable additions. Only acceptable for spec-compliant polyfills; otherwise write a helper function. → *More:* Q292 below (the LeetCode version)

Q70. How do you implement method chaining? Return `this` from each method that changes state, so the next call works on the same object: `calc.add(5).multiply(2).getResult()`. For immutable chaining (like array `map().filter()`), return a **new** instance instead. → *More:* Q300 below

Q71. How do you implement a singleton in JavaScript? The simplest singleton is an ES module that exports one instance: modules are evaluated once and cached, so every importer gets the same object. With a class, keep a `static #instance` and return it from a `static getInstance()` (or from the constructor). Singletons are global state in disguise, so they make tests harder; prefer passing dependencies in. → *More:* §10.3 How ES modules load

Q72. What are `Proxy` and `Reflect`? A `Proxy` wraps an object and intercepts operations on it through **traps** — `get`, `set`, `has`, `deleteProperty`, `apply`, `construct` and more. Uses: validation, default values, logging, change tracking for reactivity (Vue 3 uses proxies), negative array indexes. `Reflect` provides the default behaviour of each operation as a function (`Reflect.get(target, key, receiver)`), so a trap can do its extra work and then forward correctly.

```
const safe = new Proxy({}, { get: (t, k) => (k in t ? t[k] : `no ${String(k)}`) });
console.log(safe.name); // 'no name'
```

Q73. Discuss using object-oriented principles in JavaScript. JavaScript supports the four pillars: **encapsulation** (`#private` fields, closures, modules), **abstraction** (expose a small public API), **inheritance** (prototypes / `extends`), and **polymorphism** (any object with the right method works — "duck typing" — and subclasses override methods). In practice JavaScript code mixes OOP with functional style: classes for things with identity and lifecycle, plain functions and data elsewhere, composition over deep inheritance. → *More:* §9.9 · LLD Handbook

---

5. Objects, arrays, strings and collections

Q74. ⋆ What is the difference between a shallow copy and a deep copy? How do you deep-copy an object? A **shallow copy** (`{ ...obj }`, `Object.assign({}, obj)`, `[...arr]`, `arr.slice()`) copies only the top level; nested objects are still shared, so changing `copy.address.city` changes the original. A **deep copy** copies every level. Use `structuredClone(obj)` (built in; handles `Date`, `Map`, `Set`, typed arrays and circular references, but throws on functions and DOM nodes and drops prototypes/class identity).

`JSON.parse(JSON.stringify(obj))` is lossy: it drops `undefined` and functions, turns dates into strings, and throws on cycles. → *More:* §5.4 Spread, rest and copying · §17.3 Deep clone · §17.4.6 Q43

Q75. Explain mutable vs immutable. How do you make an object immutable? A mutable value can be changed in place (objects, arrays); an immutable one cannot (primitives — string methods always return new strings). Immutability makes change detection cheap (compare references, as React does) and prevents spooky action at a distance; the cost is extra copying. Tools: `const` (prevents reassignment only), `Object.freeze` (shallow — freeze recursively for deep), non-mutating methods (`toSorted`, `with`, spread), or a library like Immer. → *More:* §5.6 · §17.4.6 Q44

Q76. What is the difference between `Object.freeze`, `Object.seal` and `Object.preventExtensions`? And `const` vs `freeze`? `preventExtensions`: no new properties. `seal`: also no deleting, and existing properties become non-configurable — values can still change. `freeze`: also no changing values; everything is read-only. All three are **shallow** (nested objects stay mutable), and violations fail silently in sloppy mode but throw `TypeError` in strict mode. `const` is about the **variable** (cannot be reassigned); `freeze` is about the **object** (cannot be modified). They are independent. → *More:* §5.6

Q77. What are property descriptors? Each property has attributes: a data property has `value`, `writable`, `enumerable` (shows in `for…in` / `Object.keys`), `configurable` (can be deleted or redefined); an accessor property has `get` / `set` instead of `value` / `writable`. Read them with `Object.getOwnPropertyDescriptor(s)`, set them with `Object.defineProperty`. Properties created by `defineProperty` default to all `false`; normal assignment makes them all `true`. → *More:* §5.6

Q78. ⋆ How do you check if a key exists in an object? `Object.hasOwn(obj, 'k')` (ES2022) — own property only, the recommended check. `'k' in obj` — own **or inherited** (so `'toString' in {}` is `true`). `obj.k !== undefined` fails when the value is legitimately `undefined`. `obj.hasOwnProperty('k')` breaks for `Object.create(null)` objects or an object with its own `hasOwnProperty` key — hence `Object.hasOwn`. → *More:* §5.2

Q79. How do you loop over an object's properties? How do you check whether an object is empty? `for (const [k, v] of Object.entries(obj))` is the usual way; `Object.keys` and `Object.values` give keys or values. All three include only **own enumerable string** keys. `for…in` also visits inherited enumerable keys. Empty check: `Object.keys(obj).length === 0` (for a plain object). Order: integer-like keys first in ascending order, then other string keys in insertion order. → *More:* §5.6 · Q290 below

Q80. ⋆ What is the difference between `for…in` and `for…of`? `for…in` loops over the **enumerable property names** (strings) of an object, including inherited ones — meant for plain objects. `for…of` loops over the **values** of an **iterable** (arrays, strings, `Map`, `Set`, generators, `NodeList`) — use it for arrays. Using `for…in` on an array gives string indexes and any extra properties; `for…of` on a plain object throws `TypeError: obj is not iterable`. → *More:* §3.9 · §3.10 · §17.4.6 Q50

Q81. Give examples of object and array destructuring. Destructuring unpacks values into variables. You can rename, set defaults, nest, and collect the rest.

```
const { name, age: years = 0, address: { city } = {}, ...rest } = user;
const [first, , third = 'n/a', ...others] = list;
[a, b] = [b, a];                          // swap without a temp variable
function draw({ x = 0, y = 0 } = {}) {}     // named options with defaults
```

→ *More:* §5.5 · §7.7

Q82. ⋆ What are `JSON.stringify` and `JSON.parse`? What does `stringify` drop? `JSON.stringify(value, replacer?, indent?)` turns a value into a JSON string; `JSON.parse(text, reviver?)` turns JSON text back into values. `stringify` skips properties whose value is `undefined`, a function or a symbol (in arrays they become `null`), turns `NaN` / `Infinity` into `null`, calls `toJSON` (so dates become ISO strings), throws on `BigInt` and on circular references. A **replacer** filters or transforms values on the way out; a **reviver** transforms them on the way in (e.g. turning date strings back into `Date`). → *More:* §5.12 JSON · §17.4.6 Q49

Q83. How do you handle circular references when serializing? `JSON.stringify` throws `TypeError: Converting circular structure to JSON`. Options: use `structuredClone` (if you only need a copy), pass a replacer that tracks seen objects in a `WeakSet` and replaces repeats with a marker (simple, but it also marks an object that is merely referenced twice, not only true cycles), or restructure the data to use IDs instead of back-references.

```
const seen = new WeakSet();
JSON.stringify(obj, (k, v) => (typeof v === 'object' && v !== null ? (seen.has(v) ? '[Circular]' : (seen.add(v), v)) : v));
```

Q84. ⋆ When do you use a `Map` instead of an object? And `Set`? Use a `Map` for a dictionary with keys of any type (objects, numbers kept as numbers), frequent adds and deletes, a reliable `size`, guaranteed insertion order and no inherited keys (`'constructor'` can't clash). Use an object for fixed-shape records and JSON. A `Set` stores unique values (by `SameValueZero`, so `NaN` works) with O(1) `has`; ES2025 added set algebra: `union`, `intersection`, `difference`, `symmetricDifference`, `isSubsetOf`, `isSupersetOf`, `isDisjointFrom`. → *More:* §8.1 `Map` · §8.2 `Set` · §17.4.6 Q48

Q85. ⋆ What are `WeakMap` and `WeakSet`? How do they differ from `Map` and `Set`? Their keys must be objects (or non-registered symbols), and they hold them **weakly**: if nothing else references a key, it can be garbage-collected and its entry disappears. Therefore they are not iterable and have no `size` or `clear`. Use them to attach data to objects you don't own (DOM nodes, cache per object, private data) without causing memory leaks. → *More:* §8.3 · §17.4.7 Q53

Q86. ⋆ What is the difference between `map` and `forEach`? `map` returns a **new array** of the callback's return values (same length); use it to transform. `forEach` returns `undefined` and is only for side effects (logging, writing to the DOM). Neither can be stopped early with `break` (use `for…of`, `some` or `find`), and neither waits for `async` callbacks. → *More:* §7.5 Iteration methods

Q87. ⋆ What is the difference between `slice` and `splice`? `slice(start, end)` returns a **copy** of a portion (end exclusive), leaving the original untouched; negative indexes count from the end. `splice(start, deleteCount, ...items)` **mutates** the array — removing and/or inserting in place — and returns the removed elements. ES2023's `toSpliced` is the non-mutating version of `splice`. → *More:* §7.4 The methods table

Q88. Which array methods mutate the array? What are the non-mutating alternatives? Mutating: `push`, `pop`, `shift`, `unshift`, `splice`, `sort`, `reverse`, `fill`, `copyWithin` (and setting `length`). ES2023 added copies: `toSorted`, `toReversed`, `toSpliced`, and `with(i, v)` (copy with one element replaced). Non-mutating methods also include `map`, `filter`, `slice`, `concat`, `flat`. This matters for React state and for functions that receive arrays they don't own. → *More:* §7.4

Q89. Explain `reduce` with examples. `arr.reduce((acc, item, i, arr) => newAcc, initial)` walks the array carrying an **accumulator**; each step returns the next accumulator, and the last one is the result. Always pass `initial` (without it, the first element is used, and an empty array throws `TypeError`).

```
const total = cart.reduce((sum, item) => sum + item.price * item.qty, 0);
const countBy = words.reduce((m, w) => ((m[w] = (m[w] ?? 0) + 1), m), {});
```

→ *More:* §7.5

Q90. ⋆ How do you sort numbers correctly? Is `sort` stable? Default `sort()` converts elements to strings and compares UTF-16 code units, so `[10, 1, 2].sort()` gives `[1, 10, 2]`. Pass a comparator: `(a, b) => a - b` ascending; for strings use `a.localeCompare(b)`; for objects compare a field. Since ES2019, `sort` is required to be **stable** (equal elements keep their order), so you can sort by a secondary key first, then a primary key. `sort` mutates; use `toSorted` for a copy. → *More:* §7.6 Sorting correctly · §17.4.1 Q11

Q91. ⋆ How do you remove duplicates from an array? Primitives: `[...new Set(arr)]` (or `Array.from(new Set(arr))`). Objects by a key: `[...new Map(arr.map((o) => [o.id, o])).values()]` keeps the **last** object per id (in the position where that id first appeared); to keep the **first**, filter with a `Set` of seen ids — `const seen = new Set(); arr.filter((o) => !seen.has(o.id) && seen.add(o.id))`. `arr.filter((x, i) => arr.indexOf(x) === i)` works but is O(n²). → *More:* §8.2

Q92. What are `find`, `findIndex`, `findLast`, `some`, `every`, `includes` and `indexOf` for? `find`/`findIndex` return the first matching element/index (`undefined`/`-1` if none); `findLast`/`findLastIndex` (ES2023) search from the end. `some` is `true` if any element matches, `every` if all do (an empty array gives `true` for `every`). `includes(x)` checks presence using SameValueZero, so it finds `NaN`; `indexOf` uses `===` and cannot. → *More:* §7.5

Q93. How do you empty an array? What do `delete arr[i]` and `arr.length = 0` do? `arr.length = 0` empties the array in place (every reference sees it empty). `arr = []` makes the variable point to a new array (others still see the old one; impossible with `const`). `arr.splice(0)` also empties in place and returns

the removed items. `delete arr[i]` leaves a **hole** (`length` unchanged); use `splice` to remove. → *More:* §17.4.6 Q46

Q94. What are sparse arrays? What is the difference between `Array(3)` and `Array.of(3)`? A sparse array has **holes** — indexes with no element at all (`[1, , 3]`, `Array(3)`, `delete arr[1]`, or writing past the end). `forEach`/`map`/`filter` skip holes; `for…of` and spread treat them as `undefined`. `Array(3)` makes an empty array of length 3; `Array(3, 4)` and `Array.of(3)` make arrays of those elements. To create filled arrays use `Array.from({ length: 3 }, (_, i) => i)` or `Array(3).fill(0)`. → *More:* §7.2 Creating arrays

Q95. What is an array-like object? How do you convert it to an array? An object with numeric keys and a `length`, but without array methods — `arguments`, a `NodeList` from older APIs, a `{ length: 2, 0: 'a', 1: 'b' }`. Convert with `Array.from(x)` (works for array-likes and iterables, and takes a map function) or `[...x]` (iterables only). → *More:* §7.2

Q96. What is an iterator? How do you make an object iterable? An **iterator** is an object with a `next()` method that returns `{ value, done }`. An **iterable** is an object with a `[Symbol.iterator]()` method returning an iterator; `for…of`, spread, destructuring, `Array.from`, `Promise.all` and `new Map()` all consume iterables. Easiest way to make one: a generator method.

```
class Range { constructor(a, b) { this.a = a; this.b = b; }
  *[Symbol.iterator]() { for (let i = this.a; i <= this.b; i++) yield i; } }
console.log([...new Range(1, 3)]); // [1, 2, 3]
```

→ *More:* §8.4

Q97. What are generators and when would you use them? A generator function (`function*`) returns an iterator; each `yield` pauses the function and hands out a value, and the next `next()` resumes it (and `next(x)` makes the paused `yield` evaluate to `x`). Uses: lazy sequences (including infinite ones), custom iteration, paging through results, and (historically) async control flow before `async`/`await`. **Async generators** (`async function*`) yield promises' results and are consumed with `for await…of`. → *More:* §8.5 Generators · §12.12

Q98. What are iterator helpers (ES2025)? Methods on iterators — `map`, `filter`, `take`, `drop`, `flatMap`, `reduce`, `toArray`, `forEach`, `some`, `every`, `find` — plus `Iterator.from`. Unlike array methods, they are **lazy**: they process one element at a time and stop early, so they work on huge or infinite sequences without building intermediate arrays: `naturals().filter(isPrime).take(5).toArray()`. → *More:* §8.6 Iterator helpers (ES2025)

Q99. What do `Object.groupBy` and `Map.groupBy` do? ES2024. `Object.groupBy(items, fn)` returns a null-prototype object whose keys are the callback's return values and whose values are arrays of the items in each group; `Map.groupBy` returns a `Map`, so keys can be any value (objects, numbers). They replace the classic `reduce` grouping. → *More:* §8.7 · Q293 below

Q100. Are strings mutable? `slice` vs `substring` vs `substr`? Strings are immutable: every method returns a new string and `s[0] = 'x'` does nothing. `slice(start, end)` accepts negative indexes (from the end). `substring(start, end)` treats negatives as 0 and swaps the arguments if `start > end`. `substr(start, length)` is a deprecated legacy method; avoid it. → *More:* §6.4 · §6.6 String methods

Q101. How do you reverse a string? What goes wrong with emoji? `s.split('').reverse().join('')` works for plain ASCII but splits emoji and other characters outside the Basic Multilingual Plane, because `split('')` works on UTF-16 **code units** and those characters use two (`'👍'.length` is 2). `[...s].reverse().join('')` splits by **code points** and handles them; for user-perceived characters (flags, combined emoji, accents) use `Intl.Segmenter`. → *More:* §6.5 Unicode

Q102. What are template literals and tagged templates? Template literals are backtick strings with `${expression}` interpolation and real line breaks. A **tagged template** `tag`text ${x}`` calls `tag(strings, ...values)` with the literal parts and the evaluated values separately, so the function can escape, translate or build something else — used for HTML escaping, SQL parameter binding, styled-components and `String.raw` (no escape processing). → *More:* §6.3 Template literals

Q103. What are the common pitfalls of `Date`? What is Temporal? `Date` months are 0-based (`new Date(2026, 0, 1)` is January), objects are mutable, parsing non-ISO strings is unreliable, and date-only ISO strings (`'2026-01-01'`) are read as UTC while date-time strings without an offset are local time. Time zones other than local/UTC are hard. **Temporal** is the replacement API with immutable types (`PlainDate`,

`ZonedDateTime`, `Instant`, `Duration`); it is finished (Stage 4) for ES2027 and ships in Firefox 139+, Chrome/Edge 144+ and Node 26, but not in Node 24 or Safari yet, so use a polyfill there. For formatting, use `Intl.DateTimeFormat`. → *More:* §5.11 `Date`

Q104. How do you generate a random integer between `min` and `max` (inclusive)? `Math.floor(Math.random() * (max - min + 1)) + min`. `Math.random()` returns a float in [0, 1). It is not cryptographically secure: for tokens or IDs use `crypto.getRandomValues()` or `crypto.randomUUID()`. → *More:* §5.10 `Math`

---

6. The event loop, promises and `async` / `await`

Q105. ⋆ JavaScript is single-threaded — so how does it do asynchronous work? Your JavaScript runs on one thread with one **call stack**: one thing at a time. But the **host** (browser or Node) does the slow work — timers, network, file I/O — outside that thread, and when it finishes it queues a callback. The **event loop** runs queued callbacks one by one whenever the call stack is empty. So JavaScript never blocks waiting for I/O, but a long synchronous computation does block everything (clicks, rendering, other requests). → *More:* §12.1 · §12.2 The event loop

Q106. ⋆ Explain the event loop. What are the call stack, the task queue and the microtask queue? The **call stack** holds the functions running right now. The **task (macrotask) queue** holds callbacks from timers, I/O, UI events and messages. The **microtask queue** holds promise reactions (`then`/`catch`/`finally` callbacks, code after `await`), `queueMicrotask` callbacks and `MutationObserver` callbacks. The loop: run one task to completion; then run **all** microtasks, including ones added meanwhile; then (in browsers) render if needed; repeat. So all pending promise callbacks run before the next timer. → *More:* §12.2 · §12.3 Ordering puzzles · §17.4.5 Q35

Q107. ⋆ What is the difference between microtasks and macrotasks? Give examples of each. Microtasks: promise callbacks, `await` continuations, `queueMicrotask`, `MutationObserver`; in Node, `process.nextTick` runs in an even earlier queue. Macrotasks (tasks): `setTimeout`, `setInterval`, `setImmediate` (Node), I/O callbacks, UI events, `MessageChannel` messages. After every task the microtask queue is drained completely — which is why an infinite chain of microtasks can freeze the page just like a `while (true)` loop, while timers yield. → *More:* §12.2

Q108. ⋆ What does `setTimeout(fn, 0)` really do? It queues `fn` as a task to run **at least** 0 ms later — after the current code and all microtasks finish, and after any earlier tasks. Browsers clamp nested timers (more than five levels deep) to at least 4 ms, and background tabs to 1 s or more. The delay is a minimum, never a guarantee. To run something after the current code but before rendering or timers, use `queueMicrotask`. → *More:* §12.5 Timers

Q109. ⋆ What is a promise? What are its states? A promise is an object that stands for the eventual result of an async operation. It is **pending**, then becomes **settled** exactly once: **fulfilled** with a value or **rejected** with a reason (an error); after that it never changes. You react with `then(onFulfilled, onRejected)`, `catch`, `finally`, or `await`. Callbacks attached after settlement still run (asynchronously), which fixes the "missed the event" problem of callbacks. → *More:* §12.7 Promises

Q110. What are the pros and cons of promises over callbacks? Pros: flat chaining instead of nesting, one `catch` for a whole chain, guaranteed async and at-most-once callbacks (no "called twice" or "called synchronously" bugs), composition with `Promise.all` and friends, and `async`/`await` syntax. Cons: one-shot (a stream of events needs callbacks, async iterators or observables), not cancellable by themselves (use `AbortController`), and an unhandled rejection is easy to miss. → *More:* §12.6 · §12.7

Q111. ⋆ How does promise chaining work? `then` always returns a **new** promise. Whatever the callback returns becomes that promise's value; if it returns a promise, the chain waits for it; if it throws, the new promise rejects and control skips to the next `catch`. A `catch` that returns normally **recovers** — the chain continues fulfilled. Forgetting `return` inside a `then` is the classic bug: the chain moves on without waiting. → *More:* §12.7 · Q222 below

Q112. ⋆ What is the difference between `Promise.all`, `allSettled`, `race` and `any`? `all`: fulfils with an array of all values (in input order) when all fulfil; rejects as soon as **one** rejects. `allSettled`: waits for all and gives `{ status, value | reason }` for each — never rejects. `race`: settles like the **first** promise to settle, success or failure. `any`: fulfils with the first **success**; rejects with an

`AggregateError` only if all reject. Typical uses: parallel requests (`all`), batch with partial failures (`allSettled`), timeout (`race`), fastest mirror (`any`). → *More:* §12.8 · §17.4.5 Q42

Q113. Does `Promise.all` cancel the other promises when one rejects? No. A promise is only a result holder; the operations behind the other promises keep running and their results are ignored. To actually stop them, pass one `AbortSignal` to all of them and call `controller.abort()` in the `catch`. → *More:* §12.11 Cancellation

Q114. ⋆ How do `async` and `await` work? An `async` function always returns a promise: its `return` value fulfils it and a `throw` rejects it. `await p` pauses **that function only** until `p` settles, then resumes with the value (or throws the rejection, so `try`/`catch` works). The rest of the function is scheduled as a microtask; the caller and the event loop keep running meanwhile. It is syntax over promises, not threads. → *More:* §12.9 `async` / `await` · §17.4.5 Q36

Q115. ⋆ How do you run async operations in parallel vs in sequence with `await`? `const a = await f(); const b = await g();` is **sequential**: `g` starts only after `f` finishes (total ≈ sum). For independent operations, start both, then await together: `const [a, b] = await Promise.all([f(), g()]);` (total ≈ the slower). For a list processed one by one, use `for…of` with `await`; for a list in parallel with a limit, use a concurrency pool. → *More:* §17.4.5 Q41 · Q264 below

Q116. ⋆ Why doesn't `await` work inside `forEach`? `forEach` calls the async callback and ignores the promise it returns, so the loop finishes immediately and the code after it runs before any callback completes; errors become unhandled rejections. Use `for…of` (sequential) or `await Promise.all(items.map(async (x) => …))` (parallel). → *More:* §17.4.5 Q40

Q117. ⋆ How do you handle errors with promises and `async`/`await`? What is an unhandled rejection? With promises, end each chain with `.catch()`; with `async`/`await`, wrap awaits in `try`/`catch` (and use `finally` for cleanup). A rejection no handler ever catches is an **unhandled rejection**: browsers fire an `unhandledrejection` event and log it; Node prints the error and **exits with code 1** (the default since Node 15). Common causes: a missing `return` inside `then`, an un-awaited async call, `forEach` with async callbacks. → *More:* §12.10 Error handling across async code · §11.5

Q118. How do you cancel a `fetch` request or add a timeout? Create an `AbortController`, pass `controller.signal` to `fetch(url, { signal })`, and call `controller.abort()`; the `fetch` promise rejects with an `AbortError` `DOMException`. For a timeout use `AbortSignal.timeout(5000)` (rejects with a `TimeoutError`), and combine signals with `AbortSignal.any([a, b])`. The same signal works for event listeners, many Node APIs and your own functions. → *More:* §12.11

Q119. What is `queueMicrotask`? How does `process.nextTick` compare with `setImmediate`? `queueMicrotask(fn)` schedules `fn` in the microtask queue, like a resolved promise's `then` but without creating a promise. In Node, `process.nextTick(fn)` runs even sooner — its queue is drained before the promise microtasks. `setImmediate(fn)` is a macrotask that runs in the "check" phase, after I/O polling. Inside an I/O callback, `setImmediate` always runs before `setTimeout(fn, 0)`; in the main module their order is not guaranteed. → *More:* §12.4 Node's event loop · §17.4.5 Q37

Q120. What are `Promise.withResolvers` and `Promise.try`? `Promise.withResolvers()` (ES2024) returns `{ promise, resolve, reject }`, so you can settle a promise from outside an executor — handy for wrapping event-based APIs or building queues. `Promise.try(fn)` (ES2025) calls `fn` immediately and always returns a promise: a sync return fulfils it and a sync throw rejects it, so sync and async errors go to the same `catch`. → *More:* §12.7

Q121. When does a promise executor run? What if `resolve` is called twice? The executor function passed to `new Promise` runs **synchronously**, immediately, during the constructor. Only the first call to `resolve` or `reject` counts; later calls are ignored silently. Throwing inside the executor rejects the promise. → *More:* §17.4.5 Q38

Q122. What is `for await…of`? A loop over an **async iterable** — something whose `next()` returns promises, like an async generator, a Node readable stream, or paginated API results. Each iteration awaits the next value. It also accepts an array of promises, awaiting them in order (but they already run in parallel). `Array.fromAsync` (ES2026, in Node 24) collects one into an array. → *More:* §12.12

Q123. What is the difference between promises and observables? A promise delivers **one** value, is **eager** (the work started before you got it) and cannot be cancelled by itself. An observable (RxJS) delivers **zero to many** values over time, is **lazy** (nothing runs until someone subscribes), can be cancelled by

unsubscribing, and has operators like `map`, `debounceTime`, `switchMap`. Observables suit event streams; promises suit single requests.

Q124. What is the difference between synchronous and asynchronous functions? A synchronous function does all its work before returning; the caller waits, and the thread is blocked meanwhile. An asynchronous function starts the work and returns at once (usually a promise); the result arrives later through a callback or promise, while the thread stays free for other events. → *More:* §12.1

---

7. Errors, modules and tooling

Q125. ⋆ How do `try` / `catch` / `finally` work? What if `finally` has a `return`? `try` runs code; if it throws, `catch (err)` runs (the binding is optional since ES2019); `finally` always runs afterwards — after a normal finish, a `return`, a `break` or an exception — for cleanup. A `return` (or `throw`) in `finally` **overrides** the one from `try` / `catch`, and swallows a pending exception, so never return from `finally`. `try` / `catch` only catches errors thrown synchronously inside it or from awaited promises. → *More:* §11.2 · Q238 below

Q126. What are the built-in error types? `Error` (base), `SyntaxError` (invalid code or `JSON.parse` input), `ReferenceError` (unknown name, TDZ), `TypeError` (wrong kind of value: calling a non-function, reading a property of `undefined`), `RangeError` (value out of range, stack overflow), `URIError`, `EvalError`, and `AggregateError` (several errors, from `Promise.any`). Every error has `name`, `message` and a non-standard but universal `stack`. → *More:* §11.1

Q127. How do you create a custom error class? Extend `Error`, call `super(message, { cause })`, set `this.name`, and add any extra fields. Callers can then use `instanceof` or check `name` / `code`.

```
class HttpError extends Error {
  constructor(status, message, options) { super(message, options); this.name = 'HttpError'; this.status = status;
}
}
throw new HttpError(404, 'User not found', { cause: originalError });
```

→ *More:* §11.3 Custom errors · §11.4 `cause`

Q128. Why doesn't `try` / `catch` catch an error thrown inside a `setTimeout` callback? By the time the callback runs, the `try` block has finished and its stack frame is gone; the callback runs later from the event loop with a fresh stack. Put the `try` / `catch` inside the callback, or wrap the timer in a promise and `await` it. → *More:* §17.4.5 Q39

Q129. How do you catch errors globally? Browser: `window.addEventListener('error', …)` for uncaught exceptions (and failed resource loads, with capture) and `window.addEventListener('unhandledrejection', …)` for promises. Node: `process.on('uncaughtException')` and `process.on('unhandledRejection')` — use them to log and then exit, because the process may be in an unknown state; restart via a process manager. Error monitoring services (like Sentry) hook into these. → *More:* §11.5 · §12.10

Q130. What tools and techniques do you use to debug JavaScript? `console.log` / `table` / `dir` / `trace` / `time`; the `debugger` statement; DevTools breakpoints (line, conditional, logpoints, DOM-change, XHR/fetch, exception breakpoints), the call stack and scope panes, the Network and Performance panels; source maps (files that map built code back to the original source) to debug bundled code; `node --inspect` with Chrome DevTools or VS Code for Node; and a failing test that reproduces the bug. → *More:* §11.7 · §11.9

Q131. What are the performance implications of `try` / `catch`? In modern engines a `try` / `catch` costs essentially nothing when no error is thrown. **Throwing** is expensive (an error object and stack trace are built), so don't use exceptions for normal control flow in hot loops — return a value instead.

Q132. What is `using` (explicit resource management)? `using x = resource` declares a variable whose `[Symbol.dispose]()` method is called automatically when the block ends, even after an exception; `await using` calls `[Symbol.asyncDispose]()`. It replaces `try` / `finally` cleanup for files, locks and connections. The proposal is finished for ES2027 and already works in Node 24. → *More:* §11.6 Guaranteed cleanup

Q133. ⋆ What is the difference between ES modules and CommonJS? CommonJS (Node's original system): `require()` / `module.exports`, loaded synchronously at run time, exports are a copied value, `require` can be called anywhere. **ES modules** (the standard): `import` / `export` declared statically at the top, so tools

know the dependency graph before running (tree shaking, browser support); loaded asynchronously; exports are **live bindings**; strict mode always; top-level `await`; `this` is `undefined` at top level. Node picks the system by extension (`.mjs` / `.cjs`) or the `"type"` field in `package.json`. ESM can import CJS; Node 22.12+ can also `require()` ESM unless the module uses top-level `await`. → *More:* §10.8 CommonJS vs ES modules in Node

Q134. Named exports vs default exports? A module can have many named exports (`export const a`, imported as `import { a } from`) and at most one default (`export default`, imported under any name). Named exports keep names consistent across the codebase, are easier for auto-import and refactoring tools, and catch typos at link time; many teams prefer them. Default exports are common for "one main thing per file" (React components, config). → *More:* §10.2

Q135. What are live bindings in ES modules? An import is a read-only view of the exporting module's variable, not a copy. If the exporter later changes `export let count`, every importer sees the new value. Importers cannot assign to it (`TypeError`). In CommonJS, `const { count } = require(…)` copies the value at that moment. → *More:* §10.3

Q136. What is dynamic `import()`, and what are its use cases? `import('./module.js')` loads a module at run time and returns a promise of its namespace object. It works in CommonJS and classic scripts too. Uses: **code splitting** (load a heavy chart library only when the chart tab opens), conditional or locale-based loading, lazy routes in SPAs (`React.lazy`), loading ESM from CommonJS. Non-JavaScript modules need an **import attribute** (ES2025): `import config from './config.json' with { type: 'json' };` or `import('./config.json', { with: { type: 'json' } })`. → *More:* §10.4

Q137. What is top-level `await`? In an ES module you can use `await` outside any function (ES2022). The module's evaluation pauses until it settles, and modules that import it wait too — so use it for setup (loading config, a DB connection, a dynamic import), not for slow work. Before it, the workaround was an async IIFE. A module using it cannot be loaded with `require()`. → *More:* §10.5 Top-level `await`

Q138. What happens with circular imports? ES modules allow cycles: the graph is linked first, then evaluated depth-first, so one module in the cycle runs before the other has finished; reading a not-yet-initialised `let` / `const` / `class` export throws a TDZ `ReferenceError` (function declarations are hoisted and work). In CommonJS, the second module gets the first one's **partially filled** `module.exports`. Fix by moving shared code into a third module or deferring access into functions. → *More:* §10.3

Q139. How do you get `__dirname` and `__filename` in an ES module? They don't exist in ESM. Use `import.meta.dirname` and `import.meta.filename` (Node 20.11+), or `fileURLToPath(import.meta.url)`. `import.meta.url` is the module's own `file://` URL; `new URL('./data.json', import.meta.url)` resolves a file next to the module. → *More:* §10.6 `import.meta` and JSON modules · §15.5

Q140. What is tree shaking? A bundler optimisation that removes exports nobody imports. It relies on ES modules' **static** `import` / `export` (the bundler sees exactly what is used) and on modules being free of side effects at import time (`"sideEffects": false` in `package.json` tells the bundler it may drop unused files). CommonJS, `import * as` used dynamically, and code with top-level side effects defeat it. → *More:* §10.11 Bundlers

Q141. What do bundlers, transpilers and polyfills do? Shim vs polyfill? A **bundler** (Vite, webpack, esbuild, Rollup) follows imports and produces a few optimised files: minified, split, tree-shaken, with source maps. A **transpiler** (Babel, TypeScript, SWC, esbuild) rewrites new **syntax** into older syntax (or TS into JS). A **polyfill** adds a missing **API** at run time using existing features (`Array.prototype.at` written in JS). A **shim** is the broader term: any library that fills in or adapts an API, possibly with a different interface. → *More:* §10.11 · §10.12 Transpilers and polyfills

Q142. What do `^` and `~` mean in `package.json`? Why commit the lockfile? Versions follow **semver**: `MAJOR.MINOR.PATCH` (breaking, feature, fix). `^1.4.2` allows `>=1.4.2 <2.0.0` (any non-breaking update); `~1.4.2` allows `>=1.4.2 <1.5.0` (patches only). (For `0.x`, `^0.4.2` means `<0.5.0`.) The lockfile (`package-lock.json`) records the exact resolved version of every package in the tree, so every install and CI build gets identical code; use `npm ci` in CI. → *More:* §10.10 Semantic versioning, ranges and lockfiles

Q143. What are minification and obfuscation? Minification removes whitespace and comments and shortens local names to make files smaller and faster to download; it is a normal build step (Terser, esbuild). **Obfuscation** deliberately makes code hard to read to deter copying; it is not security — anything shipped to the browser can be read — and it is not encryption, which needs a key to reverse. → *More:* §10.11

8. The DOM and events

Q144. What is the DOM? What is the difference between `window` and `document`? The DOM (Document Object Model) is the browser's live tree of objects representing the HTML page; JavaScript reads and changes the page through it. `window` is the global object for the browser tab: timers, `location`, `history`, `localStorage`, viewport size — these browser-level APIs are called the BOM (Browser Object Model). `document` (`window.document`) is the root of the DOM tree for the loaded page. → *More:* §13.1 The DOM tree

Q145. ⋆ Explain event bubbling and capturing (the event flow). A DOM event travels in three phases: **capture** — from `window` down through the ancestors to the target; **target** — at the element itself; **bubble** — back up from the target to `window`. Listeners run in the bubble phase by default; pass `{ capture: true }` to run in the capture phase. Most events bubble; some don't (`focus`, `blur`, `mouseenter`, `load` — use `focusin` / `focusout` to get bubbling focus events). → *More:* §13.6 Propagation

Q146. ⋆ What is event delegation, and why use it? Instead of a listener on every child, put one listener on a common ancestor and use `event.target.closest(selector)` to find which child was hit. It works because events bubble. Benefits: one listener instead of thousands (less memory), and it automatically works for children added later. Check the matched element is inside the container, and remember non-bubbling events need capture.

```
list.addEventListener('click', (e) => {
  const btn = e.target.closest('button.delete');
  if (btn && list.contains(btn)) removeItem(btn.dataset.id);
});
```

→ *More:* §13.7 Event delegation

Q147. ⋆ What is the difference between `event.target` and `event.currentTarget`? `target` is the element where the event originated (the innermost element clicked). `currentTarget` is the element whose listener is running now — the one you called `addEventListener` on. With delegation they differ; `currentTarget` is `null` after the handler returns (so log it inside the handler). → *More:* §13.6

Q148. ⋆ `preventDefault()` vs `stopPropagation()` vs `stopImmediatePropagation()` vs `return false`? `preventDefault()` cancels the browser's **default action** (following a link, submitting a form, checking a checkbox) but the event still propagates. `stopPropagation()` stops the event moving to further elements, but other listeners on the current element still run. `stopImmediatePropagation()` also stops those. `return false` does nothing in `addEventListener`; only in an inline `onclick` property handler does it act like `preventDefault()` (and in jQuery, like both). → *More:* §13.8 Default actions and `preventDefault` · §13.6

Q149. What is the difference between an attribute and a property? An **attribute** is written in the HTML and lives on the element as a string (`getAttribute('value')`). A **property** is a field of the DOM object (`input.value`), with a real type. Many are linked at first, but some diverge: for an `<input>`, the `value` attribute is the **initial** value while the `value` property is the **current** text the user typed. `data-*` attributes map to `element.dataset`. → *More:* §13.4

Q150. ⋆ What is the difference between `DOMContentLoaded` and `load`? Between `async` and `defer` scripts? `DOMContentLoaded` fires when the HTML is parsed and the DOM is built (deferred scripts have run); `load` fires later, when images, stylesheets and iframes have also finished. A plain `<script>` blocks parsing while it downloads and runs. `defer` downloads in parallel and runs after parsing, in document order — the right default for app scripts. `async` downloads in parallel and runs as soon as it arrives, in any order — for independent scripts like analytics. Module scripts are deferred by default. → *More:* §13.12 When scripts run

Q151. `querySelectorAll` vs `getElementsByClassName`: what is the difference? `querySelectorAll` takes any CSS selector and returns a **static** `NodeList` (a snapshot; later DOM changes don't affect it), which has `forEach`. `getElementsByClassName` / `getElementsByTagName` return a **live** `HTMLCollection` that updates as the DOM changes (so looping while removing elements can skip items) and has no `forEach`. `querySelector` returns the first match or `null`. → *More:* §13.2 Selecting and traversing

Q152. ⋆ `innerHTML` vs `textContent` vs `innerText`? `innerHTML` reads/writes the element's HTML markup — setting it parses HTML, so untrusted input causes XSS, and it destroys and recreates child nodes (losing listeners). `textContent` reads/writes raw text of all descendants, including hidden ones, and

never parses HTML — use it for user data. `innerText` returns the **rendered** text (respects CSS, skips hidden elements) and forces a layout to compute it, so it is slower. → *More:* §13.3 · §14.10 XSS

Q153. How do you add many elements to the page efficiently? Build them off-page and insert once: append to a `DocumentFragment` (a lightweight container whose children move into the DOM in one operation) or call `parent.append(...nodes)` / `replaceChildren(...nodes)` once. This causes one layout instead of one per element. For very long lists, render only the visible rows (virtualisation). → *More:* §13.3 · §13.11

Q154. ⋆ What is the difference between reflow and repaint? What is layout thrashing? Reflow (layout) recalculates the size and position of elements — triggered by changing geometry (width, font, adding nodes). **Repaint** redraws pixels without layout changes (colour, visibility). Reflow is more expensive and may cascade. **Layout thrashing** is interleaving DOM writes with reads of layout properties (`offsetHeight`, `getBoundingClientRect`) in a loop, which forces a synchronous reflow each time. Fix: batch all reads, then all writes; animate `transform` / `opacity` (often handled by the compositor without layout or paint); use `requestAnimationFrame`. → *More:* §13.11

Q155. Why remove event listeners? How? A listener keeps its callback — and everything the callback closes over — alive while the target exists, so listeners on long-lived targets (`window`, `document`) from removed components leak memory and keep running. Remove with `removeEventListener(type, sameFunctionReference, sameCaptureFlag)` (an inline arrow can't be removed), use `{ once: true }` for one-shot listeners, or pass `{ signal }` and call `controller.abort()` to remove many at once. → *More:* §13.5 Events and listeners

Q156. What are passive event listeners? `addEventListener('touchstart', fn, { passive: true })` promises the browser that `fn` will not call `preventDefault()`, so it can start scrolling immediately instead of waiting for the handler. Browsers make `touchstart` / `touchmove` / `wheel` listeners on `window`, `document` and `body` passive by default. → *More:* §13.5

Q157. How do you create and dispatch a custom event? `el.dispatchEvent(new CustomEvent('cart:add', { detail: { id: 7 }, bubbles: true }))`, and listen with `addEventListener('cart:add', (e) => e.detail.id)`. Set `bubbles: true` if ancestors should hear it (custom events don't bubble by default). Dispatch is **synchronous**: listeners run before `dispatchEvent` returns. Plain `EventTarget` can be subclassed for non-DOM event buses. → *More:* §13.5

Q158. What is `MutationObserver`? An API that calls your callback (as a microtask, batched) when a DOM subtree changes: children added/removed, attributes changed, text changed. It replaced the slow, synchronous mutation events. Uses: reacting to DOM changes made by third-party code, auto-initialising widgets inserted later. Call `disconnect()` when done. → *More:* §13.10

Q159. How do you handle forms with JavaScript (submit, validation)? Listen for `submit` on the form (not `click` on the button, so the Enter key works too), call `preventDefault()` to stop the page reload, read values with `new FormData(form)`, and send them with `fetch`. Use HTML constraints (`required`, `type="email"`, `pattern`, `min`) and the Constraint Validation API (`checkValidity()`, `reportValidity()`, `setCustomValidity()`, `input.validity`) before custom code. Always validate again on the server. → *More:* §13.9 Forms

Q160. What are Web Components and the Shadow DOM? Web Components are the browser's native component model: **custom elements** (`class MyCard extends HTMLElement`, registered with `customElements.define('my-card', MyCard)`), **Shadow DOM** (`this.attachShadow({ mode: 'open' })` — a hidden subtree whose styles and IDs are isolated from the page), and `<template>` / `<slot>` for content. They work with any framework or none.

---

9. Browser APIs, storage, networking and security

Q161. ⋆ What are the differences between cookies, `localStorage` and `sessionStorage`? Cookies (~4 KB each) are sent to the server with every matching request; they can be `HttpOnly` (invisible to JavaScript), `Secure`, `SameSite`, and expire when you choose — use them for sessions. **`localStorage`** (~5 MB per origin, strings only, synchronous) persists until cleared and is never sent automatically. **`sessionStorage`** is the same but per tab, cleared when the tab closes. JavaScript can read both storages, so never put tokens or secrets there if XSS is possible. For large or structured data use IndexedDB. → *More:* §14.3 Web Storage · §14.4 Cookies and IndexedDB

Q162. What is the `storage` event? When one tab changes `localStorage`, every **other** tab of the same origin receives a `storage` event on `window` with `key`, `oldValue`, `newValue`. It is a simple way to sync tabs (e.g. log out everywhere); `BroadcastChannel` is the more general tool. → *More:* §14.3

Q163. What is IndexedDB, and when would you use it? An asynchronous, transactional key-value database in the browser that stores structured data (objects, files, blobs) with indexes, and can hold far more than `localStorage` (a share of free disk space). Use it for offline apps, caching API data, and drafts. Its raw API is event-based and verbose, so most people use a small promise wrapper. → *More:* §14.4

Q164. What do the cookie attributes `HttpOnly`, `Secure` and `SameSite` do? `HttpOnly` hides the cookie from `document.cookie`, so an XSS script cannot steal it. `Secure` sends it only over HTTPS. `SameSite` controls sending on cross-site requests: `Strict` never, `Lax` (the browsers' default when the attribute is missing, in Chrome and Edge) only on top-level navigations with safe methods like a link click, `None` always (requires `Secure`). Also: `Max-Age`/`Expires`, `Path`, `Domain`; the `__Host-` prefix forces `Secure`, `Path=/` and no `Domain`. → *More:* §14.4

Q165. What is the same-origin policy? A browser rule: a page may only **read** responses and data from the same **origin** — the same scheme, host and port (`https://app.com:443`). Other origins can still be sent requests (forms, images, scripts, `fetch`) but scripts cannot read the response unless the server allows it with CORS. It stops a malicious site from reading your bank's pages using your cookies. → *More:* §14.2

Q166. ⋆ What is CORS? What is a preflight request? CORS (Cross-Origin Resource Sharing) lets a **server** opt in to cross-origin reads via response headers like `Access-Control-Allow-Origin: https://app.com`. For "non-simple" requests — methods other than GET/HEAD/POST, custom headers such as `Authorization`, or `Content-Type: application/json` — the browser first sends an `OPTIONS` **preflight** asking permission, and only sends the real request if the server answers with matching `Access-Control-Allow-*` headers. With cookies, the server must send `Access-Control-Allow-Credentials: true` and an exact origin (not `*`). CORS is enforced by the browser; it is not server security, and fixing a CORS error means changing the server (or using a same-origin proxy). → *More:* §14.2

Q167. ⋆ What is XSS, and how do you prevent it? Cross-site scripting: an attacker gets their script to run in your page — **stored** (saved in the DB and shown to others), **reflected** (in a URL echoed back) or **DOM-based** (your JS writes untrusted data into the page). The script can then do anything the user can, including reading tokens. Prevention: insert user data as text (`textContent`, framework escaping), never into `innerHTML`/`eval`/`href="javascript:"`; sanitise HTML you must render with DOMPurify; set a strict **Content Security Policy**; keep session cookies `HttpOnly`; consider Trusted Types. → *More:* §14.10

Q168. ⋆ What is CSRF, and how do you prevent it? Cross-site request forgery: a malicious site makes the victim's browser send a request (e.g. a hidden form POST) to your site, and the browser attaches the victim's cookies, so the action runs as them. Prevention: `SameSite=Lax` or `Strict` cookies; a **CSRF token** (a random value in the page that the server checks on every state-changing request); checking the `Origin`/`Sec-Fetch-Site` header; never changing state on GET. APIs authenticated with an `Authorization` header (not cookies) are not exposed to CSRF. → *More:* §14.11

Q169. What is a Content Security Policy (CSP)? An HTTP response header (`Content-Security-Policy`) that tells the browser which sources of scripts, styles, images and connections are allowed. A strict policy (scripts only from your origin or with a per-response **nonce**, no inline scripts, no `eval`) means that even if an attacker injects markup, their script won't run. It is defence in depth against XSS; also `frame-ancestors` blocks clickjacking. → *More:* §14.12

Q170. Where should you store a JWT or auth token in the browser? The safest common choice is an `HttpOnly; Secure; SameSite` cookie set by the server: JavaScript (and therefore XSS) cannot read it, and `SameSite` plus CSRF tokens handle CSRF. `localStorage` is easy but any XSS can steal the token. Another pattern keeps a short-lived access token **in memory** and a refresh token in an `HttpOnly` cookie. Either way, XSS prevention comes first. → *More:* §14.4

Q171. What is clickjacking? What is prototype pollution? Clickjacking: your page is loaded in an invisible iframe on an attacker's site so the user clicks your buttons unknowingly; block framing with `Content-Security-Policy: frame-ancestors 'self'` (or `X-Frame-Options: DENY`). **Prototype pollution**: code that deep-merges untrusted JSON writes a key like `__proto__` or `constructor.prototype`, adding properties to `Object.prototype` and so to every object. Prevent it by rejecting those keys, using `Object.create(null)` or `Map` for dictionaries, and validating input against a schema. → *More:* §14.12 · §17.4.4 Q32

Q172. ⋆ How does `fetch` work? Does it reject on a 404? `fetch(url, options)` returns a promise of a `Response`. It rejects **only on network failure** (or abort/CORS failure); HTTP errors like 404 or 500 still fulfil, so check `response.ok` (status 200–299) or `response.status`. Reading the body is a second async step: `await res.json()`, `text()`, `blob()`; the body can be read only once. Options: `method`, `headers`, `body` (stringify JSON yourself), `credentials`, `signal`. Unlike `XMLHttpRequest`, it is promise-based and supports streaming.

```
const res = await fetch('/api/users', { signal: AbortSignal.timeout(5000) });
if (!res.ok) throw new Error(`HTTP ${res.status}`);
const users = await res.json();
```

→ *More:* §14.1 `fetch` and HTTP

Q173. What are Web Workers? What can't they do? A Web Worker runs a script on a separate background thread, so heavy computation (parsing, image processing) doesn't freeze the UI. It communicates with the page by `postMessage` (data is copied using structured clone, or **transferred** for `ArrayBuffer`s). Workers have no access to the DOM, `window` or `localStorage`, but they can use `fetch`, timers, IndexedDB and WebSockets. → *More:* §14.8 Web Workers

Q174. What is a service worker? What is a PWA? A service worker is a background script that sits between your page and the network: it can intercept `fetch` requests and answer from a cache (offline support), receive push messages and do background sync. It has no DOM access, requires HTTPS, and has its own lifecycle (install, activate, then controls pages). A **PWA** (progressive web app) is a website with a service worker and a web app manifest, so it can work offline and be installed like an app. → *More:* §14.8

Q175. How does client-side routing work in an SPA? A single-page application changes the URL without a page reload using the **History API**: `history.pushState(state, '', '/path')` adds an entry and `replaceState` changes the current one; the app then renders the view for the new path. Back/forward fire a `popstate` event, which the app handles by rendering the matching view. The server must return the app's HTML for every route so direct links work. → *More:* §14.7 The History API

Q176. How do you lazy-load images or implement infinite scroll? Use `IntersectionObserver`: it calls you asynchronously when a target element enters or leaves the viewport (with a `rootMargin` to start early), without scroll listeners or layout reads. For images, `<img loading="lazy">` is built in. For infinite scroll, observe a sentinel element after the last item and load the next page when it becomes visible (with a guard so only one load runs at a time). → *More:* §14.9

Q177. What is `requestAnimationFrame`? `requestAnimationFrame(cb)` runs `cb` just before the browser's next repaint (usually 60 times a second, matching the display), with a high-resolution timestamp. Use it for animations and DOM writes tied to frames instead of `setTimeout`: it is synchronised with rendering and paused in background tabs. → *More:* §14.5

Q178. How do you read query-string parameters? `new URL(location.href).searchParams.get('q')` or `new URLSearchParams(location.search).get('q')`. `URLSearchParams` also builds query strings with correct encoding (`params.set('q', 'a b&c')`). Manually, use `encodeURIComponent` for a single value; `encodeURI` is for a whole URL and leaves `&`, `?`, `/` alone. → *More:* §14.6 `URL` and `URLSearchParams`

Q179. What is `postMessage`, and how do you use it safely? `otherWindow.postMessage(data, targetOrigin)` sends data between windows, iframes or workers of different origins. Always pass an exact `targetOrigin` (not `'*'`) so data isn't delivered to the wrong site, and in the `message` handler check `event.origin` against an allow-list and validate `event.data` before using it.

Q180. Polling vs long polling vs WebSockets vs Server-Sent Events? Polling asks the server every few seconds (simple, wasteful, delayed). **Long polling** keeps a request open until there is news, then reconnects. **Server-Sent Events** (`EventSource`) is a one-way server-to-browser stream over plain HTTP with automatic reconnect — good for notifications and feeds. **WebSockets** give a persistent two-way connection — good for chat, games, collaboration. → *More:* §14.1

Q181. What is feature detection, and why prefer it over user-agent sniffing? Feature detection checks whether the API exists before using it (`if ('IntersectionObserver' in window)`, `CSS.supports(…)`), then uses it or a fallback/polyfill. User-agent sniffing guesses features from the browser name string, which is easy to spoof, frozen or reduced in modern browsers, and wrong as soon as a browser adds the feature. → *More:* §10.12

10. Performance and memory

Q182. How does garbage collection work in JavaScript? Memory is freed automatically. Engines use **reachability**: starting from roots (globals, the current call stack, active closures), everything reachable through references is kept; everything else is garbage (**mark-and-sweep**). Cycles are not a problem — two objects pointing at each other but unreachable are collected. V8 is **generational**: new objects go into a small young area collected often and cheaply; survivors are promoted to the old area, collected less often. → *More:* §17.4.7 Q51

Q183. ⋆ What causes memory leaks in JavaScript? How do you find them? Something keeps references alive that you no longer need: forgotten event listeners and `setInterval`s, growing caches/maps with no limit, closures holding large data, detached DOM nodes still referenced from JS, accidental globals, unresolved promises holding resources. Find them with DevTools Memory: take heap snapshots before and after repeating an action and compare, look for detached elements, or use the allocation timeline. Fix by cleaning up (remove listeners, clear timers, `AbortController`), bounding caches (LRU), and using `WeakMap`. → *More:* §17.4.7 Q52

Q184. What are `WeakRef` and `FinalizationRegistry`? `WeakRef(obj)` holds a reference that doesn't stop garbage collection; `deref()` returns the object or `undefined` once it's gone — useful for caches of large objects. `FinalizationRegistry` lets you register a callback that may run after an object is collected — for cleaning external resources. Both are non-deterministic (GC timing is unpredictable, callbacks may never run), so don't base program logic on them. → *More:* §8.3 · §17.4.7 Q53

Q185. ⋆ What is the difference between debouncing and throttling? When do you use each? Debounce waits until calls stop for a quiet period, then runs once — for search-as-you-type, auto-save, resize-end. **Throttle** runs at most once per interval while calls continue — for scroll position, mouse move, rate-limited API calls, infinite-scroll checks. Both reduce how often an expensive handler runs. → *More:* §17.3 Debounce · §17.3 Throttle

Q186. ⋆ How would you improve the performance of a slow web page's JavaScript? Measure first (Lighthouse, Performance panel, Core Web Vitals). Then: ship less JS (code splitting, lazy loading with dynamic `import()`, tree shaking, drop heavy libraries); load scripts with `defer`; break **long tasks** (anything over 50 ms blocks input) into chunks with `scheduler.yield()` or `setTimeout`, or move them to a Web Worker; avoid layout thrashing; debounce/throttle frequent handlers; use event delegation; virtualise long lists; cache network results; memoise expensive pure computations. → *More:* §13.11 · §10.11

Q187. What are Core Web Vitals? Google's user-centred metrics: **LCP** (Largest Contentful Paint — loading; good ≤ 2.5 s), **INP** (Interaction to Next Paint — responsiveness to clicks and keys; good ≤ 200 ms; it replaced FID in 2024), and **CLS** (Cumulative Layout Shift — visual stability; good ≤ 0.1). JavaScript mostly hurts INP (long tasks on the main thread) and LCP (render-blocking or client-rendered content).

Q188. How do you measure the performance of code? `performance.now()` gives a high-resolution timestamp for timing a piece of code; `performance.mark`/`measure` create named entries visible in DevTools; `console.time`/`timeEnd` for quick checks. For pages, use the Performance panel and Lighthouse; for real users, `PerformanceObserver` (long tasks, paint and web-vitals entries). In Node, `node --cpu-prof` or `--inspect` profiles. Benchmark with realistic data and warm-up, since the JIT optimises hot code. → *More:* §11.7 · §11.9

Q189. How does V8 run your code fast? What are hidden classes and inline caching? V8 first interprets bytecode (Ignition), collects type feedback, then compiles hot functions to optimised machine code (TurboFan, with intermediate tiers) — **JIT** (just-in-time) compilation. Objects get **hidden classes** (shapes) describing their property layout; objects created with the same properties in the same order share a shape. **Inline caches** remember the shape seen at each property access, making repeat accesses fast. So: initialise all properties in the constructor in the same order, don't `delete` properties, keep functions monomorphic (called with one shape of argument). If assumptions break, V8 **deoptimises** back to slower code. → *More:* §1.3 JavaScript engines

Q190. How do you render a list of 100,000 items without freezing the page? Don't create 100,000 DOM nodes: **virtualise** — render only the rows in (and just around) the viewport and reposition them on scroll (libraries: TanStack Virtual, react-window), or paginate. If data processing is heavy, do it in

a Web Worker or in chunks that yield to the event loop. CSS `content-visibility: auto` lets the browser skip rendering off-screen sections.

Q191. A page freezes while the user types in a search box. How do you debug and fix it? Record the interaction in the Performance panel and look for long tasks on the main thread. Usual causes: filtering a huge list or calling an API on every keystroke, and re-rendering everything. Fixes: debounce the input; cancel stale requests with `AbortController` (and ignore out-of-order responses); memoise the filtering; virtualise the result list; move heavy matching to a worker. → *More:* §12.13 Real-world async patterns

---

11. Node.js

Q192. ⋆ What is Node.js? Is it single-threaded? Node is a JavaScript runtime outside the browser: Google's V8 engine plus **libuv** (a C library providing the event loop, async I/O and a thread pool) plus core modules (`fs`, `http`, `crypto`, `stream`…). Your JavaScript runs on one main thread, but libuv uses the OS's async facilities for network I/O and a **thread pool** (4 threads by default, `UV_THREADPOOL_SIZE`) for file system, DNS lookup, `crypto` and `zlib` work. With `worker_threads` you can run JavaScript on extra threads too. → *More:* §15.1 · §15.2

Q193. ⋆ Explain the phases of Node's event loop. Each turn goes through phases: **timers** (`setTimeout`/`setInterval` callbacks that are due), **pending callbacks** (some deferred system errors), **idle/prepare** (internal), **poll** (wait for and run I/O callbacks), **check** (`setImmediate` callbacks), **close callbacks** (`'close'` events). Between every callback, Node drains the `process.nextTick` queue, then the promise microtask queue. → *More:* §12.4 Node's event loop

Q194. ⋆ Why is blocking the event loop bad in Node? How do you handle CPU-heavy work? One thread serves every request; a 2-second synchronous loop (big JSON parse, image resize, `fs.readFileSync` in a request handler, a catastrophic regex) delays **all** other clients by 2 seconds. Fixes: use async APIs; split work into chunks; move CPU work to `worker_threads` or a separate service/queue; scale across cores with multiple processes (cluster, containers). → *More:* §15.2

Q195. `worker_threads` vs `child_process` vs `cluster`? `worker_threads`: extra JavaScript threads in the same process, each with its own V8 and event loop, communicating by messages or `SharedArrayBuffer` — for CPU-heavy JS. `child_process` (`spawn`, `exec`, `fork`): separate OS processes, for running other programs or isolating work. `cluster`: forks several Node processes that share one server port to use all CPU cores (often replaced by a process manager or container orchestration). → *More:* §15.2 · §15.11

Q196. ⋆ What are streams? What is backpressure? A stream processes data piece by piece (chunks) instead of loading it all into memory. Types: **Readable** (file read, HTTP request), **Writable** (file write, HTTP response), **Duplex** (TCP socket) and **Transform** (gzip). **Backpressure** happens when the reader is faster than the writer: `write()` returns `false` and you should wait for `'drain'`. `pipeline(src, transform, dest)` from `node:stream/promises` handles backpressure, errors and cleanup for you — use it instead of `.pipe()`.

```
import fs from 'node:fs';
import zlib from 'node:zlib';
import { pipeline } from 'node:stream/promises';
await pipeline(fs.createReadStream('big.log'), zlib.createGzip(), fs.createWriteStream('big.log.gz'));
```

→ *More:* §15.9 Streams

Q197. What is a `Buffer`? A `Buffer` is Node's fixed-size chunk of raw bytes (a subclass of `Uint8Array`), used for binary data: files, network packets, images, crypto. Convert with `Buffer.from('hi', 'utf8')` and `buf.toString('base64')`. `Buffer.alloc(n)` gives zero-filled memory; `allocUnsafe` is faster but may contain old data. → *More:* §15.7

Q198. What is `EventEmitter`? Node's built-in publish/subscribe class (`node:events`): `on(event, fn)` registers listeners, `emit(event, ...args)` calls them **synchronously** in registration order, `once` listens once, `off` removes. Streams, servers and sockets are emitters. Emitting `'error'` with no listener throws, so always handle `'error'`. Warns after 10 listeners per event (a common leak hint). `events.once(emitter, 'x')` gives a promise. → *More:* §15.8 Events: `EventEmitter` · Q258 below

Q199. How do you read command-line arguments and environment variables? What do exit codes mean? `process.argv` is the array `[nodePath, scriptPath, ...args]` (`util.parseArgs` parses flags). `process.env.NAME` reads environment variables (always strings); Node can load a `.env` file with `--env-file=.env`. Exit code `0` means success, anything else failure; set `process.exitCode = 1` and let the process end naturally, rather than `process.exit()`, which cuts off pending I/O. → *More:* §15.6 `process`

Q200. `fs.readFileSync` vs `fs.readFile` vs `fs/promises`? `readFileSync` blocks the thread until the file is read — fine in startup scripts and CLIs, bad inside a server request. `fs.readFile(path, cb)` is the callback async version; `import { readFile } from 'node:fs/promises'` is the promise version for `await` — the modern default. For large files use streams. → *More:* §15.4 Files with `node:fs/promises`

Q201. How do you create an HTTP server in Node without a framework? `http.createServer((req, res) => {...}).listen(3000)`. The handler reads `req.method`, `req.url` (parse with `new URL(req.url, 'http://localhost')`), headers and the body stream, then sets `res.statusCode`, headers and calls `res.end(body)`. Frameworks (Express, Fastify, Hono) add routing, middleware (functions that run in a chain before your handler), body parsing and error handling on top. → *More:* §15.10

Q202. What is middleware (as in Express)? A function `(req, res, next)` in a pipeline that runs for each request: it can read or modify the request/response, end the response, or call `next()` to pass control on (`next(err)` jumps to error-handling middleware, which has four parameters). Used for logging, auth, body parsing, CORS, rate limiting. Order of registration is the order of execution. → *More:* §15.10

Q203. What is the difference between `dependencies`, `devDependencies` and `peerDependencies`? `npm` vs `npx`? `dependencies` are needed at run time; `devDependencies` only for development and builds (test runners, bundlers, types); `peerDependencies` are packages a library expects the host app to provide (a React component library and `react`). `npm` installs and manages packages and runs scripts; `npx` runs a package's binary without a global install. → *More:* §10.9 npm and `package.json`

Q204. How does JavaScript in Node differ from JavaScript in the browser? Same language and engine family; different host APIs. Node has no DOM, `window` or `document`, but has file system, processes, sockets, `Buffer`, `require` and full OS access; the browser is sandboxed and has the DOM, `localStorage`, rendering. The global object is `global` vs `window` (`globalThis` in both). Many web APIs now exist in both: `fetch`, `URL`, `AbortController`, `structuredClone`, web streams, `crypto.subtle`. → *More:* §15.1

Q205. How do you scale a Node.js application? Keep each process non-blocking (async I/O, offload CPU work). Run several stateless processes per machine (one per core) behind a load balancer, and several machines horizontally; keep state (sessions, caches) in shared stores like Redis or a database. Use queues for slow background jobs, streams for big payloads, caching and connection pooling for the database. → *More:* §15.2 · System Design Fundamentals

Q206. What are Deno and Bun? Alternative JavaScript runtimes. **Deno** (by Node's creator) uses V8 and Rust, is secure by default (explicit permissions for files and network), runs TypeScript natively and bundles a formatter, linter and test runner. **Bun** uses JavaScriptCore (Safari's engine) and Zig, aims for speed, and bundles a package manager, bundler and test runner. Both are largely Node-compatible; Node remains the mainstream production choice. Node has closed some gaps: it runs `.ts` files by type stripping (Q216) and has an opt-in, now stable **permission model** (`node --permission --allow-fs-read=./data app.js`). → *More:* §15.12

---

12. TypeScript basics

Q207. ⋆ What is TypeScript, and why use it? TypeScript is JavaScript plus a static type system: you annotate (or let it infer) types, and the compiler `tsc` reports type errors **before** the code runs; the types are then erased and plain JavaScript runs. Benefits: catches typos and wrong-shape bugs early, much better editor autocomplete and refactoring, types document APIs. Costs: a build/check step and learning curve. It does not check data at run time (validate external input with a schema library). → *More:* §16.8

Q208. ⋆ What is the difference between `interface` and `type`? Both describe object shapes and are mostly interchangeable. `interface` can be **reopened** (declaration merging — two declarations with the same name combine, used to augment library types) and extended with `extends`. `type` aliases can name **any** type — unions (`'a' | 'b'`), tuples, primitives, mapped and conditional types — and combine with `&`. Common

rule: `interface` for public object shapes, `type` for everything else; be consistent. → *More:* §16.10 Everyday types

Q209. ⋆ `any` vs `unknown` vs `never`? `any` turns type checking off for that value — anything goes, errors spread silently. `unknown` means "could be anything, so you must check before using it" — the safe type for external input (`JSON.parse`, `catch` errors). `never` is the type with no values: a function that always throws or never returns, or a branch that should be impossible (used for exhaustive `switch` checks). → *More:* §16.10 · §16.11 Narrowing

Q210. What is type narrowing? What are type guards and discriminated unions? Narrowing is TypeScript refining a union type inside a branch after a check: `typeof x === 'string'`, `x instanceof Date`, `'key' in x`, `Array.isArray(x)`, truthiness, equality. A **type guard** is a function returning `x is Type`, so its callers get narrowing. A **discriminated union** is a union of object types sharing a literal field (`kind: 'circle' | 'square'`); switching on that field narrows to the right member, and a `never` default catches missing cases. → *More:* §16.11

Q211. ⋆ What are generics? Type parameters that let one function or type work with many types while keeping them linked: `function first<T>(arr: T[]): T | undefined` returns the same type the array holds. Constraints restrict them: `<T extends { id: string }>`. Used everywhere in library types: `Array<T>`, `Promise<T>`, `Map<K, V>`. → *More:* §16.12 Generics and utility types

Q212. Name some utility types. `Partial<T>` (all optional), `Required<T>`, `Readonly<T>`, `Pick<T, 'a' | 'b'>`, `Omit<T, 'password'>`, `Record<K, V>` (object with keys K), `ReturnType<typeof fn>`, `Parameters<typeof fn>`, `Awaited<P>` (unwrap a promise), `NonNullable<T>`, `Exclude`/`Extract` (filter union members). They are built from mapped and conditional types. → *More:* §16.12

Q213. What do `keyof` and `typeof` mean in TypeScript types? In type position, `typeof value` gives the type of a JavaScript value (`type Config = typeof defaultConfig`). `keyof T` gives the union of T's keys (`keyof { a: 1; b: 2 }` is `'a' | 'b'`). Together: `function get<T, K extends keyof T>(obj: T, key: K): T[K]` — type-safe property access, where `T[K]` is an indexed access type. → *More:* §16.12

Q214. Enums vs union of string literals vs `as const`? What does `satisfies` do? A string-literal union (`type Status = 'idle' | 'loading'`) costs nothing at run time and is usually preferred. `enum` generates a JavaScript object and is not supported by Node's type stripping. An `as const` object (`const Status = { Idle: 'idle' } as const`) gives a runtime object plus literal types — the common enum replacement. `value satisfies Type` checks that a value matches a type **without** widening it, so you keep the precise inferred type (e.g. which keys exist). → *More:* §16.13

Q215. What is structural typing? TypeScript compares types by **shape**, not by name: any object with the required properties is compatible, even if declared separately ("duck typing" checked at compile time). Object literals passed directly get an extra check for unknown properties (excess property checking). → *More:* §16.10

Q216. Can Node 24 run TypeScript directly? Yes: `node app.ts` works without flags by **stripping** type annotations (stable in Node 24.12). It does no type checking, so still run `tsc --noEmit`; it supports only erasable syntax (no `enum`, namespaces with code, or parameter properties — enable `erasableSyntaxOnly`), needs `import type` for type-only imports (enforce with `verbatimModuleSyntax`), requires real `.ts` extensions in imports, ignores `tsconfig.json`, and won't strip files in `node_modules`. → *More:* §16.9

Q217. What are the pros and cons of languages that compile to JavaScript? Pros: types (TypeScript) or other features, earlier error detection, better tooling. Cons: a build step, source maps needed for debugging, possible mismatch between the language and the JavaScript it produces, an extra dependency, and team learning cost. TypeScript is the exception that became mainstream because it is "JavaScript plus types", not a different language. → *More:* §16.8

---

13. Output-prediction puzzles ("What will this print?")

Say the answer out loud **before** reading it. Each answer was checked by running the code in Node; files are ES modules (strict mode) unless the question says otherwise. Many more, with real output, are in §17.4 and §12.3.

Q218. ⋆ Timers, promises and `queueMicrotask`.

```
console.log('1');
setTimeout(() => console.log('2'), 0);
Promise.resolve().then(() => console.log('3'));
queueMicrotask(() => console.log('4'));
console.log('5');
```

`1 5 3 4 2`. Synchronous code first (1, 5); then the microtask queue in order of queuing (3, 4); then the timer task (2). → *More:* §17.4.5 Q35

Q219. ⋆ Nested tasks and microtasks.

```
console.log('A');
setTimeout(() => console.log('B'), 0);
Promise.resolve()
  .then(() => { console.log('C'); setTimeout(() => console.log('D'), 0); })
  .then(() => console.log('E'));
(async () => { console.log('F'); await null; console.log('G'); })();
console.log('H');
```

`A F H C G E B D`. The async function runs synchronously up to its first `await` (F). Microtasks queued during sync code: C, then G. Running C queues E (the next `then`) and timer D. Microtasks drain completely (C, G, E) before any timer; timers then run in the order created (B, D).

Q220. ⋆ `async`/`await` ordering.

```
async function a() { console.log('a1'); await b(); console.log('a2'); }
async function b() { console.log('b'); }
console.log('start');
a();
Promise.resolve().then(() => console.log('p'));
console.log('end');
```

`start a1 b end a2 p`. Calling `b()` runs its body synchronously. `await` of an already-fulfilled promise queues the continuation (a2) as a microtask — before `p`, which is queued later. → *More:* §17.4.5 Q36

Q221. ⋆ The promise executor.

```
const p = new Promise((resolve) => { console.log('A'); resolve('B'); console.log('C'); });
p.then(console.log);
console.log('D');
```

`A C D B`. The executor runs synchronously, and `resolve` does not stop it (C still prints). `then` callbacks are always asynchronous.

Q222. ⋆ Errors flowing through a chain.

```
Promise.resolve(1)
  .then((x) => { throw new Error('boom ' + x); })
  .then(() => console.log('skipped'))
  .catch((e) => { console.log(e.message); return 2; })
  .then((x) => console.log('after', x))
  .finally(() => console.log('finally'));
```

`boom 1`, `after 2`, `finally`. A throw skips `then` handlers until a `catch`; a `catch` that returns a value recovers the chain; `finally` receives no value and passes the previous result through.

Q223. What does an `async` function return?

```
async function f() { return 1; }
console.log(f());
```

`Promise { 1 }` (in Node). An `async` function always returns a promise; use `await f()` or `f().then(…)` to get `1`.

Q224. ⋆ `var` hoisting inside a function.

```
var x = 1;
function f() { console.log(x); var x = 2; }
f();
```

`undefined`. The inner `var x` is hoisted to the top of `f` and shadows the outer `x`, so at the `console.log` the local `x` exists but is still `undefined`. With `let x = 2` it would throw `ReferenceError` (TDZ). → *More:* §17.4.2 Q16

Q225. Function vs variable hoisting (a classic script).

```
console.log(typeof foo);
var foo = 1;
function foo() {}
console.log(typeof foo);
```

`function`, then `number`. Both declarations are hoisted; the function declaration wins at the start. Then the assignment `foo = 1` runs and replaces it.

Q226. ⋆ `this` in methods, arrows and nested functions.

```
const obj = {
  name: 'obj',
  regular() { return this.name; },
  arrow: () => this?.name,
  nested() { return [function () { return this?.name; }(), (() => this.name)()]; },
};
console.log(obj.regular(), obj.arrow(), obj.nested());
```

`obj undefined [ undefined, 'obj' ]`. `regular` is called on `obj`. The arrow takes `this` from module top level, which is `undefined`. Inside `nested`, the inner regular function is a plain call (`this` is `undefined` in strict code), while the inner arrow inherits `nested`'s `this`. → *More:* §17.4.3 Q25

Q227. A detached method (a CommonJS file, sloppy mode).

```
const user = { name: 'Ann', greet() { return 'Hi ' + this?.name; } };
const g = user.greet;
console.log(user.greet(), g());
```

`Hi Ann Hi undefined`. `g()` is a plain call: in sloppy mode `this` is the global object, which has no `name`. In strict mode `this` would be `undefined` (`this?.name` avoids the crash here; `this.name` would throw).

Q228. Class field arrow vs prototype method.

```
class Counter { count = 0; inc() { this.count++; } incArrow = () => { this.count++; }; }
const c = new Counter();
const { inc, incArrow } = c;
incArrow();
try { inc(); } catch (e) { console.log(e.constructor.name); }
console.log(c.count);
```

`TypeError`, then `1`. The arrow field is created per instance and captures it; the prototype method loses `this` when extracted (class bodies are strict, so `this` is `undefined`).

Q229. Closures keep separate state.

```
function make() { let c = 0; return { inc: () => ++c, get: () => c }; }
const a = make(), b = make();
a.inc(); a.inc(); b.inc();
console.log(a.get(), b.get());
```

`2 1`. Each call to `make` creates a new `c`; the two functions returned by one call share it.

Q230. ⋆ Coercion with `+` and `-`.

```
console.log("5" + 3 - 3, "5" - 3 + 3);
console.log([1, 2, 3] + [4], null + 1, undefined + 1);
console.log(1 < 2 < 3, 3 > 2 > 1);
```

`50 5` — `"5" + 3` is `"53"`, minus 3 is `50`; `"5" - 3` is `2`, plus 3 is `5`. `1,2,34 1 NaN` — arrays become strings; `null` becomes `0`, `undefined` becomes `NaN`. `true false` — `1 < 2` is `true`, `true < 3` is `1 < 3`; `3 > 2` is `true`, `true > 1` is `1 > 1`. → *More:* §17.4.1 Q1

Q231. `Number()` vs `parseInt()` on odd inputs.

```
console.log(Number(''), Number(' 12 '), Number('12px'), Number(null), Number(undefined));
console.log(Number([]), Number([5]), Number([1, 2]), parseInt('12px'), parseInt(0.0000005));
```

`0 12 NaN 0 NaN`, then `0 5 NaN 12 5`. Arrays convert via their string form (`''`, `'5'`, `'1,2'`). `parseInt(0.0000005)` is `5` because the number is first turned into the string `'5e-7'`, and parsing stops at `e`.

Q232. An object that equals three different numbers.

```
let count = 0;
const obj = { valueOf() { return ++count; } };
console.log(obj == 1, obj == 2, obj == 3);
```

`true true true`. `==` with a number converts the object with `valueOf` each time, and this one returns a new number on every call. (`===` would give `false` three times.)

Q233. ⋆ Default `sort`.

```
console.log([3, 20, 100].sort(), ['b', 'a', 'C'].sort());
```

`[ 100, 20, 3 ] [ 'C', 'a', 'b' ]`. Elements are compared as strings by UTF-16 code unit: `'1' < '2' < '3'`, and uppercase letters come before lowercase. Use `(a, b) => a - b` and `localeCompare`.

Q234. `map` with a block body.

```
console.log([1, 2, 3].map((n) => { n * 2; }));
```

`[ undefined, undefined, undefined ]`. Braces make a block body, which needs an explicit `return`. Write `(n) => n * 2`.

Q235. Holes and `length`.

```
const arr = [1, 2, 3];
arr[10] = 11;
console.log(arr.length, arr.filter(() => true).length);
```

`11 4`. Writing index 10 extends `length` to 11 and leaves holes at 3–9; `filter` skips holes.

Q236. `includes` vs `indexOf` with `NaN`, and `indexOf` with types.

```
console.log([NaN].includes(NaN), [NaN].indexOf(NaN), [1, 2, 3].indexOf('2'));
```

`true -1 -1`. `includes` uses SameValueZero (treats `NaN` as equal to itself); `indexOf` uses `===`, so it finds neither `NaN` nor the string `'2'`.

Q237. Constructors that return values.

```
function Foo() { this.a = 1; return { b: 2 }; }
function Bar() { this.a = 1; return 5; }
console.log(new Foo().a, new Foo().b, new Bar().a);
```

`undefined 2 1`. Returning an object from a constructor replaces the new object; returning a primitive is ignored. → *More:* §17.4.3 Q28

Q238. `return` inside `finally`.

```
function f() { try { return 'try'; } finally { console.log('cleanup'); } }
function g() { try { return 'try'; } finally { return 'finally'; } }
console.log(f()); console.log(g());
```

`cleanup`, `try`, `finally`. `finally` runs before the function actually returns; a `return` in `finally` overrides the one in `try`.

Q239. Generators and `next(value)`.

```
function* gen() { const x = yield 1; console.log('got', x); yield x * 2; }
const it = gen();
console.log(it.next().value); console.log(it.next(5).value); console.log(it.next().done);
```

`1`, `got 5`, `10`, `true`. The first `next()` runs to the first `yield`. The value passed to the second `next(5)` becomes the result of that `yield` expression.

Q240. Object key order.

```
console.log(Object.keys({ b: 1, 2: 1, a: 1, 1: 1 }));
```

`[ '1', '2', 'b', 'a' ]`. Integer-like keys come first in ascending numeric order, then string keys in insertion order (and keys are always strings).

Q241. Strings are immutable.

```
const s = 'abc';
s[0] = 'z';
```

In an ES module (strict mode) this throws `TypeError: Cannot assign to read only property '0' of string 'abc'`. In sloppy mode the assignment is silently ignored and `s` stays `'abc'`.

Q242. Mixing `??` with `||`.

```
console.log(0 || null ?? 'x');
```

`SyntaxError`. The language forbids mixing `??` with `||` or `&&` without parentheses, because the intended precedence is unclear. Write `(0 || null) ?? 'x'` (gives `'x'`).

Q243. ⋆ Loop closures with `let`.

```
const fns = [];
for (let i = 0; i < 3; i++) fns.push(() => i);
console.log(fns.map((fn) => fn()));
```

`[ 0, 1, 2 ]`. `let` in a `for` header creates a new binding for each iteration; with `var` it prints `[ 3, 3, 3 ]`. → *More:* §17.4.2 Q18

Q244. Array-to-primitive comparisons.

```
console.log(String([null]), String([undefined, null]), [] == '', [0] == false);
```

An empty string, `,`, `true`, `true`. `join` turns `null`/`undefined` into empty strings; `[]` becomes `''`; `[0]` becomes `'0'`, then `0`, and `false` becomes `0`.

---

14. Coding tasks: implement-from-scratch classics

Every snippet below was run against tests. In the interview, first state the contract (inputs, outputs, edge cases), write the simple version, then mention extensions (cancel, `this`, errors, leading edge). Algorithm and data-structure problems (two pointers, sliding window, BFS, DP, heaps…) are in §17.2 Coding patterns.

Q245. ⋆ Implement `debounce`.

```
function debounce(fn, wait) {
  let timer;
  const debounced = function (...args) {
    clearTimeout(timer);                                  // every call restarts the wait
    timer = setTimeout(() => fn.apply(this, args), wait);   // keep the caller's this and latest args
  };
  debounced.cancel = () => clearTimeout(timer);
  return debounced;
}
```

Follow-ups: a `leading` option (run on the first call, then ignore until quiet), `flush()` (run the pending call now), and returning a promise of the result. → *More:* §17.3 Debounce

Q246. ⋆ Implement `throttle`.

```
function throttle(fn, wait) {
  let last = 0, timer = null, lastArgs;
  return function (...args) {
    const now = Date.now();
    lastArgs = args;
    if (now - last >= wait) { last = now; fn.apply(this, args); }          // leading call
    else if (!timer) timer = setTimeout(() => {                             // one trailing call
      last = Date.now(); timer = null; fn.apply(this, lastArgs);
    }, wait - (now - last));
  };
}
```

Runs at most once per `wait`, with a trailing call so the last value is not lost. → *More:* §17.3 Throttle

Q247. ⋆ Implement `Promise.all`.

```
function promiseAll(iterable) {
  return new Promise((resolve, reject) => {
    const items = [...iterable], results = new Array(items.length);
    let left = items.length;
    if (left === 0) return resolve(results);                // empty input resolves at once
    items.forEach((item, i) => {
      Promise.resolve(item).then((v) => {                    // also accepts plain values
        results[i] = v;                                      // keep input order
        if (--left === 0) resolve(results);
      }, reject);                                            // first rejection wins
    });
  });
}
```

→ *More:* §17.3 Promise polyfills

Q248. Implement `Promise.allSettled`.

```
const promiseAllSettled = (iterable) => promiseAll([...iterable].map((p) =>
  Promise.resolve(p).then((value) => ({ status: 'fulfilled', value }),
                          (reason) => ({ status: 'rejected', reason }))));
```

Each promise is turned into one that always fulfils with a status object, so `all` never rejects. → *More:* §17.3

Q249. ⋆ Implement `Promise.race` and `Promise.any`.

```
const promiseRace = (iterable) => new Promise((resolve, reject) => {
  for (const p of iterable) Promise.resolve(p).then(resolve, reject);     // first to settle wins
});
function promiseAny(iterable) {
  return new Promise((resolve, reject) => {
    const items = [...iterable], errors = new Array(items.length);
    let left = items.length;
    if (left === 0) return reject(new AggregateError([], 'All promises were rejected'));
    items.forEach((p, i) => Promise.resolve(p).then(resolve, (e) => {
      errors[i] = e;
      if (--left === 0) reject(new AggregateError(errors, 'All promises were rejected'));
    }));
  });
}
```

Extra calls to `resolve`/`reject` after the first are ignored, which is what makes `race` this short. `race([])` stays pending forever. → *More:* §17.3

Q250. ⋆ Implement `Function.prototype.bind`.

```
Function.prototype.myBind = function (ctx, ...preset) {
  const fn = this;
  return function bound(...later) {
    if (new.target) return new fn(...preset, ...later);   // `new` ignores the bound this
    return fn.apply(ctx, [...preset, ...later]);         // partial application
  };
};
```

→ *More:* §17.3 A `bind` polyfill

Q251. Implement `call` and `apply` without using them.

```
Function.prototype.myCall = function (ctx, ...args) {
  ctx = ctx == null ? globalThis : Object(ctx);    // box primitives
  const key = Symbol('fn');                        // unique key: no clash with ctx's own props
  ctx[key] = this;                                 // call it as a method so this === ctx
  try { return ctx[key](...args); } finally { delete ctx[key]; }
};
Function.prototype.myApply = function (ctx, args = []) { return this.myCall(ctx, ...args); };
```

This mimics sloppy-mode behaviour (`null` becomes the global object, primitives are boxed); the real `call` passes `ctx` unchanged to a strict-mode function. → *More:* §17.3

Q252. ⋆ Implement a deep clone (with circular references).

```
function deepClone(value, seen = new WeakMap()) {
  if (value === null || typeof value !== 'object') return value;    // primitives (and functions) as-is
  if (seen.has(value)) return seen.get(value);                         // cycle: reuse the copy
  if (value instanceof Date) return new Date(value);
  if (value instanceof Map) { const m = new Map(); seen.set(value, m);
    value.forEach((v, k) => m.set(deepClone(k, seen), deepClone(v, seen))); return m; }
  if (value instanceof Set) { const s = new Set(); seen.set(value, s);
    value.forEach((v) => s.add(deepClone(v, seen))); return s; }
  const copy = Array.isArray(value) ? [] : Object.create(Object.getPrototypeOf(value));
  seen.set(value, copy);
  for (const key of Reflect.ownKeys(value)) copy[key] = deepClone(value[key], seen);  // incl. symbols
  return copy;
}
```

Mention that `structuredClone` is the built-in answer, and what it cannot copy (functions, DOM nodes, prototypes). → *More:* §17.3 Deep clone

Q253. ⋆ Implement deep equality.

```
function deepEqual(a, b) {
  if (Object.is(a, b)) return true;
  if (typeof a !== 'object' || typeof b !== 'object' || a === null || b === null) return false;
  if (Array.isArray(a) !== Array.isArray(b)) return false;
  const ka = Object.keys(a), kb = Object.keys(b);
  return ka.length === kb.length && ka.every((k) => Object.hasOwn(b, k) && deepEqual(a[k], b[k]));
}
```

Extensions: `Date` / `Map` / `Set` support, and a `seen` map for cycles. → *More:* §17.3 Deep equal

Q254. ⋆ Flatten a nested array (with and without recursion).

```
const flatten = (arr, depth = Infinity) => depth < 1 ? arr.slice()
  : arr.reduce((acc, x) => (Array.isArray(x) ? acc.push(...flatten(x, depth - 1)) : acc.push(x), acc), []);

function flattenIterative(arr) {                    // no recursion: explicit stack
  const stack = [...arr], out = [];
  while (stack.length) { const x = stack.pop(); Array.isArray(x) ? stack.push(...x) : out.push(x); }
  return out.reverse();
}
```

The built-in is `arr.flat(depth)` (`flat(Infinity)` for all levels). Flattening an object to dotted keys (`{ 'a.b': 1 }`) is the same recursion over `Object.entries`. → *More:* §17.3 Flatten

Q255. ⋆ Implement `curry`.

```
function curry(fn) {
  return function curried(...args) {
    return args.length >= fn.length               // enough arguments: call the original
      ? fn.apply(this, args)
      : (...more) => curried.apply(this, [...args, ...more]);   // otherwise collect more
  };
}
const add3 = curry((a, b, c) => a + b + c);
add3(1)(2)(3); add3(1, 2)(3); add3(1)(2, 3);       // all 6
```

→ *More:* §17.3 Curry

Q256. ⋆ Implement `sum(1)(2)(3)()` (infinite currying).

```
const sum = (a) => (b) => (b === undefined ? a : sum(a + b));
sum(1)(2)(3)();   // 6
```

Variant without the final `()`: return a function with a custom `valueOf` (`f.valueOf = () => total`) so `+sum(1)(2)(3)` or `sum(1)(2) == 3` works.

Q257. ⋆ Implement `memoize`.

```
function memoize(fn, keyFn = (...args) => JSON.stringify(args)) {
  const cache = new Map();
  return function (...args) {
    const key = keyFn(...args);
    if (!cache.has(key)) cache.set(key, fn.apply(this, args));   // has(): caches undefined results too
    return cache.get(key);
  };
}
```

Discuss the key (objects as arguments → `WeakMap` or a key function), memory growth (LRU limit or TTL), and memoising async functions (cache the promise and delete it on rejection). → *More:* §17.3 Memoize

Q258. ⋆ Implement an `EventEmitter` (`on`, `off`, `once`, `emit`).

```
class EventEmitter {
  #events = new Map();                                    // name -> array of listeners
  on(name, fn) {
    if (!this.#events.has(name)) this.#events.set(name, []);
    this.#events.get(name).push(fn);
    return () => this.off(name, fn);                      // handy unsubscribe function
  }
  off(name, fn) {
    const list = this.#events.get(name) ?? [];
    this.#events.set(name, list.filter((f) => f !== fn && f.original !== fn));
    return this;
  }
  once(name, fn) {
    const wrapper = (...args) => { this.off(name, wrapper); fn.apply(this, args); };
    wrapper.original = fn;                                // so off(name, fn) also removes it
    return this.on(name, wrapper);
  }
  emit(name, ...args) {
    const list = this.#events.get(name);
    if (!list?.length) return false;
    [...list].forEach((f) => f.apply(this, args));        // copy: listeners may unsubscribe
    return true;
  }
}
```

→ *More:* §17.3 EventEmitter · §15.8

Q259. ⋆ Implement an LRU cache with O(1) `get` and `put`.

```
class LRUCache {
  #map = new Map();                          // Map remembers insertion order
  constructor(capacity) { this.capacity = capacity; }
  get(key) {
    if (!this.#map.has(key)) return -1;
    const v = this.#map.get(key);
    this.#map.delete(key); this.#map.set(key, v);          // move to "most recent" end
    return v;
  }
  put(key, value) {
    this.#map.delete(key); this.#map.set(key, value);
    if (this.#map.size > this.capacity) this.#map.delete(this.#map.keys().next().value); // evict oldest
  }
}
```

In other languages (or if asked to avoid `Map` ordering) use a hash map plus a doubly linked list. → *More:* §17.3 LRU cache

Q260. ⋆ Write polyfills for `map`, `filter` and `reduce`.

```
Array.prototype.myMap = function (cb, thisArg) {
  const out = new Array(this.length);
  for (let i = 0; i < this.length; i++) if (i in this) out[i] = cb.call(thisArg, this[i], i, this);
  return out;
};
Array.prototype.myFilter = function (cb, thisArg) {
  const out = [];
  for (let i = 0; i < this.length; i++) if (i in this && cb.call(thisArg, this[i], i, this)) out.push(this[i]);
  return out;
};
Array.prototype.myReduce = function (cb, ...init) {
  let i = 0, acc;
  if (init.length) acc = init[0];
  else { while (i < this.length && !(i in this)) i++;
    if (i >= this.length) throw new TypeError('Reduce of empty array with no initial value');
    acc = this[i++]; }
  for (; i < this.length; i++) if (i in this) acc = cb(acc, this[i], i, this);
  return acc;
};
```

Points interviewers check: skipping holes (`i in this`), passing `(value, index, array)`, `thisArg`, and `reduce` without an initial value. (Use regular functions, not arrows, so `this` is the array.)

Q261. Implement `once`.

```
function once(fn) {
  let done = false, result;
  return function (...args) {
    if (!done) { done = true; result = fn.apply(this, args); }
    return result;                      // later calls return the first result
  };
}
```

(The LeetCode version returns `undefined` on later calls — read the spec.)

Q262. Implement `sleep` and `promisify`.

```
const sleep = (ms) => new Promise((resolve) => setTimeout(resolve, ms));

const promisify = (fn) => (...args) => new Promise((resolve, reject) =>
  fn(...args, (err, value) => (err ? reject(err) : resolve(value))));   // Node error-first callback
```

Node ships `util.promisify` and `node:timers/promises` (`await setTimeout(100)`).

Q263. ⋆ Implement retry with exponential backoff.

```
async function retry(fn, { retries = 3, delay = 100 } = {}) {
  for (let attempt = 0; ; attempt++) {
    try { return await fn(attempt); }
    catch (err) {
      if (attempt >= retries) throw err;              // out of attempts: give up
      await sleep(delay * 2 ** attempt);              // 100, 200, 400 ms…
    }
  }
}
```

Mention adding random **jitter** so many clients don't retry in lockstep, retrying only transient errors (timeouts, 503), and an `AbortSignal`.

Q264. ⋆ Run async tasks with a concurrency limit (a promise pool).

```
async function runWithLimit(tasks, limit) {          // tasks: functions returning promises
  const results = new Array(tasks.length);
  let next = 0;
  async function worker() {
    while (next < tasks.length) { const i = next++; results[i] = await tasks[i](); }
  }
  await Promise.all(Array.from({ length: Math.min(limit, tasks.length) }, worker));
  return results;
}
```

`limit` workers each pull the next task as soon as they finish one; `next++` is safe because JavaScript runs one piece of code at a time. → *More:* §12.13 Real-world async patterns

Q265. Implement the `new` operator and `Object.create`.

```
function myNew(Ctor, ...args) {
  const obj = Object.create(Ctor.prototype);                // 1. link the prototype
  const result = Ctor.apply(obj, args);                     // 2. run the constructor
  const isObject = result !== null && (typeof result === 'object' || typeof result === 'function');
  return isObject ? result : obj;                           // 3. object return wins
}
function objectCreate(proto) { function F() {} F.prototype = proto; return new F(); }
```

→ *More:* §9.4

Q266. Implement `instanceof`.

```
function myInstanceOf(obj, Ctor) {
  if (obj === null || (typeof obj !== 'object' && typeof obj !== 'function')) return false;
  for (let p = Object.getPrototypeOf(obj); p; p = Object.getPrototypeOf(p)) {
    if (p === Ctor.prototype) return true;
  }
  return false;
}
```

→ *More:* §9.7

Q267. Implement lodash-style `get(obj, 'a.b[0].c', fallback)`.

```
function get(obj, path, fallback) {
  const keys = Array.isArray(path) ? path : path.replace(/\[(\w+)\]/g, '.$1').split('.').filter(Boolean);
  let cur = obj;
  for (const k of keys) { if (cur == null) return fallback; cur = cur[k]; }
  return cur === undefined ? fallback : cur;
}
```

For a fixed path, optional chaining does this: `obj?.a?.b?.[0]?.c ?? fallback`.

Q268. Implement `setInterval` using `setTimeout`. Why would you?

```
function mySetInterval(fn, ms) {
  let id, stopped = false;
  const tick = () => { id = setTimeout(() => { fn(); if (!stopped) tick(); }, ms); };  // schedule the next run
after this one
  tick();
  return () => { stopped = true; clearTimeout(id); };   // the flag also works when fn() itself cancels
}
```

Without the `stopped` flag, calling the cancel function from inside `fn` would not work: `tick()` would schedule a new timer right after it. Chained timeouts guarantee a gap between runs; `setInterval` fires on schedule even if the previous (async) run has not finished, so calls can pile up.

Q269. Implement a basic `Promise` class.

```
class MyPromise {
  #state = 'pending'; #value; #handlers = [];
  constructor(executor) {
    const settle = (state, value) => {
      if (this.#state !== 'pending') return;                    // settle only once
      this.#state = state; this.#value = value;
      this.#handlers.forEach((h) => h());
    };
    try { executor((v) => settle('fulfilled', v), (e) => settle('rejected', e)); }
    catch (e) { settle('rejected', e); }
  }
  then(onFul, onRej) {
    return new MyPromise((resolve, reject) => {
      const run = () => queueMicrotask(() => {                  // callbacks are always async
        const cb = this.#state === 'fulfilled' ? onFul : onRej;
        if (typeof cb !== 'function') return (this.#state === 'fulfilled' ? resolve : reject)(this.#value);
        try { resolve(cb(this.#value)); } catch (e) { reject(e); }
      });
      this.#state === 'pending' ? this.#handlers.push(run) : run();
    });
  }
  catch(onRej) { return this.then(undefined, onRej); }
}
```

This shows the core ideas: three states, settle once, async callbacks, `then` returns a new promise. A full Promises/A+ version also **adopts** a returned promise or thenable (waits for it instead of fulfilling with it). → *More:* §12.7

Q270. Small warm-ups: FizzBuzz, `duplicate([1, 2, 3])`, unique, chunk, palindrome.

```
for (let i = 1; i <= 100; i++) console.log((i % 3 ? '' : 'fizz') + (i % 5 ? '' : 'buzz') || i);
const duplicate = (arr) => [...arr, ...arr];                    // [1,2,3,1,2,3]
const unique = (arr) => [...new Set(arr)];
const chunk = (arr, n) => Array.from({ length: Math.ceil(arr.length / n) }, (_, i) => arr.slice(i * n, i * n +
n));
const isPalindrome = (s) => { const t = s.toLowerCase().replace(/[^a-z0-9]/g, ''); return t ===
[...t].reverse().join(''); };
```

→ *More:* §17.2 Coding patterns

---

15. LeetCode "30 Days of JavaScript"

LeetCode's **30 Days of JavaScript** study plan is the most common online practice set for JavaScript-specific interview tasks. Its sections are Closures, Basic Array Transformations, Function Transformations, Promises and Time, JSON, and Classes. The problems below follow the plan's order; extra JavaScript problems often practised alongside it come at the end. Read each LeetCode statement for exact edge cases; the answers here are compact versions that pass the stated examples.

Q271. Create Hello World Function. Return a function that always returns `"Hello World"`, whatever it is called with.

```
const createHelloWorld = () => (...args) => 'Hello World';
```

Tests that you know functions are values that can be returned (higher-order functions).

Q272. Counter. `createCounter(n)` returns a function that returns `n`, then `n + 1`, and so on.

```
const createCounter = (n) => () => n++;      // postfix: return the old value, then increment
```

The parameter `n` is captured by the closure and lives on between calls. → *More:* §4.7 Closures

Q273. To Be Or Not To Be. `expect(val)` returns `{ toBe, notToBe }`: `toBe(v)` returns `true` if `v === val`, otherwise throws `"Not Equal"`; `notToBe(v)` returns `true` if `v !== val`, otherwise throws `"Equal"`.

```
const expect = (val) => ({
  toBe: (v) => { if (v !== val) throw new Error('Not Equal'); return true; },
  notToBe: (v) => { if (v === val) throw new Error('Equal'); return true; },
});
```

This is the shape of Jest's `expect` — an object of closures.

Q274. Counter II. `createCounter(init)` returns `{ increment, decrement, reset }`, each returning the new value.

```
function createCounter(init) {
  let value = init;
  return { increment: () => ++value, decrement: () => --value, reset: () => (value = init) };
}
```

Q275. Apply Transform Over Each Element in Array. Implement `map(arr, fn)` without `Array.prototype.map`; `fn` receives `(arr[i], i)`.

```
function map(arr, fn) { const out = []; for (let i = 0; i < arr.length; i++) out.push(fn(arr[i], i)); return out;
}
```

Q276. Filter Elements from Array. Implement `filter(arr, fn)` keeping elements where `fn(arr[i], i)` is truthy, without `filter`.

```
function filter(arr, fn) { const out = []; for (let i = 0; i < arr.length; i++) if (fn(arr[i], i))
out.push(arr[i]); return out; }
```

Q277. Array Reduce Transformation. Implement `reduce(nums, fn, init)`; an empty array returns `init`.

```
function reduce(nums, fn, init) { let acc = init; for (const n of nums) acc = fn(acc, n); return acc; }
```

Q278. ⋆ Function Composition. `compose([f, g, h])` returns `x => f(g(h(x)))`; an empty list gives the identity function.

```
const compose = (functions) => (x) => functions.reduceRight((acc, fn) => fn(acc), x);
```

→ *More:* §17.3 Compose and pipe

Q279. Return Length of Arguments Passed.

```
const argumentsLength = (...args) => args.length;
```

Rest parameters collect any number of arguments into a real array.

Q280. ⋆ Allow One Function Call. `once(fn)` calls `fn` the first time and returns its result; later calls return `undefined`.

```
function once(fn) {
  let called = false;
  return (...args) => { if (called) return undefined; called = true; return fn(...args); };
}
```

Q281. Memoize. `memoize(fn)` returns a version that never calls `fn` twice with the same inputs (`sum(a, b)`, `fib(n)`, `factorial(n)` — note `sum(2, 3)` and `sum(3, 2)` are different inputs).

```
function memoize(fn) {
  const cache = new Map();
  return (...args) => {
    const key = args.join(',');              // inputs are numbers here; use JSON.stringify for general data
    if (!cache.has(key)) cache.set(key, fn(...args));
    return cache.get(key);
  };
}
```

→ *More:* Q257 above

Q282. Add Two Promises. Return a promise of the sum of two promises' numbers.

```
const addTwoPromises = async (p1, p2) => (await p1) + (await p2);
```

Both promises already run in parallel (they were created before the call); `Promise.all([p1, p2])` works too.

Q283. Sleep.

```
const sleep = (millis) => new Promise((resolve) => setTimeout(resolve, millis));
```

Q284. Timeout Cancellation. `cancellable(fn, args, t)` runs `fn(...args)` after `t` ms and returns `cancelFn`; calling `cancelFn` before then prevents the call.

```
function cancellable(fn, args, t) {
  const id = setTimeout(() => fn(...args), t);
  return () => clearTimeout(id);
}
```

Q285. Interval Cancellation. Call `fn(...args)` immediately and then every `t` ms until `cancelFn` is called.

```
function cancellable(fn, args, t) {
  fn(...args);
  const id = setInterval(() => fn(...args), t);
  return () => clearInterval(id);
}
```

Q286. ⋆ Promise Time Limit. `timeLimit(fn, t)` returns an async function that resolves like `fn`, or rejects with `"Time Limit Exceeded"` if `fn` takes longer than `t` ms.

```
const timeLimit = (fn, t) => (...args) => Promise.race([
  fn(...args),
  new Promise((_, reject) => setTimeout(() => reject('Time Limit Exceeded'), t)),
]);
```

In production, clear the losing timer and use `AbortSignal.timeout(t)` to actually stop the work.

Q287. ⋆ Cache With Time Limit. A class with `set(key, value, duration)` (returns `true` if an unexpired key already existed, and overwrites it), `get(key)` (value or `-1`) and `count()` (number of unexpired keys).

```
class TimeLimitedCache {
  #store = new Map();                                  // key -> { value, expires }
  set(key, value, duration) {
    const alive = this.get(key) !== -1;
    this.#store.set(key, { value, expires: Date.now() + duration });
    return alive;
  }
  get(key) { const e = this.#store.get(key); return e && e.expires > Date.now() ? e.value : -1; }
  count() { let n = 0; for (const e of this.#store.values()) if (e.expires > Date.now()) n++; return n; }
}
```

(Using `get` inside `set` assumes `-1` is never a stored value, which holds for this problem.) Alternative: a `setTimeout` per key that deletes it (clear the old timer on overwrite).

Q288. Debounce. Same as the classic: delay each call by `t` ms and cancel the previous pending call.

```
function debounce(fn, t) {
  let id;
  return (...args) => { clearTimeout(id); id = setTimeout(() => fn(...args), t); };
}
```

→ *More:* §17.3 Debounce

Q289. ⋆ Execute Asynchronous Functions in Parallel. Given an array of functions that return promises, implement `promiseAll(functions)` without `Promise.all`.

```
function promiseAll(functions) {
  return new Promise((resolve, reject) => {
    const results = new Array(functions.length);
    let left = functions.length;
    if (left === 0) return resolve(results);
    functions.forEach((fn, i) => fn().then((v) => {
      results[i] = v;
      if (--left === 0) resolve(results);
    }, reject));
  });
}
```

→ *More:* §17.3 Promise polyfills

Q290. Is Object Empty. Return `true` for `{}` or `[]`.

```
const isEmpty = (obj) => Object.keys(obj).length === 0;
```

A version that stops at the first key without building a key array is `for (const _ in obj) return false; return true;` — but it breaks if anyone has added enumerable properties to `Array.prototype` (as the "Array Prototype Last" solution below does with plain assignment).

Q291. Chunk Array. Split `arr` into sub-arrays of length `size` (the last may be shorter), without lodash's `_.chunk`.

```js
function chunk(arr, size) {
  const out = [];
  for (let i = 0; i < arr.length; i += size) out.push(arr.slice(i, i + size));
  return out;
}
```

Q292. Array Prototype Last. Add `arr.last()` returning the last element, or `-1` for an empty array.

```js
Array.prototype.last = function () { return this.length ? this[this.length - 1] : -1; };
```

Use a regular function so `this` is the array. In real code use the built-in `arr.at(-1)`, and avoid extending built-ins (Q69). If you must, use `Object.defineProperty` so the method is non-enumerable.

Q293. ⋆ Group By. Add `arr.groupBy(fn)` returning an object whose keys are `fn(item)` and values are arrays of items.

```js
Array.prototype.groupBy = function (fn) {
  const out = {};
  for (const item of this) (out[fn(item)] ??= []).push(item);
  return out;
};
```

The standard version is `Object.groupBy(arr, fn)` (ES2024). → *More:* §8.7

Q294. Sort By. Return `arr` sorted ascending by the number `fn(item)`.

```js
const sortBy = (arr, fn) => arr.toSorted((a, b) => fn(a) - fn(b));
```

Q295. Join Two Arrays by ID. Merge two arrays of objects by `id`; for the same `id`, properties from `arr2` override `arr1`; return the result sorted by `id`.

```js
function join(arr1, arr2) {
  const byId = new Map();
  for (const o of arr1) byId.set(o.id, { ...o });
  for (const o of arr2) byId.set(o.id, { ...byId.get(o.id), ...o });   // later spread wins
  return [...byId.values()].sort((a, b) => a.id - b.id);
}
```

Q296. ⋆ Flatten Deeply Nested Array. Implement `flat(arr, n)` (flatten up to depth `n`) without `Array.prototype.flat`.

```js
function flat(arr, n) {
  if (n === 0) return arr.slice();
  const out = [];
  for (const x of arr) Array.isArray(x) ? out.push(...flat(x, n - 1)) : out.push(x);
  return out;
}
```

Q297. Compact Object. Remove all falsy values from an object or array, recursively (`compactObject([null, 0, 5, [0], [false, 16]])` → `[5, [], [16]]`).

```js
function compactObject(obj) {
  if (obj === null || typeof obj !== 'object') return obj;
  if (Array.isArray(obj)) return obj.filter(Boolean).map(compactObject);
  const out = {};
  for (const [k, v] of Object.entries(obj)) if (v) out[k] = compactObject(v);
  return out;
}
```

Q298. ⋆ Event Emitter. `subscribe(eventName, callback)` returns `{ unsubscribe() }`; `emit(eventName, args = [])` calls the callbacks in subscription order and returns an array of their results.

```
class EventEmitter {
  #events = new Map();                                  // name -> Set of entries
  subscribe(eventName, callback) {
    if (!this.#events.has(eventName)) this.#events.set(eventName, new Set());
    const subs = this.#events.get(eventName), entry = { callback };  // wrapper: same callback twice is fine
    subs.add(entry);
    return { unsubscribe: () => { subs.delete(entry); } };
  }
  emit(eventName, args = []) {
    return [...(this.#events.get(eventName) ?? [])].map(({ callback }) => callback(...args));
  }
}
```

→ *More:* Q258 above

Q299. Array Wrapper. Class `ArrayWrapper(nums)` where `obj1 + obj2` gives the sum of all elements and `String(obj)` gives `"[1,2,3]"`.

```
class ArrayWrapper {
  constructor(nums) { this.nums = nums; }
  valueOf() { return this.nums.reduce((a, b) => a + b, 0); }   // used by +
  toString() { return `[${this.nums.join(',')}]`; }            // used by String()
}
```

→ *More:* Q17 above (object-to-primitive conversion)

Q300. ⋆ Calculator with Method Chaining. `new Calculator(10).add(5).subtract(7).getResult()` → `8`; `divide(0)` throws `"Division by zero is not allowed"`.

```
class Calculator {
  constructor(value) { this.value = value; }
  add(v) { this.value += v; return this; }              // return this -> chainable
  subtract(v) { this.value -= v; return this; }
  multiply(v) { this.value *= v; return this; }
  divide(v) { if (v === 0) throw new Error('Division by zero is not allowed'); this.value /= v; return this; }
  power(v) { this.value **= v; return this; }
  getResult() { return this.value; }
}
```

Q301. Check if Object Instance of Class. Return whether a value is an instance of a class, including primitives (`5` is an instance of `Number`) and without throwing for odd inputs.

```
function checkIfInstanceOf(obj, classFunction) {
  if (obj == null || typeof classFunction !== 'function') return false;
  for (let p = Object.getPrototypeOf(obj); p !== null; p = Object.getPrototypeOf(p)) {
    if (p === classFunction.prototype) return true;      // getPrototypeOf boxes primitives
  }
  return false;
}
```

Unlike `instanceof`, this says `5` is a `Number`, because `Object.getPrototypeOf(5)` is `Number.prototype`.

Q302. Generate Fibonacci Sequence and Nested Array Generator. Write a generator yielding Fibonacci numbers forever, and one yielding the integers of a nested array in order.

```
function* fibGenerator() { let [a, b] = [0, 1]; while (true) { yield a; [a, b] = [b, a + b]; } }
function* inorderTraversal(arr) { for (const x of arr) Array.isArray(x) ? yield* inorderTraversal(x) : yield x; }
```

`yield*` delegates to another generator. Infinite generators are safe because values are produced only on demand. → *More:* §8.5 Generators

Q303. Throttle, Curry, JSON Deep Equal, Promise Pool, Call Function with Custom Context, Undefined to Null. These JavaScript problems are also commonly practised with the plan; each matches a classic above:

- **Throttle** and **Curry** → the throttle and curry answers in group 14.
- **JSON Deep Equal** → the deep-equality answer (JSON values only, so no `Date`/`Map` handling is needed).

- **Promise Pool** (`promisePool(functions, n)`) → the concurrency-limit answer, without collecting results.
- **Call Function with Custom Context** (`fn.callPolyfill(context, ...args)`) → the `call` polyfill.
- **Undefined to Null** → a recursive walk, or `JSON.parse(JSON.stringify(obj, (k, v) => (v === undefined ? null : v)))`.

Sources of the question lists

Question wording and frequency were taken from these lists and posts (titles only; all answers are this book's own):

- LeetCode — *30 Days of JavaScript* study plan (leetcode.com/studyplan/30-days-of-javascript); problem lists as mirrored in public solution repositories (e.g. github.com/Rizwan-Damani/leetcode-30-days-js, github.com/mahmoudabozied4/30-Days-of-JavaScript-LeetCode).
- sudheerj, *javascript-interview-questions* (github.com/sudheerj/javascript-interview-questions) — the 500+ question list.
- h5bp, *Front-end Developer Interview Questions* — JavaScript questions and coding questions (github.com/h5bp/Front-end-Developer-Interview-Questions).
- GreatFrontEnd — *Front End Interview Playbook: JavaScript coding* and *10 Must-Know JavaScript Coding Interview Questions* (greatfrontend.com).
- NamasteDev — "JavaScript interview questions by company" (namastedev.com).
- Facts were checked against MDN Web Docs, the ECMAScript specification and TC39 finished-proposals list, the Node.js documentation, and OWASP cheat sheets, as for the rest of this book.